Simulation software for safe, sustainable and future deltas

Delft3D FM Suite 1D2D

Deltares systems

D-Flow Flexible Mesh

User Manual

Deltares

DRAFT

D-Flow Flexible Mesh

Computational Cores and User Interface

User Manual

Released for:
Delft3D FM Suite 1D2D 2026.02

Version: 2026.02
Revision: 81102

20 June 2026

D-Flow Flexible Mesh, User Manual

DRAFT

Published and printed by:
Deltares
Boussinesqweg 1
2629 HV Delft
P.O. 177
2600 MH Delft
The Netherlands

telephone: +31 88 335 82 73
e-mail: Information
www: Deltares

For sales contact:
telephone: +31 88 335 81 88
e-mail: Sales
www: Sales & Support

For support contact:
telephone: +31 88 335 81 00
e-mail: Support
www: Sales & Support

Copyright © 2026 Deltares
All rights reserved. No part of this document may be reproduced in any form by print, photo print, photo copy, microfilm or any other means, without written permission from the publisher: Deltares.

Contents

List of Tables xvi

List of Figures xix

List of Symbols xxvii

1 A guide to this manual 1
- 1.1 Introduction . . . 1
- 1.2 Overview . . . 1
- 1.3 Typographical conventions . . . 2

2 Introduction to D-Flow Flexible Mesh 4
- 2.1 Applications . . . 4
- 2.2 Modelling capabilities and advantages . . . 5
- 2.3 Current limitations for Delft3D Flexible Mesh Suite . . . 5
  - 2.3.1 General limitations . . . 5
  - 2.3.2 Limitations for D-Flow FM 1D2D . . . 5
- 2.4 Coupling to other modules . . . 6
- 2.5 Installation of D-Flow Flexible Mesh . . . 6
- 2.6 About D-Flow FM 1D2D . . . 7
  - 2.6.1 Introduction to rural modelling in D-Flow FM 1D2D . . . 7
  - 2.6.2 Introduction to urban modelling in D-Flow FM 1D2D . . . 7
  - 2.6.3 Introduction to river modelling in D-Flow FM 1D2D . . . 8
  - 2.6.4 Overview of modelling approaches in D-Flow FM 1D2D . . . 8
  - 2.6.5 Important differences compared to SOBEK . . . 9
- 2.7 Deltares Software Development Canvas . . . 10

3 Conceptual description of hydrodynamic equations 11
- 3.1 Introduction . . . 11
- 3.2 Physical processes . . . 11
- 3.3 Hydrodynamic modelling . . . 12
  - 3.3.1 Model equations in 1D . . . 12
    - 3.3.1.1 Momentum equation in 1D . . . 13
    - 3.3.1.2 Continuity equation (1D) . . . 13
    - 3.3.1.3 Flow area . . . 13
    - 3.3.1.4 Storage area . . . 14
    - 3.3.1.5 Wetted area . . . 14
    - 3.3.1.6 Wetted perimeter . . . 14
    - 3.3.1.7 Velocity . . . 15
    - 3.3.1.8 Hydraulic radius . . . 15
    - 3.3.1.9 Momentum equation for 1D networks . . . 16
  - 3.3.2 Model equations in 2D . . . 17
    - 3.3.2.1 Momentum equations in horizontal direction (2D) . . . 17
    - 3.3.2.2 Continuity equation (2D) . . . 18
  - 3.3.3 The hydrostatic pressure assumption . . . 18

DRAFT

3.3.4 The Coriolis force . . . 19
3.3.5 Diffusion of momentum . . . 19
3.3.6 Bed friction . . . 20
3.3.6.1 Roughness definition input . . . 22
3.4 Cross sections in 1D . . . 22
3.4.1 Introduction . . . 22
3.4.2 Cross section types . . . 23
3.4.2.1 YZ and XYZ cross section . . . 23
3.4.2.2 ZW cross section . . . 24
3.4.2.3 ZWriver cross section . . . 25
3.4.2.4 Circle cross section . . . 25
3.4.2.5 Rectangle cross section . . . 25
3.4.2.6 Template cross sections . . . 25
3.4.3 Preissmann slot and closed cross sections . . . 25
3.4.4 Levees . . . 26
3.4.5 Conveyance . . . 29
3.4.5.1 Conveyance according to the vertically segmented approach 30
3.4.5.2 Conveyance according to the lumped approach . . . 35
3.4.6 1D friction definition . . . 35
3.4.6.1 Friction definition with cross section definitions . . . 35
3.4.6.2 Friction definition in friction files . . . 36
3.4.7 Interpolation of cross sections . . . 37
3.5 Equations of State for the density . . . 38
3.5.1 Overview of density formulations in D-Flow FM . . . 38
3.5.2 Thermobaricity . . . 40
3.6 Shear-stresses at closed boundaries . . . 40
3.7 Secondary flow . . . 41
3.7.1 Definition . . . 42
3.7.2 Momentum equations in horizontal direction . . . 42
3.7.3 Effect of secondary flow on depth-averaged momentum equations . . . 43
3.7.4 The depth averaged transport equation for the spiral motion intensity . . . 44
3.8 Tide generating forces . . . 44
3.9 Wave forcing in D-Flow FM . . . 46
3.9.1 Forcing by radiation stress gradients . . . 48
3.9.2 Stokes drift and mass flux . . . 49
3.9.3 Enhancement of the bed shear-stress by waves . . . 49
3.10 Assumptions underlying D-Flow FM . . . 54

4 Conceptual description of transport equation 56
4.1 Introduction . . . 56
4.2 Transport processes . . . 56
4.2.1 Advection . . . 57
4.2.2 Diffusion . . . 57
4.2.2.1 Time integration of horizontal diffusion . . . 58
4.2.3 Sources and sinks . . . 59
4.3 Boundary and initial conditions for transport equation . . . 60
4.3.1 Open boundary conditions . . . 60
4.3.2 Closed boundary conditions . . . 61
4.3.3 Thatcher-Harleman boundary conditions . . . 61
4.3.4 Initial conditions . . . 61

5 Getting started with Graphical User Interface 64
5.1 Introduction . . . 64
5.2 Overview of D-Flow FM GUI . . . 64

DRAFT

5.2.1 **Project** window . . . 65
5.2.2 The central window . . . 66
5.2.3 **Settings** window . . . 67
5.2.4 **Map** window . . . 67
5.2.5 **Messages** window . . . 68
5.2.6 **Time Navigator** window . . . 68
5.3 Dockable views . . . 68
5.3.1 Docking tabs separately . . . 68
5.3.2 Multiple tabs . . . 69
5.4 Ribbons and toolbars . . . 69
5.4.1 Ribbons (shortcut keys) . . . 69
5.4.2 File . . . 70
5.4.3 Home . . . 71
5.4.4 View . . . 71
5.4.5 Map . . . 72
5.4.6 Grid . . . 72
5.4.7 DIMR . . . 73
5.4.8 Quick access toolbar . . . 73
5.5 Important differences compared to Delft3D-FLOW GUI . . . 73
5.5.1 Project vs model . . . 74
5.5.2 Load/save vs import/export . . . 74
5.5.3 Working from the map . . . 74
5.5.4 Coordinate conversion . . . 74
5.5.5 Model area . . . 75
5.5.6 Integrated models (model couplings) . . . 75
5.5.7 Ribbons (shortcut keys) . . . 76
5.5.8 Context menus . . . 76
5.5.9 Scripting . . . 76

6 Set-up of a D-Flow FM model **77**
6.1 Introduction . . . 77
6.2 mdu-file and attribute files . . . 77
6.3 Filenames and conventions . . . 78
6.4 How to set-up a D-Flow FM model . . . 79
6.4.1 General . . . 80
6.4.1.1 Model coordinate system . . . 80
6.4.1.2 Angle of latitude for Coriolis on a Cartesian grid . . . 81
6.4.2 Area . . . 81
6.4.2.1 Grid-snapped features . . . 82
6.4.2.2 Observation points . . . 84
6.4.2.3 Observation cross-sections . . . 85
6.4.2.4 Thin dams . . . 86
6.4.2.5 Fixed weirs . . . 87
6.4.2.6 Land boundaries . . . 89
6.4.2.7 Dry points and Dry areas . . . 89
6.4.2.8 Pumps . . . 92
6.4.2.9 Weirs . . . 93
6.4.2.10 Gates . . . 95
6.4.2.11 Bridge Pillars . . . 95
6.4.3 Computational grid . . . 97
6.4.4 Bed level . . . 97
6.4.5 Time frame . . . 97
6.4.6 Initial conditions . . . 99
6.4.7 Boundary conditions . . . 101

DRAFT

6.4.7.1 Specification of boundary locations (support points) 102
6.4.7.2 Boundary data editor (forcing) 103
6.4.7.3 Import/export boundary conditions from the *Project* window 115
6.4.7.4 Overview of boundary conditions in attribute table (non-editable)117
6.4.7.5 Open Boundaries and grid snapping 118
6.4.8 Physical parameters 119
6.4.8.1 Constants 119
6.4.8.2 Roughness 119
6.4.8.3 Viscosity 120
6.4.8.4 Diffusivity 121
6.4.8.5 Infiltration 121
6.4.8.6 Meteo items 121
6.4.8.7 Wind 121
6.4.9 Sources and sinks 122
6.4.10 Intakes, outfalls and coupled intake-outfalls 124
6.4.11 Lateral discharges 126
6.4.11.1 Grid snapping and distribution of discharge 126
6.4.11.2 Forcing information for lateral discharges 127
6.4.11.3 Output for lateral discharges 127
6.4.12 Numerical parameters 127
6.4.13 Output parameters 127
6.4.14 Miscellaneous 132
6.5 Save project, MDU file and attribute files 132

7 Running a model **134**
7.1 Introduction 134
7.2 Running a scenario using the User Interface 134
7.3 Running a scenario using a batch script 135
7.3.1 Commandline executables 136
7.3.2 Running D-Flow FM 136
7.4 Parallel computing using MPI 137
7.4.1 Introduction 137
7.4.2 Partitioning a model 138
7.4.2.1 More options for the partitioning 139
7.4.2.2 Partitioning of 1D2D models 142
7.4.2.3 Partitioning the mdu-file 142
7.4.2.4 Remaining model input 142
7.4.3 Running a parallel job 142
7.4.3.1 A parallel simulation via DIMR 143
7.4.3.2 A parallel simulation on Linux via D-Flow FM 143
7.4.4 Visualizing the results of a parallel run 144
7.4.4.1 Plotting all partitioned map files with Delft3D-QUICKPLOT . 144
7.4.4.2 Merging multiple map files into one 145
7.5 Run time 148
7.5.1 Multi-core performance improvements by OpenMP 149
7.5.2 Parallel computations in coupled simulations 149
7.6 Files and file sizes 151
7.6.1 History file 151
7.6.2 Map file 152
7.6.3 Restart file 152
7.7 D-Flow FM command-line argument 153
7.8 Restart a simulation 154
7.9 Inspecting a model with the volume tool 155
7.10 Frequently asked questions 156

DRAFT

8 Visualize results **157**
8.1 Introduction . . . 157
8.2 Visualization with the User Interface . . . 157
8.3 Visualization with Delft3D-QUICKPLOT . . . 158
8.4 Visualization with Matlab . . . 158
8.5 Visualization with Python . . . 159

9 Boundary conditions in D-Flow FM **160**
9.1 Hydrodynamics boundary conditions . . . 160
9.1.1 Open boundary conditions . . . 160
9.1.1.1 Forcing information . . . 160
9.1.1.2 External forcings file . . . 166
9.1.1.3 External forcings in model definition file . . . 166
9.1.1.4 Example . . . 166
9.1.1.5 Miscellaneous . . . 168

10 Hydraulic structures **172**
10.1 Fixed weirs . . . 173
10.2 Dam break . . . 173
10.2.1 Grid snapping of a dam break . . . 174
10.2.2 Computation of breach growth . . . 175
10.2.2.1 The Verheij–Van der Knaap (2002) formula . . . 175
10.2.2.2 The Van der Knaap (2000) formula . . . 177
10.2.2.3 Breach crest level and width time series . . . 178
10.2.2.4 Method when breach grows wider than one flow link . . . 179
10.3 General structure . . . 179
10.3.1 Notation . . . 182
10.3.2 Geometry . . . 183
10.3.3 Computation of the downstream water level . . . 185
10.3.4 Turning off the computation of the energy levels . . . 187
10.3.5 Discharge equations . . . 188
10.4 Weir . . . 189
10.5 Gates . . . 190
10.6 Orifice . . . 191
10.7 Culvert . . . 192
10.8 Long Culvert . . . 195
10.8.1 Long culvert . . . 195
10.9 Bridges in 1D . . . 197
10.10 Universal weir . . . 199
10.11 Pump . . . 202
10.11.1 Notation . . . 202
10.11.2 Orientation . . . 203
10.11.3 Staged pump . . . 203
10.11.4 Non-staged pump . . . 204
10.11.5 Capacity reduction table . . . 204
10.11.6 Distribution and relaxation of pump discharge . . . 204
10.12 Compound structure . . . 205
10.13 Outlet . . . 205
10.14 Thin dams . . . 205

11 Hydrological processes **206**
11.1 Notation . . . 206
11.2 Precipitation . . . 206
11.2.1 Meteo forcing input . . . 206

DRAFT

11.2.2 Rainfall distribution across the model grid . . . 207
11.2.3 Rainfall intensities versus rainfall amounts . . . 207
11.3 Interception . . . 208
11.3.1 Numerical treatment of interception . . . 208
11.4 Infiltration . . . 208
11.4.1 Horton's equation . . . 209
11.5 Evaporation . . . 209
11.5.1 Numerical treatment of evaporation . . . 209
11.5.2 Evaporation from interception layer . . . 210
11.6 Lumped rainfall-runoff coupling . . . 210
11.7 Hydrological output . . . 210
11.7.1 Spatial hydrological output in map file . . . 210
11.7.2 Time series hydrological output in history file . . . 210

12 Bedforms and vegetation 211
12.1 Bedform heights . . . 211
12.1.1 Van Rijn (2007) bedform roughness height predictor . . . 211
12.1.2 Dune height predictor . . . 213
12.2 Vegetation models . . . 215
12.2.1 Dynamic vegetation model . . . 215
12.2.1.1 Philosophies of dynamic vegetation model . . . 215
12.2.1.2 Conceptual description . . . 216
12.3 Trachytopes . . . 217
12.3.1 Trachytope classes . . . 217
12.3.2 Averaging and accumulation of trachytopes . . . 224

13 Calibration factor for bed roughness 226

14 Wind 227
14.1 Definitions . . . 227
14.1.1 Nautical convention . . . 228
14.1.2 Drag coefficient . . . 228
14.2 File formats . . . 232
14.2.1 Defined on the computational grid . . . 233
14.2.1.1 Specification of uniform wind through velocity components . 233
14.2.1.2 Specification of uniform wind through magnitude and direction 235
14.2.2 Defined on an equidistant grid . . . 236
14.2.3 Defined on a curvilinear grid . . . 237
14.2.4 Space and time varying Charnock coefficients . . . 239
14.2.5 Rainfall . . . 239
14.2.6 Defined on a spiderweb grid . . . 240
14.2.7 Combination of several wind specifications . . . 242
14.3 Masking of points in the wind grid from interpolation ('land-sea mask') . . . 244

15 Ice modelling in D-Flow FM 246
15.1 Forced ice cover . . . 246
15.2 Dynamic ice cover . . . 246
15.2.1 Conservation laws . . . 246
15.2.2 Thermodynamics of the ice/snow cover . . . 248
15.3 Ice effect on wind drag . . . 250
15.4 Ice effect on pressure . . . 251
15.5 Ice effect on flow resistance . . . 251
15.6 Ice effect on surface exchange . . . 251
15.7 Ice effect on waves . . . 252
15.8 Ice input options . . . 252

DRAFT

15.8.1 Ice modelling via prescribed external input files . . . 255
15.8.2 Ice modelling via Semtner . . . 256
15.9 Postprocessing . . . 257
15.10 Limitations . . . 257

16 Coupling to external models via Basic Model Interface (BMI) 259
16.1 BMI interface to interact with the D-Flow FM model state . . . 259
16.2 DIMR and the BMI . . . 259
16.3 BMI access to hydraulic structure data in D-Flow FM . . . 260
16.4 BMI access to observations in D-Flow FM . . . 261
16.5 BMI access to forcings in D-Flow FM . . . 262
16.6 Coupling with D-RTC (FBC-Tools) . . . 263
16.6.1 Introduction . . . 263
16.6.2 Getting started . . . 263
16.6.2.1 User interface: the first steps . . . 263
16.6.2.2 Input D-Flow FM . . . 264
16.6.2.3 Input D-RTC . . . 264
16.7 Coupling with D-Rainfall Runoff . . . 264
16.7.1 Introduction . . . 265
16.7.2 Getting started . . . 265
16.7.2.1 User interface: the first steps . . . 265
16.7.2.2 Input D-Flow FM . . . 266
16.7.2.3 Input D-RR . . . 266
16.7.2.4 Input DIMR . . . 266
16.7.3 Technical background . . . 267

17 Numerical aspects of D-Flow Flexible Mesh 268
17.1 Topological conventions for (2D) horizontal grids . . . 268
17.2 Notation . . . 270
17.3 Definition of bed level options . . . 271
17.3.1 Piecewise constant approach for the bed level . . . 272
17.3.2 Piecewise linear approach for the bed levels . . . 273
17.3.3 Hybrid bed level approach . . . 273
17.3.4 Specification in the User Interface . . . 273
17.4 The location of open boundaries . . . 274
17.4.1 Geometry of open boundaries . . . 276
17.4.1.1 Conveyance in 2D . . . 277
17.5 Drying and flooding . . . 279
17.5.1 Dry/wet criterion at cell faces . . . 279
17.5.2 Dry/wet criterion at cell circumcenters . . . 279
17.6 Advection at open boundaries . . . 280
17.7 1D2D coupling . . . 280
17.7.1 1D2D model input . . . 281
17.7.2 Definitions . . . 281
17.7.3 1D2D internal connections (lateral and embedded) . . . 282
17.7.4 1D2D longitudinal connection . . . 285
17.7.5 1D2D vertically stacked connection . . . 286
17.8 Storage nodes . . . 288
17.8.1 Notation . . . 289
17.8.2 Urban manhole compartments . . . 289
17.8.2.1 Storage volume for urban manhole compartments . . . 289
17.8.2.2 Network topology for urban manhole compartments . . . 290
17.8.2.3 Vertical levels for urban networks . . . 291
17.8.3 Storage tables for lumped water storage . . . 291

DRAFT

17.8.4 Manhole losses . . . 291
17.9 Flow limiters . . . 293
17.9.1 Slope limiter for 1D . . . 293
17.9.2 Slope limiter for 1D or 1D2D connections with advection type 8 . . . 294
17.10 Volume tables for 1D . . . 294
17.10.1 Introduction . . . 294
17.10.2 Technical information . . . 295
17.11 Numerical treatment of lateral discharges . . . 296
17.12 Implicit 1D solver . . . 296

18 Tutorial for 1D and 1D2D modelling 298
18.1 Outline of the tutorials . . . 298
18.2 Tutorial 1: Hydrodynamics in open water . . . 298
18.2.1 Setting up a 1D model . . . 298
18.2.2 Integrating a 2D grid in the model . . . 302
18.2.3 Running the Model . . . 304
18.2.4 Exporting and saving the model . . . 304
18.3 Tutorial 2: Hydrodynamics in sewers . . . 305
18.3.1 Setting up a 1D sewer model . . . 305
18.3.2 Include the Rainfall Runoff concept . . . 307
18.3.3 Inserting NWRW catchments . . . 308
18.3.4 Running the model and visualizing the results . . . 309
18.4 Tutorial 3: Hydrodynamics in 1D2D river models . . . 309
18.4.1 Setting up the 1D model . . . 310
18.4.2 Expanding the model to a 1D2D model . . . 312
18.5 Tutorial 4: Real Time Control in a river model . . . 315
18.5.1 Import the 1D2D model of the Magdalena river . . . 316
18.5.2 Implement a PID Rule in the weir . . . 316
18.5.3 Inspecting the response of the structure . . . 318

References 320

A The master definition file 325
A.1 Overview of keywords generated by GUI . . . 325
A.2 Overview of research keywords . . . 337
A.3 Overview of deprecated and removed keywords . . . 340

B The net file for flexible meshes 343
B.1 2D grids in netCDF UGRID files . . . 343
B.2 Proposal for netCDF conventions for 1D networks . . . 343
B.2.1 History . . . 343
B.2.2 Naming conventions . . . 343
B.2.3 Requirements 1D Network . . . 344
B.2.4 1D Network: Example based on Figure B.1 . . . 344
B.2.4.1 Dimensions . . . 344
B.2.4.2 1D-Network topology . . . 345
B.2.4.3 1D-Network geometry . . . 346
B.2.4.4 Numerical discretization . . . 347
B.2.4.5 Duplicate or unique points . . . 348
B.2.5 1D Network, 1D2D Links as mesh contacts, 2D Meshes; Example based on Figure B.2 . . . 349
B.2.5.1 1D Network . . . 349
B.2.5.2 1D2D Links as mesh contacts . . . 349
B.2.6 Figures with example numbering of nodes and edges . . . 350
B.2.7 Data defined on 1D meshes . . . 352

DRAFT

B.2.8 1D Network history nc-file . . . 353
B.2.9 Keyword definition for 1D Network . . . 353
B.2.10 Keyword definition for 1D2D Links . . . 354
B.2.11 Addendum: Deltares-specific agreements for 1D2D datasets . . . 355
B.2.11.1 Attributes for data variables . . . 355
B.2.11.2 Non-unique relation between 1D mesh nodes and network topology edges . . . 355
B.2.11.3 Branch and node terminology . . . 355
B.2.11.4 Water system characteristics . . . 356
B.2.11.5 Network geometry . . . 356
B.2.11.6 Positioning of 1D mesh nodes: from offset to x, y . . . 357
B.2.11.7 Determine the chainage of nodes on a branch, ex. branch 1 of Figure B.4 . . . 357
B.2.11.8 1D2D link types . . . 358
B.2.11.9 Data compression by packing . . . 359

C Attribute files **361**
C.1 Introduction . . . 361
C.2 Polyline/polygon file . . . 361
C.3 Sample file . . . 362
C.4 Time series <tim> file (ASCII) . . . 363
C.5 Time-varying data <bc> file (ASCII) . . . 363
C.5.1 Name in a bc block . . . 366
C.5.1.1 Global forcings . . . 366
C.5.1.2 Boundary forcings . . . 366
C.5.1.3 Meteo forcings on stations . . . 367
C.5.1.4 Forcings on "point" locations . . . 367
C.5.2 Function in a bc block . . . 367
C.5.3 Quantity names in a bc block . . . 369
C.6 The external forcings file . . . 370
C.6.1 Old style external forcings . . . 370
C.6.2 New style external forcing . . . 373
C.6.2.1 Boundary definitions . . . 374
C.6.2.2 Lateral discharge definitions . . . 375
C.6.2.3 Meteorological forcings . . . 376
C.6.2.4 Source and sink definitions . . . 378
C.6.3 Accepted quantity names . . . 379
C.6.4 Overview of deprecated and removed keywords in external forcing files 383
C.7 Initial and parameter fields . . . 384
C.8 Trachytopes . . . 384
C.8.1 Area Roughness on Links (ARL-file) . . . 385
C.8.1.1 Example . . . 385
C.8.2 Trachytope Definition file (TTD-file) . . . 385
C.8.2.1 General format . . . 385
C.8.2.2 Example . . . 386
C.8.2.3 Discharge dependent format . . . 386
C.8.2.4 Water level dependent format . . . 386
C.8.2.5 Supported roughness formulations . . . 388
C.9 Fixed weirs . . . 389
C.10 Calibration Factors . . . 390
C.10.1 Calibration factor definition file (CLD-file) . . . 390
C.10.1.1 Header of the CLD-file . . . 391
C.10.1.2 Constant values . . . 391
C.10.1.3 Discharge dependent format . . . 391

DRAFT

C.10.1.4 Water level dependent format . . . 391
C.10.1.5 Example . . . 392
C.10.2 Calibration Class Area on Links (CLL-file) . . . 392
C.10.2.1 Header of the CLL-file . . . 392
C.10.2.2 Body of the CLL-file . . . 392
C.10.2.3 Example . . . 393
C.11 Sources and sinks . . . 393
C.12 Dry points and areas . . . 393
C.13 Structure INI file . . . 394
C.13.1 Weir . . . 395
C.13.2 Universal Weir . . . 395
C.13.3 Culvert . . . 396
C.13.4 Long culvert . . . 397
C.13.5 Bridge . . . 398
C.13.6 Pump . . . 399
C.13.7 Orifice . . . 400
C.13.8 Gate . . . 400
C.13.9 General Structure . . . 401
C.13.10 Dambreak . . . 402
C.13.11 Compound Structure . . . 404
C.13.12 Structure defined with a time series . . . 405
C.13.13 File version history . . . 406
C.14 Space varying wind and pressure . . . 407
C.14.1 Meteo on equidistant grids . . . 407
C.14.2 Curvilinear data . . . 411
C.14.3 Spiderweb data . . . 414
C.14.4 Meteo on a netCDF file . . . 418
C.15 Statistical output (Fourier, minimum, maximum and average) . . . 418
C.16 1D roughness INI file . . . 421
C.17 Cross section files . . . 425
C.17.1 Cross section definition file . . . 425
C.17.1.1 Circle cross section definition . . . 428
C.17.1.2 Rectangle cross section definition . . . 428
C.17.1.3 Tabulated river cross section definition . . . 429
C.17.1.4 ZW (tabulated) cross section definition . . . 430
C.17.1.5 XYZ cross section definition . . . 434
C.17.1.6 YZ cross section definition . . . 435
C.17.2 Cross section location file . . . 436
C.18 Storage node file . . . 437
C.19 1D-2D Link file . . . 440
C.20 Cache file . . . 441
C.20.1 *File version and description* . . . 442

D Initial conditions and spatially varying input 443
D.1 Introduction . . . 443
D.2 Initial and parameter fields . . . 443
D.3 Supported quantities . . . 448
D.3.1 Accepted quantity names in initial field files . . . 448
D.3.2 Water levels . . . 449
D.3.3 Initial velocities . . . 449
D.3.4 Salinity . . . 449
D.3.5 Temperature . . . 450
D.3.6 Tracers . . . 450
D.3.7 Physical coefficients . . . 450

DRAFT

D.4 Supported file formats . . . 450
D.4.1 1D field INI file . . . 451
D.4.2 Inside-polygon option . . . 452
D.4.3 Sample file . . . 452
D.4.4 GeoTIFF file . . . 452
D.4.5 Vertical profile file . . . 453
D.4.6 Map file . . . 453
D.4.7 Restart file . . . 453

E Boundary conditions specification 454
E.1 Supported boundary types . . . 454
E.1.1 Open boundaries . . . 454
E.1.2 Astronomic boundary conditions . . . 454
E.1.3 Astronomic correction factors . . . 454
E.1.4 Harmonic flow boundary conditions . . . 454
E.1.5 QH-relation flow boundary conditions . . . 455
E.1.6 Time-series flow boundary conditions . . . 455
E.1.7 Time-Series for Varying Air Density . . . 455
E.2 Boundary signal file formats . . . 455
E.2.1 The <cmp> format . . . 455
E.2.2 The <qh> format . . . 456
E.2.3 The <bc> format . . . 456
E.2.4 The netCDF-format . . . 456
E.2.4.1 Scalar quantity for point-location time-series . . . 456
E.2.4.2 Scalar Quantity for gridded data . . . 457
E.2.4.3 Vector quantity . . . 458

F Output files 460
F.1 Diagnostics file . . . 460
F.2 Demanding output . . . 460
F.2.1 The MDU-file . . . 461
F.2.2 Observation points . . . 461
F.2.2.1 The observation point file with extension <*_obs.ini> . . . 461
F.2.2.2 The observation point file with extension <*.xyn> . . . 462
F.2.3 Moving observation points . . . 463
F.2.4 Observation cross sections . . . 464
F.2.4.1 Observation cross section file with extension <*_crs.ini> . . . 464
F.2.4.2 Observation cross section file with extension <*_crs.pli> . . . 465
F.3 NetCDF output files . . . 465
F.3.1 Timeseries as netCDF his-file . . . 467
F.3.1.1 Dimensions . . . 467
F.3.1.2 Location variables . . . 467
F.3.1.3 Overview of output quantities in a netCDF his-file . . . 468
F.3.1.4 Variables on stations . . . 470
F.3.1.5 Variables on cross sections . . . 477
F.3.1.6 Mass balance output . . . 479
F.3.1.7 Variables on laterals . . . 482
F.3.1.8 Variables on sources/sinks . . . 482
F.3.1.9 Hydraulic structure output . . . 482
F.3.1.10 Geometry variables . . . 493
F.3.2 Spatial fields as netCDF map-file . . . 495
F.3.3 Class map files as netCDF clm-file . . . 514
F.3.4 Restart files as netCDF rst-file . . . 514
F.3.5 Volume tables per- gridpoint output as netCDF file . . . 514

DRAFT

F.3.6 NetCDF versions and file formats . . . 517
F.4 Shapefiles . . . 517
F.5 Post processing on his-file . . . 518
F.5.1 Max25 function . . . 518
F.5.2 Maximum based on a generic filter . . . 518
F.5.3 Maximum based on a running mean filter . . . 519
F.6 Time step analysis . . . 519

G Model generation **520**
G.1 Introduction . . . 520
G.2 Grid generation . . . 520
G.2.1 Uniform Cartesian grid . . . 520
G.2.2 Locally refined Cartesian grid . . . 520

H Spatial editor **521**
H.1 Introduction . . . 521
H.2 General . . . 521
H.2.1 Overview of spatial editor . . . 521
H.2.2 Import/export dataset . . . 522
H.2.3 Activate (spatial) model quantity . . . 523
H.2.4 Colorscale . . . 524
H.2.5 Render mode . . . 525
H.2.6 Context menu . . . 526
H.3 Quantity selection . . . 527
H.4 Geometry operations . . . 528
H.4.1 Polygons . . . 528
H.4.2 Lines . . . 529
H.4.3 Points . . . 530
H.5 Spatial operations . . . 530
H.5.1 Import point cloud . . . 531
H.5.2 Crop . . . 531
H.5.3 Delete . . . 532
H.5.4 Set value . . . 533
H.5.5 Contour . . . 534
H.5.6 Copy to samples . . . 536
H.5.7 Copy to spatial data . . . 537
H.5.8 Merge spatial data . . . 538
H.5.9 Gradient . . . 539
H.5.10 Interpolate . . . 540
H.5.11 Smoothing . . . 543
H.5.12 Overwrite (single) value . . . 545
H.6 Operation stack . . . 546
H.6.1 Stack workflow . . . 546
H.6.2 Edit operation properties . . . 547
H.6.3 Enable/disable operations . . . 548
H.6.4 Delete operations . . . 549
H.6.5 Refresh stack . . . 550
H.6.6 Quick links . . . 550
H.6.7 Import/export . . . 551

I Grid editing **552**
I.1 Overview of the toolbars . . . 553
I.2 How to start D-Grid Editor? . . . 554
I.3 How to save a mesh generated by the D-Grid Editor? . . . 554

DRAFT

J Astronomical constituents **555**
J.1 Introduction 555
J.2 List of astronomical constituents 555

DRAFT

List of Tables

3.1 Example of a ZW cross section definition . . . 24
3.2 Fitting coefficients for wave/current boundary layer model . . . 52

6.1 Overview and description of numerical parameters . . . 128
6.2 Input and output parameters of the example . . . 131
6.3 Time (after *Reference Date* in seconds) of output files . . . 131
6.4 Overview and description miscellaneous parameters . . . 132

10.1 Recommended values of input parameters in the Verheij–Van der Knaap (2002) formula. . . . 176
10.2 Recommended values for u_c and τ_c. . . . 177
10.3 Values of coefficients c_1, c_2 and $B_{\max}$ of the Van der Knaap (2000) formula for different material types. . . . 178
10.4 Mapping table for weir input parameters to general structure parameters . . . 189
10.5 Mapping table for orifice input parameters to general structure parameters . . . 191

15.1 Ice input keywords . . . 252
15.2 Keywords for ice output . . . 253
15.3 ice quantities on map- and his-files . . . 258

16.1 Supported compound variable fields for hydraulic structures in D-Flow FM via BMI. . . . 260
16.2 Supported compound variable fields for observations in D-Flow FM via BMI. . . . 261
16.3 Supported compound variable fields for forcings in D-Flow FM via BMI. . . . 262

17.1 Example storage table relating water levels to storage areas. . . . 291

A.1 Standard MDU-file with default settings. . . . 325
A.2 Overview of numerical research parameters with compulsory defaults. . . . 338
A.3 Overview of deprecated and removed keywords. . . . 340

B.1 1D2D Link table based on Figure B.2 . . . 349
B.2 1D network topology . . . 353
B.3 1D network geometry . . . 353
B.4 1D network mesh . . . 354
B.5 Composite meshes, 1D2D Links . . . 354
B.6 1D2D Links keywords . . . 354
B.7 Chainage table for branch B, Figure B.4 . . . 358

C.1 Description of <bc> format. . . . 363
C.3 Quantity names for use in <*.bc> files. . . . 369
C.4 Definition of external forcings. . . . 373
C.5 Boundary definitions in new style external forcing file. . . . 374
C.6 Lateral discharge definitions in new style external forcing file. . . . 375
C.7 Meteorological forcings in new style external forcing file. . . . 376
C.8 Source and sink definitions in new style external forcing file. . . . 378
C.9 List of accepted external forcing quantity names. . . . 380
C.10 List of quantity name and required standard name supported in NetCDF format. . . . 382
C.11 Deprecated and obsolete keywords in external forcings file. . . . 383
C.13 General part of structure file. . . . 394
C.14 Weir definition. . . . 395
C.15 Universal Weir definition. . . . 396
C.16 Culvert . . . 396

DRAFT

C.17 Long culvert . . . 397
C.18 Bridge Structure definition. . . . 398
C.19 Pump definition. . . . 399
C.20 Orifice definition. . . . 400
C.21 Gate definition. . . . 400
C.22 Specific input for the general structure. . . . 401
C.23 Specific input for the dambreak. . . . 403
C.24 Specific input for the compound structure. . . . 404
C.25 Quantities that can be set by a time series. . . . 406
C.31 Roughness definition. . . . 422
C.32 Cross section definition file. . . . 425
C.33 Circle cross section definition. . . . 428
C.34 Rectangular cross section definition. . . . 428
C.35 Tabulated river cross section definition. . . . 429
C.36 ZW cross section definition. . . . 430
C.37 Arch profile definition. . . . 431
C.38 Mouth-shaped profile definition. . . . 432
C.39 Egg profile definition. . . . 432
C.40 Ellipse profile definition. . . . 433
C.41 Inverted egg profile definition. . . . 433
C.42 Steel mouth-shaped profile definition. . . . 433
C.43 Trapezium profile definition. . . . 434
C.44 U-shaped profile definition. . . . 434
C.45 XYZ cross section definition. . . . 434
C.46 YZ cross section definition. . . . 436
C.47 Cross section location. . . . 437
C.48 Definition of Storage Nodes. . . . 438
C.49 1D-2D link definitions. . . . 440

D.1 Initial and parameter fields. . . . 443
D.2 List of accepted field quantity names in initial field file. . . . 448
D.3 1D field definition. . . . 451

F.1 Definition of observation points. . . . 461
F.2 Definition of observation cross sections. . . . 464
F.3 Global overview of output quantities in the history file. . . . 469
F.4 Output quantities of observation stations in history file. . . . 471
F.5 Output quantities on cross sections in history file. . . . 478
F.6 Output quantities for the mass balance in history file. . . . 480
F.7 Output quantities for laterals. . . . 482
F.8 Output quantities of pumps in history file. . . . 483
F.9 Output quantities of general structures in history file. . . . 484
F.10 Output quantities of weirs in history file. . . . 485
F.11 Output quantities of orifices in history file. . . . 486
F.12 Output quantities of bridges in history file. . . . 487
F.13 Output quantities of culverts in history file. . . . 488
F.14 Output quantities of long culverts in history file. . . . 489
F.15 Output quantities of dam breaks in history file. . . . 490
F.16 Output quantities of universal weirs in history file. . . . 491
F.17 Output quantities of compound structures in history file. . . . 492
F.18 Summary of output in history files of quantities for different structures. . . . 492

DRAFT

F.19 Output variables in map file of UGRID format. Column 'Location' shows the grid location: CN means cell corners. L means net links. S means pressure points, U means velocity points. S3D and U3D are the corresponding locations in 3D networks. Two more locations in 3D networks are: W, vertical velocity point on all layer interfaces, and WU, vertical viscosity point on all layer interfaces. 498
F.20 Flow analysis variables in the map file 513
F.21 Features and MDU settings for generating shapefiles 518

J.1 Astronomical constituents 555

DRAFT

List of Figures

2.1 Overview of TRL levels for software of Deltares . . . 10

3.1 Definiton of flow area (A_F) and storage area (A_S) . . . 14
3.2 Wetted perimeter . . . 14
3.3 Wetted perimeter for the main channel . . . 15
3.4 Computation of hydraulic radius . . . 15
3.5 Wetted perimeter for the main channel . . . 16
3.6 Example of river network, and coarse grid representation . . . 17
3.7 Input for map projection for specifying Coriolis parameter on the grid. . . . 19
3.8 Input parameters for horizontal and vertical eddy viscosities. . . . 20
3.9 Example of constructed vertical segments in a Y-Z profile . . . 24
3.10 Example a ZW cross section . . . 25
3.11 levee option, available in a river profile . . . 26
3.12 Flow area behind the levee as function of local water levels. . . . 28
3.13 Total area behind the levee as function of local water levels. . . . 29
3.14 Definition sketch of a vertical segment, considered in computing conveyance for a Y-Z profile and an Asymmetrical Trapezium cross-section . . . 31
3.15 Example of a 1D-model. . . . 37
3.16 Vertical profile secondary flow (v) in river bend and direction bed stress . . . 41
3.17 Schematic view of non-linear interaction of wave and current bed shear-stresses (from Soulsby *et al.* (1993b, Figure 16, p. 89)) . . . 50
3.18 Inter-comparison of eight models for prediction of mean and maximum bed shear-stress due to waves and currents (from Soulsby *et al.* (1993b, Figure 17, p. 90)) . . . 53

5.1 Start-up lay-out D-Flow FM User Interface . . . 64
5.2 **Project** window of D-Flow FM plugin . . . 66
5.3 The central window with a map and contents of the D-Flow FM model . . . 66
5.4 The model **Settings** window with General settings tab enabled . . . 67
5.5 **Map** window controlling map contents . . . 67
5.6 Log of messages, warnings and errors in **Messages** window . . . 68
5.7 **Time Navigator** window in D-Flow FM User Interface . . . 68
5.8 Docking windows on two screens within the User Interface. . . . 68
5.9 Bringing the **Time Navigator** window to the front . . . 69
5.10 Docking the **Time Navigator** window. . . . 69
5.11 Auto hide the **Properties** window . . . 69
5.12 Perform operations using the shortcut keys . . . 70
5.13 The *File* ribbon . . . 70
5.14 The User Interface options dialog. . . . 71
5.15 The *Home* ribbon. . . . 71
5.16 The *View* ribbon. . . . 71
5.17 The *Map* ribbon. . . . 72
5.18 The *Grid* editor ribbon . . . 72
5.19 The *DIMR* ribbon . . . 73
5.20 The quick access toolbar. . . . 73
5.21 Set map coordinate system using right mouse button . . . 74
5.22 Select a coordinate system using the quick search bar . . . 75
5.23 Perform operations using the shortcut keys . . . 76

6.1 The Add Model drop-down menu . . . 79
6.2 The *Select model ...* window . . . 79
6.3 Overview of the *General* tab . . . 80
6.4 Coordinate system wizard . . . 80

DRAFT

6.5 Overview of geographical features . . . 82
6.6 Overview of map ribbon . . . 82
6.7 Example of expanded *Estimated Grid-snapped features* attribute in the *Map* window . . . 83
6.8 Example of grid snapped features displayed on the central map . . . 84
6.9 Geographical and grid snapped representation of an observation point . . . 84
6.10 Attribute table with observation points . . . 85
6.11 Geographical and grid snapped representation of a cross section . . . 85
6.12 Attribute table with observation cross sections . . . 86
6.13 Geographical and grid snapped representation of a thin dam . . . 86
6.14 Attribute table with thin dams . . . 87
6.15 Geographical and grid snapped representation of a fixed weir . . . 87
6.16 Schematic representation of a fixed weir . . . 87
6.17 Attribute table with fixed weirs . . . 88
6.18 Fixed weir editor . . . 88
6.19 Geographical representation of a land boundary . . . 89
6.20 Attribute table with land boundaries . . . 89
6.21 Geographical and grid snapped representation of several dry points . . . 90
6.22 Attribute table with dry points . . . 90
6.23 Geographical and grid snapped representation of a dry area . . . 91
6.24 Attribute table with dry areas . . . 91
6.25 Polygon for pump (a) and adjustment of physical properties (b). . . . 92
6.26 Selection of the pumps . . . 93
6.27 Polygon for adjustable weir (a) and adjustment of geometrical and temporal conditions (b). . . . 94
6.28 Time series for crest level. . . . 94
6.29 Time series for crest level. . . . 95
6.30 Polygon for gate (a) and adjustment of geometrical and temporal conditions (b). 95
6.31 Selection of a bridge . . . 96
6.32 Adjustment of the properties of the pillars . . . 96
6.33 Bed level activated in the spatial editor . . . 97
6.34 Overview time frame tab . . . 98
6.35 Relation between Reference Date and the simulation start and stop time for astronomic- and harmonic-series as used in the simulation. Time-series should cover the simulation time. . . . 99
6.36 Initial conditions in the *Project* window . . . 100
6.37 The 'Initial Conditions' tab where you can specify the uniform values and the layer distributions of the active physical quantities. . . . 100
6.38 Initial water levels activated in the spatial editor . . . 100
6.39 Selecting 3 dimensional initial fields from the dropdown box in the 'Map' ribbon to edit them in the spatial editor . . . 101
6.40 Restart files in output states folder . . . 101
6.41 Restart file in initial conditions attribute . . . 101
6.42 Adding a boundary support point on a polyline in the central map . . . 102
6.43 Polyline added in *Project* window under 'Boundary Conditions' . . . 102
6.44 Geometry edit options in *Map* ribbon . . . 103
6.45 Edit name of polyline/boundary in Boundaries tab . . . 103
6.46 Overview of the boundary data editor . . . 104
6.47 Process and quantity selection in the boundary data editor . . . 105
6.48 Activate a support point . . . 106
6.49 Time zone in the Boundary conditions editor . . . 106
6.50 Specification of time series in the boundary data editor (left panel) . . . 107
6.51 Window for generating series of time points . . . 107
6.52 Csv import wizard: csv file selection . . . 108

DRAFT

6.53 Clipboard/csv import wizard: specification of how data should be parsed into columns . . . 109
6.54 Clipboard/csv import wizard: specification of how values should be parsed and columns should be mapped . . . 110
6.55 Specification of harmonic components in boundary data editor . . . 111
6.56 Selection of astronomical components from list (after pressing 'select components') . . . 112
6.57 Suggestions for astronomical components in list . . . 113
6.58 Editing harmonic/astronomic components and their corrections . . . 113
6.59 Specification of a Q-h relationship . . . 114
6.60 Example of active and total signal for multiple water level data series on one support point . . . 114
6.61 Importing or exporting boundary features — both polylines <∗.pli> and forcing <∗.bc> — from the *Project* window using the right mouse button . . . 115
6.62 Import or export a <∗.pli>-file as is or with coordinate transformation. . . . 115
6.63 Import or export a <∗.pli>-file as is or with coordinate transformation. . . . 116
6.64 Import or export a *.pli file as is or with coordinate transformation. . . . 117
6.65 Overview of all boundary conditions in attribute table . . . 118
6.66 Illustration of grid snapping for open boundaries . . . 118
6.67 The physical parameters in the **Project** window . . . 119
6.68 The section of the 'Physical Parameters' tab where you can specify roughness related parameters and formulations. . . . 119
6.69 Roughness activated in the spatial editor to create/edit a spatially varying field 120
6.70 The section of the'Physical Parameters' tab where you can specify (uniform) values for the horizontal and vertical eddy viscosity and diffusivity. . . . 120
6.71 Viscosity activated in the spatial editor to create/edit a spatially varying field . . 121
6.72 Overview of parameters in sub-tab Wind . . . 122
6.73 Activate the sources and sinks editing icon in the *Map* ribbon . . . 123
6.74 Add sources and sinks in the central map using the 'Sources and sinks' icon. . 123
6.75 Sources and sinks appearing in the *Project* window . . . 123
6.76 Specifying time series for sources and sinks in the sources and sinks editor . . 124
6.77 *Output Parameters* tab . . . 127
6.78 Overview *Output Parameters* tab . . . 129
6.79 Model/data import drop-down menu . . . 132
6.80 Model/data import wizard . . . 132

7.1 Selecting the model you want to run in the *Project* window . . . 134
7.2 Group Run in Home ribbon . . . 134
7.3 Run console D-Flow FM User Interface . . . 135
7.4 Partioning exporter dialog . . . 141
7.5 Partitioned 1D2D mesh with two partitions and illustrated ghost cells. . . . 142
7.6 Domain selector in Delft3D-QUICKPLOT for partitioned map files. . . . 145

8.1 Example of setting output (in)visible in the *Map* window . . . 157
8.2 Useful map visualization options in the Delft3D-QUICKPLOT . . . 158

9.1 User Interface view of a simple channel covered by a straightforward Cartesian grid. Boundary conditions are prescribed at the left hand side and the right hand size of the domain. . . . 166
9.2 Transitioning between initial conditions F_i and boundary forcing F_b depending on the smoothing time T_{smo} and smoothing start time $T_{start,smo}$ for a case in which the start of start time `TStart` falls within the smoothing period. . . . 169

DRAFT

9.3 Cell-centre velocity magnitude along the streamwise direction of an idealized normal-flow case. `ExtrBl` = 1 extrapolates the bed levels at the boundaries and perfect normal flow is obtained. . . . 171
9.4 Flow depth at velocity points along the streamwise direction of an idealized normal-flow case. `ExtrBl` = 1 extrapolates the bed levels at the boundaries and perfect normal flow is obtained. . . . 171
9.5 Cell-centre flow depth along the streamwise direction of an idealized normal-flow case. `ExtrBl` = 1 extrapolates the bed levels at the boundaries and perfect normal flow is obtained. . . . 171

10.1 Selection of structures (and other items) in the toolbar. . . . 172
10.2 General structure, side view . . . 180
10.3 General structure, top view . . . 180
10.4 General structure, front view . . . 181
10.5 Submerged gate flow . . . 185
10.6 Submerged weir-flow. . . . 186
10.7 Input for a gate . . . 190
10.8 Side view of a culvert . . . 192
10.9 Good modelling practice, culvert, Inverted Siphon and Siphon . . . 194
10.10 Side view of an inverted siphon . . . 195
10.11 Side view of a long culvert . . . 196
10.12 Discretization of three long culverts via 1D and 1D2D links. . . . 196
10.13 A suspension bridge . . . 197
10.14 Crest level (Y-Z) profile of a Universal weir, divided into three rectangular weir sections (2, 4 and 6) and four triangular weir sections (1, 3, 5 and 7) . . . 200
10.15 Pump station . . . 203

11.1 Vertical cross section view of wet area and hydrological area in a 1D and a 2D grid cell. . . . 206
11.2 Rainfall rates by nearest neighbour interpolation, with some areas masked out. 208

12.1 Conceptual schematization of the modelling environment, visualizing the Basic Model Interface connecting the D-Flow FM model for hydrodynamic and morphodynamic computation with the Python environment for vegetation and ecological modelling. Figure after Oorschot *et al.* (2016) and Brückner *et al.* (2019). . . . 216

14.1 Nautical conventions for the wind. . . . 228
14.2 Prescription of the dependency of the wind drag coefficient C_d on the wind speed is achieved by means of at least 1 point, with a maximum of 3 points. . 229
14.3 Grid definition of the spiderweb grid for cyclone winds. . . . 240

15.1 Conceptual diagram of the ice-growth (left) and temperature (right) model . . . 248
15.2 Illustration of computed ice thickness in case of no snow (in blue), ice thickness in case of snow (in green) and snow thickness (in red) . . . 258

16.1 An *Integrated model* in the **Project** window . . . 263
16.2 Example of a *Control flow* in D-RTC . . . 264
16.3 Example of an *Integrated Model* with D-Flow FM and D-RR. . . . 265
16.4 Hydro links in the GUI: connection between D-RR catchments and D-Flow FM lateral discharges in 1D. . . . 266

17.1 Flexible mesh topology . . . 268
17.2 Two conventional definitions of the cell center of a triangle: the circumcenter and the mass center. . . . 269

DRAFT

17.3 Perfect orthogonality and nearly perfect smoothness along the edge connecting two triangles. Black lines/dots are network links/nodes, blue lines/dots are flow links/nodes. . . . 269
17.4 Poor mesh properties due to violating either the smoothness or the orthogonality at the edge connecting two triangles . . . 270
17.5 Definition of the water levels, the bed levels and the velocities in case of two adjacent triangular cells. . . . 271
17.6 Specification of the conveyance option in the User Interface. . . . 273
17.7 Specification of the bed level treatment type in the User Interface. . . . 274
17.8 Specification of the hybrid bed options (with keywords `blminabove` and `blmeanbelow`). . . . 274
17.9 Virtual boundary 'cells' near the shaded boundary . . . 275
17.10 Position of virtual boundary point . . . 276
17.11 Bed representation with uniform depth levels (a), and locally sloping bed (b). . . . 277
17.12 A shematic view of the linear variation over the width for calculating the flow parameters. . . . 278
17.13 Discretization of internal 1D2D links. . . . 282
17.14 Lateral 1D2D links and the volume contribution to the 1D and 2D cell. . . . 284
17.15 Discretization of longitudinal 1D2D links. . . . 285
17.16 Examples uses for 1D2D vertically stacked connections. . . . 286
17.17 Difference between manhole storage nodes of type "closed" and "reservoir" . . . 289
17.18 Side view of a manhole with two compartments. At the bottom the matching network topology. . . . 290
17.19 Example of a manhole configuration. In this example the water from pipe 1 flows into the manhole and from the manhole the water flows into pipe 2 . . . 292

18.1 A schematization of the 1D model of the reach in Gieterveen. . . . 299
18.2 Cross-sections. . . . 300
18.3 Adding structures to the 1D branches. . . . 302
18.4 Interpolating the DEM samples over the created 2D raster . . . 303
18.5 Adding precipitation to the model. . . . 304
18.6 Visualizing the *Waterdepth at pressure points* results at the last time step. . . . 304
18.7 Schematization of the sewer system. . . . 306
18.8 Pipe definitions in the sewer system. . . . 307
18.9 Setting precipitation data. . . . 308
18.10 Catchment Data. . . . 309
18.11 Creating NWRW catchments and defining the outlet compartment. . . . 309
18.12 Visualizing the results using the sideview function. . . . 309
18.13 Opening an FM-model and selecting the right coordinate system. . . . 310
18.14 Schematization of the Magdalena river. . . . 310
18.15 Generating the computational gridnodes in the Magdalena river. . . . 311
18.16 The 2D grid with a close-up of the refinement around the Magdalena River. . . . 313
18.17 Mesh cells at the 1D Magdalena River are removed. . . . 313
18.18 Close-up of the grid refinement and 1D2D links in the South part of the Magdalena river. . . . 314
18.19 Flood depth at the last time step in the Magdalena river. . . . 315
18.20 Placing observation points upstream and downstream of the weir. . . . 317
18.21 Control group of the weir. . . . 317
18.22 Control group of the weir. . . . 318
18.23 Crest level of the weir changing as a result of the upstream and downstream water level. . . . 319

B.1 Simple 1D network . . . 344
B.2 Composite mesh (1D, 1D2D and 2D) . . . 349

DRAFT

B.3 Simple 1D network and mesh, all branches do have intermediate nodes 350
B.4 Simple 1D network and mesh, all branches do have intermediate nodes 351
B.5 Simple 1D network and mesh, 2 branches do not have intermediate nodes (so only end points) 351
B.6 Simple 1D network and mesh, all branches do not have intermediate nodes (so only end points) 351
B.7 Simple 1D network and mesh 351
B.8 Simple 1D network and mesh 352

C.1 Illustration of the data to grid conversion for meteo input on a separate curvilinear grid 413
C.2 Wind definition according to Nautical convention 416
C.3 Spiderweb grid definition 416

H.1 Overview of spatial editor functionality in Map ribbon 521
H.2 Importing a point cloud into the project using the context menu on “project” in the project tree 522
H.3 Importing a point cloud into the project using the "Import" drop-down menu in the Spatial Operations ribbon 523
H.4 Activate the imported point cloud in the spatial editor by double clicking it in the project tree 523
H.5 Activate the imported point cloud in the spatial editor by selecting it from the dropdown box in the Map ribbon 523
H.6 Legend button in Map ribbon 524
H.7 Activate the colorscale using the Legend in the map ribbon 524
H.8 Edit the colorscale properties using the context menu on the active layer in the Map Tree 525
H.9 Select the rendermode for the active layer in the property grid. 526
H.10 Example of a coverage rendered as colored numbers. 526
H.11 Selecting a smoothing operation for a polygon geometry from the context menu (using context menu) 527
H.12 Activating a spatial quantity by double clicking it in the project tree (in this example ‘Initial Water Level’) 527
H.13 Activating a spatial quantity by selecting it from the dropdown box in the ‘Spatial Operations’ ribbon 528
H.14 Overview of the available geometry operations in the ‘Spatial Operations’ ribbon 528
H.15 Activating the polygon operation and drawing polygons in the central map. . . . 529
H.16 Activating the line operation and drawing lines in the central map. 529
H.17 Activating the ‘Add points’ operation, drawing them in the central map and assigning a value to them. 530
H.18 Overview of the available spatial operations in the ‘Spatial Operations’ ribbon . 530
H.19 Importing a point cloud using the ‘Import’ operation from the ‘Spatial Operations’ ribbon 531
H.20 Option to perform a coordinate transformation on the imported point cloud . . 531
H.21 Appearance of import point cloud operation in the operations stack 531
H.22 Performing a crop operation on a point cloud with a polygon using ‘Crop’ from the ‘Spatial Operations’ ribbon 532
H.23 Appearance of crop operation in the operations stack 532
H.24 Performing an delete operation on a point cloud with a polygon using ‘Delete’ from the ‘Spatial Operations’ ribbon 533
H.25 Appearance of delete operation in the operations stack 533
H.26 Performing a set value operation (e.g. overwrite) on a point cloud with a polygon using ‘Set Value’ from the ‘Spatial Operations’ ribbon 534
H.27 Appearance of set value operation in the operations stack 534

DRAFT

H.28 Import a nautical chart as a georeferenced tiff file . . . 535
H.29 Set the right map coordinate system for the geotiff . . . 535
H.30 Performing a contour operation on a nautical chart using lines to define the contours and 'Contour' from the 'Spatial Operations' ribbon . . . 535
H.31 Bring the sample set to the front if it appears behind the nautical chart . . . 536
H.32 Appearance of contour operation in the operations stack . . . 536
H.33 Applying the copy to samples operation . . . 536
H.34 Copy to samples operation result . . . 537
H.35 Applying the copy spatial data operation . . . 537
H.36 Copy spatial data operation result . . . 537
H.37 Activating the merge spatial data tool from the ribbon . . . 538
H.38 The merge operation requests a pointwise combination method . . . 538
H.39 Resulting grid coverage . . . 539
H.40 Performing a gradient operation on a point cloud with a polygon using 'Gradient' from the 'Spatial Operations' ribbon . . . 539
H.41 Appearance of gradient operation in the operations stack . . . 540
H.42 Interpolation Operation options . . . 540
H.43 Averaging options . . . 541
H.44 Performing an interpolation operation on a single sample set (without using a polygon) using 'Interpolate' from the 'Spatial Operations' ribbon . . . 542
H.45 Appearance of interpolation of 'set1' to the coverage 'bed level' in the operations stack . . . 542
H.46 Performing an interpolation operation on multiple sample sets (without using a polygon) using 'Interpolate' from the 'Spatial Operations' ribbon . . . 543
H.47 Appearance of interpolation of 'set1' and 'set2' to the coverage 'bed level' in the operations stack . . . 543
H.48 Performing a smoothing operation on a point cloud with a polygon using 'Smoothing' from the 'Spatial Operations' ribbon . . . 544
H.49 Appearance of smoothing operation in the operations stack . . . 544
H.50 The cursor for the overwrite operation showing the value of the closest coverage point . . . 545
H.51 Performing an overwrite operation on a coverage point using 'Single Value' from the 'Spatial Operations' ribbon . . . 546
H.52 Appearance of overwrite operation in the operations stack . . . 546
H.54 Input for the operation (top panel), mask for the operation (middle panel) and output of the operation (bottom panel) . . . 547
H.53 The 'Operations' panel with the operations stack. In this example 'bed level' is the coverage (e.g. trunk) that is edited. The point clouds 'set 1' and 'set 2' (e.g. branches) are used to construct the 'bed level' coverage. . . . 547
H.55 Editing the value or 'Pointwise operation' of a 'Set Value' operation using the properties panel . . . 548
H.56 Disabling an operation using the boxed cross icon in the stack menu. The operation will become grey. Note the exlamation marks marking the stack 'out of sync'. . . . 549
H.57 Removing an operation from the stack using the cross icon in the stack menu . 549
H.58 Removing an operation from the stack using the context menu on the selected operation . . . 550
H.59 Refresh the stack using the 'Refresh' button so that all operation are (re-) evaluated . . . 550
H.60 Quick link to the original dataset before performing any spatial operations . . . 551
H.61 Quick link to the output after performing all (enabled) operations . . . 551

I.1 Illustration of a *mesh* with only quadrangles generated by D-Grid Editor . . . 552
I.2 Illustration of a *mesh* with triangles and quadrangles generated by D-Grid Editor 553

DRAFT

I.3 Main toolbar in D-Grid Editor 553
I.4 Toolbar for Polygons 553
I.5 Toolbar for Splines 553
I.6 Toolbar for Land Boundaries 554
I.7 Toolbar for Curvilinear Grids 554
I.8 Toolbar for Meshes 554

DRAFT

List of Symbols

Symbol	Unit	Description
O	m	Wetted perimeter

DRAFT

1 A guide to this manual

1.1 Introduction

This User Manual describes the hydrodynamic module D-Flow Flexible Mesh (D-Flow FM) which is part of the Delft3D Flexible Mesh Suite or D-HYDRO Suite. There are two different releases for this suite; one for 1D2D modelling and one for 2D3D modelling. **The current document involves 1D2D modelling with D-Flow FM 1D2D, which is meant for rural, urban and river applications.**

Note: D-Flow FM 1D2D and D-Flow FM 2D3D make use of different GUI's, but use the same computational kernel. Therefore, there are separate releases for 1D2D and 2D3D modelling.

This module is part of several Modelling suites, released by Deltares as Deltares Systems or Dutch Delta Systems. These modelling suites are build with use of the Delta Shell framework. The framework enables to develop a range of modeling suites, each distinguished by the components and — most significantly — the (numerical) modules, which are plugged in. The modules which are compliant with the Delta Shell framework are released as D-*Name of the module*, for example: D-Flow Flexible Mesh, D-Waves, D-Water Quality, D-Real Time Control and D-Rainfall Runoff.

Therefore, this User Manual is shipped with several modelling suites. On the *Start Page* links are provided to all relevant User Manuals (and Technical Reference Manuals) for that modelling suite. Other user manuals can be referenced. In that case, you need to open the specific user manual from the *Start Page* in the central window. Some texts are shared in different user manuals, in order to improve the readability.

1.2 Overview

To make this manual more accessible we will briefly describe the contents of a lot of chapters in this user manual.

If this is your first time to start working with D-Flow FM we suggest you to read Chapter 5, Getting started with Graphical User Interface and carry out the tutorial exercises of Chapter 18. These chapters explain the user interface and guide you through the modelling process resulting in your first simulations with D-Flow FM.

Chapter 2: Introduction to D-Flow Flexible Mesh, provides specifications of D-Flow FM, such as the areas of application, the standard and specific features provided, coupling to other modules and utilities.

Chapter 3: Conceptual description of hydrodynamic equations, contains a description of the conceptual model of D-Flow FM.

Chapter 5: Getting started with Graphical User Interface, gives an overview of the basic features of the D-Flow FM GUI and will guide you through the main steps to set up a basic D-Flow FM model.

Chapter 6: Set-up of a D-Flow FM model, provides practical information on the GUI, setting up a model with all its parameters and tuning the model.

Chapter 7: Running a model, discusses how to validate and execute a model run. This can either be done in the GUI, or in batch mode. The latter can also be done in combination with parallel computing by using MPI. It also provides some information on run times and file sizes.

DRAFT

Chapter 8: Visualize results, explains in short the visualization of results within the GUI. It introduces the program Delft3D-QUICKPLOT to visualize or animate the simulation results, and Matlab for general post-processing.

Chapter 10: Hydraulic structures, gives background information of the available hydraulic structures in D-Flow FM, the relevant definitions and the supported file formats.

Chapter 14: Wind, gives background information of how wind fields should be imposed, the relevant definitions and the supported file formats.

Chapter 16: Coupling to external models via Basic Model Interface (BMI), provides guidance on the integrated modelling of hydrodynamics (D-Flow FM) to external models.

Section 16.6: Coupling with D-RTC (FBC-Tools), provides guidance on the integrated modelling of hydrodynamics (D-Flow FM) and real time control of hydraulic structures (D-RTC).

Chapter 18: Tutorial for 1D and 1D2D modelling, gives you hands-on experience in using the D-Flow FM GUI to define the input of a simple problem, in validating this input, in executing the simulation and in inspecting the results.

1.3 Typographical conventions

Throughout this manual, the following conventions help you to distinguish between different elements of text.

Example	**Description**
Module **Project** **Map**	The title of a window or a sub-window is given in **bold**. Sub-windows are displayed in the **Module** window and cannot be moved. Windows can be moved independently from the **Module** window, such as the **Map** window.
Save	Item from a menu, title of a push button or the name of a user interface input field. Upon selecting this item (click or in some cases double click with the left mouse button on it) a related action will be executed; in most cases it will result in displaying some other (sub-)window. In case of an input field you are supposed to enter input data of the required format and in the required domain.
<\tutorial\wave\swan-curvi> <siu.mdw>	Directory names, filenames, and path names are expressed between angle brackets, <>. For Linux environments a forward slash (/) is used instead of the backward slash (\) for Windows environments.
“27 08 1999”	Data to be typed by you into the input fields are displayed between double quotes. Selections of menu items, option boxes etc. are described as such: for instance ‘select *Save* and go to the next window’.

DRAFT

Example	Description
`delft3d-menu`	Commands after the *Command Prompt* to be typed by you are given in the font Courier New, 10 points.
➢	In this User manual, user actions are indicated with this arrow.
[m s^{-1}] [–]	Units are given between square brackets when used next to the formulae. Leaving them out might result in misinterpretation. Most units will be in SI notation. [m AD] stands for 'meter Above Datum', which denotes a level relative to the vertical reference system in the model.

Command prompts and terminal output are shown in framed boxes with typewriter font:

```
> ./dflowfm --version
Deltares, D-Flow FM Version 1.1.149.41663, Sep 02 2015, 10:40:42
Compiled with support for:
IntGUI: no
OpenGL: no
OpenMP: yes
MPI   : yes
PETSc : yes
METIS : yes
```

DRAFT

2 Introduction to D-Flow Flexible Mesh

D-Flow Flexible Mesh (D-Flow FM) is a hydrodynamic simulation program developed by Deltares. It is the hydrodynamic module of Deltares' unique, fully integrated computer software suite for a multi-disciplinary approach with 1D, 2D and 3D modelling named Delft3D Flexible Mesh Suite (abroad) or D-HYDRO Suite (in the Netherlands). 1D2D modelling is meant for urban, rural and river applications, while 2D3D focusses on coastal seas, estuarines and rivers. It can carry out simulations of hydrodynamic flows, water quality and ecology, waves and morphology.

The term Flexible Mesh in the name refers to the flexible combination of unstructured grids consisting of triangles, quadrangles, pentagons and hexagons.

It has been designed for experts and non-experts alike. The Delft3D Flexible Mesh Suite is composed of several modules, grouped around a mutual interface, while being capable to interact with one another. D-Flow FM, which this manual is about, is one of these modules. D-Flow FM is a multi-dimensional (1D, 2D and 3D) hydrodynamic (and transport) simulation program which calculates non-steady flow and transport phenomena that result from tidal and meteorological forcing on unstructured, boundary fitted grids.

D-Flow Flexible Mesh can be coupled with the water quality module D-Water Quality. For the interaction between waves and currents the flow module may be coupled with the short-waves model D-Waves. To apply hydraulic structures in a model, D-Flow FM can be coupled to the D-Real Time Control module.

2.1 Applications

This section presents an overview of the domain of applicability of the model. This is done by making claims about the types of practical and realistic situations in which the model can be employed, and showing the nature and quality of the information that the model is capable of generating in those situations.

The purpose of providing the reader with this inventory of application types is to allow the reader to quickly recognise whether the model is indeed suitable for a particular application. The computational model of D-Flow FM can be used in a wide range of applications, which are listed below:

- ◇ Tide and wind-driven flow (i.e., storm surges).
- ◇ Density driven flow.
- ◇ Horizontal transport of matter on large and small scales
- ◇ Hydrodynamic impact of engineering works such as land reclamation, breakwaters, dikes
- ◇ Hydrodynamic impact of hydraulic structures such as barriers, gates and weirs
- ◇ Rainfall runoff in urban environments.
- ◇ Simulation of tsunamis, hydraulic jumps, bores and flood waves.
- ◇ Fresh-water river discharges in estuaries.
- ◇ Spreading of waste water discharges from coastal outfalls
- ◇ Thermal stratification in lakes, seas and reservoirs.
- ◇ Impact of cooling water intakes and waste water outlets.
- ◇ Sediment transport and morphodynamics via online coupling with hydrodynamics.
- ◇ Water quality processes via online coupling with hydrodynamics.

DRAFT

2.2 Modelling capabilities and advantages

- ◇ Hydrodynamic modelling on unstructured grids, which offers extra flexibility of (grid) modelling because triangles, quadrangles (four-sided), pentagons (five-sided) and hexagons (six-sided) are possible.
- ◇ A user-friendly and modern Grahical User Interface to set up a hydrodynamic model
- ◇ Model input is independent of the grid (trachytopes excluded), so that changes to the grid are easy and without much effort.
- ◇ Possibility of parallel computing on MPI-based High Performance Computing clusters
- ◇ In the vertical σ-layers and z-layers and $z - \sigma$-layers are possible, which offers huge flexibility for vertical layer modelling.
- ◇ Many options for hydraulic structures. Depending on the application the most suitable approach can be selected.
- ◇ High computational efficiency, leading to fast computation times
- ◇ Coupling to external software via BMI-interfaces
- ◇ Use of international standards, like outputting via NetCDF

2.3 Current limitations for Delft3D Flexible Mesh Suite

2.3.1 General limitations

We start with limitations that are not related to either D-Flow FM, but are part of other modules of the Delft3D Flexible Mesh Suite. This involves:

- ◇ 1D, 2D and 3D modelling cannot be combined in the Delft3D Flexible Mesh Suite. Currently, only 1D and 2D can be combined into so-called 1D2D applications, by using D-Flow FM 1D2D; see Deltares (2026b).
- ◇ The D-Particle Tracking module is currently of TRL level 4-6 (prototype) for 2D modelling and of TRL level 1-3 (concept) for 3D modelling.
- ◇ Nesting, which is a standard practice in automatic generation of boundary conditions from a large to a small domain, isn't available (cf. Delft3D-NESTHD module in Delft3D 4 Suite).
- ◇ Wave modelling in 3D, via D-Waves, is of TRL level 4-6 (prototype).
- ◇ Morphology modelling in 3D, via D-Morphology, is of TRL level 4-6 (prototype).

For a definition of the Technical Technology Readiness Levels (TRL) we refer to Section 2.7.

2.3.2 Limitations for D-Flow FM 1D2D

The following functionalities aren't available in D-Flow FM 1D2D:

- ◇ Coupling to D-Morphology
- ◇ Coupling to D-Waves
- ◇ Calibration and data assimilation (for which OpenDA is applied in combination with 2D and 3D modelling with D-Flow FM)
- ◇ Coupling to nearfield systems (via COSUMO)

Furthermore, the current limitations are that

- ◇ transport processes (salinity, temperature, sediments, tracers) cannot be simulated in 1D.
- ◇ the coupling to D-Water Quality is of TRL level 1-3 (concept).

DRAFT

2.4 Coupling to other modules

The hydrodynamic conditions (velocities, water elevations, density, salinity, vertical eddy viscosity and vertical eddy diffusivity) calculated in the D-Flow FM module are used as input to the other modules of the Delft3D Flexible Mesh Suite, which are:

Module	Description
D-Morphology	cohesive and non-cohesive sediment transport
D-Real Time Control	flow-triggered control of hydrodynamic structures
D-Water Quality (Delwaq)	far-field water quality

For using D-Flow FM the following utilities are important:

Module	Description
User Interface	for complete model set-up and model runs, see chapter 5 and chapter 6
Grid Editor	for generating unstructured grids
Delft3D-QUICKPLOT	for visualisation and animation of simulation results
DFMOUTPUT	◇ for merging partitioned map files into one, see section 7.4.4. ◇ output of derived properties, such as the maximum value of a time series, see section F.5.

For details on using these utility programs you are referred to the corresponding user manuals.

2.5 Installation of D-Flow Flexible Mesh

D-Flow FM consists of a set of Computational Cores and a User Interface. The User Interface is only available for Windows operating systems. The Computational Cores are available both on Windows and Linux operating systems.

On Windows, D-Flow FM can be installed via a msi-file. Double-click it to start the installation and follow the instructions. Both the Computational Cores and the User Interface will be installed. Start the application from *Start* →*All Programs* →*Deltares* or by double-clicking the short-cut on your desktop.

On Linux, the Computational Cores of D-Flow FM are available via an rpm-file or via an Apptainer Container (previously known as Singularity). See the installation manual for more details.

A Docker version is available upon request. Please contact Deltares if needed.

DRAFT

2.6 About D-Flow FM 1D2D

D-Flow FM 1D2D is an integrated software package for river, urban or rural management. Several program modules work together to give a comprehensive overview of waterway systems keeping you in control. Its integrated framework also means that D-Flow FM 1D2D can link river, canal and sewer systems for a total water management solution. It contains the following components: D-Flow Flexible Mesh (D-Flow FM), D-Real Time Control (D-RTC) and D-Rainfall Runoff (D-RR). Furthermore, a coupling to external models is possible via the Basic Model Interface (BMI). This program is easy to configure and quick to learn. It guides you in obtaining correct model descriptions. The graphically oriented interface makes it more user friendly than similar types of software. D-Flow FM 1D2D is designed to interface with your existing software. It can download information from a variety of standard data formats and GIS systems. D-Flow FM 1D2D is based on high performance computer technology. That means it can handle water networks of any size - big or small.

2.6.1 Introduction to rural modelling in D-Flow FM 1D2D

D-Flow FM 1D2D gives regional water managers a high-quality tool for modelling irrigation systems, drainage systems, natural streams in lowlands and hilly areas. Applications are typically related to optimizing agricultural production flood control, irrigation, canal automation, reservoir operation, and water quality control. D-Flow FM 1D2D can also answer questions about increased pollution loads in response to growing urbanisation. D-Flow FM 1D2D offers the support you need for effective planning, design and operation of new and existing water systems. It is a tool that can be used to simulate and solve problems in regional water management, such as irrigation construction, drainage, automation of canal systems, dredging and flood protection. The software calculates (easily, accurately and fast) the flow in simple or complex channel networks, consisting of thousands of branches, cross sections and structures. You can define all types of boundary conditions, as well as define lateral inflow and outflow using time series or standard formulae. For more detail, the rainfall run-off process of urban areas and various types of unpaved areas can be modelled, taking into account land use, the unsaturated zone, groundwater, capillary rise and the interaction with water levels in open channels. For water quality and environmental problems the Water Quality module offers almost unlimited possibilities. The graphic display superimposes your network over a (GIS or aerial photo) map of the area so you can see the canals, reservoirs, weirs, pumping stations, treatment plants, urban and rural areas at a glance. Animation options show the direction of flow through the network and by varying the thickness and colours of selected network elements, all input and computed parameters can be visualised. By clicking on the map you can draw your network and adjust any detail to suit your needs. The network can also be viewed from the side. Side view allows you to print the design and watch the water profiles and real-time operation of structures in detail.

2.6.2 Introduction to urban modelling in D-Flow FM 1D2D

D-Flow FM 1D2D offers a comprehensive modelling tool for simple or extensive urban drainage systems consisting of sewers and open channels. It is a tool that can be used to simulate and solve problems in urban drainage systems such as determination of urban drainage capacities including treatment plants, assessment of sewer overflow frequency and design of detention basins. D-Flow FM 1D2D allows you to design new urban areas or analyse and improve existing ones. You can use it to find out what measures will prevent drainage congestion, street flooding and water pollution from sewer overflows. The return period of street flooding and sewer overflows can be analysed using long time series of rainfall data or storm events. The models help you to find out how the performance of the urban drainage system can be improved by a better operation of the pumps gates and weirs. The impact of treatment plants and sewer overflows on the receiving water can be analysed by combining everything into

DRAFT

one model. It's the ideal tool for the design, management and renovation of urban sewer systems. D-Flow FM 1D2D models the rainfall run-off process for various types of paved and unpaved areas. It doesn't matter whether the urban drainage system consists of open channels and sewer pipes, storage tanks and reservoirs. Even street flow can be modelled with D-Flow FM 1D2D. The application handles all kinds of cross sections, control structures and any network configuration (branched and looped). The size and complexity of the system doesn't matter either. The computation is extremely fast, also for large networks (e.g. 25 000 sewer pipes and manholes in one model). On top of that, D-Flow FM 1D2D offers virtually any real-time control option for pumps, weirs and gates in the urban system. The graphic display superimposes your network over a (GIS or aerial photo) map of the area so you can see sewer pipes, manholes, canals weirs and pumping stations at a glance. By clicking on the map you can draw your network and adjust any detail to suit your needs. Animation options show the direction of flow and by varying the thickness and colours of network elements all input and computed parameters can be visualised. The network can also be viewed from the side and allows you to print the design and watch the filling and drying process in detail. The software conforms to strict Dutch guidelines for sewerage calculations.

2.6.3 Introduction to river modelling in D-Flow FM 1D2D

D-Flow FM 1D2D is also designed for simple and complex river systems and estuaries. It is a tool that can be used to simulate and solve problems in river water management such as flood protection, flood-risk assessment, real-time forecasting, dam break analysis, navigation and dredging. This module can be used stand-alone or in combination with other modules. It simulates the water flows, the water quality and morphological changes in river systems, estuaries and other types of alluvial channel networks. D-Flow FM 1D2D is able to work with complex cross-sectional profiles consisting of various sub-sections.

2.6.4 Overview of modelling approaches in D-Flow FM 1D2D

With D-Flow FM 1D2D six different approaches are possible:

- ◇ 1D modeling (stand-alone), using a (1D) network of computational cells (for river, rural and urban modelling)
- ◇ 2D modeling (stand-alone), using a (2D) grid with computational cells (for river, rural and urban modelling)
- ◇ 1D2D lateral coupling (in particular for river modelling)
- ◇ 1D2D embedded coupling (in particular for rural modelling)
- ◇ 1D2D longitudinal coupling (in particular for estuarine modelling)
- ◇ 1D2D vertically stacked coupling (in particular for urban modelling)

The latter four approaches apply a combination of a (1D) network and a (2D) grid, so that they contain 1D2D links. These approaches for 1D2D link types can be distinguished as follows:

1D2D lateral A sideways link between the 1D channel and 2D flood plains on the side(s), for example for rivers. 1D and 2D domain parts do not overlap. Further described in section 17.7.3.

1D2D embedded Very similar to the lateral link, but intended for smaller 1D channels, where the 2D grid cells overlap the 1D channels, for example in rural or marshland areas. Also described in section 17.7.3.

1D2D longitudinal The 1D channel is linked to the 2D grid part in the flow direction (i.e., not lateral), for example to model an upstream 1D river feeding into a 2D river or estuary. Further described in section 17.7.4.

1D2D vertically stacked A vertical link from 2D planes to 1D underground sewer networks. Further described in section 17.7.5.

DRAFT

2.6.5 Important differences compared to SOBEK

SOBEK is the Deltares' modelling suite for 1D and 1D2D modelling for flood forecasting, optimisation of drainage systems, control of irrigation systems, sewer overflow design, river morphology, salt intrusion and surface water quality, which has been developed in the 90s. At first, the SOBEK 2 software suite was developed, which was later succeeded by the SOBEK 3 suite, which is based on a Delta Shell Framework for its GUI. Currently Deltares is working on important technological innovations, in particular the possibility of unstructured grids, via the Delft3D FM Suite 1D2D and Delft3D FM Suite 2D3D for integral simulations. Both suites use the same computational core, but the have different GUIs. The Delft3D FM Suite 1D2D will be the successor of SOBEK.

In this section the main differences between D-Flow FM 1D2D and SOBEK 2/SOBEK 3 will be presented, which will appear in a future version of this user manual.

DRAFT

2.7 Deltares Software Development Canvas

For high quality, dependable and future proof of Deltares' software, the **Deltares Software Development Canvas** is applied. The canvas provides a consistent way to classify the maturity of our software products and prototypes, so that teams, strategic partners and clients, and other stakeholders know exactly what a release does—and does not—offer at any given moment software evolves through several stages: from the first idea, to experimentation, to prototypes, to stable releases, to fully mature products, and eventually—when the time comes—to phase out and replacement. Together, these stages form the software lifecycle. The Deltares Software Development Canvas describes these stages in a clear, recognisable way, making it easy to see where a product stands and what that means for its use in projects, research, or collaboration with partners and clients. Below is a brief overview of the five levels in the Deltares software lifecycle, based on the widely used Technology Readiness Level (TRL) framework:

- **Concept (TRL 1–3)**; This stage covers early ideas and experimental models, which is relevant for scientific code, new methods or exploratory prototypes. These concepts are not yet validated or supported, and pre compiled versions are only available under Pre Release Terms and Conditions. This phase is ideal for research and experimentation, helping us explore promising directions before investing in full product development.
- **Prototype (TRL 4–6)**; Prototypes are used in pilots and internal projects. They are field tested, iteratively improved and shared with a small network of strategic partners and clients. Pre compiled prototypes still fall under Pre Release Terms and Conditions. Feedback is crucial at this stage to understand their real world potential. This is where early ideas start maturing into viable software solutions.
- **Stable (TRL 7–8)**; Stable releases offer fully validated, reliable functionality tailored to specific use cases or regions. Pre compiled stable releases are supported for key accounts and partners under the Stable Release Terms and Conditions; others may still access them under Pre Release Terms and Conditions. This marks the transition from development to operational readiness, where software becomes robust and dependable.
- **General Available (GA) (TRL 9)**; GA software is widely adopted, internationally validated and fully supported. Pre compiled GA releases are available through standard licensing or service packages under GA Terms and Conditions. Achieving GA requires consistent performance across diverse contexts and organisational readiness for structured maintenance and support.
- **End of Life (EoL) and renewal**; Eventually, software reaches a point where maintenance is no longer viable or where a successor is ready. EoL marks the beginning of phasing out or replacing the software, ensuring Deltares continues to offer modern, secure and sustainable digital solutions.

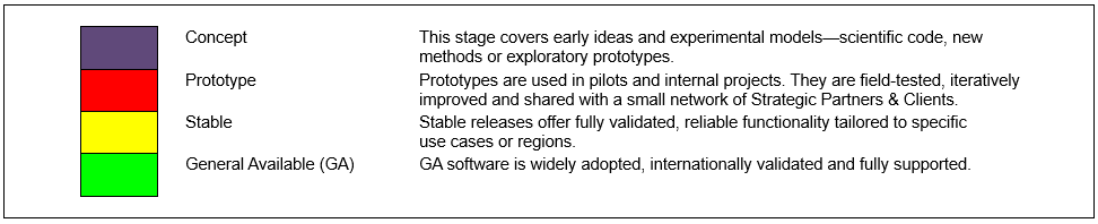


Figure 2.1: *Overview of TRL levels for software of Deltares*

DRAFT

3 Conceptual description of hydrodynamic equations

3.1 Introduction

The hydrodynamic module D-Flow FM simulates one-dimensional (1D), two-dimensional (2DH, depth-averaged) or three-dimensional (3D) unsteady flow and transport phenomena resulting from tidal and/or meteorological forcing, including the effect of density differences due to a non-uniform temperature and salinity distribution (density-driven flow). Also a combination of 1D and 2D modelling can be applied (1D2D). The hydrodynamic model is used to predict the flow in shallow seas, coastal areas, estuaries, lagoons, rivers, lakes and polders. It aims to model flow phenomena of which the horizontal length and time scales are significantly larger than the vertical scales. The many applications that are possible with D-Flow FM, either with D-Flow FM 1D2D or with D-Flow FM 2D3D have been summarized in section 2.1.

If the fluid is vertically homogeneous, a depth-averaged approach is appropriate. D-Flow FM is able to run in two-dimensional mode (one computational layer), which corresponds to solving the depth-averaged equations. Examples in which the two-dimensional, depth-averaged flow equations can be applied are tidal waves, storm surges, tsunamis, harbor oscillations (seiches) and transport of pollutants in vertically well-mixed flow regimes.

One-dimensional modelling or 1D2D modelling with D-Flow FM 1D2D is available for urban, rural and river and polder applications and sewer systems. Three-dimensional modelling with D-Flow FM 2D3D is of particular interest in transport problems where the horizontal flow field shows significant variation in the vertical direction. This variation may be generated by wind forcing, bed stress, Coriolis force, bed topography or density differences. Examples are dispersion of waste or cooling water in lakes and coastal areas, upwelling and downwelling of nutrients, salt intrusion in estuaries, fresh water river discharges in bays and thermal stratification in lakes and seas.

D-Flow FM is flexible by using an unstructured grid in the horizontal plane.

This section gives some background information on the conceptual model of the D-Flow FM module. Most of the concepts and algorithms are applicable to both the σ-model and Z-model.

3.2 Physical processes

The numerical hydrodynamic modelling system D-Flow FM solves the unsteady shallow water equations in one (transverse and depth-averaged), two (depth-averaged) or in three (multi-layer) dimensions. The system of equations consists of the horizontal equations of motion, the continuity equation, and the transport equations for conservative constituents. The equations are formulated in orthogonal Cartesian co-ordinates or in spherical co-ordinates on the globe. In Cartesian co-ordinates, the free surface level and bathymetry are related to a flat horizontal plane of reference, whereas in spherical co-ordinates the reference plane follows the Earth curvature.

The flow is forced by among others tides, currents and discharges at the open boundaries, wind stress at the free surface, pressure gradients due to free surface gradients (barotropic) or density gradients (baroclinic). Source and sinks or laterals are included in the equations to model the discharge and withdrawal of water and of constituents and substances, of which the latter are subject to one or more water quality processes.

The D-Flow FM model includes mathematical formulations that take into account the following physical phenomena:

DRAFT

- ◇ Propagation of long tidal waves (barotropic flow).
- ◇ Horizontal density gradients via the pressure term (baroclinic effects).
- ◇ Wind-driven flows; including space and time varying wind and atmospheric pressure, including tropical cyclone winds.
- ◇ Bed-stress including many options for vegetation.
- ◇ The effect of the Earth rotation (via Coriolis force).
- ◇ Heat exchange through the free water surface (via heat flux models).
- ◇ Transport of dissolved material and pollutants (via transport equation).
- ◇ Robust simulation of drying and flooding of inter-tidal flats and river winter beds.
- ◇ Turbulence modelling (via turbulence closure models) (only 3D).
- ◇ Transport of salt, heat and other conservative constituents.
- ◇ Evaporation and precipitation.
- ◇ Tide generating forces.
- ◇ Effect of secondary flow on depth-averaged momentum equations (for 3D effect in a 2D model).
- ◇ Influence of waves on the bed shear-stress (currently on for 2D).
- ◇ Wave induced stresses (radiation stress)and mass fluxes (currently on for 2D).
- ◇ Water quality processes in case of online coupling of hydrodynamics and water quality.
- ◇ Sediment transport and morphodynamics via online coupling with hydrodynamics.

3.3 Hydrodynamic modelling

D-Flow FM solves the Navier Stokes equations for an incompressible fluid, under the shallow water and the Boussinesq assumptions. In the vertical momentum equation the vertical accelerations are neglected, which leads to the hydrostatic pressure equation. The set of partial differential equations in combination with an appropriate set of initial and boundary conditions is solved on an unstructured finite volume grid.

Two coordinate references are supported in D-Flow FM:

1. Cartesian co-ordinates
2. Spherical co-ordinates

For the sake of simplicity in this chapter the equations are only in Cartesian co-ordinates.

3.3.1 Model equations in 1D

In one-dimensional (1D) models the water flow is described by the De Saint Venant (1871) equations for unsteady flow and are based upon the the following series of assumptions (Cunge *et al.*, 1980):

- ◇ The flow is one-dimensional i.e. the velocity can be represented by a uniform flow over the cross-section and the water level can be assumed to be horizontal across the section.
- ◇ The streamline curvature is small and the vertical accelerations are negligible, hence the pressure is hydrostatic.
- ◇ The effects of boundary friction and turbulence can be accounted for through resistance laws analogous to those used for steady flow.
- ◇ The average channel bed slope is small so that the cosine of the angle it makes with the horizontal may be replaced by unity.

For one-dimensional flow the momentum equation and the continuity equation may either be solved in the pure 1D form, or in the quasi 2D form. By default, D-Flow FM solves the 1D equations as if the domain had been schematized using a 2D model with a mesh of one grid cell wide. This is the quasi 2D approach. The important difference between this approach and

DRAFT

the pure 1D approach is that the 2D momentum is a vectorial quantity and only the momentum component that aligns with the downstream channel moves downstream, while the remainder is converted to potential energy (i.e. results in a water level rise). For 1D the surface area, cross-sectional area, and volume may non-linearly depend on the water level. As a result a different notation is used compared to 2D (and 3D) modelling. This will be described in the subsequent sections.

3.3.1.1 Momentum equation in 1D

The 1D momentum equation reads:

$$\frac{\partial U}{\partial t} + U\frac{\partial U}{\partial x} + g\frac{\partial \zeta}{\partial x} + \frac{gU|U|}{C^2 R} - w_f \frac{\tau_{wind}}{A_F \rho_w} = 0 \tag{3.1}$$

The first term describes the inertia, the second term the convection, the third term the water level gradient, the fourth term the bed friction and the fifth term describes the influence of the wind force, in which

U	velocity $[m/s]$
A_F	flow area $[m^2]$
C	Chézy value $[m^{1/2}/s]$
g	acceleration due to gravity $[m/s^2]$
ζ	water level $[m]$
R	hydraulic radius $[m]$
t	time $[s]$
w_f	water surface width $[m]$
x	distance along the channel axis $[m]$
ρ_w	density of fresh water $[kg/m^3]$
τ_{wind}	wind shear stress component aligned with the channel $[N/m^2]$

This is the pure 1D momentum equation whereas D-Flow FM uses, by default, a quasi 2D approach on 1D networks. See section 3.3.1.9 for more details on this choice and how to influence it.

3.3.1.2 Continuity equation (1D)

The continuity equation in 1D reads:

$$\frac{\partial A_T}{\partial t} + \frac{\partial A_F U}{\partial x} = Q_{lat} \tag{3.2}$$

with

A_T	the total cross-sectional area, i.e. the volume per unit length $[m^2]$
Q_{lat}	the lateral discharge per unit length $[m^2/s]$

The total cross-sectional area A_T is in general a non-linear function of the water level ζ. For a simple rectangular channel the cross-sectional $A_T = w_f h$ where w_f is the channel width [m] and h is the water depth [m].

3.3.1.3 Flow area

The flow area A_F of a cross-section is the area through which water is actually flowing.

In a cross-section, distinction can be made between the flow area and the storage area. In the latter part, water is stored only, see Figure 3.1.

DRAFT

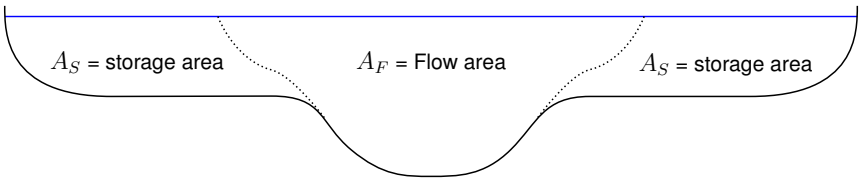


Figure 3.1: *Definiton of flow area (A_F) and storage area (A_S)*

The shape of the cross-section, and the distinction between the flow area and storage area are defined for each cross-section by user input.

3.3.1.4 Storage area

The storage area A_S of a cross-section is the area in which only water is stored i.e. the non-conveying part of the cross-section.

In a cross-section, distinction can be made between the flow area and the storage area, see Figure 3.1.

The shape of the cross-section, and the distinction between the flow area and storage area is defined for each cross-section by user input.

The total area A_T is defined as the flow area plus the storage area.

3.3.1.5 Wetted area

The wetted area (A_F [m^2]) is the part of the cross section that is filled with water. See Figure 3.2

3.3.1.6 Wetted perimeter

The wetted perimeter (O [m]) is the interface between the soil and the flowing water. See Figure 3.2.

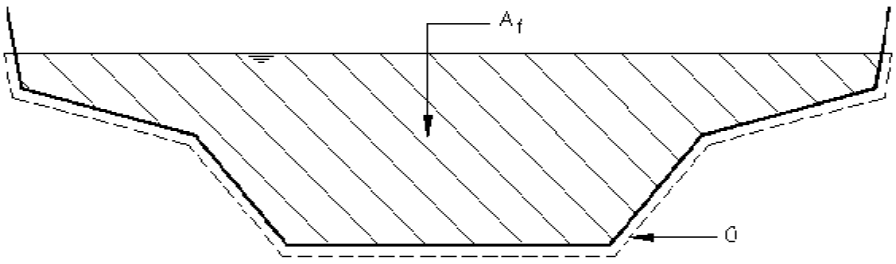


Figure 3.2: *Wetted perimeter*

In the case of a main channel and one or two floodplains in the cross-section, a wetted perimeter is computed for each section. In that case the water interface between the sections is not included in the wetted perimeter. See also Figure 3.2 and Figure 3.3. The wetted perimeter is used to compute the hydraulic radius, which is important for the computation of the bed friction.

DRAFT

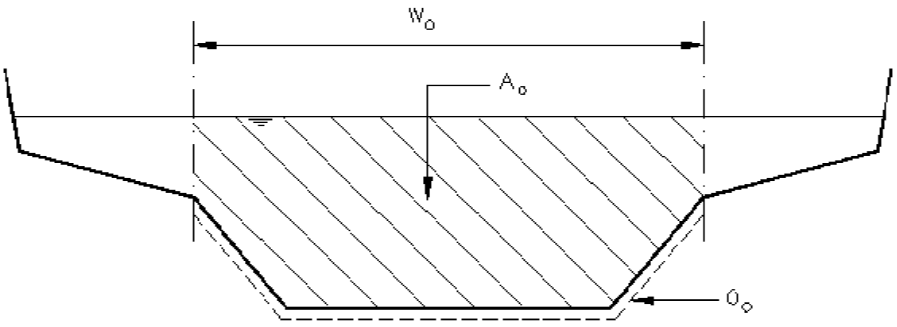


Figure 3.3: *Wetted perimeter for the main channel*

3.3.1.7 Velocity

The velocity (U) is defined as the average flow velocity in the wetted area of the cross section. It is given in [m/s] and can be written as

$$u = \frac{Q}{A_F} \tag{3.3}$$

Note: Discharges and velocities are defined in branch segments, whereas cross sections are defined in ζ-calculation points. For the computation of the velocities according to the above described formula the D-Flow 1D module uses the upstream cross section.

3.3.1.8 Hydraulic radius

The hydraulic radius is defined as the wetted area of the cross section divided by the wetted perimeter.

$$R = \frac{A_F}{O} \tag{3.4}$$

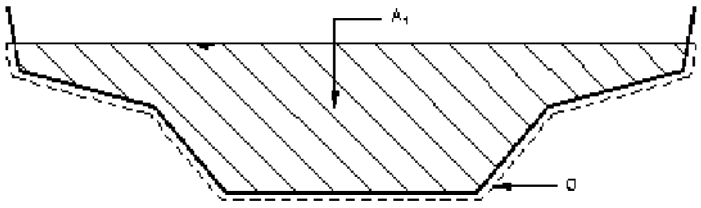


Figure 1 Computation of hydraulic radius

Figure 3.4: *Computation of hydraulic radius*

DRAFT

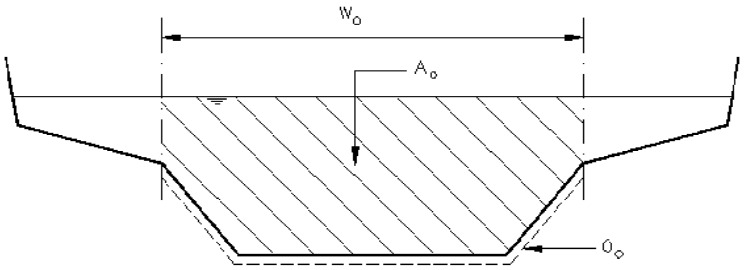


Figure 3.5: *Wetted perimeter for the main channel*

3.3.1.9 Momentum equation for 1D networks

By default, D-Flow FM always uses the two-dimensional momentum equation in the horizontal plane. This has the advantage that the 1D and 2D flow solutions are consistent and head losses at bends are included implicitly. A validation of the bend losses using the experiments of Malone and Parr (2008) showed good agreement with measurements, but there could be situations in which one might like to deviate from this default approach. A keyword `Pure1D` has been implemented that allows you to switch to the pure one-dimensional momentum equation as described in section 3.3.1.1.

Situations that may require such an approach include:

1. When converting from an existing 1D SOBEK/D-Flow 1D schematisation and desiring to obtain very similar results. An existing, calibrated model may include local friction losses to represent bend losses that are not represented by the one-dimensional momentum equation. By converting such a schematisation to D-Flow FM and using the standard `Pure1D = 0` option, you may be accounting for the same losses more than once.
2. When the resolution of your model is too coarse to represent the change in curvature accurately. Figure 3.6 shows that the angle between two subsequent segments can be much larger when using a coarse resolution, than in the actual river branch geometry. In the real geometry the change is more gradual, and although some extra energy losses may be expected, they are not as large as when using the coarse grid setup. The choice of a coarse grid representation may therefore lead to mild curvature variations wrongfully being characterized as strong curvature variations, thereby giving unwanted extra water level setup.
3. When running simulations that include morphodynamics. The reason for this is that the bend effect manifests itself by an increased water level and decreased flow velocity at the nodes. These local changes in the velocity are amplified in the sediment transport (typically the velocity raised to a certain power) which governs the morphological development. The consequence when running simulations including morphology is that locally a spike in sedimentation occurs in bends, which does not match the real behaviour.

Two keywords have been introduced to influence the momentum treatment in 1D networks.

- `Pure1D` = 0, 1, 2, 3, 4, 5, 6, 7 allows you to select the way in which the momentum should be solved along simple channels with one incoming and one outgoing channel at a node. With the default option (0) the quasi 2D approach is used. When the keyword is set to 1 or 2, the 1D momentum equation is solved using a discretization that is consistent with 2D D-Flow FM. When the value 1 is used, flow volume is used for weighting the momentum contribution of the two connected nodes, whereas the total volume is used when the value 2 is specified. For values in the range of 3 to 7, the discretization matches the various `Iadvec1D` options of SOBEK 2. The value 3 corresponds to the momentum conserving

Figure 3.6: *Example of river network with gradual change in curvature along the river, and resulting coarse model representation with strong curvature changes.*

default setting of SOBEK 2. The value 7 corresponds to an energy conserving approach, while 4 used a weighted combination of the two approaches. In case of 5 the weighted combination is used in contractions, and momentum conservation in expansions. For option 6 this is the other way around.

◇ `Junction1D` = 0, 1 allows you to select the way in which the momentum should be solved at channel junctions and confluences. This option is only applicable for `Pure1D` equal to 1 and 2. In those cases, by default the quasi 2D approach is used (0). When the keyword is set to 1, the implementation approximates a pure 1D approach. When `Pure1D` values in the range 3–7, the 1D momentum equation is always also applied at channel junctions and confluences.

When aiming for a pure 1D model, `Pure1D` = 3 is recommended. At computational points that combine 1D and 2D, the 2D approach is always used and hence the direction of the connected 1D segment is important at such locations. See also the settings description in chapter A.

3.3.2 Model equations in 2D

In this section, we will present the governing equations in two horizontal directions, which involves two momentum equations and the continuity equation.

3.3.2.1 Momentum equations in horizontal direction (2D)

The momentum equations in x- and y-direction are given by:

$$\frac{\partial U}{\partial t} + U\frac{\partial U}{\partial x} + V\frac{\partial U}{\partial y} - fV = -\frac{1}{\rho_0}\frac{\partial P}{\partial x} + F_x + \frac{\partial}{\partial z}\left(\nu_V \frac{\partial U}{\partial z}\right) + M_x \tag{3.5}$$

$$\frac{\partial V}{\partial t} + U\frac{\partial V}{\partial x} + V\frac{\partial V}{\partial y} + fU = -\frac{1}{\rho_0}\frac{\partial P}{\partial y} + F_y + \frac{\partial}{\partial z}\left(\nu_V \frac{\partial V}{\partial z}\right) + M_y \tag{3.6}$$

Where U and V are the depth-averaged currents and ν_V is the vertical eddy viscosity coefficient. Density variations are neglected, except in the baroclinic pressure terms, $\partial P/\partial x$ and $\partial P/\partial y$ represent the pressure gradients.

The forces F_x and F_y in the momentum equations represent the unbalance of horizontal Reynolds stresses.

M_x and M_y represent the contributions due to external sources or sinks of momentum (external forces by hydraulic structures, discharge or withdrawal of water, wave stresses, etc.).

3.3.2.2 Continuity equation (2D)

D-Flow FM solves the depth-averaged continuity equation, derived by integration the continuity equation, for incompressible fluids ($\nabla \bullet \boldsymbol{u} = 0$) over the total depth, taken into account the kinematic boundary conditions at water surface and bed level, and is given by:

$$\frac{\partial h}{\partial t} + \frac{\partial U h}{\partial x} + \frac{\partial V h}{\partial y} = Q \tag{3.7}$$

with U and V the depth averaged velocities. Q is representing the contributions per unit area due to the discharge or withdrawal of water, precipitation and evaporation:

$$Q = \int_0^h (q_{in} - q_{out})\, dz + P - E \tag{3.8}$$

with q_{in} and q_{out} the local sources and sinks of water per unit of volume [1/s], respectively, P the non-local source term of precipitation and E non-local sink term due to evaporation. We remark that the intake of, for example, a power plant is a withdrawal of water and should be modelled as a sink. At the free surface there may be a source due to precipitation or a sink due to evaporation.

3.3.3 The hydrostatic pressure assumption

Under the shallow water assumption, the vertical momentum equation is reduced to a hydrostatic pressure equation. Vertical accelerations due to buoyancy effects and due to sudden variations in the bed topography are not taken into account. So:

$$\frac{\partial P}{\partial z} = -\rho g \tag{3.9}$$

For water of constant density and taking into account the atmospheric pressure, it includes gradients of the free surface level, called barotropic pressure gradients. The atmospheric pressure is included in the system for storm surge simulations. The atmospheric pressure gradients dominate the external forcing at peak winds during storm events. Space and time varying wind and pressure fields are especially important when simulating storm surges.

In case of a non-uniform density the pressure gradients includes not only barotropic pressure gradient, but also vertical pressure gradient, the so called baroclinic pressure gradient. The baroclinic pressure gradient is the result of variable distribution of density and temperature in the vertical direction.

DRAFT

3.3.4 The Coriolis force

The Coriolis parameter f depends on the geographic latitude ϕ and the angular speed of rotation of the earth, Ω ($f = 2\Omega \sin \phi$). For a grid the user should specify the space varying Coriolis parameter, using a suitable projection.

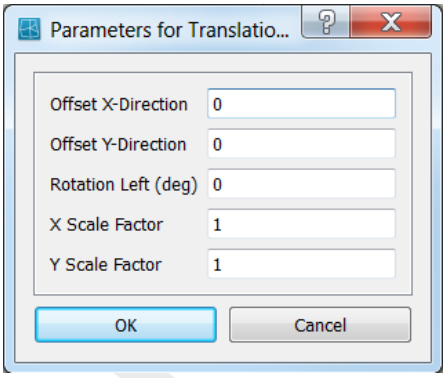


Figure 3.7: *Input for map projection for specifying Coriolis parameter on the grid.*

3.3.5 Diffusion of momentum

The forces F_x and F_y in the horizontal momentum equations represent the unbalance of horizontal Reynolds stresses. The Reynolds stresses are modelled using the eddy viscosity concept, (for details e.g. Rodi (1984)). This concept expresses the Reynolds stress component as the product between a flow as well as grid-dependent eddy viscosity coefficient and the corresponding components of the mean rate-of-deformation tensor. The meaning and the order of the eddy viscosity coefficients differ for 2D and 3D, for different horizontal and vertical turbulence length scales and fine or coarse grids. In general the eddy viscosity is a function of space and time.

For 3D shallow water flow the stress tensor is an-isotropic. The horizontal eddy viscosity coefficient, ν_H, is much larger than the vertical eddy viscosity ν_V ($\nu_H \gg \nu_V$). The horizontal viscosity coefficient may be a superposition of three parts:

1 a part due to "sub-grid scale turbulence",
2 a part due to "3D-turbulence" see Uittenbogaard *et al.* (1992) and
3 a part due to dispersion for depth-averaged simulations.

In simulations with the depth-averaged momentum and transport equations, the redistribution of momentum and matter due to the vertical variation of the horizontal velocity is denoted as dispersion. In 2D simulations this dispersion is not simulated as the vertical profile of the horizontal velocity is not resolved. Then this dispersive effect may be modelled as the product of a viscosity coefficient and a velocity gradient. The dispersion term may be estimated by the Elder formulation.

If the vertical profile of the horizontal velocity is not close to a logarithmic profile (e.g. due to stratification or due to forcing by wind) then a 3D-model for the transport of matter is recommended.

The horizontal eddy viscosity is mostly associated with the contribution of horizontal turbulent motions and forcing that are not resolved by the horizontal grid ("sub-grid scale turbulence") or by (a priori) the Reynolds-averaged shallow-water equations. in 3D, in the vertical direction, ν_V is referred to as the three-dimensional turbulence and in it is computed following a 3D-

turbulence closure model.

Therefore, in addition to all turbulence closure models in D-Flow FM a constant (space and time) background mixing coefficient may be specified by the user, which is a background value for the vertical eddy viscosity in the momentum Equation (3.5) and Equation (3.6) consequently.

The horizontal and vertical eddy viscosities can be set by user defined value under `Physical Parameters` shown in Figure 3.8.

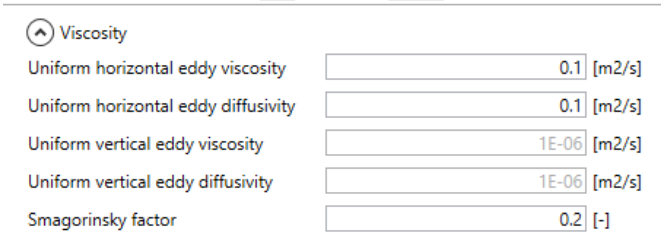


Figure 3.8: *Input parameters for horizontal and vertical eddy viscosities.*

The default approach for the computation of the horizontal viscosity in D-Flow FM is the so-called Smagorinsky approach, which results into time and space varying horizontal viscosity coefficients. Due to the explicit time integration of the horizontal viscosity term a time step criterion is applied in order to ensure stable model results. If this criterion is violated, then the horizontal viscosity coefficient at a certain flow link is reduced. Next, in the diagnostic file a warning is generated as shown below.

```
** WARNING: Viscosity coefficient was limited for 3309 links.
** WARNING: Viscosity coefficient was limited for 3269 links.
** WARNING: Viscosity coefficient was limited for 3411 links.
```

Per time step is shown in how many flow links the horizontal viscosity coefficient has been reduced. After ten warnings this message is stopped, in order to prevent that the diagnostic file will become very large. The following message is written in the diagnostic file at the end of the simulation:

```
** INFO   : Viscosity coefficient/Horizontal transport flux were limited on some links in the course of computation.
```

3.3.6 Bed friction

The boundary conditions for the momentum equations 3.5 and 3.6 at the bed ($z = z_b$) are:

$$\left.\frac{\nu_V}{H}\frac{\partial u}{\partial \sigma}\right|_{\sigma=0} = \frac{1}{\rho_0}\tau_{bx} \tag{3.10}$$

and

$$\left.\frac{\nu_V}{H}\frac{\partial v}{\partial \sigma}\right|_{\sigma=0} = \frac{1}{\rho_0}\tau_{by} \tag{3.11}$$

with τ_{bx} and τ_{by} the components of the bed stress in x- and y-direction, respectively. These equations hold in 2D and 3D parts of the model, in 1D modelling the equation reduces to a single component along the channel.

Depth-averaged flow

For 2D depth-averaged flow the shear-stress at the bed induced by a turbulent flow is assumed to be given by a quadratic friction law:

$$\boldsymbol{\tau}_b = \frac{\rho_0 g \boldsymbol{U} |\boldsymbol{U}|}{C_{2D}^2}, \tag{3.12}$$

where $|\boldsymbol{U}|$ is the magnitude of the depth-averaged horizontal velocity. In D-Flow FM we support the following equations to compute the C_{2D} coefficient. The numbers and strings following the name of the roughness formulation indicate the number and name by which the formulations are selected in the input files.

◇ **Chézy formulation**: 0, `Chezy`

$$C_{2D} = \text{Chézy coefficient [m}^{1/2}\text{/s]}. \tag{3.13}$$

◇ **Manning's formulation**: 1, `Manning`
As formulated by Manning (1891), and also known as Gauckler-Manning-Strickler formulation.

$$C_{2D} = \frac{\sqrt[6]{R}}{n} \tag{3.14}$$

where:

R	is the hydraulic radius
n	is the Manning coefficient [m$^{-1/3}$s].

◇ **Log law of the Wall**: 2, `WallLawNikuradse`

$$C_{2D} = \frac{\sqrt{g}}{\kappa} \ln\left(\frac{30R}{ek_n}\right) \tag{3.15}$$

where:

R	is the hydraulic radius
k_n	is the Nikuradse roughness length [m].

◇ **White-Colebrook's formulation**: 3, `WhiteColebrook`
As formulated by Colebrook and White (1937) and Colebrook (1939).

$$C_{2D} = 18\,{}^{10}\log\left(\frac{12R}{k_n}\right) \tag{3.16}$$

where:

R	is the hydraulic radius
k_n	is the Nikuradse roughness length [m].

◇ **Strickler's formulation based on Nikuradse**: 7, `StricklerNikuradse`
As formulated by Strickler (1923).

$$C_{2D} = 25\sqrt[6]{\frac{R}{k_n}} \tag{3.17}$$

where:

DRAFT

R is the hydraulic radius
k_n is the Nikuradse roughness length [m].

◇ **Strickler's formulation**: 8, `Strickler`
As formulated by Strickler (1923).

$$C_{2D} = k_s \sqrt[6]{R} \tag{3.18}$$

where:

R is the hydraulic radius
k_s is the Strickler roughness coefficient [m$^{1/3}$/s].

◇ **de Bos & Bijkerk's formulation**: 9, `deBosBijkerk`
As formulated by De Bos and Bijkerk (1963).

$$C_{2D} = \gamma \sqrt[3]{H} \sqrt[6]{R} \tag{3.19}$$

where:

H is the total water depth
R is the hydraulic radius
γ is the De Bos & Bijkerk coefficient [1/s].

Alluvial bed properties and land use data can be used to determine the roughness and flow resistance. Specific functionality has been implemented to make the translation of bed and land use properties into roughness and flow resistance characteristics. See section 12.3 for details.

3.3.6.1 Roughness definition input

The roughness definition (or: friction values) is different for 1D network and 2D grid parts:

◇ 1D network parts are typically defined using branch locations and cross section definitions. Therefore, the description of this is placed in section 3.4.6.
◇ 2D grid parts may use a global friction value, specified in the MDU file (chapter A) under `[physics]`, keyword `UnifFrictCoef`. Note that the available `UnifFrictType` values are limited to 0, 1, 2, 3.
◇ Alternatively, 2D grid parts may have spatially varying friction values defined via the parameter field file, specified in the MDU file under `[geometry]`, keyword `IniFieldFile`. The spatially varying friction values have to be provided in a separate data file, and interpolation of that data onto the 2D grid is controlled by several parameters in the `IniFieldFile`. All details are given in section D.2.

3.4 Cross sections in 1D

3.4.1 Introduction

The bathymetry of a 1D element is defined by means of a cross section.

The available cross section types in D-Flow FM are:

◇ YZ cross section (section 3.4.2.1)
◇ XYZ cross section (section 3.4.2.1)
◇ ZW (tabulated) cross section (section 3.4.2.2)
◇ ZWriver cross section (section 3.4.2.3)
◇ Circle cross section (section 3.4.2.4)
◇ Rectangle cross section (section 3.4.2.5)

DRAFT

◇ Template cross section (section 3.4.2.6)

These cross sections are used for calculating the conveyance and flow area of a channel at a flow velocity point and the total volume in water level points.

For the calculation of the conveyance D-Flow FM offers two different approaches.

◇ Vertically segmented approach (section 3.4.5.1).
◇ Lumped approach (section 3.4.5.2)

For YZ and XYZ type cross sections the user can choose for either the vertically segmented approach (default) or the lumped approach. For all other cross section types the lumped approach is used.

The ZWriver cross section is a mix between the lumped and a vertically segmented approach. For this cross section type the user can define three different sections. (i.e. 'Main', 'Floodplain1' and 'Floodplain2'). For each separate section a lumped conveyance is calculated and summed up to construct a total conveyance.

Cross sections can be defined at arbitrary locations on branches. The required data is then interpolated to the correct location. The interpolation is described in section 3.4.7.

3.4.2 Cross section types

3.4.2.1 YZ and XYZ cross section

In a YZ-type cross section the user specifies the bed level, i.e., Z-value at given Y-points (see Figure 3.9).

An XYZ-type cross section contains (x, y) coordinates of the bed level locations. In the computational core an XYZ-cross section is changed into a YZ-type cross section. The distance between the different (x, y) locations is used to construct the relative Y-values for the YZ cross section.

Remarks:

◇ The Y-values in the YZ cross section must be given as a metric value ([m AD]), regardless of the coordinate reference system of the model.
◇ The (x, y) coordinates in the XYZ cross section must given in the coordinate reference system of the model (that is, in metres for projected coordinate systems, or degrees for a spherical coordinate system).
◇ In case two subsequent Z-values are identical, the second value is raised with 10^{-3} m.

DRAFT

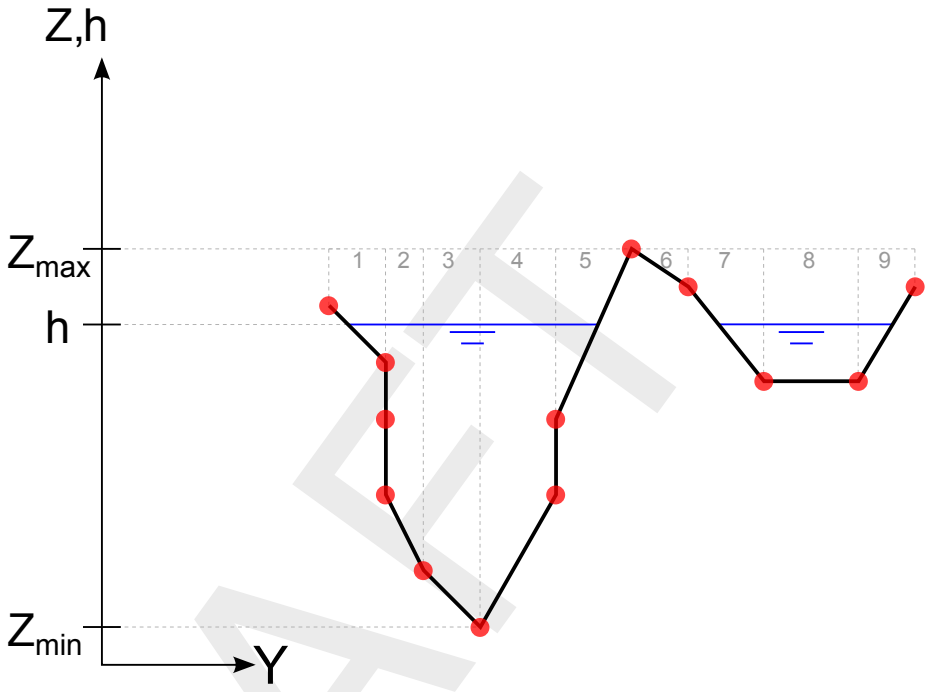


Figure 3.9: *Example of constructed vertical segments in a Y-Z profile*
13 (y_i, z_i) points red dots
9 vertical segments

3.4.2.2 ZW cross section

In a ZW-type cross section the user specifies the shape of the cross section by specifying the width at certain levels. For this cross section type it is possible to make a distinction between the flow area and the total area.

An input example for a ZW-cross section definition is given in Table 3.1. In Figure 3.10 the cross section for this example is shown. The blue shaded area shows the storage, i.e., non-flowing area.

Table 3.1: *Example of a ZW cross section definition*

Z	Width	Storage width
1	500	250
-4	450	330
-4.2	110	0
-10	33	0

DRAFT

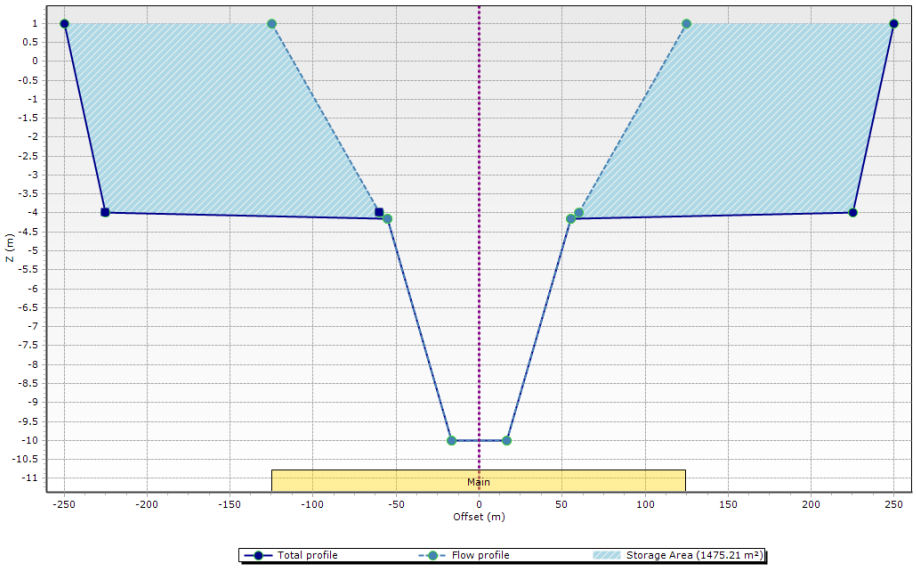


Figure 3.10: *Example a ZW cross section*

3.4.2.3 ZWriver cross section

Not yet described

3.4.2.4 Circle cross section

A circular shaped cross section is defined by its diameter. Section C.17.1.1 describes the cross section definition input.

3.4.2.5 Rectangle cross section

A rectangular cross section is defined by its width and height and whether it is open on the top side or not. Section C.17.1.2 describes the cross section definition input.

3.4.2.6 Template cross sections

A 'Template cross section' is offered by the GUI to support a number of standard shapes. These are translated into the equivalent ZW-definition. The computational kernel then treats these as a tabulated ZW-type cross section. See also the optional keyword `template` in section C.17.1.4.

3.4.3 Preissmann slot and closed cross sections

Closed cross sections are cross sections that have a 0 m width above a certain level. Closed cross sections can be used on ordinary branches. An application for this functionality is the modelling of sewer systems, where the sewer pipes are modelled as ordinary branches with a closed cross section.

Note: Long culverts are also modelled as ordinary branches in D-Flow FM.

In case a branch with a closed cross section is completely filled, the water flow becomes a pressurised flow and not a free surface flow. Since D-Flow FM is not designed for simulations with pressurised flow, a Preissmann slot is added at the top of the cross section.

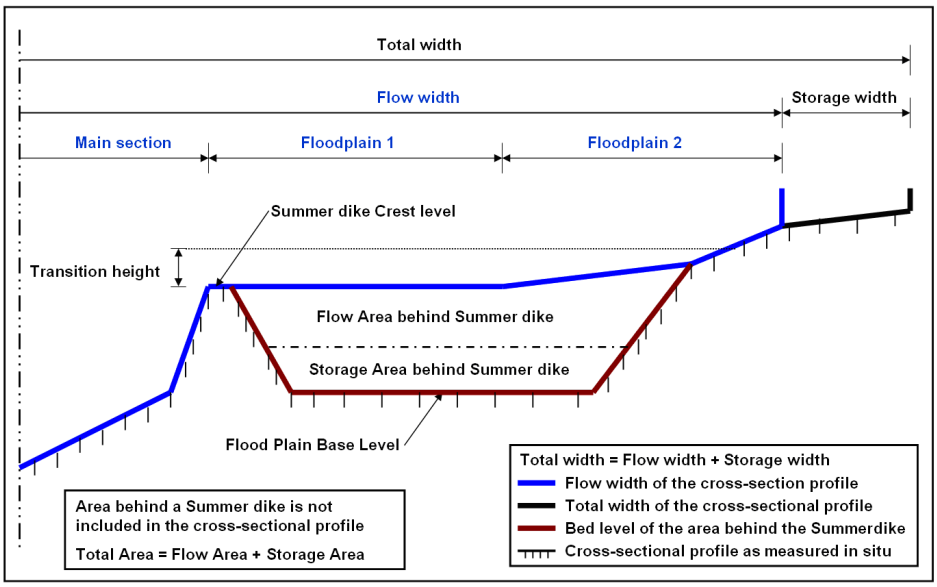


Figure 3.11: *levee option, available in a river profile*

A Preissmann slot is a narrow extension of the cross sectional area, above the part that is originally closed. As a result even closed pipes, will always have a small free surface area. This Preissmann slot is only used in calculating the amount of water in a water level point and in calculating the free surface area. The flow area and hydraulic radius are not affected by this Preissmann slot.

The width of the Preissmann slot can be set by the user, see section 17.7.5.

Closed cross sections can also be used for the hydraulic structures, bridges and culverts. Since hydraulic structures do not have any effect on the storage of a model or the free surface area, no Preissmann slot is required for closed cross sections in hydraulic structures.

3.4.4 Levees

A flood plain might be separated from a river by means of a small levee, having a crest level lower than the crest level of the main dike (see Figure 3.11). In the Netherlands large floods usually occur in winter. The main dike and the small levee are, therefore, respectively referred to as the “winter dike” and the “summer dike”.

The presence of a levee implies that the cross-sectional profile as measured in situ (see shadowed line in Figure 3.11) does not increase monotonously with rising water levels. When the water level becomes slightly higher than the crest level of the levee, first the area behind the levee is to be filled, before water levels can become any higher. This implies a local attenuation of the flood wave at water levels just above the crest level of the levee. This hydraulic phenomenon can be modelled using the levee option available at a river profile.

In defining a levee in a river profile a distinction is to be made between:

- ◇ **The cross-sectional profile**
  The cross-sectional profile comprises of a “flow area” and a “storage area”, which in Figure 3.11 are respectively located above the blue lines and the black lines. The summation of flow area and storage area is called the “total area”. Both flow area and total area are defined by its width as function of elevation. At each elevation in the cross-sectional profile

yields that storage width equals total width minus flow width. Hence, no storage is defined if total width equals flow width. The flow area of the cross-sectional profile can be divided in three separate flow sections, respectively called the (i) main section, (ii) floodplain 1 and (iii) floodplain 2 . Each such flow section can have a different roughness value and a different roughness formulation.

- ◇ **The area behind the summer-dike**
  In analogy with the cross-sectional profile, the "total area behind a levee" is divided in a "flow area" and a "storage area". The "Flood Plain Base Level" coincides with the lowest part of the area behind the levee (see Figure 3.11).

Remark:

- ◇ Both the cross-sectional profile and the area behind the levee should be mutually consistent with the cross-sectional profile as measured in situ (see shadowed line in Figure 3.11). Hence, both the flow area and the storage area behind the levee should not be included in the cross-sectional profile.

The so-called "transition height" is used for avoiding numerical oscillations. More precisely, the transition height ensures that the flow area and storage area behind the levee are gradually taken into computation. Transition height for levees is a model-wide parameter.

Lets consider the following levee properties:

- ◇ flood plain base level (z_{base}) of 2 m,
- ◇ levee crest level (z_{sd}) of 4 m,
- ◇ transition height (T) of 0.5 m,
- ◇ storage area behind the levee of 200 m^2,
- ◇ flow area behind the levee (ExtraFlowArea) of 50 m^2, and
- ◇ total area behind the levee (ExtraTotalArea) of 250 m^2.

In explaining how a levee is taken into computation, we make a distinction between:

- ◇ The actual value of the (extra) flow area behind the levee,
- ◇ The flow velocity applied to the (extra) flow area behind the levee, and
- ◇ The actual value of the (extra) total area behind the levee.

The actual (extra) flow area is a unique function of the local water level (h) in the cross-sectional profile (see Figure 3.12). More precisely:

- ◇ If $\zeta < z_{\text{sd}}$; Flow area behind the levee is zero.
- ◇ If $z_{\text{sd}} \leq \zeta \leq z_{\text{sd}} + T$; Flow area behind the levee varies as a quadratic function of water level (ζ) between zero and ExtraFlowArea (= 50 m^2).
- ◇ If $\zeta > z_{\text{sd}} + T$; Flow area behind the levee is equal to ExtraFlowArea.

The flow velocity applied to the (extra) flow area behind the levee is the flow velocity in one of the three available flow sections. Hence, either the flow velocity in the main section, in floodplain 1 or in floodplain 2. The actual flow velocity applied to the (extra) flow area behind the levee is determined as follows:

1. Default the flow velocity in floodplain 2 is used.
2. However, if the wetted area of floodplain 2 becomes less than 0.4 m^2, the flow velocity in floodplain 1 is used,
3. Furthermore, if the wetted area of floodplain 1 becomes less than 0.4 m^2, the flow velocity of the main section is used.

DRAFT

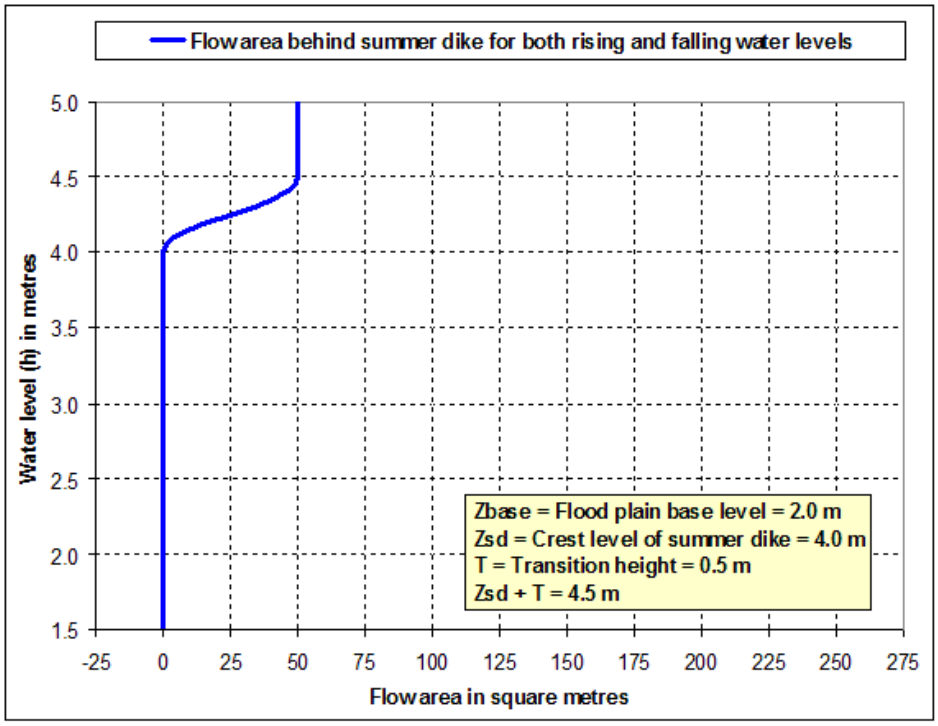


Figure 3.12: *Flow area behind the levee as function of local water levels.*

Remarks:

- ◇ Please contact Deltares if you like to use a different threshold value (i.e. not 0.4 m^2) for discerning which flow velocity is to be applied to the (extra) flow area behind the levee.
- ◇ The flow velocity applied to the (extra) flow area behind the levee is computed as follows

$$U_i = C_i\sqrt{R_i S} \qquad i \in \{\text{main section, floodplain 1, floodplain 2}\} \tag{3.20}$$

where:

U_i	average flow velocity
C_i	Chézy coefficient of the flow section
R_i	$(A_i + A_{\text{sd}})/P_i$: hydraulic radius of the flow section
A_i	wetted area of the flow section
A_{sd}	extra flow area behind the levee
P_i	wetted perimeter of the flow section
S	water level slope

- ◇ The discharge including the (extra) flow area behind the levee is computed as follows

$$Q_i = U_i(A_i + A_{\text{sd}}) \tag{3.21}$$

Figure 3.13 depicts the actual (extra) total area behind the levee as function of the local water level (ζ) in the cross-sectional profile. The (extra) total area behind the levee is added to the total area of the cross-sectional profile. There is a hysteresis in the extra total area for water levels varying from flood plain base level ($z_{\text{base}} = 2.0\ m$) to crest level of levee plus transition height ($z_{\text{sd}} + T = 4.5\ m$). We call line ABC the rising limp and line CA the falling limp of the hysteresis. The so-called “hysteresis flag” determines which limp of the hysteris is to be followed. In case the hysteresis `flag=1` the rising limp of the hysteresis is followed, else the falling limp is followed. At the onstart of a computation yields that the hysteresis `flag=1`. The actual applied extra total area behind the levee is computed as follows:

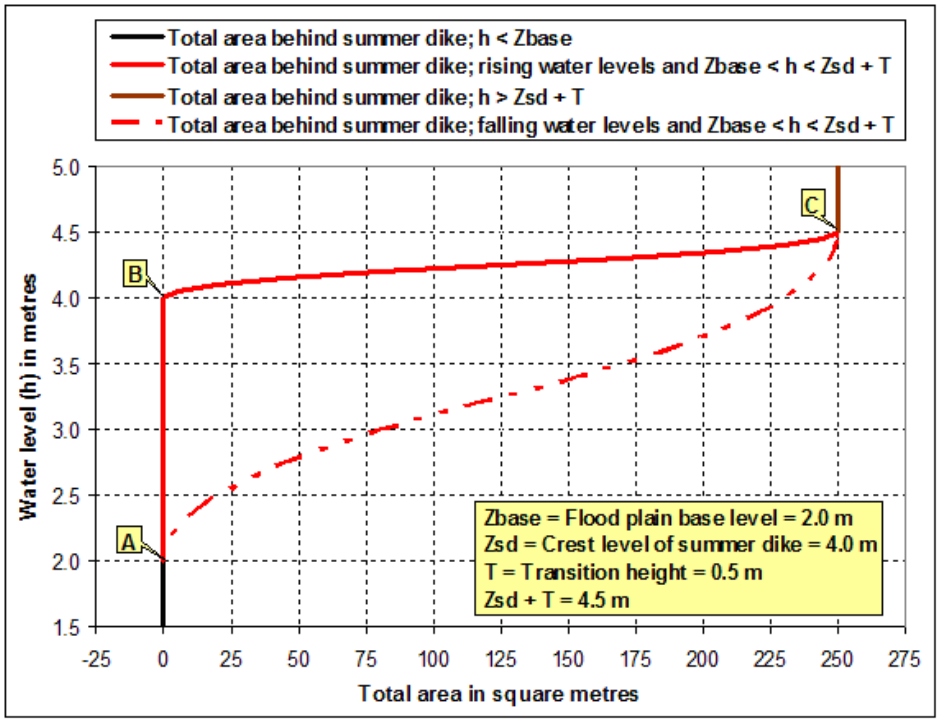


Figure 3.13: *Total area behind the levee as function of local water levels.*

1 If $\zeta < z_{\text{base}}$; Hysteresis flag is set to 1. Total area behind the levee is zero.
2 If $z_{\text{base}} \leq \zeta \leq z_{\text{sd}} + T$ and hysteresis `flag=1`;
   - ◇ For $z_{\text{base}} \leq \zeta < z_{\text{sd}}$; Total area behind the levee is zero.
   - ◇ For $z_{\text{sd}} \leq \zeta \leq z_{\text{sd}} + T$; Total area behind the levee varies as a quadratic function of water level (ζ) between zero and ExtraTotalArea (= 250 m^2)
3 If $z_{\text{base}} \leq \zeta \leq z_{\text{sd}} + T$ and hysteresis `flag=0`; Total area behind the levee varies as a quadratic function of water level (ζ) between zero and ExtraTotalArea.
4 If $\zeta > z_{\text{sd}} + T$; Hysteresis flag is set to 0. Total area behind the levee is equal to ExtraTotalArea.

3.4.5 Conveyance

The bed friction term in the 1D momentum equation is similar to the 2D bed friction in Equation (3.12):

$$\tau_b = \frac{\rho_0 g U |U|}{C^2}, \tag{3.22}$$

with Chézy coefficient C. In channels with flood plains, it is preferable to compute bed friction effects in vertical segments (more details follow), and for this purpose we rewrite the bed friction in terms of conveyance K. The conveyance is defined by

$$K = C A_F \sqrt{R}, \tag{3.23}$$

where A_F is the flow area and R is the hydraulic radius.

Substituting the conveyance in the bed friction term yields:

$$\tau_b = \frac{\rho_0 g A_F^2 U |U|}{K^2}. \tag{3.24}$$

In D-Flow FM two different approaches are implemented for calculating the 1D conveyance:

- ◇ Vertically segmented approach, this approach only applies to YZ cross section definitions. It is preferable for cross section shapes that have significant depth differences between large parts of the cross section, such as river like channels with large floodplains. (section 3.4.5.1)
- ◇ Lumped approach, this approach is preferable for smaller canals and sewer systems, also because it is better for including roughness of (nearly) vertical walls. (section 3.4.5.2)

3.4.5.1 Conveyance according to the vertically segmented approach

The vertically segmented conveyance approach is suited for modelling flow in rivers, since in this approach it is assumed that flow velocities vary significantly across the cross-sectional profile for a given water level. In line with this assumption, the cross-section is divided in vertical segments (see Figure 3.9).

A Y-Z profile is defined by a number (n) of (y_i, z_i) points. The total number of vertical segments equals: $n_{\text{seg}} = n - 1$. The bed or wall stress effects of nearly vertical walls is not well included in computing the conveyance of a Y-Z cross section. This underestimation of vertical wall stresses in computing the conveyance of a Y-Z profile, means that for a canal having a rectangular profile with a small ratio of width and water depth only, a substantial difference in discharge capacity can occur if such a canal is modelled as a rectangular cross-section (i.e. lumped conveyance approach) or as a rectangular shaped Y-Z profile (i.e. vertically segmented conveyance approach).

The conveyance (K_i) of each vertical segment is computed separately (for formulae, see conveyance per vertical segment further below). The conveyance of each inclined vertical segment is determined by solving an integral from y_i to y_{i+1}. The advantage of using an integral is that irrespective of the number of user-defined intermediate points, that lie on a straight line between (y_0, z_0) to (y_1, z_1), the same conveyance for the cross-sectional part from y_0 to y_1 is computed. The total conveyance (K_{tot}) at a particular water level is the summation of the conveyance of all the vertical segments ($K_{tot} = \sum K_i$).

Conveyance tables, applied in the vertically segmented conveyance approach

Conveyance tables are a table representation of the conveyance integral values and are used for increasing the computational performance. A conveyance table gives the conveyance, flow width and flow area of a cross-section for a finite number of water levels. Conveyance tables are used for Y-Z cross sections only. These conveyance tables are made during initialisation, before the start of a hydrodynamic calculation. During the computation the conveyance at a certain water level is determined by means of linear interpolation between table values for the two surrounding support points. In constructing the conveyance table of each user-defined cross-section, it is ensured that the difference between the interpolated conveyance and the exact conveyance is less than 1 %.

Conveyance tables are made from the bed-level up to the highest (y_i, z_i) point, irrespective of the actual location of the highest (y_i, z_i) point within the cross-sectional profile.

DRAFT

Extrapolation of conveyance tables

When the water level (ζ) rises above the highest level ($n_{\max}$) of a conveyance table, the conveyance ($K(\zeta)$) is computed as

$$K(\zeta) = a(\zeta - z_{\min})^b, \tag{3.25}$$

where $z_{\min}$ is the lowest level in the cross-sectional profile and a, b are constants derived from the conveyances at levels $n_{\max}$ and $n_{\max} - 1$.

Conveyance per vertical segment

In this section, a single vertical segment i is considered, for notational simplicity defined by two (y_0, z_0) and (y_1, z_1) points, which do not lie on top of each other ($y_0 \neq y_1$) (see Figure 3.14). The local water depth $d(y)$ at a particular point on this segment is defined as:

$$d(y) = \zeta - z(y). \tag{3.26}$$

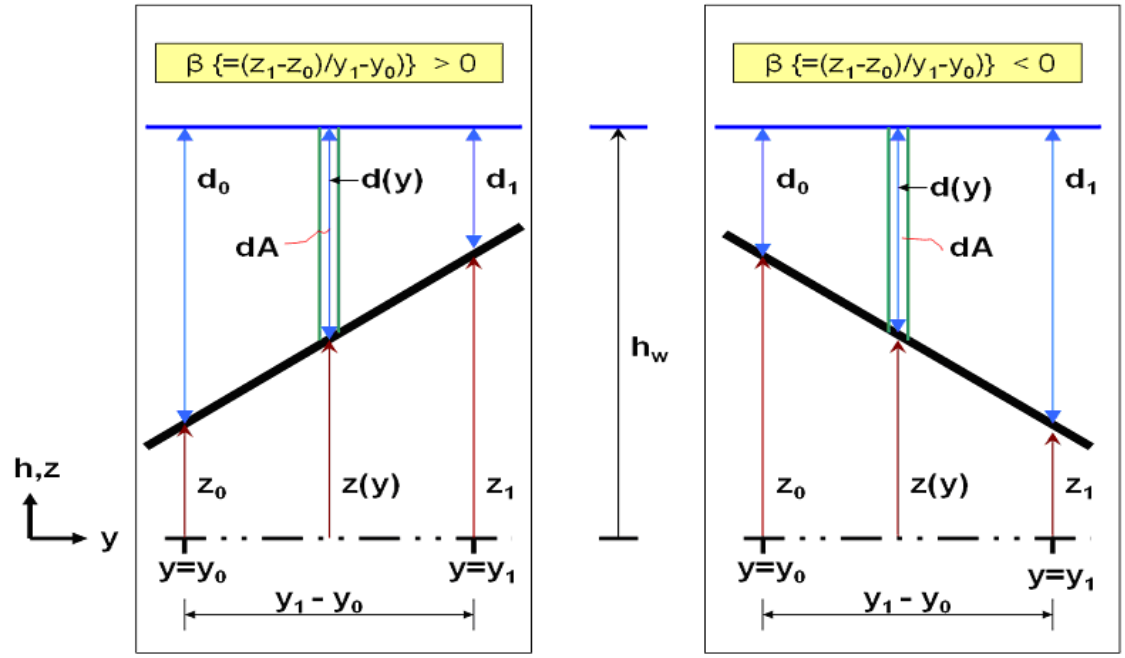


Figure 3.14: *Definition sketch of a vertical segment, considered in computing conveyance for a Y-Z profile and an Asymmetrical Trapezium cross-section*

The calculation of the conveyance integral for a vertical segment depends on the type of friction used. Below, the conveyance formulae are given for:

- ◇ Chézy roughness,
- ◇ Manning roughness,
- ◇ Strickler (k_n) roughness,
- ◇ Strickler (k_s) roughness,
- ◇ White-Colebrook (k_n) roughness, and
- ◇ Bos & Bijkerk (γ) roughness.

Chézy roughness conveyance for a vertical segment (see Figure 3.14):

$d = \max(d_0, d_1)$
if $d \leq 0$ then
    $K_i = 0$
else
    if $d < |z_1 - z_0|$ then
        if $|\beta| \leq 0.01$ then
            $K_i = k_1 = \frac{2C}{|\beta|(1+\beta^2)^{1/4}} \left(\frac{d}{2}\right)^{5/2}$
        elseif $|\beta| > 0.01$ then
            $K_i = k_2 = \frac{2C}{5|\beta|(1+\beta^2)^{1/4}} d^{5/2}$
    elseif $d \geq |z_1 - z_0|$ then
        if $|\beta| \leq 0.01$ then
            $K_i = k_3 = \frac{C}{(1+\beta^2)^{1/4}} \left(\frac{d_0+d_1}{2}\right)^{3/2} (y_1 - y_0)$
        elseif $|\beta| > 0.01$ then
            $K_i = k_4 = \frac{2C}{5|\beta|(1+\beta^2)^{1/4}} \left| d_0^{5/2} - d_1^{5/2} \right|$
        endif
    endif
endif

Manning roughness conveyance for a vertical segment (see Figure 3.14):

$d = \max(d_0, d_1)$
if $d \leq 0$ then
    $K_i = 0$
else
    if $d < |z_1 - z_0|$ then
        if $|\beta| \leq 0.01$ then
            $K_i = k_1 = \frac{2}{n|\beta|(1+\beta^2)^{1/4}} \left(\frac{d}{2}\right)^{8/3}$
        elseif $|\beta| > 0.01$ then
            $K_i = k_2 = \frac{3}{8n|\beta|(1+\beta^2)^{1/4}} d^{8/3}$
    elseif $d \geq |z_1 - z_0|$ then
        if $|\beta| \leq 0.01$ then
            $K_i = k_3 = \frac{1}{n(1+\beta^2)^{1/4}} \left(\frac{d_0+d_1}{2}\right)^{5/3} (y_1 - y_0)$
        elseif $|\beta| > 0.01$ then
            $K_i = k_4 = \frac{3}{8n|\beta|(1+\beta^2)^{1/4}} \left| d_0^{8/3} - d_1^{8/3} \right|$
        endif
    endif
endif

Strickler (k_n) roughness conveyance for a vertical segment (see Figure 3.14):

$d = \max(d_0, d_1)$
if $d \leq 0$ then
    $K_i = 0$
else
    if $d < |z_1 - z_0|$ then
        if $|\beta| \leq 0.01$ then

DRAFT

$K_i = k_1 = \frac{50 k_n^{-1/6}}{|\beta|(1+\beta^2)^{1/4}} \left(\frac{d}{2}\right)^{8/3}$
elseif $|\beta| > 0.01$ then
$K_i = k_2 = \frac{75 k_n^{-1/6}}{8|\beta|(1+\beta^2)^{1/4}} d^{8/3}$
elseif $d \geq |z_1 - z_0|$ then
if $|\beta| \leq 0.01$ then
$K_i = k_3 = \frac{25 k_n^{-1/6}}{(1+\beta^2)^{1/4}} \left(\frac{d_0+d_1}{2}\right)^{5/3} (y_1 - y_0)$
elseif $|\beta| > 0.01$ then
$K_i = k_4 = \frac{75 k_n^{-1/6}}{8|\beta|(1+\beta^2)^{1/4}} \left| d_0^{8/3} - d_1^{8/3} \right|$
endif
endif
endif

Strickler (k_s) roughness conveyance for a vertical segment (see Figure 3.14):

$d = \max(d_0, d_1)$
if $d \leq 0$ then
$K_i = 0$
else
if $d < |z_1 - z_0|$ then
if $|\beta| \leq 0.01$ then
$K_i = k_1 = \frac{2 k_s}{|\beta|(1+\beta^2)^{1/4}} \left(\frac{d}{2}\right)^{8/3}$
elseif $|\beta| > 0.01$ then
$K_i = k_2 = \frac{3 k_s}{8|\beta|(1+\beta^2)^{1/4}} d^{8/3}$
elseif $d \geq |z_1 - z_0|$ then
if $|\beta| \leq 0.01$ then
$K_i = k_3 = \frac{k_s}{(1+\beta^2)^{1/4}} \left(\frac{d_0+d_1}{2}\right)^{5/3} (y_1 - y_0)$
elseif $|\beta| > 0.01$ then
$K_i = k_4 = \frac{3 k_s}{8|\beta|(1+\beta^2)^{1/4}} \left| d_0^{8/3} - d_1^{8/3} \right|$
endif
endif
endif

White-Colebrook (k_n) roughness conveyance for a vertical segment (see Figure 3.14):

$d = \max(d_0, d_1)$
$c_1 = (1+\beta^2)^{1/4} \ln(10)$
$c_2 = \frac{k_n}{12}$
if $d \leq 0$ then
$K_i = 0$
else
if $d < |z_1 - z_0|$ then
if $|\beta| \leq 0.01$ then
if $\frac{6d}{k_n} \leq 1.0005$ then
$K_i = k_1 = \frac{36}{|\beta|(1+\beta^2)^{1/4}} 0.00022 \left(\frac{d}{2}\right)^{5/2}$
else
$K_i = k_1 = \frac{36}{|\beta|(1+\beta^2)^{1/4}} {}^{10}\log\left\{\frac{6d}{k_n}\right\} \left(\frac{d}{2}\right)^{5/2}$
endif

DRAFT

elseif $|\beta| > 0.01$ then
if $\frac{d}{c_2} \leq 1.495$ then
$K_i = k_2 = \frac{36}{5|\beta|c_1} d^{5/2} 0.00213$
else
$K_i = k_2 = \frac{36}{5|\beta|c_1} d^{5/2} \left(\ln \left\{ \frac{d}{c_2} \right\} - \frac{2}{5} \right)$
endif
elseif $d \geq |z_1 - z_0|$ then
if $|\beta| \leq 0.01$ then
if $\frac{6(d_0+d_1)}{k_n} \leq 1.0005$ then
$K_i = k_3 = \frac{18}{(1+\beta^2)^{1/4}} 0.00022 (y_1 - y_0) \left(\frac{d_0+d_1}{2} \right)^{3/2}$
else
$K_i = k_3 = \frac{18}{(1+\beta^2)^{1/4}} {}^{10}\log \left\{ \frac{6(d_0+d_1)}{k_n} \right\} (y_1 - y_0) \left(\frac{d_0+d_1}{2} \right)^{3/2}$
endif
elseif $|\beta| > 0.01$ then
if $\frac{d_0}{c_2} \leq 1.495$ and $\frac{d_1}{c_2} \leq 1.495$ then
$K_i = k_4 = \frac{36}{5|\beta|c_1} \left| d_0^{5/2} 0.00213 - d_1^{5/2} 0.00213 \right|$
elseif $\frac{d_0}{c_2} \leq 1.495$ and $\frac{d_1}{c_2} > 1.495$ then
$K_i = k_4 = \frac{36}{5|\beta|c_1} \left| d_0^{5/2} 0.00213 - d_1^{5/2} \left(\ln \left\{ \frac{d_1}{c_2} \right\} - \frac{2}{5} \right) \right|$
elseif $\frac{d_0}{c_2} > 1.495$ and $\frac{d_1}{c_2} \leq 1.495$ then
$K_i = k_4 = \frac{36}{5|\beta|c_1} \left| d_0^{5/2} \left(\ln \left\{ \frac{d_0}{c_2} \right\} - \frac{2}{5} \right) - d_1^{5/2} 0.00213 \right|$
else
$K_i = k_4 = \frac{36}{5|\beta|c_1} \left| d_0^{5/2} \left(\ln \left\{ \frac{d_0}{c_2} \right\} - \frac{2}{5} \right) - d_1^{5/2} \left(\ln \left\{ \frac{d_1}{c_2} \right\} - \frac{2}{5} \right) \right|$
endif
endif
endif
endif
endif

Bos & Bijkerk (γ) roughness conveyance for a vertical segment (see Figure 3.14):

$d = \max(d_0, d_1)$
if $d \leq 0$ then
$K_i == 0$
else
if $d < |z_1 - z_0|$ then
if $|\beta| \leq 0.01$ then
$K_i = k_1 = \frac{2\gamma}{|\beta|(1+\beta^2)^{1/4}} \left(\frac{d}{2} \right)^3$
elseif $|\beta| > 0.01$ then
$K_i = k_2 = \frac{\gamma}{3|\beta|(1+\beta^2)^{1/4}} d^3$
elseif $d \geq |z_1 - z_0|$ then
if $|\beta| \leq 0.01$ then
$K_i = k_3 = \frac{\gamma}{(1+\beta^2)^{1/4}} \left(\frac{d_0+d_1}{2} \right)^2 (y_1 - y_0)$
elseif $|\beta| > 0.01$ then
$K_i = k_4 = \frac{\gamma}{3|\beta|(1+\beta^2)^{1/4}} |d_0^3 - d_1^3|$
endif
endif

DRAFT

endif

3.4.5.2 Conveyance according to the lumped approach

The lumped conveyance for a given water depth h is computed as:

$$K(h) = A(h)C(h)\sqrt{R(h)} \tag{3.27}$$

where:

$K(h)$	Lumped conveyance for water depth h
$A(h)$	Cross-sectional area for water depth h
$C(h)$	Chézy friction value for water depth h
$P(h)$	Wetted Perimeter for water depth h
$R(h)$	Hydraulic radius for water depth h ($R(h) = A(h)/P(h)$)

Here, the water depth h follows from a water level ζ and a bed level z. More details follow in section 3.4.7.

If the water level in an open tabulated cross section rises above the highest level in a cross-sectional profile, a vertical wall is assumed, as a result:

- ◇ the flow width is set to the width at the highest level.
- ◇ the flow area is set to the total flow area of the cross section at the highest point plus the flow width at the highest level * water height above the highest point in the cross section definition.
- ◇ The wet perimeter is set to the wet perimeter at the highest point of the cross section plus 2 * the water height above the highest point in the cross section.

For a closed cross section the flow area and wet perimeter are equal to the corresponding highest level in the cross sectional profile.

3.4.6 1D friction definition

The previous section on conveyance uses friction coefficients for its calculations. The input of these friction values typically varies between the various locations in a 1D network. There are two places in the model input files where these values can be defined:

- ◇ Friction values (and types) directly in the cross section definition file.
- ◇ Friction values (and types) in friction files (separate of the cross sections), with the possibility to vary on the network branches.

These two types can also be combined in one model schematization. The following subsections contain further details.

The friction values are defined on certain locations on the network, but for conveyance calculations friction values are needed on other locations in the computational grid. The interpolation choices used for this are further explained in section 3.4.7.

3.4.6.1 Friction definition with cross section definitions

All available cross section definitions allow direct specification of the friction properties. This is done using the keywords `frictionType(s)` and `frictionValue(s)` in the cross section definition file section C.17.1.

DRAFT

Alternatively, a cross section definition may refer to separate friction definitions, using keyword `frictionId(s)`. In this case, each referenced `frictionId` should be present in one of the friction files, described in the following section.

Friction sections

Friction sections allow for different friction values in subsegments of a single cross section definition. They are defined using multiple `frictionIds` (or `frictionValues` and `frictionTypes`) for a single cross section. This is only possible for the following cross section definition types: tabulated river, xyz and yz. All other cross section definitions only offer the singular keywords `frictionId`, `frictionType` and `frictionValue`.

3.4.6.2 Friction definition in friction files

When friction values are not directly specified in the cross section definitions, they may alternatively be specified on the network branches or in named friction classes. This is done using MDU keyword `FrictFile` under `[geometry]`, and all definitions must then be placed in one or more 1D roughness definition INI (friction) files (section C.16).

Each friction file defines one or more `frictionId`s with a global friction value under `[Global]`. When defining more than one `frictionId` in a single file, only the use of `[Global]` is allowed, not `[Branch]`. An example use of this is to define multiple friction 'classes' for various materials, e.g., `pvc` or `concrete`. When multiple `frictionId`s are needed that have friction values on branches, each `frictionId` must be placed in its own friction file.

In addition to one `[Global]` value, different friction values may also be defined on branch locations under `[Branch]` blocks using keywords `branchId` and `chainage`. Spatially varying friction *within* a branch is defined using `numLocations` > 1, and the friction interpolation settings are listed in Table C.31.

Time-dependent friction

Also, time-dependent friction values on branches are possible by supplying a <*.bc> file with the time series for the friction coefficient. A branch with `functionType = timeSeries` can refer to exactly one `timeSeriesId`, which must be an existing `name` in that <*.bc> file. As a result the friction coefficient is constant along the branch and varying in time.

In section 3.4.5.1 is described how a conveyance table for YZ- and XYZ-cross section is computed. When the friction values are time-dependent, these tables need to be recalculated. The generation of a conveyance table is time consuming, therefore it is recommended to limit the number of updates.

In the MDU file the parameter `UpdateRoughnessInterval` under `[time]` can be specified (default is 1 day). The time dependent friction parameters are updated at the start of the computation and subsequently at `UpdateRoughnessInterval` intervals. This offers the user the possibility to limit the number of updates of the friction parameters and as a result also the number of conveyance tables that have to be recomputed is reduced. In case of a (X)YZ-cross section with one or more friction sections with time dependent friction values, two conveyance tables are computed, one for the start of the interval (with the corresponding friction parameters for this time instance) and one for the end of the interval (`UpdateRoughnessInterval` later). At intermediate times during the computation, a linear time interpolation of the conveyance is performed. For all the other types of cross sections the friction values are determined using a linear interpolation in time, using the friction values at the beginning of the update interval and at the end of the update interval, and the

DRAFT

conveyance is directly calculated using that interpolated friction value.

3.4.7 Interpolation of cross sections

Cross sections can be placed at arbitrary locations on the branches of a 1D model. As a result a cross section might not be available at the required location for the computational core. In those cases an interpolation is required. The interpolation method is explained, using the example network from Figure 3.15

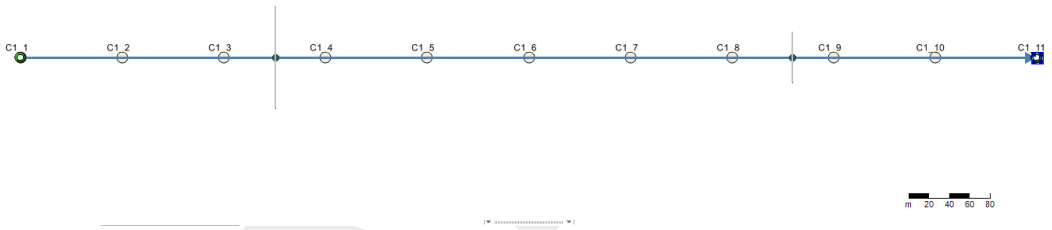


Figure 3.15: *Example of a 1D-model.*

This example contains 2 cross sections and eleven water level points.

The bedlevels are treated differently from the cross section dimensions. The interpolation of a circular cross section is different from the other cross section types. And also the conveyance is treated in a different way.

Remark:

- ◇ In D-Flow FM the user can specify interpolation over branches. The interpolation methods, described in this subsection, still holds for this case.

Interpolation of bed levels

The bed levels are interpolated to the water level points. In this example:

- ◇ C1_1 until C1_3 get the same bed level as the bed level of the first cross section.
- ◇ The bed levels for C1_4 until C1_8 will be the result of a linear interpolation between both cross sections.
- ◇ C1_9 until C1_11 get the same bed level as the bed level of the second cross section.

The absolute bed levels at cross section locations is determined by the absolute bed level of the cross section definition (in [m AD]) plus an optional shift at the cross section location (in [m]).

Interpolation of cross sectional flow data

The flow area and wet perimeter is required at the flow velocity points.

The computational core calculates a water depth for a velocity point. In the next step the flow area and the wet perimeter for the two adjacent cross sections are calculated, using the water depth from that velocity point. The two different values (both for flow area and wet perimeter) are then linearly interpolated back to the velocity point.

Interpolation of cross sectional total data

The volume of a 1D grid cell is calculated by adding up half the volume in each connected flow link: $0.5\Delta x A_T(h)$, and any additional storage volume in the node itself.

The water depth is calculated by subtraction of the bed level from the current water level. This water depth is used to calculate the total area and total width for the two adjacent cross sections. The two values for each item are then interpolated to the **velocity point**.

Interpolation of circular cross sections

Circular cross sections are handled differently from other cross sections. Only the diameter is interpolated from adjacent cross sections to the velocity point. The quantities total and flow area and wet perimeter are then calculated once using that interpolated diameter and the local water depth.

Calculation of the conveyance for YZ-type cross sections

In addition to the cross section data, also the Chézy value is required for the calculation of the conveyance. The user defined roughness values can be spatially varying, in the 1D roughness files (section C.16). In particular, these roughness locations need not be the same as the cross section locations. Therefore, the equivalent Chézy value is interpolated from the roughness locations to the location of the YZ cross sections. The conveyance is then calculated for the two adjacent cross sections of a velocity point. These conveyance values for the two cross sections are then interpolated to the velocity point.

Calculation of the conveyance for non-YZ-type cross sections

The conveyance at a velocity point for non-YZ-type cross sections is based on the interpolated flow area and wet perimeter, as described before. The Chézy value is directly interpolated from the roughness locations to the velocity point (i.e., not first to the cross section locations). Subsequently, these parameters are used to calculate the conveyance at the velocity point.

3.5 Equations of State for the density

3.5.1 Overview of density formulations in D-Flow FM

The density of water, ρ, is a function of salinity (s) and temperature (T), and optionally pressure (p). The keyword that selects the equation of state in the .mdu file is `Idensform`. The influence of salinity and/or temperature on water motion via baroclinic pressure gradients can be disabled by setting `Idensform = 0`.

Default: By default, D-Flow FM uses the UNESCO 1980 (EOS-80) equation of state with pressure set to zero (i.e., surface-referenced potential density).

DRAFT

Eckart Formulation

The Eckart formulation is an empirical equation of state derived from limited measurements (two salinities at five temperatures) from 1910 (Eckart, 1958). While the original includes pressure effects, they are negligible at low pressures and thus omitted here. The Eckart formulation is selected with `Idensform = 1`, and follows the implementation in Delft3D-FLOW.

Valid Range: $0 < T < 40$ °C, $0 < s < 40$ ppt

$$\rho = \frac{1\,000 P_0}{\lambda + \alpha_0 P_0}, \tag{3.28}$$

where:

$$\lambda = 1\,779.5 + 11.25T - 0.074\,5T^2 - (3.80 + 0.01T)s, \tag{3.29}$$
$$\alpha_0 = 0.698\,0, \tag{3.30}$$
$$P_0 = 5\,890 + 38T - 0.375T^2 + 3s. \tag{3.31}$$

Here, s is in [ppt] and T in [°C].

UNESCO 1980 Formulation (EOS-80)

The EOS-80 equation of state, based on 467 experimental data points, provides an internationally standardized expression for seawater density as a function of salinity and temperature at atmospheric pressure. It is selected via `Idensform = 2` and is recommended for most practical applications.

Valid Range: $0 < T < 40$ °C, $0.5 < s < 43$ ppt

$$\rho = \rho_0 + As + Bs^{3/2} + Cs^2 \tag{3.32}$$

where:

$$\rho_0 = 999.842\,594 + 6.793\,952 \times 10^{-2} T - 9.095\,290 \times 10^{-3} T^2 + 1.001\,685 \times 10^{-4} T^3 - 1.120\,083 \times 10^{-6} T^4 + 6.536\,332 \times 10^{-9} T^5 \tag{3.33}$$

$$A = 0.824\,493 - 4.089\,9 \times 10^{-3} T + 7.643\,8 \times 10^{-5} T^2 - 8.246\,7 \times 10^{-7} T^3 + 5.387\,5 \times 10^{-9} T^4 \tag{3.34}$$

$$B = -5.724\,66 \times 10^{-3} + 1.022\,7 \times 10^{-4} T - 1.654\,6 \times 10^{-6} T^2 \tag{3.35}$$

$$C = 4.831\,4 \times 10^{-4} \tag{3.36}$$

The standard error is approximately 3.6×10^{-3} kg/m^3 (Millero and Poisson, 1981). EOS-80 accurately reproduces the maximum density of freshwater at 4° C, which is essential for thermal stratification modeling in lakes. D-Flow FM applies EOS-80 under the assumption that pressure is zero, interpreting the output as surface-referenced potential density.

DRAFT

UNESCO 1983 Formulation (EOS-83)

The EOS-83 formulation extends EOS-80 by including pressure dependence and thermobaric effects. It is based on Millero and Poisson (1981) and adopted in UNESCO's 1983 technical addendum. It is selected via `Idensform = 3`.

Valid Range: $0 < T < 40$ °C, $0.5 < s < 43$ ppt, $0 < p < 1\,000$ dbar

At zero pressure, the density is given by:

$$\rho(S,T,0) = \rho_w(T) + A(T)S + B(T)\sqrt{S}S + CS^2 \tag{3.37}$$

where:

$$\begin{aligned}\rho_w(T) = (((((6.536\,332 \times 10^{-9}T - 1.120\,083 \times 10^{-6})T + 1.001\,685 \times 10^{-4})T \\ - 9.095\,290 \times 10^{-3})T + 6.793\,952 \times 10^{-2})T + 999.842\,594 \end{aligned} \tag{3.38}$$

$$\begin{aligned}A(T) = ((((5.3875 \times 10^{-9}T - 8.2467 \times 10^{-7})T + 7.6438 \times 10^{-5})T \\ - 4.0899 \times 10^{-3})T + 0.824493 \end{aligned} \tag{3.39}$$

$$B(T) = (-1.6546 \times 10^{-6}T + 1.0227 \times 10^{-4})T - 5.72466 \times 10^{-3} \tag{3.40}$$

$$C = 4.8314 \times 10^{-4} \tag{3.41}$$

The full pressure-corrected density is:

$$\rho(S,T,p) = \rho(S,T,0) + \Delta\rho(S,T,p) \tag{3.42}$$

where $\Delta\rho$ is computed using the pressure-dependent secant bulk modulus $K(S,T,p)$ and specific volume anomaly, following the algorithms in Millero and Poisson (1981).

D-Flow FM uses the UNESCO 1983 formulation to extend EOS-80 when pressure is non-zero. This maintains consistency with both modern oceanographic standards and legacy systems, and ensures physically realistic behavior under key conditions, such as the freshwater density maximum near 4° C.

3.5.2 Thermobaricity

Note: Thermobaricity in D-Flow FM is of TRL level 4-6 (prototype), with the definitions of the Technical Technology Readiness Levels (TRL) in Section 2.7.

Thermobaricity refers to the phenomenon where mixing two water parcels of equal density but differing temperature and pressure yields a parcel with a different density. While negligible in shallow waters—where D-Flow FM is primarily applied—thermobaric effects can become significant in deep lakes or oceanic regions, especially at depths of several hundred meters. To account for these effects, D-Flow FM allows thermobaricity to be enabled exclusively for the UNESCO 1983 formulation by setting the .mdu file keyword `thermobaricity = 1` in the input block `[physics]`. When enabled, the density calculation incorporates spatial and temporal variations in salinity, temperature and water pressure.

3.6 Shear-stresses at closed boundaries

A closed boundary is situated at the transition between land and water. At a closed boundary, two boundary conditions have to be prescribed. One boundary condition has to do with the flow normal to the boundary and the other one with the shear-stress along the boundary. A closed sidewalls is always considered as impermeable. For the shear stress along the

DRAFT

boundary, the possible conditions to be prescribed are free-slip (zero tangential shear-stress), partial-slip and full-slip.

For large-scale simulations, the influence of the shear-stresses along closed boundaries can be neglected. Free slip is then applied for all closed boundaries. For simulations of smallscale flow (e.g. laboratory scale), the influence of the side walls on the flow may no longer be neglected. This option has been implemented for closed boundaries and dry points *but not for thin dams*. The reason is that the shear stress at a thin dam is dependent on the side (double valued).

Along the side walls, the tangential shear stress is calculated based on a logarithmic law of the wal. The friction velocity u_* is determined by the logarithmic law for a rough wall, with side wall roughness y_0 and the velocity in the first grid point near the side wall. Let Δy_s be the grid size normal to the sidwall, then:

$$|\vec{u}_{\text{sidewall}}| = \frac{u_*}{\kappa} \ln\left(1 + \frac{\Delta y_s}{2y_0}\right) \tag{3.43}$$

The choice for the way tangial shear stress are computed can be inserted through the key `irov` in the MDU-file (under `[physics]`):

- ◇ the case `irov = 0` treats the side walls as free-slip walls (default),
- ◇ the case `irov = 1` treats the side walls as partial-slip walls,
- ◇ the case `irov = 2` treats the side walls as no-slip walls.

If the choice for partial-slip walls is made, then a specific wall roughness value should be prescribed in the MDU-file. For this, the keyword `wall_ks` should be used (under `[physics]` in the MDU-file). The computational engine interpretes this value as Nikuradse roughness k_s. The value for y_0 is computed as $y_0 = k_s/30$.

3.7 Secondary flow

The flow in a river bend is basically three-dimensional. The velocity has a component in the plane perpendicular to the river axis. This component is directed to the inner bend near the riverbed and directed to the outer bend near the water surface, see Figure 3.16.

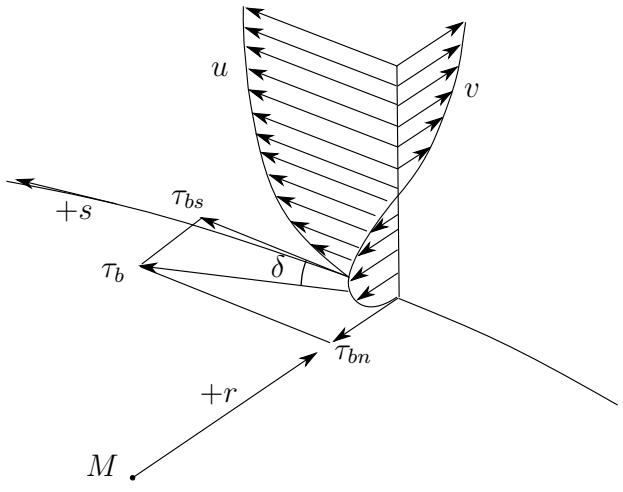


Figure 3.16: *Vertical profile secondary flow (v) in river bend and direction bed stress*

This so-called "secondary flow" (spiral motion) is of importance for the calculation of changes of the riverbed in morphological models and the dispersion of matter. It strongly varies over the vertical but its magnitude is small compared to the characteristic horizontal flow velocity. In a 3D model the secondary flow is resolved on the vertical grid, but in 2D depth-averaged model the secondary flow cannot be simulated in this way, because a vertical component doesn't exist. Therefore, it can be be simulated in a simplified way by using a so-called secondary flow model. In this section the secondary flow model in D-Flow FM is described.

3.7.1 Definition

The secondary flow will be defined here as the velocity component $v\left(\sigma\right)$ normal to the depth-averaged main flow. The spiral motion intensity of the secondary flow I is a measure for the magnitude of this velocity component along the vertical:

$$I = \int_0^1 \left|v\left(\sigma\right)\right| d\sigma \tag{3.44}$$

A vertical distribution for a river bend is given in Figure 3.16. The spiral motion intensity I leads to a deviation of the direction of the bed shear stress from the direction of the depth-averaged flow and thus affects the bedload transport direction. This effect can be taken into account in morphological simulations.

The component of the bed shear stress normal to the depth-averaged flow direction τ_{br} reads:

$$\tau_{br} = -2\rho\alpha^2 \left(1 - \frac{\alpha}{2}\right) \left|\boldsymbol{u}\right| I \tag{3.45}$$

where α is defined in Equation (3.55) and $\left|\boldsymbol{u}\right|$ is the magnitude of the depth-averaged velocity. To take into account the effect of the secondary flow on the depth-averaged flow, the depth-averaged shallow water equations have to be extended with:

- ◇ An additional advection-diffusion equation to account for the generation and adaptation of the spiral motion intensity.
- ◇ Additional terms in the momentum equations to account for the horizontal effective shear stresses originating from the secondary flow.

3.7.2 Momentum equations in horizontal direction

When applying a secondary flow model, this does not lead to any conceptual changes in the continuity equation. Therefore, equation 3.7 is still relevant. However, the momentum equations in x- and y-direction will be extended with and extra term, which is given by:

$$\frac{\partial U}{\partial t} + U\frac{\partial U}{\partial x} + V\frac{\partial U}{\partial y} - fV = -\frac{1}{\rho_0}\frac{\partial P}{\partial x} - \frac{gu\sqrt{u^2+v^2}}{C_{2D}^2 h} + F_x + F_{sx} + M_x \tag{3.46}$$

$$\frac{\partial V}{\partial t} + U\frac{\partial V}{\partial x} + V\frac{\partial V}{\partial y} + fU = -\frac{1}{\rho_0}\frac{\partial P}{\partial y} - \frac{gv\sqrt{u^2+v^2}}{C_{2D}^2 h} + F_y + F_{sy} + M_y \tag{3.47}$$

The fourth term in the right-hand side represents the effect of the secondary flow on the depth-averaged velocities (shear stresses by depth-averaging the non-linear acceleration terms).

DRAFT

3.7.3 Effect of secondary flow on depth-averaged momentum equations

To account for the effect of the secondary flow on the depth-averaged flow, the momentum equations have to be extended with additional shear stresses. To close the equations these stresses are coupled to parameters of the depth-averaged flow field. The main flow is assumed to have a logarithmic velocity profile and the secondary flow originates from a multiplication of a universal function with the spiral motion intensity, see Kalkwijk and Booij (1986). Depth averaging of the 3D equations leads to correction terms in the depth-averaged momentum equations for the effect of spiral motion:

$$F_{sx} = \frac{1}{h}\left(\frac{\partial h T_{xx}}{\partial x} + \frac{\partial h T_{xy}}{\partial y}\right) \tag{3.48}$$

$$F_{sy} = \frac{1}{h}\left(\frac{\partial h T_{xy}}{\partial x} + \frac{\partial h T_{yy}}{\partial y}\right) \tag{3.49}$$

with the shear-stresses, resulting from the secondary flow, modelled as:

$$T_{xx} = -2\beta uv \tag{3.50}$$

$$T_{xy} = T_{yx} = \beta(u^2 - v^2) \tag{3.51}$$

$$T_{yy} = 2\beta uv \tag{3.52}$$

and:

$$\beta = \beta^* \frac{h}{R_s^*} \tag{3.53}$$

$$\beta^* = \beta_c \left(5\alpha - 15.6\alpha^2 + 37.5\alpha^3\right) \tag{3.54}$$

$\beta_c \in [0, 1]$, correction coefficient specified by you,

$$\alpha = \frac{\sqrt{g}}{\kappa C_{2D}} < \frac{1}{2} \tag{3.55}$$

with R_s^* the effective radius of curvature of a 2D streamline to be derived from the intensity of the spiral motion and κ the Von Kármán constant. The spiral motion intensity is computed by Equation (3.56). The limitation on α, Equation (3.55), is set to ensure that the length scale L_a in Equation (3.63) is always positive. For $\beta_c = 0$, the depth-averaged flow is not influenced by the secondary flow.

Remark:

◇ Equation (3.55) effectively means a lower limit on C_{2D}.

DRAFT

3.7.4 The depth averaged transport equation for the spiral motion intensity

The variation of the spiral motion intensity I in space and time, is described by a depth-averaged advection-diffusion equation:

$$\frac{\partial hI}{\partial t} + \frac{\partial UhI}{\partial x} + \frac{\partial VhI}{\partial y} = h\frac{\partial}{\partial x}\left(D_H \frac{\partial I}{\partial x}\right) + h\frac{\partial}{\partial y}\left(D_H \frac{\partial I}{\partial y}\right) + hS \tag{3.56}$$

with:

$$S = -\frac{I - I_e}{T_a} \tag{3.57}$$

$$I_e = I_{be} - I_{ce} \tag{3.58}$$

$$I_{be} = \frac{h}{R_s}\left|\boldsymbol{U}\right| \tag{3.59}$$

$$I_{ce} = f\frac{h}{2} \tag{3.60}$$

$$\left|\boldsymbol{U}\right| = \sqrt{U^2 + V^2} \tag{3.61}$$

$$T_a = \frac{L_a}{\left|\boldsymbol{U}\right|} \tag{3.62}$$

$$L_a = \frac{(1 - 2\alpha)\, h}{2\kappa^2 \alpha} \tag{3.63}$$

and R_s the radius of curvature of the stream-line defined by:

$$\frac{U_s}{R_s} = -\frac{\partial U_r}{\partial s} \tag{3.64}$$

with U_s and U_r the components along and perpendicular to the streamline. The effective radius of curvature to be used for the evaluation of the coefficient β, Equation (3.54), reads:

$$R_s^* = \frac{h\left|\boldsymbol{U}\right|}{I} \tag{3.65}$$

To guarantee stability the effective radius of curvature is bounded by the following empirical relation:

$$R_s^* \geq 10h \tag{3.66}$$

The above formulas account for two sources of secondary flow:

- ◇ The centrifugal force in case of curved streamlines, I_{be}.
- ◇ The effect of the Coriolis force, I_{ce}.

3.8 Tide generating forces

Numerical models of tidal motion in coastal seas generally do not account for the direct local influence of the tide generating forces. The amount of water mass in these models is relatively small and the effect of these forces on the flow can be neglected. In that case, tidal motion can often be reproduced with sufficient accuracy just by prescribing the tidal forcing along open model boundaries.

In models covering larger seas or oceans, the contribution of the gravitational forces on the water motion increases considerably and can no longer be neglected. In a global model,

DRAFT

open boundaries are absent and the tidal motion can only be induced by including tide generating forces. Because tidal forces only play a significant role in the larger models, and because the exact position of each computational point on earth must be known, we have implemented these forces only in combination with spherical coordinates. By default, tide generating forces is switched on in spherical models. You can switch it off by setting in the mdu file: `Tidalforcing = 0`

The tide generating forces originate from the Newtonian gravitational forces of the terrestrial system (Sun, Moon and Earth) on the water mass. The equilibrium tide is the tide that would result from the gravitational forces if a solid earth would be completely covered by an ocean of about 21.5 km deep, such that the wave propagation speed would match the speed of the celestial forces over the ocean surface. The interacting frequencies that result can be grouped as diurnal, semi diurnal and long periodic. The total number of tide generating frequencies that is evaluated in the computation is a tradeoff between required accuracy and computational cost. Per default, we use a set of 60 frequencies.

A smaller set of 11 main frequencies as used in Delft3D-FLOW can also be selected by setting in the MDU file :

```
Doodsonstart =  57.555
Doodsonstop  = 275.555
Doodsoneps   =   0.030
```

The full set of 484 components can be selected by specifying:

```
Doodsonstart =  55.565
Doodsonstop  = 375.575
Doodsoneps   =   0.000
```

For the set of 60 components these numbers are:

```
Doodsonstart =  55.565
Doodsonstop  = 375.575
Doodsoneps   =   0.030
```

The earth itself is deformed by the celestial forces, this is called the solid earth tide. An estimate of this influence is based on the work of Love (1927), is included in the total tide generating potential.

Details on the implementation of tide generating forces can be found in the lecture notes of Schrama (2007).

In order to save computation cost, the tidal potential is not computed for each computational point, but on a grid with a resolution of 1 degree in both the South-North and West-East direction.

The tidal and surge motions of the seas and ocean also deform the earth, which is called tidal loading. Next to that, the water at some point is attracted by all moving water elsewhere on the globe. This is called self attraction. The combined influence of self attraction and loading is of

DRAFT

TRL level 4-6 (prototype), with the definitions of the Technical Technology Readiness Levels (TRL) in Section 2.7..

3.9 Wave forcing in D-Flow FM

In relatively shallow areas (coastal seas) wave action becomes important because of several processes:

- ◇ In the surf zone long-shore currents and a cross-shore set-up is generated due to variations in the wave-induced momentum flux (radiation stress). In case of an irregular surf zone, bathymetry strong circulations may be generated (rip currents).
- ◇ The bed shear stress is enhanced; this affects the stirring up of sediments and increases the bed friction.

These processes are accounted for in a wave-averaged manner. Some processes basically act at a specific location or interface, such as the enhanced bed shear-stress or wave breaking at the surface, while others have a (certain) distribution over the vertical, such as the energy dissipation due to bottom friction in the wave boundary layer.

The computation of waves and wave-induced effects is the domain of wave models. D-Waves supports currently one wave model: SWAN, a third generation wave model (Ris, 1997). SWAN computes the full (directional and frequency) spectrum on a rectilinear or curvilinear grid with many processes formulated in detail. For details and background you are referred to the D-Waves User Manual (Deltares, 2026f) and the references cited above.

For the sake of simplicity, we will describe in this section all wave-induced effects on a rectangular grid. In D-Flow FM, however, these terms have been implemented in an unstructured co-ordinate system. For a full wave-current interaction the currents from D-Flow FM are used in D-Waves (current refraction). The numerical grids used in the flow and wave computation often differ. SWAN works on a rectilinear or curvilinear grid, while D-Flow FM only works with unstructured grids. Unstructured SWAN is not yet supported. To use the required grid resolution in the area of interest you can use nested grids. The transformation of results between the nested grids and the grids used in the flow and in the wave computations is executed automatically and is fully transparent to you. The general procedure to derive the wave-induced forces is:

- ◇ Average the continuity equation and the momentum equations over the wave period.
- ◇ Express the residual terms that remain compared to the case without waves in terms of the wave properties, such as the wave energy, the phase and group velocity, and wave length or wave period.

Obviously, there are various averaging procedures. A straightforward approach is to define the mean motion by time averaging the equations over a wave period. The momentum equation in x-direction, averaged over the wave motion and expressed in Cartesian co-ordinates is given by:

$$\frac{\partial \bar{u}_j}{\partial t} + \bar{u}_i \frac{\partial \bar{u}_j}{\partial x_j} + ... + g\frac{\partial \bar{\zeta}}{\partial x_j} - \frac{1}{\rho}\frac{\partial \bar{\tau}_{ij}}{\partial x_i} = F_j, \tag{3.67}$$

where for i and j the summation rule applies, $i, j = \{1, 2, 3\}$, the wave averaged quantity is indicated by an overhead bar, $\bar{u}_j$ is the wave-mean velocity component, $\bar{\zeta}$ is the wave-mean free surface elevation, $\bar{\tau}_{ij}$ are the components of the wave-averaged normal stress tensor, and F_j is the wave-induced force that remains after averaging the momentum equation over the wave period. This wave induced force can generally be written as the gradient of the

radiation-stress tensor S:

$$F_i = -\frac{\partial S_{ij}}{\partial x_j}. \tag{3.68}$$

Using wave propagation models we can express S_{ij} in terms of the wave parameters, such as the wave energy, the phase and group velocity, the wave length and the wave period. The wave-induced force is computed as the gradient of the radiation stress terms.

The above given procedure can only be applied when the mean motion is uniform with depth. In most practical cases this does not apply. Then the only way to split the mean and oscillating motion is through the *Generalised Lagrangian Mean, GLM* method of Andrews and McIntyre (1978). As shown by Andrews and McIntyre (1978), Groeneweg and Klopman (1998) and Groeneweg (1999) the (depth averaged and 3D) flow equations written in a co-ordinate system moving with the Stokes drift are very similar to the ordinary Eulerian formulation. However, the wave-induced driving force due to averaging over the wave motion is more accurately expressed in wave properties. The relation between the GLM velocity and the Eulerian velocity is given by:

$$\boldsymbol{u}^L = \boldsymbol{u}^E + \boldsymbol{u}^S, \tag{3.69}$$

where $\boldsymbol{u}^L$ is the GLM-velocity vector, $\boldsymbol{u}^E$ is the ordinary Eulerian-velocity vector and $\boldsymbol{u}^S$ is the Stokes-drift vector.

The momentum equation in x-direction, averaged over the wave motion and expressed in GLM co-ordinates is given by:

$$\frac{\partial \bar{u}_j^L}{\partial t} + \bar{u}_i^L \frac{\partial \bar{u}_j^L}{\partial x_j} + ... + g\frac{\partial \bar{\zeta}}{\partial x_j} - \frac{1}{\rho}\frac{\partial \bar{\tau}_{ij}^L}{\partial x_i} = F_j^L, \tag{3.70}$$

where for i and j the summation rule applies, $i, j = \{1, 2, 3\}$, and the quantities have the same meaning as in Equation (3.67), but now in GLM co-ordinates. As shown by Groeneweg (1999) the right-hand side of Equation (3.70) contains a term related to a Stokes correction of the shear stresses. In the current implementation this term is neglected.

So in short:

- ◇ In D-Flow FM, the hydrodynamic equations are written and solved in GLM-formulation, including the wave-current interactions.
- ◇ The GLM velocities are written to the communication file and are used in all transport computations. Quantities moving with the flow are transported by the total, i.e. the GLM, velocity and not the Eulerian velocity.
- ◇ The Eulerian velocities are written to the D-Flow FM result files to be used in comparisons of computational results with measurements if `[output] EulerVelocities` in the MDU-file is set to 1.
- ◇ The only difference in the computational procedure as compared to the ordinary Eulerian formulation occurs at the boundaries. The velocity at the bottom is not the total velocity but the Eulerian velocity; so the bed-shear-stress is corrected for the Stokes drift. At lateral velocity boundaries, the total velocity must be prescribed; the implementation is such that the Eulerian velocity is prescribed in the boundary conditions (by you) and the Stokes drift is added by D-Flow FM. At water level boundaries, just the wave-mean water level must be prescribed.

DRAFT

In the following sections, we discuss the wave-current interaction terms and their possible profile in the vertical. Where applicable we distinguish between the depth-averaged and the 3D formulations.

Remarks:

- ◇ For brevity, we suppress the index L for GLM quantities. So, in this section a velocity without super-script refers to the GLM velocity, unless explicitly mentioned differently.
- ◇ A super-script S is used to indicate the Stokes drift.

3.9.1 Forcing by radiation stress gradients

The wave-induced force, i.e. the right-hand side of Equation (3.70), can be expressed in the wave parameters of the wave model that is being applied. For linear current refraction the expression can be derived analytically. To account for wave dissipation due to for instance bottom friction, wave breaking and whitecapping and wave growth due to wind one can rely on mild slope formulations with dissipation terms.

As shown by Dingemans *et al.* (1987), using the gradients of the radiation stresses in numerical models can result in spurious currents. Dingemans *et al.* (1987) showed that the divergence free part of the radiation stress is not capable of driving currents and can therefore be neglected if one is primarily interested in wave-driven currents. The remaining part of the radiation stress gradients is closely related to the wave energy dissipation, i.e. the right-hand side of Equation (3.70) can be written as:

$$F_i = \frac{Dk_i}{\omega}, \tag{3.71}$$

where D is the total energy dissipation due to waves, k_i is the wave number in i-direction and ω is the wave frequency; see Dingemans (1997) for many details and discussions on this subject.

For a depth averaged model the momentum equations in x- and y-direction, leaving out most of the terms, can be written as:

$$\frac{\partial U}{\partial t} + \ldots + \frac{gU\sqrt{U^2+V^2}}{C_{2D}^2\,(d+\zeta)} + \ldots = \ldots + F_x, \tag{3.72}$$

$$\frac{\partial V}{\partial t} + \ldots + \frac{gV\sqrt{U^2+V^2}}{C_{2D}^2\,(d+\zeta)} + \ldots = \ldots + F_y, \tag{3.73}$$

where F_x and F_y are the depth averaged wave-induced forcings and given by the gradients of the radiation stress tensor S, or following Dingemans *et al.* (1987) approximated by wave energy dissipation:

$$F_x = -\frac{\partial S_{xx}}{\partial x} - \frac{\partial S_{yx}}{\partial y} = D\frac{k_x}{\omega}, \tag{3.74}$$

$$F_y = -\frac{\partial S_{xy}}{\partial x} - \frac{\partial S_{yy}}{\partial y} = D\frac{k_y}{\omega}. \tag{3.75}$$

The dissipation rate D (a negative quantity) is computed by the wave model and read from the communication file. In SWAN, the dissipation rate may be computed from the bottom friction (orbital motion), depth-induced breaking and whitecapping.

DRAFT

You can choose to apply the radiation stress or the dissipation rate to determine the wave-induced forces.

3.9.2 Stokes drift and mass flux

In surface waves, fluid particles describe an orbital motion. The net horizontal displacement for a fluid particle is not zero. This wave induced drift velocity, the so-called Stokes-drift, is always in the direction of wave propagation. A particle at the top of the orbit beneath a wave crest moves slightly faster in the forward direction than it does in the backward direction beneath a wave trough. The mean drift velocity is a second order quantity in the wave height. The drift leads to additional fluxes in the wave averaged mass continuity equation.

The wave-induced mass fluxes M_x^S and M_y^S are found by integration of the components of the Stokes drift u^S and v^S over the wave-averaged total water depth:

$$M_x^S = \int_{-d}^{\overline{\zeta}} \rho_o u^S dz = \frac{E}{\omega} k_x \tag{3.76}$$

$$M_y^S = \int_{-d}^{\overline{\zeta}} \rho_0 v^S dz = \frac{E}{\omega} k_y \tag{3.77}$$

with E the wave energy defined as:

$$E = \frac{1}{8} \rho_0 g H_{rms}^2. \tag{3.78}$$

The mass fluxes M_x^S and M_y^S are computed by an interface program and are written to the communication file.

Remarks:

- ◇ The mass flux effect is only taken into account when D-Flow FM is coupled to D-Waves.
- ◇ The velocities written to the communication file for use in D-Waves, and D-Water Quality are based on the *total flux* velocities.
- ◇ The Eulerian velocities, which may be used in comparisons with measurements at a fixed location, are written to the hydrodynamic map and history files.

The depth-averaged Stokes drift is given by:

$$U^S = \frac{M_x^S}{\rho_0 (d + \zeta)}, \tag{3.79}$$

$$V^S = \frac{M_y^S}{\rho_0 (d + \zeta)}. \tag{3.80}$$

3.9.3 Enhancement of the bed shear-stress by waves

The boundary layers at the bed associated with the waves and the current interact non-linearly. This has the effect of enhancing both the mean and oscillatory bed shear-stresses. The bed shear-stress due to the combination of waves and current is enhanced beyond the value which would result from a linear addition of the bed shear-stress due to waves, $\boldsymbol{\tau}_w$, and the bed shear-stress due to current $\boldsymbol{\tau}_c$. For sediment transport modelling it is important to predict the maximum bed shear-stress, $\boldsymbol{\tau}_{\max}$, while the current velocity and the turbulent diffusion are determined by the combined wave-current bed shear-stress $\boldsymbol{\tau}_m$.

DRAFT

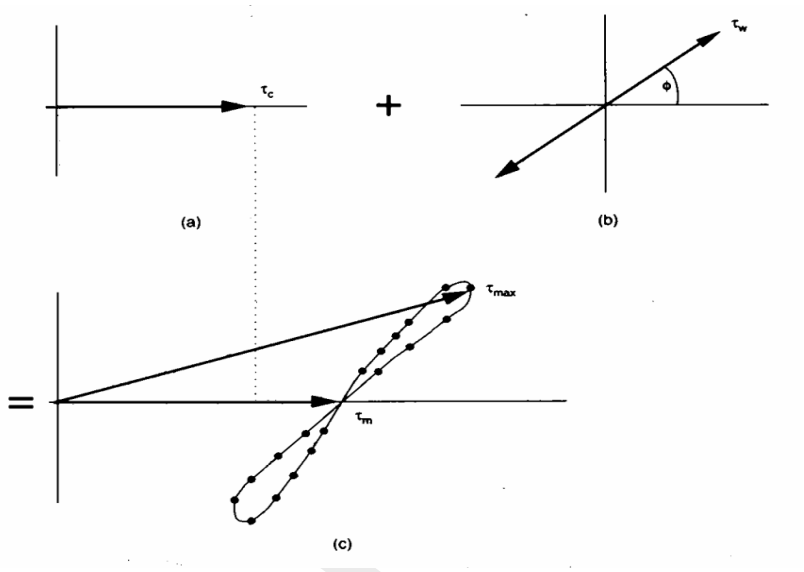


Figure 3.17: *Schematic view of non-linear interaction of wave and current bed shear-stresses (from Soulsby et al. (1993b, Figure 16, p. 89))*

Various, often very complex, methods exist to describe the bottom boundary layer under combined current and wave action and the resulting virtual roughness. Soulsby *et al.* (1993a) developed a parameterisation of these methods allowing a simple implementation and comparison of various wave-current interaction models: Fredsøe (1984); Myrhaug and Slaattelid (1990); Grant and Madsen (1979); Huynh-Thanh and Temperville (1991); Davies *et al.* (1988); Bijker (1967); Christoffersen and Jonsson (1985); O' Connor and Yoo (1988); Van Rijn *et al.* (2004). All these methods have all been implemented in D-Flow FM and can be applied in 2D and 3D modelling. However, as there are minor, but specific differences in determining certain quantities, such as determining the shear-stress at the bottom, we prefer to discuss the 2D and 3D implementation separately.

Following Soulsby *et al.* (1993b), Figure 3.17 gives a schematic overview of the bed shear-stresses for wave current interaction.

Soulsby *et al.* (1993b) fitted one standard formula to all of the models, each model having its own fitting coefficients. The parameterisation of Soulsby for the time-mean bed shear-stress is of the form:

$$|\boldsymbol{\tau}_m| = Y\left(|\boldsymbol{\tau}_c| + |\boldsymbol{\tau}_w|\right), \tag{3.81}$$

with

$$Y = X\left\{1 + bX^p(1-X)^q\right\}, \tag{3.82}$$

and for the maximum bed shear-stress:

$$|\boldsymbol{\tau}_{\max}| = Z\left(|\boldsymbol{\tau}_c| + |\boldsymbol{\tau}_w|\right), \tag{3.83}$$

with

$$Z = 1 + aX^m(1-X)^n. \tag{3.84}$$

and:

$$X = \frac{|\boldsymbol{\tau}_c|}{|\boldsymbol{\tau}_c| + |\boldsymbol{\tau}_w|}, \tag{3.85}$$

The value of the parameters a, b, p, q, m and n depends on the friction model which is parameterised, and:

$\lvert\boldsymbol{\tau}_c\rvert$	magnitude of the bed stress due to current alone
$\lvert\boldsymbol{\tau}_w\rvert$	magnitude of the bed stress for waves alone
$\lvert\boldsymbol{\tau}_m\rvert$	magnitude of the mean bed stress for combined waves and current
$\lvert\boldsymbol{\tau}_{\max}\rvert$	magnitude of the maximum bed stress for combined waves and current.

Remark:

◇ The stresses $\boldsymbol{\tau}_m$ and $\boldsymbol{\tau}_{\max}$ are assumed to have the same direction as $\boldsymbol{\tau}_c$.

Following Soulsby *et al.* (1993b) the expressions for the parameters χ $(= a, b, p, q, m, n)$ and $\mathcal{J}$ $(= I, J$; also depending on the friction model) have the form:

$$\chi = \left(\chi_1 + \chi_2 \left|\cos\phi\right|^{\mathcal{J}}\right) + \left(\chi_3 + \chi_4 \left|\cos\phi\right|^{\mathcal{J}}\right)\ {}^{10}\log\left(\frac{f_w}{C_{2D}}\right), \tag{3.86}$$

in which:

C_{2D}	drag coefficient due to current
f_w	wave friction factor
ϕ	the angle between the current direction and the direction of wave propagation.

As the radiation stress is always in the wave direction, we can derive ϕ from:

$$\left|\cos\phi\right| = \frac{\left|UF_x + VF_y\right|}{\left|\boldsymbol{U}\right|\left|\boldsymbol{F}\right|}. \tag{3.87}$$

Values of the parameters a, b, p, q and J in Equation (3.86) have been optimised by Soulsby *et al.* (1993b), see Table 3.2 and Figure 3.18.

The bed shear-stress due to flow alone may be computed using various types of formulations like Chézy, Manning or White-Colebrook, see Eqs. 3.13 to 3.16. The bed shear-stress due to current alone can be written in the form:

$$\boldsymbol{\tau}_c = \frac{g\rho_0 \boldsymbol{U}\left|\boldsymbol{U}\right|}{C_{2D}^2}. \tag{3.88}$$

The magnitude of the wave-averaged bed shear-stress due to waves alone is related to the wave orbital velocity near the bottom $\boldsymbol{u}_{orb}$ and the friction coefficient f_w:

$$\left|\boldsymbol{\tau}_w\right| = \frac{1}{2}\rho_0\, f_w\, u_{orb}^2. \tag{3.89}$$

The orbital velocity is computed from the linear wave theory and is given by:

$$u_{orb} = \frac{1}{4}\sqrt{\pi}\frac{H_{rms}\omega}{\sinh\left(kH\right)}, \tag{3.90}$$

where the root-mean-square wave height H_{rms} and the wave period $T(= 2\pi/\omega)$ are read from the communication file. The variation of the wave friction factor with relative orbital excursion at the bed under purely oscillatory flow is given by Swart (1974).

$$f_w = \begin{cases} 0.00251\exp\left[5.21\left(\frac{A}{k_s}\right)^{-0.19}\right], & \frac{A}{k_s} > \frac{\pi}{2}, \\ 0.3, & \frac{A}{k_s} \leq \frac{\pi}{2}, \end{cases} \tag{3.91}$$

DRAFT

Table 3.2: *Fitting coefficients for wave/current boundary layer model*

Model[1]	a_1	a_2	a_3	a_4	m_1	m_2	m_3	m_4	n_1	n_2	n_3	n_4	I
FR84	-0.06	1.70	-0.29	0.29	0.67	-0.29	0.09	0.42	0.75	-0.27	0.11	-0.02	0.80
MS90	-0.01	1.84	-0.58	-0.22	0.63	-0.09	0.23	-0.02	0.82	-0.30	0.19	-0.21	0.67
HT91	-0.07	1.87	-0.34	-0.12	0.72	-0.33	0.08	0.34	0.78	-0.23	0.12	-0.12	0.82
GM79	0.11	1.95	-0.49	-0.28	0.65	-0.22	0.15	0.06	0.71	-0.19	0.17	-0.15	0.67
DS88	0.05	1.62	-0.38	0.25	1.05	-0.75	-0.08	0.59	0.66	-0.25	0.19	-0.03	0.82
BK67	0.00	2.00	0.00	0.00	0.00	0.50	0.00	0.00	0.00	0.50	0.00	0.00	1.00
CJ85	-0.01	1.58	-0.52	0.09	0.65	-0.17	0.18	0.05	0.47	-0.03	0.59	-0.50	0.64
OY88	-0.45	2.24	0.16	-0.09	0.71	0.27	-0.15	0.03	1.19	-0.66	-0.13	0.12	0.77
	b_1	b_2	b_3	b_4	p_1	p_2	p_3	p_4	q_1	q_2	q_3	q_4	J
FR84	0.29	0.55	-0.10	-0.14	-0.77	0.10	0.27	0.14	0.91	0.25	0.50	0.45	3.00
MS90	0.65	0.29	-0.30	-0.21	-0.60	0.10	0.27	-0.06	1.19	-0.68	0.22	-0.21	0.50
HT91	0.27	0.51	-0.10	-0.24	-0.75	0.13	0.12	0.02	0.89	0.40	0.50	-0.28	2.70
GM79	0.73	0.40	-0.23	-0.24	-0.68	0.13	0.24	-0.07	1.04	-0.56	0.34	-0.27	0.50
DS88	0.22	0.73	-0.05	-0.35	-0.86	0.26	0.34	-0.07	-0.89	2.33	2.60	-2.50	2.70
BK67	0.32	0.55	0.00	0.00	-0.63	0.05	0.00	0.00	1.14	0.18	0.00	0.00	3.00
CJ85	0.47	0.29	-0.09	-0.12	-0.70	0.13	0.28	-0.04	1.65	-1.19	-0.42	0.49	0.60
OY88	-0.06	0.26	0.08	-0.03	-1.00	0.31	0.25	-0.26	0.38	1.19	0.25	-0.66	1.50
VR04	$Y = 0.0$ and $Z = 1.0$												

[1] FR84=Fredsøe (1984), MS90=Myrhaug and Slaattelid (1990),
HT91=Huynh-Thanh and Temperville (1991), GM79=Grant and Madsen (1979), DS88=Davies *et al.* (1988),
BK67=Bijker (1967), CJ85=Christoffersen and Jonsson (1985), OY88=O' Connor and Yoo (1988),
VR04=Van Rijn *et al.* (2004)

DRAFT

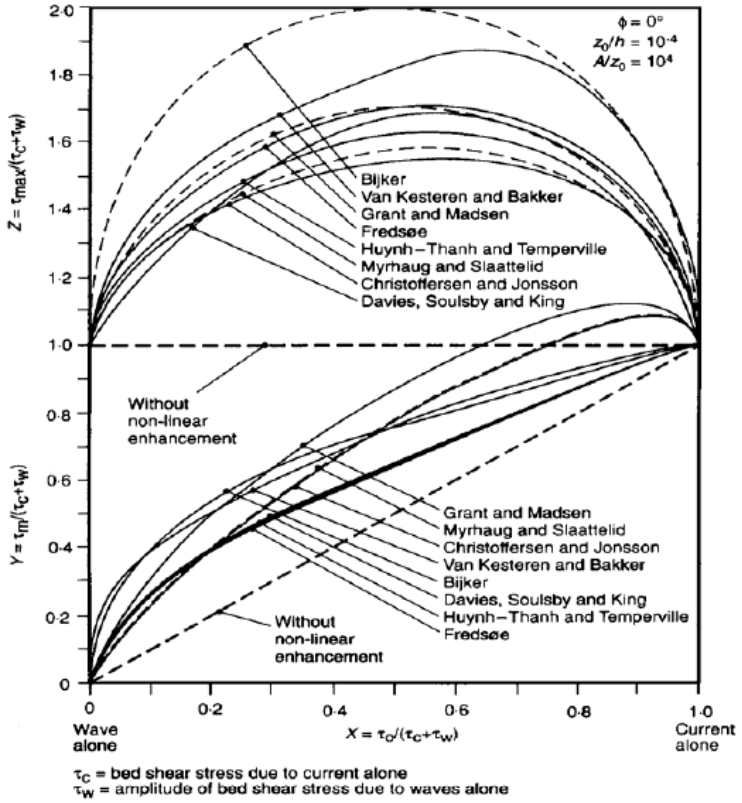


Figure 3.18: *Inter-comparison of eight models for prediction of mean and maximum bed shear-stress due to waves and currents (from Soulsby et al. (1993b, Figure 17, p. 90))*

with:

$$A = \frac{u_{orb}}{\omega}, \tag{3.92}$$

k_s is the Nikuradse roughness length-scale and ω is the wave angular frequency.

As the bed is in rest and the equations are formulated in GLM co-ordinates, we must correct the bed shear-stress used in the momentum equations for the Stokes drift. The total or effective bed shear stress is given by:

$$\boldsymbol{\tau}_b = \frac{|\boldsymbol{\tau}_m|}{|\boldsymbol{U}|}\left(\boldsymbol{U} - \boldsymbol{U}^S\right), \tag{3.93}$$

where the components of the depth-averaged Stokes drift $\boldsymbol{U}^S$ are given by Eqs. 3.79 and 3.80.

3.10 Assumptions underlying D-Flow FM

In D-Flow FM the 1D (transverse and depth-averaged), 2D (depth-averaged) or 3D (multi-layer) non-linear shallow water equations are derived from the three dimensional Navier-Stokes equations for incompressible free surface flow. The following assumptions and approximations are applied:

- ◇ The pressure is hydrostatic and the model is not capable of resolving the scales of short waves. Therefore, the basic equations are averaged in analogy with turbulence introducing so called radiation stresses.
- ◇ The effect of variable density is only taken into account in the pressure term (Boussinesq approximation).
- ◇ In a Cartesian frame of reference, the effect of the Earth curvature is not taken into account. Furthermore, the Coriolis parameter is assumed to be uniform unless specifically specified otherwise.
- ◇ In spherical co-ordinates the inertial frequency depends on the latitude.
- ◇ At the bed a slip boundary condition is assumed, a quadratic bed stress formulation is applied.
- ◇ The formulation for the enhanced bed shear-stress due to the combination of waves and currents is based on a 2D flow field, generated from the velocity near the bed using a logarithmic approximation.
- ◇ The equations of D-Flow FM are capable of resolving the turbulent scales (large eddy simulation), but usually the hydrodynamic grids are too coarse to resolve the fluctuations. Therefore, the basic equations are Reynolds-averaged introducing so-called Reynolds stresses. These stresses are related to the Reynolds-averaged flow quantities by a turbulence closure model.
- ◇ The eddy viscosity is an-isotropic. The horizontal eddy viscosity and diffusivity coefficients should combine both the effect of the 3D turbulent eddies and the horizontal motions that cannot be resolved by the horizontal grid. The horizontal eddy viscosity is generally much larger than the vertical eddy viscosity.
- ◇ For large-scale flow simulations, the tangential shear-stress at lateral closed boundaries can be neglected (free slip). In case of small-scale flow partial slip is applied along closed boundaries.
- ◇ For large-scale flow simulations, the horizontal viscosity terms are reduced to a bi-harmonic operator along co-ordinate lines. In case of small-scale flow the complete Reynold's stress tensor is computed. The shear-stress at the side walls is calculated using a logarithmic law of the wall.
- ◇ The drying and flooding procedure leads to a discontinuous movement of the closed boundaries at tidal flats.

DRAFT

- ◇ A continuity cell is set dry when all surrounding velocity points at the grid cell faces are dry or when the actual water depth at the cell centre is below zero (negative volume).
- ◇ The flux of matter through a closed wall and through the bed is zero.
- ◇ The effect of precipitation on the water temperature is accounted for.

DRAFT

4 Conceptual description of transport equation

4.1 Introduction

In D-Flow FM, transport is formulated as

$$\frac{\mathrm{d}}{\mathrm{d}t}\int\limits_{V(t)} c\,\mathrm{d}V + \int\limits_{\partial V(t)} c(\boldsymbol{u}-\boldsymbol{v})\cdot\boldsymbol{n}\,\mathrm{d}S = \int\limits_{\partial V(t)} (K\nabla c)\cdot\boldsymbol{n}\,\mathrm{d}S + \int\limits_{V(t)} s\,\mathrm{d}V, \tag{4.1}$$

where $V(t)$ is a three-dimensional control volume, c is a concentration, $\boldsymbol{u}$ the flow velocity field, $\boldsymbol{v}$ the velocity of the (vertically) moving control volume, K is a diagonal matrix $K = diag(\kappa, \kappa, \kappa_z)$ with diffusion coefficients and s a source term. For two-dimensional (depth-averaged) flow, we obtain

$$\frac{\partial hc}{\partial t} + \nabla\cdot(h\boldsymbol{u}c) = \nabla\cdot(h\kappa\nabla c) + hs, \tag{4.2}$$

where h is the water depth. In case of three-dimensional (layer-averaged) flow, with Δz a layer thickness from $z_0(x,y,t)$ to $z_1(x,y,t)$, we obtain

$$\begin{aligned}\frac{\partial \Delta z\, c}{\partial t} + \nabla\cdot(\Delta z\,\boldsymbol{u}c) + [\omega_{z_1}c]_{z=z_1} - [\omega_{z_0}c]_{z=z_0} = \nabla\cdot(\Delta z\,\kappa\nabla c) + \\ \left[\kappa_z\frac{\partial c}{\partial z} - \kappa\nabla z_1\cdot\nabla c\right]_{z=z_1} - \left[\kappa_z\frac{\partial c}{\partial z} - \kappa\nabla z_0\cdot\nabla c\right]_{z=z_0} + \Delta z\, s,\end{aligned} \tag{4.3}$$

where by $\boldsymbol{u}$ and ∇ still the horizontal components are meant, i.e. $\boldsymbol{u} = (u,v)^{\mathrm{T}}$ and $\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}\right)^{\mathrm{T}}$ and κ_z is the vertical diffusion coefficient. Furthermore ω_{z_0} and ω_{z_1} are the velocity component normal and relative to the moving $z = z_0$ and $z = z_1$ layer interfaces respectively. Note that taking $c = 1$ yields

$$\omega_{z_1} + \frac{\partial z_1}{\partial t} = \omega_{z_0} - \nabla\cdot(\Delta z\boldsymbol{u}) + \frac{\partial z_0}{\partial t}, \tag{4.4}$$

which, combined with a zero-flux condition at the bed, recursively defines ω_{z_0} and ω_{z_1} for all layers.

We apply Equation (4.2) and Equation (4.3) to transport of

- ◇ salinity,
- ◇ temperature,
- ◇ suspended sediment,
- ◇ tracers,
- ◇ other, intentionally not mentioned,

and ultimately to the water itself (the continuity equation) to obtain the relative layer interface velocities as expressed by Equation (4.4).

4.2 Transport processes

Looking at the equations that govern transport of matter, we can identify three processes, namely advection, diffusion and sources/sinks. The next sections will describe the relevant user settings.

DRAFT

4.2.1 Advection

Advection of matter is expressed by the term $\nabla \bullet (h\boldsymbol{u}c)$ in Equation (4.2) in the case of two-dimensional modelling.

Although not restricted to the advection of salinity only, the choices for *all* matter are set with keyword `limtypsa` in the mdu-file, for example

```
[numerics]
  limtypsa = 0        # first-order upwind, or
```

or

```
[numerics]
  limtypsa = 4        # Monotonized-central limiter
```

4.2.2 Diffusion

Diffusion of matter is expressed by the term $\nabla \bullet (h\kappa\nabla c)$ for two-dimensional modelling and a similar and additional terms in three dimensions, see Equation (4.3).

Clearly, we have horizontal diffusivity κ and vertical diffusivity κ_z. The horizontal diffusivity κ is a summation of

- the molecular diffusivity κ_l. We use the following values:

$$\kappa_l = \begin{cases} \frac{1}{700}\nu_l, & \text{salt,} \\ \frac{1}{6.7}\nu_l, & \text{temperature,} \\ 0\nu_l, & \text{sediment,} \\ 0\nu_l, & \text{tracers,} \end{cases} \tag{4.5}$$

  where $\nu_l = 10^{-6}\ m^2/s$ is the kinematic viscosity,
- a background diffusivity, user-specified as
  - spatially varying values by `horizontaleddydiffusivitycoefficient` in the ext-file, or
  - a uniform value `Dicouv` in the mdu-file,

  in that order of precedence, and
- a contribution from turbulent transport expressed as ν_t/σ_t, where ν_t is the eddy viscosity coefficient and σ_t is the turbulent Prandtl-Schmidt number for which the following values are used:

$$\sigma_t = \begin{cases} 0.7, & \text{salt,} \\ 0.7, & \text{temperature,} \\ 1.0, & \text{sediment,} \\ 1.0, & \text{tracers.} \end{cases} \tag{4.6}$$

Horizontal diffusion is turned off when `Dicouv=0`.

The user has to be aware of the following. The explicit nature of the horizontal diffusion terms in the time-integration method imposes a condition on the time step for numerical stability. Recall that we have a similar condition due to explicit horizontal advection. Although we

DRAFT

always decrease the time step to satisfy the advection-related time step criterion, we do not do so for diffusion. Instead, diffusion is limited such that our time step is restricted by advection only, or by a user-specified maximum time step if it is smaller. For further details, consult Deltares (2026a).

Be aware that the modelled *diffusion may be be smaller than anticipated. However, the actual* effective *diffusion encountered may be (much) larger than anticipated due to numerical diffusion of the advection scheme.*

Not considering suspended sediment, and similar to the horizontal diffusivity, the vertical diffusivity κ_z is a summation of

- ◇ the molecular diffusivity κ_l, see Equation (4.5),
- ◇ a background vertical diffusivity, user-specified as a uniform value `Dicoww` in the mdu-file, and
- ◇ a contribution from turbulent transport, similar to its horizontal counterpart, but with the horizontal viscosity coefficient now replaced by the vertical (eddy) viscosity coefficient.

Time integration is performed with a method that does not impose an additional constraint on the time step. Unlike horizontal diffusion which is limited to ensure numerical stability while maintaining the time-step size, vertical diffusion is *not* limited.

Vertical diffusion is turned off when, again not considering suspended sediment:

- ◇ if `Dicoww=0`,
- ◇ if the water depth is smaller than a threshold $10^{-2}\ m$.

For the vertical diffusivity of suspended sediment, see Deltares (2026d).

4.2.2.1 Time integration of horizontal diffusion

The advection and diffusion terms in the transport equation are integrated explicitly in time. However, for 2D models with large horizontal diffusion coefficients this will lead to a large reduction of the time step. Therefore, an implicit time integration scheme has been implemented for the horizontal diffusion term as well.To that purpose `TransportAutoTimestepdiff` has been introduced. Via `TransportAutoTimestepdiff = 3` the implicit time integration scheme is activated. The explicit time integration can be switched on via keyword `TransportAutoTimestepdiff = 0`. In this case the horizontal diffusion coefficient is probably reduced in order to satify the time step condition for stability. If so, then the following message is written in the diagnostic file:

```
** INFO   : Horizontal transport flux was limited for XX links.
```

with XX the number of links in which the horizontal diffusion coefficient has been reduced. Per time step, a warning message displays the number of flow links in which the horizontal diffusivity coefficient has been reduced. In order to prevent the diagnostic file becoming very large, the warning is shown a maximum of ten times. The following message is written to the diagnostic file at the end of the simulation:

```
** INFO   : Viscosity coefficient/Horizontal transport flux were limited on some links in the course of computation.
```

Another approach is to apply `TransportAutoTimestepdiff = 1`. Then, the horizontal diffusion coefficient isn't reduced, but the time step is probably reduced in order to satify the stability condition. If so, then the simulation will require more computation time. Depending on the application the user should choose the most suitable for for keyword `TransportAutoTimestepdiff`.

The options described above represent so-called prognostic approaches, in which the transport quantities are updated at each time step. A different approach is so-called diagnostic modelling of transport processes. Then, the transport quantities aren't updated during the simulation. In this way, the initial conditions are being used and the transport concentrates are frozen during the simulation. In case of horizontal density gradients the pressure term is relevant for the computation of water levels. Via the diagnostic approach the density gradient is taken into account in a simplified way. This avoids a possibly time step reduction for the transport equation in case of large horizontal diffusion coefficients. The latter are often required in depth-averaged simulations for modelling of salinity intrusion of estuaries. We remark that in 3D this problem does not occur, because a turbulence closure model is applied and the horizontal diffusion coefficients are small. In summary, a prognostic modelling of transport processes is to be preferred, but in some applications, in particular in depth-averaged models with large horizontal diffusion coefficients, diagnostic modelling can be a work-around. The latter can be activitated via keyword `DiagnosticTransport=1`.

4.2.3 Sources and sinks

Sources and sinks may be provided by an entry in the new ext-file as follows:

```
[SourceSink]
id             = SourceSink_1
locationFile   = sourcesink01.pli
discharge      = sourcesink_flows.bc
salinityDelta = sourcesink_changes.bc
...
```

Warning:

- ◇ Deprecated: Source-sinks could also be specified in the old <*.ext> file. This will become obsolete in an upcoming release and the example below is only kept for migration purposes.

Sources and sinks may be provided by an entry in the old ext-file as follows:

```
quantity=discharge_salinity_temperature_sorsin
```

This simultaneously prescribes sources and sinks of

- ◇ water volume itself (i.e. discharge),
- ◇ salinity, and
- ◇ temperature, and.
- ◇ any other constituents that are transported.

See section 6.4.10 for more details.

DRAFT

4.3 Boundary and initial conditions for transport equation

The equations that govern transport of matter are complemented with boundary and initial conditions. We make the distinction between "horizontal" boundaries, that are either "open" or "closed", and vertical boundaries, only relevant for three-dimensional modelling.

4.3.1 Open boundary conditions

At "horizontal" open boundaries the following boundary conditions are applied, with or without diffusion on the open boundary:

- salt, temperature and tracers:
  - inflow: user-specified Dirichlet condition,
  - outflow: homogeneous Neumann condition,
- suspended sediment: see Deltares (2026d).

Horizontal diffusive transport through open boundaries is switched on by default and can be switched off. The diffusive transport can be enabled or disabled by specifying the keyword `Diffusiononbnd` in the mdu-file, and is valid for all open boundary types. Enable diffusive transport on open boundaries (default)

```
[Numerics]
Diffusiononbnd=1
```

Disable diffusive transport on open boundaries

```
[Numerics]
Diffusiononbnd=0
```

The user-specified Dirichlet conditions are supplied in the usual manner through the ext-file, for salt

```
quantity=salinitybnd
```

and for temperature

```
quantity=temperaturebnd
```

Boundaries conditions for multiple tracers may be defined by appending their name to the `tracerbnd` keyword, for example

```
quantity=tracerbndMY_FIRST_TRACER
```

and

DRAFT

```
quantity=tracerbndMY_SECOND_TRACER
```

4.3.2 Closed boundary conditions

At "horizontal" closed boundaries zero-flux conditions are applied.

4.3.3 Thatcher-Harleman boundary conditions

Consider (a part of) an open boundary where the flow reverts from outflowing to inflowing. According to section 4.3.1, at that very moment a Dirichlet condition becomes effective and a user-specified boundary value is prescribed. This value does in general not reflect the true condition at the boundary and causes a discontinuous temporal behaviour. The so-called Thatcher-Harleman boundary condition is intended to regularize this behaviour. The actual value applied at the boundary is smoothly transformed from the last value under outflow conditions to the user-specified value under inflow conditions within a user-specified *return time*, which has to be specified in seconds. This return time is prescribed with the keyword `return_time` in the “new style” external forcings file for boundary condition, for example

```
[boundary]
quantity     = salinitybnd
locationfile = tfl_01.pli
forcingfile  = tfl.bc
return_time  = 250
```

See section C.6 for more details on this format.

4.3.4 Initial conditions

We will only consider intitial conditions for salinity, temperature and tracers. For initial sediment concentrations, see Deltares (2026d).

Initial conditions are specified in three possible ways, namely

- ◇ a horizontally spatially varying field in the usual way through the initial field file,
- ◇ a vertical profile in three dimensions, horizontally uniformly distributed, for salinity and temperature only in the initial field file, and
- ◇ uniform values for salinity and temperature in the mdu-file.

In the initial field file we have

```
[Initial]
quantity     = initialsalinity
```

for the intial salinity, and

```
[Initial]
quantity     = initialsalinitytop
```

DRAFT

for the initial salinity in the top layer in case of three-dimensional modelling. When specified, the initial salinity field is linearly distributed from the "initialsalinity" in the lowest layer to the "initialsalinitytop" in the top layer. When *not* specified, the initial salinity field is vertically uniformly distributed.

The initial temperature field is prescribed with

```
[Initial]
quantity     = initialtemperature
```

which in three dimensions is vertically uniformly distributed.

A horizontally uniformly distributed vertical profile of salinity and temperature can be prescribed with `initialverticalsalinityprofile` and `initialverticaltemperatureprofile` respectively, for example for salinity

```
[Initial]
quantity           = initialverticalsalinityprofile
dataFile           = inisal.pli
dataFileType       = polygon
interpolationMethod = constant
```

Warning:

- ◇ Deprecated: The old external forcing file is being deprecated. The example below is kept only for migration purposes:

```
QUANTITY=initialverticalsalinityprofile
FILENAME=inisal.pli
FILETYPE=10
METHOD=4
OPERAND=O
```

The polyline file contains (z, salinity) value pairs, where z is the vertical coordinate in meters. For example for linearly varying salinity from 30 to 20 ppt from -10 to 0 m:

```
L1
2 2
-10 30
  0 20
```

The initial field of an arbitrary number of tracers are prescribed in the same way, except for the user-specified tracername that is added to the keyword `initialtracer`, similar to the tracer boundary conditions, for example

```
[Initial]
quantity=initialtracerMY_FIRST_TRACER
```

and

```
[Initial]
quantity=initialtracerMY_SECOND_TRACER
```

Default values for salinity and temperature are defined in the mdu-file with `Initialsalinity` and `Initialtemperature` respectively.

DRAFT

5 Getting started with Graphical User Interface

Note: Some of the figures in this chapter are currently based on the 2D3D GUI and not yet on the 1D2D GUI. Since the D-Flow FM 1D2D and 2D3D GUIs are different, the figures may differ from your GUI for 1D2D modelling.

5.1 Introduction

This chapter gives an overview of the basic features of the D-Flow FM User Interface and will guide you through the main steps to set up a D-Flow FM model. For a more detailed description of the GUI features you are referred to chapter 6. For technical documentation you are referred to Deltares (2026a).

5.2 Overview of D-Flow FM GUI

When you start the application for the first time the lay-out might look like Figure 5.1. The basic lay-out consists of the following items:

- *Ribbon* - top
- **Project** window - up left
- The central window, containing the *Start Page*
- **Messages** and **Time Navigator** window - down centre
- **Toolbox**, **Chart**, **Region**, **Map** and **Operations** window - to the right
- **Properties** window - down left

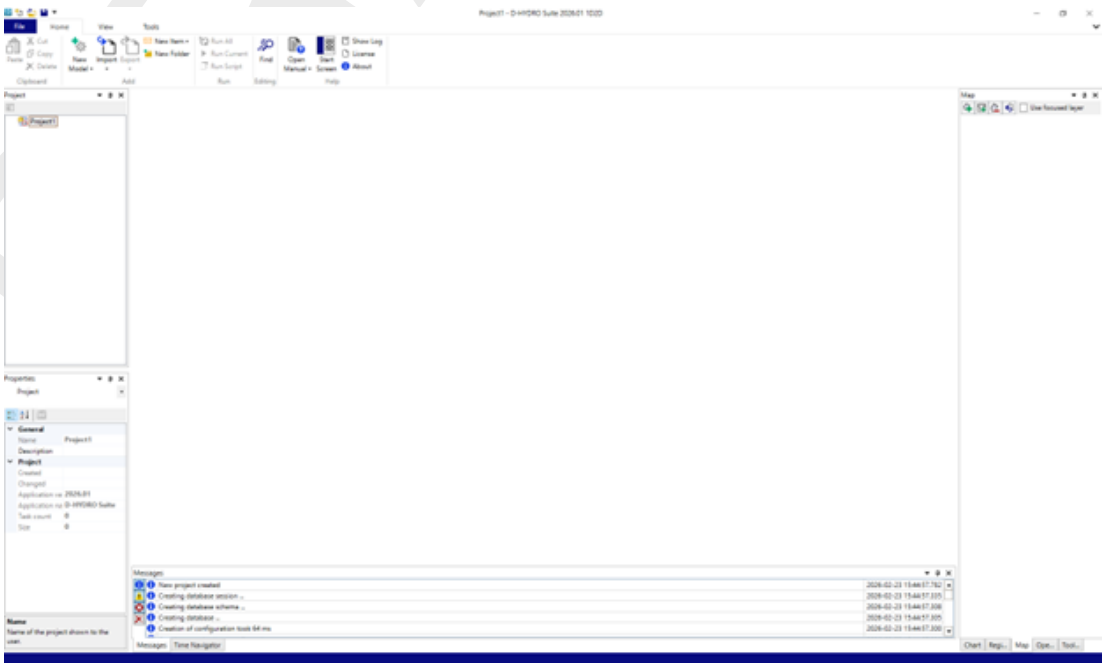


Figure 5.1: *Start-up lay-out D-Flow FM User Interface*

All the windows can be customized/hidden according to your own preferences.

These settings will be automatically saved for the next time you start the application.

The most important windows for the D-Flow FM plugin are the **Project**, **Map**, **Messages**, **Time Navigator** and **Properties** windows. The central window displays the **Start Page**. Here

the *Map view* or specific editors will be displayed.

The contents of these windows are briefly discussed in the subsections below.

5.2.1 Project window

After adding or importing a D-Flow FM model (see section 5.5.1 and section 5.5.2), the *Project* window will be extended with D-Flow FM specific features (see Figure 5.2). The *Project* window provides you with the basic steps to set up a D-Flow FM model.

The *Project* window consists of the following features:

General	general model information such as depth layer specification, model coordinate system and angle of latitude and longitude
Area	geographical (GIS based) features, such as observation points, structures, dry points and land boundaries
Grid	computational grid
Bed Level	model bed level
Time Frame	model time frame and time step
Processes	active physical processes in the model such as salinity, temperature, wind and tide generating forces
Initial Conditions	initial conditions for water levels and other physical processes
Boundary Conditions	model boundaries and boundary condition specification
Physical Parameters	physical settings for processes such as roughness, viscosity, wind and temperature
Sources and Sinks	location and time series specification for point sources and sinks
Numerical Parameters	numerical simulation settings
Output Parameters	output specification
Output	output after running the simulation

Upon clicking the items in the *Project* window the corresponding tab (in case of non-geographic model settings), attribute table (in case of geographic model settings) or editor view (in case of advanced editing options) will open in the central window. Using the right mouse button gives options such as importing/exporting model data.

DRAFT

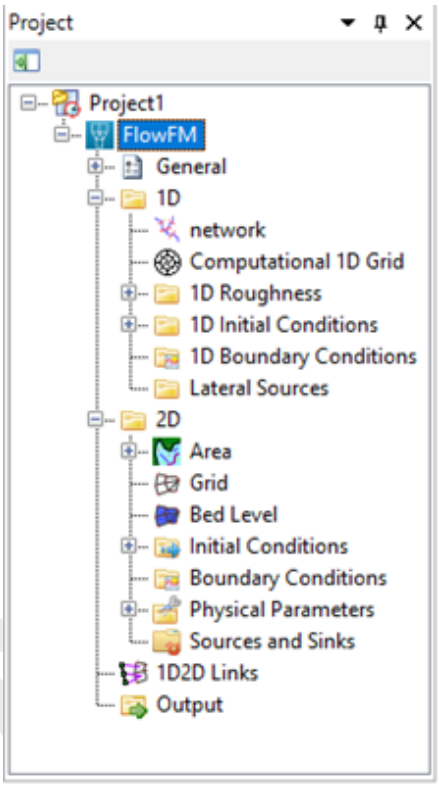


***Figure 5.2: Project** window of D-Flow FM plugin*

5.2.2 The central window

The central window shows the contents of the map or the editor you are working with. Multiple maps and editors may be docked at the central window location; each accessible as a tabs along the top (see Figure 5.3). The map is used to edit geographic model data, the editor for the overall model settings. Moreover, the contents of the central window can also be a specific editor such as the time point editor or the boundary condition editor. Each of these editors will open as a separate tab.

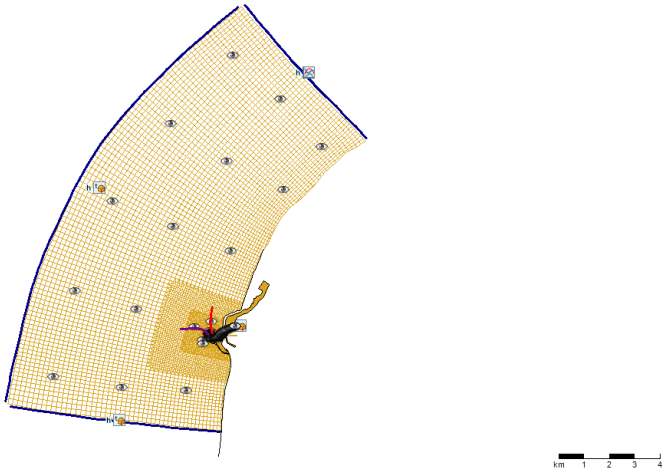


***Figure 5.3:** The central window with a map and contents of the D-Flow FM model*

5.2.3 Settings window

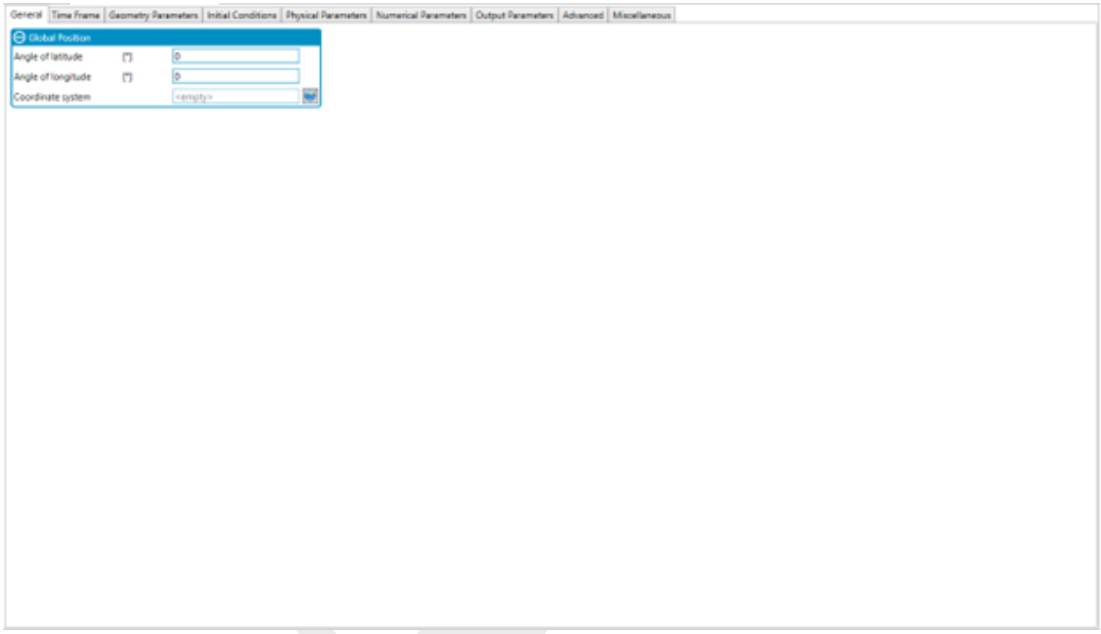


Figure 5.4: *The model* ***Settings*** *window with General settings tab enabled*

5.2.4 Map window

The **Map** window allows the user to control the visibility of the contents of the central map using checkboxes. Furthermore, the user can add (wms) layers, such as satellite imagery or open street maps (see Figure 5.5).

Note: Please note that the map usually has a different coordinate system than the model. In rendering the model attributes they are transformed to the map coordinate system (for visual inspection on the map), but the model will be saved in the model coordinate system.

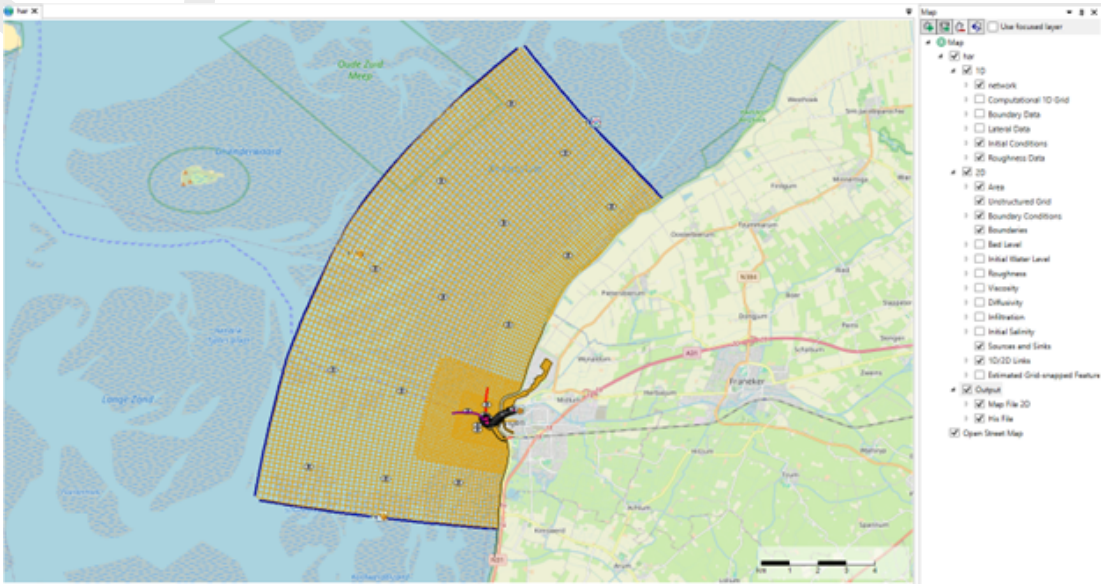


Figure 5.5: ***Map*** *window controlling map contents*

5.2.5 Messages window

The **Message** window (Figure 5.6) provides a log of information on the recent activities in the User Interface. It also provides warning and error messages.

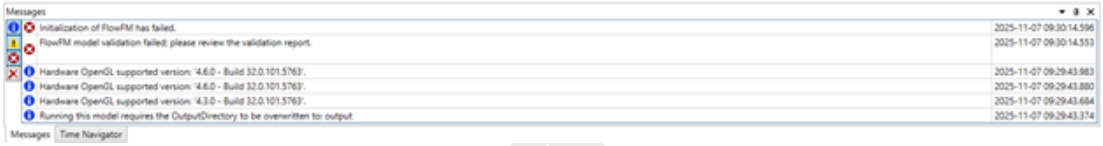


Figure 5.6: *Log of messages, warnings and errors in* ***Messages*** *window*

5.2.6 Time Navigator window

The **Time Navigator** (Figure 5.7) can be used to step through time dependent model output and other time dependent geographic features on the map.

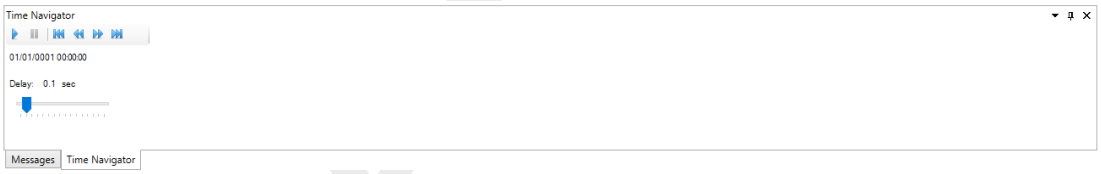


Figure 5.7: Time Navigator *window in D-Flow FM User Interface*

5.3 Dockable views

The User Interface offers lots of freedom to customize dockable views, which are discussed in this section.

5.3.1 Docking tabs separately

Within the User Interface the user can dock the separate windows according to personal preferences. These preferences are then saved for future use of the framework. An example of such preferences is presented in Figure 5.8, where windows have been docked on two screens.

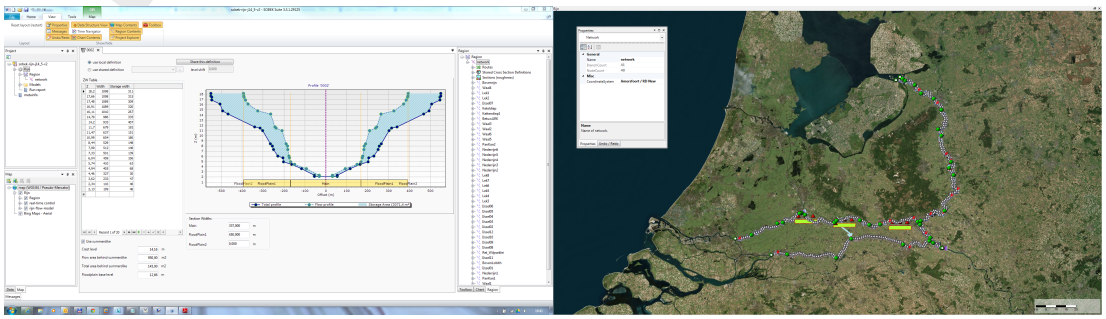


Figure 5.8: *Docking windows on two screens within the User Interface.*

DRAFT

5.3.2 Multiple tabs

In case two windows are docked in one view, the underlying window (tab) can be brought to the front by simply selecting the tab, as is shown here.

Figure 5.9: *Bringing the* ***Time Navigator*** *window to the front*

By dragging dockable windows with the left mouse button and dropping the window left, right, above or below another one the graphical user interface can be customized according to personal preferences. Here an example of the **Time Navigator** window being docked to the right of the **Messages** window.

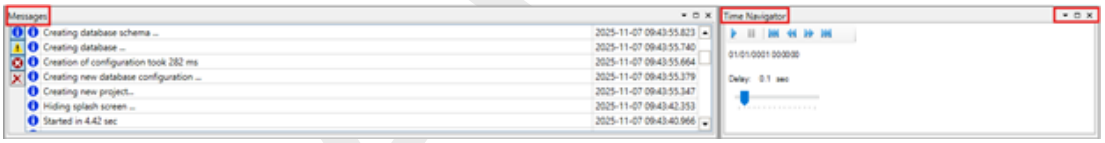


Figure 5.10: *Docking the* ***Time Navigator*** *window.*

Additional features are the possibility to remove or (auto) hide the window (top right in Figure 5.10). In case of removal, the window can be retrieved by two left mouse-clicks on *Time Navigator* in the *View* ribbon. Hiding the **Properties** window results in:

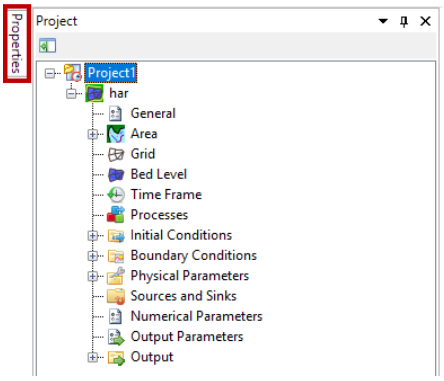


Figure 5.11: *Auto hide the* ***Properties*** *window*

5.4 Ribbons and toolbars

The user can access the toolbars arranged in *ribbons*. Model plug-ins can have their own model specific *ribbon*. The *ribbon* may be auto collapsed by activating the *Collapse the Ribbon* button when right-mouse-clicking on the *ribbon*.

5.4.1 Ribbons (shortcut keys)

The User Interface makes use of ribbons, just like Microsoft Office. You can use these ribbons for most of the operations. With the ribbons comes shortcut key functionality, providing shortcuts to perform operations. If you press `Alt`, you will see the letters and numbers to access

DRAFT

the ribbons and the ribbon contents (i.e. operations). For example, `Alt + H` will lead you to the *Home*-ribbon (Figure 5.12).

Note: Implementation of the shortcut key functionality is still work in progress.

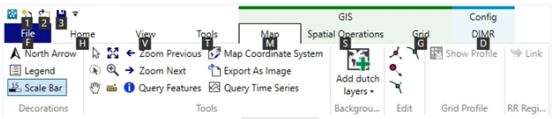


Figure 5.12: *Perform operations using the shortcut keys*

5.4.2 File

The left-most *ribbon* is the *File* ribbon. It has menu-items comparable to most Microsoft applications. Furthermore, it offers users save and open functionality, as well as the *Info* and *Options* dialogs, as shown in Figure 5.13 and Figure 5.14.

Figure 5.13: *The* File *ribbon*

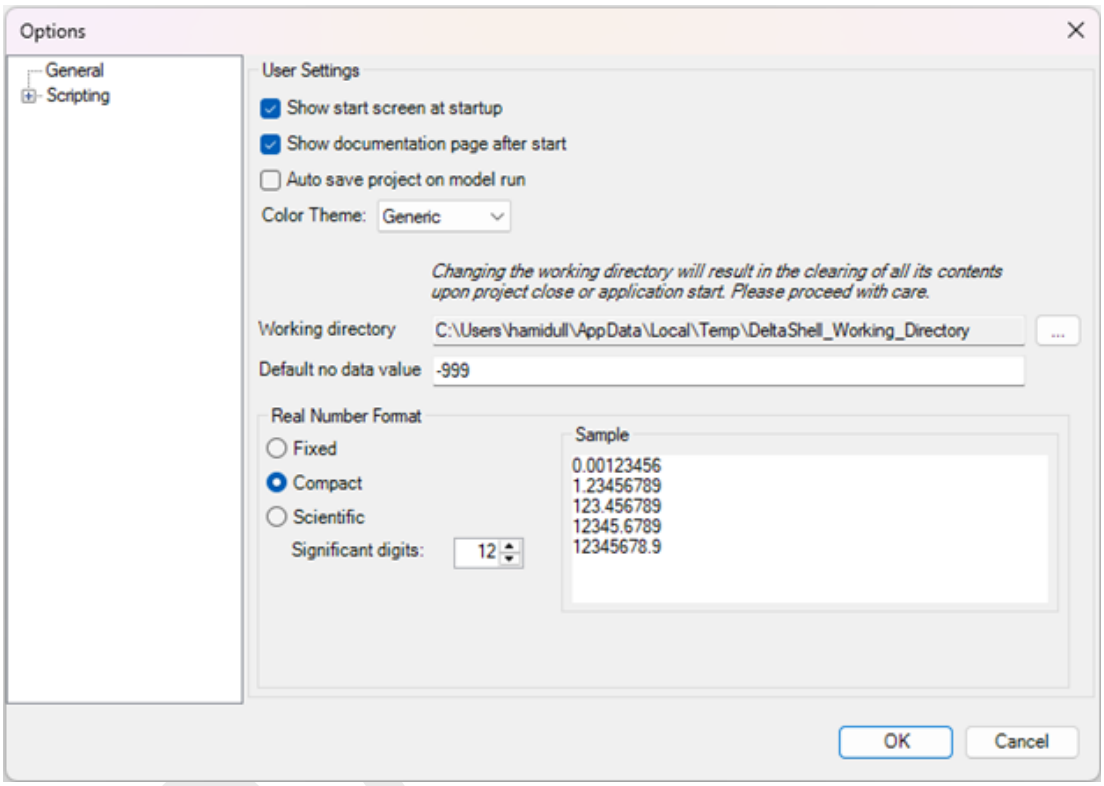


Figure 5.14: *The User Interface options dialog.*

5.4.3 Home

The second *ribbon* is the *Home* ribbon (Figure 5.15). It harbours some general features for clipboard actions, addition of items, running models, finding items within projects or views, and help functionality.

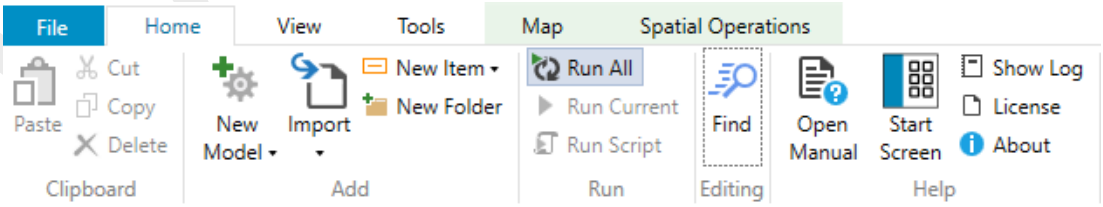


Figure 5.15: *The* Home *ribbon.*

5.4.4 View

The third *ribbon* is the *View* ribbon (Figure 5.16). Here, the user can show or hide windows.

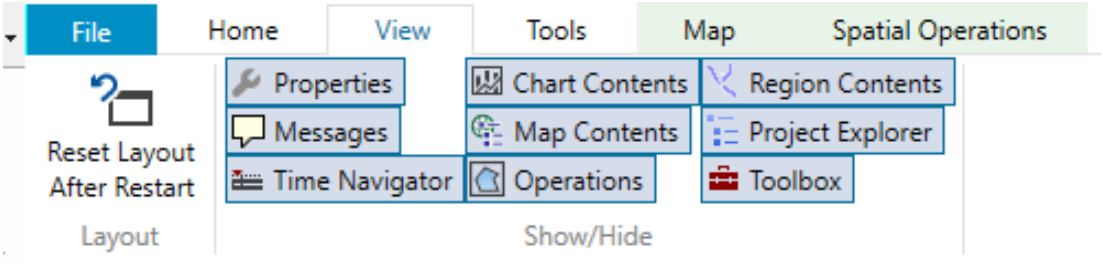


Figure 5.16: *The* View *ribbon.*

5.4.5 Map

The fourth *ribbon* is the *Map* ribbon (Figure 5.17).

Figure 5.17: *The* Map *ribbon.*

This ribbon will be used heavily, as it harbours all *Geospatial* functions, like:

- *Decorations* for the map
  - North arrow
  - Scale bar
  - Legend
  - ...
- *Tools* to customize the map view
  - Select a single item
  - Select multiple items by drawing a curve
  - Pan
  - Zoom to Extents
  - Zoom by drawing a rectangle
  - Zoom to Measure distance
  - ...
- *Edit* polygons, for example within a network, basin, or waterbody
  - Move geometry point(s)
  - Add geometry point(s)
  - Remove geometry point(s)
- Addition of *Area* elements, such as
  - Add 2D Cross-section
  - Add 2D Structure
  - Add 2D Pump
  - ...

Still, all functions of the category can be activated as they will appear in the drop-down panel.

5.4.6 Grid

D-HYDRO Suite 1d2d has an embedded grid editor which is accessable via the *Grid* ribbon (Figure 5.19).

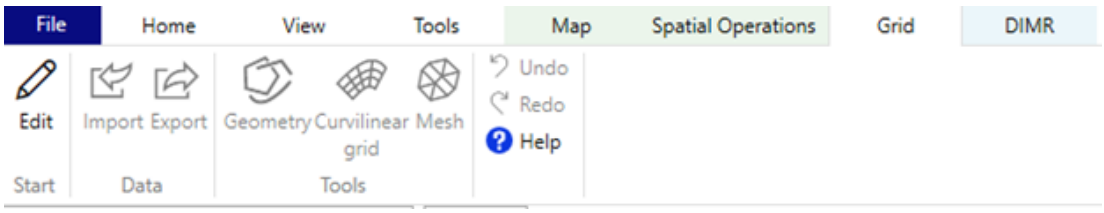


Figure 5.18: *The* Grid *editor ribbon*

DRAFT

This ribbon harbours all *Grid* editing functions, organized in the following chapters:

- *Edit* to start editing a mesh
- *Import* of data and/or grids/meshes
- *Export* of data and/or grids/meshes
- *Geometry* to work with polygons, splines, land boundaries and samples
- *Curvilinear grid* to generate / edit curvilinear grids
- *Mesh* to generate / edit unstructured meshes
- *Undo* and *Redo*

5.4.7 DIMR

The last *ribbon* is the *DIMR* ribbon (**??**).

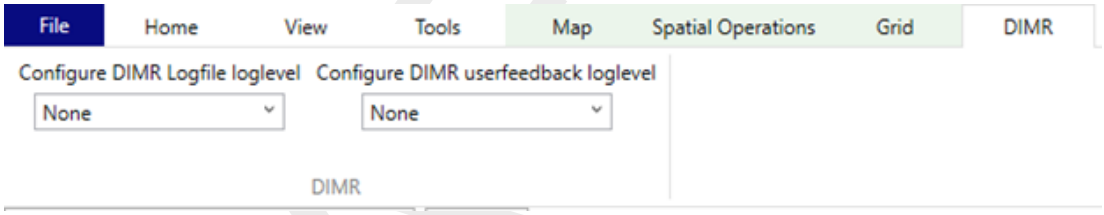


Figure 5.19: *The* DIMR *ribbon*

This ribbon enables the logging of DIMR, the Deltares Integrated Model Runner.

- *Configure DIMR Logfile loglevel*
- *Configure DIMR userfeedback loglevel*

5.4.8 Quick access toolbar

Note: The user can make frequently used functions available by a single mouse-click in the *Quick Access Toolbar*, the top-most part of the application-window. Do this by right-mouse-clicking a ribbon item and selecting *Add to Quick Access Toolbar*.

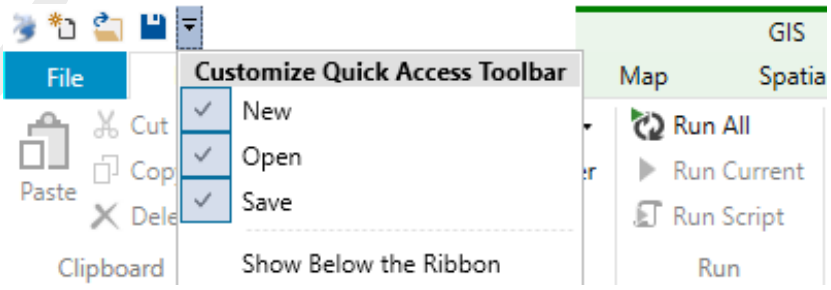


Figure 5.20: *The quick access toolbar.*

5.5 Important differences compared to Delft3D-FLOW GUI

The differences between the former Delft3D-FLOW GUI and the D-Flow FM GUI in lay-out and functionality are numerous. Here, we address only the most important differences in the workflow.

DRAFT

5.5.1 Project vs model

The entity “project” is new in the D-Flow FM GUI. In the hierarchy the entity “project” is on a higher level than the entity “model”. A project can contain multiple models, which can either run standalone or coupled. The user can run all models in the project at once (on project level) or each model separately (on model level). When the user saves the project, the project settings will be saved in a <∗.dsproj> configuration file and the project data in a <*.dsproj_data folder>. The <*.dsproj_data> folder contains folders with all input and output files for the models within the project. There is no model intelligence in the <∗.dsproj> configuration file, meaning that the models can also be run outside the GUI from the <*.dsproj_data> folder.

5.5.2 Load/save vs import/export

The user can load an existing D-Flow FM project, make changes in the GUI and, consequently, save all the project data. Loading and saving means working on the original project data, i.e. the changes made by the user overwrite the original project data. Alternatively, use *Save As* to keep the original project data and save the changes project data at another location (or with another name).

Import/export functionality can be used to copy data from another location into the project (import) or, vice versa, to copy data from the project to another location (export). Import/export is literally copying, e.g.:

- ◇ import: changes on the imported data will only affect the data in the project and not the source data (upon saving the project)
- ◇ export: the model data is copied to another location “as is”, changes made afterwards will only affect the data in the project not the exported data (upon saving the project)

5.5.3 Working from the map

One of the most important differences with the former GUI is the central map. The central map is comparable with the former “visualization area”, but with much more functionality and flexibility. The map helps you to see what you are doing and inspect the model at all times. You can use the *Region* and *Map* ribbons to add/edit model features in the map.

5.5.4 Coordinate conversion

With the map as a central feature, functionality to convert model and map coordinates is an indispensable feature. In the *General* tab you can set the model coordinate system. In the map tree you can set the map coordinate system using the right mouse button (Figure 5.21). The coordinate systems are subdivided in geographic and projected systems. Use the quick search bar to find the coordinate system you need either by name or EPSG code (Figure 5.22).

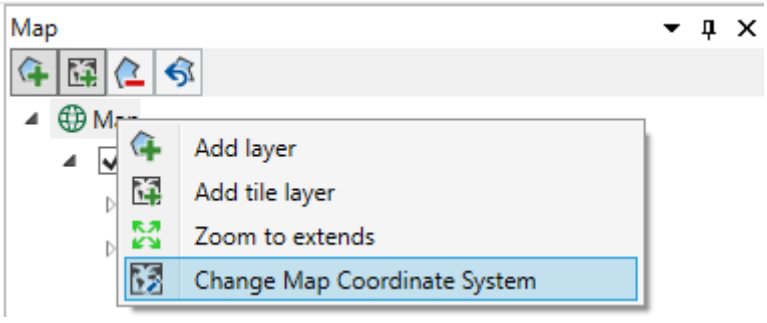


Figure 5.21: *Set map coordinate system using right mouse button*

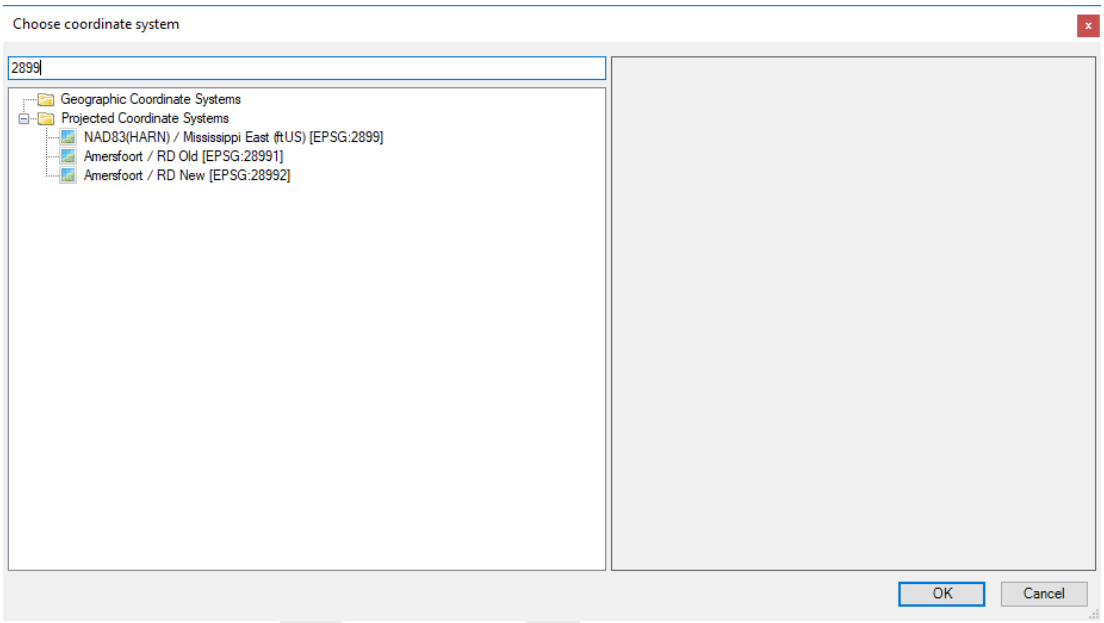


Figure 5.22: *Select a coordinate system using the quick search bar*

5.5.5 Model area

The model area contains geographical features, such as observation points & curves and obstacles. In contrast to the former GUI, these features can even exist without a grid or outside the grid and they are not based on grid coordinates, implying that their location remains the same when the grid is changed (for example by (de-)refining).

Finally, for the computations, the SWAN computational core interpolates the features to the grid. In the future we would like to show to which grid points the features are snapped before running the computation. However, this requires some updates in the SWAN computational core.

5.5.6 Integrated models (model couplings)

The D-Flow FM User Interface implements the concept of an *Integrated model* in order to couple different models, such as: hydrodynamics coupled with the controlling of structures, waves, morphology and/or water quality.

Two types of coupling are distinguished:

1 *offline* coupling and
2 *online* coupling.

Note: *Offline* is also referred to as *sequential* coupling and *online* as *parallel* coupling.

Offline

In case of an *Integrated model* with *offline* coupling, the entire hydrodynamic simulation is done first, i.e., *separately* from the second simulation. The file-based hydrodynamic output serves as input for the second simulation. As such, the hydrodynamic results drives the controlling of structures or the simulation of waves or water quality. In this offline case, there is no feedback from the waves or water quality to the hydrodynamic simulation. For many applications, this is good practice.

Online

An *online* coupling, on the other hand, exchanges data *every time* after computing a specified time interval. This tight coupling allows for direct feedback of the various processes on one another. This is crucial for controlling structures.

5.5.7 Ribbons (shortcut keys)

The User Interface makes use of ribbons, just like Microsoft Office. You can use these ribbons for most of the operations. With the ribbons comes shortcut key functionality, providing shortcuts to perform operations. If you press `Alt`, you will see the letters and numbers to access the ribbons and the ribbon contents (i.e. operations). For example, `Alt + H` will lead you to the *Home*-ribbon (Figure 5.23).

Note: Implementation of the shortcut key functionality is still work in progress.

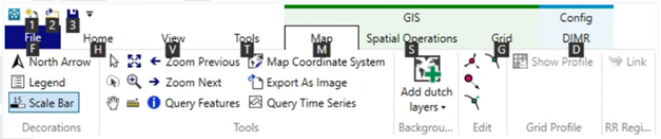


Figure 5.23: *Perform operations using the shortcut keys*

5.5.8 Context menus

Context menus are the menus that pop up using the right mouse button. These context menus provide you with some handy functionality and shortcuts specific for the selected item. The functionality is available in all User Interface windows and context dependent. You can best try it yourself to explore the possibilities.

5.5.9 Scripting

The User Interface has a direct link with scripting in Iron Python (NB: this is not the same as C-Python). This means that you can get and set data, views and model files by means of scripting instead of having to do it all manually. Scripting can be a very powerful tool to automate certain steps of your model setup or to add new functionality to the GUI. You can add a new script by adding a new item, either in the *Home*-ribbon or through the right mouse button.

DRAFT

6 Set-up of a D-Flow FM model

6.1 Introduction

In order to set up a hydrodynamic model you must prepare an input file. All parameters to be used originate from the physical phenomena being modelled. Also from the numerical techniques being used to solve the equations that describe these phenomena, and finally, from decisions being made to control the simulation and to store its results. Within the range of realistic values, it is likely that the solution is sensitive to the selected parameter values, so a concise description of all parameters is required. This input data is collected into the Master Definition Unstructured file, called a mdu-file.

In section 6.2 we discuss some general aspects of the mdu-file and its attribute files. The sections thereafter describe how the actual modelling process can be done in the GUI.

6.2 mdu-file and attribute files

The Master Definition Unstructured file (mdu-file) is the input file for the hydrodynamic simulation program. It contains all the necessary data required for defining a model and running the simulation program. In the mdu-file you can define attribute files in which relevant data (for some parameters) is stored. This will be particularly the case when parameters contain a large number of data (e.g., time-dependent or space varying data). The mdu-file and all possible user-definable attribute files are listed and described in chapter A.

Although you are not supposed to work directly on the mdu-file it is useful to have some ideas on it as it reflects the idea of the designer on how to handle large amounts of input data and it might help you to gain a better idea on how to work with this file.

The basic characteristics of an mdu-file are:

- ◇ It is an ASCII file.
- ◇ Each line contains a maximum of 256 characters.
- ◇ Each (set of) input parameter(s) is preceded by a (set of) *keyword*(s).

The results of all modules are written to platform independent binary (netCDF-)files, so also these result files you can transfer across hardware platforms without any conversion.

The mdu-file contains several sections, denoted by square brackets, below are the most relevant ones:

`[general]` this section contains the program name and its version.

`[geometry]` in this section, the main entry comprises the specification of the grid (i.e. the netCDF network file). In addition, thin dams and thin dykes can be specified.

`[numerics]` this section contains the settings of specific parts of the flow solver, such as limiters and the iterative solver type.

`[physics]` in this field, physical model parameters can be inserted, for instance related to friction modelling and turbulence modelling.

`[wind]` the wind section prescribed the dependency of the wind drag coefficient to the wind velocity through 2 or 3 breakpoints. This field also contains pressure information.

`[time]` in this section, the start time and the stop time of the simulation are specified (both as a combination of a date and time).

`[restart]` in this section, the restart file can be specified, either as a <∗_map.nc>-file or as an <∗_rst.nc> file. Because a <∗_map.nc>-file can contain multiple

DRAFT

stages, in this case also a `RestartDateTime` must be specified.
`[external forcing]` this section only contains the name of the external forcings file.
`[output]` in this section, the writing frequency of output data can be prescribed.

chapter A contains the full list of MDU sections and keywords.

6.3 Filenames and conventions

Filenames and file extensions hardly have any strict requirements in D-Flow FM, but we do advise to use the suggested file naming patterns below:

file pattern	description
mdu_name.mdu	mdu-file
∗_net.nc	Unstructured grid (network) file
∗.xyz	Sample file (for spatial fields)
∗.ldb	Land boundary file (polyline file format)
∗_thd.pli	Thin dam file (polyline file format)
∗_fxw.pliz	Fixed weir file (polyline file format with z values)
∗_crsdef.ini	1D cross section definition file (see section C.17.1)
∗_crsloc.ini	1D cross section location file (see section C.17.2)
roughness-∗ini	1D roughness variable file (see section C.16)
∗_part.pol	Partitioning polygon file (polyline file format)
∗.ext	External forcings file
pli_name.pli	Boundary condition location file (polyline file format)
*pli_name*_000X.tim	Timeseries boundary data file at point #X
*pli_name*_000X.cmp	Astronomic/harmonical component boundary data file at point #X
∗.bc	BC-format boundary data file with polyline and point labels in file
∗.xyn or ∗_obs.ini	Observation station file (see section F.2.2)
∗_crs.pli or ∗_crs.ini	Observation cross-sections file (see section F.2.4)
*mdu_name*_map.nc	Output map file
*mdu_name*_his.nc	Output his file
*mdu_name*_clm.nc	Output class map file (see section F.3.3)
mdu_name.dia	Output diagnostics (log) file

DRAFT

6.4 How to set-up a D-Flow FM model

This chapter describes how to set up a D-Flow FM model in an empty User Interface project. When you open the GUI, an empty project is automatically created. Starting from scratch, you have to create an empty D-Flow FM model in a User Interface project:

- ◇ In the ribbon menu items, go to *Home* and click on *New Model*. In the drop-down menu Figure 6.1 that appears, select "Flow Flexible Mesh Model" to add an empty D-Flow FM model.
- ◇ Or use the right mouse button on the name of your project (project1 by default). In the context menu that appears Figure 6.2, select *Add* and click *New Model*. The *Select model ...* window appears allowing you to add an empty D-Flow FM model.

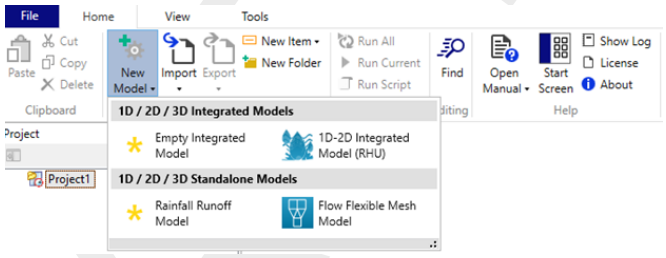


Figure 6.1: *The Add Model drop-down menu*

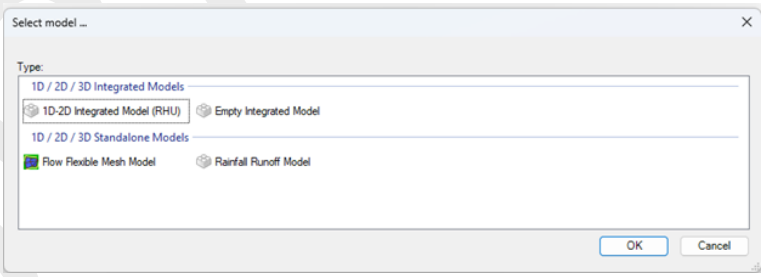


Figure 6.2: *The* Select model ... *window*

In the **Project** window, an empty model has appeared (FlowFM by default). Click the plus sign (+) before the name of the model to expand all model attributes in the **Project** window: General, Area, Grid, Bed Level, Time Frame, Processes, Initial Conditions, Boundary Conditions, Physical Parameters, Sources and Sinks, Numerical Parameters, Output Parameters and Output. In the following paragraphs, all model attributes are treated separately.

6.4.1 General

When you double click on *General* in the **Project** window, the tabbed editor for D-Flow FM appears (Figure 6.3). The *General* tab contains general model information such as the model coordinate system and the angle of latitude and of longitude.

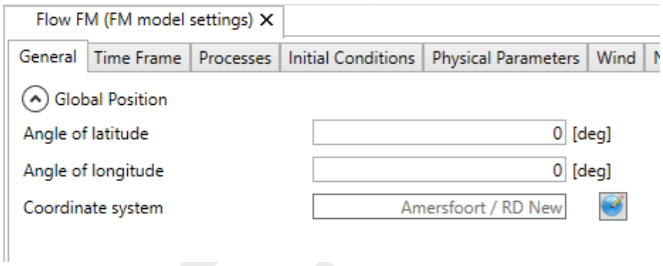


Figure 6.3: *Overview of the* General *tab*

6.4.1.1 Model coordinate system

A very important property of your model is the coordinate system in which it is specified. Within the interface, there is a clear distinction between the coordinate system of your model and all of its attributes and the coordinate system of the central map and all of its items. Both coordinate systems can be set independent from each other. Keep in mind, that the coordinate system of your model is saved and used when you run your model. Only coordinate systems supported by the computational core are supported.

The model coordinate system can be set using the globe icon next to the Coordinate system text box. After clicking this button, the coordinate system wizard is opened (Figure 6.4). This wizard allows you to choose one of many possible coordinate systems and apply it to your model. You can use the search bar to browse the various coordinate systems (searching possible by name and EPSG code). If your model is specified in a certain coordinate system already, it is possible to convert the model coordinate system using the same button. After clicking *OK*, all model attributes are converted to the system of choice. Note that you have to close and re-open all map views for the changes to take effect in these views.

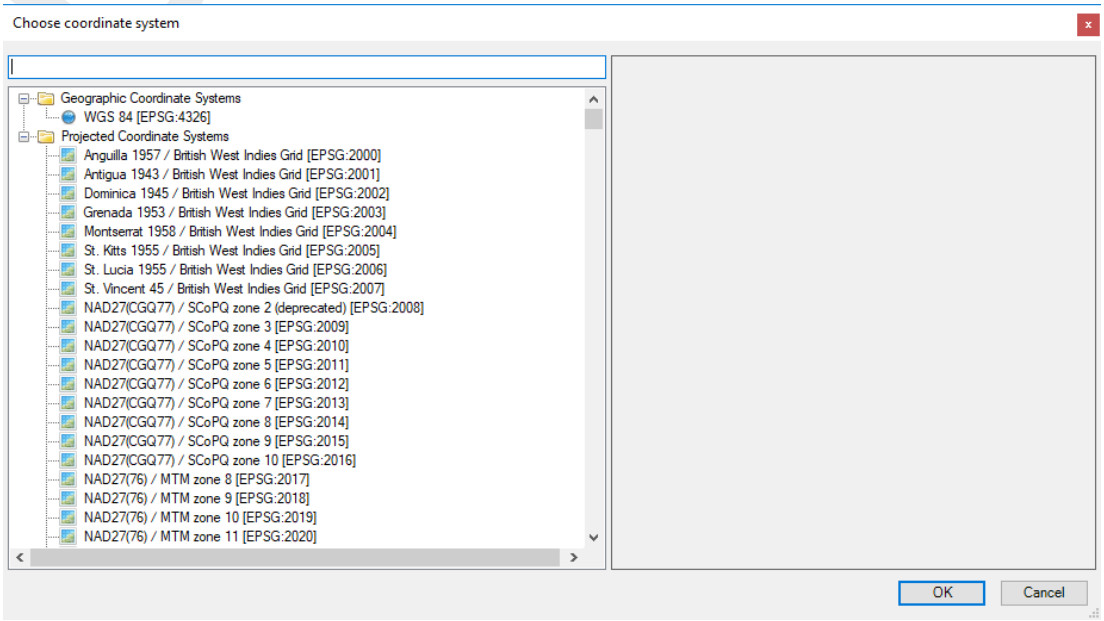


Figure 6.4: *Coordinate system wizard*

The map coordinate system applies to all items in the map **Project** window. To change the map coordinate system, navigate the menu ribbons to *Map* and click on *Map coordinate system*. Alternatively, right mouse click on *Map* in the map *Project* window and select *Change Map Coordinate System*. The coordinate system wizard appears, allowing you to set the map coordinate system of your choice. When the original model coordinate system differs from the selected map coordinate system, all map items are automatically converted to the specified map coordinate system.

6.4.1.2 Angle of latitude for Coriolis on a Cartesian grid

For a Cartesian grid, you must specify the latitude of the model area using the keyword `Anglat`; this value is used to calculate a fixed Coriolis force across the entire area. For a spherical grid, the Coriolis force is computed from the latitude values in the grid file and therefore varies with latitude. Typically, spherical coordinates are used for large areas, in the order of 500 by 500 km^2 or larger.

6.4.2 Area

The model area contains all geographical features, such as the observation points, structures and land boundaries. These features can exist without a grid or outside the grid as they are not based on grid coordinates but xy-coordinates. This means that the location of the model area features remains the same when the grid is changed (for example by (de-)refining). When you expand the model attribute *Area* in the *Project* window, a list of supported geographical features is displayed (Figure 6.5).

DRAFT

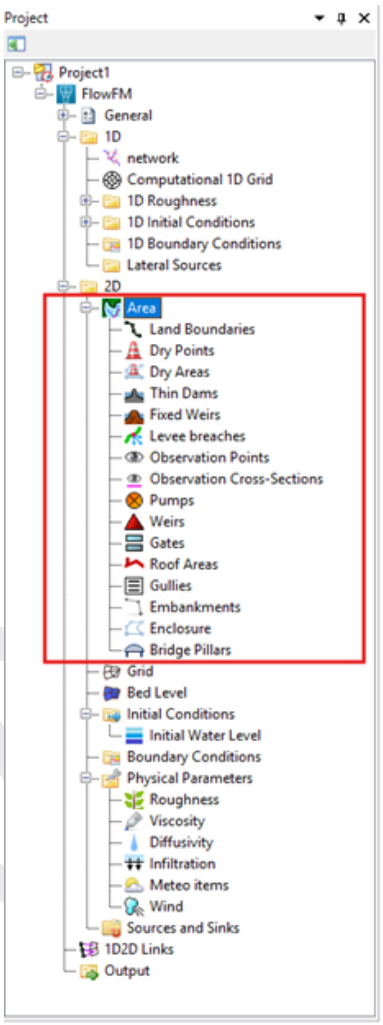


Figure 6.5: *Overview of geographical features*

Below is Figure 6.6. It displays an overview of the (*Map*) ribbon.

The red box to the right (*1D Network*, *2D Area*, *1D Region* ...) can be used to *Add ...* the various geographical features to the model: click the corresponding item in the box and use your mouse to indicate the location of the selected feature on the central map. The red box to the left (*Edit*) highlights the buttons available to edit the location of geographical features. The specifics for each feature are discussed separately in the following sections.

Figure 6.6: *Overview of the* Map *ribbon*

Importing and exporting of model features is done with the context menu by using your right mouse button on the different features in the *Project* window.

6.4.2.1 Grid-snapped features

All geographical features of your model that are described by x-, and y-coordinates have to be interpolated to your computational grid when you run your model. The computational

core of D-Flow FM automatically assigns these features to the corresponding parts of your grid. The so-called Grid-snapped features are part of the output of a simulation: *shape* files. In section 6.4.13 is described how - in short: select *Write Snapped Features* in the *Output Parameters* tab below the central map.

However, the graphical user interface allows you to inspect the grid-snapped features before a simulation run.
Note: Mark that this way, the grid-snapping is an estimation because it would take too much processing time. In the neighbourhood of *Dry Areas* the grid-snapping will be different, as the *Dry Areas* are not taken into account in this interactive *Estimated Grid-snapped features* mode.

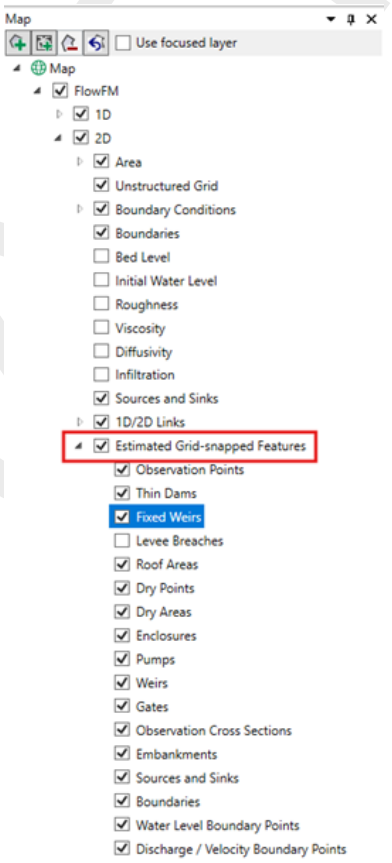


Figure 6.7: *Example of expanded* Estimated Grid-snapped features *attribute in the* Map *window*

Figure 6.7 shows a part of the *Map* window, showing the *Area* and *Estimated Grid-snapped features* attributes. The x- and y-locations of all spatial model features are shown within the *Area* attribute. You can hide or show any of these attributes by means of clicking the check boxes in front of the attributes. When you enable the Grid-snapped features, all items within the *Area* attribute, as well as all boundaries are interpolated to their corresponding locations on the computational grid. The interpolation is performed instantaneously by the computational core of D-Flow FM, which enables you to directly inspect the numerical interpretation of all features on the computational grid. Figure 6.8 shows an example of four observation points and one thin dam in the central map, showing both the x- and y-locations of these features as well as their representation on the computational grid.

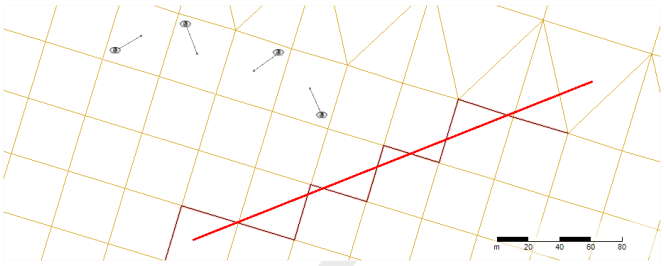


Figure 6.8: *Example of grid snapped features displayed on the central map*

6.4.2.2 Observation points

Observation points are used to monitor the time-dependent behaviour of one or all computed quantities as a function of time at a specific location, i.e., water level, velocity, salinity, temperature and concentration of the constituents. Observation points represent an Eulerian viewpoint at the results.

To add an observation point, click the corresponding icon from the *FM Region 2D / 3D* menu in the map ribbon (Figure 6.6). By clicking in the central map, observation points are placed in your model. Selected observation points (first click the *Select* icon from the "Tools" menu in the map ribbon) can be deleted using backspace or directly from the attribute table (explained below). The grid snapped representation (Figure 6.9) is indicated by a line linking the observation points to the closest cell center, indicating that output will be stored of this cell. Importing and exporting of observation points is possible via the context menu of "Observation points" in the *Project* window (right mouse button).

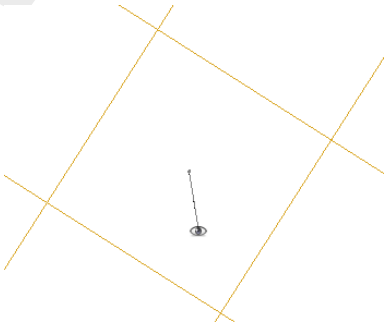


Figure 6.9: *Geographical and grid snapped representation of an observation point*

When you double click the *Observation points* attribute in the *Project* window, the observation points tab is displayed underneath the central map (Figure 6.10). This tab shows an attribute table with the names, x- and y-locations (in the model coordinate system) of the various observation points within the model. When one of the entries is selected, the corresponding observation point is highlighted in the central map.

DRAFT

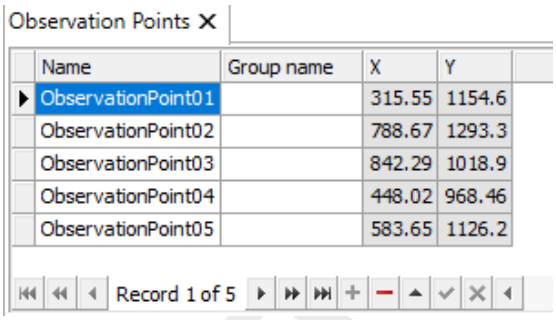


Figure 6.10: *Attribute table with observation points*

6.4.2.3 Observation cross-sections

Cross-sections (Figure 6.11) are used to store the sum of computed fluxes (hydrodynamic), flux rates (hydrodynamic), fluxes of matter (if existing) and transport rates of matter (if existing) sequentially in time at a prescribed interval.

To add a cross section, click the corresponding icon from the *FM Region 2D / 3D* menu in the map ribbon (Figure 6.6). By clicking in the central map, cross section points are added. Note that a cross section consists of a minimum of two points, but an arbitrary amount of intermediate points can be added. The last point is indicated by double clicking the left mouse button. The distance in meters (independent of the local coordinate system) in between the last point and the mouse pointer is indicated in pink; in case more than two points are used, the cumulative length of the entire cross section is shown in black. Once highlighted in the central map, a cross section is deleted with backspace or directly from the attribute table described below. The positive direction through the cross section is indicated by a pink arrow. The direction of this arrow is dependent on the order in which the cross section points are drawn (to change the direction, flip the start and end points). Importing and exporting of cross sections is possible via the context menu of “Observation cross-sections” in the *Project* window (right mouse button).

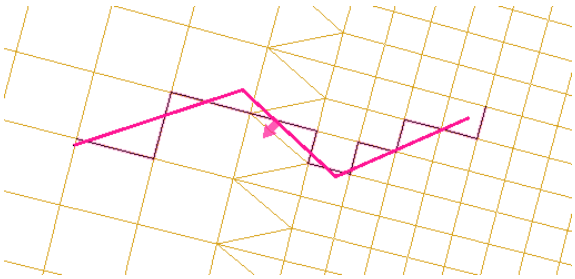


Figure 6.11: *Geographical and grid snapped representation of a cross section*

Double clicking the *Observation cross-sections* attribute in the *Project* window enables the Observation cross-sections tab underneath the central map view (Figure 6.12). Alternatively, you can double click on any cross section in the map. The attribute table displayed in the tab contains the names of the various cross sections of your model. When one of the entries is selected, the corresponding cross section is highlighted in the central map. A cross section entry can be deleted from the table via the context menu (right mouse button).

DRAFT

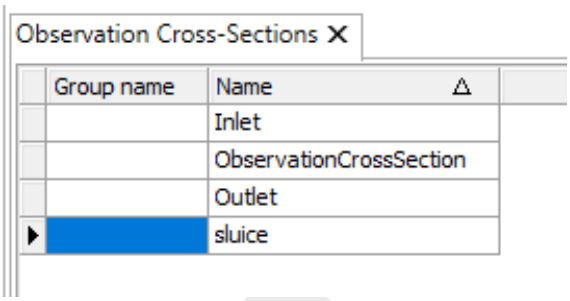


Figure 6.12: *Attribute table with observation cross sections*

6.4.2.4 Thin dams

Thin dams (Figure 6.13) are infinitely thin objects defined at the velocity points which prohibit flow exchange between the two adjacent computational cells without reducing the total wet surface and the volume of the model. The purpose of a thin dam is to represent small obstacles (e.g. breakwaters, dams) in the model which have sub-grid dimensions, but large enough to influence the local flow pattern. A thin dam is assumed to have an infinite level in the model; no water will ever overflow a thin dam.

To add a thin dam, click the corresponding icon from the *FM Region 2D / 3D* menu in the map ribbon (Figure 6.6). Adding, deleting, importing and exporting of a line feature such as a thin dam is discussed in more detail in section 6.4.2.3 on cross sections.

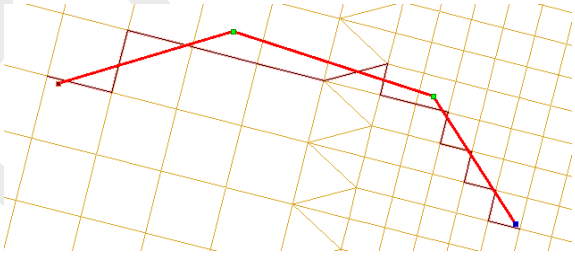


Figure 6.13: *Geographical and grid snapped representation of a thin dam*

When you double click the *Thin dams* attribute in the *Project* window, the corresponding Thin dams tab appears underneath the central map (Figure 6.14). Alternatively, you can also double click on any thin dam in the central map. Within this tab, an attribute table is shown which displays the names of all thin dams within your model. When one of the entries is selected, the corresponding item is highlighted in the central map. A thin dam entry can be deleted from the table via the context menu (right mouse button).

DRAFT

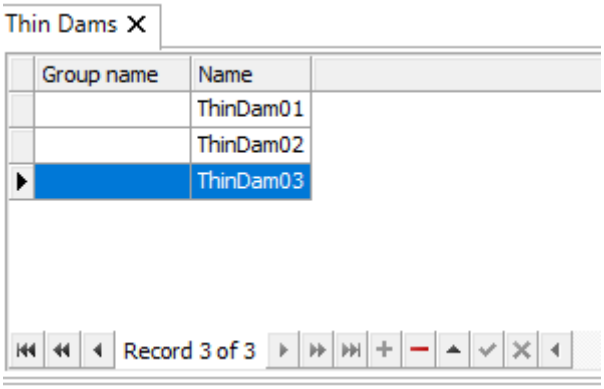


Figure 6.14: *Attribute table with thin dams*

6.4.2.5 Fixed weirs

A fixed weir (Figure 6.15) has the same function as a thin dam (section 6.4.2.4). However, unlike a thin dam, a fixed weir can be assigned both xy- and z-values. Furthermore, a fixed weir can be assigned a crest length (a thin dam is infinitely thin). The z-values correspond to the crest level of the fixed weir at the corresponding x- and y-locations; the level can vary in space, but is constant in time (Figure 6.16). Consequently, a fixed weir can overflow if the water level exceeds the crest level of the fixed weir. The level is specified with regard to the same vertical reference level as all other model items with level specifications (e.g. bed level values and initial water levels).

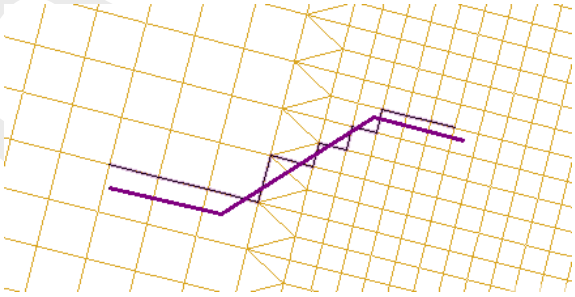


Figure 6.15: *Geographical and grid snapped representation of a fixed weir*

Figure 6.16: *Schematic representation of a fixed weir*

To add a fixed weir, click the corresponding icon from the *FM Region 2D / 3D* menu in the map ribbon (Figure 6.6). Adding, deleting, importing and exporting of a line feature such as a fixed weir is discussed in more detail in section 6.4.2.3 on cross sections.

When you double click the *Fixed weirs* attribute in the *Project* window, the corresponding Fixed weirs tab appears underneath the central map (Figure 6.17). Alternatively, you can also

double click on any fixed weir in the central map. Within this tab, an attribute table is shown which displays the names of all fixed weirs within your model. When one of the entries is selected, the corresponding item is highlighted in the central map. A fixed weir entry can be deleted from the table via the context menu (right mouse button).

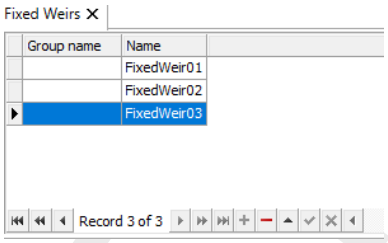


Figure 6.17: *Attribute table with fixed weirs*

When you double click on a fixed weir in the central map, the fixed weir editor opens in a separate view (Figure 6.18). On the right, a graphic representation (top view) of the fixed weir is displayed. The support point that is selected in the table is highlighted by means of a blue circle. On the left, a table is displayed showing the following properties of each support point of the fixed weir under consideration:

- X: x-coordinate of support point
- Y: y-coordinate of support point
- Crest level: crest level of the support point of support point of a fixed weir
- Ground height left: Difference between crest level and toe level at the left side
- Ground height right: Difference between crest level and toe level at the right side
- Crest width: Width of the crest of a fixed weir in perpendicular direction
- Slope left: Slope at left side of support point of a fixed weir
- Slope Right: Slope at right side at support point of support point of a fixed weir
- Roughness code: Roughness due to vegetation on a support point of a fixed weir

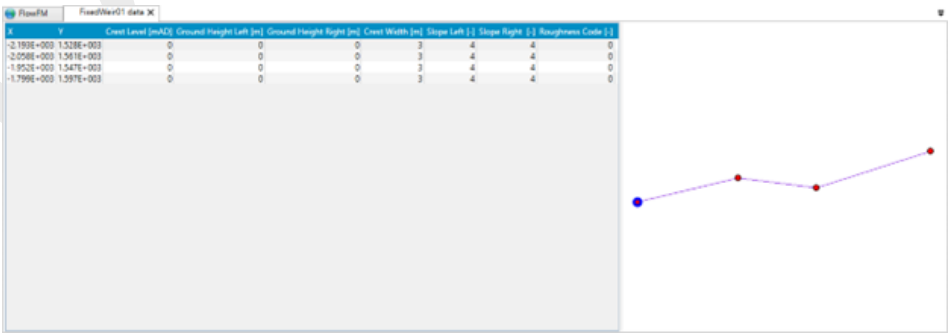


Figure 6.18: *Fixed weir editor*

6.4.2.6 Land boundaries

A land boundary (Figure 6.19) encloses the main geographic features surrounding your model and indicates the intersection of the water and land masses. When you set up your computational grid in the D-Grid Editor, a land boundary determines the onshore extent of your model. When you open the D-Grid Editor to edit your grid, then there is a toolbar for editing land boundaries; see Figure I.6.

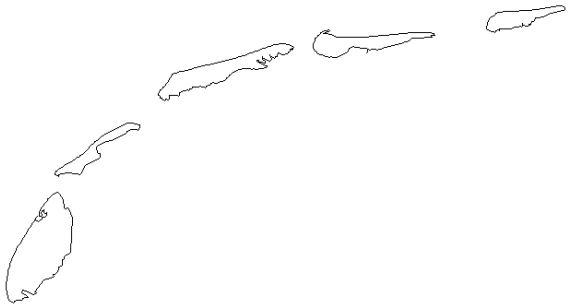


Figure 6.19: *Geographical representation of a land boundary*

To add a land boundary, click the corresponding icon from the *FM Region 2D / 3D* menu in the map ribbon (Figure 6.6). Adding, deleting, importing and exporting of a line feature such as a land boundary is discussed in more detail in section 6.4.2.3 on cross sections.

When you double click the *Land boundaries* attribute in the *Project* window, the corresponding land boundaries tab appears underneath the central map (Figure 6.20). Alternatively, you can also double click on any land boundary in the central map. Within this tab, an attribute table is shown which displays the names of all land boundaries within your model. When one of the entries is selected, the corresponding item is highlighted in the central map. A land boundary entry can be deleted from the table via the context menu (right mouse button).

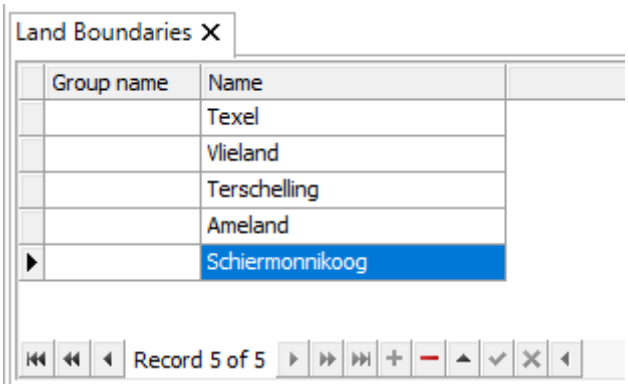


Figure 6.20: *Attribute table with land boundaries*

6.4.2.7 Dry points and Dry areas

Dry points are grid cells centred around a water level point that are permanently dry during a computation, irrespective of the local water depth and without changing the water depth as seen from the wet points. Dry areas are the same, but provide an easy way of defining many grid points as a single dry area at once.

Note that the flexibility of unstructured grids makes the use of dry points less necessary than with structured grid models, such as Delft3D-FLOW. In the interior of unstructured grids, some

or more grid cells can easily be deleted during grid manipulation. Still, a dry points file can be used to explicitly mark locations or regions inside the grid as dry cells.

Dry points in the GUI

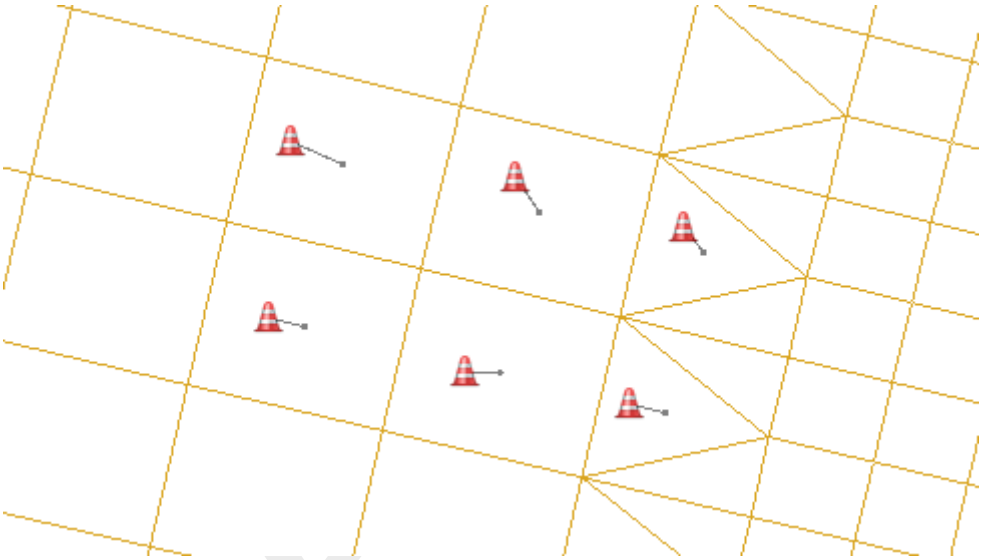


Figure 6.21: *Geographical and grid snapped representation of several dry points*

To add a dry point, click the corresponding icon from the *FM Region 2D / 3D* menu in the map ribbon (Figure 6.6). Adding, deleting, importing and exporting of a point feature such as a dry point is discussed in more detail in section 6.4.2.2 on observation points. The grid snapped representation of a dry point (Figure 6.21) is indicated by a line linking the dry point to the closest cell center.

When you double click the “Dry points” attribute in the *Project* window, the corresponding Dry points tab appears underneath the central map (Figure 6.22). Alternatively, you can also double click on any dry point in the central map. Within this tab, an attribute table is shown which displays the names of all dry points within your model. When one of the entries is selected, the corresponding item is highlighted in the central map. A dry point entry can be deleted from the table via the context menu (right mouse button).

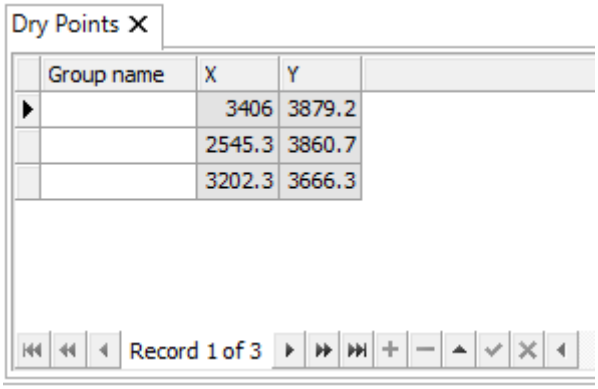


Figure 6.22: *Attribute table with dry points*

Dry areas in the GUI

Dry areas (Figure 6.23), like dry points, indicate areas that permanently dry during a computation. Instead of adding many separate dry points, you can draw a polygon that encloses all required computational cells. Only cells which centers are strictly inside the polygon are taken into account. The grid snapped representation of the dry area clearly indicates which cells are considered within the dry area.

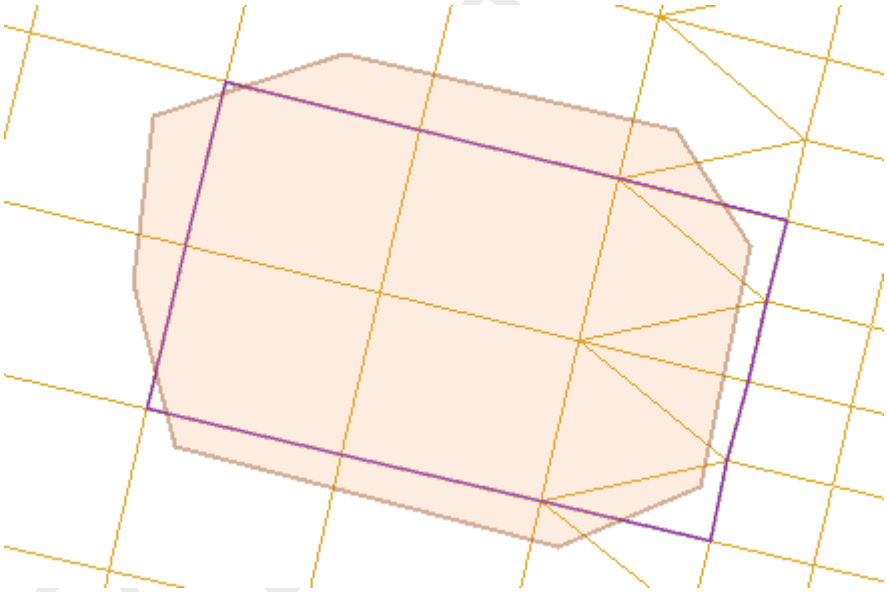


Figure 6.23: *Geographical and grid snapped representation of a dry area*

When you double click the *Dry areas* attribute in the *Project* window, the corresponding Dry areas tab appears underneath the central map (Figure 6.24). Alternatively, you can also double click on any dry area in the central map. Within this tab, an attribute table is shown which displays the names of all dry areas within your model. When one of the entries is selected, the corresponding item is highlighted in the central map. A dry area entry can be deleted from the table via the context menu (right mouse button).

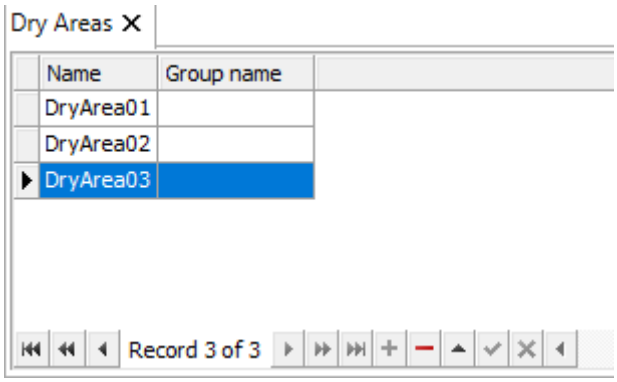


Figure 6.24: *Attribute table with dry areas*

Dry points file input

Dry points are defined by a sample file <∗.xyz>, dry areas are defined by a polygon file <∗.pol>. Add the filename to the MDU as below:

```
[geometry]
# ...
DryPointsFile = <filename.xyz> # Dry points file *.xyz, third column dummy z values,
                               # or polygon file *.pol.
```

The format of the sample file is defined in section C.3. The format of the polygon file is defined in section C.2. All grid cells that contain a sample point are removed from the model grid, and as a result do not appear in any of the output files. Alternatively, for a polygon file, all grid cells whose mass center lies within the polygon will be removed from the model grid.

6.4.2.8 Pumps

Pumps are a type of structures in D-Flow FM. Unlike the other structures, pumps can force the flow only in one direction. This direction is shown by an arrow in D-Flow FM. The flow direction of a pump can be reverted by a mouse right-click and selecting *Reverse line(s)*.

Like all other structures in D-Flow FM, the pump can be defined by a polygon. The input data of the pumps can be given by selecting and editing the pump polygon (see Figure 6.25). Right click on the pump polygon and selecting *Delete Selection* leads to deletion of the selected pump. Double clicking the pump polygons (or right-click the pump in the list and select *Open view*), it opens a tab for editing the pump properties. The tab includes pump capacity. If the pump capacity is time dependent, it can be specified by time series data (Figure 6.25).

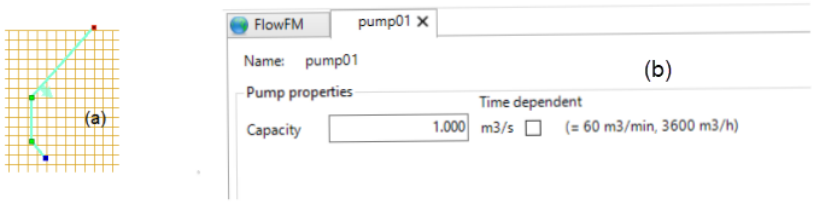


Figure 6.25: *Polygon for pump (a) and adjustment of physical properties (b).*

Right clicking the pumps attribute in the *Project* window opens a pop-down window on which you can select to import or export pumps. The pumps can be imported from and exported to a <∗.pli(z)> file or a <*shapefile*>.

Double clicking the pumps attribute in the *Project* window opens the *Pumps* tab underneath the central map. The attribute table in this tab shows all pumps with their corresponding properties. When one of the pumps is selected, the corresponding item is highlighted in the central map. Double clicking any of the pumps in the central map opens the Structure Editor underneath the central map in which all parameters related to the pump can be set (Figure 6.26).

DRAFT

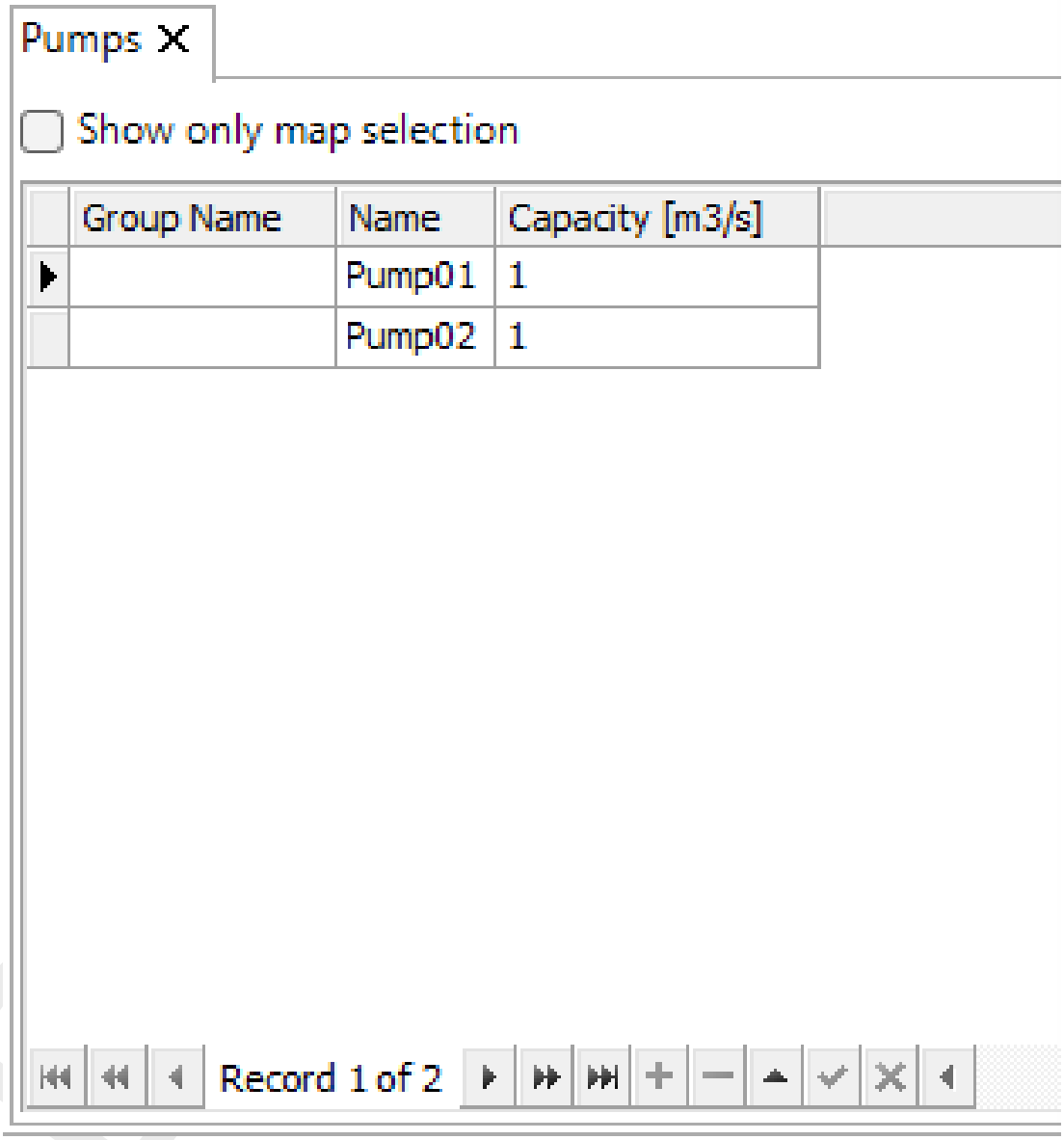


Figure 6.26: *Selection of the pumps*

6.4.2.9 Weirs

Unlike the fixed weir, weir (or adjustable weir) can be adjusted based on the user requirements. To set an adjustable weir in the computational domain, you can select the icon *Add new structure (2D)* from the toolbar, and draw a line by mouse. This line includes direction, which defines the sign of total flux passing above the structure (positive flux in the direction of weir, otherwise negative). This direction can be inverted by mouse right-click and selecting *Reverse line(s)*. By double-click on the line, you can define this structure as *Simple weir* and add the geometrical and time-dependent parameters such as *Crest level*, *Crest width*, *Crest level time series* and *Lateral concentration coefficient* (See Figure 6.27).

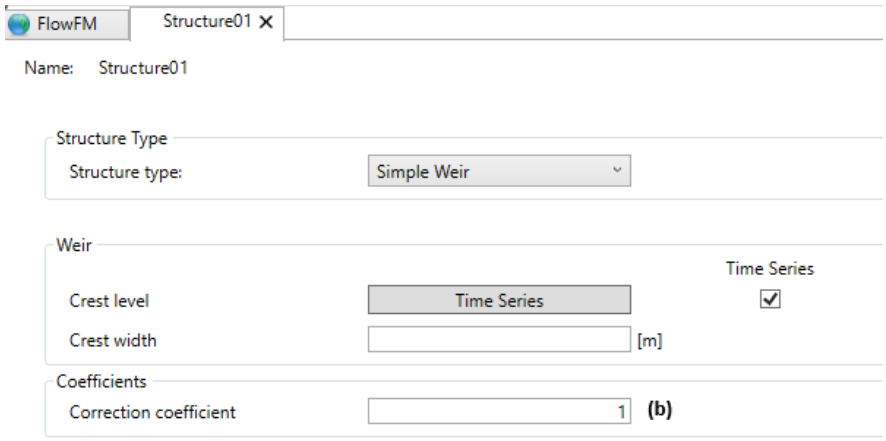


Figure 6.27: *Polygon for adjustable weir (a) and adjustment of geometrical and temporal conditions (b).*

Moreover, the time series of the crest level can be set in the case the crest level is time dependent. The time dependency diagram can be defined by the help of time series diagram as shown in Figure 6.28. The time series can also be imported (and exported) from external <csv>-file.

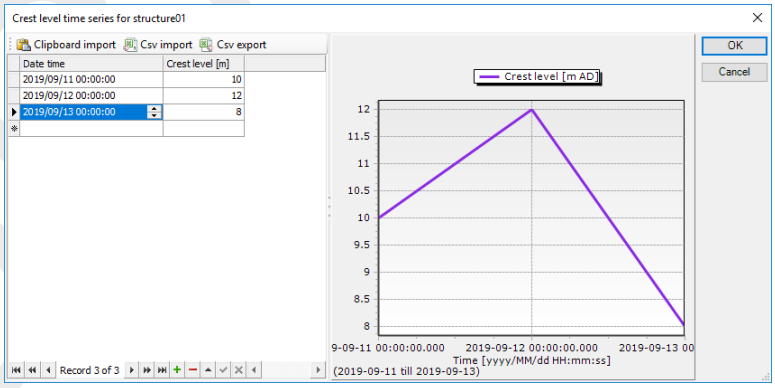


Figure 6.28: *Time series for crest level.*

The weirs can be deleted, imported and exported. By right clicking on the weir polygon, and selecting *Delete Selection* from the pop-down window, you can delete the selected weir. Right clicking the "Weirs" attribute in the *Project* window opens the "Weirs" tab opens the options for import and export. You can import weirs as polygon (<∗.pli> file) or as a structure by structure file. The weirs can also be exported to a polygon file, to a structure data file, or by the help of a shape file.

Double clicking the "Weirs" attribute in the *Project* window opens the "Weirs" tab underneath the central map. The attribute table in this tab shows all weirs with their corresponding properties. When one of the weirs is selected, the corresponding item is highlighted in the central map. Double clicking any of the weirs in the central map opens the Structure Editor as a new map view in which all parameters related to the weir can be set (Figure 6.29).

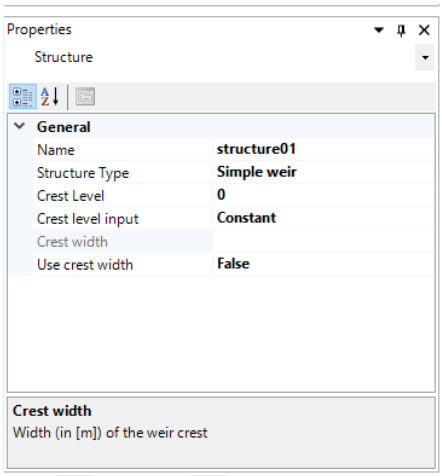


Figure 6.29: *Time series for crest level.*

6.4.2.10 Gates

In D-Flow FM the gates can be imposed by polygon, and can be edited in a similar way as the other structures (see Figure 6.30). Like the other structures, mentioned above, the gates can be imported and exported by means of structure file or $<*.\text{pli}>$ file.

Figure 6.30 shows the edit tab of the gate properties. The gate can be opened horizontally, as well as vertically.

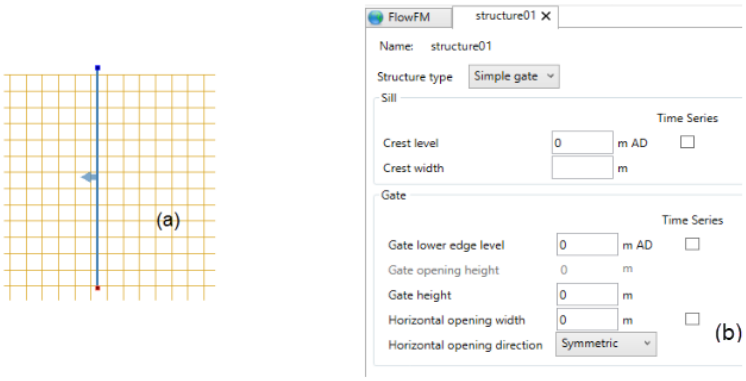


Figure 6.30: *Polygon for gate (a) and adjustment of geometrical and temporal conditions (b).*

6.4.2.11 Bridge Pillars

Remark:

- ◇ Bridge pillar functionality is currently only available for 1D and 2D modelling, but not for 3D modelling yet.

Bridge pillars are a type of structures in D-Flow FM that are relevant for the hydrodynamics. In D-Flow FM, a bridge is defined by a polygon on the central map. The user can add a bridge to the model by

- ◇ selecting the *Add new bridge pillar* icon in the *Map* ribbon; and
- ◇ defining the location of the bridge pillars by clicking on the map. The polyline for a bridge is finalized by double-clicking on the last bridge pillar.)
- ◇ repeat the above for another bridge and end by pressing the *<ESC>* key

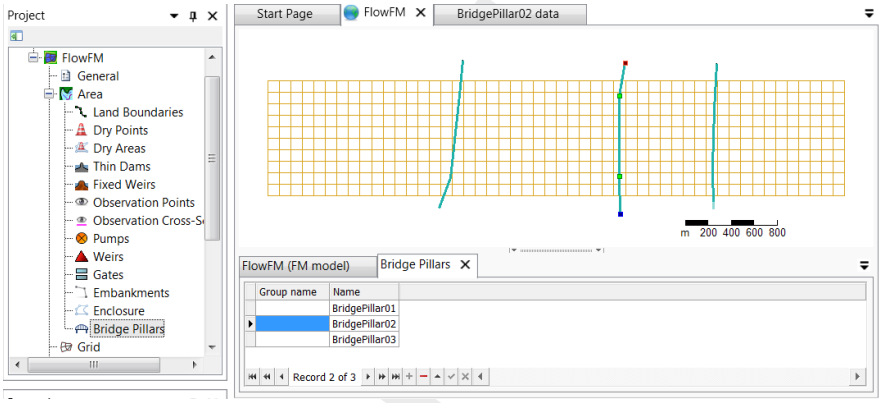


Figure 6.31: *Selection of a bridge*

Right clicking on the bridge polygon and selecting *Delete Selection* leads to deletion of the selected bridge. After double clicking a bridge polygon or right clicking the bridge in the list and selecting *Open view*, a tab for editing the bridge properties opens (Figure 6.32), with the following parameters:

- ◇ Pillar diameter [m] - (use -999.0 for an auxiliary location)
- ◇ Drag coefficient [-]: default is 1.0

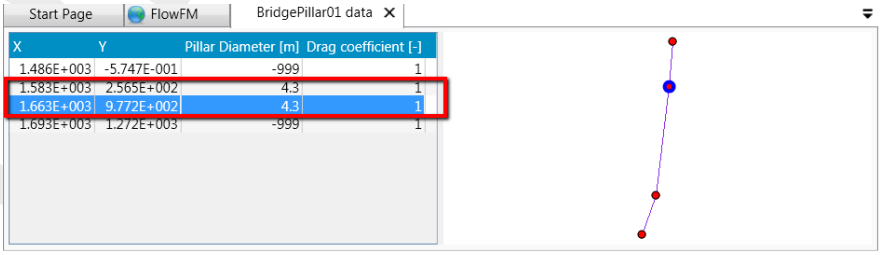


Figure 6.32: *Adjustment of the properties of the pillars*

Right clicking the *Bridge Pillars* attribute in the *Project* window (Figure 6.31) opens a pop-down window on which you can select to import or export bridges. The bridges can be imported and exported as polyline by a *<∗.pliz>* file or *<shapefile>*.

As can be seen in Figure 6.32, some of the points of the polygon are relevant pillars with real dimensions and some are not. For example, if a bridge is of curved shape, then additional points can be used to visualize such a bridge, while some of the points of a polyline do not coincide with a pillar. Such points are modelled with a pillar diameter of -999.0.

6.4.3 Computational grid

To set up a grid, click on the *Edit grid* button which opens the program D-Grid Editor. Then a lot of features are available generate the computational grid; see chapter I

6.4.4 Bed level

When you double click on *Bed level* in the *Project* window or select *Bed level* from the drop-down box in the spatial editor section of the *Map* ribbon, the spatial editor is activated (Figure 6.33). This editor can be used to generate a bathymetry for your computational grid. How to work with the spatial editor is described in chapter H. Be aware that the bathymetry in D-Flow FM is defined as the bed level (e.g. positive upward), implying that all bed levels below the reference plane are negative. By default the bed levels are defined on the net nodes.

Note: Please note that, currently, other bed level definition types (e.g. `BedlevTypes`) are not visually supported by the GUI. If you would like to switch the bed level defintions to another type, you have to set the `BedlevType` in the *Physical Parameters* tab. However, the bed level locations will not be updated accordingly in the central map.

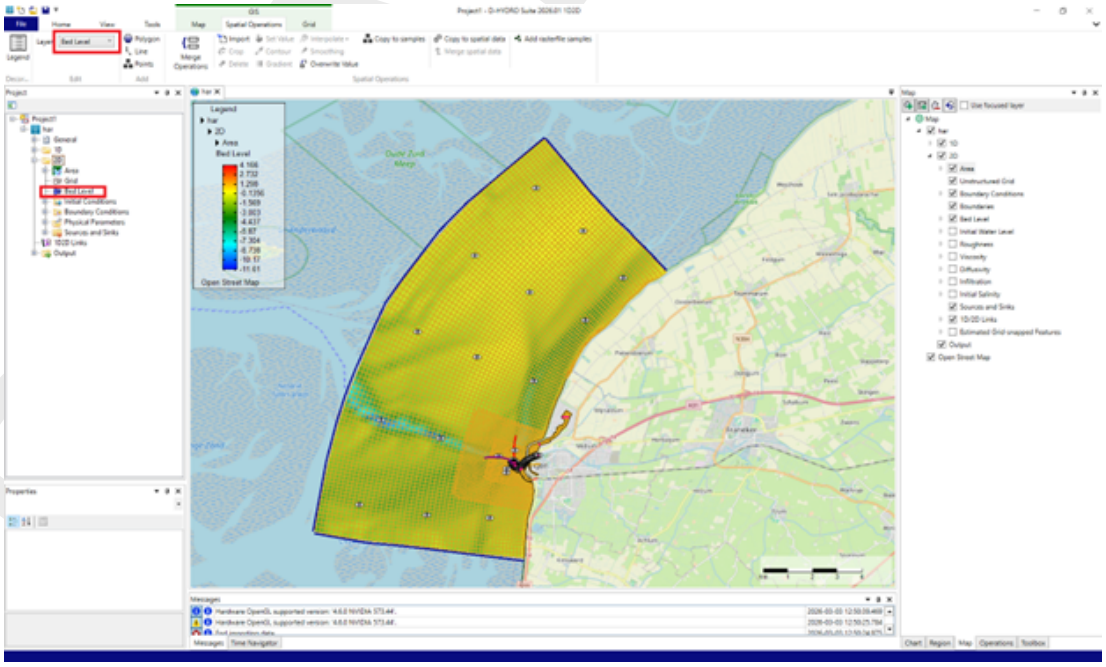


Figure 6.33: *Bed level activated in the spatial editor*

6.4.5 Time frame

In the settings tab, in the sub-tab time frame (Figure 6.34), you can specify everything related to the time frame in which your model will run.

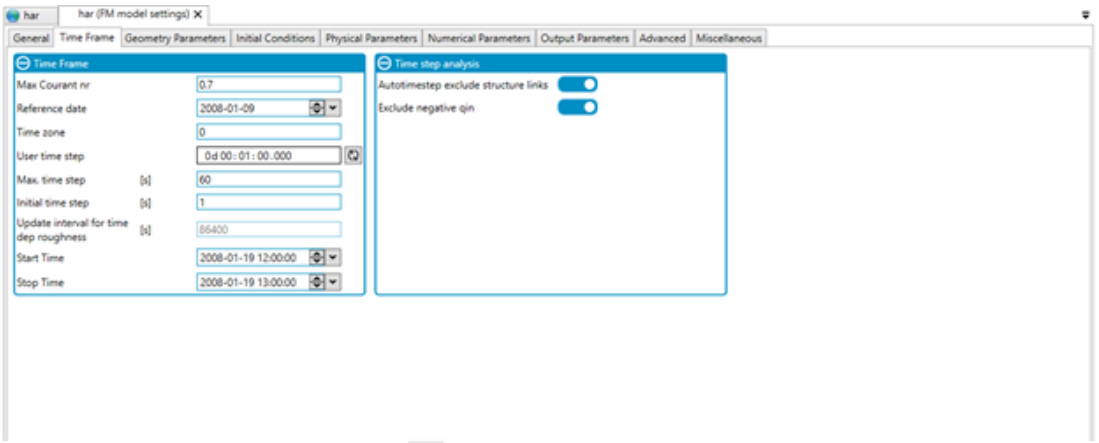


Figure 6.34: *Overview time frame tab*

In general, the time frame is defined by a reference date and a start and stop time. The time step size of your model is automatically limited (every time step) based on a Courant condition. In more detail, you must define the following input data:

Max Courant nr The maximum allowed Courant number, which is used to compute the time step size from the CFL criterium. D-Flow FM uses an explicit advection scheme, therefore a value of 0.7 or lower is advised.
Remark:
- ◇ You should check the influence of the time step on your results at least once.

Reference date The reference date of the simulation, which is by definition always at midnight (00h00m00s). It defines the (arbitrary) $t = 0$ point for all time-series as used in the simulation. In the GUI, time-specifications are always absolute, but in the underlying model input files, these are stored as time values relative to the reference date. Typically, input time-series files are specified in minutes after this $t = 0$ point. See for an illustration Figure 6.35.

Time zone The time difference between local time and UTC.
The time zone is defined as the time difference (in hours) between the local time (normally used as the time frame for D-Flow FM) and Coordinated Universal Time (UTC). The time difference can consist of a part of an hour, for example 1 hour and 30 minutes (specify as a decimal value: 1.5 (hours)). The local Time Zone is used for two processes:

- ◇ To determine the phases in local time of the tidal components when tide generating forces are included in the simulation, see section 3.8.
- ◇ To compare the local time of the simulation with the times at which meteo input is specified, e.g., wind velocities and atmospheric pressure. These can be specified in a different time zone.

If the *Time Zone* = 0 then the simulation time frame will refer to UTC.

User time step The interval that is highest in the hierarchy. It specifies the interval with which the meteorological forcings are updated. The *Max. time step* cannot be larger than the *User time step*, and it will automatically be set back if it is. Also, the output intervals should be a multiple of this *User time step*, see chapter F. Finally the computational time steps will be fitted to end up exactly at each *User time step*, such

DRAFT

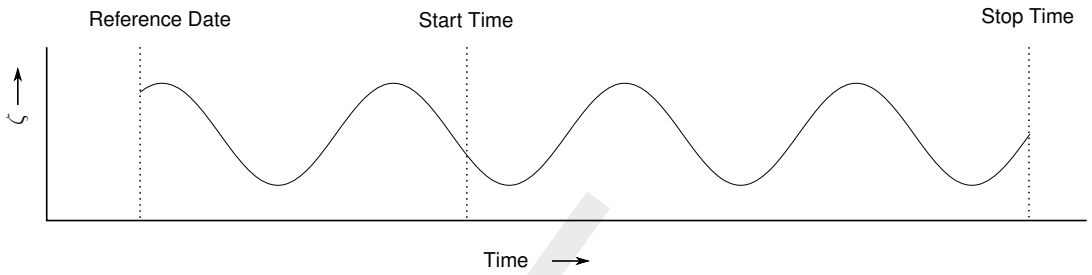


Figure 6.35: *Relation between Reference Date and the simulation start and stop time for astronomic- and harmonic-series as used in the simulation. Time-series should cover the simulation time.*

that proper equidistant output time series are produced.

Nodal update interval When using astronomic boundary conditions, the nodal factors can be updated with certain intervals, see section 3.8.

Max. time step The *Max. time step* is the upper limit for the computational time step. The automatic time step can not be switched off explicitly. (If you want to enforce a fixed time step anyway, set the parameter *Max. time step (s)* to the desired step size, and the parameter *Max. Courant nr.* to an arbitrary high value.)

Initial time step the initial time step of the model; there is no data available yet during the first time step to compute the time step automatically based on a Courant condition. The computational time step then gradually increases from *Initial time step* to the CFL-number limited time step (assuming that *Initial time step* is relatively small).

Start Time The start date and time of the simulation.

Stop Time The stop date and time of the simulation. Always make sure that the model *Stop Time* is larger than the model *Start Time* to avoid errors during your calculation.

6.4.6 Initial conditions

When expanding the initial conditions in the *Project* window, all quantities requiring an initial state are shown (Figure 6.36). The number of quantities depends on the activated physical processes in the 'Processes' tab (see **??**). The initial conditions for each quantity can be specified as a uniform value or as a coverage (e.g. a spatially varying field).

The uniform values can edited in the 'Initial Conditions' tab, which opens upon double clicking 'Initial Conditions' in the *Project* window (Figure 6.37). In this tab you can also specify the layer distribution for the initial condition specification in case of a 3 dimensional quantity (i.e. salinity). **Note:** Please note that for 3 dimensional initial conditions currently only the option 'top-bottom' is supported.

In case of spatially varying initial conditions you can double click the quantity in the *Project* window or select it from the drop-down box in the spatial editor section of the Special Operations ribbon (Figure 6.38). Then the spatial editor is activated, which you can use to edit spatially varying fields. For more information on how to use the spatial editor you are referred to chapter H. In case of 3 dimensional initial conditions, you can select the layer from the quantity dropdown box in the 'Map' ribbon (Figure 6.39).

DRAFT

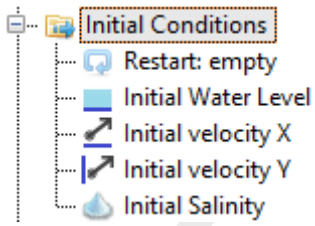


Figure 6.36: *Initial conditions in the* Project *window*

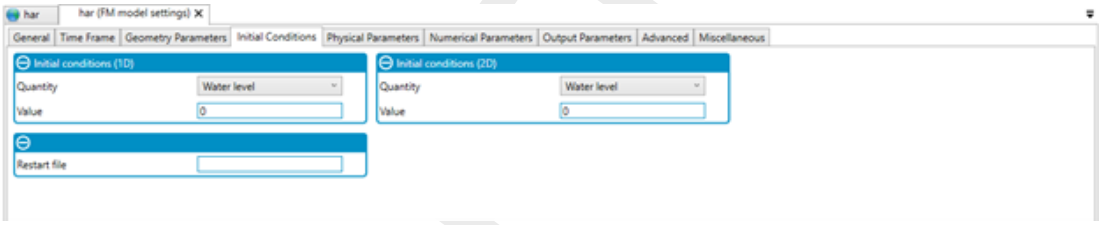


Figure 6.37: *The 'Initial Conditions' tab where you can specify the uniform values and the layer distributions of the active physical quantities.*

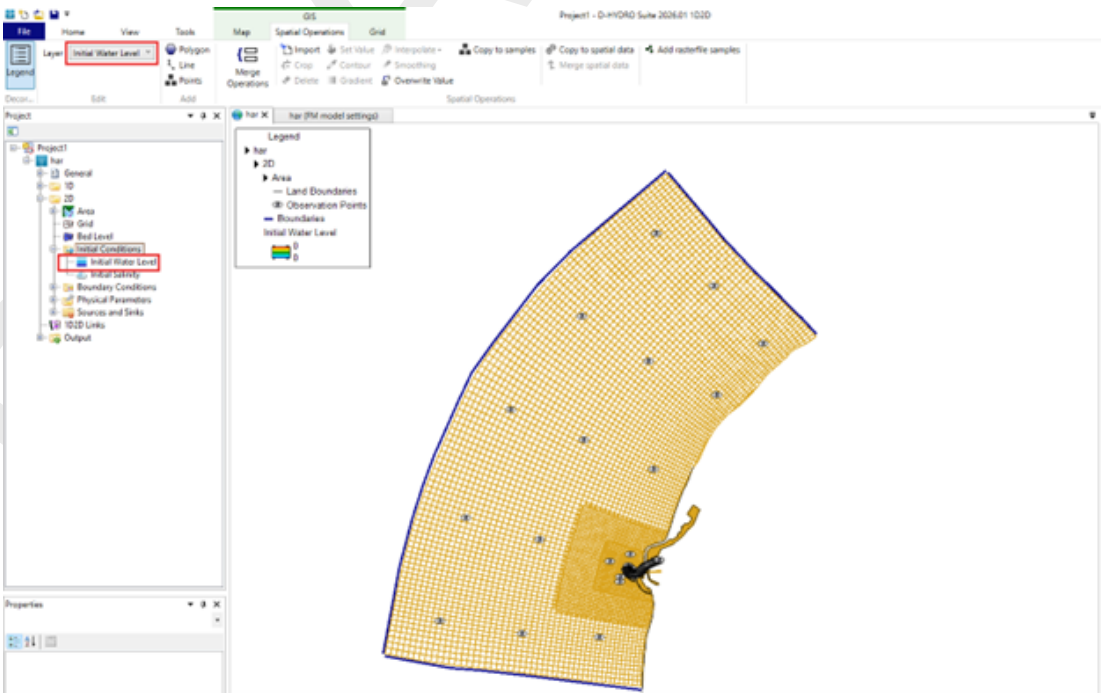


Figure 6.38: *Initial water levels activated in the spatial editor*

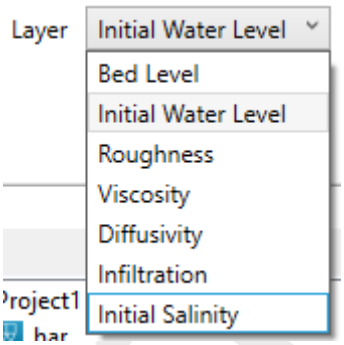


Figure 6.39: *Selecting 3 dimensional initial fields from the dropdown box in the 'Map' ribbon to edit them in the spatial editor*

Instead of defining initial conditions from scratch you can also import fields from a previous computation (using restart files). When you run a model using the D-Flow FM GUI which is writing restart files, the restart states will appear in the "Output" folder in the *Project* window (Figure 6.40). For a description of the specification of restart files, please refer to paragraph section 6.4.13. To use a restart file as initial conditions, apply the right mouse button and select "Use as initial state". The file will now appear under "Initial Conditions" in the *Project* window (Figure 6.41). To activate the restart file utilize the right mouse button and select "Use restart". The file will no longer be grey, but is now highlighted in black. The model will now restart from this file. Notice that the simulation still starts at the original *User Start Time*, rather than the time of the restart file. (The restart file only provides initial conditions.)

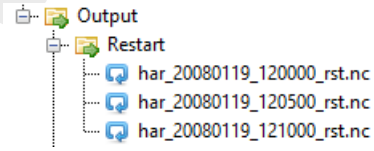


Figure 6.40: *Restart files in output states folder*

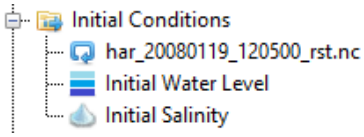


Figure 6.41: *Restart file in initial conditions attribute*

6.4.7 Boundary conditions

Boundary conditions consist of a location specification ('support points') and a forcing for that location.

Section 6.4.7.1 describes how support points can be specified in D-Flow FM User Interface. The boundary forcing can be specified in the boundary data editor.

Section 6.4.7.2 describes the functionality of the boundary data editor. Finally, section 6.4.7.4 describes how the user can get an overview of the boundary locations and forcing in the attribute table.

DRAFT

6.4.7.1 Specification of boundary locations (support points)

In D-Flow FM the boundary locations are defined as 'support points' on a polyline (<∗.pli>). The user can add a boundary polyline (<∗.pli>) in the central map by selecting the *Add Boundary* button in *FM Region 2D / 3D* of the *Map* ribbon (see Figure 6.42). The number of individual mouse clicks determines the number of support points on the polyline. The polyline is closed by a double click. Once the polyline is added it becomes visible in the *Project* window under 'Boundary Conditions' (see Figure 6.43). The polyline can be edited by the general edit operations in the 'Map' ribbon (i.e. add/delete/move individual geometry points or the complete geometry, see Figure 6.44). The name of the polyline (or 'boundary') can be edited in the Boundaries tab, which can be opened by double clicking 'Boundaries' in the *Map* window (see Figure 6.45).

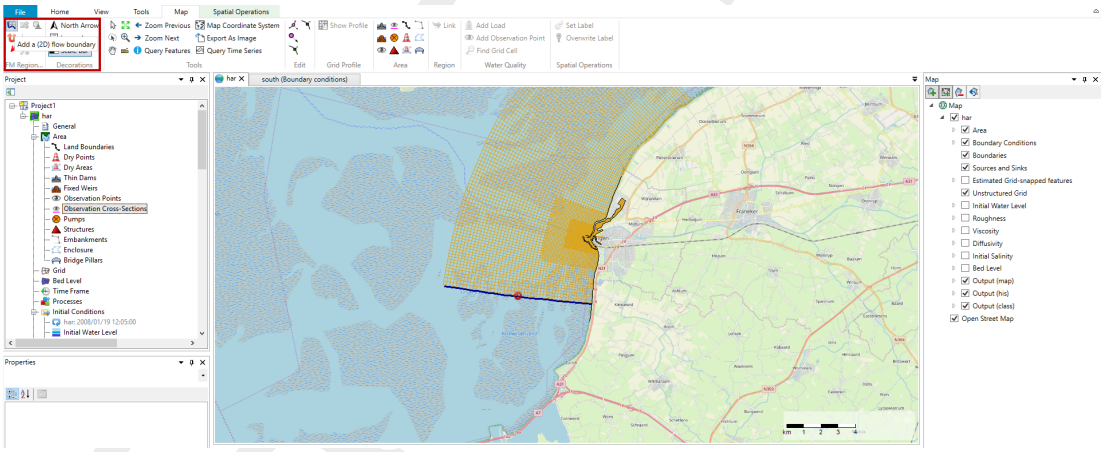


Figure 6.42: *Adding a boundary support point on a polyline in the central map. By double clicking on the polyline in the map, the boundary condition editor will open to edit the forcing data on the polyline.*

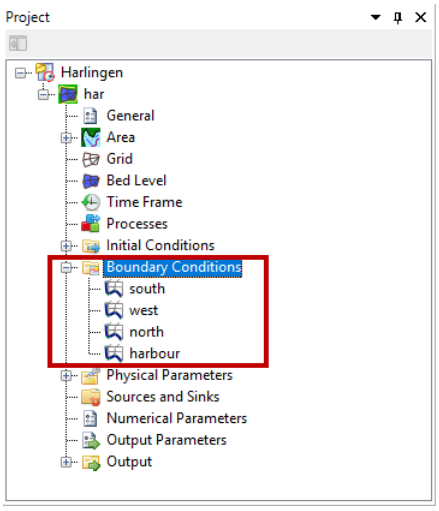


Figure 6.43: *Polyline added in* Project *window under 'Boundary Conditions'. By double clicking on the name of the polyline, the boundary condition editor will open to edit the forcing data on the polyline.*

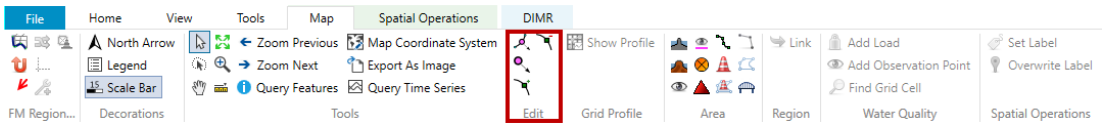


Figure 6.44: *Geometry edit options in* Map *ribbon*

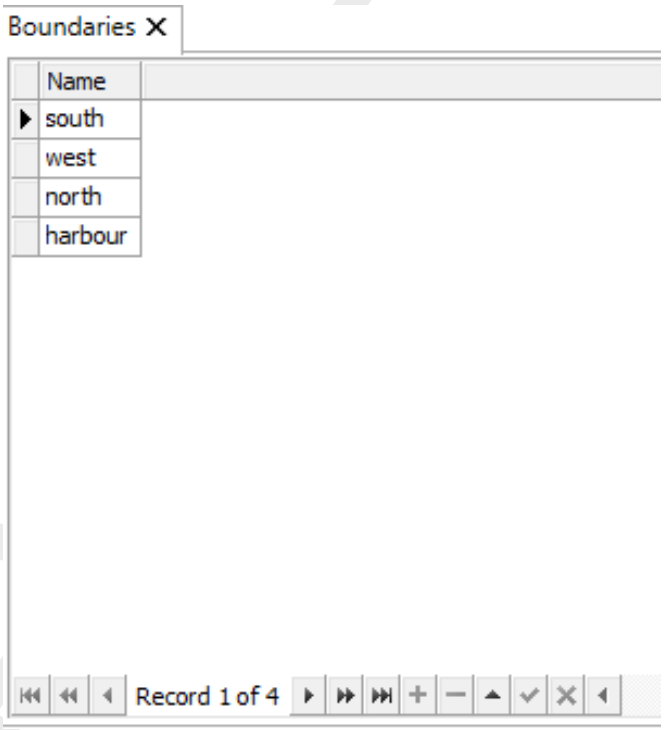


Figure 6.45: *Edit name of polyline/boundary in Boundaries tab*

6.4.7.2 Boundary data editor (forcing)

The boundary condition editor can be opened either by double clicking on the boundary name in the *Project* window (Figure 6.43) or by double clicking the boundary polyline in the map window (Figure 6.42). An overview of the boundary data editor is given in Figure 6.46. This editor can be used to specify the boundary forcing for different quantities (i.e. water level, velocity, discharge, salinity, etc.) corresponding to different processes (i.e. flow, salinity, temperature, tracers). The user can select the processes and quantities in the upper left corner. In the upper centre panel the user can select the forcing function for the selected quantity (i.e. time series, harmonic components, astronomical components or Q-h relation). The upper right corner contains a list of the support points on the polyline. The geometry view shows the location of the selected support point on the polyline (<∗.pli>). In the middle panel are some handy buttons to generate, import and export forcing data. In the lower left panel the user can specify the boundary data for the selected support point. The lower right corner shows the signal of the boundary data. The following sections describe the features of the boundary data editor in more detail.

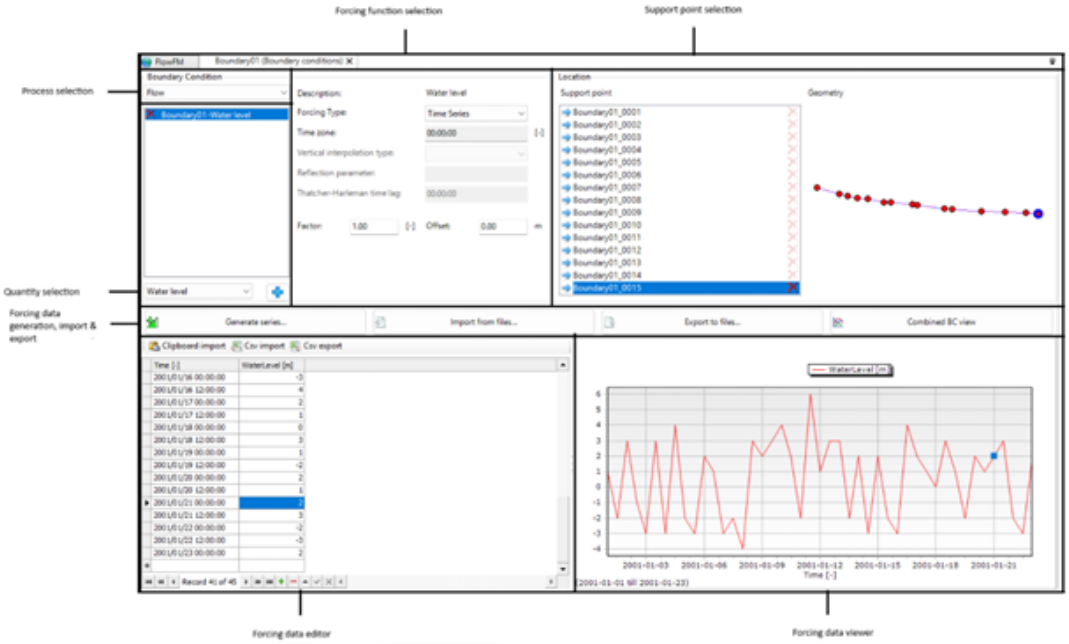


Figure 6.46: *Overview of the boundary data editor*

Process and quantity selection:

Currently, D-Flow FM supports the processes flow, salinity, temperature and tracers (**Note: tracers are not yet fully editable**). The processes available in the boundary condition editor depend on the selected processes in the processes tab (see **??**). After selecting the process from the dropdown box the user can select one of the corresponding quantities, as illustrated in Figure 6.47. For the process flow the user can choose from five principal quantities: water level, velocity, Riemann invariant, Neuman gradient and discharge. All quantities are specified per support point, except for discharges which are specified per polyline (<∗.pli>). A support point can have multiple forcing quantities of the same type (i.e. water level, velocity, Riemann, Neumann or discharge). These quantities are added up. This can be relevant to add a surge level to an astronomical water level for example. Furthermore, the user can apply normal and tangential velocities as 'add on' quantities. The following combinations of quantities are allowed:

- ◇ Water level + normal velocity
- ◇ Water level + tangential velocity
- ◇ Water level + normal velocity + tangential velocity
- ◇ Velocity (= normal) + tangential velocity
- ◇ Riemann + tangential velocity

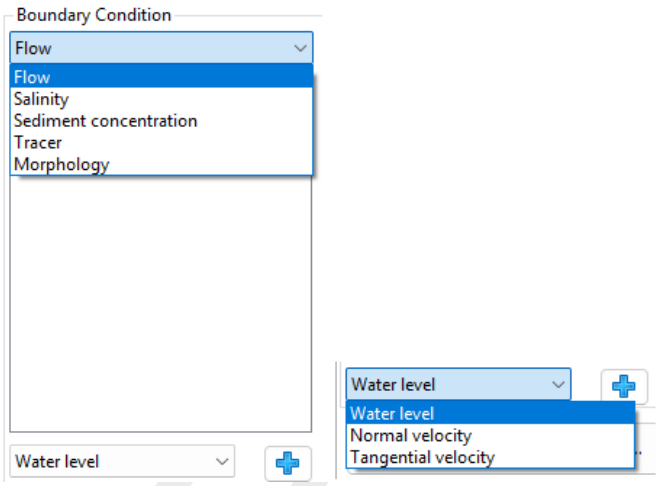


Figure 6.47: *Process and quantity selection in the boundary data editor*

Forcing function selection:

The user can select one of the following forcing functions:

- ◇ Time series
- ◇ Harmonic components
- ◇ Harmonic components + correction
- ◇ Astronomic components
- ◇ Astronomic components + correction
- ◇ Q-h relation (only for water levels)

The next section describes how data can be added for these types of forcing functions.

Add forcing data to a support point or polyline:

By default all support points on the polyline are deactivated. To add forcing data to a support point the user first needs to activate it by pressing the green 'add'-symbol in the list of support points (see Figure 6.48). Consequently, the user can specify the forcing data in the lower left panel based on the selected forcing function. Of which a preview is shown in the lower right panel. **Note: Please note that once a support point containing forcing data is made inactive, all data on the support point is lost!**

The user can choose from the forcing functions time series, harmonic components (+ correction), astronomic components (+ correction) and Q-h relation. Examples of the boundary data specifications for these different forcing functions are given below. **Note: Please note that once the user changes the forcing function of a polyline, all data on the polyline is lost!**

DRAFT

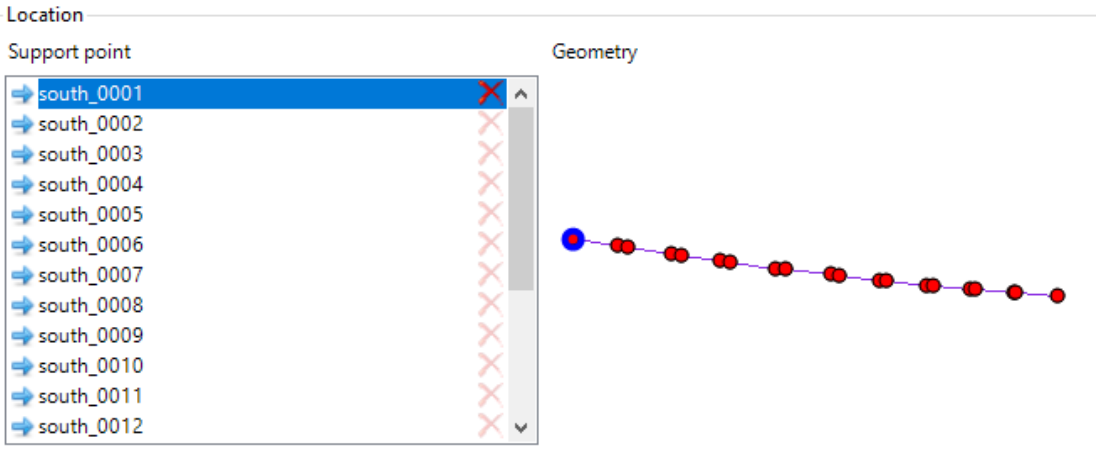


Figure 6.48: *Activate a support point*

Time series

Note: Time series have an attribute `Time zone`, as can be seen below.

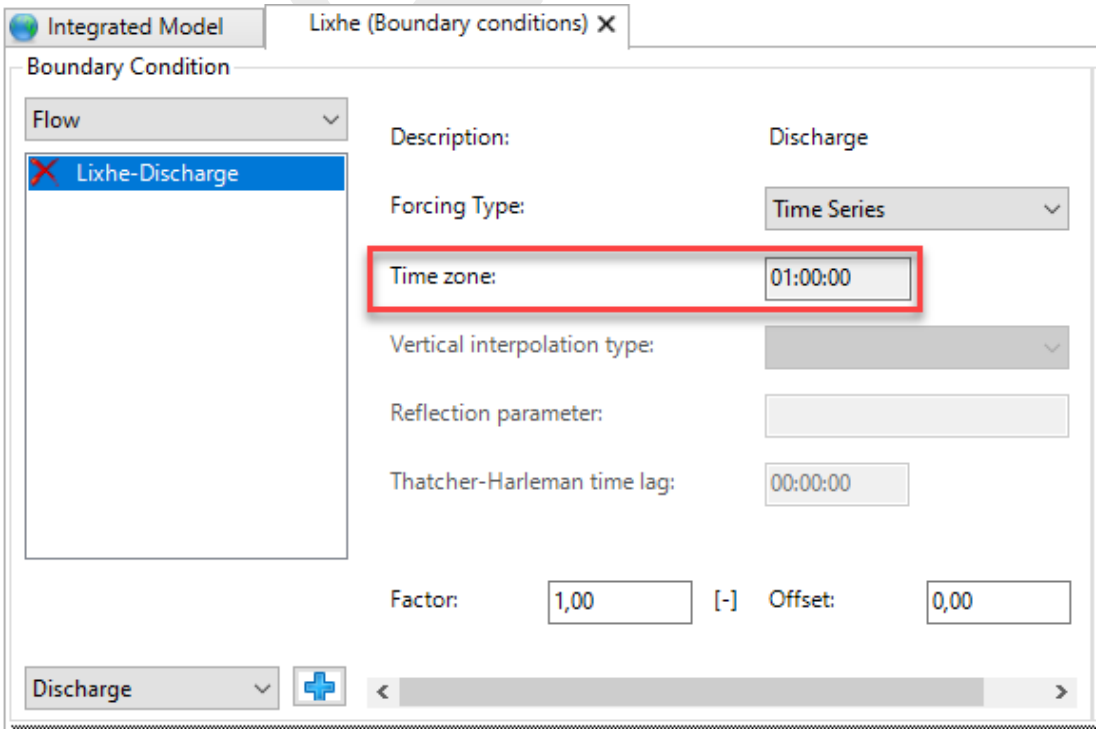


Figure 6.49: *Time zone in the Boundary conditions editor*

This attribute is un-editable, as it belongs to the data. The `Time zone` is defined as the time difference between local time and UTC. By default the `Time zone` is `0`. In case external data is used, the `Time zone` can be different - as in Figure 6.49. The `Time zone` will be used to transpose the (external) data to the `Time zone` of the time frame of the simulation (see also section 6.4.5).

The time format for time series is yyyy-mm-dd HH:MM:SS.

There are multiple ways to specify time series for the selected quantity:

◇ Specify the time series step by step in the table (Figure 6.50): the user can add or delete rows with the “plus”- and “minus”-signs below the table.

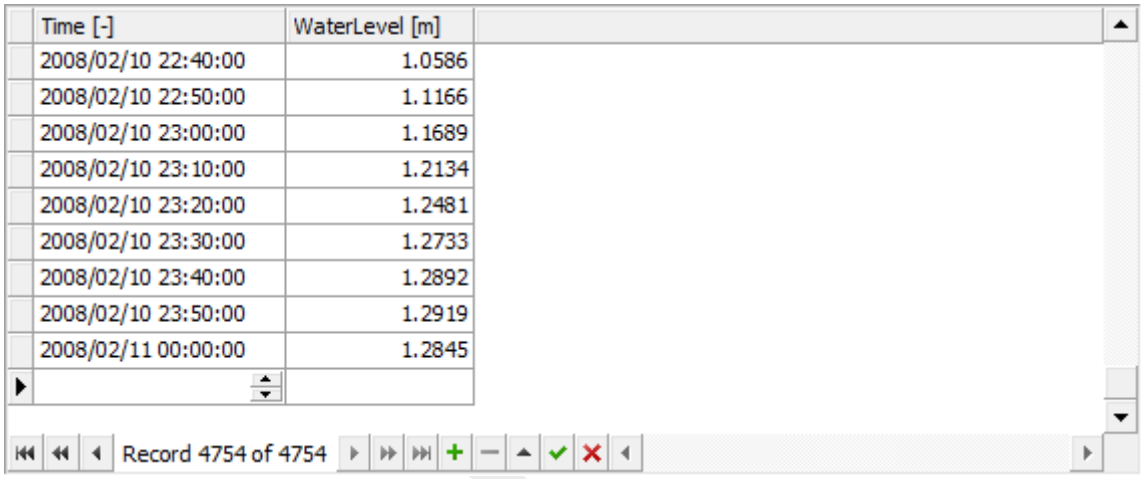


Figure 6.50: *Specification of time series in the boundary data editor (left panel)*

◇ Generate time series using the ‘Generate time series’ button (Figure 6.51): the user can specify start time, stop time and time step.

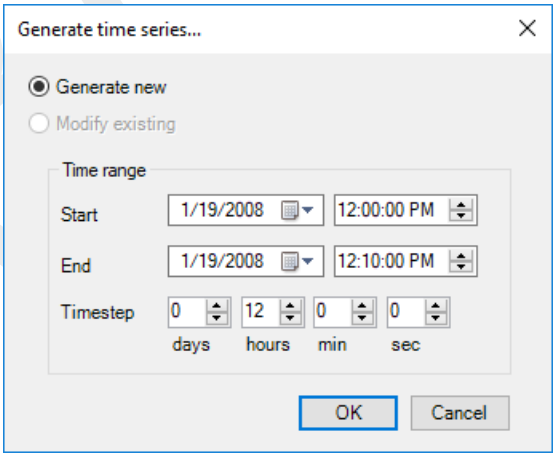


Figure 6.51: *Window for generating series of time points*

◇ Import from csv using the ‘Csv import’ button: a wizard will open in which the user can (1) select a csv-file (Figure 6.52), (2) specify how data should be parsed into columns (Figure 6.53) and (3) how the values should be parsed and mapped into columns (Figure 6.54).

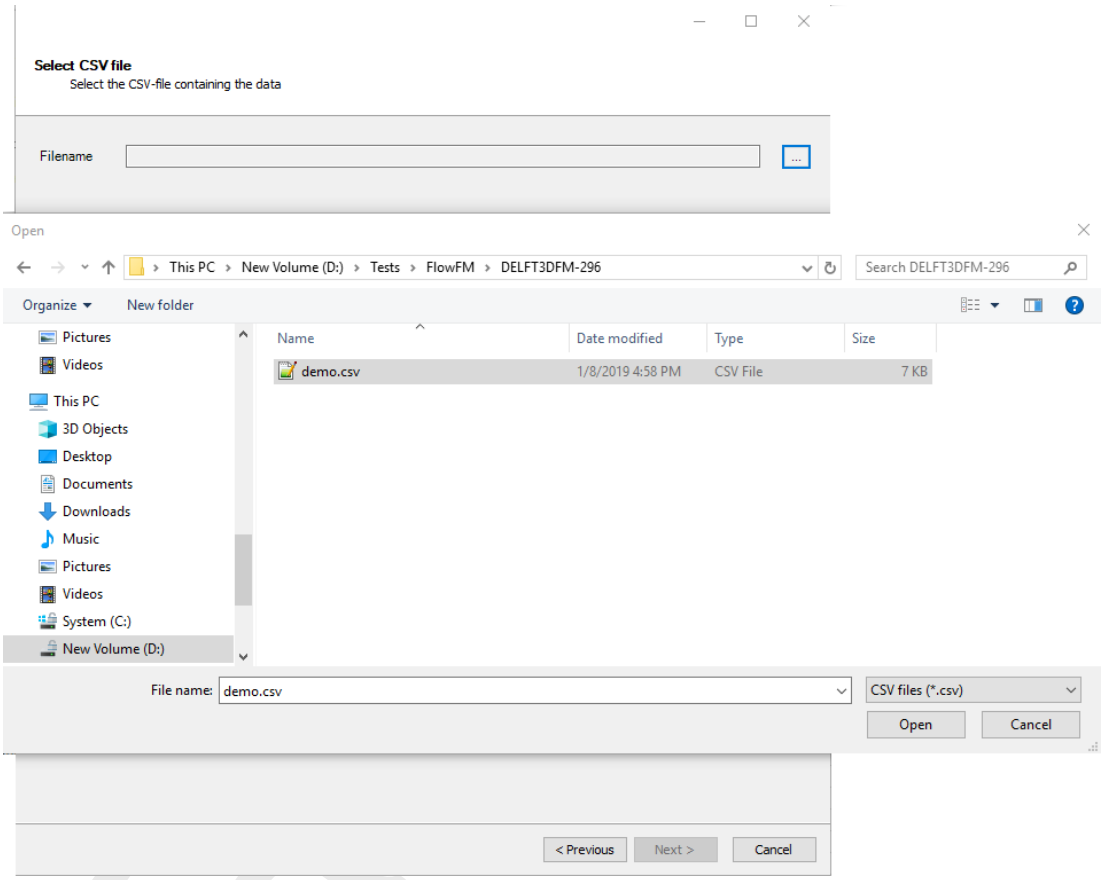


Figure 6.52: *Csv import wizard: csv file selection*

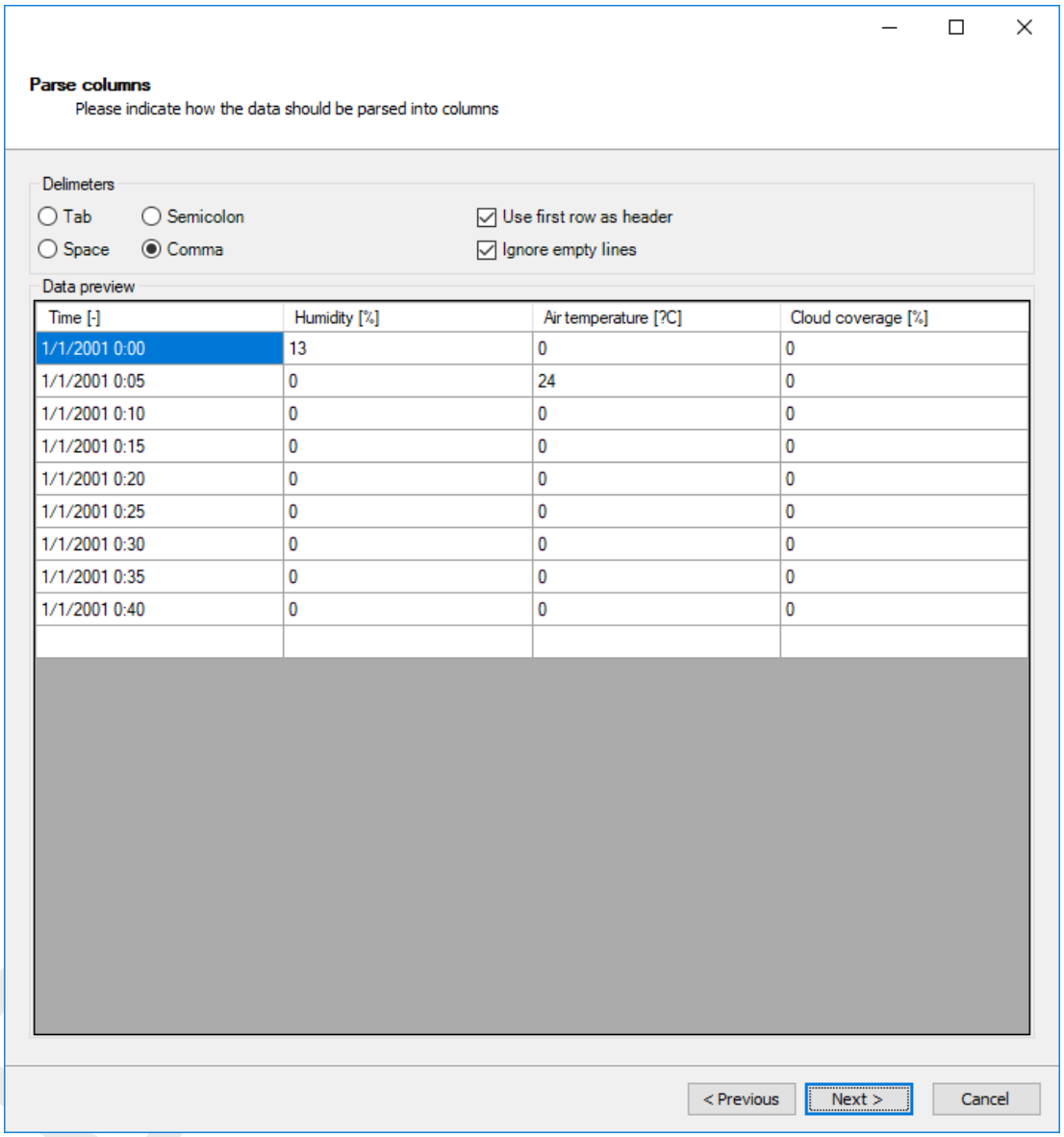


Figure 6.53: *Clipboard/csv import wizard: specification of how data should be parsed into columns*

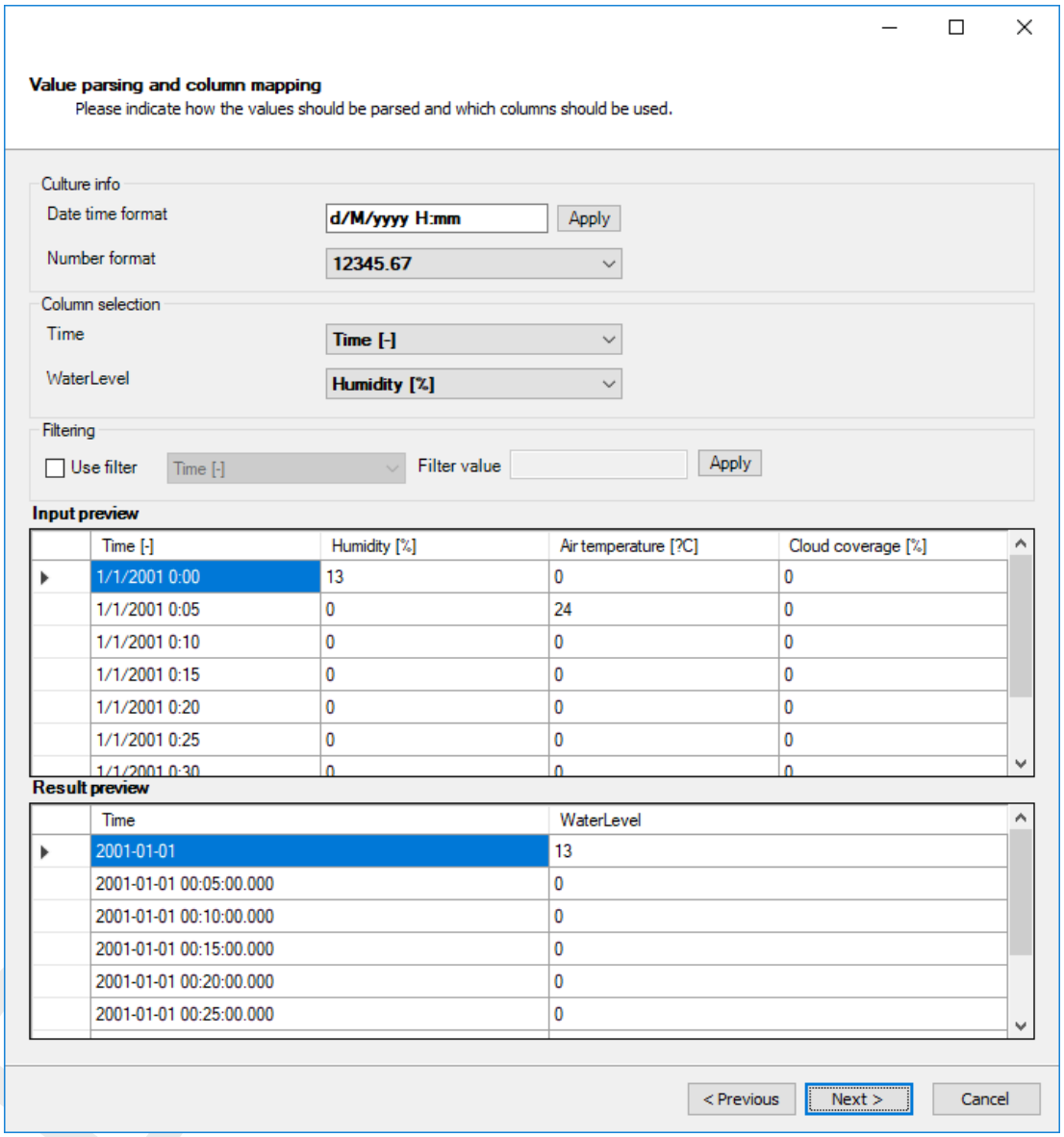


Figure 6.54: *Clipboard/csv import wizard: specification of how values should be parsed and columns should be mapped*

- ◇ Import from clipboard using the 'Clipboard import' button: a wizard will open in which the user can specify (1) how data should be parsed into columns (Figure 6.53) and (2) how the values should be parsed and mapped into columns (Figure 6.54).
- ◇ Import from attribute file <∗.bc>: with this button the user can import data from existing boundary conditions <∗.bc> file. The user has three options for importing:
  - Overwrite where matching (replace): only overwrites the forcing data for matching support points in the <∗.bc>-file and GUI input.
  - Overwrite where missing (extend): only overwrites the forcing data for matching support points in the <∗.bc>-file and in the GUI input that did not contain data.
  - Overwrite all: overwrites the forcing data for all support points in the GUI (meaning that the forcing data for non-matching support points is emptied).

Harmonic components

The harmonic components are defined by a frequency, amplitude and phase (see Figure 6.55). By default the forcing data viewer shows the harmonic component for the time frame specified for the model simulation. The options to define the harmonic components are similar to the options for time series:

- ◇ Specify components step by step: the user can add or delete rows with the “plus”- and “minus”-signs below the table.
- ◇ Select (astronomical) components using the ‘Select components’ button (Figure 6.56): the user can select astronomical components which will be transformed in the corresponding frequencies.
- ◇ Import from csv using the ‘Csv import’ button: a wizard will open in which the user can (1) select a csv-file, (2) specify how data should be parsed into columns and (3) how the values should be parsed and mapped into columns.
- ◇ Import from clipboard using the ‘Clipboard import’ button: a wizard will open in which the user can specify (1) how data should be parsed into columns and (2) how the values should be parsed and mapped into columns.
- ◇ Import from attribute file <∗.bc>: with this button the user can import data from existing boundary conditions <∗.bc> file. The user has three options for importing:
  - ◻ Overwrite where matching (replace): only overwrites the forcing data for matching support points in the bc-file and GUI input.
  - ◻ Overwrite where missing (extend): only overwrites the forcing data for matching support points in the bc-file and in the GUI input that did not contain data.
  - ◻ Overwrite all: overwrites the forcing data for all support points in the GUI (meaning that the forcing data for non-matching support points is emptied).

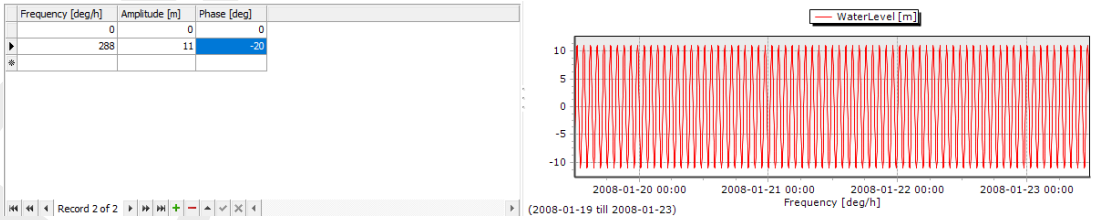


Figure 6.55: *Specification of harmonic components in boundary data editor*

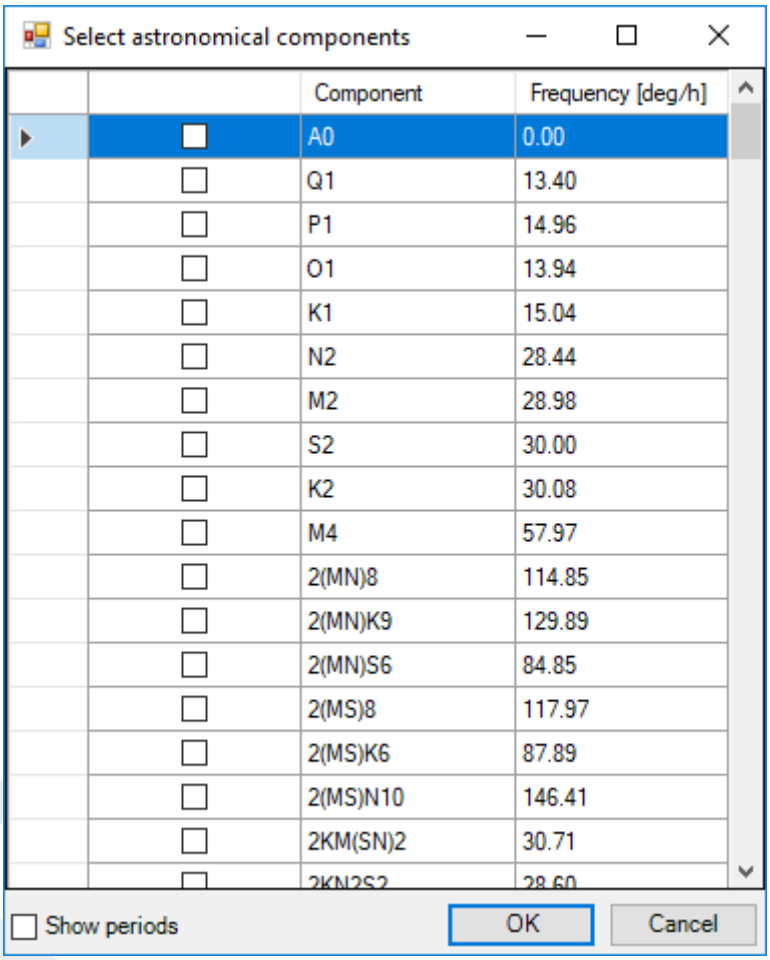


Figure 6.56: *Selection of astronomical components from list (after pressing 'select components')*

Astronomic components

Astronomical components are similar to harmonic components, with the exception that the frequency is prescribed. Instead of specifying the frequency the user can select astronomical components by name. Upon editing the component field in the table the user will get suggestions for components in a list (Figure 6.57). The most frequently used components (A0, Q1, P1, O1, K1, N2, M2, S2, K2 and M4) are put on top of the list, the other components are listed in alphabetic order. Instead of defining each component individually, the user can also make a selection of components by pressing the button 'select components' (Figure 6.56). For a complete overview of all 235 astronomic components we refer to chapter J.

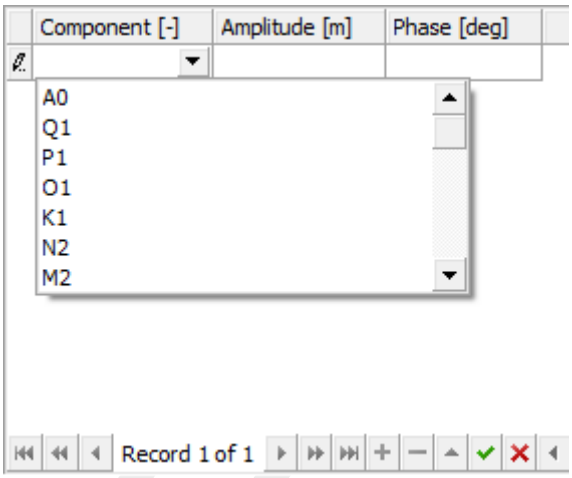


Figure 6.57: *Suggestions for astronomical components in list*

Harmonic or astronomic components with corrections

For calibration purposes the user can combine harmonic or astronomic components with corrections. The corrections are defined in terms of an amplitude (multiplication) factor and a phase difference (see Figure 6.58). This allows the user to keep track of both the original signal and the calibration coefficients. The effects of the corrections on the resulting signal are directly visualized in the forcing data viewer. **Note: Please note that the import functionality is not (yet) working properly for astronomic/harmonic boundary conditions with corrections.**

Component [-]	Amplitude [m]	Phase [deg]	Amplitude corr.	Phase corr. [deg]
A0	0.2	0	2	15
M2	1	55	5	30

Figure 6.58: *Editing harmonic/astronomic components and their corrections*

Q-h relation (only for water level)

The user can force the boundary with a Q-h relationship, but only for the quantity water level. This is a relationship between discharge and water level (see Figure 6.59). The relationship can only be prescribed per polyline, not per support point. (**Note: this functionality has not been tested extensively yet**)

DRAFT

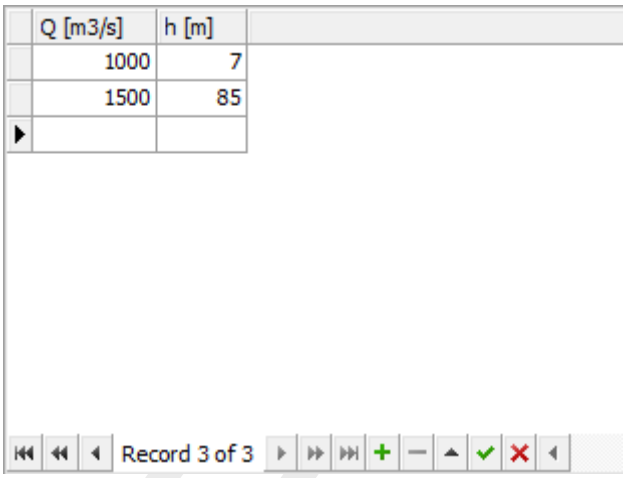


Figure 6.59: *Specification of a Q-h relationship*

Exporting boundary conditions

With the *Export to files* button the user can export all boundary forcing data for the given polyline to a $<*.bc>$-file.

View boundary data

All boundary data of the same quantity on a support point can be (pre-)viewed in the boundary data view in the lower-right panel. If multiple signals of the same process have been entered, the viewer will show the active signal in red and the total signal of all datasets in green (see Figure 6.60). In the view the user can zoom-in by dragging a box from top-left to bottom-right and zoom-out by dragging a box from bottom-right to top-left.

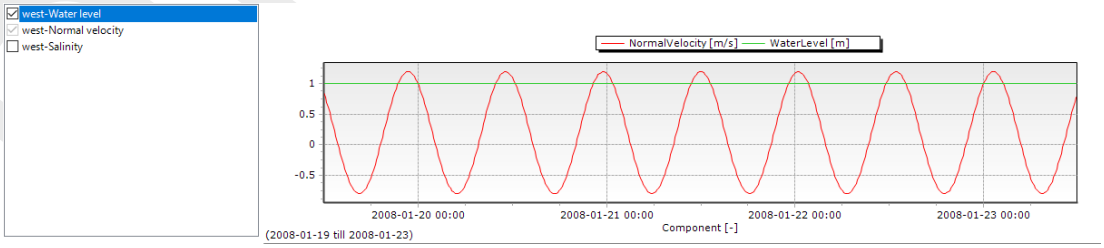


Figure 6.60: *Example of active and total signal for multiple water level data series on one support point*

To inspect multiple quantities at a support point at the same time (for example water level and normal velocity) the user can use the combined boundary data viewer by pressing the button ‘Combined BC view’. **Note: This does not work properly yet.**

6.4.7.3 Import/export boundary conditions from the *Project* window

Apart from import and export functionality per individual boundary polyline in the boundary condition editor, the GUI offers the opportunity to import and export boundary locations (<∗.pli>) and forcing (<∗.bc>) on a higher level. Hereto, you have to click the right mouse button on “Boundary Conditions” in the *Project* window and select “Import” or “Export” (Figure 6.61). Imports and exports on “Boundary Conditions” apply to all the boundary conditions whereas import and exports on a boundary polyline apply only to that boundary condition.

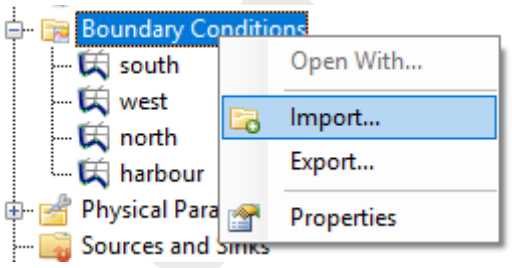


Figure 6.61: *Importing or exporting boundary features — both polylines <∗.pli> and forcing <∗.bc> — from the* Project *window using the right mouse button*

Import and export polylines

Upon importing a <∗.pli>-file with the same filename and the same polyline name(s) as the existing polyline names in the GUI, the existing polyline(s) will be replaced and all forcing data thereon will be deleted. Upon importing a polyline(s) with a different name(s), the polyline(s) will be added to the *Project* window without any forcing data on it/them. The user will be asked to import the data “as is” or to perform a coordinate transformation before the import (see Figure 6.62).

Alternatively, the user can exported created polylines to a <∗.pli>-file. Upon export the user will be asked to export the data “as is” or to perform a coordinate transformation before the export (see Figure 6.62).

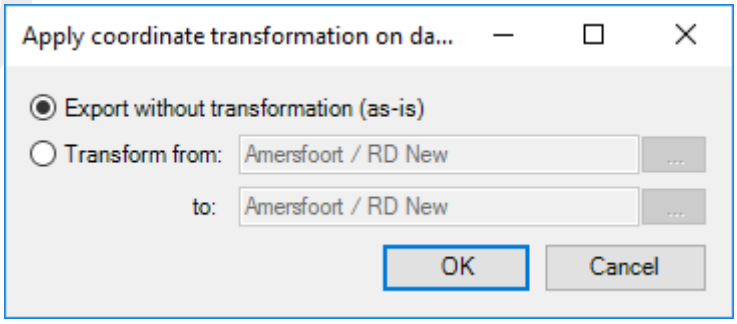


Figure 6.62: *Import or export a <∗.pli>-file as is or with coordinate transformation.*

DRAFT

Import and export boundary forcing data

Similar to the polylines, you can import and export boundary forcing data from/to a <∗.bc>-file. To import forcing data the existence of a polyline with at least one matching support point is a prerequisite. Upon importing <∗.bc> data you can select which quantities and forcing types from the <∗.bc>-file should be imported and with which overrwrite options (see Figure 6.63). Similarly, you can export boundary forcing data. As an additional exporting feature you can select whether you would like to export: 1) all forcing data into one file, 2) as separate files per boundary, 3) as separate files per process or 4) as separate files per quantity (see Figure 6.64).

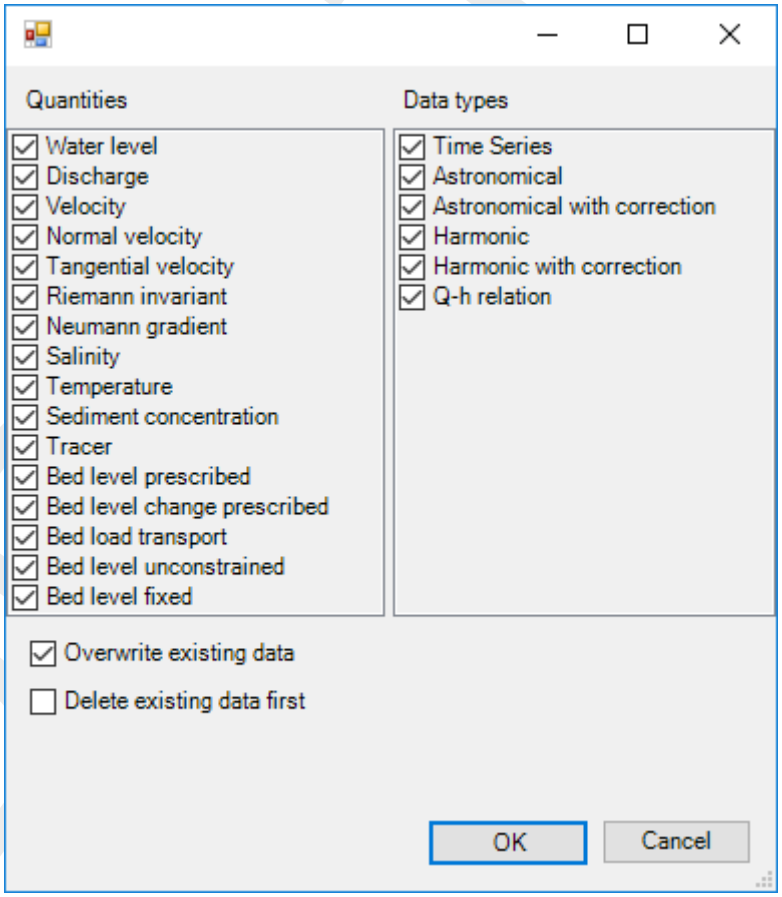


Figure 6.63: *Import or export a <∗.pli>-file as is or with coordinate transformation.*

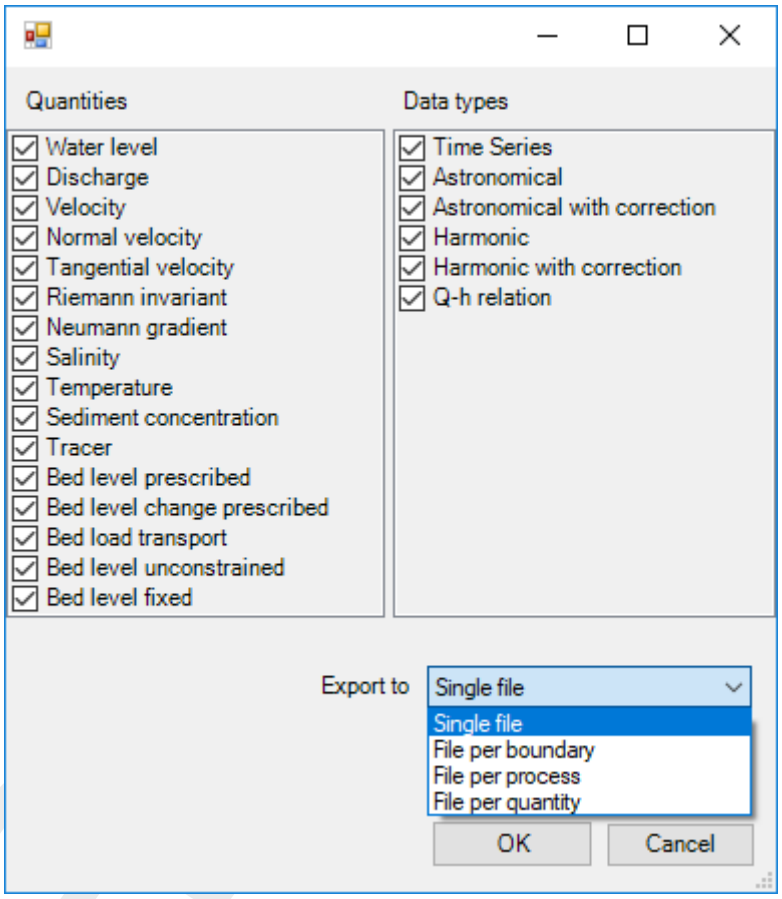


Figure 6.64: *Import or export a *.pli file as is or with coordinate transformation.*

6.4.7.4 Overview of boundary conditions in attribute table (non-editable)

The attribute table of the boundary conditions gives an overview of all specified boundary polylines and corresponding forcing (see Figure 6.65). This attribute table can be opened by double clicking 'Boundary Conditions' from the project or *Map* window. Most of the features in the attribute table are non-editable, except for the optional (multiplication) factor and offset per quantity per polyline. With these settings you can integrally multiply all data on a polyline with a factor and/or add an offset to it.

Boundary Conditions

Boundary	Quantity	Forcing type	Factor	Offset
south	WaterLevel	Time Series	1	0
south	Salinity	Time Series	1	0
west	WaterLevel	Time Series	1	0
west	NormalVelocity	Astronomical	1	0
west	WaterLevel	Time Series	1	0
west	Salinity	Time Series	1	0
north	WaterLevel	Q-h relation	1	0
north	Salinity	Time Series	1	0
harbour	Salinity	Time Series	1	0
harbour	Discharge	Time Series	1	0

Figure 6.65: *Overview of all boundary conditions in attribute table*

6.4.7.5 Open Boundaries and grid snapping

In this section grid snapping for open boundaries will be explained. For a description how to specify open boundaries we refer to section 6.4.7. A polyline for an open boundary segment should be sufficiently closed to the end of the grid. The polyline should be within a distance of three grid cells from the end of the grid, as illustrated in Figure 6.66. In this example can be seen that the upper part of the polyline is sufficiently close to the end of the grid, while the lower part isn't. This can be checked visually through the prencence of dots on the last grid line, coinciding with the end of the computational grid.

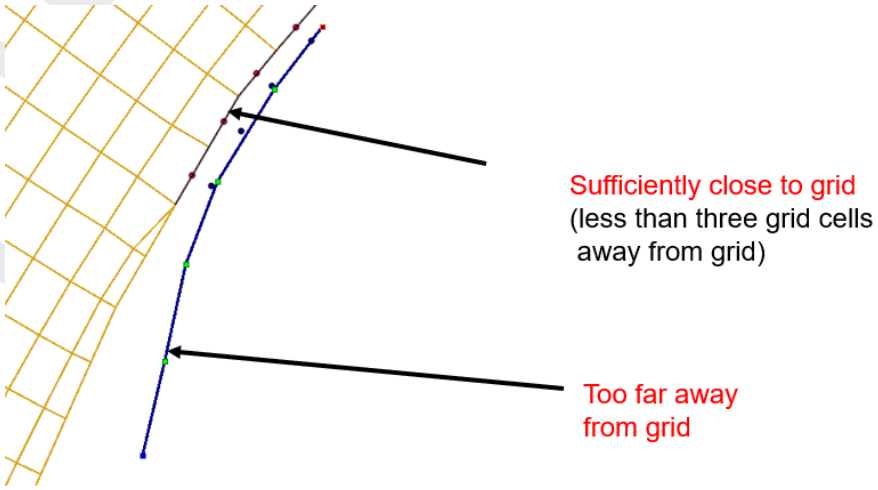


Figure 6.66: *Illustration of grid snapping for open boundaries*

DRAFT

6.4.8 Physical parameters

The physical parameters attribute is used to set all physical parameters of your model. When it is expanded in the *Project* window, it shows six attributes: *Roughness*, *Viscosity*, *Diffusivity*, *Infiltration*, *Meteo items* and *Wind*.

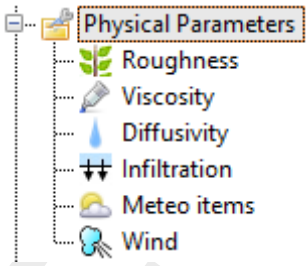


Figure 6.67: *The physical parameters in the* ***Project*** *window*

6.4.8.1 Constants

In the *Physical Parameters* tab, the user can adjust the value of *Gravity* [m/s^2] under *Others* and *Default water density* [kg/m^3] under *Density*.

6.4.8.2 Roughness

Bed roughness can be specified as a uniform value or as a coverage (e.g. a spatially varying field). The uniform values as well as the roughness formulation (i.e. Chézy, Manning, White-ColeBrook or Z_0) can be edited in the 'Physical Parameters' tab, which opens upon double clicking 'Roughness' in the *Project* window (Figure 6.68). **Note:** The latter is not yet working. In this tab you can also specify the linear friction coefficient, linear friction Umod, wall behaviour (free slip or partial slip) and wall ks for partial slip.

In case of spatially varying bed roughness you can double click the quantity in the *Project* window or select it from the dropdown box in the *Spatial Operations* ribbon (Figure 6.69). Then the spatial editor is activated, which you can use to edit spatially varying fields. For more information on how to use the spatial editor you are referred to chapter H.

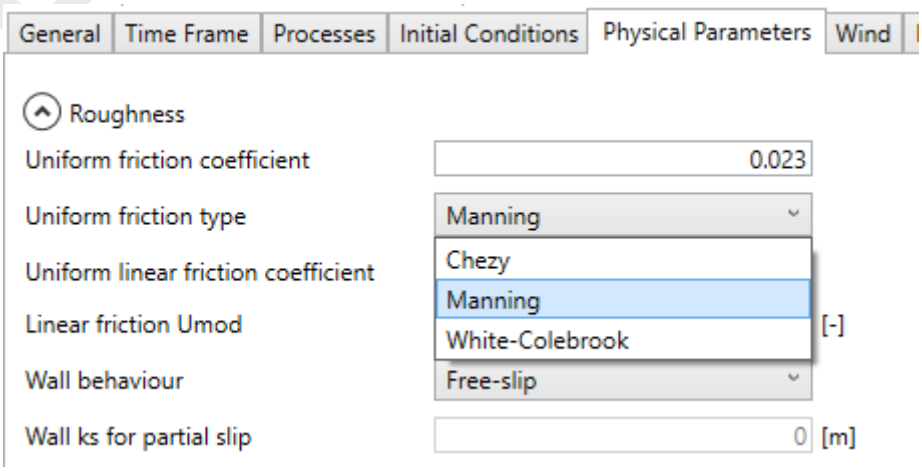


Figure 6.68: *The section of the 'Physical Parameters' tab where you can specify roughness related parameters and formulations.*

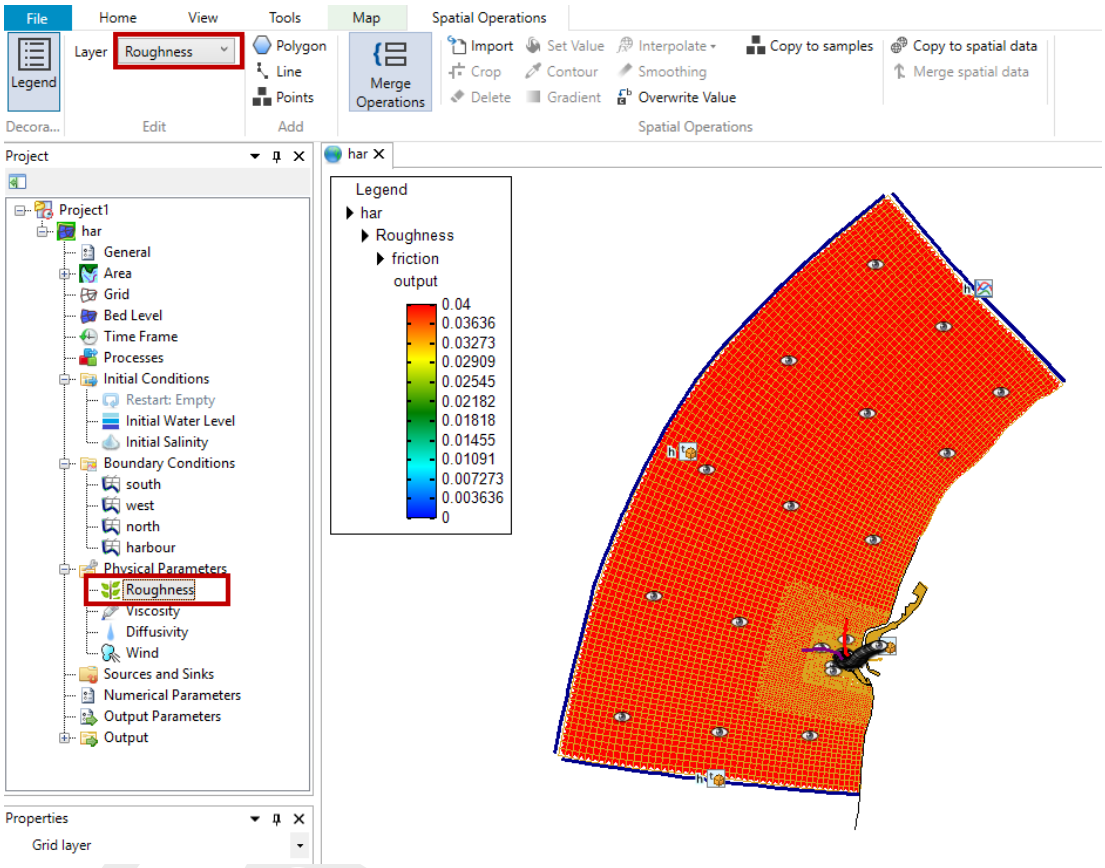


Figure 6.69: *Roughness activated in the spatial editor to create/edit a spatially varying field*

6.4.8.3 Viscosity

The eddy viscosity can be specified as a uniform value or as a coverage (e.g. a spatially varying field). In the 'Physical parameters' tab you can specify the uniform values for the horizontal and vertical (in case of 3D simulations) eddy viscosity and diffusivity (Figure 6.70).

In case of spatially varying viscosity you can double click the quantity in the *Project* window or select it from the dropdown box in the *Spatial Operations* ribbon (Figure 6.71). Then the spatial editor is activated, which you can use to edit spatially varying fields. For more information on how to use the spatial editor you are referred to chapter H.

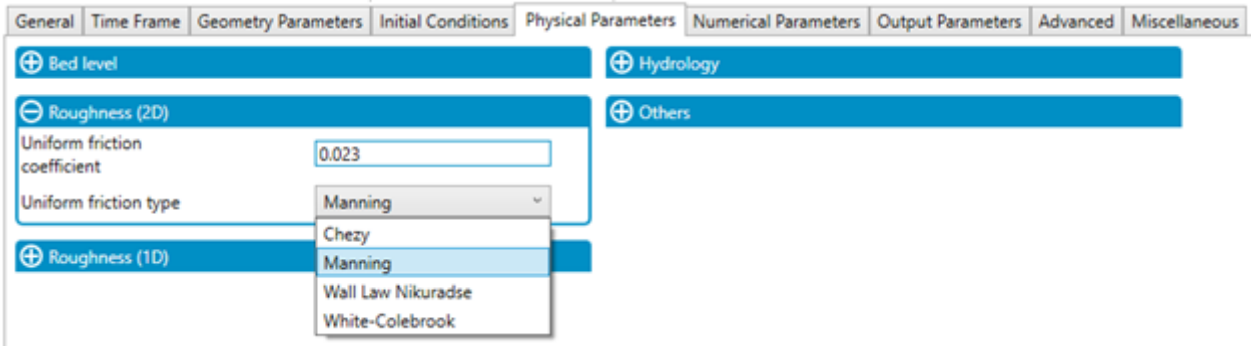


Figure 6.70: *The section of the'Physical Parameters' tab where you can specify (uniform) values for the horizontal and vertical eddy viscosity and diffusivity.*

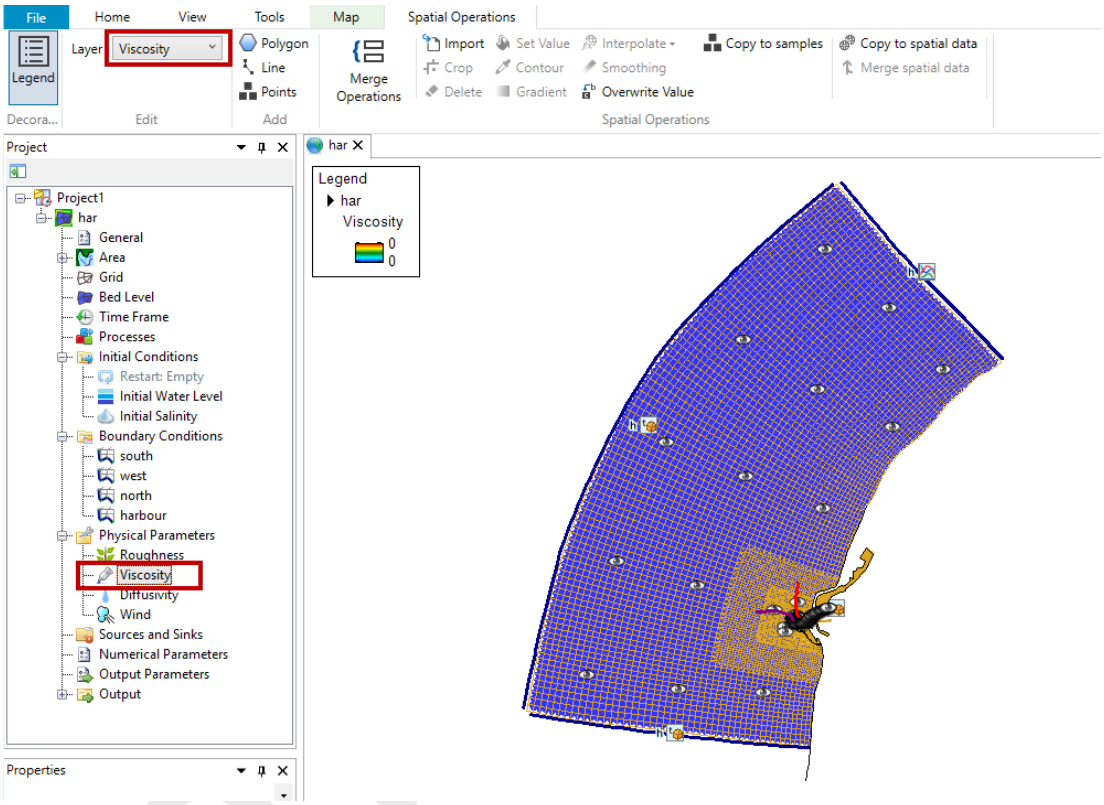


Figure 6.71: *Viscosity activated in the spatial editor to create/edit a spatially varying field*

6.4.8.4 Diffusivity

Description will be added ...

6.4.8.5 Infiltration

Description will be added ...

6.4.8.6 Meteo items

Description will be added ...

6.4.8.7 Wind

All relevant parameters, described in chapter 14 can be adjusted in the following sub-tab.

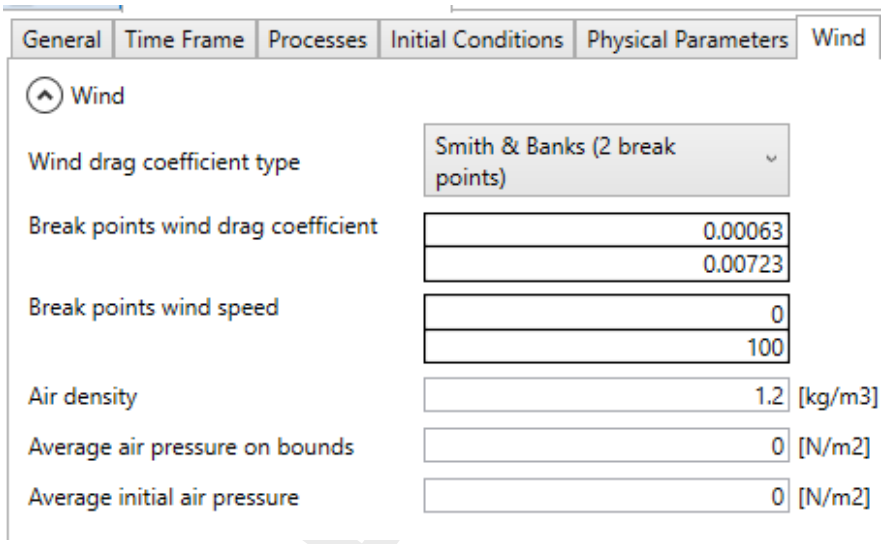


Figure 6.72: *Overview of parameters in sub-tab Wind*

6.4.9 Sources and sinks

Sources and sinks (or: intake/outfall facilities) can be used to add/extract a discharge to/from the model or to redistribute water and constituents (such as temperature and salinity) within the model. Sources and sinks consists of a location (defined by a $<*$.pli$>$-file) and time series describing the discharges (defined by a $<*$.bc$>$-file). All the hydrodynamical considerations behind sources and sinks are discussed in section 6.4.10.

Sources and sinks locations

Sources and sinks can be added to the model using the corresponding icon from the *Map* ribbon (Figure 6.73). When the sources/sinks icon is active you can add them as polyline elements in the central map using the left mouse button. Each polyline element is closed by double clicking the left mouse button.

Note: Please note that the length of the polyline elements is not taken into account in the handling of sources and sinks (e.g. it is modeled as an instant redistribution of water and constituents without delays and friction losses).

Polyline elements starting outside the model domain and going inward are sources and, vice versa, polyline elements starting inside the model domain and going outward are sinks. Polyline elements starting and ending within the model are intake/outfall type of discharges. The drawing direction determines the direction of the discharge indicated by an arrow (Figure 6.74). In case of a source the direction of the last polyline element determines the direction of flow momentum into the model.

Note: The 'reverse' line option — to switch the discharge direction without having to redraw the source/sink — is not implemented.

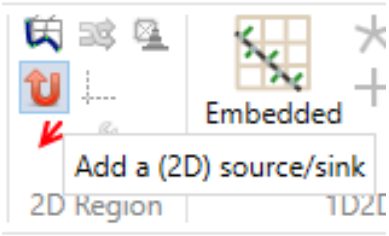


Figure 6.73: *Activate the sources and sinks editing icon in the* Map *ribbon*

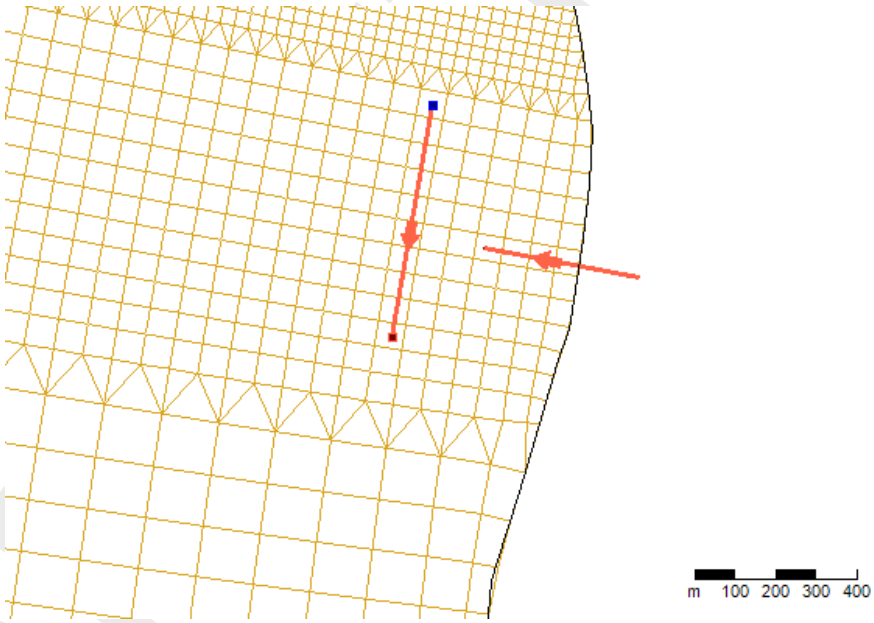


Figure 6.74: *Add sources and sinks in the central map using the 'Sources and sinks' icon.*

Sources and sinks time series

After drawing the sources/sinks locations in the central map, they appear under 'Sources and sinks' in the *Project* window (Figure 6.75). Either by double clicking the sources/sinks in the *Project* window or the line element in the central map, you can open the sources and sinks editor (Figure 6.76). In this editor you can specify the discharge time series as well as the corresponding constituents (for example salinity and temperature, depending on the active physical processes). The time series of water discharges are always defined as absolute values. For the time series of constituents the following applies:

- ◇ For sources the constituent time series are absolute values
- ◇ For sinks the constituent time series are determined by the modeled values
- ◇ For sources and sinks (intake/outfall relationships) the constituent time series are the excess values (e.g. on top of the modeled values)

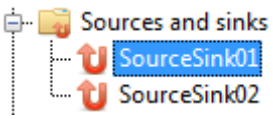


Figure 6.75: *Sources and sinks appearing in the* Project *window*

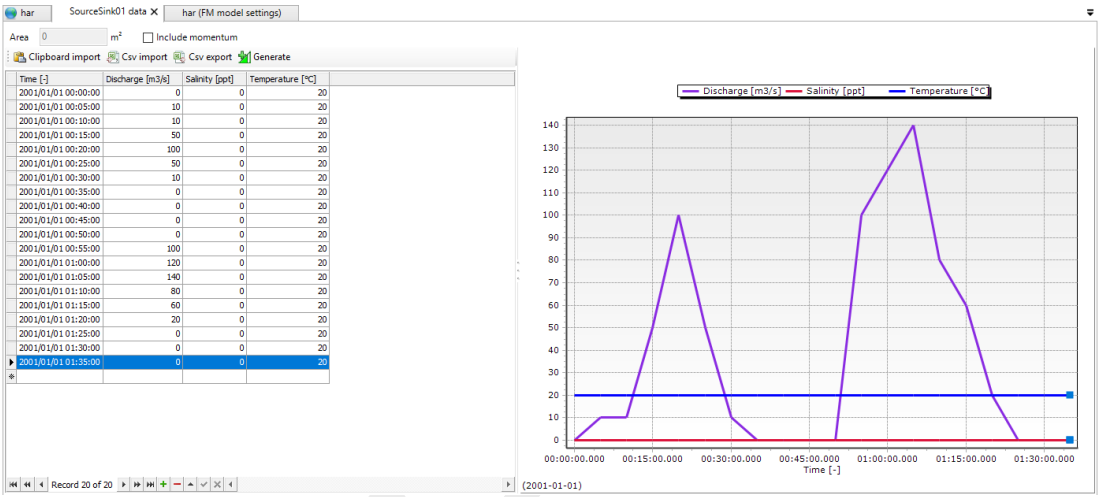


Figure 6.76: *Specifying time series for sources and sinks in the sources and sinks editor*

6.4.10 Intakes, outfalls and coupled intake-outfalls

Many engineering studies concern the design of intakes and outfalls. For instance, studies about positioning of waste water diffusers or coupled intake-outfall design for cooling water at power plants. Intakes and outfalls can be modeled using sinks and sources. A source is a point in the model where a discharge Q in [m^3/s] is prescribed by a time series ASCII file (section C.4) with at least two columns and, depending on the model settings, possibly more. For each optional constituent that is modelled, an extra column is required. Best known constituents are salinity and temperature and the values in the time series file should be specified as salinity increase in [ppt] and temperature increase in [$^\circ$C].

1 When salinity and temperature are not used in the computation:
Model expects two columns, where the first column is the time in minutes, the second is the discharge in [m^3/s].
2 When either salinity or temperature is used in the computation:
Model expects three columns, where the first column is the time in minutes, the second is the discharge in [m^3/s], the third column contains either the salinity (increase) in [ppt] or the temperature (increase) in [$^\circ$C]. Note that third column can be either salinity or temperature depending which constituent is used in the model and which is not.
3 In general, when other constituents are also modelled (e.g. one or more sediment fractions, passive tracers), the column order is as follows: Time, Discharge, Salinity?, Temperature?, (Sed.frac 1, . . ., Sed.frac n)?, Spiral flow intensity?, (Tracer 1, . . ., Tracer n)?.

The location of the source or sink is specified in a polyline file (section C.2), containing a polyline with either multiple points or just one point. If two or more points are specified, a coupled source sink pair is made, the first point is the FROM (or sink) point, the last point is the TO (or source) point. Three variants may occur:

1 Sink point side lies inside a grid cell A, source point also lies inside a grid cell B (A may equal B, but that is rarely useful): water is extracted from cell A and transported to B.
2 Sink point lies outside of the grid, source point lies in a grid cell B: water is discharged into B, a bit like an inflow discharge boundary condition.
3 Sink side lies inside a grid cell A, source side lies outside of the grid: water is extracted from A, a bit like an outflow discharge boundary condition.

Specifying a negative discharge value effectively interchanges the role of the source and sink

points. If only one point is in the $<*$.pli$>$ file, it is assumed to be a source point. Specifying a negative discharge turns the source into a sink.

For 3D computations, the polyline file should have a third column with z-values. It is good practice to change the $<*$.pli$>$ file into a $<*$.pliz$>$ file. The z-values are used to determine in which vertical grid cell the source and/or sink lie. The layer number can vary in time in sigma models.

In the case of a coupled pair of sink source points (variant 1), the third and fourth column with the salinity and temperature specification are interpreted as delta salinity and delta temperature. So the values at the source point become the values at the sink point plus the specified delta values.

The sources and sinks are specified in the new $<*$.ext$>$ as follows:

```
[SourceSink]
id            = SourceSink_1
locationFile  = chan1_westeast.pli
discharge     = chan1_westeast.bc
salinityDelta = chan1_westeast.bc
...
area          = 1.5
```

Warning:

- ◇ Deprecated: Source-sinks could also be specified in the old $<$*.ext$>$ file. This will become obsolete in an upcoming release and the example below is only kept for migration purposes.

In the old $<$*.ext$>$ file, the above example would look like:

```
QUANTITY=discharge_salinity_temperature_sorsin
FILENAME=chan1_westeast.pli  # A file 'chan1_westeast.tim', with same basename
                             # as the polyline should be present
FILETYPE=9
METHOD  =1
OPERAND =O
AREA    =1.5
```

The specified area in the last line of this file determines (together with the specified discharge) the amount of momentum uQ that is released at the source point. This is currently only implemented in advection scheme 33 (cf. Deltares (2026a) on Momentum advection). Omitting this line or specifying a zero area switches off the release of momentum at the source point. The direction of the discharged momentum is in direction of the last two points of the polyline. So a one point polyline is momentumless. Both in 2D and 3D, the momentum can only be directed in horizontal direction.

Sources and sinks are treated explicity in the numerical scheme. This implies that the actual discharged or extracted amounts of water are limited by the velocity Courant condition $Q\Delta t/V < 1$. Not doing so could lead to severe timestep restrictions. Consider for instance a specified extraction in case the extraction point has fallen dry. In that case, a real pump would not be able to extract water and the specified extraction is more likely an input error than an actual description of a physically feasible situation. So, we limit the specified discharges to the local velocity Courant restriction. By specifying observation cross-sections

(section 6.4.2.3), one can compare prescribed discharges to the discharges that were actually realised in the model. In case of large differences, the discharge should probably be distributed over a larger number of gridcells, or the extraction channel that feeds the extraction point should be dredged.

When modelling freshwater inflow from a river into a sea, the vertical distribution of the discharge that one should specify depends on the amount of detail available for modelling the saline water - fresh water interface. If the river is modeled with sufficient detail, and the river is well mixed at the upstream model boundary, the mixing process can be part of the modelling and the river can be discharged over the whole vertical. If the mixing process cannot be resolved in the model, for instance if the river is modeled as a point discharge adjacent to a closed boundary, make sure that the whole river is discharged into the top layer or in the top layers, depending on the estimated thickness of the fresh water plume at the discharge cell. If one would have distributed the river discharge over the whole vertical, the size of the fresh water plume would be underestimated because of too much mixing at the river discharge cell.

6.4.11 Lateral discharges

Lateral discharges prescribe additional inflow (or outflow) into the system on specific locations in the model. Lateral discharges are merely adding (or extracting) volume to the model and no momentum. As such, they differ from discharge boundaries (section 9.1.1.1) and source-sinks (section 6.4.10), which *can* add momentum as well. The input of lateral discharges is via the external forcings file (section C.6.2.2); the following subsections describe the grid placement and the forcing information.

6.4.11.1 Grid snapping and distribution of discharge

The spatial location of lateral discharges can be a single point or a set of grid points, and both 1D and 2D points are supported (also combined). The grid snapping depends on the type of input:

- ◇ Single point input via a 1D network node id, or branch id/chainage. The given location is snapped to the nearest 1D pressure point.
- ◇ Single point input via a single x, y-point. The given location is snapped to the grid cell (1D or 2D) in which that x, y-point lies.
- ◇ Polygon area input via a sequence of x, y-points. The given area selects all pressure points that lie inside this polygon. If the input contains multiple polygons, it is sufficient for pressure points to lie in at least one of these polygons. Note that no "nesting" of polygon rings is detected, polygons can not be used to *exclude* regions from the lateral discharge.

The latter two types also allow the user to limit the grid point snapping to either 1D or 2D, via the keyword `locationType`, but this is optional.

When more than a single grid point (cell) is selected (only possible for polygon input), the prescribed lateral discharge $Q_{\mathrm{lat},\ell}$ is distributed across all selected grid cells based on the grid cell areas. Specifically, each grid cell k in the set $\mathcal{L}_{\mathrm{lat},\ell}$ of selected grid cells receives $Q_{\mathrm{lat},\ell,k}$, defined by:

$$Q_{\mathrm{lat},\ell,k} = \frac{Q_{\mathrm{lat},\ell}}{A_{\mathrm{lat,tot},\ell}} A_k, \tag{6.1}$$

where the total area affected by this lateral is defined as:

$$A_{\mathrm{lat,tot},\ell} = \sum_{k \in \mathcal{L}_{\mathrm{lat},\ell}} A_k. \tag{6.2}$$

This area-weighting ensures that the rate of change for the water levels in all selected grid cells is the same, as far as the forcing by this lateral discharge is concerned. Thus, a lateral discharge with a polygon shape could even be used to model local rainfall.

6.4.11.2 Forcing information for lateral discharges

The forcing information for lateral discharges is the intended flow rate for each of them. It can be defined in three ways, all starting with the keyword `discharge`:

- A constant value, e.g., `discharge = 10.0`, giving a constant flow rate of 10 m^3/s.
- A prescribed time series, stored in a <.bc> or (deprecated) <.tim> file, e.g., `discharge = forcing.bc`. This is described for boundaries in section 9.1.1.1. For laterals, the data format is the same, with one exception: one field `Quantity` should have the value `lateral_discharge`.
- An externally-provided discharge, i.e., `discharge = realtime`. More information about this type is given in the section on lumped rainfall-runoff coupling (section 16.7), which uses lateral discharges.

6.4.11.3 Output for lateral discharges

The lateral discharges can also be included in the output files for a model run. The total mass balance contributions can be included in the history file (section F.3.1.6). This is the total sum of lateral in- and outflows (possibly limited, see right-hand side in Equation (17.42)).

Flow data on each of the laterals can be written to the history file, see (section F.3.1.7). This can be controlled by setting `Wrihis_lateral = 0/1` in the MDU file under `[output]` (cf. Table A.1).

6.4.12 Numerical parameters

In the numerical parameters tab, all numerical parameters related to your computation can be set. The parameters that can be set are described in Table 6.1.

6.4.13 Output parameters

Model runs can produce various types of output files. The **Map** window shows the two most-used types: *(map* and *his(tory)* output (Figure 6.77).

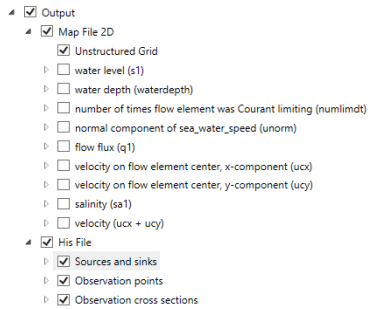


Figure 6.77: Output Parameters *tab*

The history file contains output on specific locations: time series data on observation points (section 6.4.2.2), cross-sections (section 6.4.2.3) and structures (Sections 6.4.2.8 – 6.4.2.10).

DRAFT

Table 6.1: *Overview and description of numerical parameters*

Parameter	**Description**
Max Courant number	The size of the time step, Δt, is computed by the computational kernel automatically, each time step by means of the maximum tolerable Courant number. By default, this value is 0.7.
Wave velocity fraction	The wave velocity fraction is related to stability of the computation. By using this fraction, the velocity of the flow is enlarged with the wave velocity times this fraction. The value of this fraction is 0.1 by default.
Advection type	This key depicts the ID of the advection scheme. By default this value is 33.
Water depth limiter type	The limiter type for waterdepth in continuity equation: 0 means no limiter (default), 1 is the minmod method, 2 the Van Leer method and 4 the monotone central method.
Advection velocity limiter type	The limiter type for the cell center advection velocity: 0 means no limiter, 1 the minmod method, 2 the Van Leer method and 4 the monotone central method (default).
Salinity transport limiter type	The limiter type for salinity transport: 0 means no limiter, 1 the minmod method, 2 the Van Leer method and 4 the monotone central method (default).
Solver type	Specification of the Poisson equation type solver for the pressure: 1 = sobekGS_OMP, 2 = sobekGS_OMPthreadsafe, 3 = sobekGS, 4 = sobekGS + Saadilud (default), 5 = parallel/global Saad, 6 = parallel/Petsc, 7 = parallel/GS.
Max degree in Gaussian elimination	The maximum degree in the Gauss elimination, part of the pressure solver. This key has the value 6 by default.
Fixed weir scheme	Or: fixed weir scheme. 0: none, 1: compact stencil, 2: whole tile lifted, full subgrid weir + factor.
Fixed weir contraction	Or: fixed weir contraction. This is the fixed weir flow width contraction factor, being the flow width = flow width times the fixed weir contraction.
Boundary smoothing time	Fourier smoothing time on waterlevel boundaries (s).
Linear continuity	Default 0; Set to 1 for linearizing d(Hu)/dx; link to AdvecType.
Threshold for drop losses	Apply droplosses only if local bed slope is larger than this specific value.
Theta of time integration	Level of explicitness/implicitness of time integration.
Downwind cell H on Q boundaries	Specifies the way of the upstream discharge boundary: 0 is original hu on qbnd, 1 is downwind hs on qbnd

DRAFT

The map file contains flow quantities on the entire grid at specified time intervals, and can later be used for 2D and 3D visualizations of entire flow fields. The map file can typically turn out much larger than the history file, and it is therefore advised to use larger time intervals for map files than for his files.

Additionally, three more types of output can be requested: restart files, WAQ output, and timing statistics. Restart files are a special type of map file that can later be used as initial states in other runs. Restart files contain several additional flow quantities and are written at specified intervals into one file per each restart time. Typically, one selects a large restart interval in order not to waste disk space. Timing statistics can be produced both in the diagnostics file (via *Statistics output interval*, for viewing basic simulation progress), and in a separate detailed timings file (via *Timing statistics output interval*, for detailed performance analysis).

When double clicking the *Output Parameters* in the *Project* window, the *Output Parameters* tab is highlighted below the central map. All parameters related to the output of your model run are specified here (Figure 6.78). The most common output parameters to set are the parameters related to the water quality, history, map and restart files.

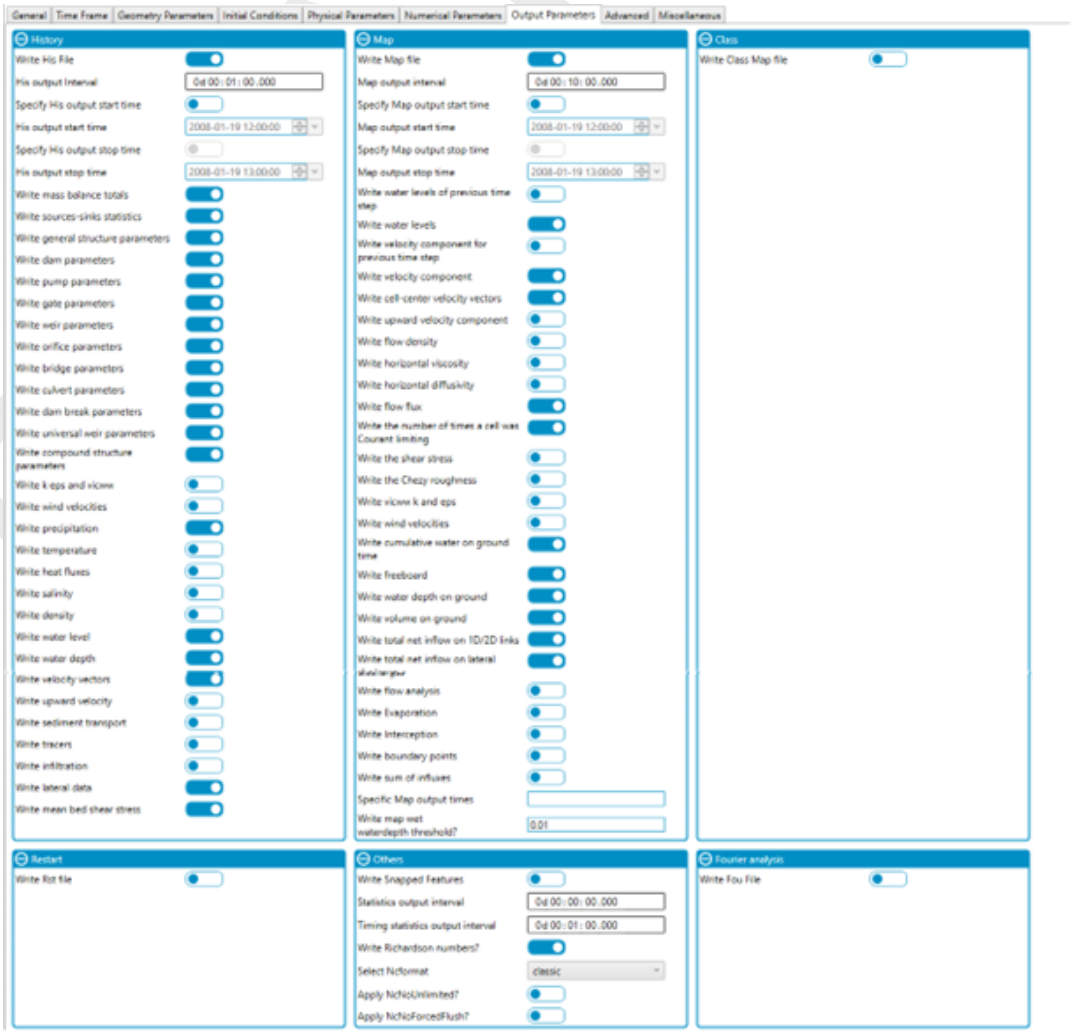


Figure 6.78: *Overview* Output Parameters *tab*

For each of the three files (water quality, history, map and restart) the input parameters are

specified in the same manner (Figure 6.78). Taking his output as an example: when *Write His File* is checked, the output of history file is enabled. *His Output Interval* determines the interval at which the output data is stored in the file; the smaller the interval, the more detailed the output and the larger the file. By default, history output is written from the start until the end of the simulation. Optionally, output can be restricted to a certain time window: to specify different output start and/or stop times, check the box next to *Specify His Output Start Time* and/or *Specify His Output Stop Time* and enter the desired times next to the parameters *His Output Start Time* and *His Output Stop Time*.

When *Write Snapped Features* is activated, then shape files with snapped data will be generated for all quantities, such as fixed weirs and thin dams. For example, for fixed weirs a crest height is specified at both end of a fixed weir polyline. In between linear interpolation is applied, which can be checked via these shape files.

Below, the various output options are described in greater detail. *There is a special requirement on the output parameters for His and Restart files that the* Output Interval*,* Output Start Time *and* Output Stop Time *must be integer multiples of* User Time Step*. Optionally, the interval (*Output Stop Time−Output Start Time*) should be an integer multiple of* User Time Step*.*

Write his file

His Output Interval	Time interval for history time series.
His Output Start/Stop time	Restrict history output to a specified time window.
Write mass balance totals	Enable detailed mass balance time series output in the his file.
Write (misc.) structure parameters	Enable time series output across general structures, pumps, weirs and gates in the his file.

Write map file

Map Output Interval	Time interval for map field time series.
Map Output Start/Stop time	Restrict map output to a specified time window.
Specific Map Output Times	File containing specific time values at which to produce additional map output snapshots. It the value is not integer, it is firstly set to the least integer larger than or equal to this value. If the computational time does not hit the specified time value, the output snapshot is chosen to be at the time closest to the specified time value.
Write water levels, etc.	Several optionals for enabling/disabling certain quantities output in the map file.

Write restart file

Restart interval	Time interval for restart files.
Rst Output Start/Stop time	Restrict restart output to a specified time window.

DRAFT

Other output options

WAQ Output Interval	Time interval for D-Water Quality files in <DFM_DELWAQ_*mdu_name**.hyd>, etc.
Simulation statistics output interval	Interval for simulation progress output (on standard out and diagnostics file).
Timing statistics output interval	Interval for detailed timings output into <*mdu_name*_timings.txt> for expert performance analysis.

Example

One example of input and output parameters (in seconds) is given in Table 6.2. Table 6.3 shows the time (after *Reference Date* in seconds) when output files are generated. We explain this example as follows.

- ◇ The history ile has output interval 18 seconds. No parameters are specified for *His Output Start Time* and *His Output Stop Time*, which means that they are automatically set equal to *Start Time* and *Stop Time* of the simulation, respectively.
- ◇ The *Map Output Interval* is 6 seconds, and *Map Output Start Time* is 15 seconds. Since the *Map Output Stop Time* is not given, it is set to equal to *Stop Time*. Moreover, we hope to have output at time given in *Specified Map Output* as 30.5 and 42.1 seconds. These two values are firstly set to 31 and 43 seconds, respectively, in the simulation. Then there will be output snapshots if the computational time hit these two integers. Otherwise, as in this example, the output snapshots will be provided when the computational time hits the time that is greater and the closest to these integers, i.e. at 31.2 and 43.2 (as seen in Table 6.3).
- ◇ Three parameters are set for the output of the Rst files: the interval, start and stop time.

Input parameters		Output parameters	
Reference Date	2007-11-19	*His Output Interval*	00:00:18
User Time Step	00:00:00.3	*Map Output Interval*	00:00:06
Start Time	2007-11-19 00:00:03	*Map Output Start Time*	2007-11-19 00:00:15
Stop Time	2007-11-19 00:00:51	*Specific Map Output*	30.5, 42.1
		Rst Output Interval	00:00:09
		Rst Output Start Time	2007-11-19 00:00:12
		Rst Output Stop Time	2007-11-19 00:00:45

Table 6.2: *Input and output parameters of the example*

His file	3, 21, 39, 51
Map file	3, 15, 21, 27, 31.2, 33, 39, 43.2, 45, 51
Restart file	3, 12, 21, 30, 39, 45

Table 6.3: *Time (after* Reference Date *in seconds) of output files*

DRAFT

6.4.14 Miscellaneous

Within the miscellaneous sub-tab, various parameters in relation to waves and equatorial settings can be adjusted. Table 6.4 gives an overview and description of these parameters.

Table 6.4: *Overview and description miscellaneous parameters*

Parameter	Description
Time step type	Leave at default.
Water level threshold	Max allowed water level difference between old and new time step in any cell. Run will abort if exceeded. (0 means disabled)
Velocity threshold	Max allowed velocity difference between old and new time step in any cell. Run will abort if exceeded. (0 means disabled)
Dry cell threshold	Flooding threshold at velocity points. Used in wetting and drying.

6.5 Save project, MDU file and attribute files

To save your D-Flow FM GUI project, navigate the menu ribbons to *File* and click *Save as*. Choose a location, specify a name and click *Save*. Your project will now be saved in a folder called <*name*.dsproj_data> and a <*name*.dsproj> file is written. Within this folder you will find all input ASCII input files of your model, output files of your model (if the model was run using the GUI) and zip folders containing your restart files. Be aware that the output files are stored within a separate folder in which the input files of your model are stored. The output folder on the same level as the folder containing model input files is empty.

To open a project, navigate the menu ribbons to *File* and click *Open*. Select the <∗.dsproj>-file of choice and click *Open*.

Importing model or data within a D-Flow FM GUI project can be achieved in two ways. Navigate the menu ribbons to *File* and click *Import*. A drop-down menu appears, allowing you to select what you want to import (Figure 6.79).

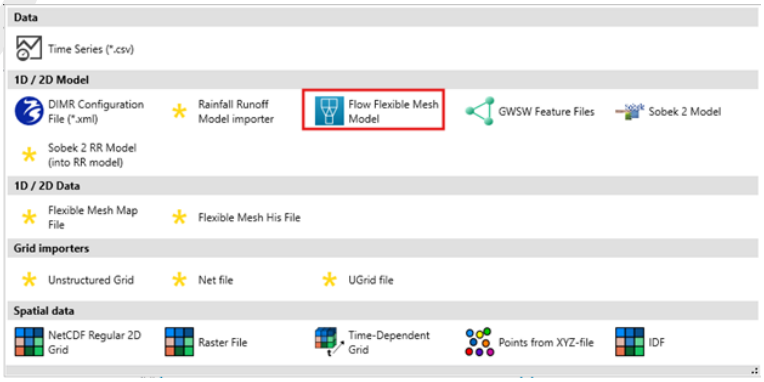


Figure 6.79: *Model/data import drop-down menu*

Alternatively, you can also right mouse click on the name of your project in the *Project* window and select *Import*. An import wizard appears, allowing you to select what you want to import (Figure 6.80).

Figure 6.80: *Model/data import wizard*

Exporting your model can be achieved in the same fashion as importing your model. All model input files will be written to the folder you select. Be aware that model files exported to a folder in which other model files with the same names are present will be overwritten.

DRAFT

7 Running a model

7.1 Introduction

After defining the input for the D-Flow FM hydrodynamic simulation, the computation can be executed either via the D-Flow FM Graphical User Interface, GUI, (on Windows), see section 7.2, or using batch scripts (on Linux and Windows). Via the GUI, the status of the computation and possible messages are displayed in a separate messages window. When using a batch script, all messages are written to the diagnostics file (section F.1) and inside the terminal (stdout and stderr).

Use a batch script (see section 7.3) in the following cases:

1 Using MPI to run in parallel, see section 7.4
2 Using some queueing mechanism on a cluster
3 Running some unattended simulations, while continuing to work in the User Interface
4 Running on Linux.

7.2 Running a scenario using the User Interface

In the *Project* window, select the model you want to run by clicking the first sub-item of the desired model (Figure 8.1).

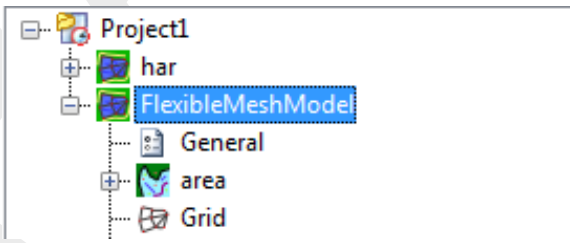


Figure 7.1: *Selecting the model you want to run in the* Project *window*

Starting the calculation can be achieved in two ways. You can navigate the menu ribbons; go to “Home” and in the group Run click on the button “Run Current”. To run all models that are opened in the *Project* window, click on “Run All” (Figure 7.2).

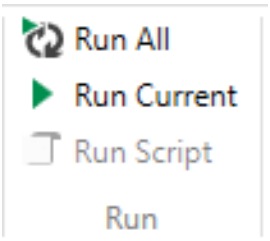


Figure 7.2: *Group Run in Home ribbon*

Alternatively, you can right mouse click the first attribute in the *Project* window of the model you want to run. Next, click “Run Model” to start the calculation. If you first select the model you want to run, the properties window (“View” “Properties”) will show several properties of the model you selected. If you set “ShowModelRunConsole” to true, the model run console of the computational core will be shown during the calculation, providing you with additional information during the model run (Figure 7.3).

DRAFT

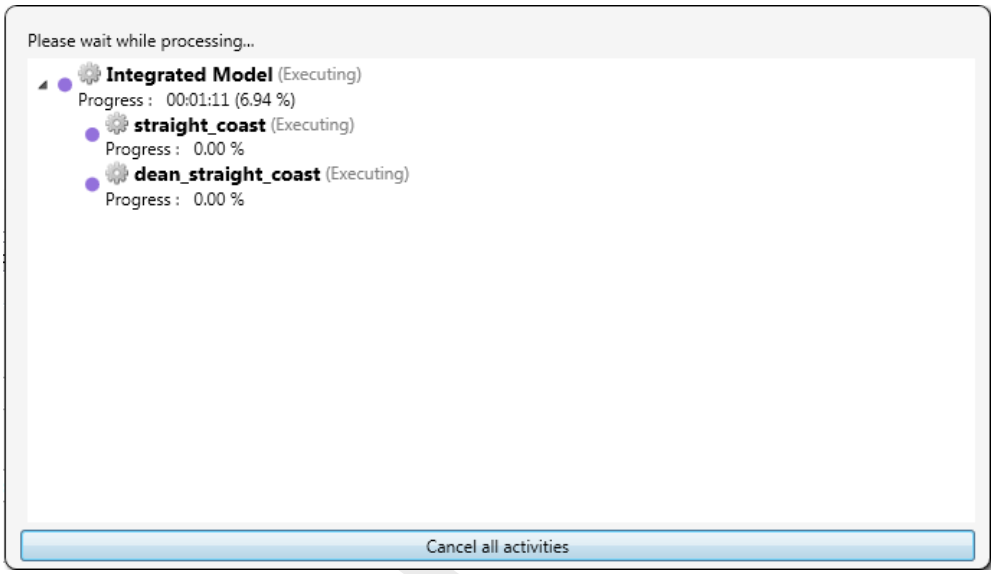


Figure 7.3: *Run console D-Flow FM User Interface*

If you cancel the run by clicking “Cancel all activities”, model results are stored up to the point where you cancel the run.

7.3 Running a scenario using a batch script

In general, the following steps have to be carried out:

1. Set-up a D-Flow FM model in the GUI on Windows.
   Currently, this is only possible for 2DH (depth-averaged) model simulations. Validate the model.
2. Export the model to either a ’DIMR configuration’ or a ’Flow Flexible Mesh model’, as described in section 5.2 of this user manual. A ’DIMR configuration’ is necessary when the computation consists of a D-Flow FM model integrated with another kernel (e.g. D-RTC or D-Waves). A ’Flow Flexible Mesh model’ is optional for a standalone D-Flow FM computation.
   In case of a 3D model, change the input for the 3D keywords (number of layers, layer percentages, . . .) via the GUI or manually. **Note:** The GUI for 3D modelling contains limited functionality for 3D yet.
3. Copy the complete set of model input files to the machine doing the computation (Windows or Linux).
4. In case of a parallel MPI computation: partition the model. Partitioning can be executed both on Windows (before copying the input files) and Linux (after copying the input files), see section 7.4.2.
5. Start the D-Flow FM model simulation. See section 7.3.2.
6. In case of a parallel MPI computation: after completion of the model simulation, map merging of the output files is often carried out in order to simplify the postprocessing. See section 7.4.4.2.
7. Use QUICKPLOT for postprocessing, see section 7.4.4 and the QUICKPLOT user manual.

DRAFT

7.3.1 Commandline executables

Executables should be called via run-scripts prepared by Deltares. To use these prepared scripts, you are supposed to write a one-lined script in your working directory.

Windows (example of a one-lined script that calls a Deltares-prepared script):
`call <installDir>\x64\bin\run_dimr.bat`

Linux (example of a one-lined script that calls a Deltares-prepared script):
`<installDir>/lnx64/bin/run_dimr.sh`

Note: Example models, with one-line-scripts, are included in the (open source) code. After registering yourself at https://oss.deltares.nl/, you can have a look at:
https://svn.oss.deltares.nl/repos/delft3d/trunk/examples
/12_dflowfm/test_data/e100_f02_c02-FriesianInlet_schematic_FM

Advantages of this construction:

- ◇ The script will take care of preparations like setting environment parameters
- ◇ The script will call the correct binary with the correct arguments
- ◇ Uniform look and feel
- ◇ In case of multiple installations, there is no confusion about what version is being used. The prepared scripts, located in an installation directory, will always use the executables in that same installation directory.

Differences between the Windows and Linux prepared run-scripts:
On Windows:

- ◇ The location of the prepared scripts is `x64\<modulename>\scripts` inside `<Your installation base dir>\plugins\DeltaShell.Dimr\kernels\` where `<Your installation base dir>` is typically: `c:\Program Files\Deltares\Delft3D Flexible Mesh Suite HMWQ (xxxx.xx)`
- ◇ The file extension is <.bat> or <.cmd>.

On Linux:

- ◇ The location of the prepared scripts is `lnx64\bin` inside `<Your installation base dir>` where `<Your installation base dir>` is typically: `/opt/delft3dfm/xxxx.xx`
- ◇ The file extension is <.sh> or no extension at all.

7.3.2 Running D-Flow FM

Export the input files in the GUI, see section 5.2. Be sure that the GUI produces the 'DIMR configuration' file too, typically called <dimr_config.xml>. Write a one-line-script to call the 'run_dimr' script in the installation directory. The easiest Windows example <run.bat>:

```
call <installDir>\x64\bin\run_dimr.bat dimr_config.xml
```

The easiest Linux example <run.sh>:

DRAFT

```
<installDir>/lnx64/bin/run_dimr.sh dimr_config.xml
```

Put the one-line-script in the same directory as file <dimr_config.xml> and execute it; on Windows double-click the script or execute it in a command box; on Linux execute it in a command box.

Note: Example models, with run scripts, are included in the (open source) code. After registering yourself at https://oss.deltares.nl/, you can have a look at:
https://svn.oss.deltares.nl/repos/delft3d/trunk/examples
/12_dflowfm/test_data/e100_f02_c02-FriesianInlet_schematic_FM

Remarks:

- ◇ Executing <dflowfm> instead of <dimr> is an option when running D-Flow FM standalone (not coupled to D-Water Quality, D-RTC, D-Waves etc.). To do this, replace 'run_dimr' by 'run_dflowfm' and the dimr config file by the mdu-file in the one-line-script.
- ◇ Calling 'run_dimr' with the argument `--help` will show the options
- ◇ Argument after the separator '`--`' are passed through to the D-Flow FM kernel (if the D-Flow FM kernel is the only kernel being started).

7.4 Parallel computing using MPI

Note: Parallel computing is of TRL level 4-6 (prototype) for 1D2D modelling.

7.4.1 Introduction

This section describes parallel computing with D-Flow FM based on the *Message Passing Interface* system (MPI). This can be run both on computing clusters with distributed memory as well as shared memory machines with multiple processors and/or multiple CPU cores. A parallelMPI computation can use multiple machines in a Linux cluster. On Windows, only one machine can be used.

The goal of parallelization of D-Flow FM is twofold: faster computations and the ability to model problems that do not fit on a single machine. A less powerful, yet possibly attractive performance improvement is offered by D-Flow FM's OpenMP-based parallelization (section 7.5.1).

Technical backgrounds on the parallel algorithms in D-Flow FM are described in the Technical Reference Manual Deltares (2026a).

Workflow of a parallel run

A parallel MPI run divides the work between multiple processes. The model is split in submodels or partitions. A separate process is started for each partition, solving the equations on that partition. It will communicate with the neighbouring processes where needed and generate output only for it's own partition. Given a whole model, the workflow is as follows:

1. Partition the model (mesh and model definition file)
2. Execute the parallelMPI job on a multi-core machine or on several machines in a (Linux) cluster
3. Visualize the results from partitioned output files.

DRAFT

7.4.2 Partitioning a model

In D-Flow FM a model is defined by the model definition file, <.mdu>, the network (mesh) file, <_net.nc>, external forcing/boundary condition files, <.ext>, et cetera. Partitioning a model concerns partitioning of <.mdu>-file and the network file <_net.nc>. Other files are shared by all submodels and do *not* need to be partitioned, they should only be available to all processes.

Partitioning a model can be done by executing `run_dflowfm`, which is slightly different on Windows and Linux. It is recommended to use a runscript to partition the model that takes care of paths and environment parameters:

◇ **Windows**
On Windows one has to use the batch file `run_dflowfm.bat`, please see the following example:

```
"<installDir>\x64\bin\run_dflowfm.bat" "--partition:ndomains=8:
icgsolver=6" example.mdu
```

Note that the double quotes, `"..."`, are used for the input arguments, in order to preserve equal-sign, =, when passing those input arguments to the batch file.

◇ **Linux**
On Linux one has to use `run_dflowfm.sh`:

```
<installDir>/lnx64/bin/run_dflowfm.sh --partition:ndomains=8:icgsolver=6
example.mdu
```

The `run_dflowfm` command reads the name of the network file from mdu-file, and generates `n` subdomain network files by the METIS software package (See Deltares (2026a) and references mentioned therein). Then, it creates `n` subdomain mdu-files where the parallel Krylov solver `icgsolver` is set to `i`, which can be 6, the PETSc solver (recommended), or, 7, the parallel CG with MILU block preconditioning.

The partition example above results into 8 subdomain mdu-files, and 8 subdomain network files. The subdomain mdu-files have names as <example_000j.mdu>, j=0,1,...,7. The different items in, e.g. <example_0000.mdu> with respect to the original mdu-file are:

```
[geometry]
NetFile       = example_0000_net.nc

[numerics]
Icgsolver     = 6 (or 7)
```

Assuming that the specified network file in the original mdu-file is <example_net.nc>, then the 8 subdomain network files have names <example_000j_net.nc>, j=0,1,...,7. In other words, they are:

```
example_0000_net.nc  example_0003_net.nc  example_0006_net.nc
example_0001_net.nc  example_0004_net.nc  example_0007_net.nc
example_0002_net.nc  example_0005_net.nc
```

By specifying `writePart_domain = 1` in the `output` section of the mdu-file, the interpreted network, including the domain partitioning, is written to the file <DFM_interpreted_idomain_example_net.nc

Note: In all the partitioning commands shown in this subsection, one can specify a network file <example_net.nc>, instead of the mdu-file <example.mdu>. This will only partition the network file. However, we strongly suggest to use the mdu-file in the partitioning commands, which partitions both the mdu-file and the network file. The reason is, when a dry points or/and enclosure file is specified in the mdu-file, the partition tool will first delete dry areas based on these files, and then do the partition. If one specifies network file in the partitioning command, then the resulting subdomain network files still include dry areas. Then after running the parallel simulation, the output map files do not contain those dry areas. These will bring inconsistence and further yield a problem if merging the map files.

7.4.2.1 More options for the partitioning

The commands in section 7.4.2 partitions the model, including partitioning the network file. This section describes the full possibilities for partitioning a network (mesh). The mesh can be automatically partitioned with the METIS software package, or manually by supplying polygons that define the subdomains. They both produce a cell colouring of the original mesh. In each subdomain, the cells are assigned the same colour, and augmented with ghost cells. This information is saved in variable "idomain" in the resulting partitioning mesh files, e.g. <example_NNNN_net.nc>, where NNNN is a subdomain index.

Partitioning with METIS

The command (on Linux):

```
<installDir>/lnx64/bin/run_dflowfm.sh --partition:ndomains=n:icgsolver=i <mdu-file>
```

is a basic command to partition with METIS. An advanced command, which enables more options, is:

```
<installDir>/lnx64/bin/run_dflowfm.sh --partition:ndomains=n[:method=0|1][:genpolygon=0|1]
[:contiguous=0|1][:ugrid=0|1][:seed=i] <mdu-file>
```

The explanation of these options is as follows:

The partition method can be chosen via setting `method=1`, the multilevel K-Way method (default), or `method=0`, the Recursive Bisection method. We refer to Karypis (2013) for more details about these two methods.

Option `genpolygon` specifies if the command generates a partition polygon file (`genpolygon=1`), or not (`genpolygon=0`, default).

By default FM enforces contiguous partitioning, i.e. `contiguous=1`. This can be switched off by setting `contiguous=0`. Only the multilevel K-Way method can enable the contiguous partition. If one requires the Recursive Bisection method, then it is not possible to enforce contiguous partitioning.

In the situation when one gets error message "*The above METIS error message is not a problem. It means that partitioning failed for k-way method with option contiguous=1 because the input graph is not contiguous. Retrying partitioning now with contiguous=0.*", it is no need to worry because it retries partitioning without enforcing contiguous.

DRAFT

Option `ugrid=1` allows to enforce UGRID output. This is required for running parallel 1D models (and map merging the output). Running parallel 2D models (and map merging) works for both non-UGRID and UGRID mesh files.

Option `seed=i` with `i` a user-defined integer allows to pass a random seed to the METIS partitioner. This allows the user to have reproducible partitioning result between separate runs.

Note: These options can be used with scripts `run_dflowfm.bat` and `run_dflowfm.sh`.
Note: Partitioning script `generate_parallel_mdu.sh` is outdated and should not be used.

Partitioning with manually supplying polygons

If the user wants to manually partition the mesh instead of applying METIS, then he has to provide a polygon file <userpol.pol> which determines the mesh partition. In this situation, he can use the following command:

```
<installDir>/lnx64/bin/run_dflowfm.sh --partition:icgsolver=i <mdu-file> <userpol.pol>
```

Notice that this command should *not* use the option `ndomains`, but adds the polygon file name at the end. If accidentally, `ndomains` *is* specified, the provided polygon will be ignored and partitioning falls back to using METIS.

Note: Similarly, to use a script `run_dflowfm.bat` or `run_dflowfm.sh`, one can remove option `ndomains`, and add the polygon file name at the end of the command, as below:

- ◇ On Windows one can use batch file `run_dflowfm.bat`, please see the following example:

  ```
  "<installDir>\x64\bin\run_dfmoutput.bat" "--partition:icgsolver=6"
  example.mdu part.pol
  ```

  Note that `" "` is used for the input arguments, in order to preserve = when passing those input arguments to the batch file.
- ◇ On Linux one can use `run_dflowfm.sh`:

  ```
  <installDir>/lnx64/bin/run_dflowfm.sh --partition:icgsolver=6
  example.mdu part.pol
  ```

Where, 'part.pol' is the name of a partition polygon file.

And the partitioning obeys the following rules:

- ◇ if the polygons have a z-value specified, it is considered a subdomain number,
- ◇ if the polygons have no z-value specified, its order determines the corresponding subdomain number,
- ◇ if a cell is not inside at least one polygon, it is assigned to subdomain 0,
- ◇ In order to avoid that subdomain 0 will be empty, which isn't allowed, the partitioning with polygons should not cover the whole model domain. Then, automatically some cells will be in subdomain 0.

DRAFT

- ◇ Thus, if $n-1$ partitioning polygons are used and some cells aren't in these polygons, then a partitioning in n subdomains is generated by the partition method.
- ◇ if a cell is inside only one polygon, it is assigned to the subdomain defined by that polygon,
- ◇ if a cell is inside more than one polygon, it is assigned to the subdomain with the highest number.

In other words, the polygons may be overlapping and the largest subdomain number is taken in the overlapping regions. If the polygons have no z-value, the polygon order determines the corresponding subdomain number, i.e. the first polygon corresponds to subdomain 1 et cetera and there is no polygon defining subdomain 0.

Partitioning the mesh with METIS via the User Interface

A separate mesh partitioning is not needed when following the instructions for partitioning a complete model in section 7.4.2. This section describes the full posibilities for partitioning a mesh.

We will firstly focus on the METIS partitioner. Partitioning the mesh from within the User Interface is achieved by the following steps. In the *Project* window, select the model you want to run by means of clicking on the desired model (Figure 8.1). A right-mouse click will open the context menu, then select *Export...*. In the window **Select Type of Data...**, choose *Partition exporter* and the partitioning dialog will appear (Figure 7.4).

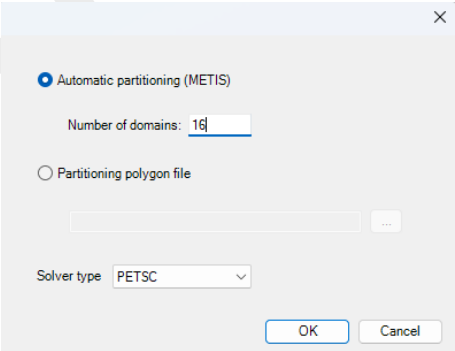


Figure 7.4: *Partioning exporter dialog*

Enter the desired amount of subdomains and leave the solver type at its default. After pressing *OK*, a file dialog will appear. Enter the name of the mdu-file, *without* any trailing '_000x' partition numbers: these will be added automatically.

Partitioning the mesh with the graphical user interface is achieved by the following steps: You are prompted for ... Note that there is *no* polygon defining subdomain 0. Parts of the mesh not confined by any polygon are assumed to be in subdomain 0. In other words, there are at least $N-1$ polygons defining N subdomains. In that case, regenerate the cell colors/domain numbers again by selecting *Operations* $\rightarrow$ *Generate domain numbers (polygon or METIS)*. Note that you will not be asked to specify the number of subdomains. The cell coloring/domain numbering is now based on the (modified) polygons and not produced by the METIS partitioner. Manual partitioning with user-specified polygons will be explained in the next section, The entered mesh filename is a *basename* that will be used to derive the partitioning filenames, e.g. <example_net.nc> will produce:

7.4.2.2 Partitioning of 1D2D models

The partitioning of models with a 1D network or even a combined 1D2D mesh is entirely the same as discussed before for a 2D mesh. This section contains some additional information about the resulting partitioned mesh files and the ghost cells in them.

The spatial discretization of the (momentum) advection and diffusion terms in the shallow water equations requires four levels of ghost cells beyond the partition boundary in each partition. More information can be found in (Deltares, 2026a, section 8.1.1). Figure 7.5 shows a partitioned 1D2D model with two partitions, where the four layers of ghost cells are clearly distinguishable in 2D. In the 1D channel, the 1D2D connections affect the ghost levels, but the partitioning algorithm accounts for this.

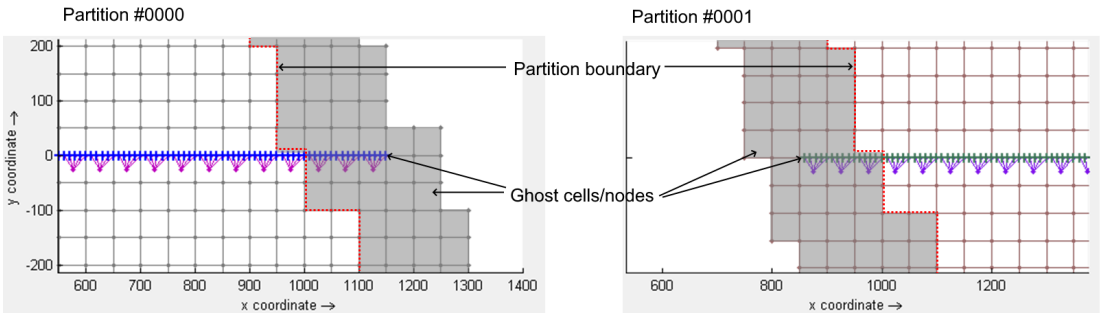


Figure 7.5: *Partitioned 1D2D mesh with two partitions and illustrated ghost cells.*

7.4.2.3 Partitioning the mdu-file

A separate mdu-partitioning is not needed when following the instructions for partitioning a complete model in section 7.4.2.

Partitioning script `generate_parallel_mdu.sh` is outdated and should not be used.

7.4.2.4 Remaining model input

A parallel run of a D-Flow FM model needs only partitioned <.mdu> and <_net.nc> files. All other model input is the same as for a standalone run, e.g., meteo forcings, boundary conditions, observation stations and more. These are generally copied to the working directory by the parallel job submission script.

7.4.3 Running a parallel job

On Windows, parallel jobs can only be executed using one node/machine/PC. On Linux, multiple nodes/machines can be used.

Note: Example models, with run scripts, are included in the (open source) code. After registering yourself at https://oss.deltares.nl/, you can have a look at:
https://svn.oss.deltares.nl/repos/delft3d/trunk/examples
/12_dflowfm/test_data/e02_f14_c040_westerscheldt

7.4.3.1 A parallel simulation via DIMR

Below is shown how to start a parallel simulation on Linux:

```
<installDir>/lnx64/bin/run_dimr.sh -c 3 dimr_config.xml
```

And for Windows:

```
call <installDir>\x64\bin\run_dimr_parallel.bat 3 dimr_config.xml
```

This assumes that the model has already been partitioned in three partitions. When running via DIMR, in the file <dimr_config.xml>, in the section about the D-Flow FM component, the following two lines must be present:

```
<process>0 1 2</process>
<mpiCommunicator>DFM_COMM_DFMWORLD</mpiCommunicator>
```

where <process> must match exactly with the number of processes.

A Linux cluster usually has a queueing system with a scheduler. Example submission scripts are available upon request.

7.4.3.2 A parallel simulation on Linux via D-Flow FM

To run a parallel simulation with a D-Flow FM model on a Linux cluster you have to prepare a submission script with the specific options that the job scheduler on your cluster requires.

Below a simple example is shown of the D-Flow FM submission script for a Linux cluster with the Grid Engine scheduler:

```
#!/bin/bash
#$ -V
#$ -q test
#$ -cwd
#$ -N My_DFlowFM_JOB
#$ -m bea
#$ -M my.email@provider.net

export LD_LIBRARY_PATH=$DFLOWFM/lib:$LD_LIBRARY_PATH
export PATH=$DFLOWFM/bin:$PATH

mpiexec -np 4 dflowfm --autostartstop YOUR_MDU_FILE.mdu >out.txt 2>err.txt
```

Assumptions related to the simplified example above:

- ◇ The D-Flow FM simulation has already been partitioned into four domains.
- ◇ The location of the D-Flow FM binaries has been stored in environment parameter 'DFLOWFM'.
- ◇ D-Flow FM is used as a standalone executable, not as part of a DIMR computation.

DRAFT

◇ The example script is used in the actual submission command on a Linux cluster, together with a specificiation of the number of nodes to be used.

The options used above are:

-V	Specify that all environment variables active within the qsub utility be exported to the context of the job.
-q	Specify the queue 'test' to be used for this job, if absent default queue is used.
-cwd	Execute the job from the current working directory.
-N	Specify the name of the job.
-m	Specifies which message type should be emailed (b=beginning of job, e=end of job, a=abort of job.
-M	Specifies the email address to send the notification.

In order to submit more complicated, e.g. multi-node simulations, additional options that are scheduler dependent have to be added.

7.4.4 Visualizing the results of a parallel run

The map and history output files (as introduced in section 6.4.13) deserve special attention in parallel runs.

The history file — with time series for observation points, structures and more — is written only by process #0, and all model-global data has already been aggregated into that single file: <*mdu_name*_0000_his.nc>.

The map file — with full-grid output of flow fields — is written for each domain separately as <*mdu_name*_000X_map.nc>. This saves communication overhead during the parallel run. The partitioned map files contain duplicate points, since each file also contains the domain's ghost nodes. For postprocessing these map files, two options are now available:

1. Direct plotting of the set of all map files in Delft3D-QUICKPLOT: the partitioned file series will be recognized automatically, and the partion results will be drawn on top of each other. For water levels this gives good results.
2. Merging the partioned map files into a single global map file with the `dfmoutput` tool. The resulting map file can then be loaded again in Delft3D-QUICKPLOT and other post-processing utilities.

7.4.4.1 Plotting all partitioned map files with Delft3D-QUICKPLOT

When opening one of the partitioned map files into Delft3D-QUICKPLOT, it will automatically detect that the map file is part of a series. An additional select list *Domain* appears, see Figure 7.6. Select either "all partitions", or a partition of your choice, and proceed with the plotting as normal (section 8.3).

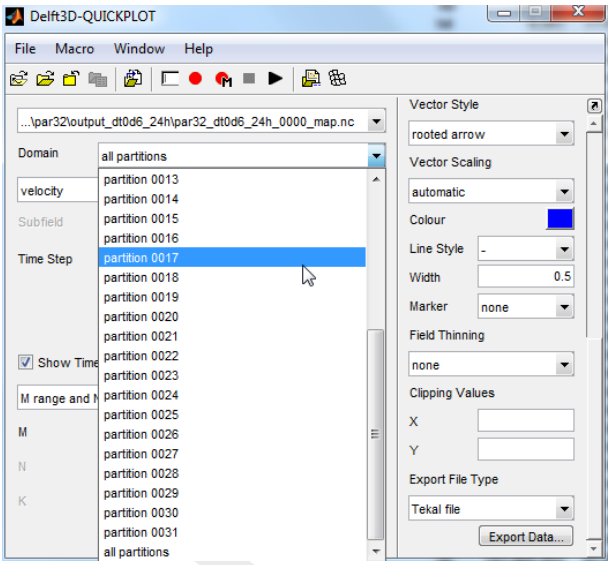


Figure 7.6: *Domain selector in Delft3D-QUICKPLOT for partitioned map files.*

7.4.4.2 Merging multiple map files into one

The partitioned map files of a parallel model run can be merged into a single global map file with the `dfmoutput` tool. It cuts off ghost nodes, and concatenates all grid points, taking care of correct global renumbering. Usage:

```
dfmoutput.exe mapmerge --infile FILE1 FILE2 [--outfile DSTFILE]
```

where `FILE1`, `FILE2` are the input files, e.g. <*mdu_name*_0000_map.nc>, <*mdu_name*_0001_map.nc>, and you can add more input files here. And `DSTFILE` is an optional output file name (the default is <*mdu_name*_merged_map.nc>).

It is recommended to use a runscript, that takes care of paths and environment parameters. Please see the following:

◇ **Windows**
On Windows one can use batch file <run_dfmoutput.bat>:

```
"<installDir>\x64\bin\run_dfmoutput.bat" mapmerge
--infile FILE1 FILE2 [--outfile DSTFILE]
```

Remarks:

- ◇ Note that for this script the number of arguments is limited to 9. Therefore, considering 2 arguments `mapmerge` and `--infile`, only maximal 7 map-file names could be passed, which means that only maximal 7 map-files can be merged if using the option `--infile`.
- ◇ If one wants to merge more than 7 map-files using this batch file, then the user should use `--listfile` instead of `--infile`, then one can use the command option `--listfile` (see the list of more advanced options below), instead of `--infile`. An example is:

```
dfmoutput.exe mapmerge --listfile list.txt
```

where file `list.txt` contains a list of names of all input map files, as shown below, with one file name on a single line.

```
model_0000_map.nc
model_0001_map.nc
model_0002_map.nc
model_0003_map.nc
model_0004_map.nc
model_0005_map.nc
model_0006_map.nc
model_0007_map.nc
```

◇ **Linux**
On Linux one can use <run_dfmoutput.sh>:

```
<installDir>/lnx64/bin/run_dfmoutput.sh -- mapmerge --infile FILE1 FILE2
[--outfile DSTFILE]
```

Note, in this command, there is a space between "`--`" and "`mapmerge`". Moreover, one can also use option "`--listfile`" instead of "`--infile`".

Remark:

◇ Since a restart file is a special type of map-file, partitioned restart files can also be merged using the above command.

The built-in help gives a list of more advanced options:

```
> dfmoutput mapmerge --help
usage:  dfmoutput mapmerge [--infile FILE1 [FILE2 FILE3...]] [--listfile LISTFILE]
        [--outfile DSTFILE] [--force] [--help] [--version]

Merge multiple map files from parallel run into one.

Optional switches:
   --infile FILE1 [FILE2...], -i FILE1 [FILE2...]
          default value
          One or more input files.
   --listfile LISTFILE, -F LISTFILE
          Pass contents of LISTFILE as input files.
   --outfile DSTFILE, -o DSTFILE
          Write output to file DSTFILE. Default: <model>_merged_map.nc
   --force, -f
          default value .false.
          Force overwriting of existing output file.
   --help, -h
          Print this help message
   --version, -v
          Print version

Examples:
   dfmoutput mapmerge         --infile model_0000_map.nc model_0001_map.nc
   dfmoutput max_running_mean --infile hisfile.nc
   dfmoutput max25            --infile hisfile.nc
   dfmoutput genfilter        --infile hisfile.nc  --intval 6
```

Merging 2D map files

2D (or 3D) map file merging has a few special aspects:

◇ All data variables and underlying spatial coordinate variables are merged into one file, cutting off all ghost cells.
◇ Near partition boundaries, the data written to the merged file is taken from the input partitioned file that "owns" the grid locations near those boundaries. For grid cells, this is the

partition in whose interior the grid cells lie. For shared mesh nodes and edges the partition is based on the lowest partition number owning the grid cell that they are part of.
- ◇ The order of the merged data values is in a so-called concatenated way. This means that all non-ghost points are concatenated domain-by-domain, keeping the local ordering for each partitioned submodel. This means that the output map file is likely to have a different ordering than the single map file from a sequential model run. This should not be an issue for any postprocessing, since all CF- and UGRID- domain variables have been merged in a consistent way, so the file remains correctly self-contained.
- ◇ Any user-defined vector dimensions for multidimensional data are also copied, if they are necessary for variables defined on the spatial domain (for example velocity components, tracers or vertical layering).

Merging 1D map files

1D map files can also be merged by the `dfmoutput` tool using the above command. There are a few special aspects to 1D map merging:

- ◇ The network data, those variables whose names start with `network1d`[1], are copied from only one of the input map files, since these have not been partitioned at all and as such need no special merging. These variables contain all information on network topology and geometry, as well as branch and node ids, some of which are needed for the valid definition of the 1D computational mesh.
- ◇ Variables of the 1D computational mesh, i.e., those variables whose names start with `mesh1d`[2], are merged in a way that cuts off the ghost nodes, just like for 2D. These variables include topology data of the 1D mesh (e.g. `mesh1d_edge_x`, `mesh1d_edge_nodes`) and data variables that are located on the 1D mesh (e.g. `mesh1d_s1`, `mesh1d_u1`). The order of the merged data depends on the topological location as follows:
  - ▫ All variables that are located on 1D nodes (pressure points), e.g., `mesh1d_node_x` and `mesh1d_s1`, are merged in the order corresponding to the global numbering of the nodes (see variable `mesh1d_flowelem_globalnr`).
  - ▫ Variables on other locations, e.g., `mesh1d_edge_x` and `mesh1d_u1`, are merged in a concatenated way, the same as merging 2D map files. This means that all non-ghost points are concatenated domain-by-domain, keeping the local ordering for each partitioned submodel.

Merging 1D2D map files

A 1D2D map file can contain both a 1D mesh and a 2D mesh, and/or mesh contacts between them. Merging 1D2D map files could be done by the `dfmoutput` tool using the above same command. Here are a few special aspects to 1D2D map merging:

- ◇ The variables on the 1D mesh and on the 2D mesh are merged separately, using the methods that are described in the above sections "Merging 1D map files" and "Merging 2D map files", respectively.
- ◇ A 1D2D map file can contain zero, one, or more mesh contacts. A mesh contact is a group of the so-called 1D2D links, where each 1D2D link connects a 1D node and a 2D face. For each mesh contact in the map file, there are a mesh contact variable, and some data variables that are located on the mesh contact, such as velocity and discharge on the 1D2D links.

[1] The exact detection of a network variable is not based on a particular name, but generically on `:cf_role=mesh_topology` and the presence of an `:edge_geometry` attribute. See Section B.2 for the specifications.

[2] The exact detection of a 1D mesh variable is not based on a particular name, but generically on `:cf_role=mesh_topology` and `:topology_dimension=1`.

A mesh contact variable has attribute `:cf_role=mesh_topology_contact`, and it gives, for each 1D2D link, the index number of its connected 1D node and 2D face.
If a mesh contact variable exists in input map files, then it is merged using the following way:

- the 1D2D links are merged in the concatenated way (see section "Merging 2D map files"). Here, the domain number of a 1D2D link is chosen to be the minimal domain number of the 1D node and 2D face that are connected by this 1D2D link.
- For a certain 1D2D link in the merged file, the index numbering of its connected 1D node and 2D faces are the global numbers, that are read directly from variable "_flowelem_globalnr" in the input map files. NOTE: when both 1D and 2D meshes are present in an input file, the global numbering of 1D nodes includes the numbering of 2D faces as well. In the merged map file, the global numbering of 1D nodes is done separately for 1D nodes, and separately for the 2D faces.
- Variables that locate on the mesh contact (1D2D links) are merged in the concatenated way (see section "Merging 2D map files").

◇ If there are more than one mesh contacts in input map files, then they are merged one by one, using the above method.

Supported situations

The map merge tool supports the following different merging situations:

◇ All input map files are in the old format, or are in the ugrid format.
◇ All input map files have only a 1D mesh, or only a 2D mesh.
◇ All input map files have both a 1D mesh and a 2D mesh.
◇ All input map files have both a 1D mesh and a 2D mesh, and at least one mesh contact.
◇ Some input map files have both a 1D mesh and a 2D mesh, some have only a 1D mesh or a 2D mesh.
◇ Some input map files have only 1D mesh, some have only 2D mesh.

7.5 Run time

The actual run time of a model can vary considerably depending on a variety of factors such as:

◇ The problem being solved, characterised by the number of active grid points, the number of layers in the vertical or the number of processes taken into account.
◇ The length of the simulation in time and the time step being used.
◇ The hardware configuration that is used and the work load of the processor.

For this reason, only some general considerations are given to determine the run time of a hydrodynamic simulation. On a PC or a workstation without separate I/O-processors the CPU time is the sum of the processor time and the I/O time.

The *processor time* required for a simulation is primarily determined by:

◇ The model definition, i.e., the number of active grid points and the number and type of the processes taken into account.
◇ The length of the simulated period in terms of the number of time steps executed.

The *I/O time* is determined by:

◇ The number of times the computed data are written to history, map, restart files and other

communication files for water quality or wave model couplings.
◇ The number of observation points, cross-sections and the number of output parameters.

The simulation performance is defined as the CPU time per grid point per time step per constituent:

$$\text{simulation performance} = \frac{\text{CPU time}}{\text{Dnt} \cdot \text{Ndx}} \quad \text{[system seconds]}$$

where:

Dnt	is the number of time steps executed
Ndx	is the number of flow nodes

The simulation performance is written to the diagnostic file at the end of the simulation.

7.5.1 Multi-core performance improvements by OpenMP

D-Flow FM has built-in support for multi-core parallellism using OpenMP[3]. This speeds up calculations by employing multiple processor cores in a single (shared-memory) computer, e.g., a modern-day notebook. OpenMP-parallellism in D-Flow FM does not scale as well as MPI-parallellism (section 7.4), but it comes for free (not any change to model input necessary) and can give a welcome performance improvement (approximately double speed on an Intel quadcore CPU). It is strongly advised to limit the number of OpenMP-threads to one less than the number of physical cores in your machine, thus also ignoring any hyperthreading. Mixing OpenMP with MPI is possible, but the optimum with respect to computation time is expected with only MPI parallellisation.

An OpenMP example on Windows:

```
set OMP_NUM_THREADS=3
call <installDir>\x64\bin\run_dimr.bat dimr_config.xml
```

An OpenMP example on Linux:

```
export OMP_NUM_THREADS=3
<installDir>/lnx64/bin/run_dimr.sh dimr_config.xml
```

7.5.2 Parallel computations in coupled simulations

D-Flow FM can run in parallel in simulations where it is coupled to other applications, like D-Waves and/or D-RTC, via DIMR. Obtaining a good performance is delicate and depends on many factors related to all modelling aspects, including hardware, operating system, queueing software, security tools, D-Flow FM itself and model setup. This section describes a recently solved performance issue and might help to get more insight.

When combining D-Flow FM with D-Waves in a coupled simulation and using parallelization, the default configuration is that D-Flow FM uses MPI (possibly combined with OpenMP) and SWAN uses OpenMP. When D-Flow FM is computing, SWAN is waiting, and when SWAN is computing, D-Flow FM is waiting; they can share resources (CPUs). **??** describes a way to

[3] http://www.openmp.org

DRAFT

check efficiency of a simulation where D-Flow FM is coupled to D-Waves. Below are hints for solving inefficiency:

Mixing MPI with OMP
If more CPUs are available than the number of MPI processes being defined, the performance might improve by using multiple OMP threads. Use this with care because the performance might worsen if the total number of processes is higher than the number of CPUs. The safest way to keep control is by specifying the number of OMP threads explicitly. This can be done for D-Flow FM and SWAN separately by using the following two environmental parameters:
```
export OMP_NUM_THREADS_SWAN=4
export OMP_NUM_THREADS=1
```
Typically, when D-Flow FM is coupled to D-Waves, D-Flow FM uses MPI partitions combined with one OMP thread (`OMP_NUM_THREADS=1`) and the number of threads used by SWAN matches with the number of available cores.
Remark:

- ◇ Deltares is improving the control on OMP threads. The number for D-Flow FM can also be set using a commandline option. More standardized options can be expected. The support for the environmental parameters, as in the example, above will pursue.

No OMP
Controlling threads is less complicated when using solely MPI-partitions. See **??** for coupling D-Flow FM (using MPI) to SWAN (using MPI). The following settings can be used for safety:
```
export OMP_NUM_THREADS_SWAN=1
export OMP_NUM_THREADS=1
```

Avoid pinning
Tasks can be pinned to a specific CPU. This might increase the performance. But when D-Flow FM is coupled to D-Waves, pinning is disastrous for the performance because D-Flow FM MPI partitions that are pinned to specific CPUs blocks these CPUs from being used by SWAN. Pinning can be enforced by the queuing system, the MPI library and the operating system. Pinning by Slurm, which is a popular job scheduler for Linux clusters, has to be avoided explicitly by adding the following line to your run-script:
```
export SLURM_CPU_BIND=none
```

Note: Do not apply the disabling of pinning globally on a cluster using something like `TaskPluginParam=none`, because a side effect will be that all processes will spin-up threads for the total host core counts, causing severe thread over-subscription, massive load spikes, and perhaps node crashes.

Avoid busy-wait
Running tasks might consume 100% CPU time while waiting for an MPI-barrier. In some situations this is the fastest way to continue the computation, but when D-Flow FM runs coupled to D-Waves, the waiting D-Flow FM partitions consume all the available CPU time and slow down the SWAN computation. In DIMR (on Linux), the call to `MPI_Barrier` is replaced by a call to `MPI_Ibarrier`, combined with a sleep, when waiting for D-Waves to finish. In case changes are needed:

- ◇ Try to get the correct behaviour by using (MPI-related) environmental parameters. For example, adding the following line to the run-script might help:
  `export I_MPI_THREAD_YIELD=3`
  Disadvantage is that this influences *all* barriers, including barriers for D-RTC, which might be desastrous for the performance. If differentiation per application is needed, try the item below:
- ◇ By default, the barrier for D-Waves on Linux uses `MPI_Ibarrier` combined with a

sleep; all other barriers use the default `MPI_Barrier`. This can be overruled per application in the DIMR config file by adding the following line to the related application block:
`<mpiBarrierSleep>200</mpiBarrierSleep>`
where the value is an integer specifying the sleep time (in milliseconds) between barrier-checks. If the value specified is zero, this will result in the usage of `MPI_Barrier` for this application. If an integer above zero is set, this will result in the usage of `MPI_Ibarrier` combined with the specified sleep period. Tests show that the exact value of the sleep period is not sensitive; if the D-Waves computation takes (more than) several seconds, any sleep-value between 10 and 500 milliseconds will do. The value of `mpiBarrierSleep` being used is echoed to stdout, for each application in the configuration, when DIMR is started with maximum debug output by adding `-d 0` to its command line arguments.

DIMR replacement
DIMR is used by default to couple applications. As an alternative, a Python script can be written replacing DIMR. The Python script can use the BMI-interface of the Deltares applications to perform the model execution and exchange data. Users can add their own applications.
Remarks:

- ◇ Arranging good performing coupled (parallel) applications via a Python replacement of DIMR is challenging.
- ◇ Deltares is currently investigating the use of preCICE[4], as alternative for DIMR.

7.6 Files and file sizes

For estimating the required disk space the following files are important:

- ◇ history file
- ◇ map file
- ◇ restart file

7.6.1 History file

The size of the history file is determined by:

- ◇ The number of monitoring points (observation points + cross-sections): H1.
- ◇ The number of quantities stored: H2.
- ◇ The number of additional process parameters, such as salinity, temperature, constituents and turbulence quantities, taken into account in the simulation: H3.
- ◇ The number of time the history data is written to the history file: H4.

You can estimate the size of a history file (in bytes) from the following equation:

size history file = H1 · (H2 + H3) · H4 · 8 bytes.

As a first approximation you can use H2 = 8.

[4] https://precice.org/

DRAFT

Example

For a 2D simulation with density driven currents (salinity and temperature), a simulated period of 12 hrs 30 min, a time integration step of 5 minutes, 30 monitoring points and each time step being stored, the size of the history file will be of the order of 384 kBytes. For the same model but now with 10 layers in the vertical the file size will increase to about 4 MBytes. These estimates show that history files are rather small. Unless the number of monitoring points is excessively large the history files are typically much smaller than the map output files.

7.6.2 Map file

The size of the map file is determined by:

- ◇ The size of the model, i.e. the number of grid cells multiplied by the number of layers (Ndxi · Kmax): M1n, and the number of flow links (open grid cell edges) multiplied by the number of layers (Lnx · Kmax): M1l.
- ◇ The number of quantities stored on grid cells and flow links: M2n, M2l, respectively.
- ◇ The number of process parameters taken into account, such as salinity, temperature, constituents and turbulence quantities: M3.
- ◇ The number of time steps for which the map file is written: M4.

Remark:

- ◇ For a more refined estimate you should distinguish between parameters that depend or not on the number of layers used (such as the water level). For a 3D simulation the latter quantities can be neglected, for a 2D simulation they must be accounted for. As a first estimate we double the number of quantities M2 in a 2D simulation.

As a first approximation you can use M2n = 5, M2l = 5 for a 3D simulation and M2n = 8, M2l = 5 for a 2D simulation.

You can estimate the size of a map file (in bytes) from the following equation:

size map file =[M1n · (M2n + M3) + M1l · M2l] · M4 · 8 bytes.

Example

For a 2D simulation with 6800 grid cells and 13000 flow links, simulation results stored for a period of 7 days, and the file is written with an interval of 60 minutes the size of the map file will be about 161 MBytes. For larger models the map file can easily become excessively large, as result the map file is less frequently written, for instance every 2 or 3 hours.

7.6.3 Restart file

A restart file is a special type of map file, where only one time snapshot per file is saved (i.e, M4 = 1). No grid or flow geometry information is stored in a restart file, except for the flow cell/link information (denoted by M5). Moreover, the restart files obtained after a parallel run contain some necessary information about parallelization (denoted by M6). Similarly to the equation of size map file above, one can write the estimating equation as follows:

size rst file = [M1n · (M2n + M3) + M1l · M2l + M5 + M6] · 8 bytes,

where M6 = 0 for a sequential run.

DRAFT

7.7 D-Flow FM command-line argument

Sometimes it's useful to run the D-Flow FM binary from the command line, without using any prepared script. On Linux this binary is named `dflowfm`. (`dflowfm-cli.exe` on Windows). On Linux a basic non-interactive run is started by:

```
> dflowfm --autostartstop MDUFILE
```

In the box below, a full list of command-line options and arguments is shown:

```
> dflowfm --help
 Usage: dflowfm [OPTIONS] [FILE]...

 FILE may be one of the following types:
  *.mdu                 model definition file
  *.cfg                 display settings file
  *.pol, *.xyz, etc.    additional model files

 Options:
   --autostart MDUFILE
       Auto-start the model run, and wait upon completion.

   --autostartstop MDUFILE
       Auto-start the model run, and exit upon completion.

   --noautostart MDUFILE
       Disable any AutoStart option in the MDU file (if any).

   --partition:OPTS (MDUFILE|NETFILE) [POLFILE]
       Partitions the unstructured model+grid in MDUFILE into multiple files.
             or: the unstructured grid in NETFILE into multiple files.

       POLFILE is an optional polygon file which defines the partitions.
       Only used when ndomains in OPTS is undefined or 0.

       OPTS is a colon-separated list opt1=val1:opt2=val2:...
         ndomains  = N     Number of partitions.
         method    = [123] Partition method: K-Way(1, default), Recursive Bisect
 ion(2), Mesh-dual(3).
         genpolygon= [01]  Generate partition polygon(1), or not (0).
         contiguous= [01]  Enforce contiguous grid cells in each domain.
                           Only available when K-Way is enabled (method=1).
         icgsolver = [67]  Parallel CG solver (When MDUFILE is specified).
                           6: PETSc (default for parallel), 7: parallel GS+CG.
         ugrid     = [01]  write cf UGRID 0.8 (0, default) or UGRID 1.0 (1)
         seed      = i     User-defined random seed value, for reproducible
                           partitionings. Only used when i not equal to 0.

   -t N, --threads N
       Set maximum number of OpenMP threads. N must be a positive integer.

   --processlibrary PROCESSLIBRARYFILE
       Specify the process library file to be used for water quality processes.

   --openprocessdllso OPENPROCESSDLLSOFILE
       Specify the open process dll/so file with additional subroutines to be
       used for water quality processes.

   --bloomspecies BLOOMSPECIESFILE
       Specify the BLOOM species definition file to be used for water quality
       processes.

   --refine:OPTS NETFILE
       Refine the unstructured grid in NETFILE from commandline.
       OPTS is a colon-separated list opt1=val1:opt2=val2:...
           hmin=VAL
           dtmax=VAL
           maxlevel=M
           connect=[01]
```

DRAFT

```
          directional=[01]
          outsidecell=[01]
          drypointsfile=<filename (*.pol, or cutcellpolygons.lst)>

 --make1d2dlinks[:OPTS] NETFILE [-o OUTPUTFILE]
     Make 1d2d links for the given NETFILE and save the resulting net.
     OPTS is a colon-separated list opt1=val1:opt2=val2:...
       method       = (1to1 | 1ton_emb | 1ton_lat | long)  Coupling method.
       linktype     = N    The link type (kn3) that will be used for all links
                           (only for method=1to1).
       connect1dend = VAL  The search distance for coupling 1D endpoints.
     OUTPUTFILE is the name under which the file will be saved.
       When not specified, the original NETFILE will be overwritten.

--cutcells NETFILE
     Cut the unstructured grid in NETFILE with the polygons specified
     in a file called 'cutcellpolygons.lst'.

--no-geom-cache
     Do not load nor save cache file with geometry information.

 -q, --quiet
     Minimal output: Only (fatal) errors are shown.

 --verbose[:level_stdout[:level_dia]], e.g., --verbose:INFO:DEBUG
     Set verbosity level of output on standard out and in diagnostics file.
     where level is in: {DEBUG|INFO|WARNING|ERROR|FATAL}
     Levels are optional, default is INFO on screen, DEBUG in dia file.

 -h, --help
     Display this help information and exit.

 -v, --version
     Output version information and exit.
```

7.8 Restart a simulation

D-Flow FM allows to restart a simulation, not from the original starting time, but from a user-specified time with all the information at that time. In other words, for a model which has a long spinup time from t_{start}, instead of restarting from t_{start}, one can restart another simulation of the same model from a later time t_{rst}, where $t_{\mathrm{start}} \leq t_{\mathrm{rst}} < t_{\mathrm{stop}}$. And the results for times $t_{\mathrm{rst}} \leq t \leq t_{\mathrm{stop}}$ are the same as in the original simulation run.

This can be achieved by following steps:

1. In order to obtain restart files <_rst.nc>, this needs to be enabled in the output parameters (see section 6.4.13 for more details about how to set output parameters).
2. Run the original model from t_{start} to t_{stop}, resulting in restart files.
3. To restart the simulation from time t_{rst}, in the mdu-file modify `TStart` to t_{rst}, *and* fill in the name of corresponding restart file in `RestartFile` of the `[restart]`-section. Moreover, place the corresponding restart file relative to the directory of this mdu-file.

To restart a parallel simulation, one can use any of the following methods:

Method 1 On each subdomain use its own restart file.

Method 2 Merge all the subdomain restart files into a single global restart file (see section 7.4.4.2), and then use it for all the subdomains. This means to put the name of this merged file as `RestartFile` in all subdomain mdu-files. This method is supported when restarting with the same, or a different domain partitioning. Moreover, such a merged restart file can also be used to restart a sequential run.

The above two methods require taking care of the name of the restart file in subdomain mdu-files. D-Flow FM automatically fills in the correct names, when partition an mdu-file to subdo-

DRAFT

main mdu-files (see section 7.4.2), in the following way:

For Method 1 When <*mduname*_YYMMDD_HHMMSS_rst.nc> is filled in `RestartFile` of the original mdu-file, partition this mdu-file gives subdomain mdu-files with `RestartFile` as <*mduname*_000X_YYMMDD_HHMMSS_rst.nc> for each subdomain <000X>.

For Method 2 If the string word "merged" is contained in the name of `RestartFile` in the original mdu-file, then this file name is kept to all the subdomain mdu-files after partitioning.

It is also possible to use a map file as a restart file, following the above approaches. Notice that in this case, one needs to specify the `RestartDateTime` in mdu-file as well. However, we strongly recommend to use <_rst.nc> file instead of a map file.

Note: Restarting from a map file is only possible for the old output format (`MapFormat`=1) and not yet for the default NetCDF UGRID output format (`MapFormat`=4).

7.9 Inspecting a model with the volume tool

With the use of the volume tool the user can generate a Netcdf file containing aggregated volumes and surface areas on 1D gridcells.

The aggregation can be performed on a complete model, on branches, on gridpoints or all three. Two matrices are written to the Netcdf file. For each level in an array of levels they contain:

1 The total volume below the level.
2 The surface area at the level.

The first index of the two matrices corresponds with the levels array. Levels start at the lowest point in the model and stops at the highest point in the model. Together with a given increment, levels are defined by successive increments from the lowest point until the highest point is reached.

The second index of the two matrices will correspond to a list with IDs. For the total volume for the complete model the ID-name will be 'Total'. For the branches the branch IDs will be used. For the gridpoints the node IDs will be used, written in a separate file also containing 1D geometry information.

The volume tool is called by:

```
usage:  dfm_volume_tool --mdufile FILE --increment INCREMENT [--output OUTPUT_TYPE]
                    [--outputfile OUTPUT_FILE] [--gridoutputfile GRID_OUTPUT_FILE]
                    [--help] [--version]
```

Required switches:

◇ `--mdufile FILE, -f FILE`
Name of the model definition file.

◇ `--increment INCREMENT, -i INCREMENT}`
Increment between volume table levels.

Optional switches:

◇ `--output OUTPUT_TYPE, -o OUTPUT_TYPE`
Output type: "`Total`", "`Branches`", "`Gridpoints`", "`All`".

- ◇ `--outputfile OUTPUT_FILE, -p OUTPUT_FILE`
  Name of the output file.
- ◇ `--gridoutputfile GRID_OUTPUT_FILE, -g GRID_OUTPUT_FILE`
  Name of the output file for the Gridpoints output option.
- ◇ `--help, -h`
  Print help message.
- ◇ `--version, -v`
  Print version.

Note: In some models storage nodes are defined with a street level of 9999.0 m. This is to denote there is no street storage, and in that case the highest level will be 9999 m. As a result the levels in the volume tables will go up to 9999 m.

Warning:

- ◇ Currently, running `dfm_volume_tool` will overwrite any existing output files (<*_map.nc>, <*_his.nc>, <*_clm.nc><*_rst.nc>). Make a backup copy first if you want to keep existing model results.

7.10 Frequently asked questions

This chapter aims to help you with common questions that may arise while using D-Flow FM.

1 **Question**
My model does not run/crashes. What's wrong?
Answer
The diagnostics file is the starting point for finding out what went wrong. See section F.1 for a detailed description of the contents of this file, and the order of model run output. Globally, ask yourself the following questions:

- ◇ Was the mdu-file found?
- ◇ If yes, was the model successfully loaded? Common mistakes are missing boundary or meteorological forcings file.
- ◇ If yes, was the time loop successfully started? Possible errors are non-writeable output files.
- ◇ If yes, does the dia file contain any messages from during the time loop? Possible errors are solver convergence errors.
- ◇ If not, did the run end successfully?

2 **Question**
I get a warning that my network is non-orthogonal. Can I loosen the orthogonality threshold?
Answer
Unfortunately, no. Orthogonality is very important for accuracy: advised orthogonality values for your grid are around 0.01, preferably lower. The current threshold is already very high at a value of 0.5. Use D-Grid Editor to improve your grid orthogonality.

3 **Question**
When running a computation with a batch script on Linux, should I call `DIMR` or `dflowfm`?
Answer
When running D-Flow FM online in combination with for example D-RTC, you have to run via DIMR. When running a standalone D-Flow FM computation, you can also choose `dflowfm`. Deltares aims on one method for running computations and that is via DIMR. But not all D-Flow FM functionality is yet available via DIMR, which involves functionality for partitioning a model (Section 7.4.2)

DRAFT

8 Visualize results

8.1 Introduction

A model run will produce two types of output:

1 a graph or history (<∗.his>-file) for a specific quantity on a location
2 a map (<∗.map>-file) for a specific quantity

Both are stored in files within the project, as described in section 7.6.

D-Flow FM provides basic visualization of the model and the model results. Advanced and tailor-made visualization is possible by the export of the his- and/or map-files, and inspection with dedicated visualization applications. Some are provided and described below.

8.2 Visualization with the User Interface

When your model run has finished, a new folder called “Output” has appeared at the bottom of the attributes of your model in the *Map* window. Inside this folder, all output quantities of the his-, and map-files can be found, as well as a folder called “States”, in which you find all restart files written during the model run. Output folders have also appeared in the *Map* window; “Output (map)” and “Output (his)”. Both the results of your map and his files can be viewed in the central map by means of enabling the desired quantities from the *Map* window (Figure 8.1).

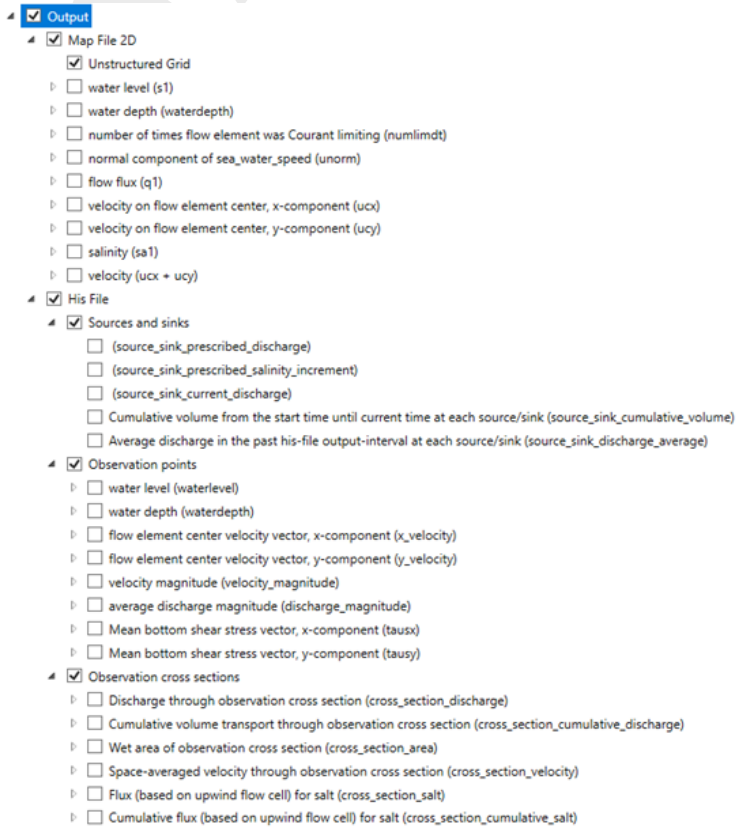


Figure 8.1: *Example of setting output (in)visible in the* Map *window*

DRAFT

The time navigator can be used to slide through the different time steps in the output files. If your model is relatively large, the drawing performance of the interface can be improved dramatically by enabling QuadTree visualizations in the properties window of the layer you want to visualize. To this end, select the layer you want to visualize in the ...

8.3 Visualization with Delft3D-QUICKPLOT

The interface of the Delft3D-QUICKPLOT allows to open the netCDF output files from a D-Flow FM simulation. In the interface you can select data fields, make a selection of time steps, stations or specify a preferred figure presentation options. After selection of a certain options you can visualize your data by using the “Quick View” button. To add another plot to an existing figure the “Add to Plot” button should be used.

When plotting the map output files from a D-Flow FM simulation, by default the presentation type “markers” will be selected. To smoothly visualize your results it is recommended to change presentation type into “polygons” and then select the option “Fill Polygons” (see the example on the Figure 8.2).

Figure 8.2: *Useful map visualization options in the Delft3D-QUICKPLOT*

See the Delft3D-QUICKPLOT User Manual for full details and a description of the routines and their use.

8.4 Visualization with Matlab

OpenEarthTools is an open-source toolbox that includes, among other components, MATLAB scripts and functions for the pre- and postprocessing of D-Flow FM files. For detailed guidance on getting started and using these tools effectively, visit: https://publicwiki.deltares.nl/display/OET/OpenEarth+tutorials#OpenEarthtutorials-GettingtheOpenEarthtools. Note: OpenEarthTools is provided without official support.

8.5 Visualization with Python

History-files can be easily visualized using *xarray* (https://docs.xarray.dev/en/stable/). Similarly, map-files can be visualized using *xugrid* (https://deltares.github.io/xugrid/). *Xugrid* is a UGRID wrapper around *xarray* that exposes many plotting options and processing options like coordinate transformation, re-indexing, rasterizing and regridding. If you want to do more advanced map-file visualization like horizontal and vertical (2DV cross section) slicing you can use *dfm_tools* (https://deltares.github.io/dfm_tools/), this package uses *xarray* and *xugrid* where possible and adds several extra features to read, process and visualize the history- and map-files of D-Flow FM. It can also works with multiple partitions and with older D-Flow FM mapformats.

DRAFT

9 Boundary conditions in D-Flow FM

9.1 Hydrodynamics boundary conditions

In section 6.4.7, the boundary conditions are discussed from the viewpoint of the user interface. In the user interface, the user can specify the locations at which particular boundary conditions are to be imposed. Using section 6.4.7 as a backdrop, the present section discusses the underlying files and file formats and the way these are interpreted by the computational kernel. Three types of boundary conditions are discussed in this section, namely open boundaries (in section 9.1.1) and closed boundaries (in section 3.6).

Lateral discharges are a special type of input that also act like boundary conditions, and are discussed in section 6.4.11.

9.1.1 Open boundary conditions

The proper prescription of an open boundary condition along a certain (part of the) rim of the grid can be achieved by considering four elements of the model:

1. the location where boundary are imposed, defined by either a polyline file (extension `.pli`), or for 1D networks the network end node id,
2. a boundary conditions file (extension `.bc`), containing the actual *forcing signals*, such as the time dependent information of the quantity under consideration and the physical nature of the quantity itself,
3. an external forcing file (extension `.ext`), in which the connection is laid between the location (polyline file or node id) and the boundary conditions file,
4. the name of the external forcing file in the master definition file (extension `.mdu`).

These four elements are subsequently discussed in the following.

Alternatively, for 1D boundaries, the bed level and flow link width can be customized per boundary. This can be useful at boundaries where the boundary water level is below the interior bed level, for example for an urban sewer outflow into open surface water. The following two options are available in the external forcings file (see also section 9.1.1.2):

- `bndBlDepth`: Custom bed level *depth below* initial water level for boundary point: $bl_{L(j)} = \zeta_{R(j)} - \texttt{bndBlDepth}$;
- `bndWidth1D`: Custom width for boundary flow link: $w_{u_j} = \texttt{bndWidth1D}$, and area and wetted perimeter are based on a rectangular profile with this width.

9.1.1.1 Forcing information

The <.bc>-file is used to store the actual boundary forcing signals. Multiple types of boundary conditions can be prescribed, configured via the `Quantity` keywords. The following subsections contain details for several types of flow boundaries.

The <.bc>-file may also contain the forcing signals for multiple boundary locations, configured via the `Name` keyword. When boundary locations have been specified by a polyline (`locationFile` in the <.ext> file), then the `Name` must refer to the polyline name, followed by the polyline point number, e.g., `Name = upstream_0001`. When instead a `nodeId` was used in the <.ext> file, the `Name` must be the same as that node id.

DRAFT

Water level

A water level signal is applied at the cell center of the virtual cell outside the grid (ghost cells or mirror cells). Water levels can be imposed as a timeseries or as a harmonic signal. In case of a harmonic signal, the period of the signal can be given as an astronomic component acronym.

If a timeseries is applied to a the first support point of a polyline named `arbitraryname`, prescribed in some polyline file, then the header of the `bc`-file is:

```
[forcing]
Name                 = arbitraryname_0001
Function             = timeseries
Time-interpolation   = linear
Quantity             = time
Unit                 = minutes since YYYY-MM-DD
Quantity             = waterlevelbnd
Unit                 = m
 [two-column data]
```

The data is to be inserted as a two-column array containing the time (in minutes) and the water level itself (in meters w.r.t. the reference level). In case of harmonic components, the header of the `bc`-file is:

```
[forcing]
Name                 = arbitraryname_0001
Function             = harmonic
Quantity             = harmonic component
Unit                 = minutes
Quantity             = waterlevelbnd amplitude
Unit                 = m
Quantity             = waterlevelbnd phase
Unit                 = degrees
 [three-column data]
```

The data is to be inserted as a three-column array containing the period (in minutes), the amplitude (in meters) and the phase (in degrees). If the period is T minutes, the amplitude is A meters and the phase is φ degrees, then the signal that is applied reads

$$h(t) = B + A\ \cos(2\pi t/T - \varphi). \tag{9.1}$$

The parameter B can be prescribed by an additional signal with a period specified equal to 0 minutes. As an example, the data series:

```
  0.0  0.5  0.0
745.0  2.0  0.0
```

represents the signal $h(t) = 0.5 + 2.0\ \cos(2\pi t/745)$, with the time t in minutes.

Discharge

A discharge boundary condition is applied at the face-center of the virtual boundary cell. For this, the face-normal velocity is used in combination with the face-center water depth. The face-based water depth in the evaluation of the flow area can optionally be set to a downwind

DRAFT

approximation (for an inflowing discharge boundary) with the option `jbasqbnddownwindhs = 1` We remark that `jbasqbnddownwindhs = 0` is the default value.

Discharges can be imposed as a timeseries or as a harmonic signal. In case of a harmonic signal, the period of the signal can be given as an astronomic component acronym.

If a timeseries is applied to a the first support point of a polyline named `arbitraryname`, prescribed in some polyline file, then the header of the `bc`-file is:

```
[forcing]
Name                 = arbitraryname_0001
Function             = timeseries
Time-interpolation   = linear
Quantity             = time
Unit                 = minutes since YYYY-MM-DD
Quantity             = dischargebnd
Unit                 = m3/s
 [two-column data]
```

The data is to be inserted as a two-column array containing the time (in minutes) and the water level itself (in meters w.r.t. the reference level). In case of harmonic components, the header of the `bc`-file is:

```
[forcing]
Name                 = arbitraryname_0001
Function             = harmonic
Quantity             = harmonic component
Unit                 = minutes
Quantity             = dischargebnd amplitude
Unit                 = m3/s
Quantity             = dischargebnd phase
Unit                 = degrees
 [three-column data]
```

The data is to be inserted as a three-column array containing the period (in minutes), the amplitude (in cubic meters per second) and the phase (in degrees).

Velocity

A velocity boundary condition is applied at the face-center of the virtual boundary cell. Values provided are interpreted as face-normal velocities. Velocities can be imposed as a timeseries or as a harmonic signal. In case of a harmonic signal, the period of the signal can be given as an astronomic component acronym.

If a timeseries is applied to a the first support point of a polyline named `arbitraryname`, prescribed in some polyline file, then the header of the `bc`-file is:

```
[forcing]
Name                 = arbitraryname_0001
Function             = timeseries
Time-interpolation   = linear
Quantity             = time
Unit                 = minutes since YYYY-MM-DD
Quantity             = velocitybnd
Unit                 = m/s
 [two-column data]
```

DRAFT

The data is to be inserted as a two-column array containing the time (in minutes) and the water level itself (in meters w.r.t. the reference level). In case of harmonic components, the header of the bc-file is:

```
[forcing]
Name                 = arbitraryname_0001
Function             = harmonic
Quantity             = harmonic component
Unit                 = minutes
Quantity             = velocitybnd amplitude
Unit                 = m/s
Quantity             = velocitybnd phase
Unit                 = degrees
 [three-column data]
```

The data is to be inserted as a three-column array containing the period (in minutes), the amplitude (in meters per second) and the phase (in degrees).

Water level gradient

Besides the option to prescribe actual water levels as a boundary condition, D-Flow FM facilitates the prescription of water level *gradients*. Such a Neumann-type boundary condition for the water level can be assigned through the keyword neumannbnd. The value of the water level gradient is applied at the face-center.

Water level gradients can be imposed as a timeseries or as a harmonic signal. In case of a harmonic signal, the period of the signal can be given as an astronomic component acronym. If a timeseries is applied to a the first support point of a polyline named arbitraryname, prescribed in some polyline file, then the header of the bc-file is:

```
[forcing]
Name                 = arbitraryname_0001
Function             = timeseries
Time-interpolation   = linear
Quantity             = time
Unit                 = minutes since YYYY-MM-DD
Quantity             = neumannbnd
Unit                 = -
 [two-column data]
```

The data is to be inserted as a two-column array containing the time (in minutes) and the water level itself (in meters w.r.t. the reference level). In case of harmonic components, the header of the bc-file is:

```
[forcing]
Name                 = arbitraryname_0001
Function             = harmonic
Quantity             = harmonic component
Unit                 = minutes
Quantity             = neumannbnd amplitude
Unit                 = -
Quantity             = neumannbnd phase
Unit                 = degrees
 [three-column data]
```

DRAFT

The data is to be inserted as a three-column array containing the period (in minutes), the amplitude (in meters per second) and the phase (in degrees). The water level gradient is interpreted positive in the outward-normal direction.

Riemann invariant

At a Riemann boundary we do not allow any outgoing perturbation with respect to some reference boundary state to reflect back from the boundary. This is achieved by prescribing the incoming Riemann invariant. Using the convention of a positive inward normal at the boundary, this can be put as $u_b + 2\sqrt{gH_b}$, with u_b the velocity at the boundary and H_b the total water depth at the boundary. When applying Riemann boundaries only currents perpendicular to the open boundary are considered and directional effects are neglected.

Using linearization and the assumption of a flow field that is initially at rest, one finds $u\sqrt{\frac{H}{g}}$ for a propagating wave according to linear wave theory. Next, a Riemann boundary can be rewritten such that is takes the form:

$$\zeta_b = 2\zeta_i - u\sqrt{\frac{H}{g}} - \zeta_0 \tag{9.2}$$

with

ζ_b	the surface level elevation at the boundary,
ζ_i	the water level of the perpendicularly incoming wave at the boundary,
H	the total water depth,
g	the gravitational acceleration,
u	the flow velocity (positive inward) and
ζ_0	the initial water level elevation.

In D-Flow FM it is sufficient that the water level of the perpendicularly incoming wave at the boundary, ζ_i, is precribed in case of a Riemann boundary. Then, in the computational kernel of D-Flow FM extra terms are added so that this open boundary acts as a Riemann boundary. Thus, in case of a Riemann boundary a water level forcing has to be prescribed on input in D-Flow FM.

Example

As an example, a simple uniform channel with two Riemann boundaries, with 1 m amplitude on the left and 0 m on the right will show a 1 m amplitude wave travelling from left to right. At the right side, the wave passes the boundary with minor reflection only. Practically no waves travel from right to left since at the right boundary no incoming wave amplitude was specified.

The water level of the incoming wave for the Riemann boundary can be prescribed as a timeseries or as a harmonic signal. In case of a harmonic signal, the period of the signal can be given as an astronomic component, (e.g. M_2), or as a period in minutes, amplitude in m and phase in degrees. If a timeseries is applied to the first support point of a polyline named *arbitraryname*, prescribed in some polyline file, then the header of the <bc>-file read:

```
[forcing]
Name                 = arbitraryname_0001
Function             = timeseries
Time-interpolation   = linear
Quantity             = time
Unit                 = minutes since YYYY-MM-DD
Quantity             = riemannbnd
```

```
Unit                    = m
 [two-column data]
```

The data is to be inserted as a two-column array containing the time (in minutes) and the water level for Riemann boundary (in meters). In case of harmonic components, the header of the bc-file is:

```
[forcing]
Name                    = arbitraryname_0001
Function                = harmonic
Quantity                = harmonic component
Unit                    = minutes
Quantity                = riemannbnd amplitude
Unit                    = m
Quantity                = riemannbnd phase
Unit                    = degrees
 [three-column data]
```

The data is to be inserted as a three-column array containing the period (in minutes), the amplitude (in meters per second) and the phase (in degrees).

Suppose, as an example, that we have:

- ◇ a flow domain with a bathymetry at a bed level equal to 0 m w.r.t. the reference level,
- ◇ an initial water level equal to 10 m w.r.t. the reference level,
- ◇ a local initial disturbance of the initial water level,
- ◇ an initial flow field at rest and
- ◇ that we aim to prevent reflections at the boundary.

In that case, it follows from Equation (9.2) that $\zeta_b = 10$ m w.r.t. the reference level is to be prescribed. Remark that this application only holds for small disturbances of ζ from ζ_b.

Discharge-water level dependency

If a relation between the discharge and the local water level is known on beforehand, then this relation can be provided to the flow model as a table by means of the keyword qhtable. This table, provides the water level ζ as a function of the computed discharge Q along the entire polyline defining the boundary. Worded differently, only one relation is given for each polyline. Note that this is opposite to other boundary types in which, for instance, a time series is given for each support point of a polyline.

If a Qh-table is applied to a polyline named arbitraryname, prescribed in some polyline file, then the header of the bc-file is:

```
[forcing]
Name                    = arbitraryname
Function                = qhtable
Quantity                = qhbnd discharge
Unit                    = m3/s
Quantity                = qhbnd waterlevel
Unit                    = m
 [two-column data]
```

The data is to be inserted as a two-column array containing the discharge (in cubic meters per second) and the water *level* (in meters w.r.t. the reference level).

9.1.1.2 External forcings file

The external forcings file <.ext> collects all boundary condition information. The file format for boundaries is described in section C.6.2.1. It contains one or more `[Boundary]` blocks, one for each boundary condition. Each block must specify the boundary type via the `quantity` keyword. The previous section already listed some boundary quantity names. Next, each block must specify the boundary location specification. This is either a polyline file name, via the `locationFile` keyword, or a 1D network node id, via the `nodeId` keyword. Finally, the actual forcing data file must be specified, via the `forcingFile` keyword. Section 9.1.1.4 contains an example of this.

9.1.1.3 External forcings in model definition file

The final step in specifying boundary conditions, is referring to the <.ext> file in the <.mdu> file. This is done via the keyword `ExtForceFileNew` under the MDU block `[external forcing]`.

9.1.1.4 Example

Recalling the four elementary actions from the beginning of section 9.1.1, the administration in files can be illustrated by means of an example. Suppose, we have a channel that is geometrically modelled by means of the grid shown in Figure 9.1. The domain is 10000 m long and 500 m wide. The bottom left corner coincides with the origin of the geometrical frame of reference.

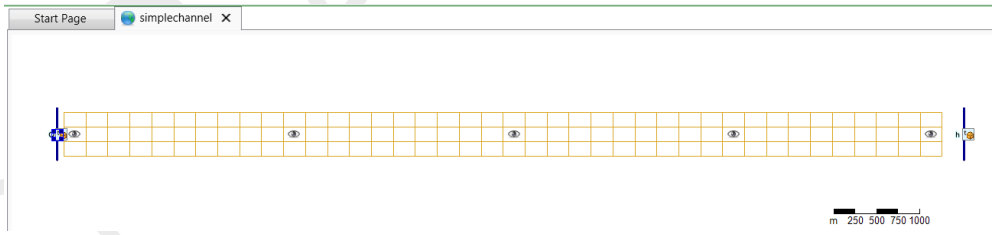


Figure 9.1: *User Interface view of a simple channel covered by a straightforward Cartesian grid. Boundary conditions are prescribed at the left hand side and the right hand size of the domain.*

The domain shown in Figure 9.1 contains two boundaries: on the left hand, a discharge boundary condition is present to present flow entering the domain, on the right hand, a water level boundary condition is set. Having two boundaries, we have two polyline files, in this example: `left.pli` and `right.pli`.

The contents of `left.pli` are:

```
boundaryleft
    2     2
    -80    -50
    -80    550
```

whereas the contents of `right.pli` are:

```
boundaryright
    2     2
```

```
  10250     -50
  10250     550
```

Notice that both polyline files contain the name of the polyline, namely `boundaryleft` and `boundaryright`, respectively. In this example, both the polylines contain *two* support points. For each of these support points, timeseries can be prescribed. In this example, we restrict ourselves to homogeneous boundary conditions. This means that we have to prescribe physical information for the first support points, i.e. for `boundaryleft_0001` and `boundaryright_0001`.

The physical information should be provided in the `bc`-file. In this example, we use the following `bc`-file:

```
[forcing]
Name                 = boundaryleft_0001
Function             = timeseries
Time-interpolation   = linear
Quantity             = time
Unit                 = minutes since 2001-01-01
Quantity             = dischargebnd
Unit                 = m3/s
  0.000000    2500.0
120.000000    3000.0
240.000000    2500.0
360.000000    2000.0
480.000000    2500.0
600.000000    3000.0
720.000000    2500.0
840.000000    2000.0
960.000000    2500.0
1080.000000    3000.0
1200.000000    2500.0
1320.000000    2000.0
1440.000000    2500.0

[forcing]
Name                 = boundaryright_0001
Function             = timeseries
Time-interpolation   = linear
Quantity             = time
Unit                 = minutes since 2001-01-01
Quantity             = waterlevelbnd
Unit                 = m
  0.000000    2.50
1440.000000    2.50
```

This file couples the timeseries for the discharge to the support point named `boundaryleft_0001`. Likewise, the water level boundary, being constant in time, is coupled to the support point named `boundaryright_0001`.

The final specification of the boundary conditions is wrapped up in the external forcing file, with extension `.ext`. In this file, the connection is laid between the quantity of the boundary condition, the name of the polyline file (in which the name of the polyline itself is given) and the forcing file (in which the physical information is provided). In our example, the file `simplechannel.ext` has the following contents:

```
[boundary]
quantity     = dischargebnd
locationfile = left.pli
forcingfile  = simplechannel.bc
```

```
[boundary]
quantity     = waterlevelbnd
locationfile = right.pli
forcingfile  = simplechannel.bc
```

The final step is to let the model know that the external forcings file `simplechannel.ext` is the one that should be used. This is to be achieved in the MDU-file:

```
[external forcing]
ExtForceFile                          =
ExtForceFileNew                       = simplechannel.ext
```

Notice that the name is specified at the keyword `ExtForceFileNew`. The other keyword, `ExtForceFile`, should be kept empty, unless *deprecated* `.tim`-files and/or `.cmp`-files are used to prescribe the physical information for the boundaries.

9.1.1.5 Miscellaneous

In the previous sections, the most essential information on the application of boundary conditions is described. Some remaining aspects are discussed in this section.

Artificial boundary layers

In general, the velocity at the boundary is prescribed as having only a component normal to the boundary (`cstbnd = 0`). This could lead to an artificial boundary layer along the boundary at offshore boundaries. To resolve this behaviour, the advection terms containing gradients perpendicular to the boundary can be switched off, by setting the keyword `cstbnd = 1` in the `[numerics]` block in the MDU-file. By default this keyword `cstbnd = 0`.

Smoothing parameter boundary conditions

The solution of the shallow water equations is uniquely determined by a set of initial and boundary conditions. The boundary conditions represent the external forcing and determine the steady state solution. The deviation between the initial condition and the steady state solution generates a transient (mass spring system analogy).

In D-Flow FM, the initial conditions for the water level and velocities are obtained from:

- ◇ The results of a previous run (warm start).
- ◇ User-prescribed (space varying or uniform) input fields (cold start).

The initial values are usually inconsistent with the boundary conditions at the start time of the simulation. This will generate a transient solution consisting of waves with eigen frequencies of the model domain. These waves may be reflected at the boundaries and generate a standing wave system. The waves should be dissipated completely by bottom friction and viscosity terms or leave the domain through the open boundaries. The damping of the transient solution determines the spin-up time of the numerical model.

To reduce the amplitude of the transient wave and the spin-up time of a model, D-Flow FM has an option to switch on the boundary forcing gradually by use of a smoothing period (parameter T_{smo}). This can by specifying the total number of seconds required for the transition using the keyword `Tlfsmo` in the `[numerics]` block in the MDU-file. With $F_i(t)$ the initial value at

the boundary, $F_b(t)$ the boundary condition signal and $F_b^{smo}(t)$ the boundary condition after smoothing, the boundary forcing is given by:

$$F_b^{smo}(t) = \alpha F_i(t) + (1-\alpha)F_b(t), \tag{9.3}$$

where t is the number of seconds since the reference date `RefDate` and

$$\alpha = \frac{T_{smo} + T_{start,smo} - t}{T_{smo}} \tag{9.4}$$

if $T_{start,smo} < t < T_{smo} + T_{start,smo}$. In case $t \geq T_{smo} + T_{start,smo}$, then the smoothing parameter is set to zero: $\alpha = 0$. The behaviour is illustrated in Figure 9.2. The situation $t < T_{start,smo}$ is not possible.

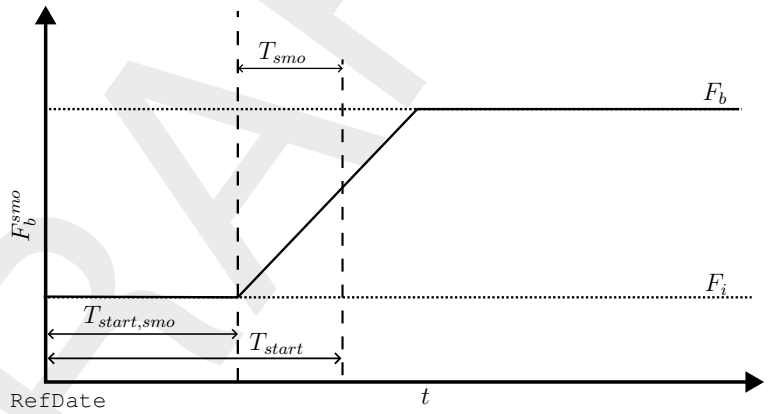


Figure 9.2: *Transitioning between initial conditions F_i and boundary forcing F_b depending on the smoothing time T_{smo} and smoothing start time $T_{start,smo}$ for a case in which the start of start time `TStart` falls within the smoothing period.*

$T_{start,smo}$ denotes the smoothing start time (not to be confused with the start time `TStart`) with respect to the reference time. This is defined using the keyword `TStartTlfsmo`. By default it is set to the start time of the simulation (i.e., smoothing is applied since the beginning), and it should be set to the start time of the original simulation when restarting (keyword `TStart`).

Smoothing is possible both for a warm and a cold start. If the initial conditions are consistent with the boundary conditions at the start time of the simulation then the smoothing time should be set to zero. When restarting a simulation (warm start), the smoothing time should be reduced by the difference between the original smoothing time (i.e., in the cold start) and the restart time, and it should be set to zero if the restart time is larger than the original smoothing time.

Remark:

- ◇ The introduction of the `TStartTlfsmo` has changed the behaviour of `Tlfsmo` in combination with a restart file as of Delft3D Flexible Mesh Suite 2024.03. Previously, when using a restart file, the simulation was assumed to be correctly adjusted to the

DRAFT

initial conditions, and therefore `Tlfsmo` was automatically set to zero. This is no longer the case. **To reproduce old model results when using a restart file, manually set `Tlfsmo` to zero in the `.mdu` file.**

Secondary boundary conditions

In section 9.1.1.1, several types of boundary conditions are discussed. In addition, two types of boundary conditions can be applied on top of the canonical types: *normal* velocities and *tangential* velocities. The associated keyword are `normalvelocitybnd` and `tangentialvelocitybnd`, respectively.

Bed level at open boundaries

For some evaluations, the solver requires the bed level at the open boundary. This value depends on the bed level in the ghost cell outside the model domain. By default, the bed level in the ghost cell is mirrored from the internal grid cell next to the boundary, which means that a zero bed slope is assumed. While this is perfectly acceptable for most real-life applications, it hinders obtaining perfect normal-flow solutions in idealized models when using `DpuOpt` = 2, i.e. the mean bed level imposed at the cell interfaces. For these specific cases, an option has been implemented to extrapolate the bed level outside the domain. When setting `[geometry]` keyword `ExtrBl` = 1, the bed levels at the ghost cells are extrapolated in such a way that the flow depth at velocity points can be exact in idealized cases.[1] This option is only available when the bed level is specified at the cell centres (`BedlevType` = 1).

Furthermore, for obtaining perfect normal flow, the downstream water level needs to be specified at the ghost cell (`izbndpos` = 0) and the conveyance needs to be computed based on a uniform bed level based on the surrounding net nodes (`Conveyance2D` = -1). Note that the boundary condition must be manually shifted by $s\Delta x$, being s the slope, from the exact value expected at the domain boundary (cell edge).[2]

Suppose a case in which a specific discharge q of 1 m^2/s is fed into a rectangular channel of constant width and constant slope s equal to -7.1356×10^{-4}. Wall friction is neglected. Friction is using Chézy with parameter C equal $3.743\,57 \times 10^1$ m$^{1/2}$/s, such that the normal flow depth, given by $h = (q^2/C^2/s)^{1/3}$, becomes 1 m. A grid with streamwise space step Δx of 20 m is employed. The bed level at the dowstream end of the domain (the cell edge) is −1 mAD, such that the water level boundary condition for obtaining perfect normal flow is 0 mAD at the cell edge. This condition is obtained by setting the water level at the ghost cell equal to $s\Delta x$ = −0.014 27 (`izbndpos` = 0).

Figure 9.3 shows the cell-centre velocity of the idealized case described above when using and not using the bed level extrapolation. Figure 9.4 shows the flow depth at velocity points and Figure 9.5 the flow depth at cell centres. When extrapolating the bed level, the flow depth at velocity points and the velocity at cell centres (and velocity points) are equal to the desired normal-flow case values. However, the flow depth at cell centres is smaller. This is because the D-Flow FM numerical scheme uses at the cell interfaces the upwind water level. If the water depth at the interface matches the normal-flow water depth, then the water depth at the upstream cell centre will be smaller since the same water level is combined with a higher cell-centre bed level.

[1] The bed levels are extrapolated in line with the internal slope at water level boundaries, but with the reverse slope at velocity/discharge boundaries since the model uses the water level of the internal grid cell at the open boundary in the latter case.

[2] One would expect that the boundary condition should only be reduced by half that value, but the water level at the cell interfaces is determined by the value of the upwind cell centre. The water level to be specified at the ghost cell should be consistent with that trend.

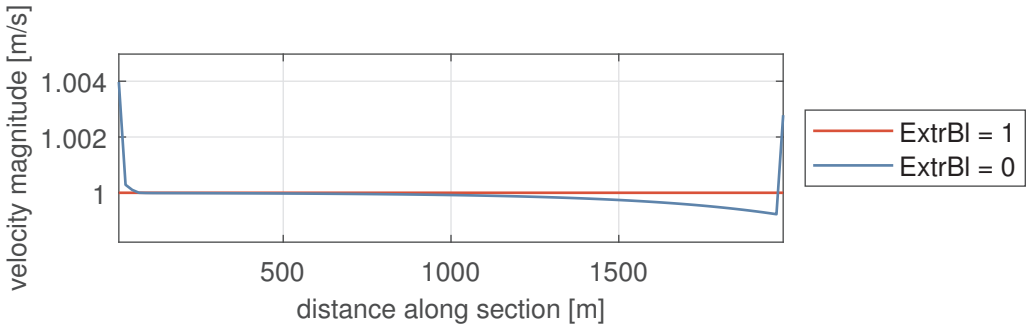


Figure 9.3: *Cell-centre velocity magnitude along the streamwise direction of an idealized normal-flow case.* `ExtrBl = 1` *extrapolates the bed levels at the boundaries and perfect normal flow is obtained.*

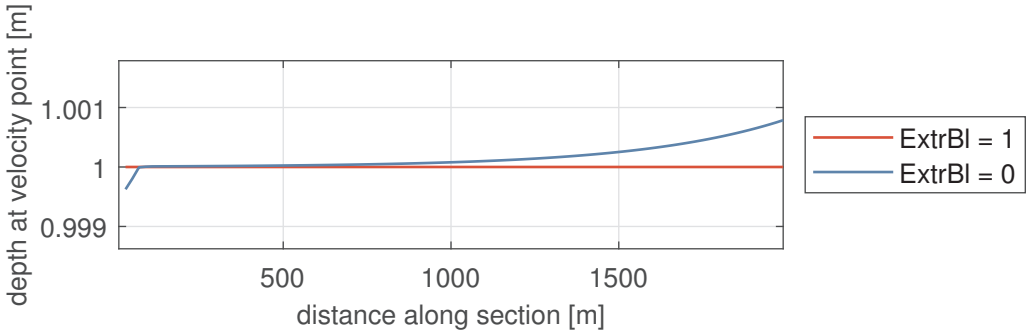


Figure 9.4: *Flow depth at velocity points along the streamwise direction of an idealized normal-flow case.* `ExtrBl = 1` *extrapolates the bed levels at the boundaries and perfect normal flow is obtained.*

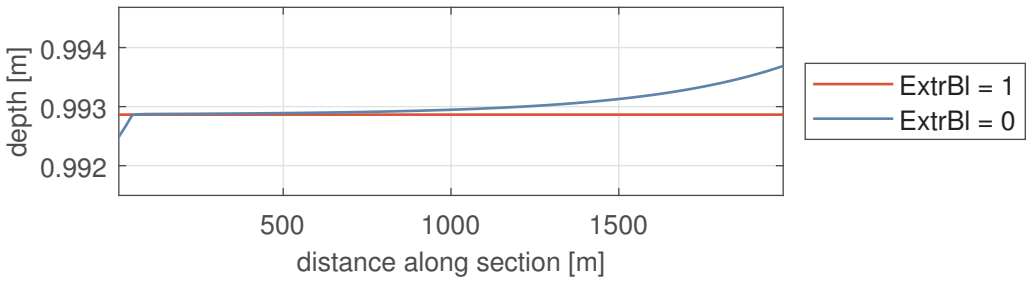


Figure 9.5: *Cell-centre flow depth along the streamwise direction of an idealized normal-flow case.* `ExtrBl = 1` *extrapolates the bed levels at the boundaries and perfect normal flow is obtained.*

10 Hydraulic structures

Obstacles in the flow may generate sudden transitions from flow contraction to flow expansion. The grid resolution is often low compared to the gradients of the water level, the velocity and the bathymetry. The hydrostatic pressure assumption may locally be invalid. Examples of these obstacles in civil engineering are: gates, barriers, dams, groynes and weirs. The obstacles generate energy losses and may change the direction of the flow.

The forces due to obstacles in the flow which are not resolved (sub-grid) on the horizontal grid, should be parameterised. The obstacles are denoted in D-Flow FM as hydraulic structures. In the numerical model the hydraulic structures should be located at velocity points of the grid. The direction of the forces should be specified at input. To model the force on the flow generated by a hydraulic structure, a quadratic energy or linear loss term is added to the momentum equation.

Note: Structure definitions can be made in Delta Shell, and will then be saved into a structures $<*$.ini$>$-file (section C.13).

The user can insert the hydraulic structures by the means of polygons on the grid. By selecting the required structure and drawing a polygon on the computational grid, the location of structure can be defined (see Figure 10.1). The supported structures in D-Flow FM are

- Fixed Weirs
- Dambreaks
- Gates
- (Adjustable) general structures, weirs, gates, orifices, and universal weirs
- Culverts
- Long culverts
- Bridges (in 1D)
- Pumps
- Compound structures
- Thin dams

In practice, the word 'barrier' is often used for structures. However, in D-Flow FM such structures are modelled as a gate or a weir in combination with a so-called Control Group (D-Real Time Control model).

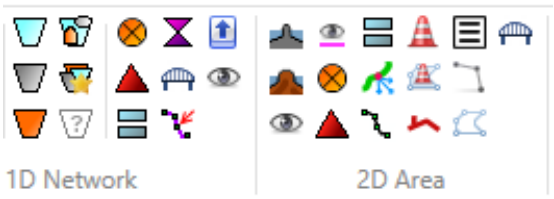


Figure 10.1: *Selection of structures (and other items) in the toolbar.*

DRAFT

10.1 Fixed weirs

In D-Flow FM, a fixed weir is a fixed non-movable construction generating energy losses due to constriction of the flow. They are commonly used to model sudden changes in depth (roads, summer dikes) and groynes in numerical simulations of rivers. Such structures are applied to keep the river in its bed for navigation purposes. Others are built, for instance, to protect an area behind a tidal weir from salt intrusion. Upstream of a weir the flow is accelerated due to contraction and downstream the flow is decelerated due to expansion. The expansion introduces an important energy loss due to turbulence. The energy loss is dependent on the shape of the weir and its crest level.

Fixed weirs are located in D-Flow FM on the velocity points of the computational grids. For a description of the input of fixed weirs, we refer you to section 6.4.2.5 and C.9.

In D-Flow FM, three different approaches are applied to simulate the energy losses by fixed weirs. First of all, a numerical approach has been implemented. Then, a special discretization of the advective terms around the fixed weir is applied. For a detailed description we refer to Deltares (2026a, section 6.7.1 – 6.7.4).

Next to the numerical approach, there is an empirical approach. Based on measurements in flume laboratories empirical formulae can be derived in order to match the measurements as accurately as possible. In this empirical approach, the energy loss due to a weir is described by the loss of energy height ($[m]$). The energy loss in the direction perpendicular to the weir is denoted as ΔE. This energy loss is added as an opposing force in the momentum equation by adding a term $-g\Delta E/\Delta x$ to the right hand side of the momentum equation, resulting in a jump in the water levels by ΔE at the location of the weir. For the computation fo the energy loss ΔE two options are availabe in D-Flow FM, namely the so-called 'Tabellenboek' and 'Villemonte' approaches. The two corresponding empirical formulas have been taken from the Simona software suite. For a detailed description of both formulas we refer to Deltares (2026a, section 6.7.5 – 6.7.6). In this empirical approach the discretization of the advective terms does not change, unlike the numerical approach in D-Flow FM for modelling fixed weirs.

The third approach is only available for fixed weirs that are located on 1D2D connections. For this approach a weir formula is used to compute the flow over the fixed weir. For a detailed description of this approach see Deltares (2026a, section 6.9.3). This approach can be turned on by setting `FixedWeirScheme1d2d=1` under the paragraph `[numerics]` in the MDU-file. **Note:** The non-1D2D fixed weirs will not be affected by this keyword.

10.2 Dam break

The dam break is a structure that models a growing breach after a dam failure or levee breach. A dam break is not really a hydraulic structure, but since it shares several properties with other structures (in terms of input and output), it is included here. The calculation of flow through a breach is based on a fixed weir (section 10.1), which models energy losses across the crest. Therefore, one important requirement of a dam break is that, a fixed weir polyline must be present on the same location as the dam break polyline. The dam break structure lowers the crest level and widens the breach width over time.

The input file format for a dam break is described in section C.13.10 and the output options in section F.3.1.9.8.

In this section, firstly, the geometrical grid snapping of dam break polylines to 2D grid lines is discussed. Then the computation of breach growth of a dam break is discussed. This includes two empirical formulas for computing breach growth: Verheij–van der Knaap (2002) and van

DRAFT

der Knaap(2000).

10.2.1 Grid snapping of a dam break

The potential location of a dambreak is defined by an input dam break polyline. This should coincide with (part of) a fixed weir polyline. When initializing a model, the geometrical snapping of these polylines to the grid cell edges is identical for both. The rule of this snapping is, if the polyline intersects with a 2d flow link or a 1d2d lateral flow link, then it is snapped to the cell edge that intersects with this flow link.

The user defines the start location of the dam breach by means of providing the coordinates (`startLocationX`, `startLocationY`). The actual starting point of the dam breach is determined by the following steps:

1 The nearest point on the dam break polyline to the start location is determined, by projecting the start location onto the polyline.
2 For all flow links in the dam break, the intersection of these flow links with the dambreak polyline are computed.
3 The flow link with its intersection that is closest to the the projected start location will be used as the actual midpoint location of the dam breach.

More details are available in the section about fixed weir snapping in (Deltares, 2026a, section 6.7.7). Note that a start location for a breach that is placed in the middle of a dam break, may not be located in the middle after snapping. This is most obvious when a dam break snaps to an even number of flow links/grid edges equal lengths. This is of particular importance when the breach width grows to about the full width of the dam break. See section 10.2.2.4 for more details.

Fixed weirs and dambreaks on a 2D grid

To correctly model the crest width of a fixed weir (and the breach width), the flow link widths at the grid-snapped polylines are adjusted. Assume that the fixed weir polyline intersects with a flow link j', associated with cell edge j. Then flow link width w_{u_j} is computed as:

$$w_{u_j} = \cos(\alpha)\ ||\textit{Edge}||_j, \tag{10.1}$$

where

w_{u_j}	is the flow link width,
$\|\|\textit{Edge}\|\|_j$	is the original length of cell edge j (i.e., the original width of flow link j'), and
α	is the angle between the fixed weir polyline and cell edge j.

In other words, w_{u_j} is the projection of the original width of edge j onto the fixed weir polyline. As a result, when summing w_{u_j} for all affected flow links j', the total width is similar to the length of the polyline.

Fixed weirs and dambreaks on 1D2D flow links

In case a fixed weir and/or a dambreak intersects a 1D2D flow link (lateral or embedded), no projection is applied. As a result the flow width w_{u_j} of this flow link is used. The calculation of the flow width of a 1D2D lateral or embedded flow link is described in Section 17.7.3.

DRAFT

10.2.2 Computation of breach growth

Two formulas are available to compute the breach growth of a dam break in D-Flow FM: Verheij–Van der Knaap(2002) and Van der Knaap (2000). These are discussed first. Alternatively, breach width and depth may be provided by the user as a time series. Finally, the details of a breach width spanning multiple flow links is discussed.

Both breach growth formulas regard the process of breach growth as two phases:

- ◇ **Phase 1**: during which the breach level, for a constant initial breach width, is lowering from its initial crest level z_{crest} (or dike level) to its minimal crest level z_{min}. In other words, this phase is a "lowering" phase. The breach level decreases while the breach width stays unchanged.
- ◇ **Phase 2**: during which the breach only grows in width under a certain condition depending on the formula in use. The Verheij–Van der Knaap (2002) formula increases the breach width based on the water level jump, as long as the flow velocity through the breach is larger than the critical flow velocity for eroding sediment/soil particles u_c. The Van der Knaap (2000) formula computes the breach width as a function of time and limits it by a maximal breach width. In other words, this phase is a "widening" phase. The breach level stays at the minimal level and the breach width increases.

10.2.2.1 The Verheij–Van der Knaap (2002) formula

One can choose to use the Verheij–Van der Knaap (2002) formula by setting the input parameter `algorithm=2` in Table C.23. The formula is as follows:

Phase 1:

For $T_{\text{start}} \leq t \leq T_{\text{start}} + t_{\text{Phase 1}}$

$$B(t) = B_0, \tag{10.2}$$

$$z(t) = z_{\text{crest}} - \left(\frac{t - T_{\text{start}}}{t_{\text{Phase 1}}}\right)(z_{\text{crest}} - z_{\text{min}}). \tag{10.3}$$

Phase 2:

For $t > T_{\text{start}} + t_{\text{Phase 1}}$ (hence $B(t) \geq B_0$):

$$z(t) = z_{\text{min}}, \tag{10.4}$$

$$B(t_{i+1}) = B(t_i) + \frac{\Delta t}{3600}\left(\frac{\partial B}{\partial t}\right)_{t_{i+1}}, \tag{10.5}$$

where

$$\left(\frac{\partial B}{\partial t}\right)_{t_{i+1}} \begin{cases} \frac{f_1 f_2}{\ln(10)} \frac{(g(h_{\text{up}} - \max(h_{\text{down}}, z_{\text{min}})))^{3/2}}{u_c^2} \frac{1}{1 + \frac{f_2\ g\ (t_{i+1} - t_1)}{3600\ u_c}} & \text{if} \quad |u| > u_c \\ 0 & \text{if} \quad |u| \leq u_c \end{cases}. \tag{10.6}$$

The notations in the above formula are:

f_1	(input parameter) Factor 1: constant factor [$-$],
f_2	(input parameter) Factor 2: constant factor [$-$],
$B(t)$	Width of the dam breach at time t [m],
B_0	(input parameter) Initial width of the dam breach [m],

DRAFT

$\left(\frac{\partial B}{\partial t}\right)_{t_i+1}$ Rate of breach width growth in time-step Δt from t_i to t_{i+1} [m/s],
g Acceleration due to gravity [m/s^2],
h_{up} Upstream water level at time t [m AD],
h_{down} Downstream water level at time t [m AD],
t Current model time [s],
$t_{\text{Phase 1}}$ (input parameter) Duration of phase 1, the time period over which the breach for a constant initial width (B_0) is lowered from its initial crest level z_{crest} (or dike level) to its minimal crest level z_{min} [s],
$t_1 = T_{\text{start}} + t_{\text{Phase 1}}$ The time [s] when the maximum breach-depth (z_{min}) is reached, i.e. when the Phase 1 finishes,
T_{start} (input parameter) Model time at which the development of the initial breach starts [s],
u Average flow velocity in the breach [m/s],
u_c (input parameter) Constant critical flow velocity for eroding sediment/soil particles.
$z(t)$ Crest level of the dam breach at time t [m AD],
z_{crest} (input parameter) Initial breach level at $t = T_{start}$ [m AD],
z_{min} (input parameter) Minimal breach level at $t = t_1$ [m AD],
Δt Computational time step [s], equal to $t_{i+1} - t_i$.

Settings of the input parameters can be found in section C.13.10.

Upstream and downstream water levels (h_{up} and h_{down})

By default the upstream and downstream water levels are computed by averaging the water-levels upstream and downstream of the dambreak. Only grid cells that are connected to a flow link that is (partly) opened by the dambreak are included in the averaging. For the averaging of upstream and downstream water level the original flow width of the open flow links is used as a weight factor.

Another possibility is to define custom up and downstream water level locations. In Section C.13.10 the input settings for this option can be found.

Recommended values of input parameters in the Verheij–Van der Knaap (2002) formula

To use proper values, a list of recommended values as well as the range of values is given in Table 10.1.

Table 10.1: *Recommended values of input parameters in the Verheij–Van der Knaap (2002) formula.*

Parameter	recommended value	Range
f_1	1.3	0.5 - 5
f_2	0.04	0.01 - 1
B_0	10	1 - 100 m
$t_{\text{Phase 1}}$	0.1 hours	0.1 - 12 hours
u_c	0.2 m/s	0.1 - 10 m/s
z_{crest}	-	-
z_{min}	-	-

DRAFT

Characterising soil strength

As a recommendation, the parameter u_c, applied in the Verheij–Van der Knaap (2002) formula may be derived from Table 10.2, where τ_c is the critical shear stress.

Table 10.2: *Recommended values for u_c and τ_c.*

Soil type	**Bodemtype (Dutch)**	u_c [m/s]	τ_c [Pa]
grass, good	gras, goed	7	185
grass, moderate	gras, matig	5	92.5
grass, bad	gras, slecht	4	62
clay, very good (compacted)	klei, zeer goed (compact; $t_{\text{ongedraineerd}}$ = 80 – 100 kPa)	1.0	4
clay, good (firm)	klei met 60 % zand, (stevig; $t_{\text{ongedraineerd}}$ = 40 – 80 kPa)	0.8	2.5
clay, moderate (little structure)	goede klei met weinig structuur	0.7	2
clay moderate (considerable structured)	goede klei, sterk	0.6	1.5
clay, bad (weak)	slechte klei (slap; $t_{\text{ongedraineerd}}$ = 20 – 40 kPa)	0.4	0.65
sand with 17 % silt	zand met 17 % silt	0.225	0.20
sand with 10 % silt	zand met 10 % silt	0.20	0.15
sand with 0 % silt	zand met 0 % silt	0.16	0.10

Another way to compute u_c is using the critical shear stress τ_c. For example, if τ_c is known, then u_c can be calculated by

$$u_c = 0.5\sqrt{\tau_c}, \tag{10.7}$$

or

$$u_c = u_{c,\text{sand}}(1 + 0.01\alpha P_{\text{clay}}) + \beta(0.65 - v), \tag{10.8}$$

where using the defaults, factor $\alpha = 15$, factor $\beta = 1$, critical flow rate for sand $u_{c,\text{sand}} = 0.2$ m/s, P_{clay} is percentage of clay, and the voids ratio v can be computed by

$$v = n/(1 - n), \tag{10.9}$$

where n is the soils pore fraction (default $n = 0.4$).

For more details of formula Verheij–Van der Knaap (2002) we refer to Verheij (2002).

10.2.2.2 The Van der Knaap (2000) formula

One can choose to use the Van der Knaap (2000) formula by setting the input parameter `algorithm=1` in Table C.23. The formula is as follows:

Phase 1:

For $T_{\text{start}} \leq t \leq T_{\text{start}} + t_{\text{Phase 1}}$

$$B(t) = B_0, \tag{10.10}$$

$$z(t) = z_{\text{crest}} - \left(\frac{t - T_{\text{start}}}{t_{\text{Phase 1}}}\right)(z_{\text{crest}} - z_{\text{min}}). \tag{10.11}$$

Phase 2:

For $t > T_{\text{start}} + t_{\text{Phase 1}}$ (hence $B(t) \geq B_0$):

$$z(t) = z_{\text{min}}, \tag{10.12}$$

$$B(t) = \min\left(c_1 \log_{10}(\frac{t}{c_2}), B_{\text{max}}\right). \tag{10.13}$$

In Equation (10.13), the coefficients c_1, c_2 and allowed maximal breach width B_{max} depend on the material of the dam. Two types of materials are possible: clay and sand. (Currently only clay is supported in D-Flow FM.) The values of these coefficients are shown in Table 10.3.

Table 10.3: *Values of coefficients c_1, c_2 and B_{max} of the Van der Knaap (2000) formula for different material types.*

Material type	c_1	c_2	B_{max}
clay	20	288	75 m
sand	67	522	200 m

Other notations in the above formula are:

$B(t)$	Width of the dam breach at time t [m],
B_0	(input parameter) Initial width of the dam breach [m],
t	Current model time [s],
$t_{\text{Phase 1}}$	(input parameter) Duration of phase 1, the time period over which the breach for a constant initial width (B_0) is lowered from its initial crest level z_{crest} (or dike level) to its minimal crest level z_{min} [s],
T_{start}	(input parameter) Model time at which the development of the initial breach starts [s],
$z(t)$	Crest level of the dam breach at time t [m AD],
z_{crest}	(input parameter) Initial breach level at $t = T_{\text{start}}$ [m AD],
z_{min}	(input parameter) Minimal breach level at $t = t_1$ [m AD],

Settings of the input parameters can be found in section C.13.10.

For more details about formula Van der Knaap (2000) we refer to Van der Knaap (2000).

10.2.2.3 Breach crest level and width time series

One can choose to explicitly prescribe the breach's crest level and width as time series by setting the input parameter `algorithm=3` in Table C.23. The provided time series must be in a 3-column <*.tim> file (in minutes), specified by parameter `dambreakLevelsAndWidths`. The times in the file are relative to T_{start} (`t0`, in seconds).

DRAFT

10.2.2.4 Method when breach grows wider than one flow link

The breach starts at only one flow link, which is defined by setting `startLocationX` and `startLocationY` in the input, see section C.13.10. The breach widths during the simulation are computed by either the formulas for breach growth or by a time series. At a certain time, if the breach width is wider than the starting flow link, D-Flow FM must determine the affected links so that the width grows, from the starting flow link, to gradually include more edges. How the breach extends beyond the starting flow link is determined by the `breachGrowth` keyword in the <mdu-file>; three options have been implemented.

- ◇ `symmetric` In this case the breach grows symmetrically on both left and right sides of the breach. This is the original implementation. This approach triggers a warning when the breach reaches the end of the snapped dam break on *one* of the sides.
- ◇ `symmetric-asymmetric` In this case the breach is assumed to grow symmetrically on both left and right sides of the breach until it reaches the end of the dam break on one of the sides; thereafter it only growth on the other side. This is the current default. This approach triggers a warning only when the breach width is larger than the full length of the snapped dam break.
- ◇ `proportional` In this case the breach is assumed to grow proportionally to the available width on the two sides of the breach. This approach triggers a warning only when the breach width is larger than the full length of the snapped dam break.

Given the breach width on one of the sides, for instance a width on the left side w_{left}, links on the left side are opened one by one in sequence gradually moving further away from the starting flow link:

- ◇ **Situation 1:** If w_{left} is smaller than the width of the considered flow link, then this is end of the searching.
- ◇ **Situation 2:** If w_{left} is larger or equal to the width of the considered flow, the value of w_{left} is decreased by this flow link width and the process continues with the next flow link that is on the left side of this link. This is repeated until Situation 1 is achieved, or the end of the dam break is reached.

In both two situations, the width and bed level of the considered link is affected. The bed level of this link is adjusted to the maximal value of its original bed level and the crest level of the dam break. The width of this link is adjusted to the effective dam breach width, i.e. the minimum of the remaining breach width w_{left} and the width of the flow link.

10.3 General structure

The General structure combines weir and gate flow into one structure. Additionally, the General structure gives the highest degree of freedom in defining the the geometry of the hydraulic structure and coefficients affecting the flow. The geometrical shape is given in Figure 10.2 and Figure 10.3. The discharge through a General structure is computed based on upstream and downstream energy levels. The input file format for a general structure is described in section C.13.9.

DRAFT

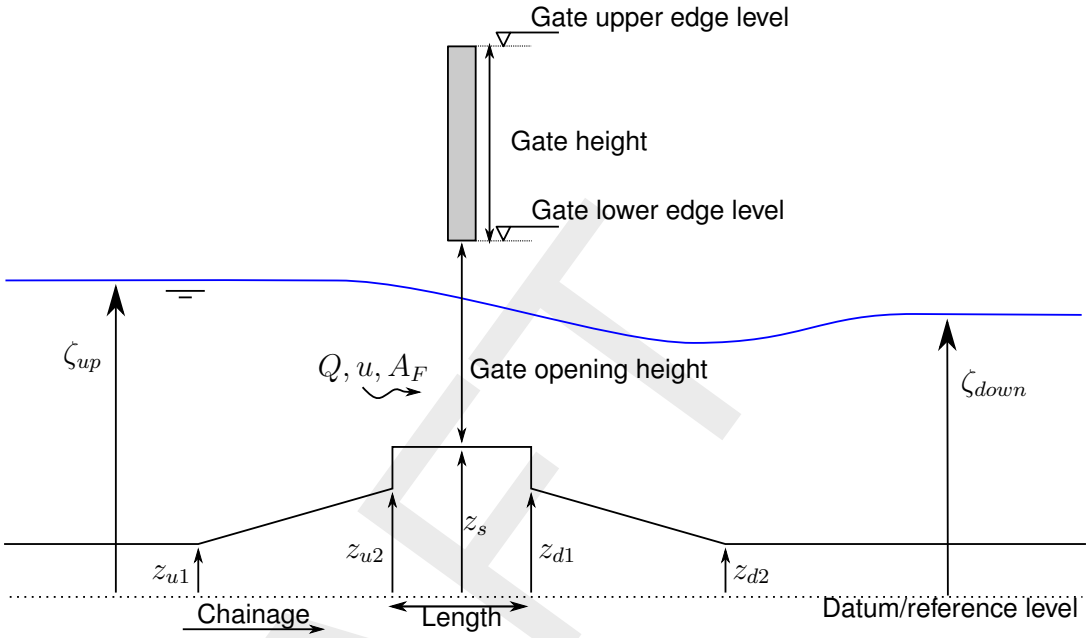


Figure 10.2: *General structure, side view*

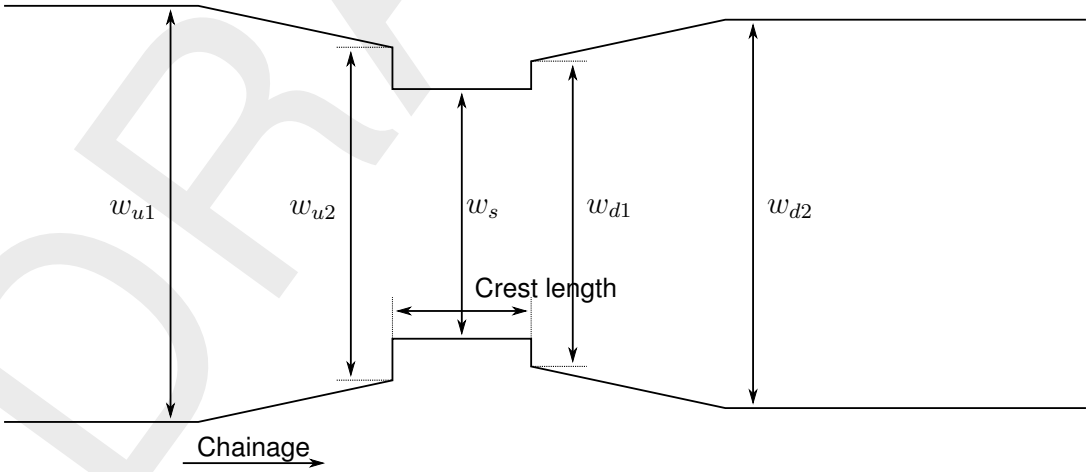


Figure 10.3: *General structure, top view*

The gate of the General structure can be configured as two doors with an opening in between as shown in Figure 10.4. When the modelled gate is a single door spanning the full width, the Gate Opening Width should be set to zero.

DRAFT

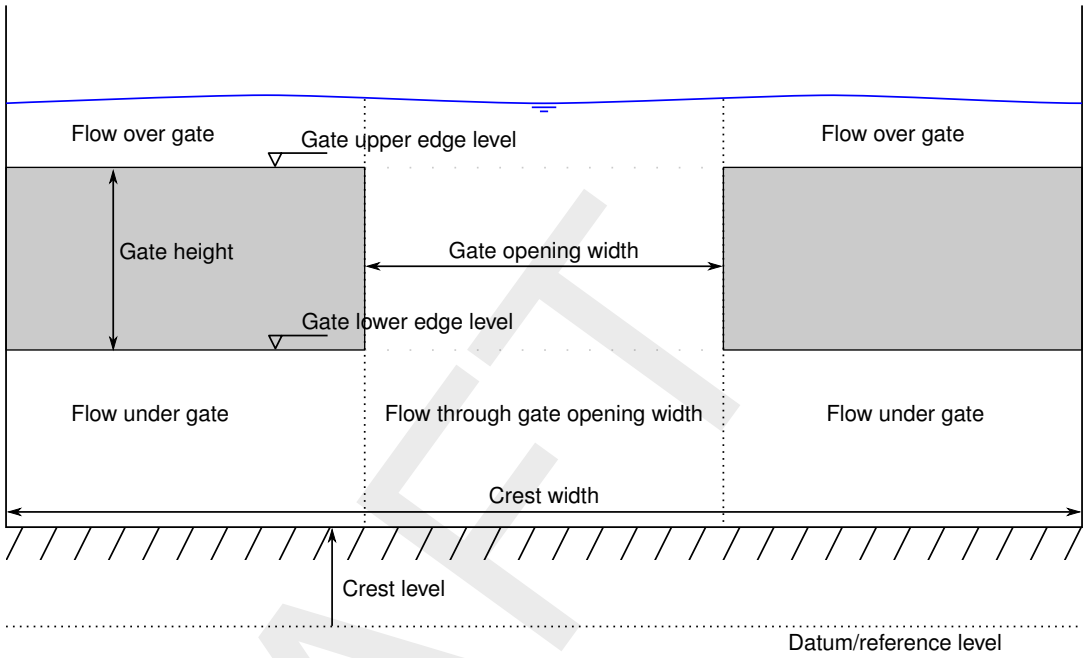


Figure 10.4: *General structure, front view*

Figure 10.4 also shows the three different geometrical parts where the General structure can have flow: under gate flow, over gate flow and between gate flow. The total discharge through the structure is calculated by adding up the discharge through the separate parts. For each part of the structure opening the downstream waterlevel is calculated and subsequently the flow type (gate or weir, free or submerged) is determined.

Remark:

- ◇ Please keep in mind that in the following part of this section the description for the General structure applies to the three different openings (with each its own flow conditions).

During the simulation a number of checks and possibly corrections on some dimensions of the general structure are performed.

- ◇ If the **`crestLevel`** is below the bed level of the surrounding water level points in the channel, the `crestLevel` (z_s) is raised to the highest bed level of these two points[1].
- ◇ If z_{u1} and/or z_{u2} are below the bed level of the upstream water level point in the channel, z_{u1} and/or z_{u2} are raised to the bed level of the upstream water level point in the channel.
- ◇ If z_{d1} and/or z_{d2} are below the bed level of the downstream water level point in the channel, z_{d1} and/or z_{d2} are raised to the bed level of the downstream water level point in the channel.
- ◇ All **widths** (w_s, w_{u1}, w_{u2}, w_{d1}, w_{d2}) are maximized with the corresponding flow width.
- ◇ If the **`gateLowerEdgeLevel`** is lower than the `crestLevel`, the `gateLowerEdgeLevel` is set to the `crestLevel`.
- ◇ If the **`gateHeight`** is less than or equal to 0 an error message is produced.
- ◇ If the **`gateOpeningWidth`** is less than 0, the `gateOpeningWidth` is set to 0.
  Else if the `gateOpeningWidth` is larger than the `crestWidth` then the `gateOpeningWidth` is set to the `crestWidth`.

[1] The bed level check is actually based on the face-based bed levels of the velocity point. In typical applications, this is equal to the maximum bed level of the two surrounding water level points. More information is available in (Deltares, 2026a, Equation (6.10)).

DRAFT

Flow across the General structure can be of the following types:

- ◇ free gate flow (i.e. supercritical gate flow),
- ◇ submerged gate flow (i.e. subcritical gate flow),
- ◇ free weir flow (i.e. supercritical weir flow), and
- ◇ submerged weir flow (i.e. subcritical weir flow).

The choice depends on the dimensions of the structure and the flow conditions. Whether or not the gate is in the flow or above the flow yields either gate or weir flow. Furthermore, the flow can be either subcritical (submerged) or supercritical (free). Both for incoming flow ("Flow") and for outgoing flow ("Reverse") contraction coefficients can be specified. These coefficients can be seen as tuning parameters for the user.

10.3.1 Notation

The used variables in this subsection are described in this paragraph.

The five geometric parameters as shown in Figure 10.2:

z_s	Crest level,
z_{u1}	Level upstream 1,
z_{u2}	Level upstream 2,
z_{d1}	Level downstream 1, and
z_{d2}	Level downstream 2

Similarly for the five values for the width of the General structure, see Figure 10.3:

w_s	Width at crest level,
w_{u1}	Width upstream 1,
w_{u2}	Width upstream 2,
w_{d1}	Width downstream 1, and
w_{d2}	Width downstream 2

Other variables used in the equations of this subsection:

C	Chézy value, representing the roughness at the downstream side of the General structure
c_{gf}	Correction coefficient for free gate flow
c_{gd}	Correction coefficient for submerged gate flow
c_{wf}	Correction coefficient for free weir flow
c_{wd}	Correction coefficient for submerged weir flow
E_1	Energy height upstream ($= \zeta_{up} + u^2/(2g)$)
g	Acceleration due to gravity
H_{s1}	Energy height with respect to the crest level ($= E_1 - z_s = \zeta_{up} + \frac{u^2}{2g} - z_s$)
h_g	Opening height of the gate ($= \texttt{gateLowerEdgeLevel} - z_s$)
L	Length of the expansion zone at the downstream side of the General structure
h_2	Water depth downstream of the structure with respect to the crest level ($= \zeta_{down} - z_{d2}$), see remark below.
ΔS_1	$= z_s - z_{d1}$
ΔS_2	$= z_{d1} - z_{d2}$
λ	Extra resistance parameter
ρ^*	$= \rho_2/\rho_1$
ρ_1	Density of the water at the upstream side of the General structure [kg/m^3]
ρ_2	Density of the water at the downstream side of the General structure [kg/m^3]
μ_{gf}	Contraction coefficient for free gate flow

DRAFT

μ_{gd} Contraction coefficient for submerged gate flow ($\mu_{gd} = \mu_{gf}$)

Remarks:

- ◇ Downstream water depth h_2 is limited to a minimal value of $0.9 \cdot \frac{2}{3} \cdot H_{s1} \cdot \left(\frac{w_s}{w_{d2}}\right)^{\frac{2}{3}}$.
- ◇ The effect of this limiter might result in an incorrect flow state. In case the downstream level z_{d2} is similar to the crest level z_s and the downstream width w_{d2} is similar to the crest width w_s, the general structure might not reach the free flow state.
- ◇ The energy heights E_1, H_{s1} can be set to ordinary water levels in case the flag `UseVelocityHeight` is set to `False`. For more information, see section 10.3.4.

10.3.2 Geometry

The crest level, gate lower edge level and gate door opening width can also be given as time series.

A (2D) hydraulic structure is given as a single geometrical object by the user, but it may cross multiple 2D grid cells (or: flow links). In case a General structure is positioned on a 1D branch, a structure crosses only 1 link. Each flow link under the influence of this structure (i.e. structure link) is treated as a General structure with the correct geometrical parameters.

Remark:

- ◇ In case a General structure crosses multiple 2d flow links, w_{u1} , w_{u2}, w_{d1} and w_{d2} are all set per individual link to the crest width per link ($w_s(L)$).

Crest width

The crest width of an individual flow link, under the influence of this structure, cannot exceed the width of this flow link. As a result, when the crest width exceeds the sum of the widths of the structure links, the crest width is reduced to the sum of the widths of the flow links. When the crest width is omitted in the input the structure width will be set to the sum of the widths of the structure links.

Remark:

- ◇ Keep in mind that in case a polyline has e.g. an angle of 45 degrees with the grid, the sum of the flow widths might be a factor of $\sqrt{2}$ larger than the polyline. Therefore it is recommended to align the grid with the structure.

In case the crest width is smaller than the sum of the widths of the structure links, the following algorithm is applied:

- ◇ Use the next definitions

$w_u(L)$	is the flow width of a structure link
N	is the total number of structure links in a structure
w_s	is the total crest width
$w_s(L)$	is the crest width of a structure link
w_{closed}	$= \sum_{L=1}^{N} w_u(L) - w_s$

DRAFT

◇ Determine smallest p and largest q where

$$\left(\sum_{L=1}^{p} w_u(L)\right) - 0.5 \cdot w_{closed} > 0 \tag{10.14}$$

$$\left(\sum_{L=1}^{q} w_u(L)\right) - 0.5 \cdot w_{closed} < 0 \tag{10.15}$$

◇ The crest widths of the structure links 1 until q will be set to 0 (i.e. closed)
◇ The crest width of structure link $p = q - 1$ will become:

$$w_s(p) = \left(\sum_{L=1}^{p} w_u(L)\right) - 0.5 \cdot w_{closed} \tag{10.16}$$

◇ A similar approach is applied to the right side of the structure.
◇ A a result the opening of the structure will be right in the middle of the set of structure links.

Example for the left part of the structure:
Assume the total crest width is 550 [m] (w_s), the total flow width is 1000 [m] and the number of structure links is set to 5 (N). So each structure link has a width of 200 is [m] ($w_u(L)$), the total flow width is indeed 1000 [m] ($\sum_{L=1}^{N} w_u(L)$) and thus w_{closed} is 450 [m]. According Equation (10.14) we get $p = 2$, and according Equation (10.15) we get $q = 1$. The partial open/closed structure link $w_s(p)$ has a crest width of 175 [m], according Equation (10.16) ($175 = 400 - 225$). The total crest width is: $w_s(2) + w_s(3) + w_s(4) = 175 + 200 + 175 = 550$ [m]. Taken into account the symmetry between left and right side of the structure.

Gate opening

The `GateOpeningWidth` is a compulsory parameter for General structures. A single gate door with no horizontal movement should use `GateOpeningWidth`=0. A structure that has one or two gate doors that do have horizontal opening and closing movement should specify the width of the horizontal opening beteen the doors.

Depending on the gate door positions, some structure links may have only flow across the weir crest in the opening, whereas others may have gate flow, or a combination of both. A similar algorithm for applying the gate door to the individual flow is performed as described for the crest width. There is an exception, because the door might not close symmetrical. The parameter `GateOpeningHorizontalDirection` defines the way a gate door is divided over the structure links. Possible values are `symmetric`, `fromLeft`, `fromRight`.

At both ends of the gate opening structure links may be partially opened in such a way that the `gateOpeningWidth` exactly matches the sum of the width of the open flow links.

Extra resistance

The extra resistance parameter λ relates to the friction force in the impulse balance, that in case of submerged gate flow or submerged weir flow is solved for the water movement at the downstream side of the General structure. The extra resistance parameter λ is given by the expression:

$$\lambda = \frac{gL}{C^2} \tag{10.17}$$

DRAFT

For an explanation of the variables see section 10.3.1.

Either the extra resistance parameter, λ, can be specified, or the length, L, can be specified by the user. In case length is used, the Chézy value of the flow link is used for calculating λ.

10.3.3 Computation of the downstream water level

In case of submerged gate flow or submerged weir flow the water level at the sill or downstream of the gate is required. This level is computed by application of the impulse balance.

Remark:

- The equations in this paragraph apply to flow in positive direction for flow in negative direction replace the next variables in the equations:
  - z_{d1} into z_{u2}
  - z_{d2} into z_{u1}
  - w_{d1} into w_{u2}
  - w_{d2} into w_{u1}

Submerged gate-flow

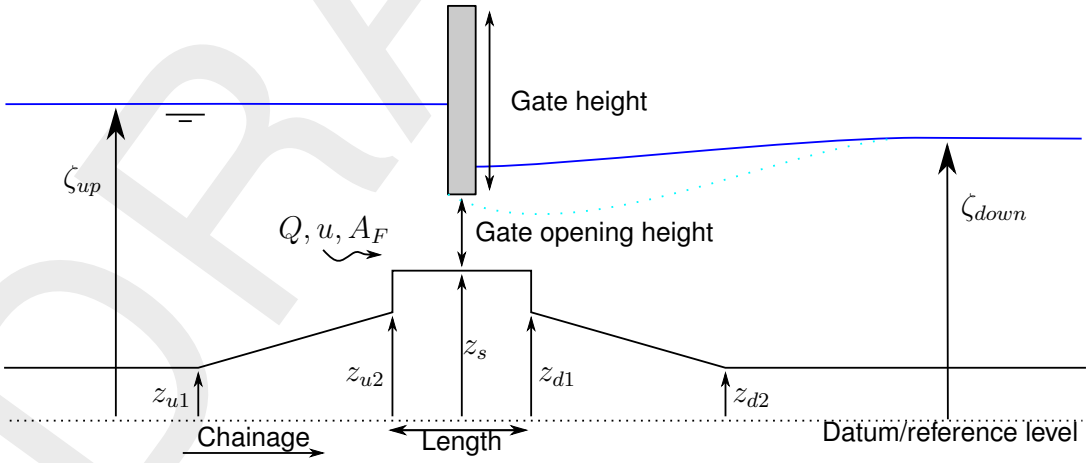


Figure 10.5: *Submerged gate flow*

The water depth on the crest h_s can be described by a second order algebraic equation:

$$A_g h_s^2 + B_g h_s + C_g = 0 \tag{10.18}$$

DRAFT

with:

$$A_g = (1+\rho^*)\left(\frac{w_{d1}}{3}+\frac{w_{d2}}{6}\right)+(1-\rho^*)\left(\frac{w_{d1}}{4}+\frac{w_{d2}}{12}\right) \tag{10.19}$$

$$\begin{aligned} B_g = {} & (1+\rho^*)\left((\Delta S_1+\Delta S_2-h_2)\left(\frac{w_{d1}}{3}+\frac{w_{d2}}{6}\right)+(2\Delta S_1+h_2)\frac{w_{d1}}{6}+\right. \\ & \left.(\Delta S_1+2h_2)\frac{w_{d2}}{6}\right)+ \\ & +(1-\rho^*)\left(\Delta S_1\frac{w_{d1}}{3}+(\Delta S_1+h_2)\frac{w_{d1}+w_{d2}}{6}\right)+ \\ & +\frac{4\rho^* c_{gd}^2\mu_{gd}^2 h_g^2 w_s^2}{w_{d2}h_2}(1+\frac{\lambda}{d_2})-4c_{gd}\mu_{gd}h_g w_s \end{aligned} \tag{10.20}$$

$$\begin{aligned} C_g = {} & (1+\rho^*)(\Delta S_1+\Delta S_2-h_2)\left(2\Delta S_1+h_2)\frac{w_{d1}}{6}+(\Delta S_1+2h_2)\frac{w_{d2}}{6}\right)+ \\ & +(1-\rho^*)\left((\Delta S_1)^2\frac{w_{d1}}{6}+(\Delta S_1+h_2)\frac{w_{d1}+w_{d2}}{12}\right)+ \\ & -\frac{4\rho^* c_{gd}^2\mu_{gd}^2 h_g^2 w_s^2 H_{s_1}}{w_{d2}h_2}(1+\frac{\lambda}{h_2})+4c_{gd}\mu_{gd}h_g w_s H_{s_1} \end{aligned} \tag{10.21}$$

For an explanation of the variables see section 10.3.1.

Equation (10.18) leads to:

$$h_s = \frac{-B_g+\sqrt{B_g^2-4A_gC_g}}{2A_g} \tag{10.22}$$

Submerged weir-flow

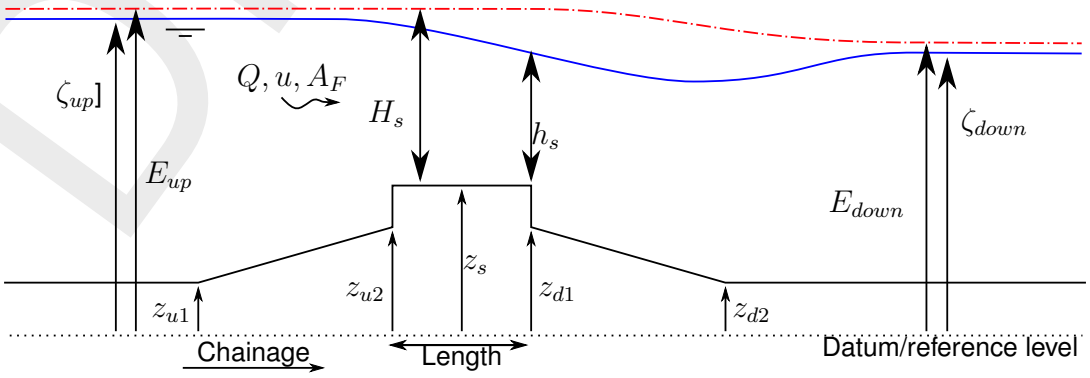


Figure 10.6: *Submerged weir-flow.*

The water depth at the sill h_s is described by a third order algebraic equation:

$$D_w h_s^3 + A_w h_s^2 + B_w h_s + C_w = 0 \tag{10.23}$$

DRAFT

with:

$$D_w = \frac{4\rho^* c_{wd}^2 w_s^2}{w_{d2} h_2}(1 + \frac{\lambda}{h_2}) \tag{10.24}$$

$$A_w = (1+\rho^*)\left(\frac{w_{d1}}{3} + \frac{w_{d2}}{6}\right) + (1-\rho^*)\left(\frac{w_{d1}}{4} + \frac{w_{d2}}{12}\right) + \\ - \frac{4\rho^* c_{wd}^2 w_s^2 H_{s_1}}{w_{d2} h_2}(1 + \frac{\lambda}{h_2}) - 4 c_{wd} w_s \tag{10.25}$$

$$B_w = (1+\rho^*)\left((\Delta S_1 + \Delta S_2 - h_2)\left(\frac{w_{d1}}{3} + \frac{w_{d2}}{6}\right) + (2\Delta S_1 + h_2)\frac{w_{d1}}{6} + \right. \\ \left. (\Delta S_1 + 2h_2)\frac{w_{d2}}{6}\right) + \\ + (1-\rho^*)\left(\Delta S_1 \frac{w_{d1}}{3} + (\Delta S_1 + h_2)\frac{w_{d1} + w_{d2}}{6}\right) + 4 c_{wd} w_s H_{s_1} \tag{10.26}$$

$$C_w = (1+\rho^*)(\Delta S_1 + \Delta S_2 - h_2)\left((2\Delta S_1 + h_2)\frac{w_{d1}}{6} + (\Delta S_1 + 2h_2)\frac{w_{d2}}{6}\right) + \\ + (1-\rho^*)\left((\Delta S_1)^2 \ \frac{w_{d1}}{6} + (\Delta S_1 + h_2)^2 \ \frac{w_{d1} + w_{d2}}{12}\right) \tag{10.27}$$

A direct method is applied to calculate h_s from equation Equation (10.23).

10.3.4 Turning off the computation of the energy levels

The input parameter `useVelocityHeight` in the <structures.ini> file (section C.13) can be used to change the default way of solving the flow through the structure. For certain situations the default setting might lead to incorrect results. The reason is that the general structure was developed for modelling structures in an open channel (river or rural applications), and the preceding approximations are based on certain assumptions. When applied to urban applications, some of these assumptions may not hold. Also important to realize is that the weir and orifice are based on the same formulations and assumptions.

The default setting, `useVelocityHeight=true`, is the situation as described in the previous section on water level computations. The upstream energy level is defined as

$$E_1 = \zeta_{up} + u^2/(2g) \tag{10.28}$$

and the downstream water depth (h_s) is determined as described in section 10.3.3.

The alternative setting, `useVelocityHeight=false`, simplifies the governing equations for E_1 and h_s. This can be of use when modelling a sewer system, where an overflow is also modelled as a general structure. Simple (gated) weir flow depending on simple water levels is preferred in such a case. The first difference compared to open water channels is that upstream horizontal velocities in an urban manhole compartment will typically be almost zero. The upstream energy height then simplifies to the upstream water level:

$$E_1 = \zeta_{up}. \tag{10.29}$$

The second difference is that the general structure's geometry simplifies to a long crest with $z_{u1} = z_{u2} = z_s = z_{d1} = z_{d2}$. The downstream water depth can no longer be derived from the accelaration across this (flat) crest, so it is simply based on downstream water level:

$$h_s = \zeta_{down} - z_s. \tag{10.30}$$

DRAFT

10.3.5 Discharge equations

The following discharge equations are applied during the computations.

◇ Free gate flow:

$$u_s = \mu_{gf} c_{gf} \sqrt{2g(E_1 - (z_s + \mu_{gf} h_g))} \tag{10.31}$$
$$A_f = w_s h_g \tag{10.32}$$
$$Q = u_s A_f = \mu_{gf} c_{gf} w_s h_g \sqrt{2g(E_1 - (z_s + \mu_{gf} h_g))} \tag{10.33}$$

◇ submerged gate flow:

$$u_s = \mu_{gd} c_{gd} \sqrt{2g(E_1 - (z_s + h_s))} \tag{10.34}$$
$$A_f = w_s h_g \tag{10.35}$$
$$Q = u_s A_f = \mu_{gd} c_{gd} w_s h_g \sqrt{2g(E_1 - (z_s + h_s))} \tag{10.36}$$
$$\tag{10.37}$$

◇ Free weir flow:

$$u_s = c_{wf} \sqrt{\frac{2}{3} g(E_1 - z_s)} \tag{10.38}$$
$$A_f = w_s \frac{2}{3}(E_1 - z_s) \tag{10.39}$$
$$Q = u_s A_f = c_{wf} w_s \frac{2}{3} \sqrt{\frac{2}{3} g} (E_1 - z_s)^{3/2} \tag{10.40}$$

◇ submerged weir flow:

$$u_s = c_{wd} \sqrt{2g(E_1 - (z_s + h_s))} \tag{10.41}$$
$$A_f = w_s h_s \tag{10.42}$$
$$Q = u_s A_f = c_{wd} w_s h_s \sqrt{2g(E_1 - (z_s + h_s))} \tag{10.43}$$

For an explanation of the variables see section 10.3.1.

Remarks:

◇ The contraction coefficient has a maximum value $\mu = 1.0$. In case the user specifies a higher value, a warning will be generated.

◇ For all the coefficients two values must be specified. One for flow in positive direction and another for flow in negative direction.

Determining the flow type

The flow type is determined by this algorithm:

◇ Determine h_s based on submerged weir flow equations

◇ Use critical weir flow depth $h_c = \frac{2}{3}(E_1 - z_s)$

◇ Test weir flow

- If ($h_s > h_c$ and $h_g > h_s$) then submerged weir flow
- Else if ($h_s < h_c$ and $h_g > h_c$) then free weir flow
- Else gate flow:

DRAFT

- The water depth at the sill h_s is recalculated using the submerged gate flow equations. The critical depth h_c is now defined as $\mu_{gf} h_g$.
  - If ($h_s < h_c$ or if h_s cannot be solved) then free gate flow
  - Else submerged gate flow

Remark:

- In case upstream water level is above gate lower edge level, there can still be **submerged weir flow** if $\zeta_s > h_c$ and $h_g > \zeta_s$ or **free weir flow** if $\zeta_s < h_c$ and $h_g > \zeta_s$.

10.4 Weir

The weir structure is a construction generating energy losses due to constriction of the flow. The dimensions of a weir are described by a crest level and a crest width. Weirs are used to regulate the flow in a waterway. A weir is treated as a general structure whose conceptual description can be found in section 10.3. The input file format for a weir is described in section C.13.1.

Table 10.4 lists the mapping of the weir input parameters to the general structure parameters as well as default values for the remaining general structure parameters.

Table 10.4: *Mapping table for weir input parameters to general structure parameters*

General structure parameters	Input variable/value for Weir
z_s	`crestLevel`
z_{u1}, z_{u2}	Upstream bedlevel of the channel
z_{d1}, z_{d2}	Downstream bedlevel of the channel
w_s, w_{u1}, w_{u2}, w_{d1}, w_{d2}	`crestWidth`
μ_{gf}, μ_{gd}	1.0
c_{gf}, c_{gd}, c_{wf}, c_{wd}	`corrCoeff`
`gateLowerEdgeLevel`	10^{10}
`gateHeight`	N/A
`gateOpeningWidth`	N/A

During the simulation a number of checks and possibly corrections on some dimensions of the weir are performed.

- If the given `crestLevel` is below the bedlevel of the surrounding waterlevel points in the channel, the `crestLevel` (z_s) is raised to the highest bedlevel of these two points (also see footnote 1).
- If the given `crestWidth` is larger than the corresponding maximum total flow width of the underlying flow link(s) then the `crestWidth` is set to that maximum flow width.

DRAFT

10.5 Gates

Constructions that partially block the horizontal flow can be modelled as so-called "gates". Its horizontal and vertical position can be specified. Upstream of the gate the flow is accelerated due to contraction and downstream the flow is decelerated due to expansion. A gate may include two type of openings, namely, in horizontal and in vertical directions. In two-dimensional simulations, the vertical effect is parameterized by a quadratic energy loss term.

The horizontal effect is mimicked by setting the velocities of the computational faces (at position of the gate) to zero. This generates structure of the horizontal flow around the gate which is more realistic. There is no transport of salt or sediment through the blocked computational faces of a gate. The width of a gate is an user defined parameters. If the user does not define it, the width is assumed to be zero, so it has no influence on the water volume.

In D-Flow FM the gates can be imposed by polygons, and can be edited in a similar way as the other structures. For more details on gates in the User Interface, we refer you to section 6.4.2.10. In Figure 10.7 the geometric parameters of a gate are shown.

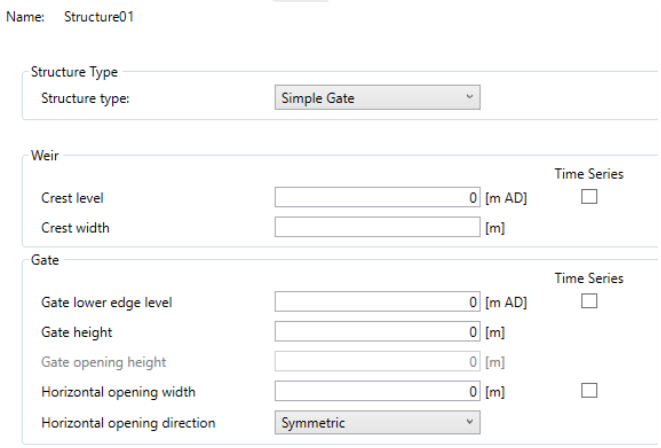


Figure 10.7: *Input for a gate*

The structure of the input file for the gates is as follows:

```
type                            = gate
id                              = gate01
locationFile                    = gate01.pli
crestLevel                      = 7
gateLowerEdgeLevel              = 15
gateOpeningWidth                = gate01_opening_width.bc
gateHeight                      = 5
gateOpeningHorizontalDirection= symmetric
crestWidth                      =
```

10.6 Orifice

The orifice structure can be used to simulate rectangle-shaped gates through which water within a channel or river is led. Orifices are used to regulate the water flow through or towards a channel or river. An orifice is treated as a general structure whose conceptual description can be found in section 10.3. The input file format for an orifice is described in section C.13.7.

Table 10.5 lists the mapping of the orifice input parameters to the general structure parameters as well as default values for the remaining general structure parameters.

Table 10.5: *Mapping table for orifice input parameters to general structure parameters*

General structure parameters	Input variable/value for Orifice
z_s	`crestLevel`
z_{u1}, z_{u2}	Upstream bedlevel of the channel
z_{d1}, z_{d2}	Downstream bedlevel of the channel
w_s, w_{u1}, w_{u2}, w_{d1}, w_{d2}	`crestWidth`
μ_{gf}, μ_{gd}	1.0
c_{gf}, c_{gd}, c_{wf}, c_{wd}	`corrCoeff`
`gateLowerEdgeLevel`	`gateLowerEdgeLevel`
`gateHeight`	10^{10}
`gateOpeningWidth`	0.0

During the simulation a number of checks and possibly corrections on some dimensions of the orifice are performed.

- ◇ If the given `crestLevel` is below the bed level of the surrounding water level points in the channel, the `crestLevel` (z_s) is raised to the highest bed level of these two points (also see footnote 1).
- ◇ If the given `crestWidth` is larger than the corresponding maximum total flow width of the underlying flow link(s) then the `crestWidth` is set to that maximum flow width.
- ◇ If the given `gateLowerEdgeLevel` is below the `crestLevel`, the `gateLowerEdgeLevel` is set to that `crestLevel`.

Flow limiter

The total flow through an orifice can be limited using the input keywords `(use)LimitFlowPos` and `(use)LimitFlowNeg` (in positive and negative direction, respectively). In this case the discharge through this structure is limited to the maximum specified discharge. When an orifice is located on a 2D grid, the flow limiter is applied for each flow link individually. The maximum discharge through each individual orifice link is weighted by the individual sill width of a link ($w_s(L)$) divided by the total width at the sill (w_s):

$$Q_{\text{lim}}(L) = \min(Q(L), Q_{\text{lim}} \cdot w_s(L)/w_s), \tag{10.44}$$

where notation from section 10.3.2 is reused. As this limitation per link based on width is only an approximation, this can mean that $\sum_{L=1}^{N} Q_{\text{lim}}(L) \leq Q_{\text{lim}}$.

10.7 Culvert

The culvert is an underground structure that normally connects two open channels. The flow through a culvert is affected by its upstream and downstream invert levels (z_{c1} and z_{c2}, respectively), its length, the size and shape of its closed cross section, its inlet loss, its friction loss, its valve loss and its outlet loss. The input file format for a culvert is described in section C.13.3.

Figure 10.8 shows a side view of a culvert.

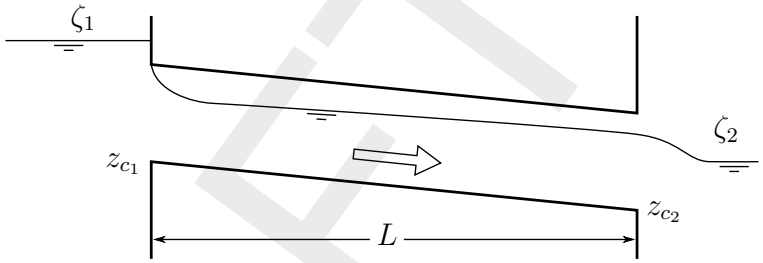


Figure 10.8: *Side view of a culvert*

Two flow conditions can occur:

Free flow when $\zeta_2 < z_{c2} + h_{c2}$

$$Q = \mu A_{fc} \sqrt{2g(\zeta_1 - (z_{c2} + h_{c2}))} \tag{10.45}$$

Submerged flow when $\zeta_2 \geq z_{c2} + h_{c2}$:

$$Q = \mu A_{fc} \sqrt{2g(\zeta_1 - \zeta_2)} \tag{10.46}$$

Where:

Q	Discharge through culvert [m^3/s]
μ	Discharge coefficient, derived from loss-coefficients [$-$]
A_{fc}	Discharge culvert flow area $\min(A_{fc1}, A_{fcvalve})$ [m^2]
	A_{fc1}: Flow area in the culvert at its upstream side [m^2]
	$A_{fcvalve}$: Flow area under the culvert valve [m^2]
g	Acceleration due to gravity [m/s^2] (≈ 9.81)
ζ_1	Upstream water level [m]
ζ_2	Downstream water level [m]
z_{c2}	Downstream culvert invert level [m]
h_{c2}	Critical culvert depth at the downstream side, $\sqrt[3]{Q^2/(gT_2^2)}$ [m]
T_2	Surface width in the culvert at its downstream side [m]

$$T_2 = \frac{A(h_{c2})}{h_{c2}} \tag{10.47}$$

$A(h_{c2})$	Flow area of the culvert at water depth (h_{c2}) [m^2]

DRAFT

Discharge coefficient (μ)

For numerical reasons the discharge coefficient (μ) is limited to a maximum of 1.0. The discharge coefficient (μ) is computed as follows:

$$\mu = \frac{1}{\sqrt{\xi_i + \xi_f + \xi_v + \xi_o}} \tag{10.48}$$

Where:

ξ_i	Inlet loss coefficient [$-$]
ξ_f	Friction loss coefficient [$-$]
ξ_v	Valve loss coefficient [$-$]
ξ_o	Outlet loss coefficient [$-$]

The inlet loss coefficient (ξ_i) can be defined as a constant value only.

The friction loss coefficient (ξ_f) is computed as follows:

$$\xi_f = \frac{2gL}{C_1^2 R} \tag{10.49}$$

Where:

L	Length of the culvert [m]
C_1	Chézy coefficient in the culvert at its upstream side [m$^{1/2}$/s]
R	Hydraulic radius [m]

$$R = \begin{cases} R_{c1} & \zeta_1 \geq z_{c1} + h_{valve} \\ R_{valve} & \zeta_1 < z_{c1} + h_{valve} \end{cases} \tag{10.50}$$

h_{valve}	Valve opening height [m]
R_{c1}	Hydraulic radius in the culvert at its upstream side [m]
R_{valve}	Hydraulic radius based on actual valve opening height [m]
z_{c1}	Upstream culvert invert level [m]

The valve loss coefficient (ξ_v) can be defined as a function of the ratio of the “Valve opening height” and the “maximum inner culvert height”.

Remarks:

- ◇ During the computations the actual valve loss coefficient (ξ_v) is derived from the user defined table, while using the ratio of the “actual valve opening height (h_{valve}) and the “actual maximum inner culvert height” ($h_{culvert}$):

  $$\xi_v = f(h_{valve}/h_{culvert}) \tag{10.51}$$

  Where f is a piecewise linear function with support points defined by the input fields `relOpening` and `lossCoeff`.
- ◇ A constant extrapolation is performed for values outside of the range of `relOpening`.

The outlet loss coefficient (ξ_o) is computed as follows:

- ◇ **Submerged flow** ($\zeta_2 = z_{c2} + h_{c2}$):

  $$\xi_o = k \left(1 - \frac{A_{fc}}{A_{fr2}}\right)^2 \tag{10.52}$$

DRAFT

Where:

k	User defined constant outlet loss coefficient $[-]$
A_{fr2}	Flow area in the branch, adjacent to the downstream culvert side $[m^2]$
A_{fc}	Culvert flow area $[m^2]$

$$A_{fc} = \begin{cases} A_{f_{cvalve}} & h_1 \geq z_{c1} + h_{gate} \\ A_{f_{c1}} & h1 < z_{c1} + h_{gate} \end{cases} \tag{10.53}$$

h_{valve}	Valve opening height $[m]$
z_{c1}	Upstream culvert invert level $[m]$
$A_{f_{cvalve}}$	Flow area under the culvert valve $[m^2]$
$A_{f_{c1}}$	Flow area in the culvert at its upstream side $[m^2]$

◇ **Free flow** ($\zeta_2 < z_{c2} + h_{c2}$):

$$\xi_o = 0 \tag{10.54}$$

Culvert cross sections, bed friction

For a culvert all available closed cross section types can be used. Culvert friction can be specified using any of the available bed friction formulations.

Culvert, Good modelling practice aspects

During the simulation a number of checks and possibly corrections on some dimensions of the culvert are performed.

◇ If the **upstream `invertLevel`** is below the bed level of the upstream water level point in the channel, the bed level of this water level point is lowered to the upstream `invertLevel`.
◇ If the **downstream `invertLevel`** is below the bed level of the downstream water level point in the channel, the bed level of this water level point is lowered to the downstream `invertLevel`.

Notice that the culvert checks are an exception to the other structure checks: the surrounding bed levels are adjusted, whereas for all other structure, the structure's own bed level(s) are adjusted.

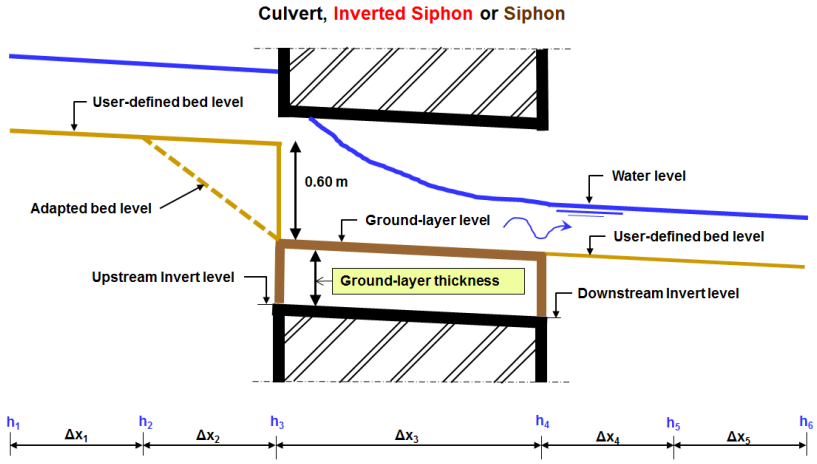


Figure 10.9: *Good modelling practice, culvert, Inverted Siphon and Siphon*

DRAFT

Inverted siphon

The inverted siphon is a special version of the culvert. An inverted siphon is a structure that normally connects two open channels, that are separated by a particular infrastructural work (e.g. dike, railroad). The inverted siphon makes an underground connection through such infrastructural work. An inverted siphon is assumed to be fully filled with water at its deepest point. In case this does not yield for your stucture, you can consider to use a culvert. The flow through an inverted siphon is affected by its upstream and downstream invert level, the size and shape of its closed cross section, its ground layer thickness, its entrance loss, friction loss, its valve loss, losses due to its bends (bend loss), and its exit loss.

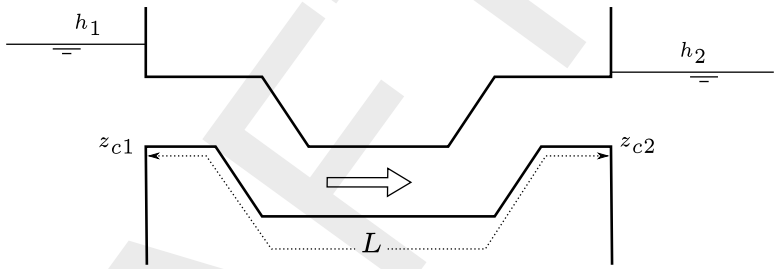


Figure 10.10: *Side view of an inverted siphon*

The bend losses are taken into account by changing Equation (10.55) into:

$$\mu = \frac{1}{\sqrt{\xi_i + \xi_f + \xi_v + \xi_o + \xi_b}} \tag{10.55}$$

Where:

ξ_b bend losses coefficient $[-]$

10.8 Long Culvert

10.8.1 Long culvert

Remark:

- ◇ Long culvert functionality is still undergoing validation and is of TRL level 4-6 (prototype), with the definitions of the Technical Technology Readiness Levels (TRL) in Section 2.7.

The long culvert is an underground pipe structure that normally connects two open waters modeled as 2D grid cells. The difference with the normal culvert is that the long culvert is modelled as pipe with a certain length, instead of being modelled as a subgrid culvert, with only energy losses at the snapped flow link of the culvert. The long culvert stores actual volume, and can optionally have multiple pressure points (which allows for a 'siphon-like' geometry with varying bed levels). A culvert can only be placed on an existing 1D pipe, while for a long culvert the pipe is generated by the kernel based on the input.

The flow through a long culvert is affected by its upstream and downstream invert levels (first and last z-coordinate, respectively), its total length, the size of its closed rectangular cross section, its friction loss, and its valve loss. The input file format for a long culvert is described in Section C.13.4.

DRAFT

Long culvert positioning

Figure 10.11 shows a side view of a long culvert. In the most simple case there will only be two points z_1 and z_2.

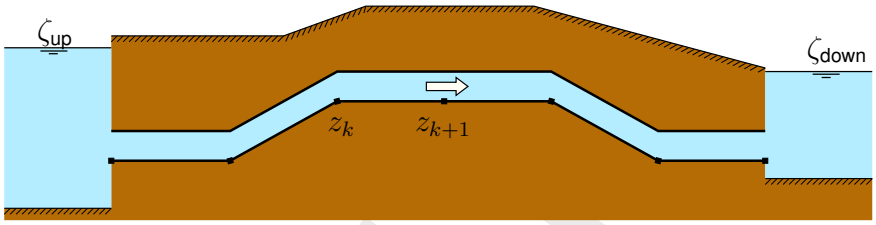


Figure 10.11: *Side view of a long culvert*

Figure 10.12 shows the discretization of long culverts in a top view. When the long culvert input polyline has more than two points, internal 1D computational pressure points are automatically generated and coupled via 1D2D links to the inlet and outlet 2D grid cells.

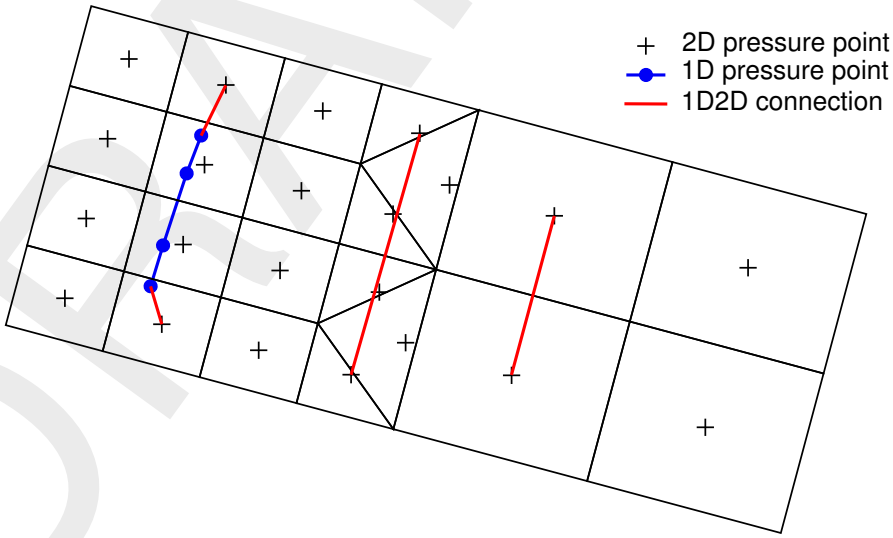


Figure 10.12: *Discretization of three long culverts via 1D and 1D2D links.*

The 1D2D links are of the vertically stacked type (see Section 17.7.5), such that pipe flow can go underneath the 2D grid cells. As a result, the flow link bed levels at the inlet and outlet side are fully determined by the long culvert's z-values (cf. Equation (17.19)).

Long culvert geometry

The 1D representation of long culverts results in regular 1D pipe flow, with a rectangular cross section with the user-defined dimensions (cf. Section 3.4).

Long culvert losses

The friction losses for long culverts are identical to the friction losses in regular 1D pipe flow (cf. Equation (3.24)). The valve losses are approximated by the user-defined relative valve opening, which is a dimensionless number between 0 and 1. The real discharge through the long culvert is reduced by a linear scaling of the maximum pipe cross sectional area:

$$A_{u_j,\text{culv}} := \theta_{\text{area,culv}} A_{u_j,\text{culv}}, \tag{10.56}$$

where j, culv is the first flow link at the inlet side of the long culvert and $\theta_{\text{area,culv}}$ is the relative flow area at the valve.

Long culverts in parallel

During initialization, the kernel generates the long culvert 1D and 1D2D links on-the-fly based on the input files. However this same approach is not feasible for parallel models, as the the result of the partitioning process must be preserved for these links. For this reason, as part of the partitioning process these new links are properly partitioned and subsuquently written to the partitioned netfiles, complete with branch and crossection information.

10.9 Bridges in 1D

This section contains a description of bridges in 1D. For modelling of bridge pillars in 2D we refer to section 6.4.2.11

The bridge structure is a structure that can be used to model the flow underneath a bridge (and across). The bridge's geometry is defined by either or both of:

- ◇ An abutment definition (as a cross section definition), which determines its wetted flow area and hydraulic radius. The flow area of the bridge should be smaller than the surrounding flow area of the channel.
- ◇ A pillars definition.

A bridge definition has to consist of at least one geometry definition, so at least an abutment definition or pillars definition has to be given in the input.

Figure 10.13 shows an example bridge where flow area is restricted by abutments, the bridge deck and two bridge pillars.

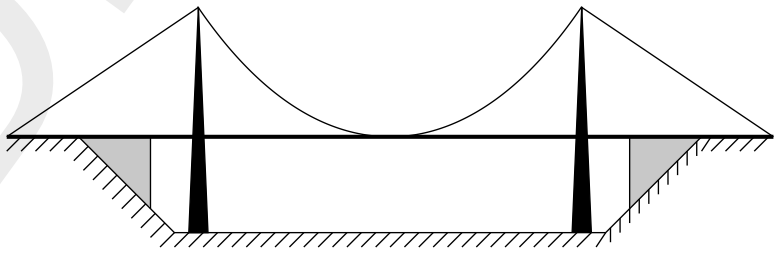


Figure 10.13: *A suspension bridge*

A limitation of the bridge structure is, that no free flow over the bridge can be computed. If free flow occurs over the bridge, it is advised to use a weir instead.

The general description of the discharge through a bridge is given by:

$$Q = \mu A_f \sqrt{2g(\zeta_1 - \zeta_2)} \tag{10.57}$$

Where

Q	Discharge through bridge [m^3/s]
μ	Discharge coefficient derived from loss-coefficients [-] (see Equation (10.58))
A_f	Wetted area [m^2] of flow through bridge at upstream side
g	Acceleration due to gravity [m/s^2] (≈ 9.81)

ζ_1	Upstream water level [m]
ζ_2	Downstream water level [m]

For the bridge the discharge coefficient is defined as:

$$\mu = \max\left(1, \frac{1}{\sqrt{\xi_i + \xi_f + \xi_o + \xi_y}}\right) \tag{10.58}$$

Where

ξ_i	Entrance loss coefficient.
ξ_f	Friction loss coefficient.
ξ_o	Exit loss coefficient.
ξ_y	Pillar loss coefficient.

Remark:

- ◇ For numerical reasons (e.g. validity of structure Equation (10.57)) the discharge coefficient (μ) is limited to a maximum of 1.0. In other words, if μ, according to Equation (10.58) becomes larger than 1.0, the actual applied discharge coefficient in Equation (10.57) is capped at 1.0. This means that for a given discharge, the water level difference over such bridge might be larger than anticipated, when considering the defined friction loss coefficients.

Abutment bridge

By defining an abutment bridge, the coefficients ξ_i, ξ_f and ξ_o in Equation (10.58) get a non-zero value. Also the flow area through the bridge is affected by the additional cross section definition.

When desired, the bridge deck can affect the flow through the bridge, for example by having the cross section closed. A limitation however, is that flow over the bridge deck cannot be taken into account.

The ξ_i loss coefficient is an input parameter

The ξ_o coefficient is defined as:

$$\xi_o = k\left(1 - \frac{A_f}{A_{f_2}}\right)^2, \tag{10.59}$$

where

k	Constant exit loss coefficient
A_f	Wetted area [m^2] of flow through bridge at upstream side, including the bridge pillars.
A_{f_2}	Wetted area [m^2] of flow in branch at downstream side of bridge

The ξ_f coefficient is defined as:

$$\xi_f = \frac{2gL}{C^2 R}, \tag{10.60}$$

where

g	Acceleration due to gravity [m/s^2] (≈ 9.81)

DRAFT

L	Length of bridge [m]
C	Chézy coefficient [$m^{1/2}/s$]
R	Hydraulic radius [m]

During the simulation a check and possibly correction on the vertical position of the bridge can be performed, see MDU option `ChangeStructureDimensions`:

- ◇ If the bed level of the bridge is below the bed level of the surrounding water level points in the channel, the bedlevel of the bridge is raised to the highest bed level of the two water level points (also see footnote 1).
- ◇ In case the bed level is raised, the complete cross section is also raised.

Pillars

A bridge can have one or more pillars that affect the discharge through the bridge. By defining pillars, the coefficient ξ_y in Equation (10.58) get a non-zero value:

$$\xi_y = \beta \frac{\alpha_y}{A_f}, \tag{10.61}$$

where

β	Parameter [$-$] depending on shape of pillar [s] (shape factor). Normally between 0.22 and 1.56
A_f	Wetted area [m^2] of flow through bridge at upstream side
α_y	Area [m^2] of wetted part of pillar [s] perpendicular to the flow direction, considered at upstream side

10.10 Universal weir

The universal weir structure is a construction generating energy losses due to constriction of the flow. The geometrical shape of the universal weir's crest can be defined as a Y-Z profile (see Figure 10.14).
It is assumed that the discharge across a universal weir is the summation of the discharges across each individual weir section. A weir section is the space between to subsequent support point of the Y-Z profile.

The crest level profile of a universal weir is divided into:

- ◇ *rectangular weir sections*, having a horizontal bed and
- ◇ *triangular weir sections*, having a sloping bed.

Rectangular weir sections are considered as a broad-crested weir. Triangular weir sections are considered as (the half of) a broad-crested weir with truncated triangular control section (Bos, 1989).

DRAFT

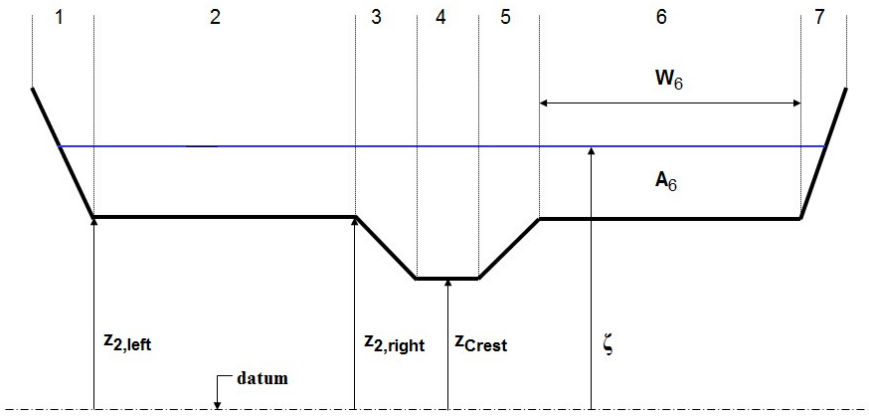


Figure 10.14: *Crest level (Y-Z) profile of a Universal weir, divided into three rectangular weir sections (2, 4 and 6) and four triangular weir sections (1, 3, 5 and 7)*

Parameters depicted in Figure 10.14 are:

A_i	Flow area of section i [m^2]
W_i	Width of weir section i [m]
$z_{i,left}$	Elevation at the left side of weir section i [mAD]
$z_{i,right}$	Elevation at the right side of weir section i [mAD]
z_{Crest}	Crest level of the Universal weir (output parameter only), equal to the lowest elevation of its crest level profile [mAD]
ζ	Water level [mAD]

Following equations yield for a universal weir:

$$Q = \sum_{i}^{N}(u_i A_i) = \sum_{i}^{N} Q_i \tag{10.62}$$

$$A = \sum_{i}^{N} A_i \tag{10.63}$$

$$U_{structure} = Q/A \tag{10.64}$$

where:

A	Flow area of the universal weir [m^2]
A_i	Flow area of weir section i [m^2]
N	Number of weir sections i [$-$]
Q	Discharge over the universal weir [m^3/s]
Q_i	Discharge over weir section i [m^3/s]
u_i	Flow velocity in weir section i [m/s]
$U_{structure}$	Average flow velocity over the universal weir [m/s]

Following equations yield for a rectangular weir section i:

$$z_{i,crest} = min(z_{i,left},\ z_{i,right}) \tag{10.65}$$

$$ml_{rectangular,i} = 2/3 \tag{10.66}$$

◇ Free rectangular weir flow if $\left(\frac{\zeta_2 - z_{i,crest}}{\zeta_1 - z_{i,crest}}\right) \leq ml_{rectangular,i}$

$$u_i = c_e \sqrt{\frac{2}{3} g} \sqrt{\zeta_1 - z_{i,crest}} \tag{10.67}$$

$$A_i = \frac{2}{3} W_i (\zeta_1 - z_{i,crest}) \tag{10.68}$$

$$Q_i = u_i \, A_i = c_e \, \frac{2}{3} \sqrt{\frac{2}{3} g} \quad W_i \, (\zeta_1 - z_{i,crest})^{3/2} \tag{10.69}$$

◇ Submerged rectangular weir flow if $\left(\frac{\zeta_2 - z_{i,crest}}{\zeta_1 - z_{i,crest}}\right) > ml_{rectangular,i}$

$$u_i = c_e \sqrt{2g(\zeta_1 - \zeta_2)} \tag{10.70}$$

$$A_i = W_i (\zeta_2 - z_{i,crest}) \tag{10.71}$$

$$Q_i = u_i \, A_i = c_e \, W_i \, (\zeta_2 - z_{i,crest}) \, \sqrt{2g(\zeta_1 - \zeta_2)} \tag{10.72}$$

Following equations yield for a triangular weir section i:

$$z_{i,crest} = min(z_{i,left}, \; z_{i,right}) \tag{10.73}$$

$$dz_i = |z_{i,left} - z_{i,right}| \tag{10.74}$$

$$ml_{triangular,i} = \begin{cases} \frac{4}{5} & \text{if } (\zeta_1 - z_{i,crest})) \leq 1.25 \, dz_i \\ \frac{2}{3} + \frac{1}{6} \frac{dz_i}{(\zeta_1 - z_{i,crest})} & \text{if } (\zeta_1 - z_{i,crest})) > 1.25 \, dz_i \end{cases} \tag{10.75}$$

◇ Free triangular weir flow if $\left(\frac{\zeta_2 - z_{i,crest}}{\zeta_1 - z_{i,crest}}\right) \leq ml_{triangular,i}$

$$u_i = c_e \sqrt{2g(1 - ml_{triangular,i})(\zeta_1 - z_{i,crest})} \tag{10.76}$$

$$A_i = \begin{cases} W_i \left(\frac{(ml_{triangular,i} \; (\zeta_1 - z_{i,crest}))^2}{2 \; dz_i} \right) & \text{if} \quad \frac{\zeta_1 - z_{i,crest}}{1/ml_{triangular,i}} \leq dz_i \\ W_i \left(\frac{\zeta_1 - z_{i,crest}}{1/ml_{triangular,i}} - \frac{dz_i}{2} \right) & \text{if} \quad \frac{\zeta_1 - z_{i,crest}}{1/ml_{triangular,i}} > dz_i \end{cases} \tag{10.77}$$

$$Q_i = u_i \, A_i \tag{10.78}$$

◇ Submerged triangular weir flow if $\left(\frac{\zeta_2 - z_{i,crest}}{\zeta_1 - z_{i,crest}}\right) > ml_{triangular,i}$

$$u_i = c_e \sqrt{2g(\zeta_1 - \zeta_2)} \tag{10.79}$$

$$A_i = \begin{cases} W_i \left(\frac{(\zeta_2 - z_{i,crest})^2}{2 \; dz_i} \right) & \text{if} \quad \zeta_2 - z_{i,crest} \leq dz_i \\ W_i \, (\zeta_2 - z_{i,crest} - \frac{dz_i}{2}) & \text{if} \quad \zeta_2 - z_{i,crest} > dz_i \end{cases} \tag{10.80}$$

$$Q_i = u_i \, A_i \tag{10.81}$$

DRAFT

where:

A_i	Flow area of weir section i [m^2]
c_e	Discharge coefficient, applicable to all rectangular weir sections as well as to all triangular weir sections of a universal weir [$-$]
dz_i	Vertical distance between the elevation at the left side and the right side of a triangular weir section i [m]
g	Acceleration due to gravity [m/s^2] (≈ 9.81)
$ml_{rectangular,i}$	*Water-level-based* modular limit of a rectangular weir section i. Its value is always equal to 2/3 [$-$]
$ml_{triangular,i}$	*Water-level-based* modular limit of a triangular weir section i. Its value depends on the actual water depth. [$-$]
Q_i	Discharge over weir section i [m^3/s]
u_i	Velocity in weir section i [m/s]
W_i	Width of weir section i [m]
$z_{i,crest}$	Crest level of weir section i [mAD]
$z_{i,left}$	Elevation at the left side of weir section i [mAD]
$z_{i,right}$	Elevation at the right side of weir section i [mAD]
ζ_1	Water level at the upstream side of the universal weir [mAD]
ζ_2	Water level at the downstream side of the universal weir [mAD]

During the simulation a check and possibly correction on the crest level of the universal weir is performed.

- ◇ If the bed level of the bridge is below the bed level of the surrounding water level points in the channel, the bed level of the bridge is raised to the highest bed level of the two water level points (also see footnote 1).
- ◇ In case the bed level is raised, the complete weir profile is also raised.

10.11 Pump

The pump structure forces the flow of water in one direction at a certain discharge. This discharge is essentially a predescribed value for the pump, depending on the type of pump. A pump can either be a staged pump or a non staged pump. The input file format for a pump is described in section C.13.6.

A **staged pump** contains control rules to switch on or off the pump and to set the actual capacity depending on the water levels at both sides of the pump.

A **non-staged pump** has its capacity prescribed directly. The capacity can be defined as a constant value, a time series or controlled by RTC.

Both types of pumps can optionally have a **reduction factor** table defined. When a pump pumps against a bigger head difference, its effective discharge may be lower than the maximum capacity.

10.11.1 Notation

The used variable in this section are described below.

ζ_{ss}^n	Water level at the suction side of the pump at the current time level.
ζ_{ds}^n	Water level at the delivery side of the pump at the current time level.
h_{ss}^n	Water depth at the suction side of the pump at the current time level.
V_{ss}^n	Water volume at the suction side of the pump at the current time level.

DRAFT

10.11.2 Orientation

Water is always pumped from the suction side to the delivery side. The orientation of the pump in the model is defined by either the branch orientation (1D) or the direction of the polyline (2D, see section C.13). It may be convenient to orientate the pump opposite to the branch orientation of the channel. This is controlled by the input parameter `orientation`. `positive` means the suction side is at the (geometrical) upstream side of the pump and `negative` means that the suction side is at the (geometrical) downstream side of the pump.

10.11.3 Staged pump

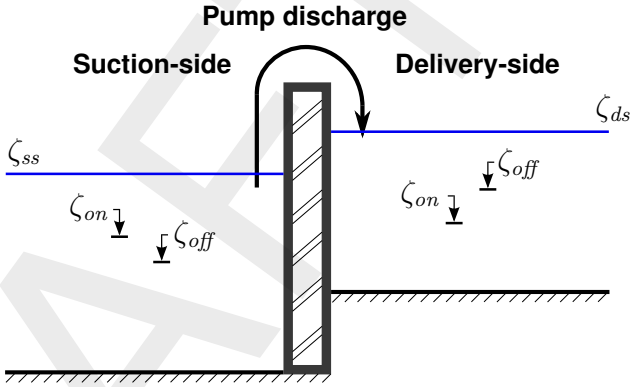


Figure 10.15: *Pump station*

Pump stages

A pump station may consist of multiple pump stages. Each stage can have a different pump capacity and switch on and switch off level. At each point in time, the pump can be in one pump stage only. The highest stage that is turned on, will be the actual stage, regardless of the state of the lower stages.

The pump can be controlled at either the **suction side** only, the **delivery side** only or on **both sides**.

The state of pump stage$_i$ is determined by:

- **suction side:**
  - **if** no suction side control **then** $s_{ss,i}^{n+1} = True$
  - **else if** ζ_{ss}^n > $\zeta_{ss,on,i}$ **then** $s_{ss,i}^{n+1} = True$
  - **else if** ζ_{ss}^n < $\zeta_{ss,off,i}$ **then** $s_{ss,i}^{n+1} = False$
  - **else** $s_{ss,i}^{n+1} = s_{ss,i}^{n}$
- **delivery side:**
  - **if** no delivery side control **then** $s_{ds,i}^{n+1} = True$
  - **if** ζ_{ds}^n < $\zeta_{ds,on,i}$ **then** $s_{ds,i}^{n+1} = True$
  - **else if** ζ_{ds}^n > $\zeta_{ds,off,i}$ **then** $s_{ds,i}^{n+1} = False$
  - **else** $s_{ds,i}^{n+1} = s_{ds,i}^{n}$
- **if** $s_{ss,i}^{n+1} == True$ **and** $s_{ds,i}^{n+1} == True$ **then** pump stage$_i$ is on

Where:

$s_{ss,i}^{n+1}$	Denotes whether the suction side control for the next time step is on ($True$) or off ($False$).
$s_{ds,i}^{n+1}$	Denotes whether the delivery side control for the next time step is on ($True$) or off ($False$).
$\zeta_{ss,on,i}$	Start level at suction side for stage i.
$\zeta_{ss,off,i}$	Stop level at suction side for stage i.
$\zeta_{ds,on,i}$	Start level at delivery side for stage i.
$\zeta_{ds,off,i}$	Stop level at delivery side for stage i.

Subsequently, the actual pump stage will be set to the stage with the highest sequence number that is turned on.

10.11.4 Non-staged pump

A non staged pump has either a constant capacity, a time series for the capacity or can be controlled by an external process, such as real time control.

10.11.5 Capacity reduction table

A capacity reduction table can be defined both for the staged and non-staged pump. The capacity reduction table contains capacity reduction factors as a function of the pump head (i.e., $\zeta_{ds} - \zeta_{ss}$). The pump heads (key '`head`' in input) in a capacity reduction table should be in increasing order. Capacity reduction factors (key `reductionFactor` in input) should be equal or larger than 0 and equal or less than 1. The actual pump discharge equals the current pump capacity multiplied by the capacity reduction factor. When no capacity reduction table is defined, in effect a capacity reduction factor equal to 1 is applied.

Please note that the pump head at $t = t^n$ is used to determine the capacity reduction factor to be applied in the time-step from $t = t^n$ to $t = t^{n+1}$.

10.11.6 Distribution and relaxation of pump discharge

A single pump in a 2D model may span multiple flow links, in which case the pump discharge is distributed evenly over the included flow links. Flow links are included when the water depth at the suction side is above a certain threshold: $h_{ss}^n > 0.01$. A single, representative water level at the suction side is then determined by averaging the separate water levels at the suction side of the pump over the included flow links. The delivery side water level is determined similarly. These two values are used for staged pumps or, optionally, for a capacity reduction table.

The actual pump discharge may be limited (relaxed) even further when total potential pump volume is above the total available volume of water at the suction side. In other words, it can happen that the actual discharge through the pump is limited below capacity by water volume on suction side because the suction side has not enough volume of water to be sucked.

The actual discharge is computed by:

$$q_{\text{pump}}^n = \min(q_{\text{pump}}, 0.9 \cdot \sum_{\text{open pump links}} V_{ss}^n / \Delta t^n). \tag{10.82}$$

The pump capacity is then equally distributed over the open pump links.

10.12 Compound structure

A compound structure consists of several structures (structure elements) parallel to each other at the same location. See Table C.13 for a list of possible structure types. The only exception is that compound structures cannot be nested. Each structure element in a compound is treated independently. The discharge through each structure element is then summed up to the total discharge for the compound structure.

The user should keep in mind that geometric checks on structure parameters are performed for each structure element independently. For example the total structure width might be larger than the actual width of the channel, even though the widths of the underlying structures have been checked. As a result the user must be extra careful to check for inconsistencies in the input.

10.13 Outlet

Urban sewer systems contain outlets (or outfalls) to discharge excess water into open water, waste water treatment plants, or other 'external' downstream systems. Often, an outlet is combined with some sewer overflow structure that has a certain crest level. As long as the sewer water level does not reach this crest, no outflow occurs. In reality outlets are constructions part of the sewer system, but in the model schematization these outlets simply act as boundary conditions. In other words, there is *no* separate hydraulic structure type for the outlet.

As an example, the model schematization near an outlet can look as follows: the outlet node is a regular end node in the network (details in section B.2). A waterlevel boundary, associated via the `nodeId`, is set to the outside water level (details in section C.6.2.1). Next in the network, connected to the boundary node, can be a regular internal node. On the connection between these two nodes, the overflow structure can be placed, for example a weir (section 10.4) or a pump (section 10.11).

When the actual overflow structure is such that it only allows outflow and no inflow, the model schematization should use a hydraulic structure that has an `allowedFlowDir` setting, for example the universal weir (section 10.10).

10.14 Thin dams

Thin dams are similar to fixed weirs. The only difference between the thin dams and fixed weirs are in their crest levels. Thin dams, in principle, include infinitely high crest levels and hence, they do not allow water flux. Similar to the other structures, the thin dams can be selected from the toolbar and drawn by a polygon. D-Flow FM adjusts the polygon to the nearest velocity points. The input data for a thin dam is identical to those for fixed-weir, except for the crest level.

DRAFT

11 Hydrological processes

Hydrological processes describe how water moves across land and through the hydrologic cycle. For certain hydrodynamic simulations these processes can not be ignored, for example for pluvial flooding. The actual flow over land is still described by shallow water hydrodynamics (chapter 3), but additionally it is possible to add hydrological parameter fields and boundary conditions to a model schematization.

11.1 Notation

The used symbols in this chapter build upon the hydrodynamics notation introduced in section 17.2. In this section, additional symbols are introduced for the hydrological processes.

Hydrology:

$A_{\mathrm{re},k}$ Bottom area of a grid cell for rain and evaporation.

The difference between wet area A_k and "rainfall-evaporation area" $A_{\mathrm{re},k}$ is quite subtle. While they are the same in a 2D-description[1], they usually differ in the 1D case. Figure 11.1 shows this by comparing two channels in the 1D and the 2D case.

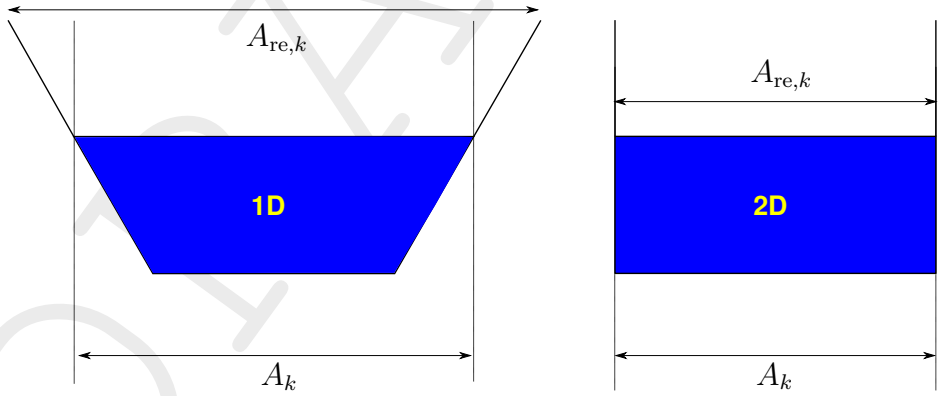


Figure 11.1: *Vertical cross section view of wet area and hydrological area in a 1D and a 2D grid cell.*

11.2 Precipitation

Precipitation falling onto the surface (land or water) acts as a source of water for the system. The state of precipitation (solid or liquid) is ignored, so all precipitation that falls, contributes directly to the mass balance. Hereafter, it is often referred to as rainfall.

11.2.1 Meteo forcing input

Rainfall is prescribed by a data file containing the rainfall intensities or amounts, and this file should be referenced in the external forcings file. An example rainfall block looks like:

Example:

```
[General]
fileVersion         = 2.01
fileType            = extForce
```

[1] This is not the case when Conveyance2D=3.

DRAFT

```
[Meteo]
quantity            = rainfall
forcingFile         = precip.nc
forcingFileType     = netcdf
targetMaskFile      = roofs.pol
targetMaskInvert    = true
interpolationMethod = nearestNb
```

The external forcings syntax for meteo is further described in section C.6.2.3. The supported data formats for the actual rainfall are:

- ◇ Global (spatially uniform) time series in <.bc> ascii format (section C.5);
- ◇ Time series on meteo stations in netCDF (section E.2.4);
- ◇ Gridded meteo data (space- and time-dependent) in netCDF (section C.6.2.3);

11.2.2 Rainfall distribution across the model grid

The total rainfall amount is determined at the start of every timestep and added to the total volume source term for each grid cell. The total rainfall flux entering a grid cell is simply the local rainfall intensity times the maximum grid cell area, *if* that grid cell is allowed to capture rainfall. A grid cell is allowed to capture rainfall when it satisfies the following conditions:

- ◇ The grid cell is 2D[2];
- ◇ Or: the grid cell is 1D and is *not* overlapped by any other 2D grid cell. This excludes 1D embedded grid points and 1D sewer pipes below street level;
- ◇ Finally, the cell must be selected by the target mask if and only if such a target mask is present.

The maximum grid cell area used for catching rainfall is defined as:

- ◇ normal bed area for 2D grid cells;
- ◇ sum of maximum flow link areas for 1D open channel grid cells; these areas are based on the maximum flow width of the relevant cross sections and half the flow link length;
- ◇ the bottom area of a 1D storage node, if `useTable=false`;
- ◇ Note: the bottom area of a 1D storage node with `useTable=true` is yet undefined.

The rainfall intensity at each grid cell is interpolated from the source data to the cell centers. The type of interpolation depends on the settings supplied. Figure 11.2 shows nearest neigbour interpolation of three meteo stations onto the model grid.

Additionally, an optional target mask can be supplied in a polygon file. The grid cells whose center lies outside (or inside) of the polygon(s) are then excluded from receiving rainfall.

11.2.3 Rainfall intensities versus rainfall amounts

Some datasets represent the precipitation as an intensity (e.g., millimeters per day). The external forcings quantity name is then `quantity = rainfall_rate`. Such input timeseries can be interpolated in time as a linear function, so the resulting rainfall intensity during the simulation is continuous and piecewise linear. Other datasets may represent the precipitation as an amount per each data time interval (e.g. total millimeters for the past interval). Such input timeseries can be interpolated in time as a block function. The external forcings quantity name is then `quantity = rainfall`. The average intensity is derived as the total rain amount divided by the data time interval (which can also be non-equidistant in time). The

[2] In 3D models, "2D" means that the cell is 2D in the horizontal input grid.

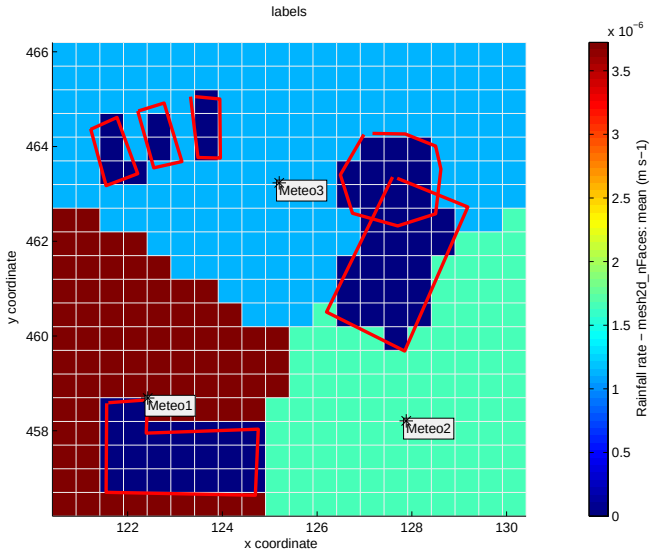


Figure 11.2: *Rainfall rates by nearest neighbour interpolation, with some areas masked out.*

resulting rainfall intensity during the simulation is then typically discontinuous and piecewise constant in time.

11.3 Interception

Interception in a hydrological context, refers to interception of rain such that it does not reach the ground. The precipitation can be intercepted by, e.g., leaves or roofs. Interception is modelled by maintaining a separate basket on each grid cell, that stores some water from precipitation. Once it is full, all precipitation continues to fall onto the ground directly. Interception can be enabled by setting `InterceptionModel=1` in the MDU file under `[hydrology]` (see chapter A). The thickness of the interception layer is then prescribed via spatial input (see Initial field file in section D.2).

11.3.1 Numerical treatment of interception

Interception is treated explicitly. It is not handled in the matrix solver, but directly subtracted from current rain intensities. This behaviour is described by equations (11.1) and (11.2):

$$Q_{\mathrm{icept},k} = \min\left(Q_{\mathrm{in,rain},k}, A_{\mathrm{re},k}/\Delta t \cdot \left(H_{\mathrm{icept},k} - h_{\mathrm{icept},k}\right)\right), \tag{11.1}$$

$$Q_{\mathrm{in,rain},k} \mathrel{-}= Q_{\mathrm{icept},k}, \tag{11.2}$$

where $H_{\mathrm{icept},k}$ is the thickness of the interception layer and $h_{\mathrm{icept},k}$ is the water depth in the interception layer. Evaporation from the interception layer is treated in section 11.5.2.

11.4 Infiltration

Infiltration of water into the soil can be modeled in several ways. Three infiltration models are currently supported:

- ◇ Maximum (constant) infiltration capacity
- ◇ Function of pressure
- ◇ Horton's infiltration equation

11.4.1 Horton's equation

Horton's infiltration equation (Horton, 1933) is an empirical formula which describes the trend of the infiltration capacity. A decreasing infiltration capacity is described by

$$f(t) = f_{min} + (f_{max} - f_{min})e^{-k_d t}, \text{ if the surface is wet}$$

and a recovering infiltration capacity is described by

$$f(t) = f_{min} - (f_{min} - f_{max})e^{-k_r t}, \text{ if the surface is dry,}$$

where the symbols used are:

t	Time	[hr]
$f(t)$	Infiltration capacity at t	[mm/hr]
f_{max}	Maximum infiltration capacity at $t = 0$	[mm/hr]
f_{min}	Minimum infiltration capacity	[mm/hr]
k_d	Time factor of decreasing infiltration capacity	[1/hr]
k_r	Time factor of recovering infiltration capacity	[1/hr]

It is assumed that at the beginning of each rainfall event, the infiltration capacity is at its maximum f_{max}. When the rainfall intensity is higher than the minimum infiltration capacity f_{min} or there is water stored on the surface, the infiltration capacity will decrease. The infiltration capacity will start to recover when the rainfall intensity is lower than the minimum infiltration capacity f_{min} and there is no water stored on the surface.

11.5 Evaporation

Water loss via evaporation can be enabled by setting `Evaporation=1` in the MDU file under `[external forcing]`. The potential evaporation $E_{\mathrm{p},k}$ is then prescribed via spatial input (see Initial field file in section D.2). The actual evaporation is only limited by the available volume as described in section 11.5.1. Other hydrological factors such as soil moisture, are not modelled.

Note: Originally, evaporation was realized by specifying negative rainfall intensities. This functionality is kept for backwards-compatibility only and should not be used for new models.

11.5.1 Numerical treatment of evaporation

Evaporation is a negative source of volume and as such is subtracted from the right-hand side source term of the continuity equation (3.7). This is described by equation (11.3)

$$Q_{\mathrm{in},k} \mathrel{-}= \min\left(\frac{1}{2}V_k/\Delta t + Q_{\mathrm{in,rain},k}, |E_{\mathrm{p},k} \cdot A_{\mathrm{re},k}|\right) \tag{11.3}$$

where $Q_{\mathrm{in,rain},k}$ describes the incoming rain. The negative evaporation flux is limited based on the available water volume in the grid cell in each timestep. The relaxation factor of $\frac{1}{2}$ limits the negative source term contribution when little water is available, to avoid negative water depths.

Once all source terms have been collected in $Q_{\mathrm{in},k}$, the real calculation of new water levels is done. The resulting water level at that grid point is then calculated implicitly by the matrix solver. Or, in the case of a negative source term $Q_{\mathrm{in},k}$, it is solved explicitly in the continuity equation.

DRAFT

11.5.2 Evaporation from interception layer

Evaporation from the interception layer can occur simultaneously with open water evaporation in the same grid cell. It is computed explicitly after precipitation was added (if any). The negative evaporation flux can simply be subtracted from the interception layer, under the condition that it is capped by volume available in the interception layer. This is described by equations (11.4) and (11.5)

$$Q_{\mathrm{evap,icept},k} = \min\left(h_{\mathrm{icept},k} \cdot A_{\mathrm{re},k}/\Delta t, |E_{\mathrm{p},k} \cdot A_{\mathrm{re},k}|\right) \tag{11.4}$$

$$h_{\mathrm{icept},k} \mathrel{-}= Q_{\mathrm{evap,icept},k} \cdot \Delta t/A_{\mathrm{re},k} \tag{11.5}$$

where $Q_{\mathrm{evap,icept},k}$ is the actual evaporation flux from the interception layer.

11.6 Lumped rainfall-runoff coupling

Rainfall-runoff processes can also be computed in a *lumped* way, instead of the *distributed* way in section 11.2. In this case, the rainfall runoff is computed elsewhere (for example by D-Rainfall Runoff) and translated into a set of single point lateral discharges in the D-Flow FM model schematization. This is further discussed in a separate section 16.7. The current chapter continues to focus on *distributed* hydrology instead.

11.7 Hydrological output

When hydrological processes are enabled in a model schematization, additional output can be produced in the various output files.

11.7.1 Spatial hydrological output in map file

The map file can contain extra variables for the current hydrological fluxes, for example rainfall intensities or actual evaporation. Table F.19 lists all available output quantities.

11.7.2 Time series hydrological output in history file

The history file can contain some extra hydrological volume totals that contribute to the overall mass balance. This output is enabled by one global setting in the MDU: `[output] Wrihis_balance = 1`. More details on this volume output is available section F.3.1.6.

DRAFT

12 Bedforms and vegetation

The ground surface, vegetation and unresolved buildings and other flow-obstructing objects exert shear stresses on the passing flow. The total flow resistance is commonly characterised by means of a bulk bed roughness coefficient of type Chézy, Manning or White-Colebrook as discussed in section 3.3.6. However, such an approach overestimates the bed shear stresses in the presence of vegetation, and other processes that depend on that quantity, such as sediment transport, will give incorrect results. Furthermore, lumping all flow resistance into bed roughness will result in incorrect velocity and turbulence profiles in 3D models. It's better to distinguish between the roughness of the bed surface itself and flow resistance higher up in the water column. This chapter describes different features of D-Flow FM to represent vegetation and an alluvial bed.

Within the main stream flow the shear stresses are largely determined by the local conditions of the alluvial bed (bed composition and bedform characteristics). For this type of conditions, bedform roughness predictors have been implemented (see section 12.1).

In floodplains of rivers and in the intertidal areas of estuaries, the flow resistance is determined by the combination of the roughness of the ground surface and vegetation causing flow blockage higher up in the water column and turbulence production. D-Flow FM includes a number of methods to represent vegetation (see section 12.2). Some of the vegetation features may also be used to represent other objects causing flow resistance, such as unresolved buildings, fences, street furniture, vehicles, etc.

Flow resistance may be represented in a 2D numerical model by means of the trachytope approach (see section 12.3) .

12.1 Bedform heights

There are various types of bedforms; each type influences the hydro- and morphodynamics in its own way. Following Van Rijn (2007) we distinguish between roughness heights for ripples, mega-ripples and dunes. D-Flow FM contains a module to determine the dune, mega-ripple, and ripple roughness heights according Van Rijn (2007) as described in section 12.1.1. D-Flow FM also implements a number of predictors for dune characteristics (height and length) which may be used to determine the effective dune roughness height; see section 12.1.2. Those dune characteristics may also be used in dredging operations and active layer thickness (see Deltares (2026d)).

12.1.1 Van Rijn (2007) bedform roughness height predictor

As proposed by Eq. (5a)–(5d) in Van Rijn (2007) the roughness height predictor for ripples is given by

$$k_{s,r} = \alpha_r \begin{cases} 20 D_{\text{silt}} & \text{if } D_{50} < D_{\text{silt}} \\ k^*_{s,r} & \text{otherwise} \end{cases} \tag{12.1}$$

where α_r is a calibration factor, $D_{\text{silt}} = 32\ \mu$m and $k^*_{s,r}$ is given by

$$k^*_{s,r} = \begin{cases} 150 f_{cs} D_{50} & \text{if } \psi \leq 50 \\ (182.5 - 0.65\psi) f_{cs} D_{50} & \text{if } 50 < \psi \leq 250 \\ 20 f_{cs} D_{50} & \text{if } \psi > 250 \end{cases} \tag{12.2}$$

in which ψ is the current-wave mobility parameter defined as $U^2_{wc}/[(s-1)gD_{50}]$ using $U^2_{wc} = U^2_w + u^2_c$ and relative density $s = \rho_s/\rho_w$, peak orbital velocity near bed $U_w =$

$\pi H_s/[T_r \sinh(2kh)]$, depth-averaged current velocity u_c, significant wave height H_s, wave number $k = 2\pi/L$, wave length L derived from $(L/T_p \pm u_c)^2 = gL\tanh(2\pi h/L)/(2\pi)$, relative wave period T_r, peak wave period T_p, water depth H and factor $f_{cs} = (0.25D_{\text{gravel}}/D_{50})^{1.5}$ in which $D_{\text{gravel}} = 0.002$ m. The value of $k_{s,r}$ is limited to values ranging from D_{90} to 2% of the water depth.

Following Eq. (6a)–(6d) by Van Rijn (2007) the roughness height predictor for mega-ripples is given by

$$k_{s,mr} = \alpha_{mr} \begin{cases} 0 & \text{if } D_{50} < D_{\text{silt}} \\ k^*_{s,mr} & \text{otherwise} \end{cases} \tag{12.3}$$

where α_{mr} is a calibration factor and $k^*_{s,mr}$ is given by

$$k^*_{s,mr} = \begin{cases} 0.0002 f_{fs}\psi H & \text{if } \psi \leq 50 \\ (0.011 - 0.00002\psi) f_{fs} H & \text{if } 50 < \psi \leq 550 \\ 0.02 & \text{if } \psi > 550 \text{ and } D_{50} \geq D_{\text{sand}} \\ 200 D_{50} & \text{if } \psi > 550 \text{ and } D_{\text{silt}} \leq D_{50} < D_{\text{sand}} \end{cases} \tag{12.4}$$

in which $f_{fs} = (D_{50}/1.5D_{\text{sand}})$ and $D_{\text{sand}} = 0.000062$ m. The value of $k_{s,mr}$ is limited to values smaller than 0.2 m.

Following Eq. (7a)–(7d) by Van Rijn (2007) the roughness height predictor for dunes is given by

$$k_{s,d} = \alpha_d \begin{cases} 0 & \text{if } D_{50} < D_{\text{silt}} \\ k^*_{s,d} & \text{otherwise} \end{cases} \tag{12.5}$$

where α_d is a calibration factor and $k^*_{s,d}$ is given by

$$k^*_{s,d} = \begin{cases} 0.0004 f_{fs}\psi H & \text{if } \psi \leq 100 \\ (0.048 - 0.00008\psi) f_{fs} H & \text{if } 100 < \psi \leq 600 \\ 0 & \text{if } \psi > 600 \end{cases} \tag{12.6}$$

Since these formulations do not depend on the equations for the dune characteristics given in the previous section, the advection and relaxation effects represented by Equation (12.13) cannot be included these Van Rijn (2007) formulations. Therefore, a separate relaxation option has been implemented for all three bedform roughness heights according Van Rijn (2007) formulations as given by

$$\begin{aligned} k_{s,r} &= (1-\alpha_r)k_{s,r,new} + \alpha_r k_{s,r,old} && \text{where } \alpha_r = e^{-\Delta t/T_r} \\ k_{s,mr} &= (1-\alpha_{mr})k_{s,mr,new} + \alpha_{mr} k_{s,mr,old} && \text{where } \alpha_{mr} = e^{-\Delta t/T_{mr}} \\ k_{s,d} &= (1-\alpha_d)k_{s,d,new} + \alpha_d k_{s,d,old} && \text{where } \alpha_d = e^{-\Delta t/T_d} \end{aligned} \tag{12.7}$$

where Δt is the hydrodynamic time step of bed roughness updating, and T_r, T_{mr} and T_d are different relaxation time scales for the three bedform types. Note that if a relaxation time scale T_* becomes zero, the corresponding relaxation factor α_* will be zero and the relaxation effect disappears.

Remarks:

- ◇ In case of a simulation with multiple sediment fractions, the D_{50} of the grain mixture in the surface layer is used.
- ◇ Non-erodible layers and partial transport effects are not taken into account to reduce the roughness height.

DRAFT

12.1.2 Dune height predictor

Four dune height predictors have been implemented: Van Rijn (1984), Fredsøe (1982) based on Meyer-Peter and Müller (1948) or Engelund and Hansen (1967), and a power relation. One of these predictors can be selected using the keyword `BdfH` in the `[bedform]` block. In case of Van Rijn (1984) the dune height h_d is computed as

$$h_d = 0.11\varepsilon H \left(\frac{D_{50}}{H}\right)^{0.3} (1 - e^{-T/2})(25 - T) \tag{12.8}$$

where ε is a calibration coefficient and T is the dimensionless bed shear parameter. Fredsøe (1982) formulated an approach to obtain a dune height predictor for a general sediment transport formula $s = f(U, H, k)$ depending on depth-averaged flow magnitude U, water depth H and Nikuradse bed roughness k as

$$h_d = \varepsilon s H / \left(U\frac{\partial s}{\partial U} - H\frac{\partial s}{\partial H} - k\frac{\partial s}{\partial k}\right) \tag{12.9}$$

If one applies this approach to the transport formula by Meyer-Peter and Müller (1948) one obtains

$$h_d = \frac{24}{63}\varepsilon H \max(1 - \frac{\theta_{cr}}{\mu\theta}, 0) \tag{12.10}$$

where μ is the efficiency factor $(C/C_{g,90})^{1.5}$ based on a grain related bed roughness of $3D_{90}$ and θ_{cr} is the critical mobility parameter based on the Shields curve or given by the user. If one applies the formulation of Fredsøe (1982) to the transport formula by Engelund and Hansen (1967) one obtains

$$h_d = \frac{1}{5}\varepsilon H \tag{12.11}$$

The latter two formulations have been implemented in D-Flow FM. Finally, the power relation dune height predictor reads

$$h_d = a_H H^{b_H} \tag{12.12}$$

The dune height formulations given by equations 12.8–12.12 are approximations for the dune height under equilibrium conditions. Under time and spatially varying conditions, the dune height will need time to adjust; this effect has been implemented by means of an advection relaxation equation.

$$\frac{\partial h_d}{\partial t} + u_H\frac{\partial h_d}{\partial x} + v_H\frac{\partial h_d}{\partial y} = \frac{h_{d,\text{eq}} - h_d}{T_H} \tag{12.13}$$

By default these effects are neglected, such that $h_d = h_{d,\text{eq}}$. By means of the keyword `BdfRlx` one can select four options for computing the variables u_H, v_H and T_H: (i) constant relaxation time scale T_H, (ii) constant relaxation distance L_H, (iii) relaxation distance proportional to dune length L_d, and (iv) relaxation distance dependent on dune length L_d and water depth H. In the first three advection cases, the direction of the bedform migration is assumed to be equal to the direction of the depth averaged velocity in all cases, i.e.

$$u_H = c_H\frac{u}{U} \tag{12.14}$$

$$v_H = c_H\frac{v}{U} \tag{12.15}$$

DRAFT

where u and v are the x and y components of the depth averaged velocity, U the flow velocity magnitude $\sqrt{u^2+v^2}$, and c_H is the bedform celerity (magnitude) iven by a power relation derived for a generic transport formula approximation $s = a_s U^{b_s}$ as

$$c_H = \frac{b_s s}{H} = \frac{a_s b_s U^{b_s}}{H} = \frac{a_C U^{b_C}}{H} \tag{12.16}$$

while the relaxation time scale T_H is either given, or computed from $T_H = L_H/c_H$ where the relaxation distance L_H is either given or proportional to the dune length $_H = f_{LH} L_d$. In the fourth advection case T_H is computed again using $T_H = L_H/c_H$ but in that case the relaxation distance L_H is given by

$$L_H = \min\left[\left(\frac{H_{\max}}{H}\right)^{b_{LH}}, \phi_{\max}\right] L_d \tag{12.17}$$

and the bedform celerity is given by

$$c_H = a_{cH} \max\left[\left(\frac{H}{H_{\max}}\right)^{b_{cH}}, \gamma_{\min}\right] \frac{s_b}{H(1-\mathrm{Fr}^2)} \tag{12.18}$$

where s_b is the total bedload transport rate, Fr^2 is the squared Froude number U^2/gH and $H_{\max}$, b_{LH}, $\phi_{\max}$, a_{cH}, b_{cH} and $\gamma_{\min}$ are constants that can be set. In this case the bedform migration direction is assumed to be equal to the direction of the bedload transport vector $\boldsymbol{s}_b$.

For the dune length L_d used in some of the formulations above, you can choose either one of the following two options using keyword `BdfL`. If you choose to compute the dune height according Van Rijn (1984) then it will be computed as

$$L_d = 7.3H \tag{12.19}$$

otherwise it will be computed using the power relation

$$L_d = a_L H^{b_L} \tag{12.20}$$

Note that only the dune height is subject to the advective and relaxation behaviour formulated in Equation (12.13); for the dune length always these equilibrium formulations are used.

Finally, you may select one of three methods for computing the bedform roughness height by means of the keyword `BdfRou`; these methods are based on the formulations of Van Rijn (1984), Van Rijn (2007) or a power relation. This value will only be used if you use the trachytope option for specifying bed roughness and include either one of alluvial roughness formulae 105 and 106.

$$k_{s,d} = 1.1 h_d \left(1 - e^{-25 h_d/L_d}\right) \tag{12.21}$$

and in case of the power relation this quantity is computed as

$$k_{s,d} = a_R h_d^{b_R} \tag{12.22}$$

The roughness height related to ripples is set in both cases equal to $3D_{90}$. In case of Van Rijn (2007) the roughness heights neither depend on the computed dune height nor on the dune length; in that case the equations given in the following section are used.

DRAFT

12.2 Vegetation models

D-Flow FM includes a number of methods to include vegetation resistance in 2D models; each method has its own benefits and limitations.

method	benefit	limitation
bed friction	common practice; transferable across modelling systems	vegetation included in bed friction: bed shear stress too large; static vegetation map
trachytopes	separate terms for bed friction and vegetation drag	static vegetation map
dynamic	dynamic vegetation via online coupling; separate terms for bed friction and vegetation drag	more time-consuming

The first method is the common practice in a wide variety of hydraulic models: a spatially varying bed friction coefficient (see section 3.3.6). The trachytopes method is described in section 12.3. For an introduction of the dynamic vegetation option, you are referred to section 12.2.1.

12.2.1 Dynamic vegetation model

D-Flow FM offers the possibility of coupling to a dynamic vegetation model via the Basic Model Interface (BMI). By doing so a fast and flexible coupling platform is created to answer questions about the design and evaluation of Nature-based solutions (NbS), thereby enabling assessments that were considered too complicated and costly before. The vegetation dynamics is implemented in an open source ecological model written in Python. The code of the model is available at https://github.com/deltares-research/NBSDynamics/. The following sections briefly describe the philosophy and concepts of the vegetation model. For details, the user is referred to the GitHub website indicated above.

12.2.1.1 Philosophies of dynamic vegetation model

Faced with the problems of climate change and socio-economic pressure in many of the world's deltas and rivers, there is a rising interest in Nature-based solutions as a means to combine affordable and adaptive flood risk reduction with other ecosystem services. This has created a demand for tools that can quantify the development and performance of natural or nature-based systems like salt marshes, mangroves and river floodplains and the ecosystem services they provide. The NbS Dynamics modelling suite provides the tools to simulate interactions between hydrodynamics, sediment dynamics, morphodynamics and vegetation/ecology.

The ecological model can be as simple as a single habitat suitability rule, or endlessly complex involving multiple species, age classes and stressors. To facilitate the assessment of ecological processes, Delft3D Flexible Mesh has been expanded with summarizing statistics of ecologically relevant parameters (e.g. inundation time, bed shear stress). The ecological model is not restricted to vegetation only. Other biota that interact with hydrodynamics or morphology, such as mussels, algae and microfytobenthos, can also be computed with this modelling suite.

A substantial improvement over the earlier academic tools is the direct exchange of parameters through memory pointers, instead of via files. This is much faster, allows for flexible

DRAFT

exchange intervals (i.e. only when really needed) and allows for relatively independent development of both parts of the coupled code.

12.2.1.2 Conceptual description

The vegetation module in Python is coupled to a hydrodynamic D-Flow FM model. The Python environment has two functions: it contains the vegetation module and it 'orchestrates' the interaction between the hydrodynamic and the vegetation modules. 'Orchestrating' means that Python is used to define when and via which parameters the models interact. To do so, it uses the Basic Model Interface (BMI) technique (Hutton *et al.*, 2020; Peckham *et al.*, 2013). The BMI interface of D-Flow FM is described in more detail in Section 16.1.

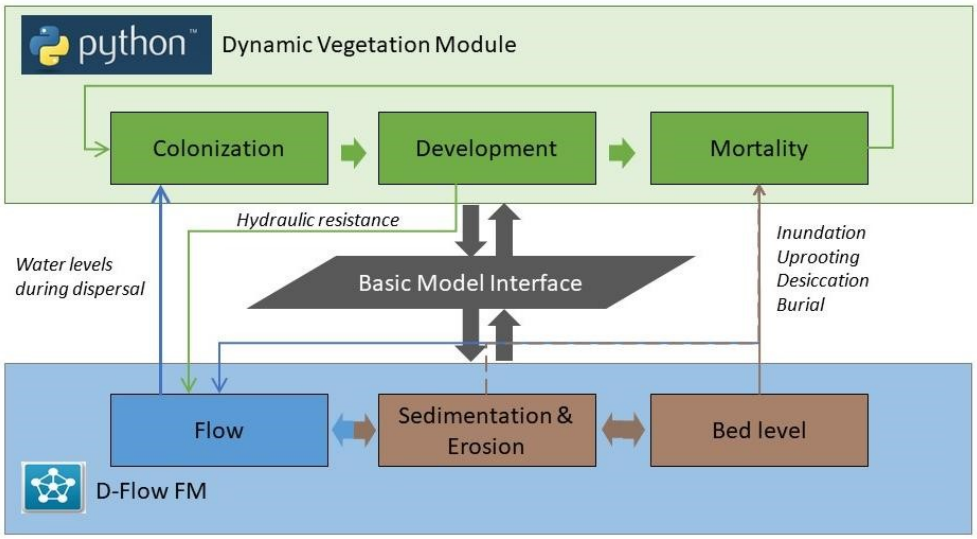


Figure 12.1: *Conceptual schematization of the modelling environment, visualizing the Basic Model Interface connecting the D-Flow FM model for hydrodynamic and morphodynamic computation with the Python environment for vegetation and ecological modelling. Figure after Oorschot et al. (2016) and Brückner et al. (2019).*

The modelling environment is illustrated by a conceptual schematization in Figure 12.1. The hydrodynamic module D-Flow FM computes for example the water level, flow velocity and bed shear stress and passes these results to the vegetation module (Python). The vegetation module then computes the vegetation biomass, expressed as a stem density, stem height and stem diameter. These vegetation parameters are used as inputs for D-Flow FM in the next time step.

12.3 Trachytopes

This functionality allows you to specify the bed roughness and flow resistance on a sub-grid level by defining and using various land use or roughness/resistance classes, further referred to as trachytopes after the Greek word $\tau\rho\alpha\chi\acute{\upsilon}\tau\eta\varsigma$ for roughness. The input parameters and files to use the trachytopes functionality are described in section C.8.

At every time step (or less frequent as requested by the user) the trachytopes are converted into a representative bed roughness C, k or n and optional linear flow resistance coefficient λ per velocity point with index j.

$$M = -\frac{1}{2}\lambda_j u_j \left|\boldsymbol{u}_j\right| \tag{12.23}$$

To save computational time the user may choose to update the computed bed roughness and resistance coefficients less frequently than every time step. See section C.8 for a description of the keywords and input files associated with this feature.

The following two sections describe the various classes of trachytopes distinguished and the way in which they are combined, respectively.

12.3.1 Trachytope classes

Three base classes of trachytopes are distinguished: area classes, line classes and point classes. The area classes (type range 51–200) basically cover the whole area, therefore, they are generally the dominant roughness factor. The line classes (type range 201–250) may be used to represent hedges and similar flow resistance elements; it will add anisotropy to the roughness field. The point class (type range 251–300) represents a set of point flow resistance elements. The next six sections provide an overview of the various trachytope formulae implemented.

Special classes (1–50)

In addition to the three base classes two special trachytope classes have been defined: a flood protected area and a composite trachytope class. The first class represents a sub-grid area that is protected from flooding and thus does not contribute to the bed roughness; however, the effect on the flow resistance should be taken into account. The second class can be used to make derived trachytope classes that are a combination of two other trachytopes: an area fraction α of trachytope type T_1 and an area fraction β (often equal to $1-\alpha$) of trachytope type T_2.

FormNr	Name	Formula
Special classes (1–50)		
1	flood protected area	area fraction shows up as f_b in Eqs. 12.75 and 12.78
2	composite trachytope	fraction α of type T_1 and fraction β (generally $\beta = 1-\alpha$) of type T_2

DRAFT

Area trachytope classes (51–200)

The class of area trachytopes is subdivided into three types: simple (51–100), alluvial (101–150) and vegetation (151–200). Four simple area trachytopes have been implemented representing the four standard roughness types of flow module.

FormNr	Name	Formula
51	White-Colebrook value	k
52	Chézy value	C
53	Manning value	$C = \sqrt[6]{h}/n$
54	z_0 value	$k = 30 z_0$

Six alluvial trachytopes have been implemented.

FormNr	Name	Formula
101	simplified Van Rijn	Equation (12.24)
102	power relation	Equation (12.25)
103	Van Rijn (1984)	Equations 12.26 to 12.34
104	Struiksma	Equations 12.35 to 12.38
105	bedforms quadratic	Equation (12.39)
106	bedforms linear	Equation (12.40)

The first alluvial roughness formula is a simplified version of the Van Rijn (1984) alluvial roughness predictor

$$k = A h^{0.7} \left[1 - e^{-B h^{-0.3}}\right] \tag{12.24}$$

it is obtained from Equation (12.26) by noting that $h_b \propto h^{0.7}$ and $L_b \propto h$ and ignoring the grain related roughness. The parameters A and B can be calibrated by the user. The second formula implemented is a straightforward general power law

$$C = A h^B \tag{12.25}$$

where A and B are calibration coefficients. The Van Rijn (1984) alluvial roughness predictor reads

$$k = k_{90} + 1.1 h_b \left(1 - e^{-25 h_b / L_b}\right) \tag{12.26}$$

where the bedform height h_b and length L_b are given by

$$h_b = 0.11 h \left(\frac{D_{50}}{h}\right)^{0.3} \left(1 - e^{-T/2}\right) (25 - T) \tag{12.27}$$

$$L_b = 7.3 h \tag{12.28}$$

where h is the local water depth and the transport stage parameter T is given by

$$T = \frac{u_*'^2 - u_{*,cr}^2}{u_{*,cr}^2} \tag{12.29}$$

where u'_* is the bed shear velocity given by

$$u'^2_* = gu^2/C^2_{g,90} \tag{12.30}$$

where

$$C_{g,90} = 18\ ^{10}\log(12h/k_{90}) \text{ and } k_{90} = 3D_{90} \tag{12.31}$$

and $u_{*,cr}$ is the critical bed shear velocity according Shields given by

$$u^2_{*,cr} = g\Delta D_{50}\theta_c \tag{12.32}$$

given

$$\theta_c = \begin{cases} 0.24/D_* & \text{if } D_* \leq 4 \\ 0.14D_*^{-0.64} & \text{if } 4 < D_* \leq 10 \\ 0.04D_*^{-0.10} & \text{if } 10 < D_* \leq 20 \\ 0.013D_*^{0.29} & \text{if } 20 < D_* \leq 150 \\ 0.055 & \text{if } 150 < D_* \end{cases} \tag{12.33}$$

where

$$D_* = D_{50}\left(\frac{g\Delta}{\nu^2}\right)^{1/3} \tag{12.34}$$

This predictor does not contain any calibration coefficients but requires D_{50} and D_{90} data from the morphology module. It does not include the advective and relaxation behaviour that is available by explicitly simulating the dune height as described in section 12.1 combined with trachytope number 106.

The second alluvial roughness predictor proposed by (Struiksma, pers. comm.) allows for a lot of adjustments, it reads

$$\frac{1}{C^2} = (1-\xi)\frac{1}{C^2_{90}} + \xi\frac{1}{C^2_{min}} \tag{12.35}$$

where

$$C_{90} = A_1\ ^{10}\log(A_2h/D_{90}) \tag{12.36}$$

and

$$\xi = \frac{\max(0, \theta_g - \theta_c)}{\theta_m - \theta_c}\frac{\theta^2_m - \theta_c\theta_g}{(\theta_m - \theta_c)\theta_g} \tag{12.37}$$

which varies from 0 at $\theta_g \leq \theta_c$ to 1 at $\theta_g = \theta_m$ where

$$\theta_g = \frac{u^2}{C^2_{90}\Delta D_{50}} \tag{12.38}$$

and $A_1, A_2, \theta_c, \theta_m, C_{\min}$ are coefficients that the user needs to specify. This formula requires also D_{50} and D_{90} data from the morphology module. The fifth formula is based on Van Rijn (2007) and reads

$$k = \min(\sqrt{k^2_{s,r} + k^2_{s,mr} + k^2_{s,d}}, \frac{h}{2}) \tag{12.39}$$

DRAFT

It uses the roughness heights of ripples k_r, mega-ripples k_{mr} and dunes k_d. These formulae depend on sediment properties D_{50} and D_{90} data which may be either specified as part of the roughness type or obtained from the morphology module. The sixth formula is similar, but uses a linear addition

$$k = \min(k_{s,r} + k_{s,mr} + k_{s,d}, \frac{h}{2}) \tag{12.40}$$

Four vegetation based area trachytopes have been implemented. Two formulae (referred to as 'Barneveld') are based on the work by Klopstra *et al.* (1996, 1997) and two on the work by Baptist (2005).

FormNr	Name	Formula
151	Barneveld 1	Eqs. 12.41 – 12.50, $C_D = 1.65$
152	Barneveld 2	Eqs. 12.41 – 12.47, 12.51 – 12.53
153	Baptist 1	Eqs. 12.54 and 12.55
154	Baptist 2	Eqs. 12.56, 12.58 and 12.59

The formula by Klopstra *et al.* (1997) reads

$$C = \frac{1}{h^{3/2}} \left\{ \begin{array}{l} \frac{2}{\sqrt{2A}} \left(\sqrt{C_3 e^{h_v\sqrt{2A}} + u_{v0}^2} - \sqrt{C_3 + u_{v0}^2} \right) + \\ \frac{u_{v0}}{\sqrt{2A}} \ln \left(\frac{(\sqrt{C_3 e^{h_v\sqrt{2A}} + u_{v0}^2} - u_{v0})(\sqrt{C_3 + u_{v0}^2} + u_{v0})}{(\sqrt{C_3 e^{h_v\sqrt{2A}} + u_{v0}^2} + u_{v0})(\sqrt{C_3 + u_{v0}^2} - u_{v0})} \right) + \\ \frac{\sqrt{g(h-(h_v-a))}}{\kappa} \left((h - (h_v - a)) \ln \left(\frac{h-(h_v-a)}{z_0} \right) - a \ln \left(\frac{a}{z_0} \right) - (h - h_v) \right) \end{array} \right\} \tag{12.41}$$

where

$$A = \frac{nC_D}{2\alpha} \tag{12.42}$$

$$C_3 = \frac{2g(h - h_v)}{\alpha\sqrt{2A}(e^{h_v\sqrt{2A}} + e^{-h_v\sqrt{2A}})} \tag{12.43}$$

$$a = \frac{1 + \sqrt{1 + \frac{4E_1^2\kappa^2(h-h_v)}{g}}}{\frac{2E_1^2\kappa^2}{g}} \tag{12.44}$$

and

$$z_0 = ae^{-F} \tag{12.45}$$

where

$$E_1 = \frac{\sqrt{2A}C_3 e^{h_v\sqrt{2A}}}{2\sqrt{C_3 e^{h_v\sqrt{2A}} + u_{v0}^2}} \tag{12.46}$$

DRAFT

and

$$F = \frac{\kappa\sqrt{C_3 e^{h_v\sqrt{2A}} + u_{v0}^2}}{\sqrt{g(h-(h_v-a))}} \tag{12.47}$$

Here, h is the water depth, h_v is the vegetation height, and $n = mD$ where m is the number of stems per square metre and D is the stem diameter. For the first implementation the parameter α in Equation (12.43) is given by

$$\alpha = \max(0.001, 0.01\sqrt{hh_v}) \tag{12.48}$$

and the velocity within the vegetation is approximated by $u_{v0}\sqrt{i}$ where

$$u_{v0}^2 = \frac{2g}{C_D n} \tag{12.49}$$

and i is the water level gradient. For emerged vegetation the first implementation reads

$$\frac{1}{C^2} = \frac{C_D n h}{2g} \tag{12.50}$$

The second implementation of Klopstra *et al.* (1996) is based on a modification by Van Velzen *et al.* (2003); it is identical except for the following modifications to Eqs. 12.48 – 12.50. The main difference between the two implementations is the inclusion of the roughness C_b of the bed itself (without vegetation). The parameter α in Equation (12.43) is now given by

$$\alpha = 0.0227 h_v^{0.7} \tag{12.51}$$

and the velocity within the vegetation is approximated by $u_{v0}\sqrt{i}$ where

$$u_{v0}^2 = \frac{h_v}{\frac{C_D h_v n}{2g} + \frac{1}{C_b^2}} \tag{12.52}$$

and i is the water level gradient. For emerged vegetation the second implementation reads

$$\frac{1}{C^2} = \frac{C_D n h}{2g} + \frac{1}{C_b^2} \tag{12.53}$$

For large values of C_b the latter two equations simplify to the corresponding equations of the first implementation. The first implementation requires vegetation height h_v and density n as input parameters (the drag coefficient C_D is equal to 1.65); for second implementation you'll also need to specify the drag coefficient C_D and the alluvial bed roughness k_b (C_b in Equation (12.53) is computed as $18\,^{10}\log(12h/k_b)$).

The first implementation of the roughness predictor by Baptist (Baptist, 2005) reads for the case of submerged vegetation

$$C = \frac{1}{\sqrt{\frac{1}{C_b^2} + \frac{C_D n h_v}{2g}}} + \frac{\sqrt{g}}{\kappa}\ln(\frac{h}{h_v}) \tag{12.54}$$

where n is the vegetation density ($n = mD$ where m is the number of stems per square metre and D is the stem diameter). The second term goes to zero at the transition from

DRAFT

submerged to emerged vegetation. At that transition the formula changes into the formula for non-submerged vegetation which reads

$$C = \frac{1}{\sqrt{\frac{1}{C_b^2} + \frac{C_D n h}{2g}}} \tag{12.55}$$

which is identical to the non-submerged case of the second implementation of the work by Klopstra *et al.* (1996) (see Equation (12.53)).

The drawback of the three vegetation based formulations above is that they parameterize the flow resistance by means of the bed roughness. Consequently, the presence of vegetation will lead to a higher bed roughness and thus to a higher bed shear stress and larger sediment transport rates in case of morphological computations. Therefore, we have included a $-\frac{\lambda}{2}u^2$ term in the momentum equation where λ represents the flow resistance of the vegetation. For the case of non-submerged vegetation $h < h_v$ the flow resistance and bed roughness are strictly separated

$$C = C_b \quad \text{and} \quad \lambda = C_D n \tag{12.56}$$

In the case of submerged vegetation $h > h_v$ the two terms can't be split in an equally clean manner. However, we can split the terms such that the bed shear stress computed using the depth averaged velocity u and the net bed roughness C equals the bed shear stress computed using the velocity u_v within the vegetation layer and the real bed roughness C_b.

$$\frac{u^2}{C^2} = \frac{u_v^2}{C_b^2} \tag{12.57}$$

With this additional requirement we can rewrite Equation (12.54) as

$$C = C_b + \frac{\sqrt{g}}{\kappa} \ln(\frac{h}{h_v}) \sqrt{1 + \frac{C_D n h_v C_b^2}{2g}} \tag{12.58}$$

and

$$\lambda = C_D n \frac{h_v}{h} \frac{C_b^2}{C^2} \tag{12.59}$$

which simplify to Equation (12.56) for $h = h_v$. Both formulae by Baptist require vegetation height h_v, density n, drag coefficient C_D and alluvial bed roughness C_b as input parameters.

Linear trachytope classes (201–250)

Two formulae have been implemented for linear trachytopes such as hedges or bridge piers.

FormNr	Name	Formula
201	hedges 1	Eqs. 12.60 to 12.62
202	hedges 2	Eqs. 12.63 to 12.65

The first implementation reads

$$\frac{1}{C^2} = \frac{h}{2g} \frac{L_{hedge}}{W_{cell} L_{cell}} \frac{1-\mu^2}{\mu^2} \tag{12.60}$$

DRAFT

where L_{hedge} is the projected length of the hedge, W_{cell} and L_{cell} are the width and length of the grid cell. The ratio L_{hedge}/W_{cell} may be interpreted as the number of hedges that the flow encounters per unit width. The second ratio is thus the inverse of the average distance between these hedges within the grid cell. The last term may be loosely referred to as the drag of the hedge, which is determined by the hedge pass factor μ given by

$$\mu = 1 + 0.175n\left(\frac{h}{h_v} - 2\right) \tag{12.61}$$

if the hedge extends above the water level ($h_v > h$) and is given by

$$\mu = 1 - 0.175n\left(\frac{h}{h_v}\right) \tag{12.62}$$

if the hedge is fully submerged ($h > h_v$) where n is a dimensionless hedge density. The second implementation reads

$$\frac{1}{C^2} = \frac{C_D n L_{hedge} h}{2g L_{cell} W_{cell}} \tag{12.63}$$

or equivalently

$$C = \sqrt{\frac{2gL_{cell}W_{cell}}{hL_{hedge}}}\left(\sqrt{\frac{1}{C_D n}}\right) \tag{12.64}$$

for non-submerged conditions and

$$C = \sqrt{\frac{2gL_{cell}W_{cell}}{hL_{hedge}}}\left(\frac{h_v}{h}\sqrt{\frac{1}{C_D n}} + m_0\sqrt{\frac{\left(\frac{h-h_v}{h}\right)^2}{1-\left(\frac{h-h_v}{h}\right)^2}}\right) \tag{12.65}$$

for submerged conditions. We recognize the same ratio $L_{cell}W_{cell}/L_{hedge}$ that represents the average distance between hedges. Equation (12.63) can be directly compared to similar equations for area trachytopes (Equation (12.50)), point trachytopes (Equation (12.66)). Note that the formula for computing the loss coefficient for a bridge explicitly includes the reduction in the flow area and the resulting increase in the effective flow velocity, whereas the above mentioned trachytope formulae don't.

Point trachytope classes: various (251–300)

One formula for point trachytopes has been implemented. It may be used to represent groups of individual trees or on a smaller scale plants.

FormNr	Name	Formula
251	trees	Eqn. 12.66

The implemented formula reads

$$C = \sqrt{\frac{2g}{C_D n \min(h_v, h)}} \tag{12.66}$$

where $n = mD$ with m the number of trees per unit area and D the characteristic tree diameter, h_v is the vegetation height and h is the local water depth. The formula is identical to Equation (12.55) except for the fact that the point trachytope formula has no bed roughness and area associated with it. The generalization of Equation (12.66) to the submerged case ($h > h_v$) lacks the extra term in Equation (12.54).

DRAFT

12.3.2 Averaging and accumulation of trachytopes

Point and linear roughnesses are accumulated by summing the inverse of the squared Chézy values C_i.

$$\frac{1}{C_{pnt}^2} = \sum_i \frac{1}{C_{pnt,i}^2} \tag{12.67}$$

$$\frac{1}{C_{lin}^2} = \sum_i \frac{1}{C_{lin,i}^2} \tag{12.68}$$

The area roughnesses are accumulated weighted by the surface area fraction f_i. These roughnesses are accumulated as White-Colebrook roughness values and as Chézy values; for the latter values both the linear sum ("parallel") and the sum of inverse of squared values ("serial") are computed. Roughness values are converted into each other as needed based on the local water depth.

$$k_{area} = \sum_i f_i k_i \tag{12.69}$$

$$\frac{1}{C_{area,s}^2} = \sum_i f_i \frac{1}{C_i^2} \tag{12.70}$$

$$C_{area,p} = \sum_i f_i C_i \tag{12.71}$$

For the fraction of the grid cell area for which no roughness class is specified the default roughness is used, via keywords `UnifFrictCoef` and `UnifFrictType`.

The flow resistance coefficients are also accumulated proportionally to the surface area fraction f_i associated with the trachytope considered. For the fraction of the grid cell area for which no flow resistance is specified, obviously none is used.

$$\lambda = \sum_i f_i \lambda_i \tag{12.72}$$

The final effective bed roughness of the grid cell may be computed by either one of the following two methods.

Method 1

The total mean roughness is computed by summing the White-Colebrook values for the areas and line and point resistance features.

$$k_m = k_{area} + k_{lin} + k_{pnt} \tag{12.73}$$

where $k_{lin} = 12h10^{-C_{lin}/18}$ and $k_{pnt} = 12h10^{-C_{pnt}/18}$. The effect of the water free area fraction f_b is taken into account by means of the following empirical relation in which $C_m = 18\ ^{10}\log(12h/k_m)$ is the mean Chézy value corresponding to the total mean White-Colebrook roughness value obtained from Equation (12.73).

$$f_b = \max(\min(0.843, f_b), 0.014) \tag{12.74}$$

$$C_{\text{total}} = C_m \left(1.12 - 0.25 f_b - 0.99\sqrt{f_b}\right) \tag{12.75}$$

DRAFT

The resulting C_{total} value is used in the computation. This method together with trachytope classes 1, 51, 101, 151 and 201 corresponds to the NIKURADSE option of the WAQUA/TRIWAQ flow solver.

Method 2

The total roughness is computed by first averaging over the serial and parallel averages of the Chézy values according

$$C_{area} = \alpha_s C_{area,s} + (1 - \alpha_s) C_{area,p} \tag{12.76}$$

where $\alpha_s = 0.6$ by default. Subsequenty the effect of the water free area fraction f_b is taken into account by means of the following empirical relation (identical to Equation (12.75) of method 1).

$$f_b = \max(\min(0.843, f_b), 0.014) \tag{12.77}$$

$$C_{area,corr} = C_{area} \left(1.12 - 0.25 f_b - 0.99\sqrt{f_b}\right) \tag{12.78}$$

Finally the Chézy value representing the total bed roughness is computed by accumulating the inverses of the squared Chézy values.

$$\frac{1}{C_{total}^2} = \frac{1}{C_{area,corr}^2} + \frac{1}{C_{lin}^2} + \frac{1}{C_{pnt}^2} \tag{12.79}$$

The resulting C_{total} value is used in the computation. This method together with trachytope classes 1, 51, 52, 53, 101, 152, 202 and 251 corresponds to the ROUGHCOMBINATION option of the WAQUA/TRIWAQ flow solver.

DRAFT

13 Calibration factor for bed roughness

The current chapter explains the effect of the calibration factor. The calibration factor is multiplier of the roughness. The calibration factor may be constant, discharge- or water-level-dependent. By assigning different areas to different calibration classes, each region can be independently calibrated. Calibration roughness definitions can also be combined by assigning multiple field definitions and a weighting for each gridcell edge (net link). This approach allows thus both abrupt and gradual transitions and the modeller can control how the calibration factor is going to be used. The approach is similar to the roughness and link definitions used in the trachytopes module for alluivial- and bedform-roughness (see chapter 12).

The calibration factor is defined by means of two files (see section C.10):

- ◇ Calibration factor definition file (CLD-file). This file defines the calibration factor (e.g. constant, water-level- or discharge dependent).
- ◇ Calibration factor area file (CLL-file). This file associates calibration factor defintions with the edges of the model grid and include a relative weighting.

The resulting weighted calibration factor is multiplied by the roughness type as expressed by the `UnifFrictCoef` keyword in the [physics] section in the .mdu file (see Appendix A). This implies that the effect of the calibration will be different depending on the definition of `UnifFrictCoef`. For example, if the `UnifFrictCoef`=0 (i.e. Chézy) a local calibration factor of 0.5 will imply an increase of the bed shear stress, whereas if `UnifFrictCoef`=2 (i.e. White-Colebrook) a reduction of the bed shear stress may be expected. A background calibration factor equal to one is applied if the sum of the weights at a single link is lower than one.

The calibration factor approach cannot be combined with multiple roughness types as specified through the external forcing file. This will lead to an error. Such a spatial variation in the roughness can be achieved by defining these areas through the trachytopes module.

It is good to be aware of the following known differences with the trachytopes module:

- ◇ The weighting of the calibration factors is done for all entries in the calibration area defintion file. Averaging for the trachytopes area file is only done for the last sequence of trachytope area defintions at one and the same location.
- ◇ When a calibration factor definition is imposed at a location outside of the grid, and this calibration factor definition depends on a water level station or cross-section which is also outside of the domain, the model crashes, however the trachytopes module allows this.

DRAFT

14 Wind

Various external influences can exert a force on the flow field. One of these influences is the wind. The force exerted by the wind is coupled to the flow equations as a shear stress. The magnitude is determined by the following widely used quadratic expression:

$$|\boldsymbol{\tau}_s| = \rho_a C_d U_{10}^2 \tag{14.1}$$

where:

ρ_a	the density of air.
U_{10}	the wind speed 10 meter above the free surface (time and space dependent).
C_d	the wind drag coefficient, dependent on U_{10}.

In order to specify the wind shear stress, a drag coefficient is required as well as the wind field in terms of velocity magnitude and wind direction. In this chapter, the backgrounds are provided of how wind fields should be imposed, in addition to section 6.4.8.7. Relevant definitions are addressed in section 14.1, whereas supported file formats are addressed in section 14.2.

The density of air in the computation of the wind stress is considered constant as a default, but it is also possible to use a time- and space-varying air density. The air density can be prescribed directly as a timeseries field or it can be computed from prescribed time- and space-varying air pressure, air temperature and dew point temperature (see section E.1.7).

The steps to compute the air density [kg m^{-3}] are (following ECMWF (2023)):

$$e = e_0 \exp(17.502(T_d - T_0)/(T_d - 32.19)) \tag{14.2}$$

$$q_v = \frac{e}{p + \varepsilon^*(p - e)} \tag{14.3}$$

$$\varepsilon^* = \frac{R_v}{R_d} - 1 \tag{14.4}$$

$$T_v = (1 + \varepsilon^* q_v)T \tag{14.5}$$

$$\rho_a = \frac{p}{R_d T_v} \tag{14.6}$$

where

e	water vapour saturation pressure [Pa].
e_0	water vapour saturation pressure over water at T_0 [611.21 Pa].
T_0	triple point temperature [K].
T_d	dew point temperature [K].
q_v	specific humidity [g kg^{-1}].
p	atmospheric pressure [Pa].
R_v	gas constant for water vapor [461.5249 J kg^{-1} K^{-1}].
R_d	gas constant for dry air [287.0596 J kg^{-1} K^{-1}].
T_v	virtual temperature [K].
T	air temperature [K].

14.1 Definitions

When imposing wind conditions, two definitions are respected: a definition for the wind direction (see section 14.1.1) and a definition regarding the drag coefficient (see section 14.1.2).

DRAFT

14.1.1 Nautical convention

The wind direction is defined according to the nautical definition, i.e. relative to true North and positive measured clockwise. In Figure 14.1 the wind direction is about +60 degrees, i.e. an East-North-East wind.

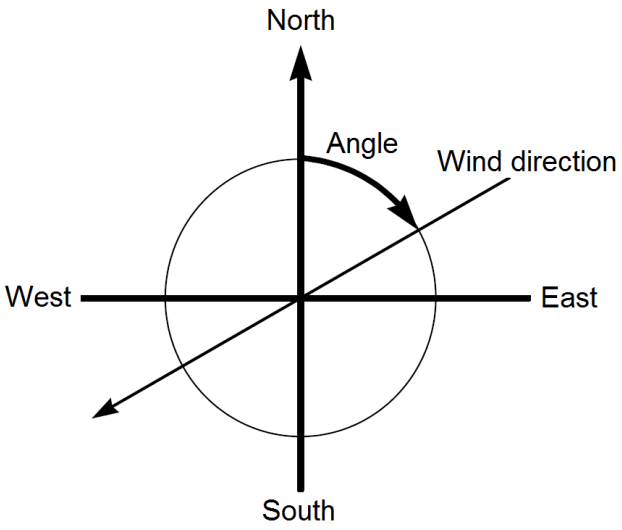


Figure 14.1: *Nautical conventions for the wind.*

14.1.2 Drag coefficient

The user can select how the wind drag coefficient should be computed, by specifying the type of wind drag formulation in the mdu-file. For this purpose, the keyword `ICdtyp` should be used. Nine options are available:

- `ICdtyp = 1` – constant drag coefficient,
- `ICdtyp = 2` – linearly varying drag coefficient (cf. Smith and Banke (1975)),
- `ICdtyp = 3` – piecewise linearly varying drag coefficient (cf. Smith and Banke (1975)),
- `ICdtyp = 4` – Charnock (1955) formulation,
- `ICdtyp = 5` – Hwang (2005a) and Hwang (2005b) formulation,
- `ICdtyp = 6` – Wüest and Lorke (2003) formulation,
- `ICdtyp = 7` – Hersbach (2011) formulation,
- `ICdtyp = 8` – Charnock (1955) formulation plus viscous effect,
- `ICdtyp = 9` – Garratt (1977) formulation.

If one of the Smith & Banke type dependencies is selected, the additional entries `Cdbreakpoints` and `Windspeedbreakpoints` come into play. In the following sections, these various options are described in more detail.

Smith & Banke type formulation

When specifying a Smith & Banke type dependency, the definition as sketched in Figure 14.2 should be kept in mind.

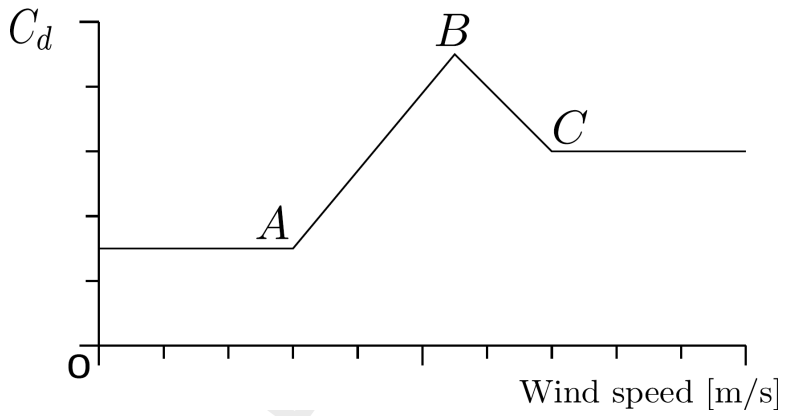


Figure 14.2: *Prescription of the dependency of the wind drag coefficient C_d on the wind speed is achieved by means of at least 1 point, with a maximum of 3 points.*

From this sketch, it can be seen that the wind drag is considered as dependent on the wind speed in a piecewise linear way. The options, that are facilitated in this respect, are:

- ◇ define *one* set of coordinates (breakpoint A), specifying a constant drag coefficient, valid for all wind speeds,
- ◇ define *two* sets of coordinates (breakpoints A and B), specifying a linearly varying dependency for one range of wind speeds,
- ◇ define *three* sets of coordinates (breakpoints A, B and C), specifying a piecewise linear dependency for two ranges of wind speeds.

Remark that for the latter two options, the drag coefficient is taken constant for wind speeds lower/higher than the lowest/highest specified wind speed, with a drag coefficient equal to the drag coefficient associated with the lowest/highest specified lowest/highest wind speed. In case of three breakpoints, the expression reads:

$$C_d\left(U_{10}\right) = \begin{cases} C_d^A, & U_{10} \leq U_{10}^A, \\ C_d^A + \left(C_d^B - C_d^A\right) \dfrac{U_{10} - U_{10}^A}{U_{10}^B - U_{10}^A}, & U_{10}^A \leq U_{10} \leq U_{10}^B, \\ C_d^B + \left(C_d^C - C_d^B\right) \dfrac{U_{10} - U_{10}^B}{U_{10}^C - U_{10}^B}, & U_{10}^B \leq U_{10} \leq U_{10}^C, \\ C_d^C, & U_{10}^C \leq U_{10}, \end{cases} \tag{14.7}$$

By means of the entries `Cdbreakpoints` and `Windspeedbreakpoints`, the coordinates of the breakpoints (see Figure 14.2) can be specified. Typical values associated with the Smith and Banke (1975) formulation are $C_d = 6.3 \times 10^{-4}$ for $U = 0$ m/s and $C_d = 7.23 \times 10^{-3}$ for $U = 100$ m/s. In this case, the entries in the mdu-file should be specified as follows:

```
[wind]
ICdtyp                                = 2
```

```
Cdbreakpoints                    =  0.00063   0.00723
Windspeedbreakpoints             =  0.00000 100.00000
```

Relative wind

In D-Flow FM, there is the possibility to compute wind shear stress based on the *relative* wind velocity, i.e. relative to the flow velocity. This becomes important when the flow is predominantly forced by the wind and the flow velocity is in the order of the wind velocity. In this way, one can avoid that the wind still forces the flow, despite a zero (or small) difference between flow and wind speed. In case of relative wind Equation (14.1) is changed to

$$|\boldsymbol{\tau}_s| = \rho_a C_d \left(U_{10} - \alpha U_{flow}\right)^2 \tag{14.8}$$

where:

ρ_a	the density of air.
U_{10}	the wind speed 10 meter above the free surface (time and space dependent).
C_d	the wind drag coefficient, dependent on U_{10}.
U_{flow}	the flow velocity.
α	the relative wind factor.

This option can be switched on via keyword `Relativewind`. Then, a value between 0 and 1 has to be specified, which acts as a factor for relative wind, as shown in Equation (14.8).

Charnock formulation

The Charnock formulation (see Charnock (1955)) is based on the assumption of a fully developed turbulent boundary layer of the wind flow over the water surface. The associated wind speed profile follows a logithmic shape. In the Charnock formulation, the wind speed is considered at 10 meters above the free water surface, hence yielding the following expression:

$$\frac{U_{10}}{u_*} = \frac{1}{\kappa} \ln\left(\frac{z_{10}}{z_0}\right) \tag{14.9}$$

with κ the Von Kármán constant, z_{10} the distance to the water surface (equal to 10 m), u_* the friction velocity and U_{10} the wind speed at 10 m above the water surface. The drag coefficient C_d is defined as:

$$C_d = \frac{u_*^2}{U_{10}^2}. \tag{14.10}$$

Charnock (1955) has proposed to represent the friction of the water surface as z_0 according to:

$$z_0 = \frac{b\, u_*^2}{g}, \tag{14.11}$$

with g the gravitation acceleration and b a specific constant. Charnock (1955) has proposed $b = 0.012$. The value of the constant b can be specified in the mdu-file by the user by means of one single value for `Cdbreakpoints`. Since the above relation yields an implicit relation for u_*, the system is solved for iteratively. The user should be aware of interpretation of the specified wind field as the wind field at 10 m above the water surface. See Table C.9 for a space and time varying Charnock coefficient b.

DRAFT

Hwang formulation

The dynamic roughness could also be related to the steady state wave conditions of the flow field under consideration. The connection of the wave parameters with the drag coefficient as elaborated by Hwang (2005a) is available within D-Flow FM through `ICdtyp = 5`, given a wave field. The Hwang-formulation interprets the user defined wind speed as the wind speed at 10 m above the water surface.

The drag coefficient is computed as:

$$C_d = \left[\frac{1}{\kappa}\ln\left(\frac{k_p z_{10}}{k_p z_0}\right)\right]^{-2} \tag{14.12}$$

with $z_{10} = 10$ m, κ the Von Kármán constant. With wavelength scaling, $k_p z_0$ is the natural expression of the dimensionless roughness, where k_p is the wave number of the spectral peak, computed on the basis of the actual water depth and the provided peak period T_p as wave field. Further following Hwang (2005a),

$$k_p z_0 = \pi \exp\left(-\kappa C_{\lambda/2}^{-0.5}\right) \tag{14.13}$$

in which $C_{\lambda/2}$ is the drag coefficient at half the wavelength above surface. This parameter $C_{\lambda/2}$ is computed as:

$$C_{\lambda/2} = A_{10}\left(\frac{\omega_p U_{10}}{g}\right)^{a_{10}} \tag{14.14}$$

in which $A_{10} = 1.289 \times 10^{-3}$, $a_{10} = 0.815$, U_{10} the wind speed at 10 m above the water surface and ω_p the wave peak frequency ($\omega_p = 2\pi/T_p$). Thus, the drag coefficient C_d is defined.

Wuest formulation

Wüest and Lorke (2003) define the drag coefficient as

$$C_d = \begin{cases} 0.00063 + 0.000066 U_{10} & \text{if } U_{10} > 4 \\ 0.0044 \max(0.1, U_{10})^{-1.15} & \text{if } U_{10} \leq 4 \end{cases} \tag{14.15}$$

Hersbach formulation

Hersbach (2011) defines the drag coefficient as

$$C_d = \left(\frac{\kappa}{b_n^{\text{fit}}}\right)^2 \tag{14.16}$$

where b_n^{fit} is split into a viscous b_n^{ν} and Charnock b_n^{α} contribution as

$$b_n^{\text{fit}} = \left[(b_n^{\nu})^p + (b_n^{\alpha})^p\right]^{1/p} \tag{14.17}$$

$$b_n^{\nu} = -1.47 + 0.93 \ln R \tag{14.18}$$

$$b_n^{\alpha} = 2.65 - 1.44 \ln A - 0.015 (\ln A)^2 \tag{14.19}$$

in which

$$A = \frac{\alpha_{ch}}{g z_{10}} (\kappa U_{10})^2 \tag{14.20}$$

DRAFT

and

$$R = \frac{z_{10}}{\alpha_M \nu_{\text{air}}} \kappa U_{10} \tag{14.21}$$

where z_{10} equals the elevation at which the velocity is given, i.e. 10 m. The values of the constants α_{ch} and α_M can be specified in the mdu-file by the user by means of two values for `Cdbreakpoints`.

Charnock with viscous

When this option is selected, the drag coefficient is computed according Equation (14.10) where u_* is determined iteratively using

$$z_0 = \frac{b_1 u_*^2}{g} + \frac{b_2 \nu_{\text{air}}}{u_*} \tag{14.22}$$

and Equation (14.9). The values of the constants b_1 and b_2 can be specified in the mdu-file by the user by means of two values for `Cdbreakpoints`.

Garratt formulation

Garratt (1977) defines the drag coefficient as

$$C_d = \min(10^{-3}(0.75 + 0.067 U_{10}), 0.0035) \tag{14.23}$$

This formulation can also be represented as a Smith & Banke type formulation.

```
[wind]
ICdtyp                          =  2
Cdbreakpoints                   =  0.00075   0.0035
Windspeedbreakpoints            =  0.00000  41.0448
```

14.2 File formats

The wind field should be provided by means of meteo forcing file(s). Most commonly used types are various ASCII-formatted files and NetCDF. Section C.6.2.3 lists all supported formats. This file should contain the grid on which the wind field is defined as well as the wind velocity vector(s).

D-Flow FM currently supports four types of wind field prescriptions, i.e. four grid types on which the wind field can be given. This wind grid does not need to be the same as the computational grid. The grid options to provide the wind data on are:

1. the computational grid — in this case, no specific wind grid is provided. The provided wind field is considered to be uniform over the entire model area. The wind field can be time dependent.
2. an equidistant grid — in this case, a wind field can be prescribed that varies both in space and in time. A Cartesian ArcInfo-type file or NetCDF file should be provided on which the wind field is defined.
3. a curvilinear grid — this case is conceptually similar to the previous type (the equidistant grid) in the sense that a wind field can be imposed that both varies in space and time. However, for ArcInfo-type data files, a separate file should be provided in which a curvilinear grid is defined (a classic $<*.$grd$>$-type file as known from Delft3D-FLOW) on which the wind field is defined. For NetCDF this is not necessary: all meteo grid coordinates must be present in the same NetCDF file that contains the meteo forcing data itself.

DRAFT

4 a spiderweb grid — this type of wind specification is specially devoted to cyclone winds. In this case, a cyclone wind field is given on a polar grid with the center ('eye') of the cyclone being the origin of the polar coordinate system. The location of this eye and the associated wind field usually varies in time.

Each of these filetypes can be assigned through the entry in the new external forcings file (the <∗.ext>-file) named `forcingFileType`.

Warning:

- ◇ Deprecated: In the old <*.ext> file, the keyword was called `FILETYPE`. Now, the new external forcings file is the only place where meteo must be specified.

In this chapter, the various types of wind field specifications are highlighted subsequently. Each of the options is illustrated by means of an example.

14.2.1 Defined on the computational grid

In D-Flow FM, the specification of the wind on the computational grid is comparable to the specification of a spatially uniform wind, since no separate wind grid is provided to the model. The specification of a uniform wind field can be done in two ways:

1 componentwise: as velocity in the longitudinal x-direction [m/s] and in the latitudal y-direction [m/s] — with `forcingFileType=bcAscii` (Deprecated: formerly this was via `FILETYPE=1` and a <*.tim> file in the old <*.ext> file).
2 by magnitude [m/s] and direction [degN] (see Figure 14.1) — with `forcingFileType = uniMagDir` (Deprecated: formerly this was `FILETYPE=2` in the old <*.ext> file).

These two types are treated below separately.

14.2.1.1 Specification of uniform wind through velocity components

Since no particular wind grid is used, only timeseries for the x-component and the y-component of the wind need to be specified. The specification of these timeseries can be done separately (one single file for the x-component and one single file for the y-component) or jointly (one single file containing the x-component and the y-component of the wind).

Uniform wind should be provided as a <*.bc> file.

Warning:

- ◇ Deprecated: formerly, this was via a <*.wnd>-file containing either 2 colums (in case of separate specification of the x-component and y-component of the wind) or 3 columns (in case of joint specification of the velocity components). In either case, the first column contains the time in minutes with respect to the overall reference time.

Example

As an example, a uniform wind field is applied to a certain model. The uniform wind is provided in a <*.bc> file named `windxdirydir.bc`. The contents of this wind file are:

```
[Forcing]
Name                = global
Function            = timeSeries
Time-interpolation  = linear
Vector              = windxy:wx,wy
```

```
Quantity            = time
Unit                = minutes since 2006-01-01 0:00:00
Quantity            = wx
Unit                = ms-1
Quantity            = wy
Unit                = ms-1
   0.00000   10.00000   10.00000
  60.00000  -10.00000  -10.00000
```

Warning:

- ◇ Deprecated: The old external forcing file is being deprecated, as well as timeseries files <*.tim> and <*.wnd>. The example below is kept only for migration purposes:

Before, the uniform wind could also be provided in a <*.wnd> file named `windxdirydir.wnd`. The contents of this wind file are:

```
   0.00000   10.00000   10.00000
  60.00000  -10.00000  -10.00000
```

The first column denotes the time in minutes with respect to the reference date (specified in the mdu-file). The second column denotes the wind velocity in x-direction, whereas the third column denotes the wind velocity in y-direction; both wind components are provided in one single file.

The connection with the flow model itself is laid through the external forcings file. The actual specification of the wind is in this case:

```
[Meteo]
quantity            = windxy
forcingFile         = windxdirydir.bc
forcingFileType     = bcAscii
interpolationMethod = linearSpaceTime
operand             = override
```

Warning:

- ◇ Deprecated: The old external forcing file is being deprecated. The example below is kept only for migration purposes:

```
QUANTITY =windxy
FILENAME =windxdirydir.wnd
FILETYPE =1
METHOD   =1
OPERAND  =O
```

Since the two components are given in one single file, the `QUANTITY` is set to `windxy`. If two separate files would have been provided, the `QUANTITY` would have been set to `windx` and `windy` over two separate datablocks in the external forcings file.

DRAFT

14.2.1.2 Specification of uniform wind through magnitude and direction

Instead of specifying the separate components of the wind field, the uniform wind vector can also be prescribed through its magnitude and direction (see Figure 14.1).

This kind of specification should be done by means of one single file, containing three columns, representing the time (in minutes with respect to the reference date), the velocity magnitude [m/s], not necessarily positive, and the direction (nautical convention).

Example

As an example, the previous uniform wind case is reformulated as a case with magnitude and direction of the wind field prescribed as two timeseries. The unimagdir wind is provided in a <*.bc> file. The contents of this wind file are:

```
[Forcing]
Name                = global
Function            = timeSeries
Time-interpolation  = linear
Vector              = windxy:magnitude,direction
Quantity            = time
Unit                = minutes since 2006-01-01 0:00:00
Quantity            = magnitude
Unit                = ms-1
Quantity            = direction
Unit                = degree
   0.00000   14.14213562373095  225.00000
  60.00000  -14.14213562373095  225.00000
```

The connection with the flow model itself is laid through the external forcings file. The actual specification of the wind is in this case:

```
[Meteo]
quantity            = windxy
forcingFile         = windinput.bc
forcingFileType     = bcAscii
interpolationMethod = linearSpaceTime
operand             = override
```

Warning:

- ◇ Deprecated: The old external forcing file and timeseries file is being deprecated. The example below is kept only for migration purposes:

The unimagdir wind could also be provided in a file named <windinput.wnd>. The contents of this file are:

```
   0.00000   14.14213562373095  225.00000
  60.00000  -14.14213562373095  225.00000
```

The first column denotes the time in minutes with respect to the reference date (specified in the mdu-file). The second column denotes the wind velocity magnitude, whereas the third column denotes the wind direction. Note that there is a clear difference between the above case and a case in which the magnitude is kept positive (14.1421 m/s) and the direction varies (and hence *rotates*!) from 225 degN to 45 degN.

DRAFT

The old external forcings file would then look like:

```
QUANTITY =windxy
FILENAME =windinput.wnd
FILETYPE =2
METHOD   =1
OPERAND  =O
```

14.2.2 Defined on an equidistant grid

The vector components of the velocity vectors can also be specified on a distinct grid, either of equidistant type or of curvilinear type. In both cases, the characteristics of the grid should be provided. In case of an equidistant grid, the grid is specified in arcinfo-style. That means, the constant grid sizes Δx and Δy should be specified such that a grid is spanned with respect to the location of the lower left corner of the grid (either the center of the lower left cell or the lower left corner of the lower left cell).

Example

As an example, a grid with $\Delta x = \Delta y = 100$ m is spanned, based on the center of the lower left cell, located at $x = y = 60$ m with respect to the origin. The input data for the x-component and the y-component should be specified separately, in two distinct files. The input of the x-component data should be given in an $<*$.amu$>$-type file, such as $<$windxdir.amu$>$ as an example:

```
### START OF HEADER
### This file is created by Deltares
### Additional commments
FileVersion      =    1.03
filetype         =    meteo_on_equidistant_grid
NODATA_value     =    -9999.0
n_cols           =    5
n_rows           =    4
grid_unit        =    m
x_llcenter       =     60
y_llcenter       =     60
dx               =    110
dy               =    110
n_quantity       =    1
quantity1        =    x_wind
unit1            =    m s-1
### END OF HEADER
TIME = 0 hours since 2006-01-01 00:00:00 +00:00
10 10 10 10 10
10 10 10 10 10
10 10 10 10 10
10 10 10 10 10
TIME = 1 hours since 2006-01-01 00:00:00 +00:00
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
```

For the y-component data, a similar file (e.g. $<$windydir.amv$>$) should be provided. In addition, the pressure could be specified in a similar file (e.g. $<$pressure.amp$>$). Note that `x_llcorner` and `y_llcorner`, instead of `x_llcenter` and `y_llcenter`, are also supported.

Wind on an equidistant grid has been provided a filetype specification as `forcingFileType=ArcInfo`.

DRAFT

The connection with the flow model itself is laid through the external forcings file. The actual specification of the wind is in this case:

```
[Meteo]
quantity            = windx
forcingFile         = windxdir.amu
forcingFileType     = ArcInfo
interpolationMethod = linearSpaceTime
operand             = override

[Meteo]
quantity            = windy
forcingFile         = windydir.amv
forcingFileType     = ArcInfo
interpolationMethod = linearSpaceTime
operand             = override

[Meteo]
quantity            = atmosphericpressure
forcingFile         = pressure.amp
forcingFileType     = ArcInfo
interpolationMethod = linearSpaceTime
operand             = override
```

Warning:

◇ Deprecated: The old external forcing file is being deprecated. The example below is kept only for migration purposes:

```
QUANTITY =windx
FILENAME =windxdir.amu
FILETYPE =4
METHOD   =2
OPERAND  =O

QUANTITY =windy
FILENAME =windydir.amv
FILETYPE =4
METHOD   =2
OPERAND  =O

QUANTITY =atmosphericpressure
FILENAME =pressure.amp
FILETYPE =4
METHOD   =2
OPERAND  =O
```

14.2.3 Defined on a curvilinear grid

In analogy with the wind specification on an equidistant grid, the wind can be specified on a curvilinear grid. This curvilinear grid should be provided as a classic $<*$.grd$>$-file as known from Delft3D-FLOW. A difference with the equidistant grid wind is the necessity to compile all data blocks (i.e. pressure, x-component and y-component) in one single file. This file should have the extension $<*$.apwxwy$>$. The sequence of this datablock is:

1 pressure,
2 x-velocity component,
3 y-velocity component.

DRAFT

Example

As an example, a curvilinear grid named <meteo.grd> is present, providing the underlying coordinates of the wind data field. The input data, comprising the atmospheric pressure, the x-velocity component *and* the y-velocity component, are given in one single file (as is compulsory). The contents of the example <meteo.apwxwy>-file is:

```
### START OF HEADER
### This file is created by Deltares
### Additional commments
FileVersion      =    1.03
filetype         =    meteo_on_curvilinear_grid
NODATA_value     =    -9999.0
grid_file        =    meteo.grd
first_data_value =    grid_llcorner
data_row         =    grid_row
n_quantity       =    3
quantity1        =    air_pressure
unit1            =    Pa
quantity2        =    x_wind
unit2            =    m s-1
quantity3        =    y_wind
unit3            =    m s-1
### END OF HEADER
TIME = 0.0 hours since 2006-01-01 00:00:00 +00:00
101325 101325 101325 101325 101325
101325 101325 101325 101325 101325
101325 101325 101325 101325 101325
101325 101325 101325 101325 101325
10 10 10 10 10
10 10 10 10 10
10 10 10 10 10
10 10 10 10 10
10 10 10 10 10
10 10 10 10 10
10 10 10 10 10
TIME = 1.0 hours since 2006-01-01 00:00:00 +00:00
101325 101325 101325 101325 101325
101325 101325 101325 101325 101325
101325 101325 101325 101325 101325
101325 101325 101325 101325 101325
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
-10 -10 -10 -10 -10
```

Note that `grid_llcenter`, instead of `grid_llcorner`, is also supported. On the contrary, `grid_column` is *not* supported instead of `grid_row`.

Wind on a curvilinear grid has been provided a filetype specification as `FILETYPE=6`. The connection with the flow model itself is laid through the external forcings file. The actual specification of the wind is in this case:

```
QUANTITY =airpressure_windx_windy
FILENAME =meteo.apwxwy
FILETYPE =6
METHOD   =3
OPERAND  =O
```

DRAFT

Notice that `METHOD=3` is chosen for wind on a curvilinear grid, instead of `METHOD=2` in case of wind on an equidistant grid.

14.2.4 Space and time varying Charnock coefficients

The value for the Charnock coefficient b in Equation (14.11) can be a constant given in the mdu-file, but also be given as a space and time varying field.

For a space and time varying Charnock coefficient, the user should provide a netCDF-file with meteorological forcing, including Charnock coefficients, and use the file specification `forcingFileFormat=netcdf`.

The specification is in this case in the new <*.ext> file:

```
[Meteo]
quantity            = airpressure_windx_windy_charnock
forcingFile         = meteo.nc
forcingFileType     = netcdf
interpolationMethod = linearSpaceTime
operand             = override
```

Warning:

- ◇ Deprecated: The old external forcing file is being deprecated. The example below is kept only for migration purposes:

```
QUANTITY =airpressure_windx_windy_charnock
FILENAME =meteo.nc
FILETYPE =11
METHOD   =3
OPERAND  =O
```

14.2.5 Rainfall

Rainfall can be defined as `rainfall` and `rainfall_rate`. Both can be provided as a timeseries, but they have a different meaning for the value at each time. The difference is: `rainfall` is the cumulative precipitation [mm] (for each time in the timeseries, measured since the previous time in that timeseries) and `rainfall_rate` is the instantaneous precipitation [mm/day] on a particular time. For example, `rainfall` data files could provided 10-minute precipitation sums on 10-minute intervals.

The specification is in the new <*.ext> file:

```
[Meteo]
quantity            = rainfall_rate
forcingFile         = rad_nl25_rac_mfbs_5min.nc
forcingFileType     = netcdf
forcingVariableName = image1_image_data
interpolationMethod = linearSpaceTime
operand             = override
```

Warning:

- ◇ Deprecated: The old external forcing file is being deprecated. The example below is kept only for migration purposes:

```
QUANTITY=rainfall_rate
FILENAME=rad_nl25_rac_mfbs_5min.nc
VARNAME =image1_image_data
FILETYPE=11
METHOD=3
OPERAND=O
```

The unit of rainfall rate is [mm/day]. So, this is the case for rainfall rate input either via a <*.bc> file or a netCDF-file.

14.2.6 Defined on a spiderweb grid

Cyclone winds can be imposed by means of a 'spiderweb'-like polar grid. The origin typically coincides with the cyclone eye and can move in time. The origin of the spiderweb should be given as longitude (for x_{eye}) and latitude (for y_{eye}). The number of rows (discretisation in radial direction) and the number of columns (discretisation in angular direction) should be given, as well as the radius of the grid (in meters). The definition of the spiderweb grid is illustrated in Figure 14.3.

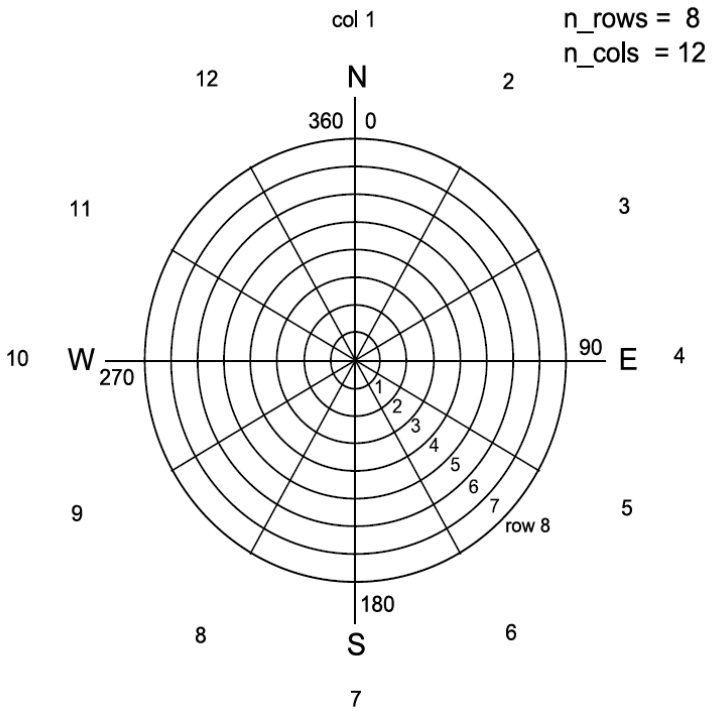


Figure 14.3: *Grid definition of the spiderweb grid for cyclone winds.*

The files containing the spiderweb data and metadata have the extension <∗.spw>. They consist of a global header containing properties that are not varying in time, followed by blocks of data for subsequent time levels. Each of these data blocks is headed by a set of properties for the corresponding time level. A detailed description of the spiderweb file format can be found in section C.14.3.

Through the specified unit for atmospheric pressure in the file, it can be specified whether the values should be interpreted as mbar (=hPa), instead of Pa, which is the default. Specifying

the spiderweb merge fraction β in `spw_merge_frac` allows for linear fading of wind speed and pressure drop towards the outer rim. For a spiderweb with radius R, the weigth assigned to the spiderweb wind and pressure at a radius r is given by $(R - r)/\beta R$ for r between $(1 - \beta)R$ and R. The weight equals unity within the inner circle and zero beyond the outer rim.

Example

As an example, a spiderweb grid named <spwsimple.spw> is present, providing the underlying coordinates of the wind data field. The input data, comprising the atmospheric pressure drops, the wind velocity magnitudes (in [m/s]) *and* the wind directions (in [degN]), are given in one single file (as is compulsory). The contents of the example <spwsimple.spw>-file is:

```
### Spiders web derived from TRACK file: gonu.trk
### This file is created by Deltares
### All text on a line behind the first # is parsed as commentary
### Additional commments
FileVersion      = 1.03
filetype         = meteo_on_spiderweb_grid
### Spiders web derived from TRACK file: gonu.trk
### This file is created by Deltares
### All text on a line behind the first # is parsed as commentary
### Additional commments
NODATA_value     = -1001
n_cols           = 4
n_rows           = 4
spw_radius       = 600000.0
spw_merge_frac   = 0.75
spw_rad_unit     = m
### END OF HEADER
TIME             = 340000.00   minutes since 2005-01-01 00:00:00 +00:00
x_spw_eye        =  265.00
y_spw_eye        =   33.00
p_drop_spw_eye    = 7000.000
 5.000000  5.000000  5.000000  5.000000
10.000000 10.000000 10.000000 10.000000
15.000000 15.000000 15.000000 15.000000
20.000000 20.000000 20.000000 20.000000
   270.00      0.00     90.00    180.00
   270.00      0.00     90.00    180.00
   270.00      0.00     90.00    180.00
   270.00      0.00     90.00    180.00
  4000.00   4000.00   4000.00   4000.00
  3000.00   3000.00   3000.00   3000.00
  2000.00   2000.00   2000.00   2000.00
  1000.00   1000.00   1000.00   1000.00
TIME             = 380000.00   minutes since 2005-01-01 00:00:00 +00:00
x_spw_eye        =  275.00
y_spw_eye        =   18.00
p_drop_spw_eye    = 8000.000
 5.000000  5.000000  5.000000  5.000000
10.000000 10.000000 10.000000 10.000000
15.000000 15.000000 15.000000 15.000000
20.000000 20.000000 20.000000 20.000000
   270.00      0.00     90.00    180.00
   270.00      0.00     90.00    180.00
   270.00      0.00     90.00    180.00
   270.00      0.00     90.00    180.00
  4000.00   4000.00   4000.00   4000.00
  3000.00   3000.00   3000.00   3000.00
  2000.00   2000.00   2000.00   2000.00
  1000.00   1000.00   1000.00   1000.00
```

Wind on a spiderweb grid has been provided a filetype specification as `FILETYPE=5`. The connection with the flow model itself is laid through the external forcings file. The actual specification of the wind in the new <*.ext> file:

DRAFT

```
[Meteo]
quantity            = airpressure_windx_windy
forcingFile         = spwsimple.spw
forcingFileType     = spiderWeb
interpolationMethod = linearSpaceTime
operand             = override
```

Warning:

◇ Deprecated: The old external forcing file is being deprecated. The examples below are kept only for migration purposes:

```
QUANTITY =airpressure_windx_windy
FILENAME =spwsimple.spw
FILETYPE =5
METHOD   =1
OPERAND  =O
```

Notice that `METHOD=1` is chosen for wind on a spiderweb grid, instead of `METHOD=2` in case of wind on an equidistant grid and `METHOD=3` in case of wind on a curvilinear grid.

Alternatively, the spiderweb wind and pressure can be specified in the external forcings file as separate quantities referring to the same file (or specify one and omit the other):

```
QUANTITY =windxy
FILENAME =spwsimple.spw
FILETYPE =5
METHOD   =1
OPERAND  =O

QUANTITY =airpressure
FILENAME =spwsimple.spw
FILETYPE =5
METHOD   =1
OPERAND  =O
```

(NB. There is a slight difference between both specifications regarding their effect on the flow. The first approach requires an additional interpolation of wind on the velocity points, which in some cases of coarse computational grids and small scale wind could introduce smoothing.)

14.2.7 Combination of several wind specifications

The combination of the various wind specification types can *only* be achieved if the `quantity` of the winds to be combined is the same, for instance `quantity=windx`. The option `operand=+` can be used to add a wind field to an existing wind field.

Example

If the uniform wind is to be combined with a wind specified on an equidistant grid, then the wind field could be assigned in the new external forcings file as follows:

```
[Meteo]
quantity            = windx
```

DRAFT

```
forcingFile          = windxdir.bc
forcingFileType      = bcAscii
interpolationMethod = linearSpaceTime
operand              = override

[Meteo]
quantity             = windy
forcingFile          = windydir.bc
forcingFileType      = bcAscii
interpolationMethod = linearSpaceTime
operand              = override

[Meteo]
quantity             = windx
forcingFile          = windxdir.amu
forcingFileType      = ArcInfo
interpolationMethod = linearSpaceTime
operand              = add

[Meteo]
quantity             = windy
forcingFile          = windydir.amv
forcingFileType      = ArcInfo
interpolationMethod = linearSpaceTime
operand              = add
```

Warning:

- ◇ Deprecated: The old external forcing file is being deprecated. The examples below are kept only for migration purposes:

```
QUANTITY =windx
FILENAME =windxdir.wnd
FILETYPE =1
METHOD   =1
OPERAND  =O

QUANTITY =windy
FILENAME =windydir.wnd
FILETYPE =1
METHOD   =1
OPERAND  =O

QUANTITY =windx
FILENAME =windxdir.amu
FILETYPE =4
METHOD   =2
OPERAND  =+

QUANTITY =windy
FILENAME =windydir.amv
FILETYPE =4
METHOD   =2
OPERAND  =+
```

The same is possible for e.g. a uniform wind and two spiderweb cyclones:

```
QUANTITY=windxy
FILENAME=uni.tim
FILETYPE=2
METHOD=1
OPERAND=O

QUANTITY =windxy
FILENAME =spwsimple.spw
FILETYPE =5
```

DRAFT

```
METHOD   =1
OPERAND  =+

QUANTITY =windxy
FILENAME =spwsimple2.spw
FILETYPE =5
METHOD   =1
OPERAND  =+

QUANTITY =atmosphericpressure
FILENAME =spwsimple.spw
FILETYPE =5
METHOD   =1
OPERAND  =+

QUANTITY =atmosphericpressure
FILENAME =spwsimple2.spw
FILETYPE =5
METHOD   =1
OPERAND  =+
```

In the above example, the wind is first prescribed as a spatially uniform time-varying field, onto which the spiderweb winds are added. Note that the uniform wind has the `'O'` operand, which means that it overrides rather than adds. However, since the default wind is zero, one might just as well have used the `'+'` operand. If the spiderweb merge fraction is specified in the spiderweb file, a gradual transition between the spiderweb wind and the background wind is applied using the linearly varying weight as described above. The same is done for atmospheric pressure, if specified except that pressure is added to, whereas wind is averaged with the background values. Without any input, the background atmospheric pressure is set to `PavBnd` in the section `[wind]` in the mdu-file, if present and zero otherwise.

14.3 Masking of points in the wind grid from interpolation ('land-sea mask')

Warning:

- ◇ Deprecated: The old external forcing file is being deprecated. Similar (but not equivalent) functionality is available via the `targetMaskFile` keyword. More details are listed in Section C.6.2.3.

A mask can be supplied by the user to prevent selected points in the wind grid from contributing to the wind interpolation on velocity points, e.g. to exclude land points. This feature was included to conform to SIMONA and therefore implemented in the same way.

For each individual grid point for which to interpolate from the wind grid:

- ◇ Masked wind points are excluded from the interpolation.
- ◇ The total of the weight factors for the remaining wind points is determined.
- ◇ If this total falls below `1E-03`, the mask is ignored and the original bilinear weights are used.
- ◇ Otherwise, the weights for the remaining wind points are normalised again.

The effect of the mask, when applied as a land-sea mask, is that for velocity points close to shore the interpolated wind is no longer influenced by the wind over land (which would otherwise yield a zone of points with reduced wind near the shore).

Specification and format of the mask file

The name of the mask file, if any, is specified in the <.ext> file, labelled `SOURCEMASK`, directly following the `FILENAME` specification, e.g.:

```
QUANTITY   =windxy
FILENAME   =meteo.wxwy
SOURCEMASK =meteo_mask.asc
FILETYPE   =6
METHOD     =3
OPERAND    =O
```

The mask file itself has the same layout as the wind file, though the number of required header fields is reduced, e.g.:

```
FileVersion      =    1.03
.
.

unit1            =    Pa
### END OF HEADER
1       1       1       1       1
1       1       1       0       0
1       1       1       0       0
1       1       0       0       0
```

The lines in the header are ignored. The number of columns and rows in the matrix of ones and zeros should match those of a block (for a single variable and a single timestep) in the meteo files. Zeros signify the position of rejected points (and ones those of the accepted points) in the wind grid.

DRAFT

15 Ice modelling in D-Flow FM

Note: Ice modelling in D-Flow FM is of TRL level 4-6 (prototype), because for example drying-flooding and restarting of ice and snow haven't been implemented yet in combination with ice modelling. Moreover, ice functionality in D-Flow FM is currently limited to a non-moving ice cover. The definitions of the Technical Technology Readiness Levels (TRL) are in Section 2.7.

The ice cover may either be externally specified or dynamically computed based on the thermodynamics of the system. Frazil ice is not yet included.

The presence of an ice cover may influence

- ◇ Wind shear, see section 15.3
- ◇ Thermal exchange between water and atmosphere, see section 15.2.2
- ◇ Precipitation and evaporation, see section 15.6
- ◇ Wave growth and propagation, see section 15.7
- ◇ Water pressure, see section 15.4
- ◇ Surface resistance, see section 15.5

15.1 Forced ice cover

In the case of a forced ice cover, the ice thickness $h_i(x, y, t)$ and the ice cover fraction $A_i(x, y, t)$ must be specified by the user via the initial and parameter fields file (Chapter D). (Formerly, this used to be in the old external forcings file.) A snow layer is not yet supported in the forced ice cover mode. These two quantities may be specified via

- ◇ ASCII equidistant grid
- ◇ ASCII curvilinear grid
- ◇ NetCDF curvilinear grid

The quantity names for h_i and A_i are `seaIceThickness` and `seaIceAreaFraction`, respectively.

15.2 Dynamic ice cover

The dynamic ice module consists of a thermodynamic model based on a single ice layer concept with snow on top (Semtner Jr., 1976). The ice model keeps track of three spatial quantities, namely the ice thickness $h_i(x, y, t)$, the snow thickness $h_s(x, y, t)$ and the ice cover fraction $A_i(x, y, t)$. Snowfall is prescribed via the precipitation input. In case of precipitation and air temperatures below zero, then this is considered as snowfall; see also section 15.6. Decay of the snow thickness is computed by the thermodynamic model.

15.2.1 Conservation laws

Although the current version does not yet include horizontal transport, it is foreseen that the functionality will be extended to include horizontal transport of the ice cover based on the elastic-viscous-plastic (EVP) sea-ice rheology of Hunke and Dukowicz (1997). Hence, the equations below include the ice velocity components $u_i(x, y, t)$ and $v_i(x, y, t)$. The equations for the conservation of the mass of ice and snow are given by

$$\frac{\partial(A_i h_i)}{\partial t} + \frac{\partial(u_i A_i h_i)}{\partial x} + \frac{\partial(v_i A_i h_i)}{\partial y} = S_{\text{ice}} + D_{\text{ice}} \tag{15.1}$$

$$\frac{\partial(A_i h_s)}{\partial t} + \frac{\partial(u_i A_i h_s)}{\partial x} + \frac{\partial(v_i A_i h_s)}{\partial y} = S_{\text{snow}} + D_{\text{snow}} + Q_{\text{snow}} \tag{15.2}$$

DRAFT

In these equations, the right-hand side contains thermodynamic terms S_* and diffusion terms D_*, with $*$ depending on the ice equation to be solved. The snowfall is represented by Q_{snow}. The equation for ice cover fraction A_i reads

$$\frac{\partial A_i}{\partial t} + \frac{\partial (u_i A_i)}{\partial x} + \frac{\partial (v_i A_i)}{\partial y} = S_A + D_A \tag{15.3}$$

with $0 \leq A_i \leq 1$. The ice cover fraction or concentration represents the fraction of a computational cell that is covered by ice. Due to horizontal transport of the ice cover (from ice floes to icebergs), it is possible that only a part of a computational cell is covered by ice (and snow). The ice velocities u_i and v_i are computed via the momentum equations

$$M\left\{\frac{\partial u_i}{\partial t} + \frac{\partial (u_i u_i)}{\partial x} + \frac{\partial (v_i u_i)}{\partial y} - f v_i\right\} = Mg\frac{\partial \zeta}{\partial x} + \tau_a^x + \tau_w^x + F^x \tag{15.4}$$

$$M\left\{\frac{\partial v_i}{\partial t} + \frac{\partial (u_i v_i)}{\partial x} + \frac{\partial (v_i v_i)}{\partial y} + f u_i\right\} = Mg\frac{\partial \zeta}{\partial y} + \tau_a^y + \tau_w^y + F^y \tag{15.5}$$

with ζ the water elevation, f the Coriolis force and M the ice mass, g the acceleration due to gravity, F the internal stresses according to Hibler III (1979) and τ_a the air and τ_w the water stresses, respectively. The internal ice stress F includes a compressive ice strength according to

$$P = p^* h_i e^{-C(1-A_i)} \tag{15.6}$$

with p^* and C empirical constants of 2.5 × 10^4 and 20, respectively.

However, without horizontal movement, the ice velocity components $u_i(x, y, t)$ and $v_i(x, y, t)$ are currently always zero and the ice fraction A_i is always equal to zero (no ice in a computational cell) or one (computational cell completely filled with ice). The Equations 15.1 and 15.2 simplify to

$$\frac{\partial h_i}{\partial t} = S_{\text{ice}} \tag{15.7}$$

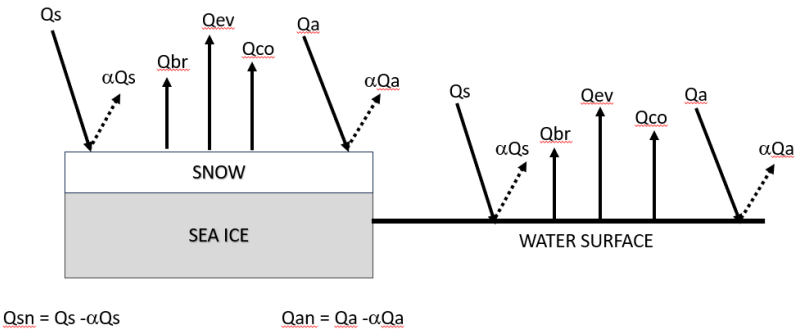
$$\frac{\partial h_s}{\partial t} = S_{\text{snow}} + Q_{\text{snow}} \tag{15.8}$$

DRAFT

15.2.2 Thermodynamics of the ice/snow cover

The vertical growth rate (or decay) of ice thickness is determined by the ice thermodynamics. The model's thermodynamic interactions between snow, ice, water, and atmosphere are shown in the left panel of Figure 15.1. The heat fluxes through the ice are based on a simple one layer ice model with snow on top, according to Semtner Jr. (1976). The right panel in Figure 15.1 represents the (standard) temperature model in D-Flow FM when ice is absent.

Figure 15.1: *Conceptual diagram of the ice-growth (left) and temperature (right) model*



The total heat flux through the free surface reads

$$Q_{\text{tot}} = Q_{\text{s,n}} + Q_{\text{a,n}} - Q_{\text{br}} - Q_{\text{ev}} - Q_{\text{co}} \tag{15.9}$$

with $Q_{\text{s,n}}$ the net incident solar radiation (short wave), $Q_{\text{a,n}}$ the net incident atmospheric radiation (long wave), Q_{br} the back radiation (long wave), Q_{ev} the evaporative heat flux (latent heat) and Q_{co} the convective heat flux (sensible heat). The subscript n refers to a net contribution. All heat flux terms of Equation (15.9) are described in more detail in (Deltares, 2026c, **??**). The net atmospheric radiation $Q_{\text{a,n}}$ and the back radiation Q_{br} can be combined into the effective back radiation Q_{ebr} according to

$$Q_{\text{ebr}} = Q_{\text{br}} - Q_{\text{a,n}} \tag{15.10}$$

such that

$$Q_{\text{tot}} = Q_{\text{s,n}} - Q_{\text{ebr}} - Q_{\text{ev}} - Q_{\text{co}} \tag{15.11}$$

and hence

$$Q_{\text{tot without ebr}} = Q_{\text{tot}} + Q_{\text{ebr}} = Q_{\text{s,n}} - Q_{\text{ev}} - Q_{\text{co}} \tag{15.12}$$

The effective back radiation Q_{ebr} for water typically reads

$$Q_{\text{ebr}} = \alpha T_{\text{s,w}}^4 \tag{15.13}$$

with $T_{\text{s,w}}$ the (absolute) water surface temperature in K and $\alpha = \varepsilon\sigma(0.39 - 0.05\sqrt{e_a})(1.0 - 0.6F_c^2)$ in J m^{-2} s^{-1} K^{-4}, in which e_a is the vapour pressure, F_c the cloudiness factor, ε the emissivity factor and σ the Stefan-Boltzman constant. In the numerical sea-ice model the temperature is assumed to vary linearly between the top and bottom of the ice layer. If an ice (and possibly snow) cover is present, the effective back radiation heat flux is no longer computed by Equation (15.13) but via

$$Q_{\text{ebr}} = \alpha T_X^4 \tag{15.14}$$

in which T_X either the (surface) ice temperature T_i or the snow temperature T_s depending on the presence of a snow layer (threshold thickness: 1 mm). The surface heat balance tells us that the net total heat flux Q_{tot} is equal to the conductive thermal flux Q_{c} through the ice. By assuming a constant thermal conductive coefficient k_i for ice, the heat conduction through the ice is given by

$$Q_{\text{c}} = -k_i \frac{T_X - T_f}{h_i} \tag{15.15}$$

which combined with the surface heat balance and Equation (15.12) gives

$$Q_{\text{tot}} = Q_{\text{tot without ebr}} - Q_{\text{ebr}} = -k_i \frac{T_X - T_f}{h_i} \tag{15.16}$$

in which T_f is the freezing temperature of (sea) water. T_f is the temperature of the ice surface at the ice-water interface. When there is a snow layer on top of the ice cover, the ice thickness h_i in Equation (15.15) should be replaced by the effective ice thickness $h_i^* = h_i + h_s k_i / k_s$, with h_s and k_s the thickness and the thermal conductive coefficient for the snow layer. A value of 2.04 W m^{-1} K^{-1} is applied for k_i and 0.31 W m^{-1} K^{-1} for k_s.

By expanding the new (surface) ice/snow temperature T_X as $T_p + \Delta T$, with T_p the previous value of T_X and linearizing Equation (15.13) in ΔT via $T_X^4 = T_p^4 + 4T_p^3 \Delta T$, the temperature T_X can be determined by iteration, via

$$Q_{\text{tot without ebr}} - \alpha T_p^4 - 4\alpha T_p^3 \Delta T = \frac{k_i}{h_i^*}(T_p - T_f) + \frac{k_i}{h_i^*}\Delta T \tag{15.17}$$

Next, Equation (15.17) can be rewritten to

$$Q_{\text{tot without ebr}} - \alpha T_p^4 - \frac{k_i}{h_i^*}(T_p - T_f) = \Delta T \left(4\alpha T_p^3 + \frac{k_i}{h_i^*} \right) \tag{15.18}$$

resulting in an air temperature correction ΔT per iteration as follows

$$\Delta T = \frac{Q_{\text{tot without ebr}} - \alpha T_p^4 + \frac{k_i}{h_i^*}(T_p - T_f)}{4\alpha T_p^3 + \frac{k_i}{h_i^*}} \tag{15.19}$$

Note that the above-described iterative procedure is often applied in sea-ice models, e.g.: Wang *et al.* (2005), Mellor and Kantha (1989). In subroutine `preprocess_icecover.f90` this is coded with the following variables (units in brackets)

$$\texttt{coef1} = \alpha T_X^4 \text{ [J m}^{-2}\text{ s}^{-1}\text{]} \tag{15.20}$$

$$\texttt{coef2} = 4\alpha T_X^3 \text{ [J m}^{-2}\text{ s}^{-1}\text{ K}^{-1}\text{]} \tag{15.21}$$

$$\texttt{Qlong_ice} = \alpha \text{ [J m}^{-2}\text{ s}^{-1}\text{ K}^{-4}\text{]} \tag{15.22}$$

$$\texttt{effective_ice_thickness} = h_i^* \text{ [m]} \tag{15.23}$$

$$\texttt{qh_air2ice} = Q_{\text{tot without ebr}} \text{ [J m}^{-2}\text{ s}^{-1}\text{]} \tag{15.24}$$

$$\texttt{Qlong} = Q_{\text{ebr}} \text{ [J m}^{-2}\text{ s}^{-1}\text{]} \tag{15.25}$$

After convergence the back radiation Q_{br} is computed via

$$Q_{\text{ebr}} = \texttt{coef1} + \texttt{coef2}\,\Delta T \tag{15.26}$$

Since $Q_{\text{tot without ebr}}$ is known in advance of the iteration process, the total heat flux Q_{tot} between the air and the ice can now be computed; see Equation 15.16. In practice, we observed

DRAFT

that only two or three iterations for are typically required for convergence. In the code maximally five iterations are applied.

So far the heat flux between the air and the ice or snow layer has been described in this section. Now we will focus on the computation of the heat flux beneath the ice. In grid cells in which ice is present, the heat flux between the ice and the water reads

$$Q_{\text{ice-water}} = -C_{Tz}(T_f - T_s) \tag{15.27}$$

where T_s is the water temperature at the uppermost grid layer near the surface. The heat transfer coefficient C_{Tz} is given by

$$C_{Tz} = \frac{u_*}{B_T + P_{rt}\ln(-z/z_0)/\kappa} \text{ with } B_T = 3P_{rt}\sqrt{\frac{z_0 u_*}{\nu}}P_t^{2/3} \tag{15.28}$$

with u_* the friction velocity, P_{rt} a turbulent Prandtl number, z is the vertical coordinate corresponding to the temperature T, z_0 the roughness length and κ the Von Karman constant. The molecular sublayer correction is represented by B_T, P_t is the molecular Prandtl number and ν is the kinematic viscosity of water.

The freezing temperature of seawater T_f depends on the salinity concentration (Fofonoff and Millard Jr., 1983) of the upper water layer has been taken from and reads in ℃

$$T_f = (-0.0575 + 0.001710523\sqrt{S} - 0.0002154996S)S \tag{15.29}$$

with S the salinity concentration in ppt. This formula is also used in NEMO (Madec *et al.*, 2022). Noted that for fresh water (S = 0 ppt) the freezing point equals 0 ℃.

15.3 Ice effect on wind drag

The wind drag assuming open water $C_{d,w}$ is computed as described in section 14.1.2. Subsequently, we have implemented 6 options to compute the effective wind drag including the ice cover. These options can be specified by setting the keyword `ModifyWindDrag` in the `[ice]` block of the mdu-file to the indicated string.

- ◇ `none`: In this case there is no effect of the ice cover.

$$C_{d,\text{eff}} = C_{d,w} \tag{15.30}$$

- ◇ `linear`: In this case the drag is assumed to be proportional to the water area fraction A_w.

$$C_{d,\text{eff}} = C_{d,w}A_w \tag{15.31}$$

- ◇ `cubic`: This option is consistent with the default "IceCube" option of ADCIRC (Chapman and Massey).

$$C_{d,\text{eff}} = \max(C_{d,w}, c_0 + c_1A_i + c_2A_i^2 + c_3A_i^3) \tag{15.32}$$

with c_0 = 0.000 75, c_1 = 0.0075, c_2 = –0.009, and c_3 = 0.002.

- ◇ `lupkes_birnbaum`: This option follows equation 22 of Lüpkes and Birnbaum (2005)

$$C_{d,\text{eff}} = C_{d,w}A_w + C_{d,i}A_i + C_{d,f} \text{ where} \tag{15.33}$$

$$C_{d,f} = 0.34A_i^2\frac{A_w\left(A_w^{0.8} + \frac{1}{2}(1 - \frac{1}{2}A_i)^2\right)}{a_r + 90A_iA_w} \tag{15.34}$$

where $a_r = 31$ and $C_{d,i}$ is the ice skin drag equal to 0.0015.

DRAFT

◇ `andreas`: This option follows Andreas *et al.* (2010)

$$C_{d,\text{eff}} = c_0 + c_1 A_i A_w \tag{15.35}$$

with c_0 = 0.0015, and c_1 = 0.002 233.

◇ `raysice`: This option is consistent with the "RaysIce" option of ADCIRC (Chapman et al).

$$C_{d,\text{eff}} = \max(C_{d,w}, c_0 + c_1 A_i A_w) \tag{15.36}$$

with c_0 = 0.001 25, and c_1 = 0.005.

◇ `joyce`:
This option follows equations 6 to 7 of Joyce *et al.* (2019)

$$C_{d,\text{eff}} = C_{d,w} A_w + C_{d,i} A_i + 4 C_{d,f,\text{max}} A_i A_w \tag{15.37}$$

with $C_{d,i}$ and $C_{d,f,\text{max}}$ given by the user keywords `iceSkinDrag` and `maximumIceFormDrag`, respectively.

15.4 Ice effect on pressure

The ice cover exerts a pressure on the water column in addition to the atmospheric pressure, such that the total pressure P at the surface becomes

$$P = P_a + A_i \rho_i g h_i \tag{15.38}$$

where P_a is the atmospheric pressure [Pa], A_i is the fraction of the area covered by ice [-], ρ_i is the density of the ice [kg m^{-3}], g is the gravitational acceleration [m s^{-2}], and h_i is the thickness of the ice layer [m]. This effect can be switched off by setting `ApplyPressure` to false.

15.5 Ice effect on flow resistance

The vectorial boundary condition for the momentum equations 3.5 and 3.6 at the water-ice interface ($z = \zeta$) is:

$$\left.\frac{\nu_V}{H}\frac{\partial \boldsymbol{u}}{\partial \sigma}\right|_{\sigma=1} = \frac{1}{\rho_0}\boldsymbol{\tau}_{s,\text{ice}} \tag{15.39}$$

where

$$\boldsymbol{\tau}_{s,\text{ice}} = \rho_0 A_i C_{d,\text{ice}} \left|\boldsymbol{U}_{\text{flow}} - \boldsymbol{U}_{\text{ice}}\right| \left(\boldsymbol{U}_{\text{flow}} - \boldsymbol{U}_{\text{ice}}\right) \tag{15.40}$$

where ρ_0 is the water density [kg m^{-3}], $C_{d,\text{ice}}$ is the drag coefficient [-] of the ice as given by the user via `IceFricValue`, and $\boldsymbol{U}_{\text{flow}}$ is the near-surface flow velocity [m s^{-1}] and $\boldsymbol{U}_{\text{ice}}$ is the drift velocity of the ice [m s^{-1}] (currently, always 0). This effect can be switched on by setting `ApplyFriction` to true.

15.6 Ice effect on surface exchange

The effect of ice on the surface heat exchange is included in the thermodynamic model and always switched on, see section 15.2.2. The effect on evaporation is to be documented. Snowfall is prescribed via the precipitation input. In case of precipitation and air temperatures below zero, then this is considered as snowfall. Decay of the snow thickness is computed by the thermodynamic model. Currently both rainfall and snowfall is applied is areas with ice in case of precipitation and air temperatures below zero. In future this will be improved. The switch `ReduceSurfExch` does not yet have an effect.

DRAFT

15.7 Ice effect on waves

This effect can be switched on by setting `ReduceWaves` to true. This effect is currently implemented as a reduction factor on the contribution of the wave forces to the momentum equation. Hence, it will not reduce the effect of waves at other places, such as wave streaming. It is foreseen that this implementation will be updated to more generally reduce the waves where an ice cover is present. Note, SWAN can also include the effect of an ice cover, so in coupled D-Flow FM-D-Waves simulation, it's better to include the effect of the ice cover also on the D-Waves side. Unfortunately, this option is only available for a forced ice cover, and not yet for a dynamic ice cover.

15.8 Ice input options

This section describes how to switch on ice modelling in D-Flow FM. The main mdu-input file of D-Flow FM contains several blocks of input, such as **geometry**, **numerics** and **output**. In case of ice modelling an extra block with ice input has to be added, **ice**, as illustrated below.

```
[ice]
iceCoverModel = Semtner          # Type of ice model (None, External, Semtner)
```

Currently, three options are available, namely:

- `None`: No ice cover included. This is the default setting.
- `External`: The ice cover thickness and ice area fraction are prescribed by the user via external forcings.
- `Semtner`: The ice thickness is dynamically computed by D-Flow FM, and varies in space and time based on the meteorological input (air temperature, wind, cloudiness, relative humidity, etc.). This option is based on the approach of Semtner Jr. (1976).

Examples for the latter two cases are given in the following sections. However, first we'll give an overview of all keywords that can be specified in the **ice** block.

Table 15.1: *Ice input keywords*

Keyword	**Record description**
ice	
`iceCoverModel`	specifies the ice cover model to be used
`applyPressure`	logical flag indicating whether the ice pressure term should be included (1 logical, default: `true`)
`applyFriction`	logical flag indicating whether the ice friction term should be included (1 logical, default: `false`)
`reduceSurfExch`	logical flag indicating whether surface exchange should be reduced (1 logical, default: `false`)
`reduceWaves`	logical flag indicating whether waves should be reduced (1 logical, default: `false`)

continued on next page

Table 15.1 — continued from previous page

Keyword	Record description
modifyWindDrag	effect of ice on wind drag (1 string) ◇ none (default) ◇ linear ◇ cubic ◇ lupkes_birnbaum ◇ andreas ◇ raysice ◇ joyce
iceAlbedo	albedo of the ice cover (1 float, default: 0.75)
iceDensity	density of the ice (1 float, default: 917 kg m^{-3})
iceSkinDrag	skin drag of ice floes (1 float, default: 1.5×10^{-3}) only for modifyWindDrag = joyce
maximumIceFormDrag	maximum form drag of ice floes (1 float, default: 2.5×10^{-3}) only for modifyWindDrag = joyce
snowAlbedo	albedo of the snow cover (1 float, default: 0.9)
iceFricType	type of friction specification at interface between ice and water (1 string) ◇ cdrag (default)
iceFricValue	drag coefficient at the ice-water interface (1 float, default: 0.005)
addIceToHis	Obsolete: use **output** wriHis_ice_default instead. logical flag indicating whether the ice quantities should be written to the his-file (1 logical, default: false)
addIceToMap	Obsolete: use **output** wriMap_ice_default instead. logical flag indicating whether the ice quantities should be written to the map-file (1 logical, default: false)

The output of ice quantities to map- and his-file can be switched on by means of the following keywords in the **output** block of the mdu-file.

Table 15.2: *Keywords for ice output*

Keyword	Record description
output	
wriHis_ice_default	flag specifying whether ice quantities should be written to the his-file (1 logical, default: false)
wriHis_ice_open_water_level	

continued on next page

DRAFT

Table 15.2 — continued from previous page

Keyword	Record description
	his-file output flag for the water level in absence of an ice cover, i.e. the water level in cracks and holes in the ice cover (1 logical, default: `wriHis_ice_default`)
`wriHis_ice_lower_surface_height`	his-file output flag for the bottom elevation of the ice cover, typically the elevation of the ice-water interface (1 logical, default: `wriHis_ice_default`)
`wriHis_ice_surface_height`	his-file output flag for the surface elevation of the ice cover, typically the elevation of the ice-air interface (1 logical, default: `wriHis_ice_default`)
`wriHis_ice_area_fraction`	his-file output flag for the fraction of the grid cell covered by ice (1 logical, default: `wriHis_ice_default`)
`wriHis_ice_thickness`	his-file output flag for the thickness of the ice cover (1 logical, default: `wriHis_ice_default`)
`wriHis_ice_pressure`	his-file output flag for the grid cell averaged pressure exerted by the ice cover (1 logical, default: `wriHis_ice_default`)
`wriHis_ice_temperature`	his-file output flag for the temperature of the ice cover (1 logical, default: `wriHis_ice_default`)
`wriHis_snow_thickness`	his-file output flag for the thickness of the snow layer on top of the ice cover (1 logical, default: `wriHis_ice_default`)
`wriHis_snow_temperature`	his-file output flag for the temperature of the snow cover (1 logical, default: `wriHis_ice_default`)
`wriMap_ice_default`	flag specifying whether ice quantities should be written to the map-file (1 logical, default: `false`)
`wriMap_ice_open_water_level`	map-file output flag for the water level in absence of an ice cover, i.e. the water level in cracks and holes in the ice cover (1 logical, default: `wriMap_ice_default`)
`wriMap_ice_lower_surface_height`	

continued on next page

Table 15.2 — continued from previous page

Keyword	Record description
	map-file output flag for the bottom elevation of the ice cover, typically the elevation of the ice-water interface (1 logical, default: `wriMap_ice_default`)
`wriMap_ice_surface_height`	map-file output flag for the surface elevation of the ice cover, typically the elevation of the ice-air interface (1 logical, default: `wriMap_ice_default`)
`wriMap_ice_area_fraction`	map-file output flag for the fraction of the grid cell covered by ice (1 logical, default: `wriMap_ice_default`)
`wriMap_ice_thickness`	map-file output flag for the thickness of the ice cover (1 logical, default: `wriMap_ice_default`)
`wriMap_ice_pressure`	map-file output flag for the grid cell averaged pressure exerted by the ice cover (1 logical, default: `wriMap_ice_default`)
`wriMap_ice_temperature`	map-file output flag for the temperature of the ice cover (1 logical, default: `wriMap_ice_default`)
`wriMap_snow_thickness`	map-file output flag for the thickness of the snow layer on top of the ice cover (1 logical, default: `wriMap_ice_default`)
`wriMap_snow_temperature`	map-file output flag for the temperature of the snow cover (1 logical, default: `wriMap_ice_default`)

Remark:

- ◇ For technical reasons, the `wriHis_ice_default` keyword does not yet work properly. For selecting his-file outputs, use the quantity specific keywords.

15.8.1 Ice modelling via prescribed external input files

In case of a prescribed ice thickness, the initial and parameter fields file (Chapter D) has to be extended with a reference to the ice model input.

```
[Parameter]
quantity            = seaIceThickness
dataFileName        = ice_forcing.HICE
dataFileType        = curviGrid
interpolationMethod = linearSpaceTime
operand             = override
```

DRAFT

Warning:

- ◇ Deprecated: The old external forcing file is being deprecated. The example below is kept only for migration purposes:

When using the old external forcings file (Section C.6) the ice-related forcing could for example be:

```
QUANTITY=sea_ice_thickness
FILENAME=ice_forcing.HICE
FILETYPE=6
METHOD=3
OPERAND=O
```

For both the initial and parameter fields file as well as the old external forcings file, the file ice_forcing.HICE contains a space- and time-varying ice thicknesses on a computational grid, similar to wind forcing as explained in Sections 12.2.2 and 12.2.3. The suffix of the filename is arbitrary, but we prefer a representative name like HICE (thickness of ice) or future options like AICE (area/fraction of grid cell covered by ice). An example is shown below. In this example a 5-by-5 grid and only two points in time are used (0 and 6 hours after T_start). The ice thickness in this example is 0.0, 1.0, 1.15 or 1.26 m. In this way, an external ice thickness can be prescribed. We remark that only an ice thickness can be prescribed and not a snow thickness. The latter can have a significant effect on the ice growth or thaw, but does not directly influence the water flow. Therefore, it is sufficient to only prescribe a fixed ice thickness.

```
### START OF HEADER
filetype          =    meteo_on_curvilinear_grid
NODATA_value      =    -999.000
grid_file         =    extdata.grd
n_quantity        =    1
### END OF HEADER
/* TIME (HRS)       0.0  20000101
     0.00     1.00     1.00     1.00     0.00
     0.00     1.15     1.15     1.00     0.00
     0.00     1.15     1.15     1.00     0.00
     0.00     1.00     1.00     1.00     0.00
     0.00     1.00     1.00     1.00     0.00
/* TIME (HRS)       6.0 20000101
     0.00     1.00     1.00     1.00     0.00
     0.00     1.26     1.26     1.00     1.00
     0.00     1.26     1.26     1.00     0.00
     0.00     1.00     1.00     1.00     0.00
     0.00     1.00     1.00     1.00     1.00
```

15.8.2 Ice modelling via `Semtner`

Via this option the ice thicknesses are computed by the computational engine of D-Flow FM, based on the model input prescribed for the Composite heat flux model in D-Flow FM (`temperature=5` in the mdu-file). See Chapter 11 for a more detailed explanation of this model. At present, ice modelling in D-Flow FM is only possible in combination with the Composite heat flux model, because this is currently the only heat flux model in D-Flow FM that computes temperatures that can be compared with measurements (i.e. so-called absolute heat flux model).

With this heat flux model, no additional input is needed to compute ice growth and thaw. The input is the same as used for regular thermodynamic modelling with the Composite heat flux model. This means that the required input for ice modelling is air temperature, relative humidity and cloudiness. Furthermore, wind forcing input is used in the computation of ice

thicknesses. Thus, just adding `IceCoverModel = Semtner` to the mdu-file is enough to include ice modelling in a simulation; see section 15.2.2. As the name reveals, the implemented ice module is based on Semtner Jr. (1976).

In D-Flow FM it is possible to compute temperature below zero. This is of use for ice modelling, because for example the freezing point of saline water with a salinity concentrations of 35 ppt is close to –2 ℃; see also Eq. (17). The following keyword `salinityDependentFreezingPoint = 1` enables negative temperature values in D-Flow FM. Then the water surface has the freezing point temperature, which is computed according to Equation 15.29. The user has to provide keyword `Tempmin = -2` (or another value). This is a global lower limiter for the water temperature values and its default value of 0 does not allow the negative values. Both of these keywords have to be added in the input block `[physics]`.

The description above is sufficient for the computation of ice thicknesses without any occurrence of snow. However, snow can have a significant impact on the ice thickness. Snow on top of ice acts like an 'insulating blanket' and can significantly reduce the growth of ice thickness. In D-Flow FM, for the computation of snow thickness, rainfall input and air temperature input are used. If there is any rainfall in a model, ideally specified with space- and time-varying input, and if the air temperature is below zero, D-Flow FM assumes that this represents snowfall. In the example below, the snowfall (rainfall at air temperatures below zero) is 10 cm in one day. Note that the below file is a time series file (*.tim), as described in section C.4.

```
    0.000000     0.0000000   !  rainfall or snowfall in mm/day
  72000.0000     0.0000000
  72001.0000   100.0000000
  73441.0000   100.0000000
  73450.0000     0.0000000
 217440.0000     0.0000000
```

To illustrate, Figure 15.2 shows ice thicknesses (in blue) and snowfall (in red) for a cold winter. In early December ice growth starts till a maximum thickness of 10 cm. This is followed by a period of thaw, that makes the ice almost completely disappear. From mid-December a second period of below-zero temperatures occurs, leading to a maximum ice thickness of about 25 cm with snow and 35 cm without any snow. This clearly illustrates that the ice growth is slowed down by a snow layer.

15.9 Postprocessing

The ice quantities available on the map and history files are listed in Table 15.3. Each quantity can be individually selected for output, see section 15.8.

Note that time series can be retrieved by selecting model output in an individual grid cell. This was done to generate Figure 15.2. In general, the map output frequency will be much lower than the history output frequency. However, time variation of ice and snow is much slower than that of other quantities (water levels, currents, salinity, etc.).

15.10 Limitations

Ice modelling in D-Flow FM currently has the following limitations:

- ◇ No horizontal transport of ice
- ◇ No open boundaries for ice
- ◇ No wetting and drying in combination with ice modelling

DRAFT

Figure 15.2: *Illustration of computed ice thickness in case of no snow (in blue), ice thickness in case of snow (in green) and snow thickness (in red)*

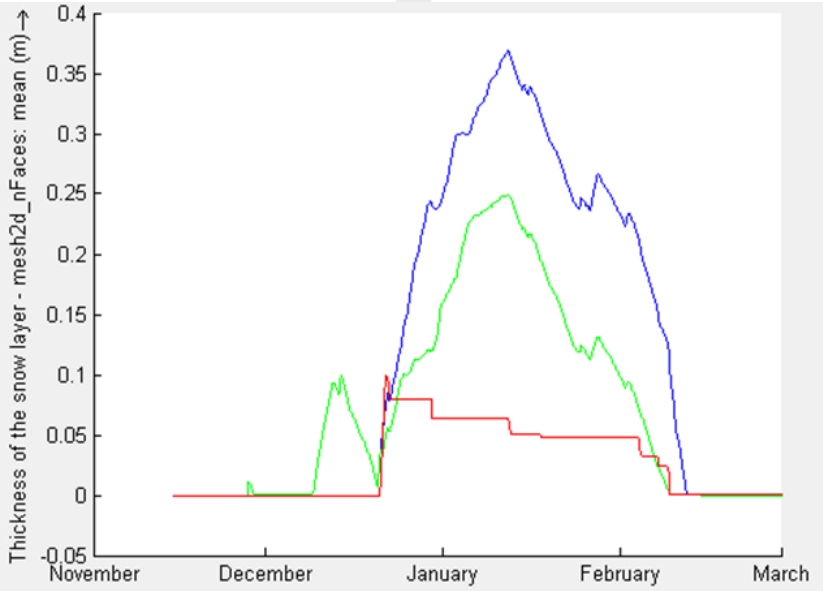


Table 15.3: *ice quantities on map- and his-files*

long name	name on file	units	dimensions
Sea surface height of open water	ice_s1	m	mesh2d_nFaces
Lower surface height of ice cover	ice_zmin	m	mesh2d_nFaces
Upper surface height of ice cover	ice_zmax	m	mesh2d_nFaces
Fraction of the surface area covered by floating ice	ice_area_fraction	-	mesh2d_nFaces
Thickness of floating ice cover	ice_thickness	m	mesh2d_nFaces
Pressure exerted by the floating ice cover	ice_pressure	$N\,m^{-2}$	mesh2d_nFaces
Temperature of floating ice cover	ice_temperature	K	mesh2d_nFaces
Thickness of the snow layer	snow_thickness	m	mesh2d_nFaces
Thickness of the snow layer	snow_temperature	kelvin	mesh2d_nFaces

16 Coupling to external models via Basic Model Interface (BMI)

This chapter is on the *coupling* of hydrodynamics to external models. When running a coupled model the DIMR program takes care of the exchange of data between components, and each component will be addressed using BMI calls. Section 16.1 contains the technical details of the D-Flow FM implementation of the BMI.

The other sections of this chapter go into more detail for a specific external model: coupling to D-RTC in section 16.6, coupling to D-RR in section 16.7, .

16.1 BMI interface to interact with the D-Flow FM model state

The D-Flow FM kernel shared library offers an API that implements the Basic Model Interface (BMI). The BMI allows one to initialize, run, and finalize a model component via standardized interface calls; furthermore it allows one to get and set various quantities such that model components can dynamically react to the computed results. General details on the BMI can be found in Peckham *et al.* (2013). This section discusses some of the D-Flow FM specifics.

Many variables in D-Flow FM's model state can be inquired and changed via BMI calls. Setting a variable can be done via a call to `set_var(varname, userdata)`, which copies the data from the `userdata` array to the D-Flow FM memory associated with the `varname` variable. Alternatively, one may use a call to `get_var(varname, xptr)`, which returns a pointer to the requested variable, and directly assign new values to the D-Flow FM memory from outside the running simulation. The latter approach may be faster because less data needs to be copied.

16.2 DIMR and the BMI

The DIMR program takes care of the exchange of data between components, and each component will be addressed using BMI calls. The data to be exchanged is defined in the <dimr.xml> configuration file, under the `<dimrConfig><coupler><item>` elements. More details on DIMR can be found in Deltares (2026h).

The variable names specified there need to be supported by the addressed component. For instance, for the coupling to D-RTC (see Section 16.6) typically scalar quantities need to be exchanged that are defined at specific locations (such as observation points or structures). Since BMI only identifies variables by a unique variable name without any location selection, D-Flow FM identifies both location and quantity by introducing *compound* variable names (not to be confused with a compound structure). A compound variable name takes the form:

```
varname = "groupname/locationid/fieldname"
```

For example the variable name `"weirs/Lith01/CrestLevel"` will select the crest level at the weir named "Lith01".

The `groupname` part of the variable name identifies the location or object type. Currently known group names are listed below. Unless noted otherwise, please check Section 16.3 for the list of quantities supported for those groups of locations.

- `pumps`
- `weirs`
- `orifices`
- `gates`

DRAFT

- ◇ generalstructures
- ◇ culverts
- ◇ dambreak
- ◇ sourcesinks
- ◇ observations (observation points, see Section 16.4)
- ◇ crosssections (observation cross sections, see Section 16.4)
- ◇ laterals (see Section 16.5)

The locationid part of the variable name is the unique identifier of the intended object. More details on the specific identifier used can be found in the subsequent sections on the various object groups. The fieldname identifies the quantity at the selected location. Which quantities are supported depends on the groupname.

16.3 BMI access to hydraulic structure data in D-Flow FM

To access quantities defined at structures use a compound variable name as explained in Section 16.2. The locationid part of the compound variable name is, for hydraulic structures, equal to the id key in the structure input file (Table C.13). Which quantities can be addressed via the BMI, depends on the structure type indicated by the groupname. Table 16.1 lists the supported field names for each hydraulic structure group.

Table 16.1: *Supported compound variable fields for hydraulic structures in D-Flow FM via BMI.*

Group name	Field name	Access	Remark
pumps			
	Capacity	read-write	Only for non-staged pumps. Prescribed capacity, not necessarily actual discharge.
weirs			
	CrestLevel	read-write	
orifices			
	crestLevel	read-write	
	gateLowerEdgeLevel	read-write	
gates			
	CrestLevel	read-write	
	GateHeight	read-write	
	GateLowerEdgeLevel	read-write	
	GateOpeningWidth	read-write	
generalstructures			
	CrestLevel	read-write	
	GateHeight	read-write	
	GateLowerEdgeLevel	read-write	
	GateOpeningWidth	read-write	
longculverts			
	ValveRelativeOpening	read-write	
culverts			

(continued on next page)

DRAFT

Group name	Field name	Access	Remark
			(continued from previous page)
	ValveOpeningHeight	read-write	
dambreak			Currently only read-only, not settable.
	DambreakS1up	read-only	
	DambreakS1dn	read-only	
	DambreakBreach_depth	read-only	
	DambreakBreach_width	read-only	
	DambreakInstantaneous_discharge	read-only	
	DambreakCumulative_discharge	read-only	
sourcesinks			
	Discharge	read-write	Prescribed discharge, not necessarily actual discharge.
	ChangeInSalinity	read-write	Prescribed change, not necessarily actual change.
	ChangeInTemperature	read-write	Prescribed change, not necessarily actual change.

16.4 BMI access to observations in D-Flow FM

To access quantities defined at observation points and observation cross sections use a compound variable name as explained in Section 16.2. The `locationid` for observation points should match the `name` specified in the relevant input file (See Section F.2.2). All observation point names must be unique when accessing the data via BMI. The same holds for the `locationid` for observation cross sections (See Section F.2.4).

Table 16.2 lists the supported field names for each observation group. All supported variables/fields are read-only, evidently, as observations are only 'observing' the flow. Typically, these variables are used as input to hydraulic triggers in an RTC-coupling.

Table 16.2: *Supported compound variable fields for observations in D-Flow FM via BMI.*

Group name	Field name	Access	Remark
observations			Observation points
	water_level	read-only	
	water_depth	read-only	
	velocity	read-only	
	discharge	read-only	
	salinity	read-only	Only when salinity transport is on.
	temperature	read-only	Only when temperature transport is on.
	<tracername>	read-only	Only when that named tracer is in the model.
crosssections			Observation cross sections
	discharge	read-only	Space-integrated along cross section.
	velocity	read-only	Space-averaged along cross section.
			(continued on next page)

Group name	Field name	Access	Remark
			(continued from previous page)
	water_level	read-only	Space-averaged along cross section.
	water_depth	read-only	Space-averaged along cross section.

Remarks:

- ◇ In 3D models, BMI access to 3D quantities is currently still very rudimentary. D-Flow FM's BMI API provides a pointer to the *first* layer. This means that for sigma models, the value at the bottom layer is returned. For z-layers this will only be the actual bottom layer value if that particular location has its bed level in layer number 1. For shallower locations where layer number 1 is inactive, only a dummy fill value is returned.
- ◇ When using `get_var()` a pointer is returned, which allows reading values in higher layers (using offset "layer index $\times$ 8 bytes"). This requires expert knowledge; it is not available in DIMR-based couplings.
- ◇ This holds for salinity, temperature, *<tracername>* and also velocity and discharge.

16.5 BMI access to forcings in D-Flow FM

To access quantities defined as local forcings (currently only lateral discharges) use a compound variable name as explained in Section 16.2. The `locationid` for lateral discharges should match the `id` specified for them in the external forcings input (See Section C.6.2.2). Table 16.3 lists the supported field names for each forcing group.

Table 16.3: *Supported compound variable fields for forcings in D-Flow FM via BMI.*

Group name	Field name	Access	Remark
laterals			
	water_discharge	read-write	Prescribed discharge, not necessarily actual discharge (when negative).
	water_level	read-only	Water level, averaged value over the nodes in the lateral.
	water_volume	read-only	Total water volume per layer, for each lateral.
	water_discharge_per_layer		
		read-write	Prescribed discharge, distributed over the layers.
	incoming/<constituent>	read-write	Concentration of a transported quantity in incoming direction, array with 1 value per layer.
	outgoing/<constituent>	read-only	Concentration of a transported quantity in outgoing direction, averaged value over the lateral, per layer.

Remark:

- ◇ <constituent> can be water_salinity, water_temperature or <tracername>.

16.6 Coupling with D-RTC (FBC-Tools)

This section is on the *coupling* of hydrodynamics and real-time control — more specific: feedback control — of hydraulic structures.

16.6.1 Introduction

A coupled flow-rtc model can be run either as an Integrated Model from within the D-Flow FM User Interface, or from the commandline using the DIMR program (Deltares Integrated Model Runner).

In case of a flow-rtc coupling

- ◇ the flow model will exchange for example *Water levels*
- ◇ subsequently, the rtc model will prescribe for example the *Crest level* of a controlled structure, based on the exchanged *water levels*
- ◇ in case of an *online* coupling this *Crest level* will influence the hydrodynamic simulation.

16.6.2 Getting started

16.6.2.1 User interface: the first steps

First, the user must add a *real-time control* model to the *flow-fm* model. The **Project** window will look like this.

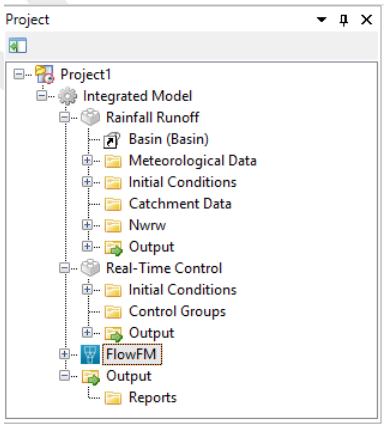


Figure 16.1: *An* Integrated model *in the* ***Project*** *window*

Secondly, the user will model the control flow for an hydraulic structure. This may look like this.

DRAFT

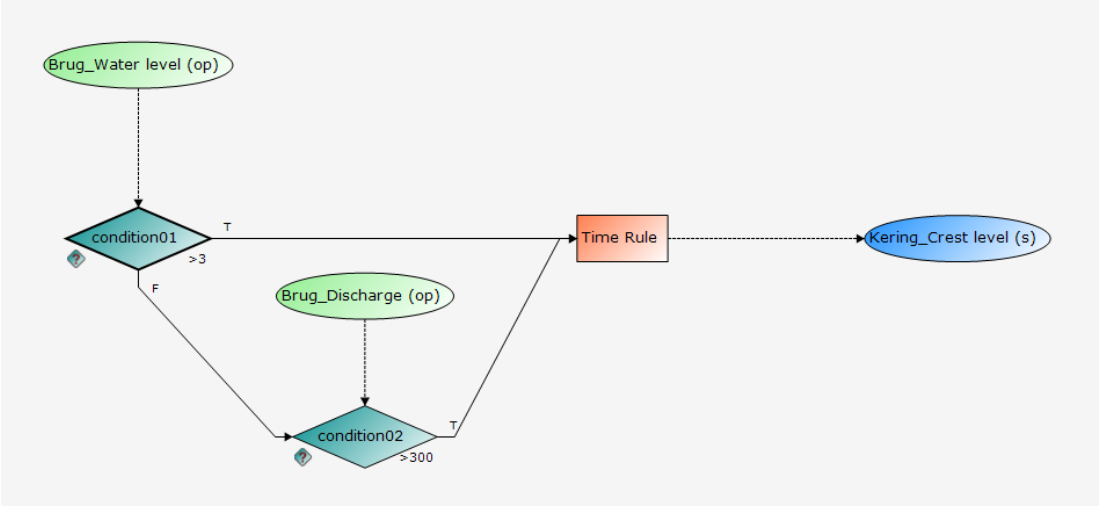


Figure 16.2: *Example of a* Control flow *in D-RTC*

Full documentation on D-RTC is available in its own User Manual. This section is limited to the details of running coupled flow-rtc models.

16.6.2.2 Input D-Flow FM

The hydraulic structures that are normally driven by scalar values or time series in <∗.bc> files, will now be fed by D-RTC. Replace the timeseries file name by the `realtime` keyword in the <structures.ini> file (section C.13):

```
[Structure]
type         = weir         # Type of structure
id           = weir01       # Name of the structure
locationFile = weir01.pli   # *.pli; Polyline geometry definition for 2D structure
crestLevel   = realtime     # Crest level in [m]
```

Also, the MDU file may optionally contain an observation file and/or a cross section file, such that D-RTC's triggers can be set to respond to the computed values at these locations.

```
[output]
ObsFile      = obs.xyn
CrsFile      = river_crs.pli
```

16.6.2.3 Input D-RTC

The input of D-RTC consists of several <∗.xml> files and a toplevel <settings.json>. We refer to the D-RTC User Manual (Deltares, 2026e) for further details.

16.7 Coupling with D-Rainfall Runoff

This section is on the *coupling* of hydrodynamics and lumped rainfall-runoff.

16.7.1 Introduction

The Delta Shell framework implements the concept of an *Integrated model* in order to couple different models, for example: hydrodynamics coupled with lumped rainfall-runoff. In this case, only a so-called *online* coupling is relevant.

Online

An *online* coupling exchanges data *every time* after computing a specified time interval. This tight coupling allows for direct feedback of the various processes on one another. This is crucial for D-RR processes that are dependent on surface water levels computed by D-Flow FM.

A coupled D-Flow FM–D-RR model can be run either as an Integrated Model from within Delta Shell, or from the commandline using the dimr program (Deltares Integrated Model Runner).

In case of a D-Flow FM–D-RR coupling

- ◇ the D-Flow FM model might exchange for example *Water levels*;
- ◇ subsequently, the D-RR model will prescribe typically the *discharge* of one or more lateral discharges. Depending on the type of runoff model selected, this can be based on the exchanged *water levels*.

16.7.2 Getting started

16.7.2.1 User interface: the first steps

The GUI presents a coupled model as an *Integrated Model*, an example is shown in Figure 16.3.

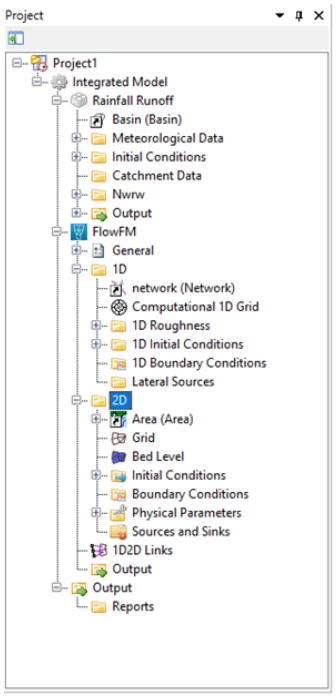


Figure 16.3: *Example of an* Integrated Model *with D-Flow FM and D-RR.*

The D-RR catchment nodes will yield runoff discharges that are to be coupled to D-Flow FM

(1D) lateral discharges. In the GUI, these couplings are called hydro links, as shown in Figure 16.4.

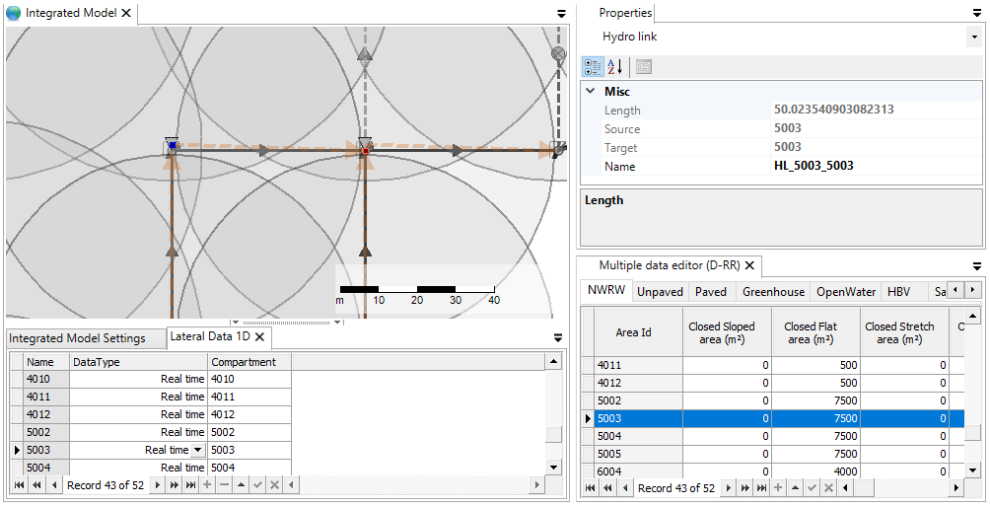


Figure 16.4: *Hydro links in the GUI: connection between D-RR catchments and D-Flow FM lateral discharges in 1D.*

Full documentation on D-RR is available in its own User Manual (D-RR_UM, 2019). The remainder of this section is limited to the details of running coupled D-Flow FM–D-RR models.

16.7.2.2 Input D-Flow FM

The lateral discharges that are normally driven by scalar values or time series in <∗.bc> files, will now be fed by D-RR. Replace the timeseries file name by the `realtime` keyword in the external forcings file (section C.6.2.2):

```
[Lateral]
id             = bnd_0230_007v
type           = discharge
nodeId         = NetworkNodeA
discharge      = realtime      # Discharge in [m3/s]
```

16.7.2.3 Input D-RR

The input of D-RR consists of the definition of catchments, and boundary conditions (and more). We refer to the D-RR User Manual (D-RR_UM, 2019) for further details.

16.7.2.4 Input DIMR

Both D-Flow FM and D-RR are used as dynamic libraries (DLL's on Windows, so's on Linux). DIMR is a small executable steering both dynamic libraries. Its input file, usually called "dimr_config.xml", looks like this:

```
<?xml version="1.0" encoding="iso-8859-1"?>
<dimrConfig xmlns="http://schemas.deltares.nl/dimrConfig"
            xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
            xsi:schemaLocation="http://schemas.deltares.nl/dimrConfig
            content.oss.deltares.nl/schemas/dimr-1.2.xsd">
   <documentation>
      <fileVersion>1.2</fileVersion>
      <createdBy>Deltares, Coupling team</createdBy>
```

```
      <creationDate>2018-08-28T10:06:09.3197094Z</creationDate>
    </documentation>
  <control>
    <parallel>
      <startGroup>
        <time>0 60 7200</time>
        <start name="myNameRR"/>
        <coupler name="rr2flow"/>
      </startGroup>
      <start name="myNameDFlowFM"/>
    </parallel>
  </control>
  <component name="myNameDFlowFM">
    <library>dflowfm</library>
    <workingDir>dflowfm</workingDir>
    <inputFile>FlowFM.mdu</inputFile>
  </component>
  <component name="myNameRR">
    <library>rr_dll</library>
    <workingDir>rr</workingDir>
    <inputFile>Sobek_3b.fnm</inputFile>
  </component>
  <coupler name="rr2flow">
    <sourceComponent>myNameRR</sourceComponent>
    <targetComponent>myNameDFlowFM</targetComponent>
    <item>
      <sourceName>catchments/0230.007v/water_discharge</sourceName>
      <targetName>laterals/bnd_0230_007v/water_discharge</targetName>
    </item>
    <item>
      <sourceName>catchments/WR-8475_02/water_discharge</sourceName>
      <targetName>laterals/bnd_WR8475_02/water_discharge</targetName>
    </item>
    <logger>
      <workingDir>.</workingDir>
      <outputFile>to_dflowfm.nc</outputFile>
    </logger>
  </coupler>
</dimrConfig>
```

16.7.3 Technical background

The technical background of the D-Flow FM–D-RR coupling is the same as for the D-Flow FM–D-RTC coupling, so for details refer to section 16.1. The only relevant BMI-exchangeable variable here is `laterals/bnd_0230_007v/water_discharge`, as listed in Table 16.3. Additionally, read-only access to `laterals/<lateralid>/water_level` is available.

DRAFT

17 Numerical aspects of D-Flow Flexible Mesh

This section contains numerical aspects of D-Flow FM.

17.1 Topological conventions for (2D) horizontal grids

In the horizontal a computational cell in a D-Flow FM grid (sometimes referred to as a 'network') consists of corner nodes and edges connecting the corner nodes. Such a grid cell should contain at least three and at most six corner nodes, representing triangles, quadrangles, pentagons and hexagons, respectively. The following topological conventions are used:

- **net nodes**: corners of a cell (triangles, quadrangles, ...),
- **net links**: edges of a cell, connecting netnodes,
- **flow nodes**: the cell circumcentre, in case of triangles the exact intersection of the three perpendicular bisectors and hence also the centre of the circumscribing circle,
- **flow links**: a line segment connecting two flownodes.

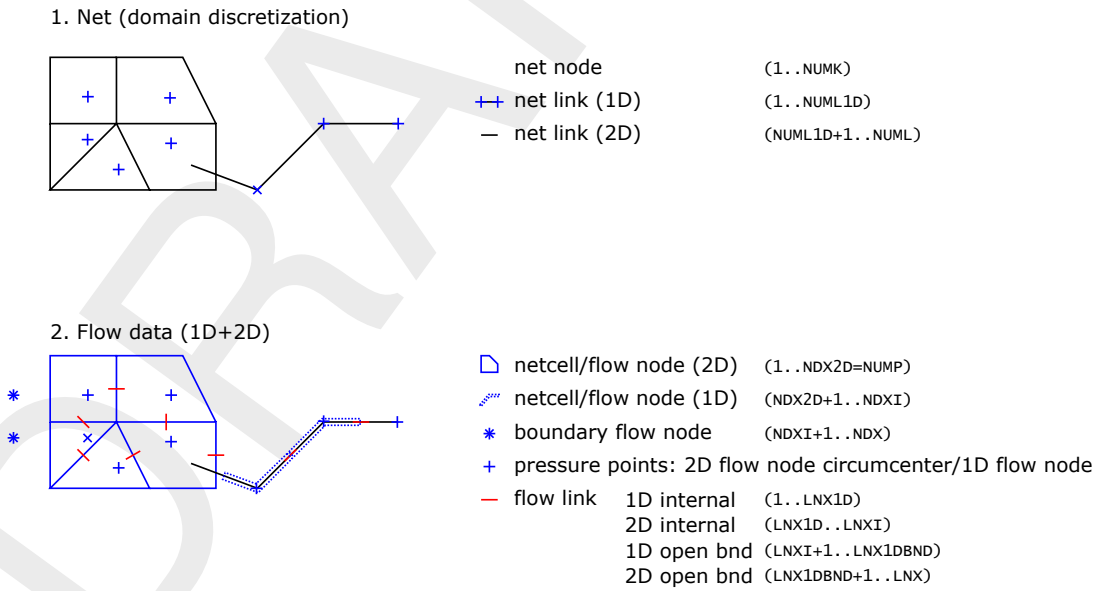


Figure 17.1: *Flexible mesh topology*

This mesh topology is illustrated in Figure 17.1. The 'center' of a cell can be defined in multiple ways. To illustrate this, two conventional cell center definitions for a triangle are highlighted in Figure 17.2. The two displayed cell centers have different properties:

1. the circumcenter is the location within the triangle which is the center point of a circle that intersects the triangle at each corner node of the triangle; as a consequence, the orthogonal projection of the center point to each face of the triangle divides each face into two exactly equidistant pieces,
2. the mass center (or centroid) is the center of gravity; as a consequence, a line through a corner node and the mass center divides a face into two exactly equidistant pieces under an angle not necessarily equal to 90°.

DRAFT

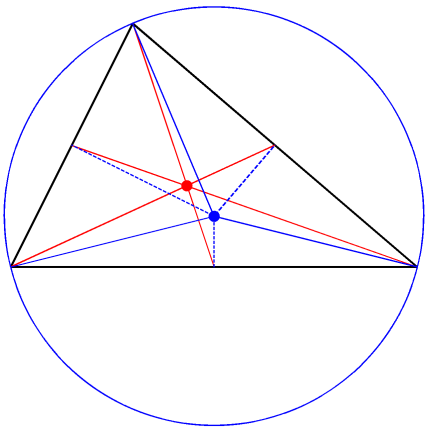


Figure 17.2: *Two conventional definitions of the cell center of a triangle: the circumcenter and the mass center.*

D-Flow FM utilizes the **circumcenter** as the basis of the definition of the elementary flow variables 'water level' and 'flow velocity'. The water level is defined at the circumcenter, whereas the face normal flow velocity is defined at the orthogonal projection of the circumcenter onto the cell face, i.e., the midpoint of the cell face.

Important properties of the mesh are the *orthogonality* and *smoothness*. The *orthogonality* is defined as the cosine of the angle φ between a flowlink and a netlink. Ideally 0, angle $\varphi = 90^\circ$. The *smoothness* of a mesh is defined as the ratio of the areas of two adjacent cells. Ideally 1, the areas of the cells are equal to each other. A nearly ideal setup is shown in Figure 17.3.

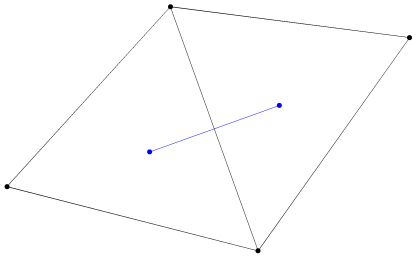


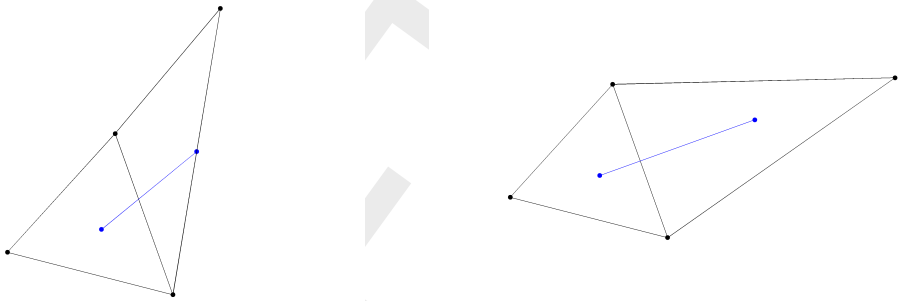
Figure 17.3: *Perfect orthogonality and nearly perfect smoothness along the edge connecting two triangles. Black lines/dots are network links/nodes, blue lines/dots are flow links/nodes.*

It is quite easy (and therefore dangerous) to generate meshes that violate the orthogonality and smoothness requirements. In Figure 17.4, two different setups of two gridcells are shown with different mesh properties.

The left picture of Figure 17.4 shows how orthogonality can be detoriated by skewing the right triangle with respect to the left triangle. While having the same area (perfect smoothness), the mutually oblique orientation results in poor orthogonality. In this particular case, the centre

of the circumscribing circle is in principle located outside the right triangle. Such a triangle is denoted as an 'open' triangle, which is bad for computations.

The opposite is shown in the right picture of Figure 17.4 in which the right triangle has strongly been elongated, disturbing the smoothness property. However, the orthogonality is nearly perfect. Nonetheless, both meshes need to be improved to assure accurate model results.



(a) *Perfect smoothness, but poor orthogonality.* ***(b)*** *Perfect orthogonality, but poor smoothness*

Figure 17.4: *Poor mesh properties due to violating either the smoothness or the orthogonality at the edge connecting two triangles. Black lines/dots are network links/nodes, blue lines/dots are flow links/nodes.*

17.2 Notation

The used symbols in this chapter are described in this section.

Time variables:

n	Time index of the current time step.
Δt_n	The computational time step size of time step n.

Computational grid and primary shallow water variables:

k	Computational grid index for a pressure point ("flow node"),
j	Computational grid index for a velocity point ("flow link"),
V_k	Volume of water in grid cell k,
A_k	Wet area of the horizontal water surface in grid cell k.
A_{u_j}	Wet cross-sectional area ("flow area") of flow link j.
A_{1j}/A_{2j}	Face-based cross-sectional area flow link j based on left or right side, respectively.

When a grid cell k is mentioned, this is essentially the same as a grid point (or calculation point) k. The term 'grid cell' better captures the concept of a cell-integrated value (such as volume), compared to a cell-averaged point value (such as water level).

Lateral discharges:

ℓ	Index of a lateral discharge,
$\mathcal{L}_{\text{lat},\ell}$	The set of grid cells selected for lateral discharge ℓ.
$Q_{\text{lat},\ell,k}$	The discharge of lateral ℓ for grid cell k,
$Q_{\text{lat},k}$	Total of all lateral discharges for grid cell k,
$Q_{\text{in},k}$	Total of all source terms for grid cell k,

DRAFT

17.3 Definition of bed level options

In section 17.1, the locations of the primary flow variables (water level and flow velocity) have been described. Through Figure 17.2 it has been clarified that the water level is computed at the location of the circumcenter (cell center), whereas the face normal flow velocity is computed at the midpoint of each cell face (face center).

For the computation of these two primary variables, the level of the bed must be known both at the cell center and at the face center. The user can specify the way these values are interpreted from the available bathymetry by means of the MDU-file keywords `BedlevType`, `DpuOpt` and `conveyance2D`. For a proper understanding of the possibilities of D-Flow FM, Figure 17.5 is provided with a three-dimensional representation of two adjacent triangular cells.

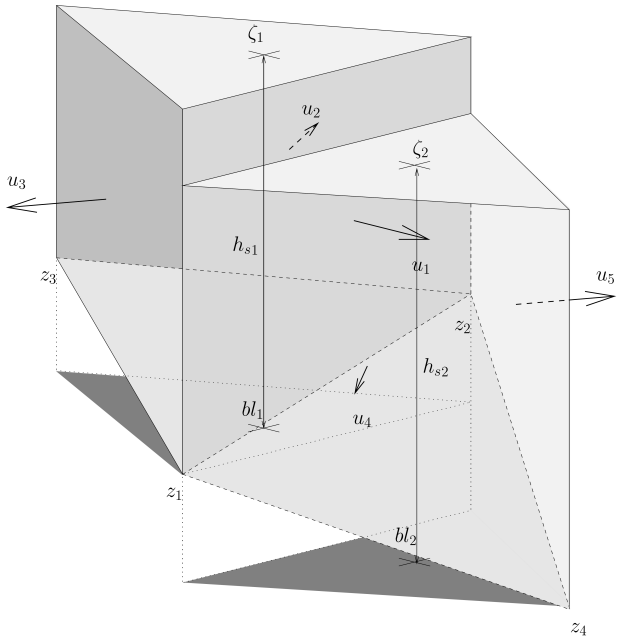


Figure 17.5: *Definition of the water levels, the bed levels and the velocities in case of two adjacent triangular cells.*

To the keyword `BedlevType`, the values `1`, `2`, `3`, `4`, `5` and `6` can be assigned. For the keyword `conveyance2D`, the values `-1`, `0`, `1`, `2` and `3` are available. The keyword `DpuOpt` can be either `1` or `2`; the latter option is currently only available in combination with D-Morphology.

The most common case is the definition of the bed levels at the corner nodes of the cell and a choice for the keyword `BedlevType = 3`. In cases where simulations with sediment transport and/or morphological updating are done (D-Morphology), only the combination of `BedlevType = 1` and `conveyance2D = -1` is allowed. Note that this requirement affects the calibration of hydrodynamic models. If a model is constructed with a potential later application of D-Morphology in mind, the hydrodynamic calibration should be performed with the combination of `BedlevType = 1` and `conveyance2D = -1`. Depending on the choice for `conveyance2D`, the face center bed levels and cell center bed levels are computed.

17.3.1 Piecewise constant approach for the bed level

In principle, the bed levels are considered as piecewise constant in space. This approach is followed if `conveyance2D` < 1. The keyword `BedlevType` can be used in combination with the piecewise constant approach as highlighted in the table below. Note that the bed level is defined with respect to the reference level (not to be confused with a water depth given as a positive value downwards). With `conveyance2D` < 1, we have:

Keyword	Value	Cell center bed level	Face center bed level
`BedlevType`	1	user specified	the highest[1] cell center bed level considering the two cells next to the face
`BedlevType`	2	the lowest face center bed level considering all the faces of the cell	user specified
`BedlevType`	3	the lowest face center bed level considering all the faces of the cell	the mean corner bed level considering the two corner nodes the face is connecting
`BedlevType`	4	the lowest face center bed level considering all the faces of the cell	the lowest corner bed level considering the two corner nodes the face is connecting
`BedlevType`	5	the lowest face center bed level considering all the faces of the cell	the highest corner bed level considering the two corner nodes the face is connecting
`BedlevType`	6	the mean corner bed level considering all the corners of the cell	the highest[1] cell center bed level considering the two cells next to the face

The former Delft3D-FLOW version, running on curvilinear meshes, has utilized the piecewise constant approach for bed levels as well. The treatment of the bed level itself is, however, different from D-Flow FM. In Delft3D-FLOW, the user can distinctly specify the treatment type for the cell center bed level and the face center bed level. In D-Flow FM, the choice for the one implies the choice for the other. A strict, *exact* match of settings for which Delft3D-FLOW and D-Flow FM treat the bed similarly, is not facilitated. Hence, the user himself should take care in comparing the Delft3D-FLOW results and D-Flow FM results when it comes to the bed level treatment settings.

[1] This is the default behaviour in case `DpuOpt`=1. When `DpuOpt`=2, the mean value of the two cells next to the face is used. This option is only available in combination with D-Morphology.

DRAFT

17.3.2 Piecewise linear approach for the bed levels

A piecewise linear bed level approach can be chosen for through setting `conveyance2D` ≥ 1. In this case, only the approach for `BedlevType` equal to `3`, `4` and `5` is affected. With `conveyance2D` ≥ 1, we have:

Keyword	Value	Cell center bed level	Face center bed level
`BedlevType`	`3`, `4`, `5`	the lowest corner node bed level considering all the nodes of the cell	linearly varying from the bed level at the one corner node to the bed level at the other corner node

Notice that the choice for either `BedlevType` equal to `3`, `4` or `5` does not imply different bed level treatment approaches. For the case `conveyance2D` ≥ 1, the bed is assumed linearly varying within a face only to compute the wet cross-sectional area of the vertical face; it should, however, be remarked that for the computation of the water column volume in a cell, this linear variation of the bed is *not* taken into account.

17.3.3 Hybrid bed level approach

In addition to the previously described bed level treatment approaches, D-Flow FM facilitates a hybrid approach for the computation of the cell center bed level by means of the keywords `blminabove` and `blmeanbelow`. In this approach, the cell center bed level is computed as the mean of the associated face center bed levels, be it only below the user specified level `blmeanbelow`. For levels above the user specified level `blminabove`, the minimum value of the associated face center bed levels is used. In between these two user specified levels, the bed levels are constructed by means of a linear interpolation between the two approaches' results.

17.3.4 Specification in the User Interface

The specification of the treatment of the bed level locations can be achieved through the tab fields 'Numerical Parameters' and 'Physical Parameters'.

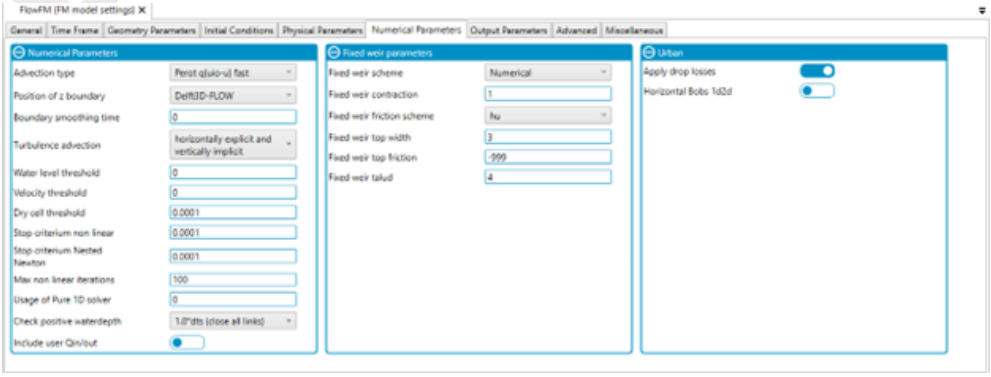


Figure 17.6: *Specification of the conveyance option in the User Interface.*

The tab field 'Numerical Parameters' contains the choice for the conveyance options: see Figure 17.6).

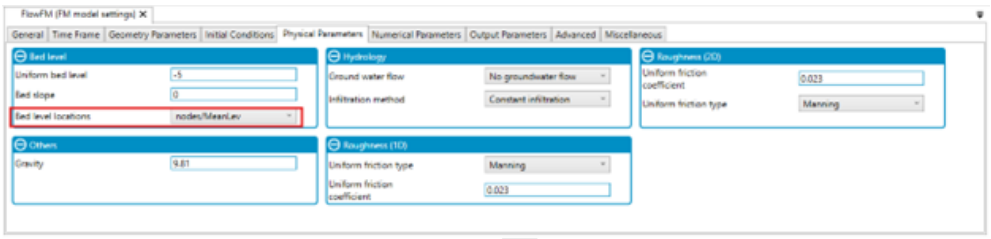


Figure 17.7: *Specification of the bed level treatment type in the User Interface.*

The tab field 'Physical Parameters' contains the choice for the Bed level locations. Five options are facilitated: `BedlevType` numbers 1 up to 5; the option 6 is not facilitated in the user interface.

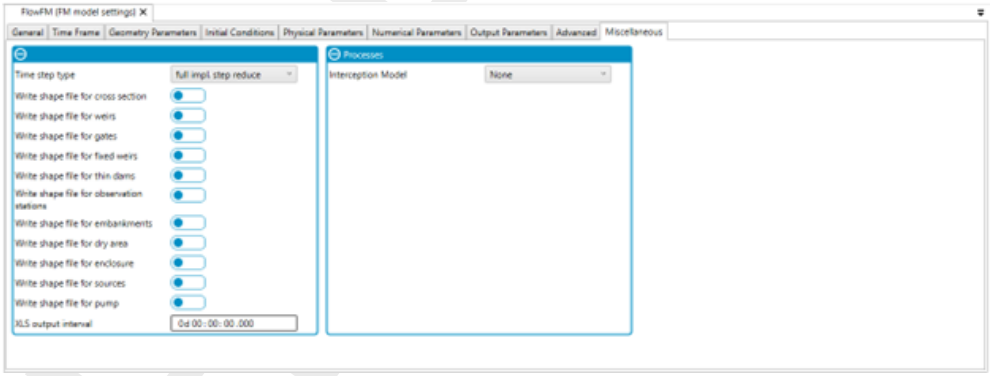


Figure 17.8: *Specification of the hybrid bed options (with keywords* `blminabove` *and* `blmeanbelow`*).*

By means of the tab field 'Miscellaneous', the hybrid options with the keywords `blminabove` and `blmeanbelow` can be enabled.

17.4 The location of open boundaries

The boundaries of a river, an estuary or a coastal sea are in general curved and are less smoothly represented on a structured grid. The boundary becomes irregular and may introduce significant discretization errors. To reduce these errors unstructured grids are used. The unstructured grids also allow local grid refinement in areas with large horizontal gradients.

The flow engine needs the location for the boundary conditions. In 2D models, this is done by providing a polyline with support points (also see section 6.4.7.1). By default, these support points are a means to construct a series of virtual cell centers along the boundary rim of the grid. Figure 17.9 provides an image of the concept of virtual boundary cells in 2D.

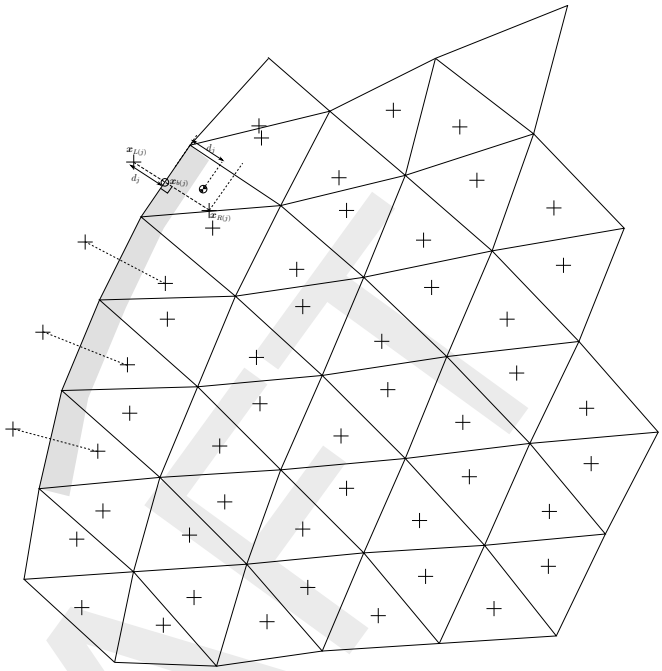


Figure 17.9: *Virtual boundary 'cells' near the shaded boundary; $\boldsymbol{x}_{Lj}$ is the virtual 'cell' pressure point near boundary edge j; $\boldsymbol{x}_{R(j)}$ is the inner cell's circumcenter; $\boldsymbol{x}_b(j)$ is the point on edge j that is nearest to the inner cell's circumcenter*

The support points are stored as one single polyline file per boundary condition, marking the rim along which the boundary conditions should hold. The polyline should be drawn in the vicinity of the rim. The user can specify the size of this 'vicinity' by means of the MDU-file keyword `OpenBoundaryTolerance`. The keyword specifies the search tolerance factor between the boundary polyline and the grid cells. The unit of this keyword is the cell size unit (i.e., not metres). By default, this value is 3, which loosely means that in the vicinity of $3\Delta x$ of the grid rim is searched of a boundary condition polyline.

In 1D, each boundary location can also be defined as a polyline in the vicinity of a network end point, but alternatively it can also be directly placed on that end point, by referring to its node id, using `nodeId` in the <.ext> file. A 1D boundary is obviously defined on a single point, and receives also one single virtual point. The precise details about the placement of this mirrored virtual boundary point follow in the text below.

The actual location of a specific boundary location point can be computed in two different ways, dependent on the user's choice for the keyword `izbndpos` in the mdu-file:

1 `izbndpos = 0`: construction of the boundary condition point by means of orthogonal mirroring of the closest cell center (default),
2 `izbndpos = 1`: construction of the boundary condition point as the orthogonal projection of the closest cell center onto the grid rim.

The following explanation uses both the mass center and the circumcenter, which are often different in 2D grid cells. For 1D gids, these two are identical.

Option `izbndpos = 0` 'mirrors' the internal cell closest to the boundary outward, to produce the virtual boundary point. The mirroring of the closest cell center is conducted as follows. First, the orthogonal distance from the cell's mass center to the actual rim of the grid is com-

puted: d_j in Figure 17.9. Second, the cell circumcenter (pressure point) $\boldsymbol{x}_{L(j)}$ is projected orthogonally onto the grid cell outer edge. The virtual boundary cell's pressure point ($\boldsymbol{x}_{R(j)}$) is defined at a distance $\frac{1}{2}d_j$ starting at that projected point $\boldsymbol{x}_{b(j)}$. Sometimes the flat area of the cell, say A , at the rim can give rise to an adaptation of this distance. If $\frac{1}{2}\sqrt{A} > d_j$, then the center of the virtual cell is located at the a distance $\frac{1}{2}\sqrt{A}$ away from the grid.

The first part in Figure 17.10 shows how in a regular grid the mass center and circumcenter coincide, with an exactly mirrored virtual boundary point. Also, the resulting placement of the pressure points in the 2D and 1D example grid is identical.

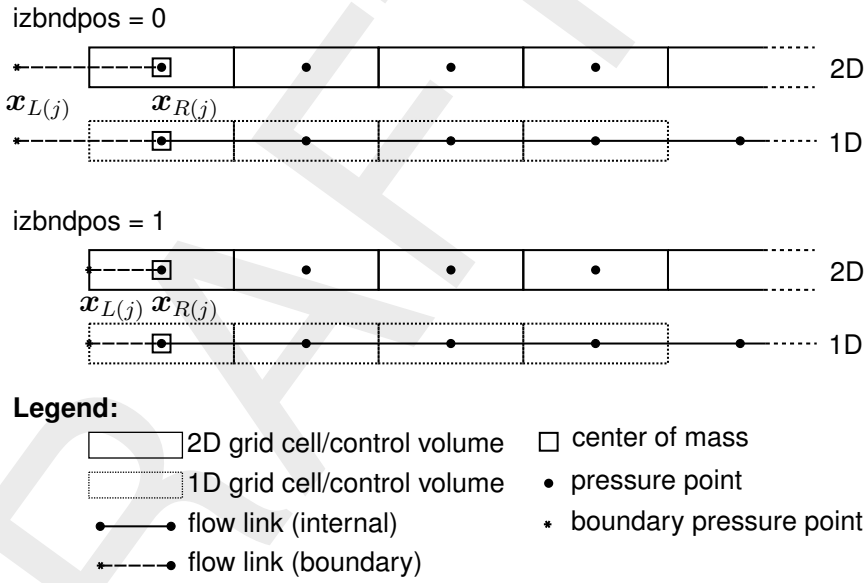


Figure 17.10: *Position of virtual boundary point, depending on* `izbndpos` *setting.*

Option `izbndpos = 1` is mainly intended to force a water level boundary exactly on the outer edge of a 2D grid cell. The second part in Figure 17.10 shows how the pressure point now lies on that 2D edge. In 1D networks, the virtual pressure point is placed *half* an edge length outward (instead of a full edge length for `izbndpos = 0`). This ensures that it is possible to create a 2D gridded channel and an exactly equivalent 1D channel branch, with the same pressure point locations.

17.4.1 Geometry of open boundaries

The virtual boundary points as introduced above inherit some additional geometric parameters from their neighbouring internal grid point:

- ◇ the bed level of the boundary point is set equal to the bed level of the internal pressure point: $bl_{L(j)} = bl_{R(j)}$;
- ◇ the boundary flow link width is different for 1D and 2D boundaries:
  - 2D: The boundary flow link width is equal to the 2D edge width: $w_{u_j} = w'_{u_j}$
  - 1D: The boundary flow link j inherits the entire cross section geometry of the first neighbouring internal flow link j' (that is: width, area, perimeter). If no such cross section is available on link j', then the default uniform width `UniformWidth1D` is used.

17.4.1.1 Conveyance in 2D

Bed friction often plays a major role in the discharge capacity and expected maximum water levels of channels and gullies. If we model a trapezoidal channel with sloping sidewalls on a grid with 6 grid cells across the width of the channel, a typical cross section using the standard bed representation with uniform depth appears per cell (Figure 17.11a). In D-Flow FM, we allow for representation of a locally sloping bed as shown in Figure 17.11b.

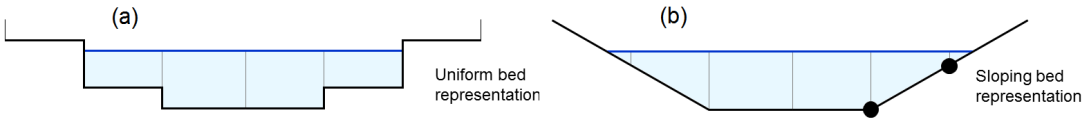


Figure 17.11: *Bed representation with uniform depth levels (a), and locally sloping bed (b).*

The most left and the most right cell are not yet wet in the uniform bed representation. In the sloping bed representation, these outer cells are partly wet, yielding a more accurate description of the total wet cross sectional area. The user can select whether to apply this more accurate description or not. This can only be done only in combination with a net-node based bathymetry description, i.e. using the keyword `BedlevType = 3` in the `.mdu` file.

For the bed friction in 2D models, one implicitly assumes a fully developed vertical velocity profile, using a logarithmic function of the water depth for the White-Colebrook bed friction formulation, or a one-sixth power function of the water depth for the Manning formulation. In a sloping cell, the local water depth is varying over the width of the cell. In the deeper part flow velocities will be higher than in the shallower part. Bed stresses will vary over the width of a cell with water depth and with the velocity. Integrating the stresses over the width of a cell one can derive the resulting total stress.

The bed friction term is not only a function of the normal velocity component in the direction of the flow link itself, but also depends on the tangential velocity component and with that on the total velocity. For each of the four components water depth, normal velocity, total velocity and Chézy parameter, we assume a linear variation over the width (Figure 17.12).

If we have in the `.mdu` file `Conveyance2D = 3`, the 2D analytic conveyance description using these four linearly varying components is applied. This option shows best grid convergence behaviour. Good grid convergence implies that the converged answer can be achieved on a coarser grid, thus saving computational costs.

If one sets `Conveyance2D = 2`, the tangential velocity component is assumed zero. This method is only applicable on curvilinear grids that are aligned with the flow direction.

If one sets `Conveyance2D = 1`, both the normal and tangential velocity component are assumed constant over the width. Effectively one obtains the so called 'lumped' bed friction approach, with hydraulic radius $R = A/P$, A being the wet cross sectional area and P the wet perimeter. This method works equally well as methods 2 and 1, provided that there is sufficient resolution of a gully in the grid. It is found that when a gully is resolved by more than about 10 or 12 cells, it provides almost identical answers as method 2, while saving some 10 % computational overhead compared to method 2.

Setting `Conveyance2D = -1`, the hydraulic radius is based on a uniform bed level at the velocity point taken as the average bed level of the two surrounding net nodes.

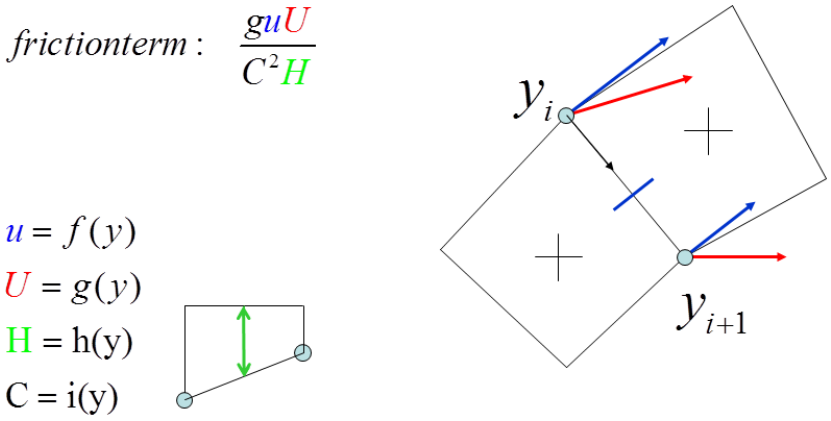


Figure 17.12: *A shematic view of the linear variation over the width for calculating the flow parameters.*

Setting `Conveyance2D = 0`, the hydraulic radius is based on a uniform bed level at the velocity point taken as the average bed level of the two surrounding water level points. This option is not advisable because cell bed levels are taken as the minimum value of the bed levels of attached link. So a min-max operator is invoked, which is not suitable for accuracy.

Keyword `Conveyance2D` deals with the bed level at the velocity points (i.e., at flow links or cell edges) in the case of a net-node based bathymetry description (i.e. `BedlevType = 3`). When using a cell-centre based bathymetry description (i.e. `BedlevType = 1`) `[geometry]` keyword `Dpuopt` can be used to this end. Using the default value `Dpuopt = 1`, the bed level at links is the maximum of the adjacent cell-centre bed levels (i.e. of the bed levels connected by a link). Setting `Dpuopt = 2` computes the bed level at links as the mean value between the adjacent cell-centre bed levels.

Please note that setting `Dpuopt = 2` should not be confused with a piece-wise linear interpretation of the bed level. The bed level for `BedlevType = 1` is considered piece-wise constant and that is still the way in which the volumes are computed. The keyword `Dpuopt` only changes the way in which the bed level at the interface between two tiles is computed.

The default maximum bed level at links causes the flow depth at links to be minimum, which guarantees that there is no flooding of a dry cell of which the bed level is higher than the water lower of the wet cell. On the contrary, using the mean bed level at links may cause a positive flow depth at the link while the bed level of the dry cell is higher than the water level of the wet cell.

Computing the bed level at links as the mean value of the adjacent cell-centres (i.e. `Dpuopt = 2`) is particularly relevant for morphodynamic simulations. This increases the accuracy of the results, as the default maximum option causes a first-order upwind/downwind reconstruction while the mean ensures a second-order central reconstruction.

DRAFT

17.5 Drying and flooding

Estuaries and coastal embayments contain large, shallow, and relatively flat areas separated and interleaved by deeper channels and creeks. When water levels are high, the entire area is covered by water. But as tide falls, the shallow areas are exposed, and ultimately the flow is confined only to the deeper channels. The dry tidal flats may occupy a substantial fraction of the total surface area. The accurate reproduction of covering or uncovering of the tidal flats is an important feature of numerical flow models based on the shallow water equations.

Many rivers have compound channels, consisting of a main channel that always carries flow (the summer bed) and one or two flood plains which only convey flow during extreme river discharges (the winter bed). The summer bed is surrounded by low dykes, which could be overtopped if the river discharge increases. The winter-bed is surrounded by much higher dykes, which are designed to protect the polders against water levels due extreme river discharges. The flooding of the flood plains increases the drainage capacity of the river and reduces the local water level gradients.

In a numerical model, the process of drying and flooding is represented by removing grid points from the flow domain that become *dry* when the tide falls and by adding grid points that become *wet* when the tide rises. Drying and flooding is constrained to follow the sides of grid cells. In this section, we specify the algorithms which have been used to determine the moment when a grid cell (water level point) or cell boundary (velocity point) becomes dry or wet. Drying and flooding gives a discontinuous movement of the closed boundaries and may generate small oscillations in water levels and velocities. The oscillations introduced by the drying and flooding algorithm are small if the grid sizes are small and the bottom has smooth gradients.

Essential elements of the wetting and drying algorithm are the definition of the water level, the definition of the bed level and the criteria for setting a velocity and/or water level point wet or dry. In the following subsections, these three items will be discussed.

17.5.1 Dry/wet criterion at cell faces

The parameter ε_{hu} (c.f. `Epshu` Table A.1) describes the threshold depth at cell faces. When the water depth at cell faces exceeds this ($h_u > \varepsilon_{hu}$) value the cell is considered wet and otherwise it is dry.

17.5.2 Dry/wet criterion at cell circumcenters

A research keyword `TestDryingFlooding` (Table A.1) is available for determining if the water depth is positive at cell circumcenters. To compute this, a variable d_{trsh} is introduced and when the water depth exceeds this value a cell is considered to be wet ($h > d_{trsh}$).

The default value is `TestDryingFlooding = 0` (D-Flow FM), and considers the threshold for positivity as $d_{trsh} = 0$. Option 1 is the Delft3D 4 default:

$$d_{trsh} = \max(10^{-9}, \min(\varepsilon_{hu}, 10^{-3})).$$

The last option 2 sets ($d_{trsh} = 0$) and furthermore considers a minimum volume for the transport solver based on the product of ε_{hu} and the flow cell area. The last setting is introduced to improve the stability of the transport computations, and can be activated if the default option leads to instabilities in the constituents.

DRAFT

17.6 Advection at open boundaries

In hydrodynamic models, it is common practice to apply high-resolution grids in domains of limited extent, for instance to investigate the influence of a dam design on local flow patterns in a coastal area. Then, the challenge is to prescribe appropriate boundary conditions for such a detailed model. Tidal motion is often important and therefore water levels have to be prescribed. If an overall model with good reproduction of water levels and velocities is available, then it is common practice to start with water levels from the overall model on the open alongshore boundary and on both cross-shore boundaries of the detail model. Then, often the reproduction of water levels is accurate, but the reproduction of flow velocities might be less accurate. In particular, the residual flow velocities might be inaccurate. Obtaining a good velocity reproduction by prescribing only water levels often only works for the larger models (e.g. > 30 km) so that bed friction, pressure gradient and acceleration can balance each other. In smaller models, however, this might not be the case. Even small water level differences on the open boundaries may introduce unrealistic high or low velocities. A remedy might be to apply velocity boundaries or other types of boundary conditions (Neumann-type, ...) on the cross-shore boundaries. However, this often reduces reproduction quality of water levels or might introduce strange flow patterns. Also the existence of stratification may also lead to unrealistic currents near open boundaries.

Therefore, in D-Flow FM the keyword `Zerozbndinflowadvection` can be activated in order to suppress unphysical currents near open boundaries. This applies for all water level open boundaries. This keyword has to be specified in group [numerics] and the options are `Zerozbndinflowadvection`=0,1 or 2. A value of 0 results into the default Neumann approach for currents, yielding that the gradient in velocity at the open boundary is zero. This approach is the default and applied when keyword `Zerozbndinflowadvection` is not specified. Applying a value of 1 forces cell center velocities to zero on the open boundary at *inflow* for all water level boundaries. A value of 2 forces cell center velocities to zero on the open boundary for both *inflow and outflow* for all water level boundaries.

Another option is to apply a so-called *advection* boundary, which can be specified via keyword `uxuyadvectionvelocitybnd` as part of the boundary conditions in the ext-file; see Section C.6.3. For this option, both the x- and y-components of the velocity vector at the boundary have to be specified in the boundary condition file, in bc format (`.bc`). To be more precise, one needs to prescribe the x- and y-components of the velocity vector in the circumcenter of the virtual boundary cells (outside the domain boundary). It is not necessary to rotate these components in the direction normal to some local boundary. They are from West to East and from South to North components.

17.7 1D2D coupling

The discretization of the model domain and the shallow water equations defined on that domain can be done in one, two and three dimensions. In a combined 1D2D model, some parts are modelled as 1D branches (e.g., channel flow or pipe flow) and other parts as 2D horizontal grids. The coupling between these 1D and 2D domain parts can use several 1D2D link types, depending on the application.

The following 1D2D link types can be distinguished:

1D2D lateral A sideways link between the 1D channel and 2D flood plains on the side(s), for example for rivers. 1D and 2D domain parts do not overlap. Further described in section 17.7.3.

1D2D embedded Very similar to the lateral link, but intended for smaller 1D channels, where the 2D grid cells overlap the 1D channels, for example in rural or marshland areas. Also described in section 17.7.3.

1D2D longitudinal The 1D channel is linked to the 2D grid part in the flow direction (i.e., not lateral), for example to model an upstream 1D river feeding into a 2D river or estuary. Further described in section 17.7.4.

1D2D vertically stacked A vertical link from 2D planes to 1D underground sewer networks. Further described in section 17.7.5.

17.7.1 1D2D model input

A coupled 1D2D model should contain the definition of the 1D2D links as "mesh contacts" in the network netCDF file. The details are described in section B.2.11.8.

All other model input that normally applies to regular flow links, can in principle also be applied to 1D2D links. For example, an observation cross section (section 6.4.2.3), or a fixed weir polyline (section 10.1) is allowed to cross 1D2D links as well.

17.7.2 Definitions

The following sections make intensive use of the mesh topology definitions in (Deltares, 2026a, section 6.1). Also of importance are the concepts of "bed level type" and "conveyance type", introduced in the same Technical Reference in Sections 6.1.2 and 6.2.2, respectively. The definitions below are specific for the 1D2D context.

Topology and connectivity

The following definitions are used for the connectivity at 1D2D links: which neighbouring cells (1D and 2D) are involved and which mesh nodes.

- ◇ 1D2D link j is a special type of flow link that connects a 1D mesh node with a 2D mesh cell,
- ◇ for longitudinal links, link j refers to two mesh nodes $l(j) = l(j')$ and $r(j) = r(j')$, where j' is the 2D vertical face (i.e., the cell edge) that is crossed by 1D2D link j,
- ◇ for all other types of 1D2D links, link j refers to two mesh nodes $l(j)$ and $r(j)$ that are simply the two end nodes of the corresponding mesh edge,
- ◇ 1D2D link j contains neighbors cells $L(j)$ and $R(j)$, corresponding to the start and end point, respectively. The side that connects to the 2D cell refers to a 2D mesh cell, and the other side to the 1D cell around the 1D mesh node.
- ◇ 1D2D link j contains neighbors cells $L(j)$ and $R(j)$, corresponding to the start and end point, respectively. The side that connects to the 2D cell refers to a 2D mesh cell, and the other side to the 1D cell around the 1D mesh node.
- ◇ For convenience, new notation is introduced to directly refer to the 1D cell and 2D cell of a 1D2D link: $C_{1D}(j)$ and $C_{2D}(j)$, respectively.

Bed levels

The effective bed levels depend on the type of 1D2D connection, therefore those details can be found in the following sections. Here, we only introduce the concept of *preliminary cell- or node-based bed levels* $bl_{*,l/r(j)}$, which are defined as:

$$bl_{*,l/r(j)} = \begin{cases} bl_{L/R(j)}, & \text{bed level type} \in \{1, 6\}, \\ z_{l/r(j)}, & \text{bed level type} = 3, \\ bl_{u_j}, & \text{otherwise.} \end{cases} \tag{17.1}$$

DRAFT

Storage volume

The storage volume in a 1D2D link is contributed half to the left and half to the right mesh cell, as follows:

$$\begin{aligned} V_{L(j)} &+= 0.5\Delta x_j A_{1j}, \\ V_{R(j)} &+= 0.5\Delta x_j A_{2j}, \end{aligned} \tag{17.2}$$

where Δx_j is the link length. This length depends on the type of 1D2D link, therefore those details can be found in the following sections. Next, the face-based cross-sectional areas $A_{1/2_j}$ are defined as the cross sectional area for the two face-based water depths, respectively:

$$\begin{aligned} h_{1j} &= \zeta_{L(j)} - bl_{1j}, \\ h_{2j} &= \zeta_{R(j)} - bl_{2j}. \end{aligned} \tag{17.3}$$

The area calculation for $A_{1/2_j}$ depends on the cross sectional shape, which differs for the various types of 1D2D links. More details can be found in the following sections.

Remark:

- ◇ The face-based cross-sectional areas $A_{1/2_j}$, which determine storage, differ slightly from the wet flow area A_{u_j}, which determines conveyance. The first is based on the water depth from Equation (17.3), whereas the latter is based on water depth h_{u_j}, which is the water depth based on an upwind reconstruction (Deltares, 2026a, Equation (6.14)).

17.7.3 1D2D internal connections (lateral and embedded)

The 1D2D lateral and embedded connections serve two purposes, but are treated numerically in one and the same way: via so-called 1D2D *internal* links. Both can be used to model 'sideways' exchange of water between a 1D channel and neighbouring 2D grid cells. The difference between lateral and embedded connections is that for lateral connections, the 2D grid cells are supposed to *not* overlap with the 1D computational cells, whereas for embedded connections the 2D grid cells overlap the 1D channels. This may result in double storage above the extent of the 1D computational cells, but this may sometimes be preferrable over fully aligning 2D model grids with relatively small 1D channels.

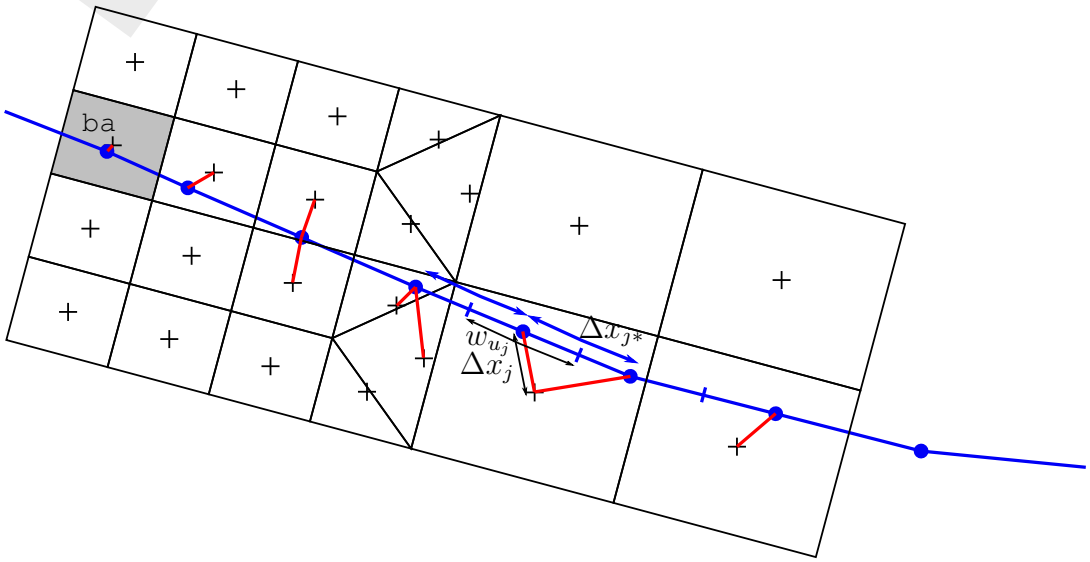


Figure 17.13: *Discretization of internal 1D2D links.*

Lengths and widths

The distance between the 1D and 2D pressure points for an internal connection j is defined as:

$$\Delta x_j = \max\left(\|\boldsymbol{x}_{R(j)} - \boldsymbol{x}_{L(j)}\|,\ 0.5\sqrt{\mathtt{ba}}\right) \tag{17.4}$$

where `ba` represents the area of the 2D grid cell connected to the 1D2D connection.

The width of a 1D2D internal flow link corresponds with either the "length" of the embankment on the 1D channel side or the width of the 2D cell. This length at the 1D channel side is approximated by the average of all regular 1D links connected to the 1D cell:

$$w_{u_{1D}} = \sum_{j^* \in \mathcal{J}_{1D}(j)} \Delta x_{j^*} / N_{1D}(j), \tag{17.5}$$

where the subset of regular 1D links neighbouring 1D2D link j is defined as:

$$\mathcal{J}_{1D}(j) = \{j^* \in \mathcal{J}(C_{1D}(j)) \,|\, j^* \text{ is a regular 1D link}\}, \tag{17.6}$$
$$N_{1D}(j) = |\mathcal{J}_{1D}(j)|. \tag{17.7}$$

In case the 1D2D link intersects the 2D cell, the width ($w_{u_{2D}}$)at the 2D side of the 1D2D link is set to the width of the intersected face of the 2D cell, otherwise $\sqrt{ba}$ is used. Finally w_{u_j} is set to the minimum of $w_{u_{1D}}$ and $w_{u_{2D}}$.

Figure 17.13 shows the most common situation of a straight 1D channel: 1D2D link j has two neighbouring regular 1D links, so the width of j is half of the two flow link lengths of them added together.

Bed levels

The 1D and 2D grid cells connected by the link each begin with their own bed level, as defined in the usual way via the `BedlevType` keyword (see section 17.3). The flow link's bed levels are defined solely by the bed level of the 2D cell of the link:

$$\begin{aligned} bl_{1_j} &= bl_{C_{2D}(j)} \\ bl_{2_j} &= bl_{C_{2D}(j)}. \end{aligned} \tag{17.8}$$

In the usual sense, the cell-based bed level on the left and right mesh cell are minimized by the flow link bed levels such that link bed levels are never below their connected cell's bed levels:

$$\begin{aligned} bl_{L(j)} &= \min(bl_{L(j)}, bl_{1j}), \\ bl_{R(j)} &= \min(bl_{R(j)}, bl_{2j}). \end{aligned} \tag{17.9}$$

Storage volume

The storage volume in a 1D2D internal link differs from the usual approach in Equation (17.2). Instead, the link volume is contributed half to the 1D cell and a quarter to the 2D mesh cell, as follows:

$$\begin{aligned} V_{C_{2D}(j)} &+= 0.5 \cdot 0.5 \Delta x_j A_{1j}, \\ V_{C_{1D}(j)} &+= 0.5 \Delta x_j A_{2j}, \end{aligned} \tag{17.10}$$

DRAFT

The quarter volume contribution is explained in Figure 17.14. On the 1D side a rectangular control volume of $\frac{1}{2}\Delta x_j$ is used.[2] When multiple 1D2D links are connected to the same 2D mesh cell, the volume contribution of these 1D2D links on the 2D side ($\Delta V_{c_{2D}(j)}$) might overlap in the corners. Therefore, half the storage of such rectangles (or in fact: the depicted triangular volumes) is added.

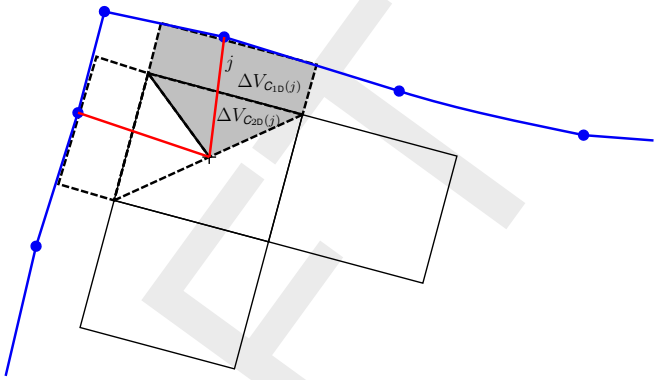


Figure 17.14: *Lateral 1D2D links and the volume contribution to the 1D and 2D cell.*

Face-based water depth h_{u_j}

The water depth on a 1D2D internal link j is defined as:

$$h_{u_j} = \zeta_{u_j} - \max(bl_{1_j}, bl_{2_j}) \tag{17.11}$$

Where the upwind water level ζ_{u_j} is defined according to (Deltares, 2026a, Equation (6.15)).

Wet cross-sectional area A_{u_j}

The wet cross-sectional area A_{u_j} is calculated as a rectangular cross section with width w_{u_j} and water depth h_{u_j}.

Remark:

- ◇ When MDU option `Conveyance2D` is switched on, the 2D bed slope is included in the area calculation. This is not yet documented here.

Advection treatment

For internal 1D2D connections advection scheme 8 is used, as described in Deltares (2026a). For 1D and 1D2D connections with type 8 advection scheme a slope limiter is used. This slope limiter is described in section 17.9.2.

In case fixed weirs are located on lateral 1D2D connections, it is also possible to use a weir formula to compute the flow over these connections. See section 10.1 for more information.

Conveyance

The conveyance on the 1D2D link is calculated as described in section 3.4.5.2. The Chézy roughness value is computed from a default Manning's roughness value of 0.035.

[2] Currently, the code has an additional factor $\frac{1}{2}$ also on the 2D side of the flow link, similar to the 1D side. This needs a closer look.

DRAFT

17.7.4 1D2D longitudinal connection

The 1D2D longitudinal connection type can be used to model 1D channel flow feeding into a downstream 2D domain part, or vice versa. The coupling is based on conveyance interpolation and the coupling is oriented perpendicular to the 2D grid cell edges, which should typically approximate the 1D channel flow direction.

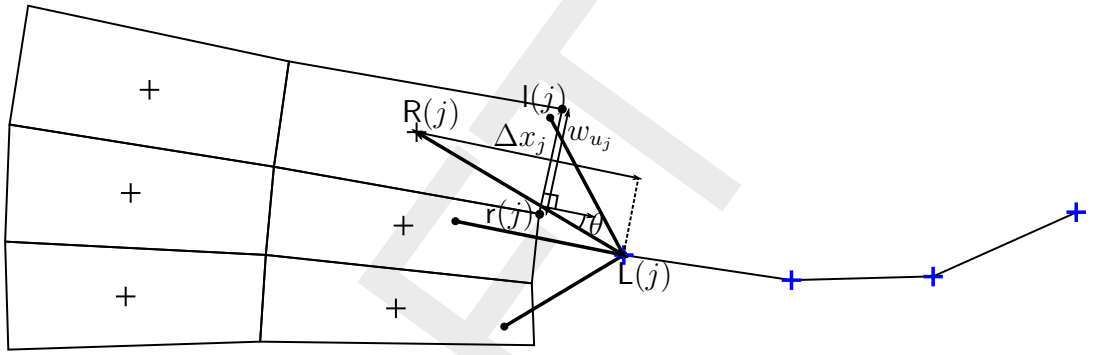


Figure 17.15: *Discretization of longitudinal 1D2D links.*

Lengths and widths

Longitudinal connections often come in the form of a 'fan-shaped' set of several 1D2D links ending up in the same number of 2D grid cells. The flow link length for each of them is intended to reflect 'unidirectional' flow perpendicular to the crossed 2D grid cell edge. As a result, the distance between the 1D and 2D pressure points for a link j is defined as:

$$\Delta x_j = \|\boldsymbol{x}_{R(j)} - \boldsymbol{x}_{L(j)}\| \cos\theta, \tag{17.12}$$

where θ is the angle between the direct 1D2D link $L(j) \rightarrow R(j)$ and the normal of the crossed 2D grid cell edge $l(j) \rightarrow r(j)$. This definition overrides the default length definition as it is used in (Deltares, 2026a, Figure 6.6).

The width (i.e., maximum flow width) of link j is defined as:

$$w_{u_j} = w_{u'_j}, \tag{17.13}$$

i.e., equal to the 2D grid cell edge's width. The latter is introduced in (Deltares, 2026a, Figure 6.4).

Bed levels

The 1D and 2D grid cells connected by the link each have their own bed level, as defined in the usual way via the `BedlevType` keyword (see section 17.3). The flow link's bed levels are defined by:

$$\begin{aligned} bl_{1_j} &= bl_{*,l(j)} \\ bl_{2_j} &= bl_{*,r(j)}, \end{aligned} \tag{17.14}$$

where the preliminary cell- or node-based bed levels $bl_{*,l/r(j)}$ were already defined in Equation (17.1). In other words: the flow link's bed levels are directly inherited from the left and right side. This overrides the 'face-based' bedlevel definition for regular flow links as given in (Deltares, 2026a, Equation (6.10)).

Remark:

DRAFT

◇ Note that the above definition 17.14 is not satisfactory for bed level type=1. It is not intended to use both the 1D and 2D bedlevels bl, but only the 2D bed level should be used. The code is subject to change on this aspect.

Face-based water depth h_{u_j}

The water depth on a 1D2D link j is defined as:

$$h_{u_j} = \begin{cases} \zeta_{u_j} - \min(bl_{1_j}, bl_{2_j}) & \text{if conveyance type} > 1 \\ \zeta_{u_j} - \max(bl_{1_j}, bl_{2_j}) & \text{otherwise,} \end{cases} \tag{17.15}$$

where the upwind water level ζ_{u_j} is defined according to (Deltares, 2026a, Equation (6.15)).

Wet cross-sectional area A_{u_j}

The wet cross-sectional area A_{u_j} is inherited from the 2D grid cell edge, and as such is computed according to Algorithm 5 in Deltares (2026a). This under the condition that all geometrical (re-)definitions in the preceding sections are used as input to the algorithm.

Advection treatment of Pure1D

To be filled in.

17.7.5 1D2D vertically stacked connection

The 1D2D vertically stacked connection can be used to model the interaction between a 1D sewer system and the 2D domain above this sewer system. The vertically stacked connection can particularly be used for street inlets and manhole cover openings. Figure 17.16 shows two examples of this.

Figure 17.16: *Examples uses for 1D2D vertically stacked connections. Left: street inlet. Right: manhole cover. (Source: GAW/Stichting RIONED)*

Lengths and widths

The distance between the 1D and 2D pressure points for a connection j is defined as:

$$\Delta x_j = \max\left(\|\boldsymbol{x}_{R(j)} - \boldsymbol{x}_{L(j)}\|,\ 0.5\sqrt{\mathtt{ba}}\right) \tag{17.16}$$

where $\mathtt{ba}$ represents the area of the 2D grid cell connected to the 1D2D connection.

DRAFT

By default, all street inlets will have the same rectangular cross section with a width of 0.2 m and height of 0.1 m. These default global values for the dimensions can be changed by specifying `Uniformwidth1Dstreetinlets` and `Uniformheight1Dstreetinlets` in the `[geometry]` section of the MDU file. Alternatively, individual street inlets can be configured with different dimensions, using the 1D-2D link file, see section C.19.

In addition to street inlets, also manhole cover openings can be modelled by a 1D2D vertically stacked connection. The default shape for vertically stacked connections is rectangular. Even when the manhole opening would be circular, the connection is typically a vertical pipe, as opposed to street inlet-pipes, which are almost horizontal. As a result, the flow from street level into the manhole will occur simultaneously from all sides. Modelling this vertically stacked connection with a rectangular profile, i.e., with a flat bed, is a good choice, because the full profile width can start to flow once there is a bit of water at the street level. It is *not* recommended to set this rectangle's width to the circle circumference, since that will overestimate the pipe's capacity. The reason is that once the pipe flow is completely filling up the pipe, it is the pipe's conveyance that is limiting the flow.

A more sensible choice for the profile dimensions of the manhole cover is a width value that yields an equivalent shape in terms of lumped conveyance, i.e., with the same hydraulic radius.

Assuming a circle with radius r, the desired hydraulic radius equals:

$$R = A/P = \pi r^2/(2\pi r) = r/2. \tag{17.17}$$

A square profile of width w and the same height $h = w$, has hydraulic radius:

$$R = A/P = w^2/(4w) = w/4. \tag{17.18}$$

So, one could set the `openingWidth` and `openingHeight` both to $2r$, i.e., the circle diameter.

Bed levels

The 1D and 2D mesh cells that are connected by the connection each have their own bed level, as defined in the usual way via the `BedlevType` keyword (see section 17.3). The flow link's bed levels are defined by:

$$bl_{1j}/_{2j} = \begin{cases} z_{l/r(j)}, & \text{if and only if } z_{l/r(j)} \text{ is defined, and } l/r(j) \text{ is an endpoint, respectively.} \\ bl_{L/R(j)}, & \text{if and only if } L/R(j) \text{ is the 2D side, respectively,} \\ bl_{*,l/r(j)}, & \text{otherwise.} \end{cases} \tag{17.19}$$

where the preliminary cell- or node-based bed levels $bl_{*,l/r(j)}$ were already defined in Equation (17.1). In other words: the flow link's bed levels are directly taken from the mesh node levels, if they are available. Otherwise, the 2D side of the flow link gets the 2D bed level, and the 1D side gets the preliminary cell- or node-based bed levels, which effectively comes down to the 1D mesh node level for most models. This overrides the 'face-based' bedlevel definition for regular flow links as given in (Deltares, 2026a, Equation (6.10)).

In the usual sense, the cell-based bed level on the left and right mesh cell are lowered (if needed) to the flow link bed levels such that link bed levels are never below their connected cell's bed levels:

$$\begin{aligned} bl_{L(j)} &= \min(bl_{L(j)}, bl_{1j}), \\ bl_{R(j)} &= \min(bl_{R(j)}, bl_{2j}). \end{aligned} \tag{17.20}$$

DRAFT

Storage volume

The storage volume in a 1D2D vertically stacked link is computed slightly different from the standard way in Equation (17.2). Only the 1D mesh node receives the regular storage contribution (that is, half the link length), and the 2D mesh cell does not. The code is subject to change on this point.

The cross sectional area is computed for a rectangular shape, with the above-mentioned width and height.

Remarks:

- ◇ When the nonlinear "Nested Newton" option is switched off (`nonlin1d` < 2) the cross section is not a closed rectangle, but an open rectangular-shape channel. As a result, this might lead to undesirable large storage in the 1D2D pipes in the sewer network, since the pipe's storage area contribution to the 1D mesh node ($= 0.5 w_{u_j} \Delta x_j$) can store water even above the pipe diameter.
- ◇ Additionally, for numerical robustness, some extra storage is added by a Preissmann slot of uniform width. The default value is 0.001 m, and this can be changed by setting the MDU keyword `Slotw1D` under `[geometry]`.
- ◇ The option `setHorizontalBobsFor1d2d` in MDU section `[numerics]` makes it possible to override the bed levels of the flow links by:

$$bl_{1_j} = bl_{2_j} = \max(bl_{1_j}, bl_{2_j}). \tag{17.21}$$

  As a result, not any storage in the 1D2D sewer connection pipe is applied until the water level in the sewer network exceeds the bed level of the 2D area.
  This setting has no effect on the friction calculation, since the face based water depth h_{u_j} is already equal to $\zeta_{u_j} - \max(bl_{1_j}, bl_{2_j})$.

Face-based water depth h_{u_j}

The water depth on a 1D2D vertically stacked link j is defined as:

$$h_{u_j} = \zeta_{u_j} - \max(bl_{1_j}, bl_{2_j}) \tag{17.22}$$

Where the upwind water level ζ_{u_j} is defined according to (Deltares, 2026a, Equation (6.15)).

Wet cross-sectional area A_{u_j}

The wet cross-sectional area A_{u_j} is calculated for a rectangular cross section using the minimized water depth $\min(h_{u_j}, \text{pipe height})$.

Conveyance

The conveyance on the 1D2D link is calculated as described in section 3.4.5.2. The Chézy roughness value is computed from a default Manning's roughness value of 0.035.

17.8 Storage nodes

Storage nodes can be used in 1D models to define custom storage dimensions. Two example uses for storage nodes are:

1. Urban manhole compartments, where bed levels, areas, street level and storage area are relevant.
2. Lumped storage areas (typically open water) tied to a 1D node, where the storage levels and areas may be provided in a table.

DRAFT

The file format details for the storage node file are described in section C.18. The sections below further describe the numerical details. The storage areas defined for these nodes are added to the normal 1D storage as defined by all connected 1D channels (or pipes) and their cross section shapes. When a model schematization also contains meteo forcings for precipitation, it is important to realize that the additional storage areas are *not* used to capture extra precipitation. Only the open water area of the grid cells (or 1D channels) is used for that.

17.8.1 Notation

The geometric and hydraulic parameters relevant for storage nodes are listed below:

$bl_{\mathrm{st},k}$	Bed level (or bottom) of storage node k;
$bl_{\mathrm{gr},k}$	Street (or ground) level of storage node k;
$A_{\mathrm{st}}(\Omega_k)$	Bottom area of storage node k (computational cell Ω_k);
$V_{\mathrm{st,max},k}$	Maximum storage volume of storage node k (without any pipes)

17.8.2 Urban manhole compartments

Manhole compartments on 1D nodes form a basic modelling element for all urban schematisations. A manhole is in fact a small reservoir that interconnects two or more sewer pipes or is an end node. Its geometry and storage volume is describe in the following subsections.

17.8.2.1 Storage volume for urban manhole compartments

The available storage volume in and above an urban manhole compartment is defined by the bed level $bl_{\mathrm{st},k}$ of node k, the street (or ground) level $bl_{\mathrm{gr},k}$ at that node and the bottom area $A_{\mathrm{st}}(\Omega_k)$ of the manhole node. The maximum storage volume of the node *within* the manhole compartment (without any pipes) equals:

$$V_{\mathrm{st,max},k} = (bl_{\mathrm{gr},k} - bl_{\mathrm{st},k}) \cdot A_{\mathrm{st}}(\Omega_k) \tag{17.23}$$

Next, the *type* of the storage node defines whether and how excess water can be stored above the street level. Two storage types can be chosen. Figure 17.17 shows the differences between them.

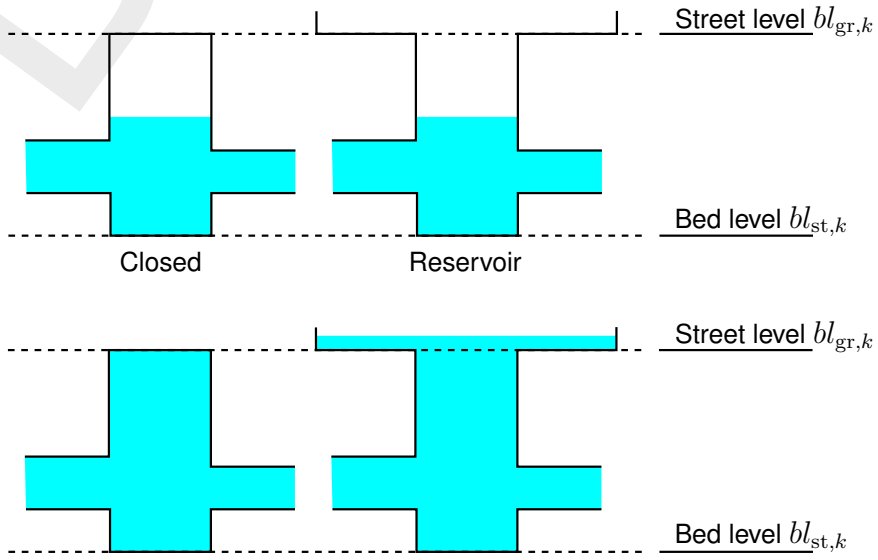


Figure 17.17: *Difference between manhole storage nodes of type "closed" and "reservoir"*

◇ **Reservoir** In case the storage type "reservoir" is chosen, there will be an open connection between the manhole and the street surface. This means that, if the water level in the

DRAFT

manhole exceeds the street level, it will inundate the "street storage area", which can be set by `streetArea`. Additionally, to easily switch from a 1D model with lumped street storage to a 1D2D model with excess storage on 2D grid cells at the street level, the global setting `useStreetStorage` can be changed to `false`. In this case, storage above ground level continues with the same bottom area $A_{\mathrm{st}}(\Omega_k)$.

◇ **Closed** In case the storage type "closed" is chosen, there will be no storage volume available above street level. For example, when a manhole has a fixed lid at the street level.

17.8.2.2 Network topology for urban manhole compartments

A single urban manhole may have multiple compartments. In that case, the network file in the model schematization must have one network node for each compartment. Figure 17.18 shows an example manhole with two compartments.

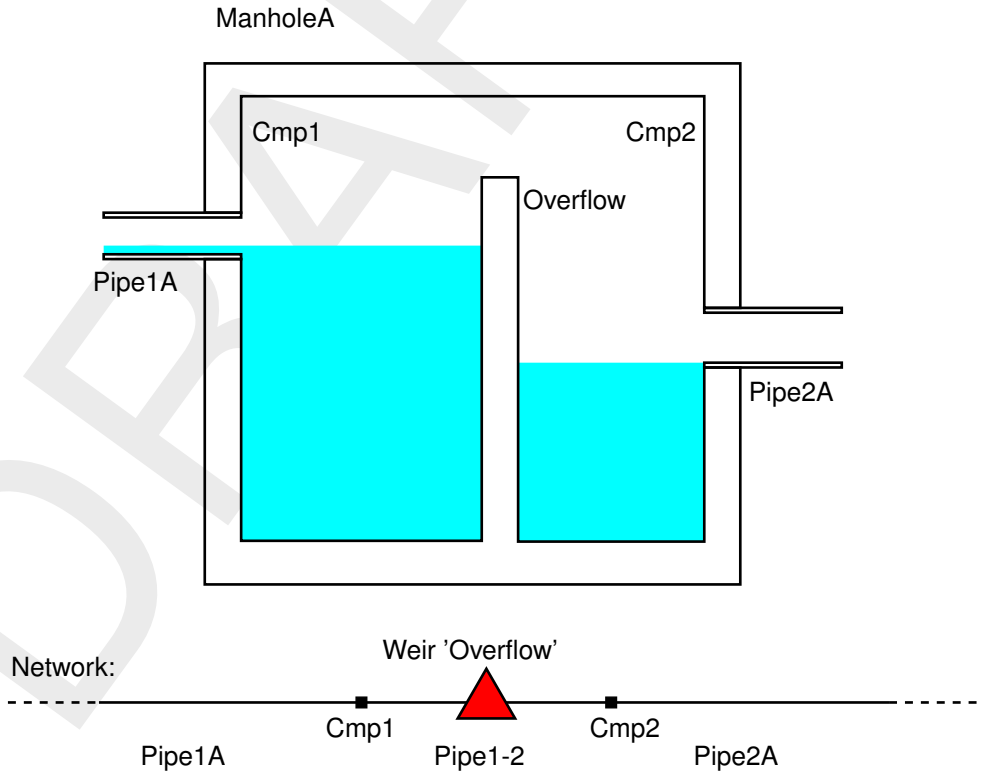


Figure 17.18: *Side view of a manhole with two compartments. At the bottom the matching network topology.*

The two sewer pipes become edges in the network and additionally, a third network edges must be present in the network to model the connection between compartment 'Cmp1' and 'Cmp2'. The type of connection between the compartments determines which type of hydraulic structure is best set on that edge. In this example, a sewer overflow structure is modelled as a weir with a crest level and a crest width. The two compartments become network nodes, and each network node will have a computational grid point placed on it.

The length of pipes is normally equal to the distance between the manhole compartment centers, but in this special case of a connection between two compartments within one manhole, it can be useful to override this Euclidean distance by a custom length. This edge length is defined as part of the network topology in the net file (see section B.2.4.2).

DRAFT

17.8.2.3 Vertical levels for urban networks

The vertical levels that define the placement of manhole compartments and connecting pipes originate from two types of input:

- ◇ A manhole compartment corresponds with a 1D pressure point and the bed level $bl_{\mathrm{st},k}$ and street (or ground) level $bl_{\mathrm{gr},k}$ are directly defined in the node file (section C.18).
- ◇ The invert levels of a sewer pipe are defined via the cross section location file (section C.17.2). The relative `shift` in that file is effectively the absolute invert level, because the closed cross sections (typically `circle` or `rectangle`) have no own absolute level (i.e., 0 m AD).

17.8.3 Storage tables for lumped water storage

Lumped water storage on 1D nodes can also be defined in a table that defines storage areas depending on water levels. The table uses blockwise interpolation.

$$i_{\mathrm{st},k}(\zeta) = \begin{cases} \max i|_{h_{\mathrm{st},k,i+1}<\zeta} & \text{if } \zeta > h_{\mathrm{st},k,2}, \\ 0 & \text{otherwise.} \end{cases} \tag{17.24}$$

$$V_{\mathrm{st},k}(\zeta) = \sum_{i=1}^{i_{\mathrm{st},k}(\zeta)} [(h_{\mathrm{st},k,i+1} - h_{\mathrm{st},k,i}) \cdot A_{\mathrm{st},k,i}] + (\zeta - h_{\mathrm{st},k,i_{\mathrm{st},k}(\zeta)}) \cdot A_{\mathrm{st},k,i_{\mathrm{st},k}(\zeta)} \tag{17.25}$$

Table 17.1: *Example storage table relating water levels to storage areas.*

$\mathbf{i = 1, N_{st}}$	$\mathbf{h_{st,k,i}}$	$\mathbf{A_{st,k,i}}$
1	-2.0	120
2	-1.5	950
3	4.3	1050

17.8.4 Manhole losses

In an urban model sewage water is transported through sewage pipes. In general, between these sewage pipes manholes are located. During the flow through a manhole from one pipe to another, an energy loss might occur. In this section it is explained how these losses can be added to a model.

The symbols, used in this paragraph are:

D_i	Diameter of pipe i (see Figure 17.19).
v_i	Flow velocity in pipe i (see Figure 17.19).
θ	Angle between pipes 1 and 2 (see Figure 17.19).
ΔH	Energy loss.
K_{loss_type}	Loss coefficient.
g	Gravity acceleration.

Subscript 1 denotes the pipe, where the flow is directed into the manhole. Subscript 2 denotes the pipe, where the flow is directed out of the manhole.

DRAFT

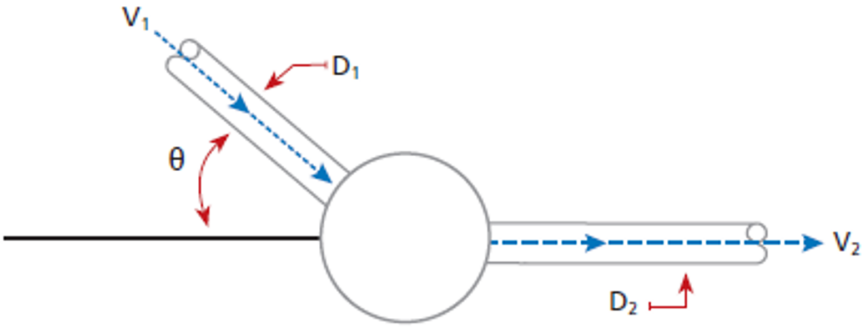


Figure 17.19: *Example of a manhole configuration. In this example the water from pipe 1 flows into the manhole and from the manhole the water flows into pipe 2*

During the flow through a manhole from one pipe to another, an energy loss might occur. There may be several causes for this energy loss. In D-Flow FM the following potential energy losses can be taken into account:

◇ *Exit loss*
The exit loss can be used to model the energy loss due to the expansion and mixing of the flow into the manhole.

$$\Delta H_{exit_loss} = K_{exit_loss} \left(\frac{v_1^2}{2g} \right) \tag{17.26}$$

◇ *Entrance loss*
The entrance loss can be used to model the energy loss due to the contraction of the flow from the manhole into the sewer pipe.

$$\Delta H_{entrance_loss} = K_{entrance_loss} \left(\frac{v_2^2}{2g} \right) \tag{17.27}$$

◇ *expansion loss*
The expansion loss can be used to model the expansion of the flow from pipe 1 to pipe 2.

$$\Delta H_{expansion_loss} = K_{expansion_loss} \left(\frac{v_1^2}{2g} - \frac{v_2^2}{2g} \right) \tag{17.28}$$

◇ *contraction loss*
The contraction loss can be used to model the contraction of the flow from pipe 1 to pipe 2.

$$\Delta H_{contraction_loss} = K_{expansion_loss} \left(\frac{v_2^2}{2g} - \frac{v_1^2}{2g} \right) \tag{17.29}$$

◇ *bend loss*
The bend loss is used to calculate the energy loss due to a change of direction of the flow, within a manhole. The user can specify a table that defines a piecewise linear function of the angle loss as a function of the angle θ, i.e. $K_{bend_loss}(\theta)$

$$\Delta H_{bend_loss} = K_{bend_loss}(\theta) \left(\frac{v_1^2}{2g} \right) \tag{17.30}$$

DRAFT

In most cases only a subset of these types of loss terms will be applied, because some of them are interchangeble.

The total energy loss is computed by adding up all individual losses. An extra condition for the total energy loss is, that this total energy loss must not be less than $0.05v_2^2/(2g)$ and may not be greater than $v_1^2/(2g) + 0.5v_2^2/2g$.

Remarks:

- ◇ These losses can be applied at storage nodes of nodeType `compartment`.
- ◇ The default behaviour is that no manhole losses are applied.
- ◇ It is possible to define the loss coefficients for all `compartment` type storage nodes at once (*global*) and/or for a storage node specific. The individual defined loss coefficients will override the global loss coefficients for a specific manhole.
- ◇ The exit loss and the bend loss are taken into account on pipe 1.
- ◇ The entrance loss is taken into account on pipe 2.
- ◇ The components for expansion loss and contraction loss are put on either side of the manhole.
- ◇ Regarding the expansion and contraction loss for manholes with more than 2 pipes connected, a total flow area of the pipes that discharges into the manhole and the total flow area of the pipes that flows out of the manhole is computed. These two total areas determine if there is an expansion or contraction. Assume the flow velocity $v > 0$ in case the flow of a pipe is directed into the manhole. E.g. in case of an expansion the total loss is:

$$\Delta H_{expansion_loss} = K_{expansion_loss} \left(\sum_{j|v_j>0} \frac{v_j^2}{2g} - \sum_{j|v_j<0} \frac{v_j^2}{2g} \right) \tag{17.31}$$

- ◇ Regarding the bend loss for manholes with more than 2 pipes connected the bend losses are calculated in a different way. The pipe with the largest discharge from the manhole into the pipe is considered the reference pipe. The angle with respect to this reference pipe is determined for each pipe with a flow into the manhole, and the corresponding energly loss is applied on this same pipe.

17.9 Flow limiters

17.9.1 Slope limiter for 1D

This subsection describes the slope limiter for the pressure gradient in case the water level drops below the invert level of a channel or pipe.

The pressure gradient in Equation (3.5) and Equation (3.6) for 1D with constant density can be rewritten into

$$\frac{1}{\rho_0}\frac{\partial P}{\partial x} = \frac{\partial \zeta}{\partial x} \tag{17.32}$$

In the numerical discretization as summarized in (Deltares, 2026a, Algorithm 16) this term becomes:

$$\frac{\partial \zeta}{\partial x} \approx -\texttt{slope} = -\frac{\zeta_{L(j)} - \zeta_{R(j)}}{\Delta x(j)} \tag{17.33}$$

In sewer systems the bed level of a manhole can be lower than the invert levels of the incoming pipes, as is shown in Figure 17.18. This situation can also occur in open channel models, but

DRAFT

this is less common, therefore we will focus on sewer systems here. In case the downstream water level drops below the invert level of the pipe, the pressure gradient in Equation (17.33) is not correct anymore. The downstream water level should in this case be equal to the downstream invert level of the pipe plus the critical depth. Since calculating the critical depth for an arbitrary shaped cross section requires a lot of computational effort the critical depth is neglected.

In the remainder of this section the upstream water level is assumed at $L(j)$, which means that the 'drop' on the $R(j)$-side has to be limited. In case the upstream water level is at $R(j)$, L and R have to be swapped in the following equations.

In case `drop1D=1` in `[numerics]` the water level slope is limited to:

$$\texttt{slope} = \frac{\zeta_{L(j)} - \max\left(\zeta_{R(j)}, z_{c,R(j)}\right)}{\Delta x(j)} \tag{17.34}$$

where $z_{c,R(j)} = bl_{2_j}$ is the invert level of the pipe at the downstream cell of face j.

Remarks:

◇ Keep in mind that this limiter is based on the previous time step:

$$\begin{aligned}\texttt{slope} = \theta\frac{\zeta^{n+1}_{L(j)} - \zeta^{n+1}_{R(j)}}{\Delta x(j)} + (1-\theta)\frac{\zeta^{n}_{L(j)} - \zeta^{n}_{R(j)}}{\Delta x(j)} \\ + \frac{\zeta^{n}_{R(j)} - \max\left(\zeta^{n}_{R(j)}, z_{c,R(j)}\right)}{\Delta x(j)}\end{aligned} \tag{17.35}$$

◇ As a result, in a dynamic situation, the limiter is not completely exact.

17.9.2 Slope limiter for 1D or 1D2D connections with advection type 8

The slope limiter that is applied for 1D and 1D2D connections with advection type 8 differs slightly from the regular approach. The main difference with the limiter, described in section 17.9.1 is in the way $z_{c,R(j)}$ is computed. Repeating Equation (17.34):

$$\texttt{slope} = \frac{\zeta_{L(j)} - \max\left(\zeta_{R(j)}, z_{c,R(j)}\right)}{\Delta x(j)}, \tag{17.36}$$

where now for this slope limiter:

$$\begin{aligned} z_{c,R(j)} &= \max\left(bl_{1_j}, bl_{2_j}\right) + \tfrac{2}{3}h_{u_j} \\ h_{u_j} &= \zeta_{u_j} - \max\left(bl_{1_j}, bl_{2_j}\right) \end{aligned}$$

In other words: the critical waterdepth $\frac{2}{3}h_{u_j}$ is added to the limiting level.

17.10 Volume tables for 1D

17.10.1 Introduction

Volume tables contain the total volume for each 1D cell as a function of the water depth. These volume tables are used by the solver to compute the volume in a grid cell for the current water level, via table lookup and simple interpolation. Using the volume tables will improve the performance of a model, since this evaluation of the volume in a cell is more efficient than recalculating the volume every time using the surrounding channel cross section information directly.

DRAFT

The volume tables are pre-calculated based on the cross sections at the location of the 1d cell (see section 3.4), the bed level information and – when present – storage nodes (see section 17.8).

The generation of the volume tables requires some extra initialization time, but during the time integration the use of volume tables is much more efficient in comparison with the default computation. As a result the total required computation time is reduced. The initialization costs can be avoided by storing the volume tables in a file, and reading them directly from this file in subsequent runs for the same model.

17.10.2 Technical information

Computing the volume in a cell without the use of volume tables requires the computation of the total area for each connected flow link, i.e., a full integration of the cross section shapes every time, for every cell. This is a time costly operation.

In a volume table only one interpolation in this table is required per grid cell. And since the volume tables use equidistant increments for the waterdepth, the lookup of the required location in the table is very simple.
The use of volume tables has a drawback. In between two table entries, any variations in cross sectional shape or storage shape are simplified to a linear variation, which makes the volume calculation slightly less accurate.

The equidistant height (increment h_{inc}) between two table entries can be defined by the user in the MDU file (chapter A). The volume table starts at index 1 (depth = 0) and ends at the depth for the heighest nearby cross section level. A smaller increment results in more accurate results, but also more table entries. When the size of the tables increases, the result will be (1) more computation time and (2) larger memory usage.

For each grid cell k the number of levels N_k is computed:

- ◇ For all cross sections that contribute to the storage of grid cell k the highest level of the cross section definition is used to compute the maximum height $h_{k,\max}$.
- ◇ The number of levels of the volume table in grid cell k depends on the highest level $h_{k,\max}$:

$$N_k = \lceil h_{k,\max}/h_{\text{inc}} \rceil + 1 \tag{17.37}$$

The volumes V_k^i for all table levels $i = 1, 2, \ldots, N_k$ are computed using the cross section information on the connected links and the storage node information on that grid cell. During the simulation, a linear interpolation is used for computing the volume in a grid cell for a given waterdepth. As a result the water surface area is presumed to be constant between two increments, and also constant above the highest level $h_{k,\max}$. The water surface area (A_k^i) is also stored in the volume tables:

$$A_k^i = \begin{cases} \frac{V_k^{i+1} - V_k^i}{h_{\text{inc}}} & \text{for } i = 1, 2, \ldots, N_k - 1, \\ A_k(N^k \cdot h_{\text{inc}}) & \text{for } i = N^k. \end{cases} \tag{17.38}$$

The calculation of the volume for a grid cell k with a given depth h_k using tables is now as follows:

$$V_k(h_k) = V_k^i + (h_k - (i-1) \cdot h_{\text{inc}}) \cdot A_k^i, \tag{17.39}$$

where

$$i = \max(0, \lfloor h_k/h_{\text{inc}} \rfloor) + 1. \tag{17.40}$$

17.11 Numerical treatment of lateral discharges

Lateral discharges are merely a source of volume, and as such they are added to the right-hand-side source term of the continuity equation (Equation (3.7)). First all lateral contributions are summed for each grid cell. This allows that multiple different laterals are contributing to the same grid point:

$$Q_{\text{lat},k} = \sum_{\ell} \sum_{k \in \mathcal{L}_\ell} Q_{\text{lat},\ell,k}, \tag{17.41}$$

where $\mathcal{L}_{\text{lat},\ell}$ is the subset of grid cells affected by lateral ℓ.

Next, positive discharges, i.e., inflows, are always fully added to the source term $Q_{\text{in},k}$. Negative discharges, i.e., outflows are limited to the available water volume in the grid cell in each timestep.

$$Q_{\text{in},k} \mathrel{+}= \begin{cases} Q_{\text{lat},k}, & \text{if } Q_{\text{lat},k} \geq 0, \\ -\min\left(|Q_{\text{lat},k}|, \frac{1}{2} V_k/\Delta t\right), & \text{if } Q_{\text{lat},k} < 0. \end{cases} \tag{17.42}$$

The relaxation factor of $\frac{1}{2}$ limits the negative source term contribution when little water is available.

Once all source terms have been collected in $Q_{\text{in},k}$, the real calculation of new water levels is done. A positive source term is always added to the right hand side of the continuity equation (3.7), and the resulting water level at that grid point is calculated implicitly by the matrix solver. A negative source term $Q_{\text{in},k}$ is solved explicitly in the continuity equation.

17.12 Implicit 1D solver

Note: The implicit 1D solver is not a released functionality, it is work in progress.

A powerful characteristic of D-Flow FM is that it can be applied to 1D, 2D, and 3D cases. This is made possible by using a generic unstructured numerical scheme. One of the characteristics of this scheme is that the time integration is done explicitly for a part of the advection term and, as such, there is a timestep limitation for the stability of the numerical scheme. The Courant number based on the flow velocity u must be below one[3]. This limitation is not a severe restriction when the timescale of interest is in the order of the resulting timestep.

However, there are cases in which this timestep restriction is severe and limits the applicability of this numerical scheme. One of these cases is one-dimensional morphodynamic modelling. One-dimensional modelling is a suitable approach when the spatial scale of interest is several times the width of the river. When the focus is on the morphodynamic behaviour at this spatial scale, the timescale of interest is several orders of magnitude larger than the timestep required for numerical stability. The consequence is that the computation needs a huge number of timesteps, resulting in a long simulation time, without gaining accuracy. Actually, the accuracy of the simulated morphodynamics decreases. This is because, in typical applications, the Courant number related to morphodynamic development (i.e., based on the bed celerity) is about three to five orders of magnitude smaller than one when the timestep is restricted by the

[3] The fast wave part $\sqrt{gh}$, where g is the acceleration due to gravity and h the flow depth, is treated implicitly, hence it does not come with a timestep limit.

DRAFT

flow velocity, as the bed celerity and the flow velocity differ by that factor. The small Courant number in the explicit time integration of the bed level causes large numerical diffusion.

For instance, consider a study in which you are interested in the morphodynamic development of a river at spatial scales of several to hundreds of kilometres over the period of months to decades. The river has a flow width of 500 m. A one-dimensional approach is appropriate since the spatial scale of interest is, at least, 10 times the flow width. For obtaining sufficient spatial accuracy, this spatial scale needs to be resolved by, at least, 10 cells. Hence, the space-step Δx will be similar to the flow width. Considering a flow velocity of 1.25 m s^{-1}, the timestep for stability of the numerical scheme, $\Delta t = \Delta x/u$, is 400 s. This contrasts with the timescale of interest of months and longer, and even with the timescale of change in the boundary conditions (i.e., upstream discharge) of days. Assuming a sediment transport rate q_b of 1 × 10^{-4} m^2 s^{-1} (including pores), a flow depth h equal to 4 m, and the sediment transport computed using Engelund and Hansen (1967), the bed celerity $c_\mathrm{b} = 5q_\mathrm{b}/h/(1-\mathrm{Fr}^2)$ (where Fr is the Froude number defined as $u/\sqrt{gh}$, equals 1.3 × 10^{-4} m s^{-1} and the Courant number for morphodynamics is 1.0 × 10^{-4}, leading to large numerical diffusion in the computation of the morphodynamic propagation.

For reducing the computational time and the numerical diffusion, an implicit flow solver has been implemented in D-Flow FM that is only applicable to one-dimensional cases. Networks of one-dimensional branches are allowed, but it cannot be applied to 1D-2D cases. The numerical scheme solves the Barré de Saint-Venant (1871) equations using the Preissmann and Cunge (1961) fully implicit scheme. The numerical implementation comes from SOBEK-RE.

One core difference with the default scheme is that it is multivalued at nodes, meaning that at each node (i.e., where several branches join) there is a value for all variables (e.g., bed level, velocity, water level) for each branch.

This feature is currently under development and not all functionality available for the default scheme is available in combination with the implicit solver. Furthermore, it cannot be run in parallel (although there is typically no need for parallelization because of its speed).

The implicit scheme is activated by setting `[numerics]` flag `FlowSolver` to `implicit1d`.

DRAFT

18 Tutorial for 1D and 1D2D modelling

This chapter includes a number of basic tutorials on setting up and running a hydrodynamic simulation with Delft3D Flexible Mesh Suite 1D2D. Different functionalities of the D-Flow FM 1D2D and D-Real Time Control modules will be discussed in the upcoming exercises. Each section provides a step-by-step guide to set up different hypothetical models with randomly chosen dimensions and parameters.

Note: in this chapter, read for Delft3D Flexible Mesh Suite either “D-HYDRO Suite” or “Delft3D Flexible Mesh Suite”.

18.1 Outline of the tutorials

Explore the basic functionalities of D-Flow FM through four tutorials in this chapter.

- Tutorial 1 (section 18.2): Hydrodynamics in open water: small stream
- Tutorial 2 (section 18.3): Hydrodynamics in sewers
- Tutorial 3 (section 18.4): Hydrodynamics in open water: larger river model
- Tutorial 4 (section 18.5): 2D model with D-RTC components

18.2 Tutorial 1: Hydrodynamics in open water

In this first tutorial, we set up a 1D hydrodynamical river model for an open water system, using a small stream in Gieterveen as a hypothetical case study. The tutorial is divided into the following steps:

- Setting up a 1D model (section 18.2.1)
- Integrating a 2D grid (section 18.2.2)
- Running the model (section 18.2.3)
- Exporting and saving the model (section 18.2.4)

18.2.1 Setting up a 1D model

In this part of the tutorial a 1D model is created for a small reach in the Netherlands. Other systems for which 1D modelling is used are:

- Polders
- Small to large rivers
- Sewer systems and urban drainage

Getting started

To start with the tutorial, you need to have the D-Flow FM software with a valid license installed on your pc.

- Click on the Delft3D Flexible Mesh Suite icon on the desktop of your computer and choose Empty project.
- Click on *New Model* in the main toolbar at the top of the Delft3D Flexible Mesh Suite GUI.
- A new window pops up in which you have to select *Flow Flexible Mesh Model*.
- In the *Project Window*, click on the + sign in front of *FlowFM* and double click on *General*. Set the coordinate system to Amersfoort RD New [EPSG:28992].
- Close the *FlowFM (FM model settings)* tab.
- Open the Map tab and click on *Add dutch layers* → *Schematic layer*.
- With one right-click on the *BRT Achtergrondkaart WMTS* in the Map Tree on the right you can choose *Zoom to extents* to see the Map.
- Zoom in to the North-East of the Netherlands and find Groningen. Follow the railroad from

DRAFT

Groningen to Zuidbroek in the East, and continue to follow it Southward to Nieuwediep. South of Nieuwediep, you will find Gieterveen. Zoom in on Gieterveen and look for the stream called de Beek.

Note: When the model area lies outside the Netherlands, a map can be added using *Add New WMS Layer* in the top of the Map Tree. Open Street Map is a worldwide map and is often chosen as background layer in projects.

Drawing the 1D river

You will create a hypothetical model of the Beek from scratch, starting with drawing the stream.

- Open the Map tab and click on *Add new branch*.
- Draw de Beek as shown in figure 18.1. To do so, start in the South East corner and click on multiple points on the river to define the geometry. When you reach the confluence in the middle, double click to finish the branch. Start again in the South West corner to draw the second branch, finish the branch by double clicking on the node in the confluence. The node changes from boundary node to connection node. Draw the third branch starting from the connection node up to the North and finish by double clicking, creating the third boundary node.
- When the shape of the branch needs needs to be corrected, you can add, remove or move geometry points afterwards by using the tools in the *Edit* section of the Map tab. When moving a geometry point make sure the cursor is on top of the point to avoid moving the whole branch.
- By clicking on the Nodes and or the Branches, you can rename them in the *Properties* window. Call the the nodes that lie most South-East, in the middle, South-West and in the North respectively NodeSE, NodeMid, NodeSW and NodeN (if they were not already named that way).
- Click on the selection button and draw a rectangle around the three channels to select the channels.
- Right-click and select *Generate computational Grid Nodes in selected branch(es)*. Then choose to generate new grid points, positioned on a preferred length of 50 meter. When zooming into a channel, you see the geometry points that define the shape of the branch in green, and the Grid Nodes in white.

Figure 18.1: *A schematization of the 1D model of the reach in Gieterveen.*

Setting the cross-sections

Now that the shape of the channel and the computational grid nodes are defined. It is time to define the dimensions of the reach. This can be done using different types of cross-sections. These are explained in 3.4. In this tutorial the ZW cross-section is used.

- ◇ Select *Add CrossSection ZW* and create one upstream and one downstream cross-section for each branch (six in total). In 1D models, the cross-sections are places as points on the branch, afterwards the dimensions are defined.
- ◇ Open the Region-tree and double click on one of the cross-sections listed there. Specify the dimensions of the six cross-sections as shown in figure 18.2. The dimensions of the defined cross-sections are interpolated between the cross sections of the same branch. Another way to edit the cross-sections is by using the selection tool and right-click on the cross-section in the schematization. Choose *edit* and select the cross-section.

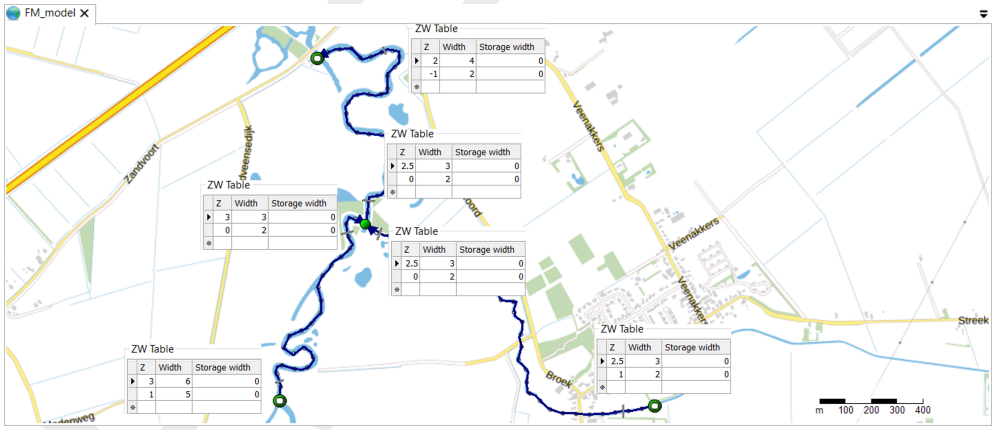


Figure 18.2: *Cross-sections.*

Setting the boundary conditions

Boundary conditions are used to define what happens at the boundaries of a model. You can choose for water level boundaries or discharge boundaries, either as a constant value, a time dependent series or let the water level and the amount of discharge dependent on each other. Sometimes the model has an integrated Real Time Control (RTC) model, in that case real time data can be used as boundary conditions for the model. Other times, for calibration or scenario purposes, you can use historical data as input or for the boundary conditions.

- ◇ In the Project Window, open the Project Tree and click on *1D* → doubleclick on *1D Boundary Conditions*. The Boundary Data editing window opens.
- ◇ Select NodeSE in the editing window.
- ◇ Change the DataType of NodeSE to *Q: constant flow*.
- ◇ Right-click on NodeSE (in the Boundary Data editing window) and select *open view*. Set the constant flow in NodeSE to 0.4 m3/s.
- ◇ Repeat this step for the other nodes, where NodeMid stays 'None', NodeSW receives a constant discharge (Q:constant flow) of 1 m3/s and NodeN is defined by a constant water level (H:Constant water level) of 0.8m.

Setting the roughness

The roughness coefficient is used to define the roughness of the bedlevel.

- ◇ Open the Project Tree and click on *General*. Open the Physical Parameters tab and set the Uniform friction type of the 1D system (Roughness 1D) to Manning with a friction

coefficient of 0.02. In this window the general settings are defined. Besides the general roughness settings, it is also possible to change the roughness for different channels.

- ◇ Open the Project Tree and click on *1D* → *1D Roughness* → doubleclick on *Channels*. The Channels editing window opens.
- ◇ All the channels are in default set to the Global Value that was defined in the previous step. On the larger reach, there are more boats then on de Beek (Channel_1D_1), therefore, the latter has more plants growing in the river bed. This increases the roughness of the bed. Change the *Specification* of de Beek to Branch Constant, the *roughness type* to Manning and the value to 0.03, indicating a channel with some stones and plants in the channel bed.

General Model Settings

Before you are able to run a model, the computational parameters need to be set. This involves for example the *Time Frame* and *Initial Conditions*. Most parameters can be set in parameter screens accessible through the Project Tree, under *General*.

- ◇ Open the Time Frame settings and set the Reference date to 2023-01-01, the Start Time to 2023-01-01 and the Stop Time to 2023-01-03.
- ◇ Open the Initial Conditions settings and set the 1D initial conditions to 1m Water depth.

Add structures

Now that the basic 1D model is created, it is time to add some structures. Figure 18.3 shows the locations of the two structures and the lateral source that you are going to add.

First you will add the culvert, this structure can represent a narrowing due to an arch or a pipe connecting two streams with each other.

- ◇ Open the Map tab and select the culvert from the *Network* section. Install the culvert at the location where de Beek flows under the road. When the culvert is placed, press the Esc button in order to leave the culvert mode and change the cursor to the selection tool again.
- ◇ Select the culvert and right-click → Edit → Culvert_1D_1 to open the editing window.
- ◇ Set the Geometry type to Arch and define the dimensions of the culvert in the editing window. Set the width and the height to 0.5 m and change the Inlet and Outlet level to 0.5 mAD.
- ◇ Close the editing window and select the weir from the *Network* section in the Map tab. Install the weir and change to the selection tool. Then double click on the weir to open the editing window. Set the weir properties with a crest level of 1.5 mAD and a crest width of 2.0 m.
- ◇ Close the editing window and click on the lateral source . Then install the lateral source on the 1D branch and set the properties to Constant flow data of 0.25 $[m3/s]$.

DRAFT

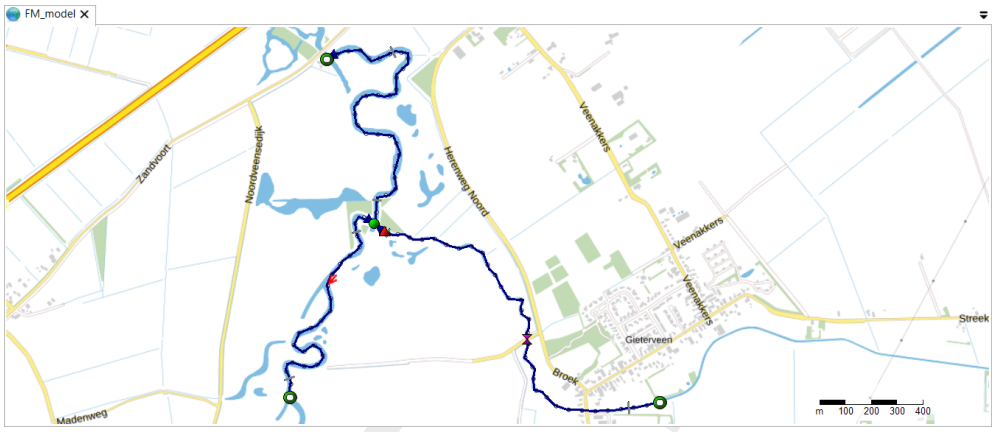


Figure 18.3: *Adding structures to the 1D branches.*

18.2.2 Integrating a 2D grid in the model

This part of the tutorial will show how you can add a 2D grid of the DEM to the model.

- ◇ Start by creating a 2D grid. Open the *Grid-tab* and enable the editing mode by clicking once on the *Edit* button. In the tools section of the menu, the type of function can be selected. After a tool is selected, the type of action can be selected.
- ◇ Click on *Geometry* tool and select *Draw/Edit* in the Polygons section. Draw a polygon around the project area. End the polygon by a double-click.
- ◇ Click on *Mesh* in the Tools section. Select the polygon. Note that *Rectangular in polygon* in the Create section is enabled. Click on *Rectangular in polygon*. Set up a grid with a grid block size of 15x15 and 0 degrees grid angle. Generate the grid by clicking on *OK*. Then unable the editing mode by clicking on *Edit* button and save the changes. Note that the grid color changes from black to orange. It looks like one big orange square, but when you zoom in you will see the 15x15 meter grid lines.
- ◇ Open the Spatial Operations tab and select the Bed level layer in the dropdown menu. Click on *Import*. Choose the `DEM_Gieterveen.xyz-file` that was provided with this tutorial. Import the file without transformation.
- ◇ At this point, a $2D$ grid is created and the bedlevel samples are imported. Open the *Operations-tree* in the right lower corner and select the output of Set 1. Open the *Spacial Operations* tab at the top and click on *Interpolate*. Choose to interpolate the selected set and set the parameters to 'Overwrite where missing' and 'Triangulation'. Each gridcel will receive one bedlevel value, see figure 18.4.
- ◇ Expend in the Map Tree the following *FM_model* → *2D* → *Bed Level* → *Interpolate 1* right-click on Output and click on Legend Properties. Choose the color settings and NoData values to your preference and click on *Generate* → *Apply* → *OK*.
- ◇ Open the Map Tree, right-click on *FM_model* → *2D* → *Bed Level* and choose *order* → *Bring to back* to show the bedlevel layer underneath the 1D elements.

Note: The PDOK website provides open source DEM and DTM data for the Netherlands.

DRAFT

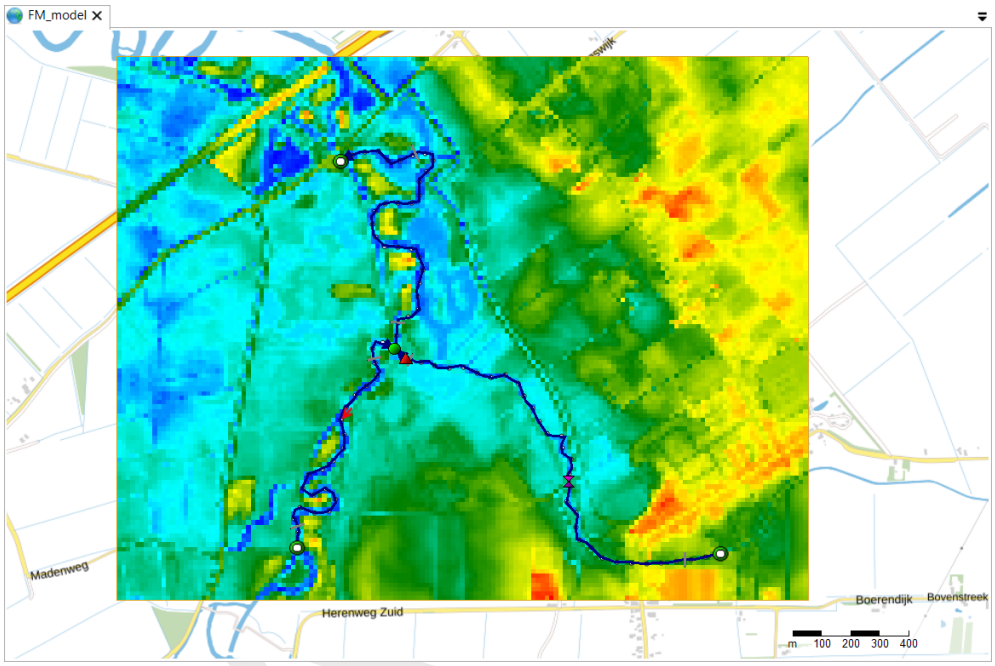


Figure 18.4: *Interpolating the DEM samples over the created 2D raster*

Linking the 1D model to the 2D network

In order to create exchange of water between the 1D and the 2D system, 1D2D links are needed. There are four types of 1D2D coupling, as explained in chapter 17.7. Since the current model consists of small channels where the 2D grid cells overlap the 1D channels, the 1D2D embedded coupling is most suitable.

- ◇ Open the Map tab and select the Embedded 1-to-1 links in the 1D2D Links section.
- ◇ Click on *Generate Links* and draw a polygon around the 1D model.
- ◇ Zoom in to one of the calculation nodes on the branch and check whether the calculation nodes now have a 1D2D link with the nearest 2D grid cell.

Adding meteorological forcing

There are two ways to include meteorological data in the model: by adding an RR model and specifying the data per catchment or by forcing the data on the whole 2D grid. In this example, we use the second option.

- ◇ In the project window, open the *FM_model* → *2D* → *Physical Parameters* → double click on *Meteo items*.
- ◇ The Meteo Editor window opens. Click on *Add* and select Global Fm Meteo precipitation time series, press *OK*.
- ◇ The window shows different options for adding data. For simplicity reasons, click on *Generate* and create a 12-hourly dataset between 01-01-2023 and 03-01-2023.
- ◇ Define the precipitation event as shown in figure 18.5.

Note: By selecting a range of cells in a column, adding one value and then pressing ctrl+enter, all the cells will receive that value.

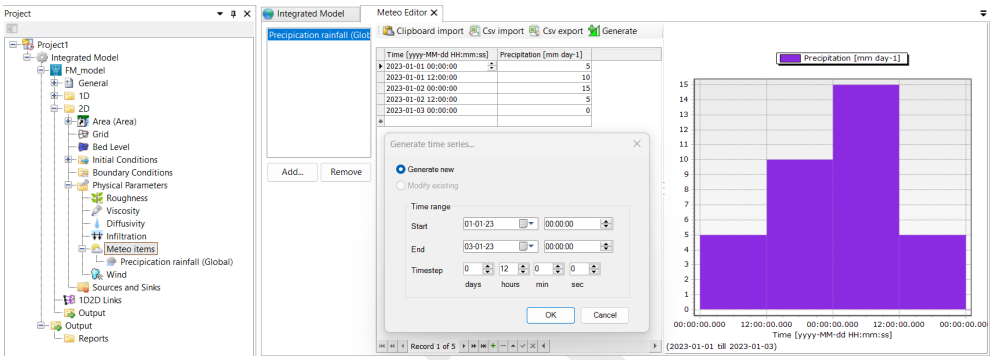


Figure 18.5: *Adding precipitation to the model.*

18.2.3 Running the Model

To check if the model set-up in the previous steps is complete, right-click on *FlowFM* and select *Validate*. If all the bullets are checked, the model is ready to run. If not, check the message and add or change that feature in the model. To do so, right-click on *FlowFM* and select *Run Model*.

Viewing the Result

Chapter 8 explains in more detail what the possibilities are to visualize the results in D-Flow FM. This model has been set-up correctly if the 2D waterdepth map at the last time step looks the same as figure 18.6

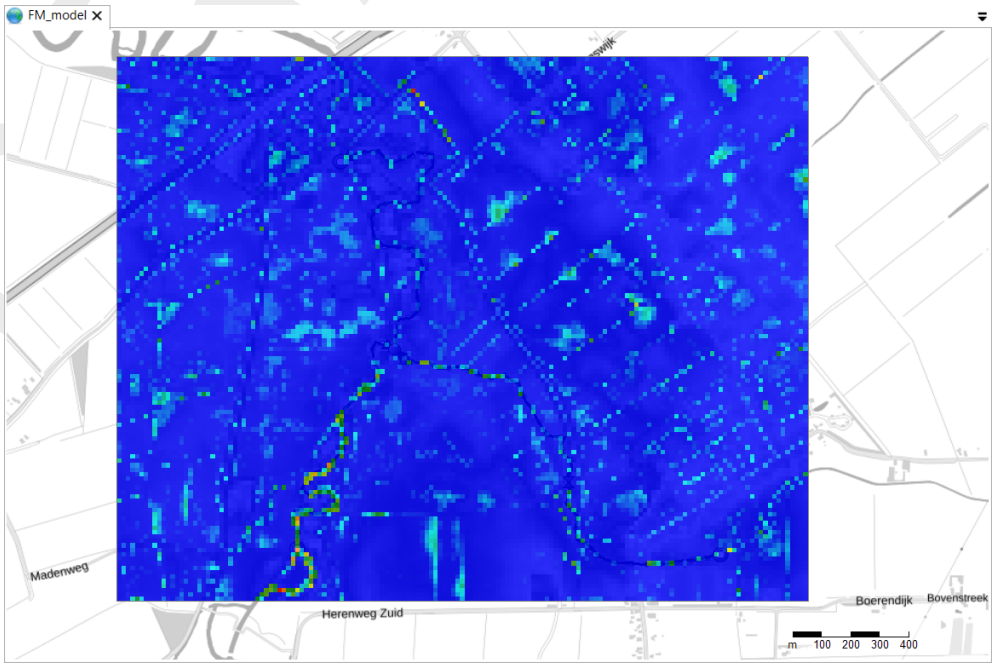


Figure 18.6: *Visualizing the* Waterdepth at pressure points *results at the last time step.*

18.2.4 Exporting and saving the model

After creating the model, you can choose between saving or exporting the model. When saving the model, the output will be saved as well, which makes it easy to open the model again and watch the results in the GUI. To save the model, use the *Save model* function in

the upper left corner of the main window. Exporting the model saves just the model settings, the advantage is that it consumes less storage space and that the model is validated first and will give an error if it is unable to run. In order to export the model, select the FM_model in the Project Window and click on Export in the Home tab. Then choose to export the *DIMR configuration*.
Another reason to export the model, is that it offers the opportunity to run the model outside the GUI, using a batch file. This is faster, and makes it easier to experiment with different scenario's. Chapter 7.3 explains how to set up a batch file in more detail.

18.3 Tutorial 2: Hydrodynamics in sewers

In this tutorial, you will set-up a 1D sewer system around Westminster Abbey in London. Sewer systems are modelled using the Rainfall Runoff module. The tutorial is divided into the following steps:

- ◇ Setting up a 1D sewer model
- ◇ Include the Rainfall Runoff concept
- ◇ Inserting NWRW catchments

18.3.1 Setting up a 1D sewer model

- ◇ Click on the Delft3D Flexible Mesh Suite icon on the desktop of your computer and start an empty project.
- ◇ Click on *New Model* in the main toolbar at the top of the Delft3D Flexible Mesh Suite GUI.
- ◇ A new window pops up in which you have to select *Empty Integrated Model*.
- ◇ Make sure the Integrated Model in the Project Window is selected and click again on *Home* → *New Model* → *Flow Flexible Mesh Model*. Repeat this step and add a *Rainfall Runoff Model*. When done correctly, the models are listed in the Integrated Model section of the Project Window.
- ◇ Double-click on *General* and set the coordinate system to *EPSG:27700* (OSGB36 / British National Grid).
- ◇ Click on the *Time Frame* tab. Set the Reference date and the Start time to 2024-06-01 00:00:00. Set the Stop Time to 2024-06-07 00:00:00.
- ◇ Click on *Initial Conditions* and set the Initial conditions (1D) to a water level of -0.5 mAD.
- ◇ Double-click on *Integrated Model* in the project tree so that the *Integrated Model Settings* window opens. Adjust the time settings of the Rainfall Runoff model so that they correspond with the FlowFM model.
- ◇ Click on *Map Coordinate System* in the Map tab and choose the right coordinate system there as well.
- ◇ Click on *Add New WMS Layer* at the top of the Map Tree and add an Open Street Map. When a window pops-up with the question to adjust the map coordinate system, press *No*.
- ◇ Zoom into London and find Westminster Abbey.

Note: It is also possible to add an Integrated Model directly from the start page of Delft3D Flexible Mesh Suite.

Make a schematization of the sewer system
In this part of the tutorial you are going to create the schematization as shown in figure 18.7.

DRAFT

A sewer system is a 1D system and therefore you use the tools in the *1D Network* part of the Map tab.

- Click on *Add new pipe* and draw the seven black pipes (from West to East) as shown in figure 18.7. To draw the pipe, click on the start point and then double click on the end point of the pipe. The second until the sixth pipe are connected and the first and the last pipes are a couple meters apart from each other. In order to make the connecting pipes, make sure that the end of a pipe, and the start of a new pipe are connected to the same manhole (orange node).
- Click on *Add new sewer connection* and connect the downstream node of the first pipe with the upstream node of the second pipe. Repeat this for the sixth and the seventh pipe. Finally, add one sewer connection between the last pipe and the imaginary overflow point in the Thames.
- Add one more sewer connection from the downstream node of the seventh pipe towards the River Thames. This connection represents a sewer overflow.
- Select a pump and insert it on the first sewer connection. Insert an orifice on the second sewer connection and place a weir between the Black Rod's Garden and the Thames.

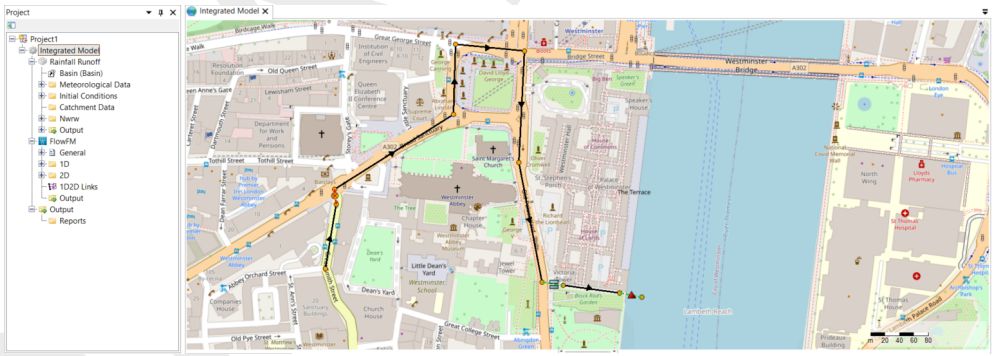


Figure 18.7: *Schematization of the sewer system.*

Define the instrument parameters

- Click on the *Select* tool in the *Tools* part of the Map tab.
- Right-click on the pump → edit → Pump_1D_1. Set the pump capacity to 5 m3/s. Set the parameter *Start suction* to 0.5 mAD and the parameter *Stop suction* to 0 mAD.
- Right-click on the orifice → edit → Orifice_1D_1 and set the Crest level, Crest width and Lower edge level to -0.8 mAD,3.0 m and -0.7 mAD respectively.
- Right-click on the weir → edit → Weir_1D_1 and set the crest level and crest width to -0.5 mAD and 5.0 mAD respectively.

Set the drainage system parameters

- Open the Map Tree and expand *FlowFM* → *1D* → *Network* → *double-click on Pipes*. A window opens with dimensions of the seven pipes that were added.
- Right-click on the first pipe and click on *Open view*. The editing window of the first pipe opens.
- Click on *Create new shared definition* and click on the three dots on the right side of the new definition to change the dimension. Change the Type to Rectangle with a Height of 1.00 m and a Width of 2.00 m. Close the editing window and move back to the pipe settings. Make sure the new definition is selected.
- Move back to the Integrated Model view and change the rest of the pipe parameters (invert

DRAFT

levels) as show in figure 18.8. Note that the definition of each pipe is changed to the definition created in the step above.

- ◇ Double-click on the *Compartments* in the Map Tree, so that the data opens in the Multiple Data Editor. Change the Bottom level of all the compartments to -2m. Set the surface level for each compartment to 1m.
- ◇ Double-click on the *sewerconnections* in the Map Tree. Set the parameters 'invert level from' and 'invert level to' of the first sewer connection 0.4 and 0.6 respectively. The pump is placed on this sewer connection to make sure the water can be pumped towards the higher level in the second pipe.

	Name	From manhole	To manhole	From compartment	To compartment	Geometry le...	Length (m)	Order...	Invert level from	Invert level to	Sewer type	Sewer...	Type	Width	Definition
▸	Pipe_1D_1	Manhole001	Manhole002	Compartment001	Compartment002	168.548137726	168.548137726	-1	0	-0.4	Combined	None	Rectangle	2	SewerProfile_1
	Pipe_1D_2	Manhole003	Manhole004	Compartment003	Compartment004	305.222940417	305.222940417	-1	-0.2	-0.2	Combined	None	Rectangle	2	SewerProfile_1
	Pipe_1D_3	Manhole004	Manhole005	Compartment004	Compartment005	144.519019542	144.519019542	-1	-0.2	-0.2	Combined	None	Rectangle	2	SewerProfile_1
	Pipe_1D_4	Manhole005	Manhole006	Compartment005	Compartment006	156.615621237	156.615621237	-1	-0.2	-0.4	Combined	None	Rectangle	2	SewerProfile_1
	Pipe_1D_5	Manhole006	Manhole007	Compartment006	Compartment007	243.333791213	243.333791213	-1	-0.4	-0.6	Combined	None	Rectangle	2	SewerProfile_1
	Pipe_1D_6	Manhole007	Manhole008	Compartment007	Compartment008	254.176974336	254.176974336	-1	-0.6	-0.8	Combined	None	Rectangle	2	SewerProfile_1
	Pipe_1D_7	Manhole009	Manhole010	Compartment009	Compartment010	141.449179372	141.449179372	-1	-0.8	-1	Combined	None	Rectangle	2	SewerProfile_1

Figure 18.8: *Pipe definitions in the sewer system.*

18.3.2 Include the Rainfall Runoff concept

This part of the tutorial explains how to include the Rainfall Runoff module. There are different Rainfall Runoff concepts available in Delft3D Flexible Mesh Suite.

- ◇ NWRW concept
- ◇ Paved/unpaved
- ◇ HBV
- ◇ Sacramento

Within the Netherlands, the NWRW (named after the Nationale Werkgroep Riolering en Waterkwaliteit) concept is often used. A NWRW model describes the dry weather flow and the transformation in time of rainfall into runoff entering the sewer system. The processes that are included are surface storage, infiltration and runoff delay. This tutorial will explain how the NWRW concept can be implemented.

Include meteorological data

- ◇ Click on *Integrated model* → *Rainfall Runoff* → *Meteorological Data* → *Precipitation*.
- ◇ Note the dropdown menu on the right, and change *Global* to *Meteo stations*
- ◇ Write *Station_1* in the textbox and click on *Add*.
- ◇ Select the station and click on *Generate Time Series* . Create a rainfall set from 01-06-2024 until 07-06-2024, with daily timesteps.
- ◇ Write *Station_2* in the textbox and click on *Add*. A new column is added to the Time Series table. Fill the table as in figure 18.9.
- ◇ Double-click on *Evaporation* and click on *Generate Time Series*. Create a Global daily timeseries from 01-06-2024 until 07-06-2024 and set the Evaporation values to 1 mm for each day.

DRAFT

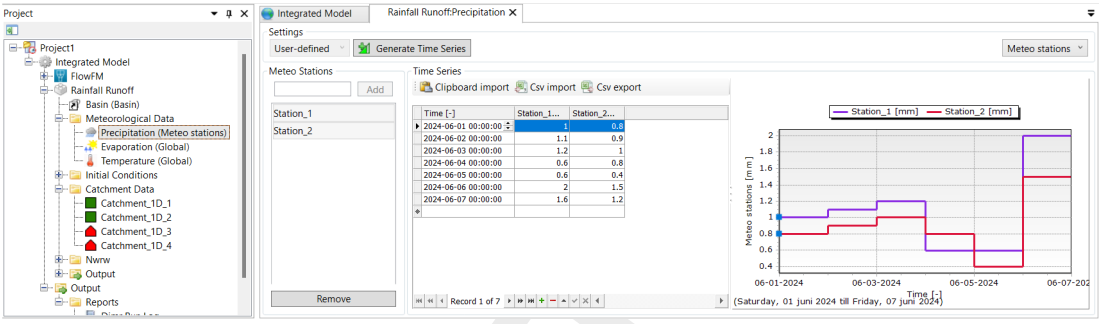


Figure 18.9: *Setting precipitation data.*

18.3.3 Inserting NWRW catchments

The sewer system is now installed but there are no households or surfaces connected to the system. In this part of the tutorial you learn to set-up NWRW catchments.

- ◇ Open the Map Tree and expand *FlowFM* → *1D* → *Network* → *Manholes*. Select the Manholes and check *Use focused layer* at the top of the Map Tree window. In this mode, it is only possible to select manholes, which will help in the upcoming steps. When the focussed layer is turned on, only the selected elements are shown in color.
- ◇ Select the most right manhole, that lies in the Thames, right-click on the manhole and choose *Upgrade to outlet compartment*, see figure 18.11b. This means that the manhole is now able to discharge water from the sewer onto the Thames.
- ◇ Open the Map Tree and expand *FlowFM* → *1D* → *double-click on Boundary Data*. Then change the DataType of the outlet compartment (Manhole011) into *H: Constant water level*. Right-click on this manhole and select *open view*. Set the Water level boundary condition to -2 m, representing a constant water level in the Thames.
- ◇ Return to the Integrated Model map view, and select the remaining ten manholes.
- ◇ Right-click somewhere on the map and select *Add NWRW catchment* → *Selected manholes*, see figure 18.11a.
- ◇ Uncheck the *Use focused layer* box at the top of the Map Tree window.
- ◇ Move to the Project Window *Rainfall Runoff* → *double-click on Catchment Data*, the Multiple data editor of the (NWRW) catchments opens. In this table, you can define the specification of the catchment that is connected to each manhole and the amount of people and companies in this catchment.
- ◇ For this exercise, set the *Open Flat area* of the first eight compartments to 2000 m2 and the last two to 5000 m2. Set the *Unpaved Flat area* for each catchment to 500 m2. Set the Dry Weather Flow (DWF) of the first six catchments to *Weekend_day* for inhabitants and *Week_day* for companies, with 100 inhabitants and 10 companies for each catchment. Set the DWF for the last four catchments to *Week_day* for inhabitants and *Weekend_day* for companies, with 50 inhabitants and 20 companies for each catchment. Finally, set the *Meteostation identification* of the first six catchments to Station_1 and the last four catchments to Station_2, see figure 18.10.
- ◇ Move to the Project Window *Rainfall Runoff* → *Nwrw* → *double-click on Dryweather Flow Definition*. In this window, you can define different types of dry weather flow, constantly spread out over the day or variable per hour. There are already some default flows defined, you can make new ones or adjust the examples. Change the Weekend_day Daily volume to 210 L/day. Keep the Week_day Daily volume at 240 L/day.
- ◇ Move to the Project Window *Rainfall Runoff* → *Nwrw* → *double-click on Nwrw Surface Settings*. Set the *Surface storage* for all the layers to 2 mm and the *Maximum infiltration capacity* to 0.5 mm/h.

DRAFT

Integrated Model | Multiple data editor (D-RR)

NWRW | Unpaved | Paved | Greenhouse | OpenWater | HBV | Sacramento

Area Id	Clo... Slo...	Clo... Flat	Clo... Str...	Open Slop...	Open Flat area (m²)	Open Stretch	Roof Sl...	Roof Flat	Roof St...	Unp... Slo...	Unpaved Flat area	Unp... Str...	Number of units	DWF definition	Number of units	DWF definition	Meteostation identification
Compartment001	0	0	0	0	2000	0	0	0	0	0	500	0	100	Weekend_day	10	Week_day	Station_1
Compartment002	0	0	0	0	2000	0	0	0	0	0	500	0	100	Weekend_day	10	Week_day	Station_1
Compartment003	0	0	0	0	2000	0	0	0	0	0	500	0	100	Weekend_day	10	Week_day	Station_1
Compartment004	0	0	0	0	2000	0	0	0	0	0	500	0	100	Weekend_day	10	Week_day	Station_1
Compartment005	0	0	0	0	2000	0	0	0	0	0	500	0	100	Weekend_day	10	Week_day	Station_1
Compartment006	0	0	0	0	2000	0	0	0	0	0	500	0	100	Weekend_day	10	Week_day	Station_1
Compartment007	0	0	0	0	2000	0	0	0	0	0	500	0	50	Week_day	20	Week_day	Station_2
Compartment008	0	0	0	0	2000	0	0	0	0	0	500	0	50	Week_day	20	Week_day	Station_2
Compartment009	0	0	0	0	5000	0	0	0	0	0	500	0	50	Week_day	20	Week_day	Station_2
Compartment010	0	0	0	0	5000	0	0	0	0	0	500	0	50	Week_day	20	Week_day	Station_2

Figure 18.10: *Catchment Data.*

(a) *Setting the manholes to NWRW catchments.* ***(b)*** *Setting the manhole to an outlet compartment.*

Figure 18.11: *Creating NWRW catchments and defining the outlet compartment.*

18.3.4 Running the model and visualizing the results

- ◇ Right-click on *Integrated Model* and click on *Run Model*.
- ◇ Click on *Add route* on the right side in the Map tab. Click on the first manhole of the sewer system and then on the last manhole to draw a route through the system. Then click on *Show Side View*.
- ◇ Drag the bar in the Time Navigator at the bottom of the screen to see the water level changes over time. Figure 18.12 shows how the sideview should look at a certain timestep.

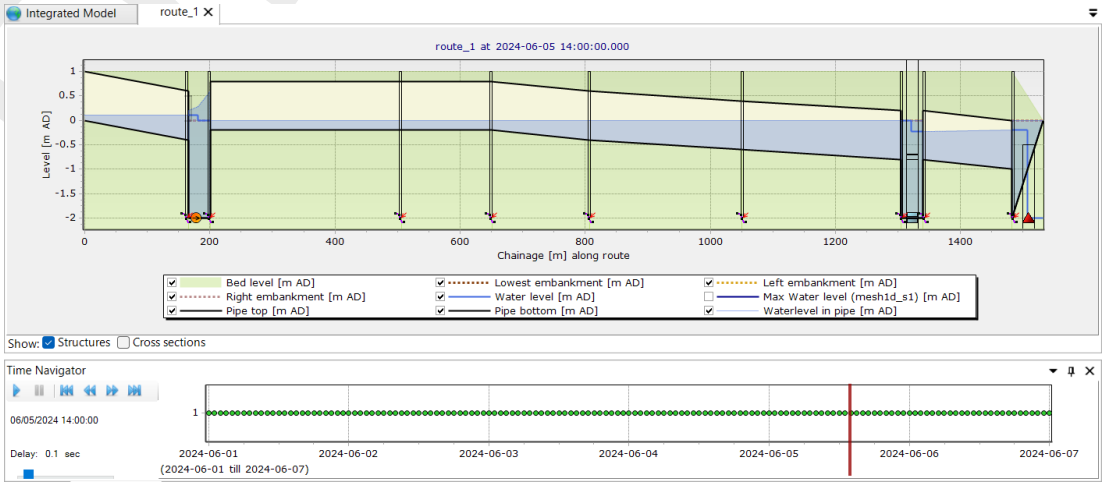


Figure 18.12: *Visualizing the results using the sideview function.*

18.4 Tutorial 3: Hydrodynamics in 1D2D river models

In this tutorial, you will set-up a 1D2D model of the Magdalena river in Colombia. The tutorial is devided into the following steps:

- ◇ Setting up the 1D model

- ◇ Expanding the 1D model to a 1D2D model
- ◇ Refining the 2D grid near the stream

18.4.1 Setting up the 1D model

- ◇ Open the Delft3D Flexible Mesh Suite program and start a new FM model. Choose the coordinate system *Bogota 1975/UTM zone 18N* and click on *Create* as shown in figure 18.13.
- ◇ Click on *Add New WMS Layer* at the top of the Map Tree and add an Open Street Map.
- ◇ Zoom into the city of Baranquilla, located at the North coast of Colombia.

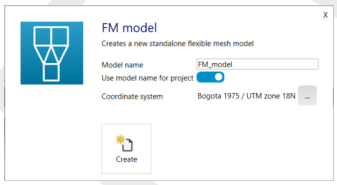


Figure 18.13: *Opening an FM-model and selecting the right coordinate system.*

- ◇ In the 1D Network section, choose *Add new branch (Freeform)* .
- ◇ Draw a line that starts near Córdoba Tetón and end at the coast in Baranquilla, representing the Magdalena river. Release the button to finish the drawing of the branch.
- ◇ When the shape of the branch needs needs to be corrected, you can add , remove or move geometry points afterward.
- ◇ Click on *Zoom to Extent* to get an overview of the river, see figure 18.14.

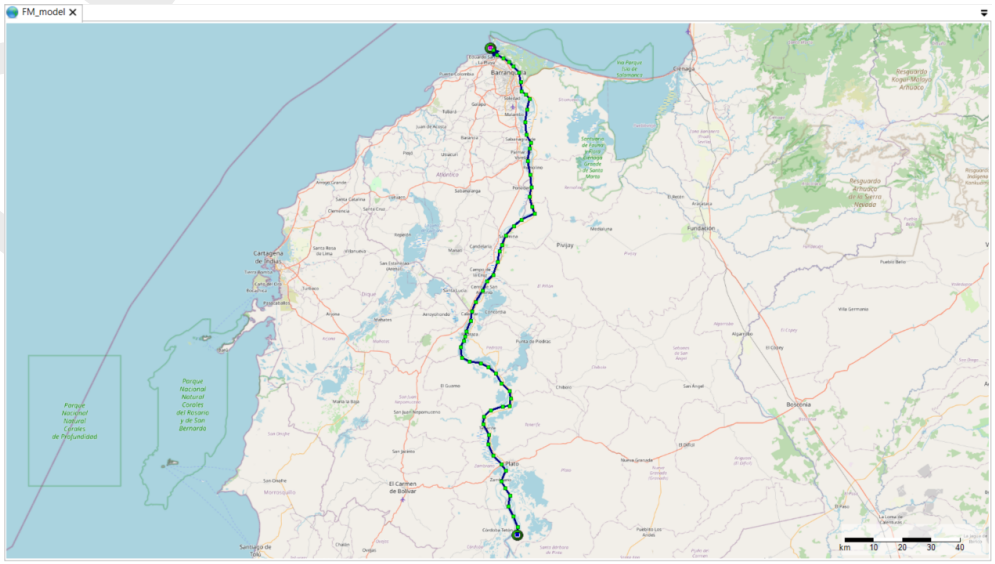
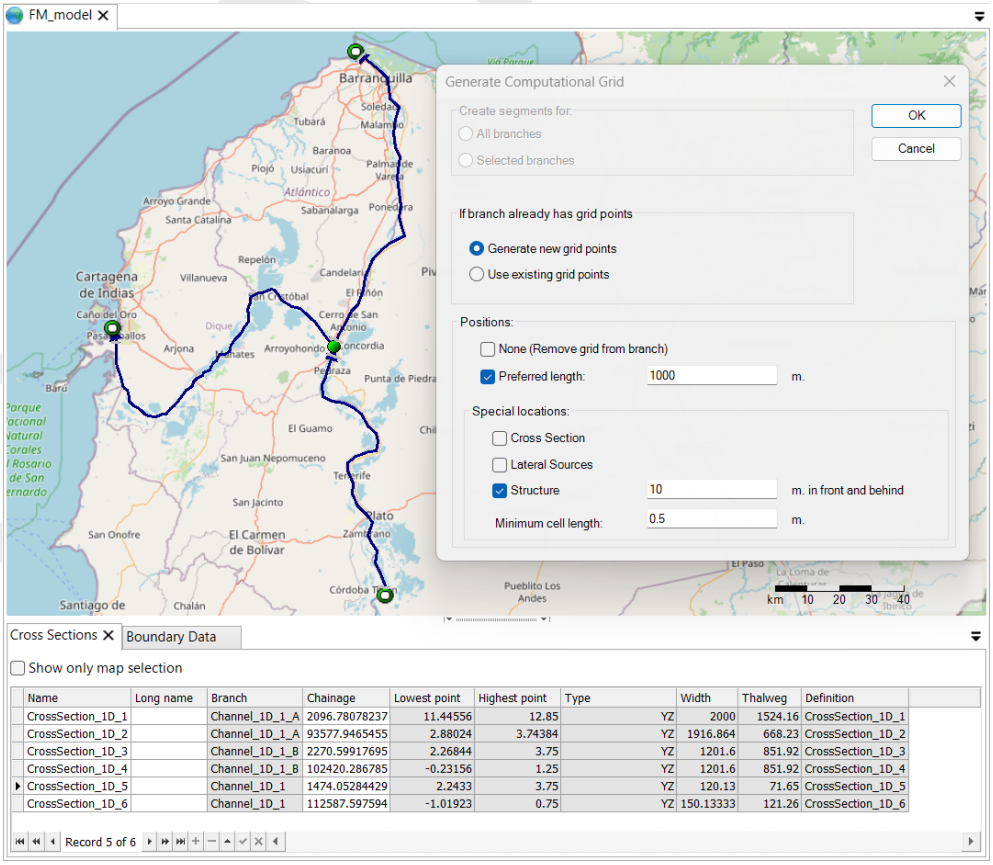


Figure 18.14: *Schematization of the Magdalena river.*

Set-up the river geometry

- ◇ Zoom in to Calamar and note that the river there is a distributary called the Canal del Dique, seperating Atlantico from Bolivar.
- ◇ Select the *Split Branch* button in the *1D Network* section and split the branch at the location where the Canal del Dique splits from the Magdalena river. Use again the *Add new branch (Freeform)* function to draw the Canal del Dique up to Pasacaballos.

- ◇ Add a YZ-cross-section to each begin and end point of the three branches. Start with the most Southern point of the Magdalena River, move up to the most Northern point of the Magdalena River and then add a cross-section at the start of the Canal del Dique and one at Pascaballow.
- ◇ Go to the Map Tree and click on *FM_model* → *1D* → *network* and double-click on *Cross Sections*. The multiple data editor opens, where you can see the six cross-sections that were added in the previous step. Right-click on a cross-section and select *Open view* to edit the cross-section geometry.
- ◇ Use the Excel file *1D_YZ_CrossSections.xlsx*, in the data folder corresponding to this tutorial, to define the geometry of the different cross-sections (each cross-section is defined on a different tab in the Excel sheet). When done correctly the Multiple Data Editor will look like the table in the bottom of figure 18.15.
- ◇ Double-click on *FM_model* in the project window to move back to the overview of the model.
- ◇ Right-click somewhere in the map and select *Generate computational gridnodes*.
- ◇ Choose to generate new grid points with a preferred length of 1000 m, as shown in figure 18.15.



Name	Long name	Branch	Chainage	Lowest point	Highest point	Type	Width	Thalweg	Definition
CrossSection_1D_1		Channel_1D_1_A	2096.78078237	11.44556	12.85	YZ	2000	1524.16	CrossSection_1D_1
CrossSection_1D_2		Channel_1D_1_A	93577.9465455	2.88024	3.74384	YZ	1916.864	668.23	CrossSection_1D_2
CrossSection_1D_3		Channel_1D_1_B	2270.59917695	2.26844	3.75	YZ	1201.6	851.92	CrossSection_1D_3
CrossSection_1D_4		Channel_1D_1_B	102420.286785	-0.23156	1.25	YZ	1201.6	851.92	CrossSection_1D_4
CrossSection_1D_5		Channel_1D_1	1474.05284429	2.2433	3.75	YZ	120.13	71.65	CrossSection_1D_5
CrossSection_1D_6		Channel_1D_1	112587.597594	-1.01923	0.75	YZ	150.13333	121.26	CrossSection_1D_6

Figure 18.15: *Generating the computational gridnodes in the Magdalena river.*

Defining the model settings

- ◇ Open the *Time Frame* settings in the project window and set the Reference date and the Start time to $2024-01-01$. Set the Stop time to $2024-01-07$ and leave the rest of the settings in default.
- ◇ Open *FM_model* → *1D* → *Boundary Conditions*, right-click on Node001 → *Open view*. Set the data type to Q(t): Flow time series and click on Csv import. Choose the file

1D_BoundaryConditions.csv from te data folder and import the boundary data. When clicking through the Wizard make sure that the right delimiters are chosen (the first value should be 100 m3s on 2024-01-01 00:00:00).

- ◇ Change the boundary conditions of the boundary nodes that are connected to the sea to constant water level of 0m.
- ◇ Open the multiple data editor of the *Roughness data* by double-clicking on *FM_model* $\rightarrow$ *1D* $\rightarrow$ *1D Roughness* $\rightarrow$ *Channels*. Set the Specification of each branch to *Branch Constant*. Set the Manning value of the most Southern branch to $0.03\,\mathrm{s}\cdot\mathrm{m}^{-\frac{1}{3}}$ and the other branches to $0.02\,\mathrm{s}\cdot\mathrm{m}^{-\frac{1}{3}}$.
- ◇ Add a 1D weir on the Canal del Dique, downstream of Mahates (see figure 18.16 for the location). Set the Crest level of the weir to 1.5 mAD and the width to 120 m by selecting the weir, right-click $\rightarrow$ edit.
- ◇ Finally, validate the 1D model by right-click on *FM_model* $\rightarrow$ *Validate* to check if the model-setup is complete.

18.4.2 Expanding the model to a 1D2D model

In this part of the tutorial, the 2D grid will be added to the 1D model.

Adding a 2D grid

- ◇ Open the Grid tab and click on *Edit* to activate the grid editing mode.
- ◇ Click on *Geometry* tool and select *Draw/Edit* in the Polygons section. Draw a polygon around the 1D model. The polygon should exclude the boundaries at sea but widely include the rest of the model. To complete the polygon double-click at the last polygon point.
- ◇ Click on *Mesh* in the Tools section. Select the polygon. Note that *Rectangular in polygon* in the Create section is enabled. Click on *Rectangular in polygon*. Set up a grid with a grid block size of 2500x2500 and 0 degrees grid angle. Generate the grid by clicking on *OK*. Then unable the editing mode by clicking on *Edit* button and save the changes. The grid is generated correctly if it turns orange.

Refining the grid around the stream

- ◇ Activate again grid editing mode and click on *Import*. Make sure the file type in the search bar is set to *Polygon files (*.shp, *.pol)*, and choose the polygon *river_buffer.shp* from the data folder.
- ◇ Select the polygon by clicking on *Mesh* and then selecting the polygon that was imported.
- ◇ Choose *Refine in polygons* in the Operations section. Then select *Include intersected faces* and *Use cell mass center*. Set the *Maximum number of iterations* to 1 iteration for the grid refinement. Refine the grid by clicking on *OK*.
- ◇ Click on *Edit* to save the grid. Zoom in on the 1D river and note that the grid size around the stream has become smaller, see figure 18.16.

DRAFT

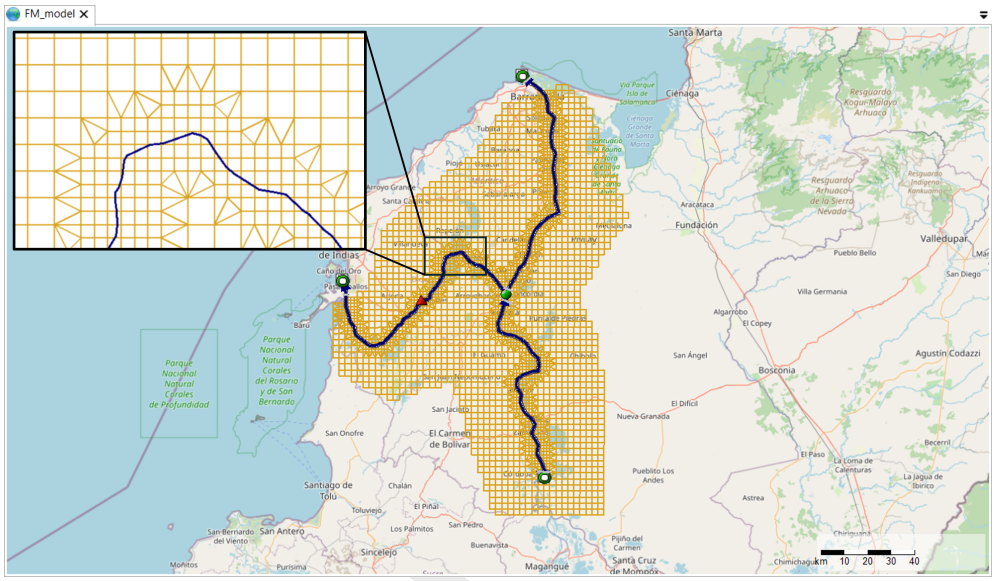


Figure 18.16: *The 2D grid with a close-up of the refinement around the Magdalena River.*

Delete the mesh underneath the channels

In larger rivers, adding the Digital Terrain Model (DTM) underneath a channel can cause an overestimation of the storage volume in the model. The river storage is accounted for in the 1D watercourse through the cross-sections and in the DTM, as the satellite measures the water level height. The storage volume above the water level at the time of the measurement up to the riverbank might therefore be included twice. To preclude this, it is advisable to delete the mesh beneath the 1D channel and utilize lateral 1D2D links to connect the 1D system with the 2D grid. It is recommended to use the Lateral links instead of Embedded 1-to-1 links, when river width exceeding twice the DTM resolution.

- ◇ Activate again the grid editing mode and import the polygon *river_buffer_for_clipping.shp*.
- ◇ Click on *Mesh* to open the mesh tools. Select the polygon and click on *Delete Mesh* in the Edit section. Set the *Delete option* to *Intersecting* and click on *OK*.
- ◇ Click on *Edit* to save the grid. Zoom in on the 1D river and note that the grid around the stream been deleted, as shown in figure 18.17.

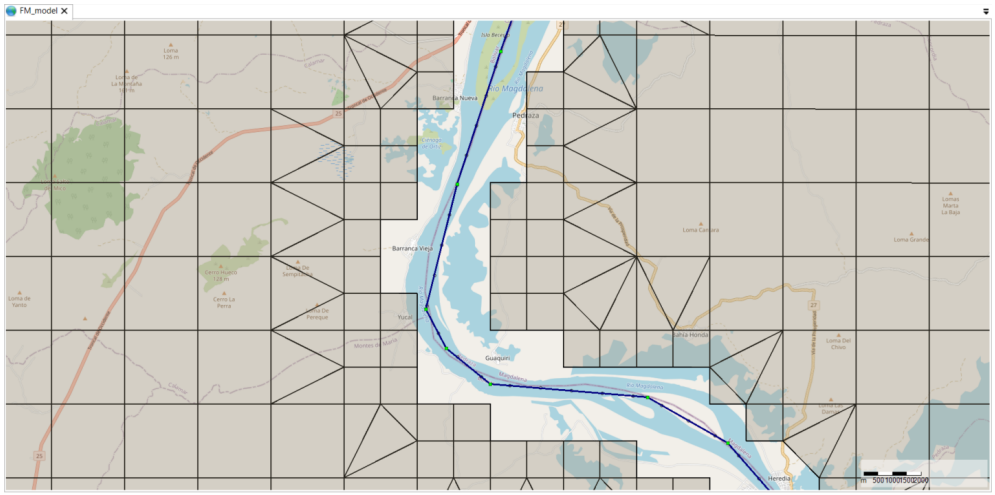


Figure 18.17: *Mesh cells at the 1D Magdalena River are removed.*

DRAFT

Interpolating the bedlevel on the grid to include a DEM

- ◇ Open the Spatial Operations tab and select the *Bed Level Layer* in the dropdown menu on the left. Choose *Bed Level* and click on *Add rasterfile samples*.
- ◇ Import the file *2D_DTM_Magdalena.tif*. Choose to import without transformation.
- ◇ Open the operations window on the right bottom and select the imported raster samples. Click on *Interpolate* → *Interpolate selected set* in the Spatial Operations tab set the interpolation method to *Averaging* and then *SimpleAveraging* with minimum sample points, Relative search cell size and Spatial Projection Type to 1, 1.01 and 2D respectively. Averaging is chosen in this example because each grid cell includes a lot of bedlevel samples.

Generate the 1D2D links

- ◇ Open the Map tab and open the dropdown menu in the *1D2D Links* section.
- ◇ Choose *Lateral* and click on *Generate links*.
- ◇ Draw a polygon around the model and end with a double-click. Set the *Search radius* to 5000 and click on OK.
- ◇ Zoom in to check if the 1D2D links are created correctly. Figure 18.18 shows an example of the grid refinement and the 1D2D links in one of the segments of the model.
- ◇ Zoom in on the 1D2D links that are connected to the boundary node in the South. Since this node includes the boundary condition implemented earlier, it is not possible to connect this node to the 2D grid. To delete the 1D2D links in this part of the model, select *1D2D links* in the Map Tree and activate the *Use focused layer* at the top of the Map Tree. Click on the selection tool and select the links that connect the boundary node and the first computational grid node with the 2D grid. When the links are selected, right-click → *Delete selection* (or press the delete button on your keyboard). By using the focused layer, it is only possible to select 1D2D links, making sure that you will not delete the wrong model elements.

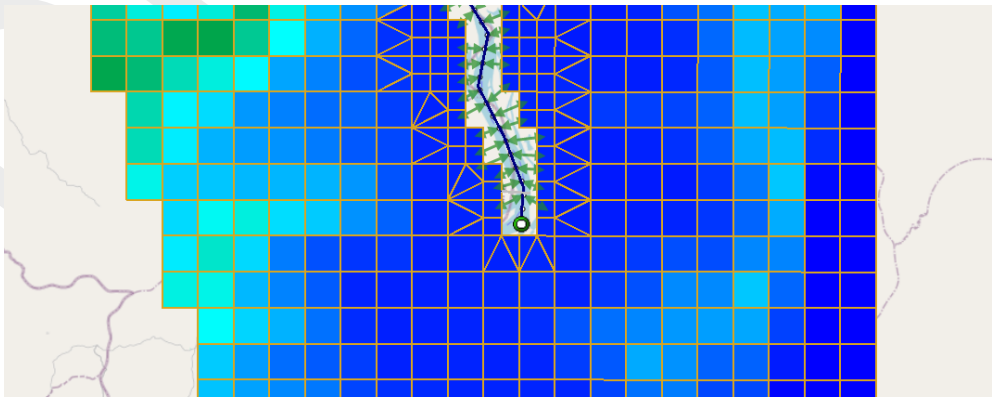


Figure 18.18: *Close-up of the grid refinement and 1D2D links in the South part of the Magdalena river.*

Run the model and inspect the flood

- ◇ Right-click on *FlowFM* in the project window and click on *Validate* to validate the model set-up. Then right-click again on on *FlowFM* and choose *Run Model* to start the simulation.
- ◇ Inspect the 2D waterdepth output by selecting the *FlowFM* → *Output* → *Map File 2D* → *Water depth at pressure points* in the Map Tree. Adjust the legend settings of the output by righ-click on the output parameter, choosing *Legend Properties* and setting the No Data value to 0.
- ◇ Drag the bar in the *Time Navigator* section to see the flood developing over time, see

figure 18.19.

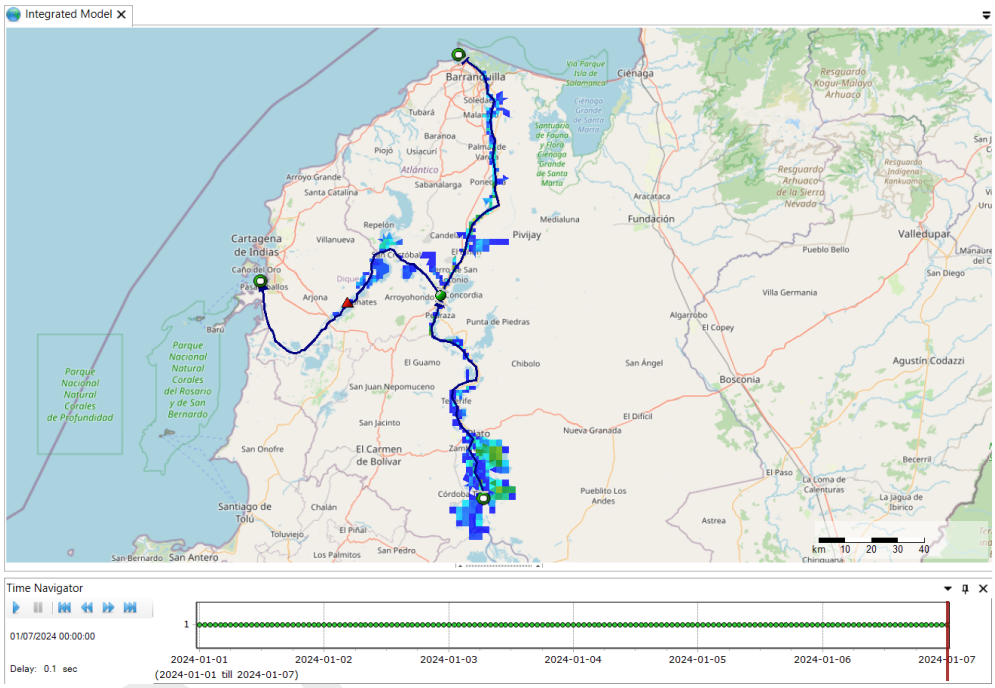


Figure 18.19: *Flood depth at the last time step in the Magdalena river.*

18.5 Tutorial 4: Real Time Control in a river model

The Real Time Control (RTC) module is used to model feedback control of hydraulic structures. A hydrodynamic 1D or 1D2D model can be extended with the RTC module in the format of an integrated model. Feedback control in hydraulic models can be represented with the following elements:

- ◇ Condition: a desired (range of) water level or discharge in the river
- ◇ Operating rules: the algorithm that is designed to maintain the right condition with certain structure settings
- ◇ Input data location (control parameter): location where water level or discharge values are measures (real time)
- ◇ Output data location (controlled parameter): the hydraulic structure that is controlled like a pump or a weir gate height

There are different types of control systems possible in the RTC module.

- ◇ Lookup table rule: used to operate a structure as a function of a control parameter such as water level or discharge at an observation point
- ◇ Time rule: used to operate a structure as a function of time (for some structures this can be implemented without using the RTC module)
- ◇ Proportional Integral Derivative (PID) rule: used to operate a structure in a control loop feedback mechanism to maintain a certain water level or discharge value in the system continuously
- ◇ Interval rule: used to operate a structure so that a hydraulic parameter is maintained. The structure is controlled in fixed timesteps and/or with a fixed change within the defined capacity of the structure.
- ◇ Relative from time/value rule: used to operate a structure as a function of time relative to the moment that the rule is activated.

◇ Invertor rule: used to model the activation of hydraulic bypasses from certain water levels in a river network by connecting lateral sources.

In this tutorial we will set up a PID controller. The tutorial is divided into the following steps:

◇ Importing the 1D model
◇ Implementing the PID rule
◇ Inspecting the response of the structure

18.5.1 Import the 1D2D model of the Magdalena river

◇ Open the Delft3D Flexible Mesh Suite program and choose *Dimr import*. Select the *dimr_config.xml* file in the *data* folder → *input model* folder. The model of the Magdalena river is loaded.
◇ Click on *Add New WMS Layer* at the top of the Map Tree and add Open Street Map.
◇ Right-click on *FM_model* in the project window and select *Run Model*.
◇ After running the model, select the *FM_model* → *Output* → *Map File 2D* → *Water depth at pressure points* in the Map Tree. Use the Time Navigator in the bottom of the screen to inspect how the flood develops over time. Note that upstream of the weir, more flooding occurs than downstream of the weir.

18.5.2 Implement a PID Rule in the weir

The PID controller is the most complex control system in the D-RTC module. It continuously maintains a specific water level or discharge in the hydraulic model using the PID rule. In this tutorial, we expand the weir in the 1D2D model of the Magdalena River with a PID rule to reduce upstream flood risk. The control system measures water levels upstream and downstream of the weir, maintaining a safe upstream water level by adjusting the weir's crest level. When the upstream water level is too high, the crest level is lowered, and when it is too low, the crest level is raised. To prevent downstream flooding, the crest level can only be adjusted if the downstream water level does not exceed a critical threshold.

To control the weir with a PID rule, the D-RTC module must be added to the integrated model.

◇ Select Integrated Model in the Project Window and click on *Home* → *New Model* → *Real-Time Control Model*. When correctly executed, the D-RTC model appears below the D-Flow FM model in the Integrated Model section of the Project Window.
◇ Open *Integrated model* → *Real_Time_Control* in the Project Window. Right-click on *Control Groups* and click on *Add New Control Group*. Select *PID Rule with condition*.
◇ A window opens where the PID rule settings can be defined.

As previously described, the PID rule in this exercise will control the weir's crest level based on the water levels both upstream and downstream of the weir.

◇ Move to the Integrated model in the Map View, and select the *observation point* button in 1D Network section in the Map tab. Insert an observation point upstream and downstream of the weir, at the locations shown in figure 18.20.
◇ Name the upstream observation point *ObservationPoint_1D_1* and the downstream observation point *ObservationPoint_1D_2*. Renaming can be done via the properties window or the multiple data editor of the observation points (Go to Map Tree and click on *FM_model* → *1D* → *network* and double-click on *Observation Points*).

DRAFT

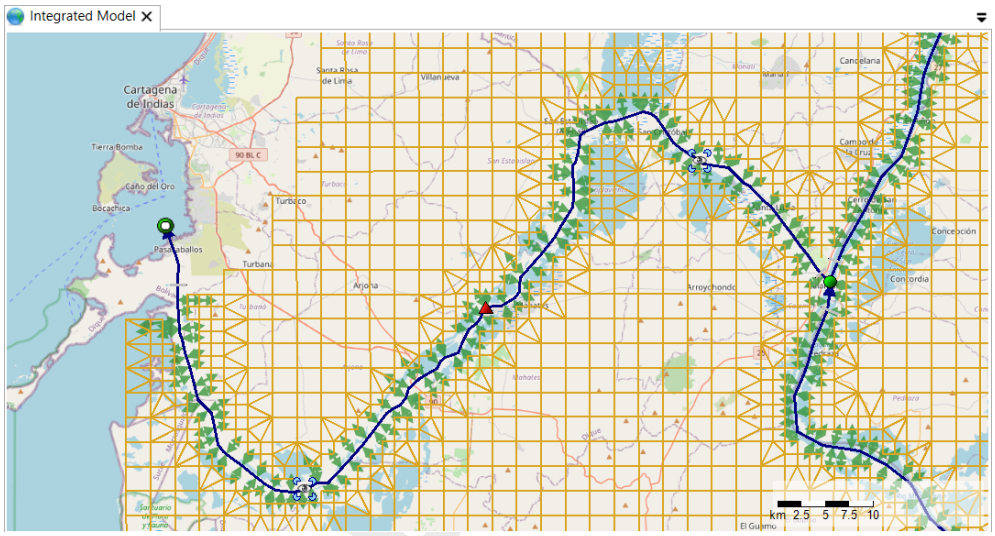


Figure 18.20: *Placing observation points upstream and downstream of the weir.*

◇ In the Project Window open *Integrated model* → Real_Time_Control → Control Groups and double-click *Control Group 1*. The Control group opens in a new window.
◇ Drag the boxes of the PID controller to the middle of the screen and arrange them as shown in 18.21

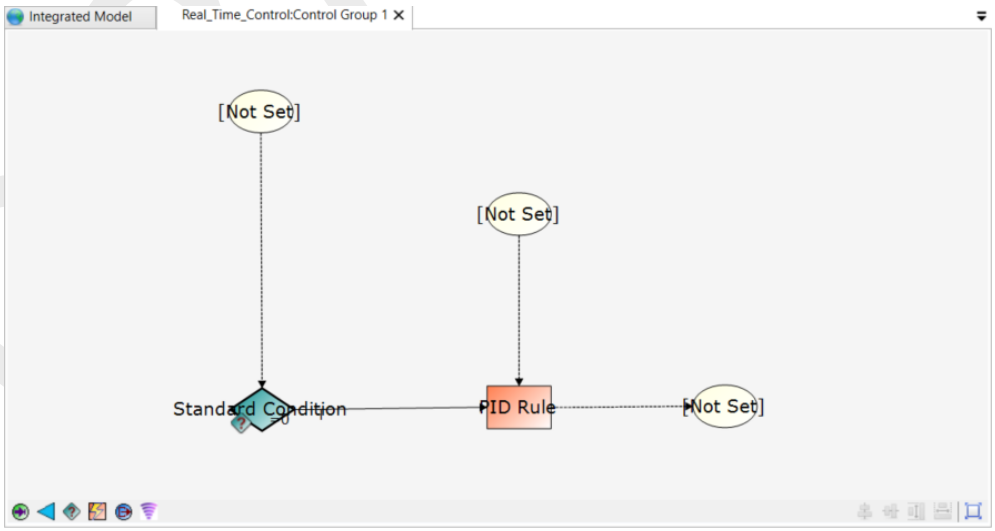


Figure 18.21: *Control group of the weir.*

◇ Note that the round boxes represent the input and output locations. Start by right-clicking on the left input location and select the Water level (op) of the observation point *ObservationPoint_1D_2* as input parameter. This represents the water level downstream of the weir. Remark that the input location is connected to the standard condition.
◇ Click on the standard condition and define the parameters in the Properties window. Set the *Operation* to "<" and the *Value* to 1.4 mAD. These settings ensure that the rule applied to the weir activates only when the downstream water level is below 1.4 mAD, preventing downstream floods.
◇ Right-click on the second input location and select the Water level (op) of the observation point *ObservationPoint_1D_1* as input parameter. This represents the water level upstream of the weir. This input location is connected to the PID rule and serves as the data point that the PID rule aims to maintain at a specific set point.

- ◇ Right-click on the output location and select the Crest level (s) of the weir *Weir_1D_1* as output parameter of the PID Rule. This means the weir's crest level is the controlled parameter by the PID rule to maintain the water level at the observation point *ObservationPoint_1D_1*, when the water level in the observation point *ObservationPoint_1D_2* is below 1.4 mAD.
- ◇ Click on the PID rule to view the settings in the *Properties window*. Set the *Constant set point* at 4.8, meaning that the goal of the PID rule is to maintain the water level in the observation point *ObservationPoint_1D_1* at this level. Then set the *Limits* parameters to a *Minimum* of 0.6 mAD, a *maximum* of 1.5 mAD and a *Maximum speed* of 0.0001. This means that the crest level of weir can deviate between 0.6 mAD and 1.5 mAD, and is able to increase and decrease with 0.1 mmsec (which is in practice very low but choosen for this example so that the effect of the parameter will be visible in the results). Finally, set the *Kp* value to 2, the *Ki* value to 0.1 and the *Kd* value to 0. These parameters are called the *Gain factors* and determine how the the crest level of the weir changes ('gains') each timestep. The Kp, Ki and Kd values determine the gain on the error, on the time integral of the error and on the derivate of the error respectively. The error is the difference between the measured water level in the observation point *ObservationPoint_1D_1* and the setpoint of the PID rule. See figure 18.22 to check if the PID controller is set-up correctly.
- ◇ Right-click on *Integrated Model* in the Project Window and select *Run Model*.

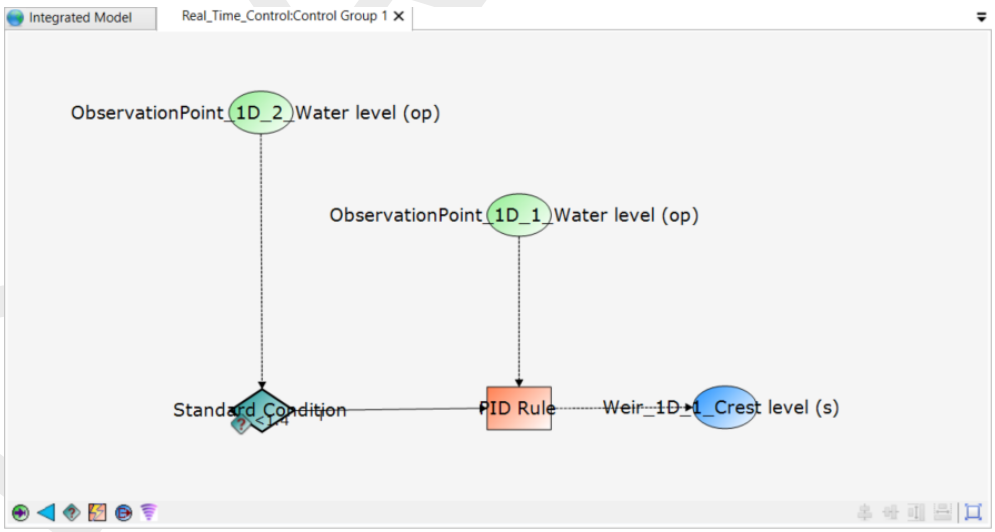


Figure 18.22: *Control group of the weir.*

18.5.3 Inspecting the response of the structure

To check if the weir with a PID controller works, you can plot its behavior and the water levels at observation points.

- ◇ Select the weir.
- ◇ Click on *Query Time Series* in the *Tools* section of the Map tab. A list of Time Dependent Spatial Data for the selected weir location appears. Select *Crest level of weir (weirgen_crest_level)*. A graph and a table open in a new window, showing the changes in the weir's crest level over time.
- ◇ Open the *Integrated Model* window again, without closing the window with the graph.
- ◇ Select both the upstream and downstream observation points by holding the *ctrl* key during the selection process. Click again on *Query Time Series* and choose *water level (waterlevel)*. The water levels are plotted in the same graph as the crest level of the weir. See figure 18.23. Note that the crest level of the weir remains high as long as the downstream water level is below the acceptable threshold and the upstream water level is below the

PID controller's set point. When the upstream water level reaches the set point, the crest level of the weir drops to its lowest possible value, reducing the upstream water level and increasing the downstream water level.

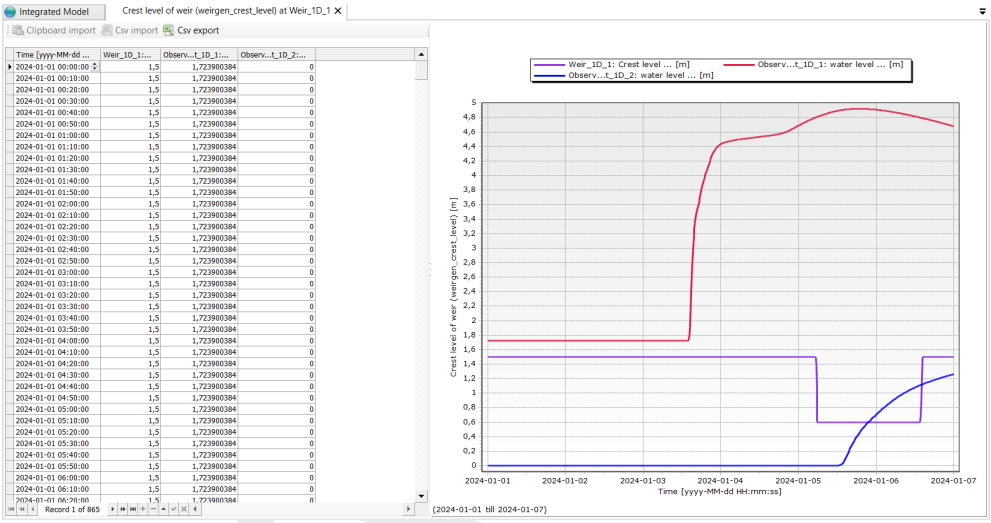


Figure 18.23: *Crest level of the weir changing as a result of the upstream and downstream water level.*

To better understand the impact of PID controller parameters, change the Ki value to 0.001 and then to 2, and compare the results with the graph above. Note that a low Ki value results in a slower response of the controlled structure, making the system more stable and smooth. However, if the Ki value is too low, the response may be too slow to be effective. Increasing the Ki value leads to a faster response but may cause overshooting or oscillations in the structure's response.

DRAFT

References

Andreas, E. L., H. T. W., A. A. Grachev, P. O. G. Persson, C. W. Fairall, P. S. Guest and R. E. Jordan, 2010. "Parametrizing turbulent exchange over summer sea ice and the marginal ice zone." *Quarterly Journal of the Royal Meteorological Society* 136: 927–943. DOI: 10.1002/qj.618.

Andrews, D. G. and M. E. McIntyre, 1978. "An exact theory of nonlinear waves on a Lagrangian-mean flow." *Journal of Fluid Mechanics* 89 (4): 609–646.

Baptist, M. J., 2005. *Modelling floodplain biogeomorphology*. Ph.D. thesis, Delft University of Technology.

Bijker, E. W., 1967. *Some considerations about scales for coastal models with moveable bed*. Tech. Rep. 50, WL | Delft Hydraulics, Delft, The Netherlands.

Bos, 1989. *Discharge Measuring Structures*. International Institute for Land Reclamation and Improvement/ILRI, Wageningen, The Netherlands.

Bos, W. de and C. Bijkerk, 1963. "Een nieuw monogram voor het berekenen van waterlopen." *Cultuurtechnisch Tijdeschrift* 3 (4): 149–155.

Brückner, M. Z. M., C. Schwarz, W. M. van Dijk, M. van Oorschot, H. Douma and M. G. Kleinhans, 2019. "Salt Marsh Establishment and Eco-Engineering Effects in Dynamic Estuaries Determined by Species Growth and Mortality." *Journal of Geophysical Research: Earth Surface* 124 (12): 2962-2986. DOI: https://doi.org/10.1029/2019JF005092, URL https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2019JF005092.

Charnock, H., 1955. "Wind-stress on a water surface." *Q. J. Royal Meteorol. Soc.* 81: 639–640.

Christoffersen, J. B. and I. G. Jonsson, 1985. "Bed friction and dissipation in a combined current and wave motion." *Ocean Engineering* 12 (5): 387–423.

Colebrook, C., 1939. "Turbulent flow in pipes, with particular reference to the transition region between the smooth and rough pipe laws." *Journal of the Institution of Civil Engineers* 11 (4): 133–156.

Colebrook, C. and C. White, 1937. "Experiments with Fluid Friction in Roughened Pipes." *Proceedings of the Royal Society of London, Series A, Mathematical and Physical Sciences* 161 (906): 367–381.

Cunge, J. A., F. M. Holly and A. Verwey, 1980. *Practical aspects of computational river hydraulics*. Boston; London; Melbourne; Boston; London; Melbourne: Pitman. Includes index.

D-RR_UM, 2019. *SOBEK 3 / D-Rainfall Runoff User Manual*. Deltares, Delft. Version: 3.6.1.

Davies, A. G., R. L. Soulsby and H. L. King, 1988. "A numerical model of the combined wave and current bottom boundary layer." *Journal of Geophysical Research* 93 (C1): 491–508.

Deltares, 2026a. *D-Flow Flexible Mesh Technical Reference Manual*. Deltares, Delft.

Deltares, 2026b. *D-Flow Flexible Mesh User Manual; Released for Delft3D FM Suite 1D2D*. Deltares, Delft.

Deltares, 2026c. *D-Flow FM Hydro- and Morphodynamics User Manual; Released for Delft3D FM Suite 2D3D*. Deltares, Delft.

Deltares, 2026d. *D-Morphology 1D/2D/3D User Manual*. Deltares, Delft.

DRAFT

Deltares, 2026e. *D-Real Time Control User Manual; Released for Delft3D FM and SOBEK*. Deltares, Delft, 1.4 ed.

Deltares, 2026f. *D-Waves User Manual; Simulation of short-crested waves with SWAN*. Deltares, Delft, 1.2 ed.

Deltares, 2026g. *Delft3D-FLOW User Manual; Simulation of multi-dimensional hydrodynamic flows and transport phenomena, including sediments*. Deltares, Delft, 4.05 ed.

Deltares, 2026h. *DIMR Technical Reference Manual*. Deltares, Delft.

Dingemans, M. W., 1997. *Water Wave Propagation over Uneven Bottoms, Vol. 1 and 2*. Advanced Series on Ocean Engineering, Vol. 13. World Scientific, London.

Dingemans, M. W., A. C. Radder and H. J. de Vriend, 1987. "Computation of the driving forces of wave-induced currents." *Coastal Engineering* 11: 539–563.

Eckart, C., 1958. "Properties of water, Part II. The equation of state of water and sea water at low temperatures and pressures." *American Journal of Science* 256: 225–240.

ECMWF, 2023. *IFS DOCUMENTATION – Cy48r1, Operational implementation 27 June 2023, PART IV: PHYSICAL PROCESSES*. Tech. rep., ECMWF.

Engelund, F. and E. Hansen, 1967. *A monograph on Sediment Transport in Alluvial Streams*. Teknisk Forlag, Copenhagen.

Fofonoff, N. P. and R. C. Millard Jr., 1983. *Algorithms for the computation of fundamental properties of seawater*. UNESCO Technical Papers in Marine Sciences 44, UNESCO, Paris, France.

Fredsøe, J., 1982. "Shape and dimensions of stationary dunes in rivers." *Journal of the Hydraulics Division, ASCE* 108: 932–947.

Fredsøe, J., 1984. "Turbulent boundary layer in wave-current interaction." *Journal of Hydraulic Engineering* 110: 1103–1120.

Garratt, J. R., 1977. "Review of Drag Coefficients over Oceans and Continents." *Review Articles in Monthly Weather Review* 105 (7): 915–929.

Grant, W. D. and O. S. Madsen, 1979. "Combined wave and current interaction with a rough bottom." *Journal of Geophysical Research* 84 (C1): 1797–1808.

Groeneweg, J., 1999. *Wave-current interactions in a generalized Lagrangian mean formulation*. Delft University of Technology, Delft, The Netherlands. Ph.D. thesis.

Groeneweg, J. and G. Klopman, 1998. "Changes of the mean velocity profiles in the combined wave-current motion in a GLM formulation." *Journal of Fluid Mechanics* 370: 271–296.

Hersbach, H., 2011. "Sea Surface Roughness and Drag Coefficient as Functions of Neutral Wind Speed." *Journal of Physical Oceanography* 41 (1): 247–251.

Hibler III, W. D., 1979. "A dynamic thermokdynamic sea ice model." *Journal of Physical Oceanography* 9 (4): 815–846.

Horton, R. E., 1933. "The Rôle of infiltration in the hydrologic cycle." *Eos, Transactions American Geophysical Union* 14 (1): 446-460. DOI: 10.1029/TR014i001p00446, URL https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/TR014i001p00446.

Hunke, E. C. and J. K. Dukowicz, 1997. "An elastic-ciscous-plastic model for sea ice." *Journal of Physical Oceanography* 27 (9): 1849–1867.

DRAFT

Hutton, E. W. H., M. D. Piper and G. E. Tucker, 2020. "The Basic Model Interface 2.0: A standard interface for coupling numerical models in the geosciences." *Journal of Open Source Software* 5 (51): 2317. DOI: 10.21105/joss.02317, URL https://doi.org/10.21105/joss.02317.

Huynh-Thanh, S. and A. Temperville, 1991. "A numerical model of the rough turbulent boundary layer in combined wave and current interaction." In R. L. Soulsby and R. Bettes, eds., *Sand transport in rivers, estuaries and the sea*, pages 93–100. Balkema Rotterdam.

Hwang, P., 2005a. "Comparison of the ocean surface wind stress computed with different parameterization functions of the drag coefficient." *J. Oceanogr.* 61: 91–107.

Hwang, P., 2005b. "Drag coefficient, dynamic roughness and reference wind speed." *J. Oceanogr.* 61: 399–413.

Joyce, B. R., W. J. Pringle, D. Wirasaet, J. J. Westerink, A. J. van der Westerhuysen, R. Grumbine and J. Feyen, 2019. "High resolution modeling of western Alaskan tides and storm surge under varying sea ice conditions." *Ocean Modelling* 141. DOI: 10.1016/j.ocemod.2019.101421.

Kalkwijk, J. P. T. and R. Booij, 1986. "Adaptation of secondary flow in nearly horizontal flow." *Journal of Hydraulic Research* 24 (1): 19–37.

Karypis, G., 2013. *METIS - A Software Package for Partitioning Unstructured Graph, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices, Version 5.1.0*. Tech. rep., Department of Computer Science and Engineering, University of Minnesota.

Klopstra, D., H. J. Barneveld and J. M. Van Noortwijk, 1996. *Analytisch model hydraulische ruwheid van overstroomde moerasvegetatie*. Tech. Rep. PR051, HKV consultants, Lelystad, The Netherlands. Commissioned by Rijkswaterstaat/RIZA, The Netherlands.

Klopstra, D., H. J. Barneveld, J. M. Van Noortwijk and E. H. van Velzen, 1997. "Analytical model for hydraulic roughness of submerged vegetation." In *The 27th IAHR Congress, San Francisco, 1997; Proceedings of Theme A, Managing Water: Coping with Scarcity and Abundance*, pages 775–780. American Society of Civil Engineers (ASCE), New York.

Knaap, F. Van der, 2000. *Breach growth as a function of time*. Tech. Rep. Q2655, WL | Delft Hydraulics, Delft, The Netherlands. Memos 2 and 3 of May 21 and September 5 respectively.

Love, A. E. H., 1927. *A Treatise on the Mathematical Theory of Elasticity*. Cambridge University Press, 4th ed.

Lüpkes, C. and G. Birnbaum, 2005. "Surface drag in the arctic marginal sea-ice zone: a comparison of different parameterisation concepts." *Boundary-Layer Meteorology* 117: 179-211. DOI: 10.1007/s10546-005-1445-8.

Madec, G., M. Bell, R. Bourdallé-Badie, J. Chanut, E. Clementi, A. Coward, M. Drudi, I. Epicoco, C. Ethé, D. Iovino, D. Lea, C. Lévy, N. Martin, S. Masson, P. Mathiot, F. Mele, S. Mocavero, A. Moulin, S. Müller, G. Nurser, G. Samson and D. Storkey, 2022. *NEMO ocean engine*. Scientific Notes. Institut Pierre-Simon Laplace, Paris, France, v4.2 ed.

Malone, T. and A. D. Parr, 2008. *Bend Losses in Rectangular Culverts*. Tech. Rep. K-TRAN: KU-05-5, Kansas Department of Transportation and Kansas State University.

Manning, R., 1891. "On the flow of water in open channels and pipes." *Transactions of the Institution of Civil Engineers of Ireland* 20: 161–207.

DRAFT

Mellor, G. L. and L. Kantha, 1989. “An ice-ocean coupled model.” *Journal of Geophysical Research* 94 (8): 10937–10954.

Meyer-Peter, E. and R. Müller, 1948. “Formulas for bed load transport.” In *Proceedings of the 2nd Congress IAHR, Stockholm*, vol. 2, pages 39–64.

Milgram, J., 1983. “Mean flow in round bubble plumes.” *Journal of Fluid Mechanics* 133: 345–376.

Millero, F. J. and A. Poisson, 1981. “International one-atmosphere equation of state of sea water.” *Deep-Sea Research* 28A (6): 625–629.

Myrhaug, D. and O. H. Slaattelid, 1990. “A rational approach to wave-current friction coefficients for rough, smooth and transitional turbulent flow.” *Coastal Engineering* 14: 265–293.

O’ Connor, B. A. and D. Yoo, 1988. “Mean bed friction of combined wave-current flow.” *Coastal Engineering* 12: 1–21.

Oorschot, M. v., M. Kleinhans, G. Geerling and H. Middelkoop, 2016. “Distinct patterns of interaction between vegetation and morphodynamics.” *Earth Surface Processes and Landforms* 41 (6): 791-808. DOI: https://doi.org/10.1002/esp.3864, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/esp.3864.

Peckham, S. D., E. W. H. Hutton and B. Norris, 2013. “A component-based approach to integrated modeling in the geosciences: The design of CSDMS.” *Computers & Geosciences* 53: 3–12.

Preissmann, A. and J. A. Cunge, 1961. “Calcul des intumescences sur machines électroniques.” In *Proceedings of 9th Congress of International Association for Hydraulic Research (IAHR), Dubrovnik, Yugoslavia, 1961*, pages 656–664. IAHR.

Rijn, L. C. van, 1984. “Sediment transport, Part III: bed form and alluvial roughness.” *Journal of Hydraulic Engineering* 110 (12): 1733–1754.

Rijn, L. C. van, 2007. “Unified View of Sediment Transport by Currents and Waves. I: Initiation of Motion, Bed Roughness, and Bed-Load Transport.” *Journal of Hydraulic Engineering* 133 (6): 649–667.

Rijn, L. C. van, D. R. Walstra and M. v. Ormondt, 2004. *Description of TRANSPOR2004 and implementation in Delft3D-ONLINE*. Tech. Rep. Z3748.10, WL | Delft Hydraulics, Delft, The Netherlands.

Ris, R. C., 1997. *Spectral Modelling of Wind Waves in Coastal Areas*. Communications on Hydraulic and Geotechnical Engineering, report 97-4. Delft University of Technology, Delft, The Netherlands. Ph.D. thesis.

Rodi, W., 1984. “Turbulence models and their application in Hydraulics, State-of-the-art paper article sur l’etat de connaissance.” *IAHR* Paper presented by the IAHR-Section on Fundamentals of Division II: Experimental and Mathematical Fluid Dynamics, The Netherlands.

Saint-Venant, A. J. C. Barré de, 1871. “Théorie du mouvement non permanent des eaux, avec application aux crues des rivières et à l’introduction des marées dans leur lit.” *Comptes Rendus des séances de l’Académie des Sciences* 70(2): 237–240. (in French).

Schrama, E., 2007. “Tides. Lecture Notes AE4-876.”

Semtner Jr., A. J., 1976. “Numerical simulation of the Arctic Ocean circulation.” *Journal of Physical Oceanography* 6 (4): 409–425.

DRAFT

Smith, S. D. and E. G. Banke, 1975. “Variation of the sea surface drag coefficient with wind speed.” *Quarterly Joournal of the Royal Meteorological Society* 101: 665–673.

Soulsby, R. L., A. G. Davies, J. Fredsøe, D. A. Huntley, I. G. Jonnson, D. Myrhaug, R. R. Simons, A. Temperville and T. J. Zitman, 1993a. “Bed shear stresses due to combined waves and currents.” In *Abstracts-in-depth of the Marine Science and Technology G8-M overall workshop, Grenoble.*, pages 2.1-1/2.1–4. EC.

Soulsby, R. L., L. Hamm, G. Klopman, D. Myrhaug, R. R. Simons and G. P. Thomas, 1993b. “Wave-current interaction within and outside the bottom boundary layer.” *Coastal Engineering* 21: 41–69.

Strickler, A., 1923. *Beiträge zur Frage der Geschwindigheits-formel und der Rauhigkeitszahlen fur Ströme, Kanäle und geschlossene Leitungen.* Tech. Rep. Mitteilungen des Eidgenössischen Amtes für Wasserwirtshaft, No. 16. (Some contributions to the problem of the velocity formula and roughnessfactors for rivers, canals, and closed conduits.).

Swart, 1974. *Offshore sediment transport and equilibrium beach profiles*. Ph.D. thesis, Delft University of Technology, Delft, The Netherlands. Delft Hydraulics Publ. 131.

Uittenbogaard, R. E., J. A. T. M. van Kester and G. S. Stelling, 1992. *Implementation of three turbulence models in 3D-TRISULA for rectangular grids*. Tech. Rep. Z81, WL | Delft Hydraulics, Delft, The Netherlands.

Velzen, E. H. van, P. Jesse, P. Cornelissen and H. Coops, 2003. *Stromingsweerstand vegetatie in uiterwaarden*. Tech. Rep. 2003.029, Rijkswaterstaat/RIZA.

Verheij, H., 2002. *Modification breach growth model in HIS-OM*. Tech. Rep. Q3299, WL | Delft Hydraulics, Delft, The Netherlands. (in Dutch).

Wang, J., Q. Liu, M. Jin, M. Ikeda and F. J. Saucier, 2005. “A Coupled Ice-Ocean Model in the Pan-Arctic and North Atlantic Ocean: Simulation of Seasonal Cycles.” *Journal of Oceanography* 61: 213–233.

Wüest, A. and A. Lorke, 2003. “Small-Scale Hydrodynamics in Lakes.” *Annual Review of Fluid Mechanics* 35 (1): 373–412.

DRAFT

A The master definition file

A.1 Overview of keywords generated by GUI

The mdu-file contains the key information of the flow model. Besides the names of the relevant user specified files, such as the grid file and the external forcings file, the values of various model parameters should be specified in the MDU-file. In Table A.1 the model parameter settings are given as well as the associated default setting for these parameters that are generated by the Graphical User Interface of D-Flow FM. If a model parameter has a unit, then this unit will also be given in [ ] in the column of Description in this table.

The basename of the mdu-file, without `.mdu`, is also used as the model identification string, and is often denoted as *mdu_name* throughout this User Manual. It is equivalent to Delft3D-FLOW's *runid* concept.

Table A.1: *Standard MDU-file with default settings.*

Keyword	Default setting	Description
`[general]`		
`Program`	`D-Flow FM`	Program.
`Version`	`1.2.60 ↩ .64623M`	Version number of computational kernel.
`fileType`	`modelDef`	File type. Do not edit this.
`fileVersion`	`1.09`	File version. Do not edit this.
`GuiVersion`	`1.5.4.45545`	Version number of GUI.
`AutoStart`	`0`	Autostart simulation after loading MDU or not (0=no, 1=autostart, 2=autostartstop).
`PathsRelativeToParent`	`0`	Whether or not (1/0) to resolve file names (e.g. inside the *.ext file) relative to their direct parent, instead of to the toplevel MDU working dir.
`[geometry]`		
`NetFile`		<*_net.nc>. See chapter B.
`DryPointsFile`		Dry points file <*.xyz>, third column dummy z values, or polygon file <*.pol>.
`GridEnclosureFile`		Enclosure file <*.pol> to clip outer parts from the grid.
`StructureFile`		File <*.ini> containing list of hydraulic structures. See section C.13.
`GulliesFile`		Polyline file <*_gul.pliz>, containing lowest bed level along talweg x, y, z level.
`RoofsFile`		Polyline file <*_roof.pliz>, containing roofgutter heights x, y, z level.
`IniFieldFile`		Initial and parameter field file <*.ini>. See section D.2.
`WaterLevIniFile`		Initial water levels sample file <*.xyz>.
`LandBoundaryFile`		Only for plotting.
`ThinDamFile`		<*_thd.pli>, Polyline(s) for tracing thin dams.
`FixedWeirFile`		<*_fxw.pliz>, Polyline(s) x, y, z, z = fixed weir top levels (formerly fixed weir).
`PillarFile`		<*_pillar.pliz>, Polyline file containing four colums with x, y, diameter and Cd coefficient for bridge pillars.
`UseCaching`	`1`	Use caching for geometrical/network-related items (0: no, 1: yes) (section C.20).
`FrictFile`		Location of the files with roughness data for 1D. See section C.16.

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
CrossDefFile		Cross section definitions for all cross section shapes. See section C.17.
CrossLocFile		Location definitions of the cross sections on a 1D network. See section C.17.
StorageNodeFile		File containing the specification of storage nodes and/or manholes to add extra storage to 1D models. See section C.18.
1D2DLinkFile		File containing the custom parameterization of 1D-2D links. See section C.19.
AllowBndAtBifurcation	0	Allow 1d boundary node when connecting branch leads to bifurcation (1: yes, 0: no).
ProflocFile		<*_proflocation.xyz>) x, y, z, z = profile refnumber.
ProfdefFile		<*_profdefinition.def>) definition for all profile nrs.
ProfdefxyzFile		<*_profdefinition.def>) definition for all profile nrs.
ManholeFile		File containing manholes (e.g. <*.dat>).
PartitionFile		<*_part.pol>, polyline(s) x, y.
WaterLevIni	0.	Initial water level [m AD].
Bedlevuni	-5.	Uniform bed level [m AD], (only if BedlevType>=3), used at missing z values in netfile.
Bedslope	0.	Bed slope inclination [-], sets zk = bedlevuni + x*bedslope and sets zbndz = xbndz*bedslope.
BedlevType	3	1: at cell center (tiles xz, yz, bl, bob=max(bl)), 2: at face (tiles xu, yu, blu, bob=blu), 3: at face (using mean node values), 4: at face (using min node values), 5: at face (using max node values), 6: with bl based on node values.
Blmeanbelow	-999.	if not -999d0, below this level [m] the cell centre bedlevel is the mean of surrouding netnodes.
Blminabove	-999.	if not -999d0, above this level [m] the cell centre bedlevel is the min of surrouding netnodes.
AngLat	0.	Angle of latitude S-N [deg], 0=on Equator (and thus no Coriolis force). Only required for Coriolis on Cartesian grids and for heat flux modelling.
AngLon	0.	Angle of longitude E-W [deg], 0=Greenwich Mean Time. Only required for heat flux modelling.
Conveyance2D	-1	-1:R=HU, 0:R=H, 1:R=A/P, 2:K=analytic-1D conv, 3:K=analytic-2D conv.
Nonlin1D	1	Non-linear 1D volumes, applicable for models with closed cross sections. 1=treat closed sections as partially open by using a Preissmann slot, 2=Nested Newton approach, 3=Partial Nested Newton approach. (For a description of these methods, see (Deltares, 2026a, **??**sec:nestedNewton).
Nonlin2D	0	Non-linear 2D volumes, only i.c.m. BedlevType=3 and Conveyance2D>=1.
slotw1D	0.001	Minimum slotwidth 1D [m].
slotw2D	0.001	Minimum slotwidth 2D [m].
Uniformwidth1D	2.	Uniform width for 1D profiles and 1d2d internal links [m].
Uniformheight1D	3.	Uniform height for 1D profiles and 1d2d internal links [m].

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
Uniformwidth1Dstreetinlets	0.2	Uniform width for street inlets [m].
Uniformheight1Dstreetinlets	0.1	Uniform height for street inlets [m].
Uniformtyp1Dstreetinlets	-2	Uniform cross section type for street inlets (1: circle, 2: rectangle, -2: closed rectangle).
Uniformwidth1Droofgutterpipes	0.1	Uniform width for roof gutter pipes [m].
Uniformheight1Droofgutterpipes	0.1	Uniform height for roof gutter pipes [m].
Uniformtyp1Droofgutterpipes	-2	Uniform cross section type for type roof gutter pipes (1: circle, 2: rectangle, -2: closed rectangle).
Sillheightmin	0.0	Fixed weir only active if both ground heights are larger than this value [m AD].
Makeorthocenters	0	(1: yes, 0: no) switch from circumcentres to orthocentres in geominit.
Dcenterinside	1.	Limit cell center [-]; 1.0:in cell <-> 0.0:on c/g.
circumcenterMethod	allNetlinks↩ Loop	Circumcenter computation method (internalNetlinksEdge = iterate each edge, internalNetlinksLoop = iterate each loop, allNetlinksLoop = iterate each edge and each loop).
circumcenterTolerance	1d-15	Tolerance for convergence of circumcenter method [m].
Bamin	1E-06	Minimum grid cell area [m^2], i.c.m. cutcells.
OpenBoundaryTolerance	3.	Search tolerance factor between boundary polyline and grid cells. [Unit: in cell size units (i.e., not meters)].
RenumberFlowNodes	1	Renumber the flow nodes (1: yes, 0: no).
Kmx	0	Number of vertical layers. NB. If keyword zLayerGrowthFactor is used, then number of layers is determined by D-Flow FM [-].
Layertype	1	1 = sigma-layers, 2 = z- or z-sigma-layers.
Numtopsig	0	Number of sigma-layers on top of z-layers in case of z-sigma-layers.
zLayerGrowthFactor	1.	Growth factor of z-Layer thickness starting below the level specified by Dztopuniabovez till the bed [-].
Keepzlayeringatbed	2	Z layering at bed: 0=original bed level, 1=adapted bed level, 2= Ztbml approach of Delft3D-FLOW.
dxmin1D	0.001	Minimum 1D link length [m].
dxDoubleAt1DEndNodes	true	Whether a 1D grid cell at the end of a network has to be extended with $0.5\Delta x$.
ChangeVelocityAtStructures	false	Ignore structure dimensions for the velocity at hydraulic structures, when calculating the surrounding cell centered flow velocities.

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
ChangeStructureDimensions	true	Change the structure dimensions in case these are inconsistent with the channel dimensions. ◇ weirs, orifices, general structures: 1. In case the crest width exceeds the surface width, the crest width is set to the surface width; 2. In case the crest level is lower than the bed level, the crest level is set to the bed level. ◇ bridges: 1. In case the crest width exceeds the surface width, the crest width is set to the surface width; 2. In case the flow area of the bridge exceeds the upstream flow area the flow area of the bridge is set to the upstream flow area. ◇ universal weirs: only the crest level is checked and changed. **NOTE: It is strongly advised not to change this parameter (true). Since turning this option off can lead to instabilities and unrealistic results.**
[volumeTables]		
useVolumeTables	0	Use volume tables for 1D grid cells (see section 17.10) (0: no, 1: yes).
increment	0.1	The height increment for the volume tables [m].
useVolumeTableFile	0	Read and write the volume table from/to file (0: no, 1: yes).
[numerics]		
CFLMax	0.7	Maximum Courant nr [-].
AdvecType	33	Adv type, 0=no, 33=Perot q(uio-u) fast, 3=Perot q(uio-u).
AdvecCorrection1D2D	0	Advection correction of 1D2D link volume (0: regular advection, 1: link volume au*dx, 2: advection on 1D2D switched off.)
TimeStepType	2	0=only transport, 1=transport + velocity update, 2=full implicit step_reduce, 3=step_jacobi, 4=explicit.
maxNonlinearIterations	100	Maximal iterations in non-linear iteration loop before a time step reduction is applied.
setHorizontalBobsFor1d2d	0	Bobs are set to 2D bedlevel, to prevent incorrect storage in sewer system (0: no, 1:yes).
Limtyphu	0	Limiter type for waterdepth in continuity eq., 0=no, 1=minmod,2=vanLeer,3=Koren,4=Monotone Central.
Limtypmom	4	Limiter type for cell center advection velocity, 0=no, 1=minmod,2=vanLeer,4=Monotone Central.
Limtypsa	4	Limiter type for salinity transport, 0=no, 1=minmod,2=vanLeer,4=Monotone Central.
Pure1D	0	Purely 1D advection along branches (0: original quasi 2D advection using velocity vector, 1: pure 1D using flow volume vol1_f, 2: pure 1D using volume vol1, 3–7: pure 1D following SOBEK 2) (section 3.3.1.9).
Junction1D	0	Advection at 1D junctions (0: original quasi 2D advection using velocity vector, 1 = same as along 1D branches); keyword only used if Pure1D = 1 or 2, for other values of Pure1D the approach at junctions always matches the branches (section 3.3.1.9).

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
Icgsolver	4 or 6	Solver type, 4 = sobekGS + Saad-ILUD (default sequential), 6 = PETSc (default parallel), 7= CG+MILU (parallel).
LogSolverConvergence	0	Print time step, number of solver iterations and solver residual to diagnostic output (0: no, 1: yes).
Maxdegree	6	Maximum degree in Gauss elimination.
FixedWeirScheme	9	6 = semi-subgrid scheme, 8 = Tabellenboek, 9 = Villemonte (default).
FixedWeirContraction	1.	flow width = flow width*FixedWeirContraction. [-]
Fixedweirtopwidth	3	Uniform width of the groyne part of fixed weirs [m].
Fixedweirtalud	4	Uniform talud slope of fixed weirs.
Fixedweirtopfrictcoef		Uniform friction coefficient of the groyne part of fixed weirs [the unit depends on frictiontype].
FixedweirRelaxationcoef	0.6	Fixed weir relaxation coefficient for computation of energy loss.
FixedweirScheme1d2d	0	Fixed weir scheme for 1d2d links (0: same as fixedweirscheme, 1: lateral iterative fixed weir scheme).
Fixedweir1d2d_dx	50.0	Extra delta x for lateral 1d2d fixed weirs.
Izbndpos	0	Position of z boundary, 0=mirroring of closest cell (as in Delft3D-FLOW), 1=on net boundary.
Tlfsmo	0.	Fourier smoothing time on water level boundaries [s].
Slopedrop2D	0.	Apply droplosses only if local bottom slope > Slopedrop2D [m], <=0 =no droplosses.
Drop1D	0	Limit the downstream water level in the momentum equation to the downstream invert level, BOB_{down} ($\zeta^*_{down} = \max(\mathrm{BOB}_{down}, \zeta_{down})$).
Chkadvd	0.1	Check advection terms if depth < chkadvd [m].
Teta0	0.55	Theta (implicitness) of time integration [-], 0.5 < Theta < 1.0.
cstbnd	0	Delft3D-FLOW type velocity treatment near boundaries for small coastal models (1) or not (0).
maxItVerticalForester	0	Forester iterations for all constituents (0: no vertical filter, > 0: max nr of iterations).
TransportAutoTimestepdiff	0	Auto Timestepdiff in Transport, (0 : lim diff, no lim Dt_tr, 1: no lim diff, lim Dt_tr, 2: no lim diff, no lim Dt_tr, 3: implicit (only 2D)).
Implicitdiffusion2D	0	Implicit diffusion in 2D (0: no, 1:yes).
Turbulencemodel	3	0=no, 1 = constant, 2 = algebraic, 3 = k-epsilon, 4 = k-tau.
AntiCreep	0	Include anti-creep to suppress artifical vertical diffusion (0: no, 1: yes).
Barocponbnd	1	Include baroclinic pressure on open boundaries (0: no, 1: yes).
maxitpresdens	1	Max nr of iterations in pressure-density coupling, only used if thermobaricity is true.
DiagnosticTransport	0	No update of transport quantities, also known as diagnostic transport (0: no, 1: yes).
Maxwaterleveldiff	0.	Upper bound [m] on water level changes, (<= 0: no bounds). Run will abort when violated.
Maxvelocitydiff	0.	Upper bound [m/s] on velocity changes, (<= 0: no bounds). Run will abort when violated.
Maxvelocity	0.	Upper bound [m/s] on velocity (<= 0: no bounds). Run will abort when violated.

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
Waterlevelwarn	0.	Warning level [m AD] on water level (<= 0: no check).
Velocitywarn	0.	Warning level [m/s] on normal velocity(<= 0: no check).
Velmagnwarn	0.	Warning level [m/s] on velocity magnitude (<= 0: no check).
MinTimestepBreak	0.	Smallest allowed timestep [s], checked on a sliding average of several timesteps. Run will abort when violated.
Epshu	0.0001	Threshold water depth for wetting and drying [m].
EpsMaxlev	1d-8	Stop criterium for non linear iteration.
EpsMaxlevm	1d-8	Stop criterium for Nested Newton loop in non-linear iteration.
FlowSolver	generic1d2d3d	Flow solver: generic1d2d3d (generic solver), implicit1d (implicit 1D solver).
[physics]		
unifFrictCoef	0.023	Uniform friction coefficient [the unit depends on unifFrictType].
unifFrictType	1	Uniform friction type (0: Chezy, 1: Manning, 2: White-Colebrook, 3: White-Colebrook in WAQUA).
unifFrictCoef1D	0.023	Uniform friction coefficient in 1D links [the unit depends on unifFrictType].
unifFrictCoef1D2D	0.023	Uniform friction coefficient in 1D2D links [the unit depends on unifFrictType].
unifFrictCoefLin	0.	Uniform linear friction coefficient (0: no friction) [m/s].
Vicouv	0.1	Uniform horizontal eddy viscosity [m^2/s].
Dicouv	0.1	Uniform horizontal eddy diffusivity [m^2/s].
Vicoww	1.d-6	Background vertical eddy viscosity [m^2/s].
Dicoww	1.d-6	Background vertical eddy diffusivity [m^2/s].
Vicwminb	0.	Minimum viscosity in production and buoyancy term [m^2/s].
Xlozmidov	0.	Ozmidov length scale [m], default=0.0, no contribution of internal waves to vertical diffusion.
Smagorinsky	0.2	Add Smagorinsky horizontal turbulence [-]: vicu = vicu + ((Smagorinsky*dx)**2)*S.
Elder	0.	Add Elder contribution [-]: vicu = vicu + Elder*kappa*ustar*H/6); e.g. 1.0.
irov	0	Wall friction, 0=free slip, 1 = partial slip using wall_ks.
wall_ks	0.	Nikuradse roughness [m] for side walls, wall_z0=wall_ks/30.
Rhomean	1000.	Average water density [kg/m^3].
Idensform	2	Density calulation (0: uniform, 1: Eckart, 2: UNESCO, 3: UNESCO83).
thermobaricity	0	Include pressure effects on water density. Only works for idensform = 3 (UNESCO83).
Ag	9.81	Gravitational acceleration [m/s^2].
TidalForcing	0	Tidal forcing, if jsferic=1 (0: no, 1: yes).
Doodsonstart	55.565	Doodson start time for tidal forcing [s].
Doodsonstop	375.575	Doodson stop time for tidal forcing [s].
Doodsoneps	0.0	Doodson tolerance level for tidal forcing [s].
VillemonteCD1	1.0	Calibration coefficient for Villemonte [-]. Default = 1.0.
VillemonteCD2	10.0	Calibration coefficient for Villemonte [-]. Default = 10.0.

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
Salinity	0	Include salinity, (0: no, 1: yes).
InitialSalinity	0.	Initial salinity concentration [ppt].
Sal0abovezlev	-999.	Salinity 0 above level [m].
DeltaSalinity	-999.	uniform initial salinity [ppt].
saliMax	-999.	Limit the salinity to max value.
saliMin	0.	Limit the salinity to min value.
InitialTemperature	6.	Initial temperature [°C].
BackgroundwaterTemperature	20.	Background water temperature for Excess heat flux model and for Equation of State for models without temperature. [°C].
tempMax	-999.	Limit the temperature to max value.
tempMin	0.	Limit the temperature to min value.
salinityDependentFreezingPoint	0	Enable salinity-dependent freezing point (0 = no, 1 = yes). tempMin should be below 0 degrees Celsius.
Secchidepth	1.	Water clarity parameter [m].
Stanton	0.0013	Coefficient [-] for convective heat flux ( ), if negative, then Cd wind is used.
Dalton	0.0013	Coefficient [-] for evaporative heat flux ( ), if negative, then Cd wind is used.
SecondaryFlow	0	Secondary flow (0: no, 1: yes).
BetaSpiral	0.	Weight factor [-] of the spiral flow intensity on flow dispersion stresses (0d0 = disabled).
BreachGrowth	symmetric-asymmetric	Method for distributing dam breach width over dam break flow links: symmetric, proportional, or symmetric-asymmetric.
[wind]		
ICdtyp	2	Wind drag coefficient type (1: Const, 2: Smith&Banke (2 pts), 3: S&B (3 pts), 4: Charnock 1955, 5: Hwang 2005, 6: Wuest 2005, 7: Hersbach 2011 (2 pts), 8: 4+viscous).
Cdbreakpoints	6.3d-4 7.23d-3	Wind drag breakpoints [-], e.g. 0.00063 0.00723.
Windspeedbreakpoints	0. 100.	Wind speed breakpoints [m/s], e.g. 0.0 100.0.
Rhoair	1.2	Air density [kg/m^3].
computedAirdensity	0	Compute air density (0: no, 1: yes). Requires quantities airpressure, airtemperature and dewpoint in <ext>-file.
rhoWaterInWindStress	0	Water density used in computation of wind stress (0: space and time constant value specified via keyword Rhomean, 1: space and time varying local (surface) density of model).
Stresstowind	0.	Switch between Wind speed (=0) and wind stress (=1) approach for wind forcing.
Relativewind	0.	Wind speed [kg/m^3] factor relative to top-layer water speed*relativewind (0d0=no relative wind, 1d0=using full top layer speed).
Windpartialdry	1	Reduce windstress on water if link partially dry, only for BedlevType=3, 0=no, 1=yes (default).
PavBnd	0.	Average air pressure on open boundaries [N/m^2], only applied if value > 0.
PavIni	0.	Initial air pressure [N/m^2], only applied if value > 0.

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
[grw]		
Infiltrationmodel	0	Infiltration method (0: No infiltration, 1: Interception layer, 2: Constant infiltration capacity, 3: model unsaturated/saturated (with grw), 4: Horton).
UnifInfiltrationCapacity	0	Uniform maximum infiltration capacity [m/s].
[hydrology]		
InterceptionModel	0	Interception model (0: none, 1: on, via layer thickness).
[time]		
refDate	20010101	Reference date [yyyymmdd]. By default midnight is taken (00h00m00s)
tZone	0.	Data Sources in GMT are interrogated with time in minutes since refdat-Tzone*60 [min].
tUnit	S	Time units in MDU [D, H, M or S].
dtUser	300.	User timestep in seconds [s] (interval for external forcing update & his/map output).
dtNodal	21600.	Time interval [s] for updating nodal factors in astronomical boundary conditions.
dtMax	30.	Maximum timestep in seconds [s].
dtInit	1.	Initial timestep in seconds [s].
tStart	0.	Start time with respect to RefDate [Tunit].
tStop	86400.	Stop time with respect to RefDate [Tunit].
startDateTime		Computation start datetime (yyyymmddhhmmss), when specified, overrides TStart. hhmmss may be omitted, default: 000000.
stopDateTime		Computation stop datetime (yyyymmddhhmmss), when specified, overrides TStop. hhmmss may be omitted, default: 000000.
updateRoughnessInterval	86400.	Update interval for time dependent roughness parameters [s].
tStartTlfsmo	TStart	Start time with respect to RefDate of Fourier smoothing time on water level boundaries [Tunit].
startDateTimeTlfsmo		Computation start datetime with respect to RefDate of Fourier smoothing time on water level boundaries (yyyymmddhhmmss), when specified, overrides TStartTlfsmo.
timeStepAnalysis	0	Create timestep-analysis file (0: no, 1: yes). See Appendix F.6.
[restart]		
RestartFile		Restart file, only from NetCDF-file, hence: either *_rst.nc or *_map.nc.
RestartDateTime		Restart time [yyyymmddhhmmss], only relevant but obligatory in case of restart from *_map.nc. hhmmss may be omitted, default: 000000.
[external forcing]		
ExtForceFile		Old format for external forcings file *.ext, link with tim/cmp-format boundary conditions specification.

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
ExtForceFileNew		New format for external forcings file *.ext, link with bc-format boundary conditions specification. Multiple external forcing files can be specified separated by spaces. See section C.6.2.
Rainfall		Include rainfall, (0=no, 1=yes).
QExt		Include user Qin/out, externally provided, (0=no, 1=yes).
Evaporation		Include evaporation in water balance, (0=no, 1=yes).
WindExt		Include wind, externally provided, (0=no, 1=reserved for EC, 2=yes).
[hydrology]		
InterceptionModel	0	Interception model (0: none, 1: on, via layer thickness). See Section 11.3.
[trachytopes]		
TrtRou	N	Flag for trachytopes (Y=on, N=off).
TrtDef		File (*.ttd) including trachytope definitions.
TrtL		File (*.arl) including distribution of trachytope definitions.
DtTrt	60.	Interval for updating of bottom roughness due to trachytopes in seconds [s].
[output]		
Wrishp_crs	0	Writing cross sections to shape file (0=no, 1=yes).
Wrishp_dambreak	0	Writing dambreaks to shape file (0=no, 1=yes).
Wrishp_dryarea	0	Writing dry areas to shape file (0=no, 1=yes).
Wrishp_enc	0	Writing enclosures to shape file (0=no, 1=yes).
Wrishp_emb	0	Writing embankments file (0=no, 1=yes).
Wrishp_fxw	0	Writing fixed weirs to shape file (0=no, 1=yes).
Wrishp_gate	0	Writing gates to shape file (0=no, 1=yes).
Wrishp_genstruc	0	Writing general structures to shape file (0=no, 1=yes).
Wrishp_obs	0	Writing observation points to shape file (0=no, 1=yes).
Wrishp_pump	0	Writing pumps to shape file (0=no, 1=yes).
Wrishp_src	0	Writing sources and sinks to shape file (0=no, 1=yes).
Wrishp_thd	0	Writing thin dams to shape file (0=no, 1=yes).
Wrishp_weir	0	Writing weirs to shape file (0=no, 1=yes).
OutputDir		Output directory of map-, his-, rst-, dat- and timings-files, default: DFM_OUTPUT_<modelname>. Set to . for no dir/current dir.
WAQOutputDir		Output directory of Water Quality files.
FlowGeomFile		*_flowgeom.nc Flow geometry file in NetCDF format.
ObsFile		Space separated list of files, containing information about observation points. See section F.2.2.
DeleteObsPointsOutsideGrid	0	Delete observation points outside the grid (0=no, 1=yes).
CrsFile		Space separated list of files, containing information about observation cross sections. See section F.2.4.
FouFile		Name of attribute file that defines the *_fou.nc Fourier output file in NetCDF format, see C.15.

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
FouUpdateStep	0	Fourier output type: 0 = every user step ; 1 = every computational step ; 2 = equal to his output.
HisFile		*_his.nc History file in NetCDF format.
HisInterval	0.	History output, given as 'interval' 'start period' 'end period' [s].
XLSInterval	0.	Interval between XLS history [s].
MapFile		*_map.nc Map file in NetCDF format.
MapInterval	0.	Map file output, given as 'interval' 'start period' 'end period' [s].
RstInterval	0.	Restart file output, given as 'interval' 'start period' 'end period' [s].
ComInterval	DtUser	Comfile write times, given as 'interval' 'start period' 'end period' [s] w.r.t. Refdate
MapFormat	4	Map file format, 1: NetCDF, 4: NetCDF UGRID.
NcFormat	3	Format for all NetCDF output files (3: classic, 4: NetCDF4+HDF5).
NcMapDataPrecision	single	Precision for NetCDF data in map files (double or single).
NcHisDataPrecision	single	Precision for NetCDF data in his files (double or single).
NcCompression	0	Whether to apply compression (1 = yes, 0 = no) to the Map (*_map.nc) and Fourier (*_fou.nc) output files. NOTE: This option only works when NcFormat = 4.
NcNoUnlimited	0	Write full-length time-dimension instead of unlimited dimension (1: yes, 0: no). (Might require NcFormat=4.)
NcNoForcedFlush	0	Do not force flushing of map-like files every output timestep (1: yes, 0: no).
NcWriteLatLon	0	Write extra lat-lon coordinates for all projected coordinate variables in each NetCDF file (for CF-compliancy) (1: yes, 0: no).
Several His file options		See section F.3.1.
wriHis_balance	1	Write mass balance totals to his file, (1: yes, 0: no).
wriHis_structure_gen	1	Write general structure parameters to his file, (1: yes, 0: no).
wriHis_structure_dam	1	Write dam parameters to his file, (1: yes, 0: no).
wriHis_structure_pump	1	Write pump parameters to his file, (1: yes, 0: no).
wriHis_structure_gate	1	Write gate parameters to his file, (1: yes, 0: no).
wriHis_structure_weir	1	Write weir parameters to his file, (1: yes, 0: no).
wriHis_structure_orifice	1	Write orifice parameters to his file, (1: yes, 0: no).
wriHis_structure_bridge	1	Write bridge parameters to his file, (1: yes, 0: no).
wriHis_structure_culvert	1	Write culvert parameters to his file, (1: yes, 0: no).
wriHis_structure_longculvert	1	Write long culvert parameters to his file, (1: yes, 0: no).
wriHis_structure_damBreak	1	Write dam break parameters to his file, (1: yes, 0: no).
wriHis_structure_uniWeir	1	Write universal weir parameters to his file, (1: yes, 0: no).
wriHis_structure_compound	1	Write compound structure parameters to his file, (1: yes, 0: no).
wriHis_lateral	1	Write lateral data, (1: yes, 0: no).
wriHis_velocity	1	Write velocity magnitude in observation point to his file, (1: yes, 0: no).

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
wriHis_discharge	1	Write discharge magnitude in observation point to his file, (1: yes, 0: no).
wriHis_sourcesink	1	Write sources-sinks statistics to his file, (1: yes, 0: no).
wriHis_turbulence	1	Write k, eps and vicww to his file, (1: yes, 0: no).
wriHis_wind	1	Write wind velocities to his file, (1: yes, 0: no).
wriHis_rain	1	Write precipitation to his file, (1: yes, 0: no).
wriHis_airdensity	0	Write air density to his file (1: yes, 0: no).
wriHis_infiltration	1	Write infiltration to his file, (1: yes, 0: no).
wriHis_temperature	1	Write temperature to his file, (1: yes, 0: no).
wriHis_waves	1	Write wave data to his file, (1: yes, 0: no).
wriHis_heat_fluxes	1	Write heat fluxes to his file, (1: yes, 0: no).
wriHis_salinity	1	Write salinity to his file, (1: yes, 0: no).
wriHis_density	1	Write density to his file, (1: yes, 0: no).
wriHis_waterlevel_s1	1	Write water level to his file, (1: yes, 0: no).
wriHis_bedlevel	1	Write bed level to his file, (1: yes, 0: no).
wriHis_waterdepth	0	Write water depth to his file, (1: yes, 0: no).
wriHis_velocity_vector	1	Write velocity vectors to his file, (1: yes, 0: no).
wriHis_upward_velocity_component	0	Write upward velocity to his file, (1: yes, 0: no).
wriHis_sediment	1	Write sediment transport to his file, (1: yes, 0: no).
wriHis_constituents	1	Write tracers to his file, (1: yes, 0: no).
wriHis_zcor	1	Write vertical coordinates to his file, (1: yes, 0: no).
wriHis_taucurrent	1	Write mean bed shear stress to his file, (1: yes, 0: no).
wriHis_velocity	1	Write velocity magnitude to his file, (1: yes, 0: no).
wriHis_wqBot3d	0	Write 3D water quality bottom variables to his file, (1: yes, 0: no).
Several Map file options		See section F.3.2.
wriMap_waterlevel_s0	1	Write water levels at old time level to map file, (1: yes, 0: no).
wriMap_waterlevel_s1	1	Write water levels at new time level to map file, (1: yes, 0: no).
wriMap_evaporation	0	Write evaporation to map file, (1: yes, 0: no).
wriMap_velocity_component_u0	1	Write velocities at old time level to map file, (1: yes, 0: no).
wriMap_velocity_component_u1	1	Write velocities at new time level to map file, (1: yes, 0: no).
wriMap_velocity_vector	1	Write cell-center velocity vectors to map file, (1: yes, 0: no).
wriMap_upward_velocity_component	0	Write upward velocity component to map file, (1: yes, 0: no).
wriMap_density_rho	1	Write density to map file, (1: yes, 0: no).
wriMap_horizontal_viscosity_viu	1	Write horizontal viscosity to map file, (1: yes, 0: no).
wriMap_horizontal_diffusivity_diu	1	Write horizontal diffusivity to map file, (1: yes, 0: no).
wriMap_flow_flux_q1	1	Write fluxes to map file, (1: yes, 0: no).
wriMap_spiral_flow	1	Write spiral flow to map file, (1: yes, 0: no).
wriMap_numlimdt	1	Write numlimdt to map file, (1: yes, 0: no).

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
wrixyz_numlimdt	0	Write numlimdt to xyz file, (1: yes, 0: no). This option is useful when a map file is not written.
wriMap_taucurrent	1	Write bottom friction to map file, (1: yes, 0: no).
wriMap_chezy	0	Write chezy roughness in flow elements to map file, (1: yes, 0: no)
wriMap_chezy_on_flow_links	0	Write chezy roughness on flow links to map file, (1: yes, 0: no)
wriMap_input_roughness	0	Write chezy input roughness on flow links to map file, (1: yes, 0: no).
wriMap_turbulence	1	Write turbulence to map file, (1: yes, 0: no).
wriMap_rain	0	Write rainfall rate to map file, (1: yes, 0: no).
wriMap_wind	1	Write winds to map file, (1: yes, 0: no).
wriMap_airdensity	0	Write air density to map file, (1:yes, 0:no).
writek_CdWind	0	Write wind friction coefficients to tek file (1: yes, 0: no).
wriMap_heat_fluxes	0	Write heat fluxes to map file, (1: yes, 0: no).
wriMap_fixed_weir_energy_loss	0	Write energy losses of fixed weirs to map file, (1: yes, 0: no).
wriMap_wet_waterdepth_threshold	2d-5	Waterdepth threshold above which a grid point counts as 'wet'. Defaults to $0.2 \cdot$Epshu. It is used for wriMap_time_water_on_ground, wriMap_waterdepth_on_ground and wriMap_volume_on_ground.
wriMap_time_water_on_ground	0	Write cumulative time when water is above ground level (only for 1D nodes) to map file (1: yes, 0: no).
wriMap_freeboard	0	Write freeboard (only for 1D nodes) to map file (1: yes, 0: no).
wriMap_waterdepth_on_ground	0	Write waterdepth that is above ground level to map file (only for 1D nodes) (1: yes, 0: no).
wriMap_volume_on_ground	0	Write volume that is above ground level to map file (only for 1D nodes) (1: yes, 0: no).
wriMap_total_net_inflow_1d2d	0	Write current total 1D2D net inflow (discharge) and cumulative total 1D2D net inflow (volume) to map file (only for 1D nodes) (1:yes, 0:no).
wriMap_total_net_inflow_lateral	0	Write current total lateral net inflow (discharge) and cumulative total lateral net inflow (volume) to map file (only for 1D nodes) (1:yes, 0:no).
wriMap_water_level_gradient	0	Write water level gradient to map file (only for 1D links) (1:yes, 0:no).
wriMap_flow_analysis	0	Write flow analysis data to the map file (1:yes, 0:no).
wriMap_volume1	0	Write volumes to map file (1: yes, 0: no).
wriMap_waterdepth	1	Write water depths to map file (1: yes, 0: no).
wriMap_waterdepth_hu	0	Write water depths on u-points to map file (1: yes, 0: no).
wriMap_ancillary_variables	0	Write ancillary variables attributes to map file (1: yes, 0: no).
wriMap_flowarea_au	0	Write flow areas au to map file (1: yes, 0: no).
wriMap_velocity_magnitude	1	Write cell-center velocity vector magnitude to map file (1: yes, 0: no).
wriMap_velocity_vectorq	0	Write cell-center velocity vectors (discharge-based) to map file (1: yes, 0: no).
wriMap_flow_flux_q1_main	0	Write flow flux in main channel to map file (1: yes, 0: no).
wriMap_interception	0	Write interception to map file (1: yes, 0: no).

(continued on next page)

DRAFT

(continued from previous page)

Keyword	Default setting	Description
wriMap_windstress	0	Write wind stress to map file (1: yes, 0: no).
wriMap_CdWind	1	Write wind friction coeffs to tek file (1: yes, 0: no).
wriMap_DTcell	0	Write time step per cell based on CFL (1: yes, 0: no).
wriMap_bnd	0	Write boundary points to map file (1: yes, 0: no).
wriMap_Qin	0	Write sum of all influxes to map file (1: yes, 0: no).
wriMap_wqBot3d	0	Write 3D water quality bottom variables to map file (1: yes, 0: no).
wriMap_every_dt	0	Write output to map file every computational timestep, between start and stop time from MapInterval, (1: yes, 0: no).
writePart_domain	0	Write interpreted network file (DFM_interpreted_idomain_*.nc) containing domain partition info (1: yes, 0: no).
MapOutputTimeVector		File (.mpt) containing fixed map output times (s) w.r.t. RefDate.
ComOutputTimeVector		File (.ctv) containing fixed comfile write times (s) w.r.t. RefDate.
FullGridOutput	0	Full grid output mode for layer positions (0: compact, 1: full time-varying grid layer data).
EulerVelocities	0	Write Eulerian velocities, (1: yes, 0: no).
ClassMapFile		Name of class map file.
WaterlevelClasses	0.	Series of values between which water level classes are computed.
WaterdepthClasses	0.	Series of values between which water depth classes are computed.
ClassMapInterval	0	Interval [s] between class map file outputs.
WaqInterval	0.	Interval [s] between DELWAQ file outputs.
StatsInterval	-60.	Interval [s] between screen step outputs in seconds simulation time, if negative in seconds wall clock time.
TimingsInterval	0.	Timings output interval TimingsInterval.
Richardsononoutput	0	Write Richardson number, (1: yes, 0: no).

Remark:

- ◇ HisInterval, MapInterval, RstInterval, ClassMapInterval, ComInterval have been implemented such that all the intervals (in [s]) which are written (compared to TStart, which is in TUnit should be a multiple of DtUser (in [s]).

A.2 Overview of research keywords

In Table A.1 in this appendix an overview of keywords in the master definition file is given that can be specified by the user or should be added manually, respectively. In addition there are several research keywords in the computational kernel of D-Flow Flexible Mesh that should, in principle, not be used and not be changed by the user. These keywords haven't been documented yet and aren't tested in the D-Flow FM test benches yet. Therefore, using the default setting of these research keywords is strongly recommended. Because these keywords are listed in the diagnostic file, for the sake of completeness they are listed in the table below. If a keyword has a unit, then this unit will also be given in [ ] in the column of Description in this table.

DRAFT

Table A.2: *Overview of numerical research parameters with compulsory defaults.*

Keyword	Default setting	Description
[geometry]		
stripMesh	0	Flag specifying whether grid nodes and edges not connected to active grid cells/faces should be removed from the administration, 0 = no, 1 = yes. This option is only relevant in sequential mode; the partitioning step of a parallel run always performs this action. Switching this on reduces the clutter in the output map-file and simplifies data exchange via BMI.
calculateBedLevel-OverNonActiveLinks	false	Flag specifying whether the bed levels must be computed over all links of a cell including the closed boundaries, thin dams and dry points (=true), or only over the flow links (=false).
[numerics]		
Cffacver	0.	Factor for including (1-CFL) in HO term vertical (0d0: no, 1d0: yes).
Corioadamsbashfordfac	0.5	Adams Bashforth factor for Coriolis term.
Corioconstant	0	0=no change in Coriolis (default), 1=Coriolis constant in Spherical models, 2=Beta plane approach both in Cartesian and Spherical coordinates.
Drop3D	1	Apply drop losses in 3D if z upwind is below bob + 2/3 hu*drop3D (0: no, 1: yes).
Horadvtypzlayer	0	Horizontal advection treatment of z-layers for dam breaks (0: default, 2: sigma-like).
Icoriolistype	5	0=No Coriolis force, 5=default approach for Coriolis (depth dependent and similar to Delft3D-FLOW).
Jbasqbnddownwindhs	0	0=original hu on qbnd, 1=downwind hs on qbnd.
Filterorder	2	First-order or second order filter to suppress checkerboarding.
Keepstbndonoutflow	1	Keep salinity and temperature signals on boundary cells at outflow: 0 = no (like Thatcher-Harleman), 1 = yes (use prescribed boundary conditions).
Logprofatubndin	1	ubnds inflow: 0=uniform U1, 1 = log U1, 2 = user3D.
Logprofkepsbndin	0	inflow: 0=0 keps, 1 = log keps, 2 = user3D.
Newcorio	1	Temporary keyword for Coriolis improvement on open boundary; 0=No, 1=yes.
TestDryingFlooding	0	Drying flooding algorithm (0: D-Flow FM, 1: Delft3D-FLOW, 2: Similar to 0, and volume limitation in the transport solver based on Epshu).
Turbulenceadvection	3	Turbulence advection (0: none, 1=Upwind explicit, 2=Central explicit, 3: horizontally explicit and vertically implicit, 4=Central implicit.
Windhuorzwsbased	0	Computation of water depth at cell face for wind forcing; 0=Face-based (upwind approach of Delft3D-FLOW), 1=Finite volume-based.
rhoInterfaces	-1	Estimate rho at 3D layer interfaces for baroclinic pressure gradient method; -1 = original linear interpolation, 0 = improved linear interpolation, 1 = recompute from salinity and temperature, 2 = use cell density.
c1e	1.489	c1e coefficient in turbulence model.
c3eStable	0.	c3e coefficient (for stable stratification) in k-eps turbulence model.

(continued on next page)

DRAFT

Keyword	Preferred setting	Description
		(continued from previous page)
c3eUnstable	1.489	c3e coefficient (for unstable stratification) in k-eps turbulence model.
TKEMin	1d-32	Minimum turbulence kinetic energy (TKE) value in k-eps turbulence model.
EPSMin	1d-32	Minimum turbulent dissipation rate (EPS) value in k-eps turbulence model.
TAUMin	1d-32	Minimum turbulent time scale (TAU) value in k-tau turbulence model.
[physics]		
thermobaricityInPressure ↩ Gradient	0	Apply thermobaricity in computing the baroclinic pressure gradient (0 = no, 1 = yes).
freeConvectionCoefficient	0.14	Free convection turbulence coefficient [-].
secchiDepthNonPenetrative	0.0	Water clarity parameter for non-penetrative radiation [m].
secchiDepthNonPenetrativeFraction	0.0	Fraction of solar radiation that falls in non-penetrative spectrum [-].
[time]		
autoTimeStep	1 (in 2D), 5 (in 3D)	Automatic timestepping limitted by the CFL condition. Control which flows are used in the CFL limit. 1: for 2D-models; use only outflows, 3: for 3D-models; use horizontal outflows (per cell-layer), 4: for 3D-models; use horizontal in- and outflows (per cell-layer), 5: for 3D-models; use in- or outflows (per cell-column).
[calibration]		Roughness calibration. Details in Chapter 13.
UseCalibration	0	Activate calibration factor friction multiplier (0: no, 1: yes).
DefinitionFile		File (*.cld) containing calibration definitions. Details in Section C.10.1.
AreaFile		File (*.cll) containing area distribution of calibration definitions. Details in Section C.10.2.
[processes]		Online water quality processes. Details in **??**.
SubstanceFile		Substance file name. Details in **??**.
AdditionalHistoryOutputFile		Extra history output filename. Details in **??**.
StatisticsFile		Statistics definition file. Details in **??**.
DtProcesses	0.0	waq processes time step [s]. Must be a multiple of DtUser. If DtProcesses is negative, water quality processes are calculated with every hydrodynamic time step.
VolumeDryThreshold	1d-3	Volume [m^3] below which segments are marked as dry. Details in **??**.
DepthDryThreshold	1d-3	Water depth [m] below which segments are marked as dry. Details in **??**.

(continued on next page)

DRAFT

Keyword	Preferred setting	Description
		(continued from previous page)
`[veg]`		Dynamic vegetation model, more details in Section 12.2.1.
`Vegetationmodelnr`	`0`	Vegetation model nr, (0: no, 1: Baptist DFM).
`Clveg`	`0.8`	Stem distance factor [-].
`Cdveg`	`0.7`	Stem Cd coefficient [-].
`Cbveg`	`0.0`	Stem stiffness coefficient [-].
`Rhoveg`	`0.0`	Stem Rho, if > 0, bouyant stick procedure [kg/m^3].
`Stemheightstd`	`0.0`	Stem height standard deviation fraction, e.g. 0.1 [-].
`Densvegminbap`	`0.0`	Minimum vegetation density in Baptist formula. Only in 2D. [1/m^2].
If expchistem < 0		
`Expchistem`	0	[-]
`Expchileaf`	0	[-]
`Uchistem`	0	[m/s]
`Uchileaf`	0	[m/s]
`Arealeaf`	0	[m^2]
`Cdleaf`	0	[-]

Note: The data block `[processes]` for online water quality processes is not valid for D-Flow FM 1D2D, because this module doesn't have a water quality component yet.

A.3 Overview of deprecated and removed keywords

The following table lists in alphabetical order per data block the deprecated keywords (which are currently still supported by D-Flow FM, but may be removed in a future release) and the removed keywords (which are no longer processed). When the functionality of the keyword has is now supported via another keyword then the new keyword is specified. Check the description of the new keyword in Table A.1 for more details on how to use it.

Table A.3: *Overview of deprecated and removed keywords.*

Keyword	Action
`[Geometry]`	
`bathymetryFile`	Removed since March 2022. See `[Geometry]` keyword `bedLevelFile` in this table.
	(continued on next page)

Keyword	Action
	(continued from previous page)
bedLevelFile	Removed since March 2022. Use [Geometry] keyword iniFieldFile instead. That keyword should point to a file with the following content where my_bedlevel_data.xyb is the name of your bed level samples file which was originally referenced by the removed keyword. [General] fileVersion = 2.00 fileType = iniField [Initial] quantity = bedlevel dataFile = my_bedlevel_data.xyb dataFileType = sample interpolationMethod = triangulation
BotLevUni	Removed since March 2022. Use [Geometry] keyword bedLevUni instead.
botLevType	Removed since March 2022. Use [Geometry] keyword bedLevType instead.
manholeFile	Removed since March 2022. Use [Geometry] keyword storageNodeFile instead.
noOptimizedPolygon	Removed since March 2022. Option no longer supported.
thindykeFile	Removed since March 2022. Use [Geometry] keyword fixedWeirFile instead.
old format of general structure	Removed since October 2023. Use new keywords instead. The old/new keywords are: widthleftW1/upstream1Width, levelleftZb1/upstream1Level, widthrightW2/upstream2Width, levelrightZb2/upstream2Level, widthleftWsdl/downstream1Width, levelleftZbsl/downstream1Level, widthrightWsdr/downstream2Width, levelrightZbsr/downstream2Level, widthcenter/crestWidth, levelcenter/crestLevel, gateheight/gateLowerEdgeLevel, gatedoorheight/gateHeight, door_opening_width/gateOpeningWidth, lower_edge_level/gateLowerEdgeLevel, opening_width/gateOpeningWidth, sill_level/crestLevel, door_height/gateHeight, horizontal_opening_direction/gateOpeningHorizontalDirection, crest_level/crestLevel.
OrgFloorLevTopLayDef	Removed since November 2025. Use [Geometry] keyword FloorLevTopLay instead.
Tsigma	Removed since November 2025. Was used in the no longer supported Layertype = 4 option.
VertplizFile	Removed since November 2025. Was used in the no longer supported Layertype = 3 option.
circumcenterMethod	Deprecated since March 2026. Once the keyword is removed/becomes obsolete, the allNetlinksLoop method will be used.
circumcenterTolerance	Deprecated since March 2026. Once the keyword is removed/becomes obsolete, a fixed tolerance will be used.
[Numerics]	
facLaxTurb	Removed since August 2025. Use [Numerics] keyword turbulenceTimeIntegrationFactor instead.
Vertadvtypsal	Deprecated since October 2025. Use keyword verticalAdvectionType instead.
Vertadvtyptem	Deprecated since October 2025. Use keyword verticalAdvectionType instead.
maxItVerticalForesterSal	Deprecated since June 2026. Use keyword maxItVerticalForester instead.
	(continued on next page)

DRAFT

Keyword	Action
	(continued from previous page)
maxItVerticalForesterTem	Deprecated since June 2026. Use keyword maxItVerticalForester instead.
KeepZLayeringAtBed	Deprecated since June 2026. Use [Geometry] keyword KeepZLayeringAtBed instead.
facLaxTurbHor	Removed since August 2025. Use [Numerics] keyword turbulenceTimeIntegrationMethod instead.
facLaxTurbVer	Removed since August 2025. Option no longer supported.
hkad	Removed since March 2022. Option no longer supported.
iThinDykeScheme	Removed since March 2022. Use [Numerics] keyword fixedWeirScheme instead.
thinDykeContraction	Removed since March 2022. Use [Numerics] keyword fixedWeirContraction instead.
transportTimeStepping	Removed since August 2022. Option no longer supported.
transportMethod	Removed since August 2022. Value 1 is the default and only available option, no further input needed. Value 2 for diagnostic transport is now under its own keyword. Use [Numerics] keyword diagnosticTransport=1 instead.
[Physics]	
secchiDepth2	Deprecated since April 2026. Use [Physics] keyword secchiDepthNonPenetrative instead.
secchiDepth2Fraction	Deprecated since April 2026. Use [Physics] keyword secchiDepthNonPenetrativeFraction instead.
Superseded by default information included in the history file.	
[Output]	
writeBalanceFile	Removed since March 2022. Superseded by default information included in the history file.
[Processes]	
wriWaqBot3dOutput	Removed since August 2024. Replaced by fine-grained [Output] keywords wriHis_wqBot3d and wriMap_wqBot3d.
thetaVertical	Deprecated since October 2025. Use keyword verticalAdvectionType instead.

DRAFT

B The net file for flexible meshes

The net file contains the full computational grid (mesh) information. It is a netCDF file that can contain 2D grids, as well as 1D networks, and combinations of these. The netCDF conventions used for this file are:

- ◇ CF-conventions ($\geq$ 1.7) and UGRID-1.0 for the 2D grids. More information on: http://ugrid-conventions.github.io/ugrid-conventions/.
- ◇ A Deltares proposal for an extension to the UGRID convention for 1D network specifics. This proposal follows in section B.2. It is part of the Deltares ($\geq$ 0.10) conventions. This proposal builds on the simple geometries concept introduced in CF-conventions version 1.8.
- ◇ As a result the global `Conventions` attribute will typically read `CF-1.8 UGRID-1.0 Deltares-0.10`.

B.1 2D grids in netCDF UGRID files

D-Flow FM reads net files that adhere to the 2D rules in the UGRID conventions. At the minimum this means that the file must contain the mesh node coordinates and for all grid cells ('faces') the corner node numbers in the face_node_connectivity table.

B.2 Proposal for netCDF conventions for 1D networks

This section contains the Deltares proposal for conventions for netCDF datasets containing 1D networks (graphs) and data defined on these networks. It is part of the Deltares-0.11 conventions. These newly proposed conventions should currently be seen as an extension on top of the regular CF-conventions ($\geq$1.8) and, in particular, on top of the UGRID (1.0) conventions. Key new aspects are:

- ◇ Definition of 1D mesh locations in a *network coordinate space*, instead of a regular two-dimensional coordinate space (e.g., Cartesian).
- ◇ Definition of the network branch geometries adhering to the proposal for geometries in the CF-1.8 conventions[1].

B.2.1 History

- ◇ Deltares-0.11 (2020-03-20): New `flag_bounds` attribute for packed data (Section B.2.11.9).
- ◇ Deltares-0.10 (2019-08-21): Consistent renaming of variables and attribute names, singular instead of plural.
- ◇ Deltares-0.9 (2019-04-25): Duplicate mesh nodes on network (connection) nodes discouraged.
- ◇ Deltares-0.8 (2017-04-21): Initial version for 1D network extension to regular UGRID.

B.2.2 Naming conventions

The 1D domains considered in this proposal are mainly described on two levels:

1. The *network* is a 1D subspace (typically in a 2D or 3D space) defined by a 1D network topology and geometry.
2. The *1D mesh* is a discrete mesh defined in the network coordinate space.

Both two levels can to a large extent be described following the UGRID conventions. As a result, the UGRID naming conventions are followed, amongst others:

[1] https://github.com/cf-convention/cf-conventions/blob/master/ch07.adoc#geometries

DRAFT

- ◇ A *node* is a point, possibly with some edges connected to it.
- ◇ An *edge* is a connection between two nodes.

Occasionally the term *branch* is used in this proposal. This is not an officially proposed attribute nor keyword, but is often shorthand notation for: a branch is an edge in the network topology. In hydrodynamics, the network domain is often defined by several branches, and the computational 1D mesh points are placed on these branches. All proposed netCDF variable attributes will continue to use the word *edge*, though.

B.2.3 Requirements 1D Network

1 In this document, all indexes are 1-based, but this can also be 0-based, specified by the UGRID attribute `start_index`.
2 Mesh nodes need not necessarily be numbered increasingly along a network branch, see Figure B.4.
3 Mesh edges need not necessarily be numbered increasingly along a network branch, see Figure B.4.

B.2.4 1D Network: Example based on Figure B.1

A simple example is given in this section, Figure B.1. Also the networks given in section B.2.6 need to be supported, with special attention to Figure B.4.

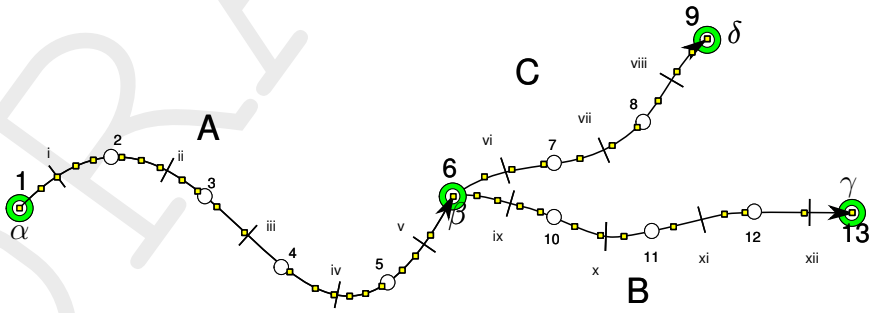


Figure B.1: *Simple computational 1D network:*
3 network edges (A, B, C),
4 network nodes (green circles, α, β, γ, δ),
46 network geometry nodes (yellow squares),
13 1D mesh nodes defined on the network (white circles, latin numbers),
12 1D mesh edges defined on the network (roman numbers)

B.2.4.1 Dimensions

network1d_nNodes = 4; network nodes α, β, γ, δ
network1d_nEdges = 3; branches A, B, C
network1d_nGeometryNodes = 46; yellow squares per branch (including both end points)
mesh1d_nNodes = 13 (13=6+5+4-(3-1); 3 branches are connected to node 6)
mesh1d_nEdges = 12 (12=5+4+3); mesh edges are between mesh nodes

```
dimensions:
    network1d_nNodes = 4 ;
    network1d_nEdges = 3 ;
    network1d_nGeometryNodes = 46 ;
    mesh1d_nNodes = 13 ;
    mesh1d_nEdges = 12 ;
```

DRAFT

```
    time = UNLIMITED ; // (2 currently)
    Two = 2 ;
```

B.2.4.2 1D-Network topology

The 1D network topology serves as the coordinate space in which a 1D mesh discretization will later be defined. The network is largely based on the UGRID conventions for its topology (i.e., nodes and edges) and additionally uses an optional `edge_geometry` to define the precise network branch geometries (more about this in the next Section).

```
uint network1d ;
    network1d:cf_role = "mesh_topology" ;
    network1d:edge_dimension = "network1d_nEdges" ;
    network1d:edge_geometry = "network1d_geometry" ;
    network1d:edge_length = "network1d_edge_length" ;
    network1d:edge_node_connectivity = "network1d_edge_nodes" ;
    network1d:long_name = "Network topology" ;
    network1d:node_coordinates = "network1d_node_x network1d_node_y" ;
    network1d:node_dimension = "network1d_nNodes" ;
    network1d:topology_dimension = 1 ;
```

network1d_node_x/y

```
double network1d_node_x(network1d_nNodes) ;
    network1d_node_x:standard_name = "projection_x_coordinate" ;
    network1d_node_x:long_name = "x-coordinate of network nodes" ;
    network1d_node_x:units = "m" ;
double network1d_node_y(network1d_nNodes) ;
    network1d_node_y:standard_name = "projection_y_coordinate" ;
    network1d_node_y:long_name = "y-coordinate of network nodes" ;
    network1d_node_y:units = "m" ;
```

```
network1d_node_x = -187.96667, 2195.7333, 4071.4928, 3445.4246 ;
network1d_node_y = 720.81667, 708.71667, 690.94861, 1540.1838 ;
```

network1d_edge_nodes

```
uint network1d_edge_nodes(network1d_nEdges, Two) ;
    network1d_edge_nodes:cf_role = "edge_node_connectivity" ;
    network1d_edge_nodes:long_name = "Start and end nodes of network edges" ;
    network1d_edge_nodes:start_index = 1 ;
```

```
 network1d_edge_nodes =
  1, 2,
  2, 3,
  2, 4 ;
```

network1d_edge_length

```
double network1d_edge_length(network1d_nEdges) ;
    network1d_edge_length:long_name = "Real length of branch geometries" ;
    network1d_edge_length:unit = "m" ;
```

DRAFT

```
network1d_edge_length = 2500, 2100, 1600 ;
```

B.2.4.3 1D-Network geometry

The network topology essentially defines a graph. Additionally, the actual geometrical shapes of connecting branches may be defined. This is done using the new geometry descriptions as introduced in CF-1.8[2]. The full network geometry typically consists of multiple *geometries* (i.e., branches) and each of them is defined by a list of geometrical *points*.

```
uint network1d_geometry ;
    network1d_geometry:geometry_type = "line" ;
    network1d_geometry:long_name = "1D Geometry" ;
    network1d_geometry:node_coordinates = "network1d_geom_x network1d_geom_y" ;
    network1d_geometry:node_count = "network1d_geom_node_count" ;
```

network1d_node_count

```
uint network1d_geom_node_count(network1d_nEdges) ;
    network1d_geom_node_count:long_name = "Number of geometry nodes per branch" ;
```

```
network1d_geom_node_count = 22, 13, 11 ;
```

This variable is needed, because all geometry is stored in contiguous ragged arrays. So the branches contain the following 46 geometry points (including the end points):

```
Branch A: 1, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
          18, 19, 20, 21, 22, 23, 24, 2;
Branch B: 2, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 3;
Branch C: 2, 36, 37, 38, 39, 40, 41, 42, 43, 44, 4.
```

Note that each list of geometry nodes also includes both end points of the branch, this implies that the connection node 2 appears three times in this array of 46 entries.

network1d_geom_x/y

```
double network1d_geom_x(network1d_nGeometryNodes) ;
    network1d_geom_x:standard_name = "projection_x_coordinate" ;
    network1d_geom_x:long_name = "x-coordinate of branch geometry nodes" ;
    network1d_geom_x:units = "m" ;
double network1d_geom_y(network1d_nGeometryNodes) ;
    network1d_geom_y:standard_name = "projection_y_coordinate" ;
    network1d_geom_y:long_name = "y-coordinate of branch geometry nodes" ;
    network1d_geom_y:units = "m" ;
```

```
network1d_geom_x = -187.96667, -127.96887, -53.84786, 56.174104, 187.13333,
    352.5, 529.96667, 683.23333, 828.43333, 892.96667, 993.8, 1082.5333,
```

[2] https://github.com/cf-convention/cf-conventions/blob/master/ch07.adoc#geometries

DRAFT

```
    1219.6667, 1405.2, 1492.2591, 1598.5306, 1712.2164, 1818.4835, 1912.3747,
    2007.9644, 2094.9, 2195.7333, 2195.7333, 2269.2456, 2381.5612, 2460.2412,
    2577.8276, 2774.8643, 2911.5187, 3029.1051, 3235.6759, 3438.7319,
    3582.0791, 3817.2519, 4071.4928, 2195.7333, 2275.9165, 2377.6129,
    2539.6914, 2739.9061, 2901.9847, 3041.8172, 3140.3355, 3226.4949,
    3321.8353, 3445.4246 ;
network1d_geom_y = y_1, ..., y_46
```

Note that the bold numbers are the begin and end nodes of a geometry.

B.2.4.4 Numerical discretization

The actual 1D mesh dataset — which goes as input into a model run, or results as output from a model run — is the discretized 1D mesh defined on the previously described 1D network. The definition is largely based on the UGRID conventions, but additionally uses location specification in the network 1D coordinate space using branch indexes and offsets.

```
uint mesh1d ;
    mesh1d:cf_role = "mesh_topology" ;
    mesh1d:long_name = "Topology data of 1D mesh" ;
    mesh1d:coordinate_space = "network1d" ;
    mesh1d:edge_dimension = "mesh1d_nEdges" ;
    mesh1d:edge_node_connectivity = "mesh1d_edge_nodes" ;
    mesh1d:edge_coordinates = "mesh1d_edge_branch mesh1d_edge_offset" ;
    mesh1d:node_coordinates = "mesh1d_node_branch mesh1d_node_offset" ;
    mesh1d:node_dimension = "mesh1d_nNodes" ;
    mesh1d:topology_dimension = 1 ;
```

mesh1d_edge_nodes

```
uint mesh1d_edge_nodes(mesh1d_nEdges, Two) ;
    mesh1d_edge_nodes:long_name = "Start and end nodes of mesh edges" ;
    mesh1d_edge_nodes:start_index = 1 ;
```

```
mesh1d_edge_nodes =
  1, 2,
  2, 3,
  3, 4,
  4, 5,
  5, 6,
  6, 7,
  7, 8,
  8, 9,
  6, 10,
  10, 11,
  11, 12,
  12, 13 ;
```

mesh1d_edge_branch

```
uint mesh1d_edge_branch(mesh1d_nEdges) ;
    mesh1d_edge_branch:long_name = "Index of branch on which mesh edges are located" ;
    mesh1d_edge_branch:start_index = 1 ;
```

```
mesh1d_edge_branch = 1, 1, 1, 1, 1, 3, 3, 3, 2, 2, 2, 2 ;
```

DRAFT

mesh1d_node_branch and mesh1d_node_offset

```
uint mesh1d_node_branch(mesh1d_nNodes) ;
    mesh1d_node_branch:long_name = "Index of branch on which mesh nodes are located" ;
    mesh1d_node_branch:start_index = 1 ;

double mesh1d_node_offset(mesh1d_nNodes) ;
    mesh1d_node_offset:_FillValue = 0. ;
    mesh1d_node_offset:long_name = "Offset along branch of mesh nodes" ;
    mesh1d_node_offset:units = "m" ;
```

```
mesh1d_node_branch = 1, 1, 1, 1, 1, 1, 3, 3, 3, 2, 2, 2, 2;
mesh1d_node_offset = 0, 500, 1000, 1500, 2000, 2500, 700, 1400, 2100, 400,
    800, 1200, 1600 ;
```

B.2.4.5 Duplicate or unique points

The association of discrete mesh locations with branch indexes needs to be unique in the dataset, but the network may allow multiple possible choices. Referring to Figure B.1, consider mesh point 6. It lies on the connection node between branches A, B and C. This proposal does not prescribe any particular choice for which branch index should be used for point 6. The application should choose this. The location at the end (offset) of branch A is equivalent to the start of branch B as well as the start of branch C. If the application needs duplicated mesh points on network connection nodes, the dataset should explicitly contain all these separate points.

In a similar way, the connectivity of mesh nodes is not defined in any special way by our proposal. Two mesh nodes are connected if and only if they appear as a tuple in the edge_node_connectivity table, as defined by the UGRID conventions.

Another related topic is the association of mesh edges with network branches. For example, when data is defined on mesh edges (e.g., flow rates) the dataset must contain two edge location variables, defining the network edge index and offset for each mesh edge. These variables can be referred to in the mesh's `edge_coordinates` attribute as well as the variable's `coordinates` attribute. Our proposal does *not* allow the association of mesh edges to network edges via an implicitly inferred way from the association of mesh nodes to network edges. Figure B.5 shows the need for this in branches B and D: the two distinct flow rates through the two u-points (one on each branch) can not be inferred from the surrounding mesh nodes, as the begin and end points of these two branches are the same.

DRAFT

B.2.5 1D Network, 1D2D Links as mesh contacts, 2D Meshes; Example based on Figure B.2

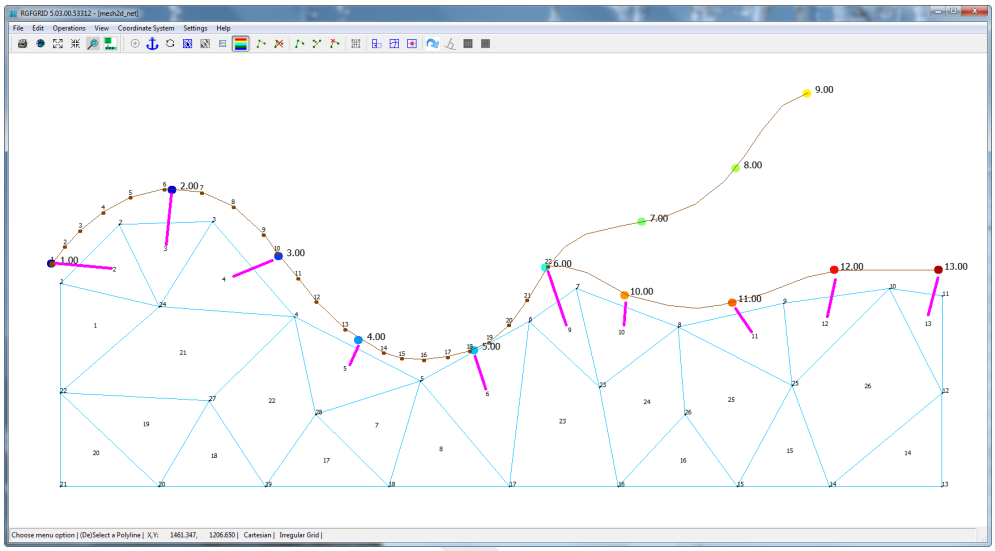


Figure B.2: *Composite mesh (1D Network, 1D2D Links and 2D Meshes),*
1D Mesh*: coloured dots (=computational 1D mesh), ζ-points; numbered dots, 1d network geometry for branch A (Figure B.1).*
2D Mesh*: light blue with node and face numbers.*
1D2D Link*: links are pink coloured.*
Python script: `create_ugrid_1d2d_map.py`

B.2.5.1 1D Network

See section B.2.4

B.2.5.2 1D2D Links as mesh contacts

1D2D Links may be defined between the computational nodes of the 1D Network (ζ-points) and the faces of the 2D Meshes (ζ-points) using the concept of *mesh contacts* from a Deltares-proposed UGRID-extension[3], see the attribute `contact` of the link1d2d-array below.

```
uint composite_mesh ;
    composite_mesh:cf_role = "parent_mesh_topology" ;
    composite_mesh:meshes = "mesh1d mesh2d" ;
    composite_mesh:mesh_contact = "link1d2d" ;
uint link1d2d(nLink1D2D_edge, Two) ;
    link1d2d:cf_role = "mesh_topology_contact" ;
    link1d2d:contact = "mesh1d:node mesh2d:face" ;
    link1d2d:start_index = 1 ;
```

Table B.1: *1D2D Link table based on Figure B.2*

mesh1d:node	mesh2d:face
1	2
2	3
3	4

[3] https://publicwiki.deltares.nl/display/NETCDF/Mosaics+of+meshes

4	5
5	6
6	9
10	10
11	11
12	12
13	13

B.2.6 Figures with example numbering of nodes and edges

For each of the figures in this section a netCDF-file is available and also the corresponding Python script to generate the netCDF4 file.

- ◇ Figure B.2. Script `create_ugrid_1d2d_map.py`.
- ◇ Figure B.3. Script `create_ugrid_1d_network_mesh_1_node_edge.py`.
- ◇ Figure B.4. Script `create_ugrid_1d_network_mesh_2_node_edge.py`.
- ◇ Figure B.5. Script `create_ugrid_1d_network_mesh_3_node_edge.py`.
- ◇ Figure B.6. Script `create_ugrid_1d_network_mesh_4_node_edge.py`.
- ◇ Figure B.7. Script `create_ugrid_1d_network_mesh_5_node_edge.py`.
- ◇ Figure B.8. Script `create_ugrid_1d_network_mesh_6_node_edge.py`.

The branch lengths of a network are given by the array `edge_length` and can not fully determined by the `node_offset` array, especially as several branches are connect between the same network nodes, see Figures B.4 to B.8. The Figures B.5 to B.8 does contain branches without intermediate nodes, so some branches does only have a begin- and an end-node.

The vertical bars in the figures mark the location of the middle of an edge indicating the velocity point for a staggered grid as used by Deltares hydrodynamic modules D-Flow 1D and D-Flow FM.

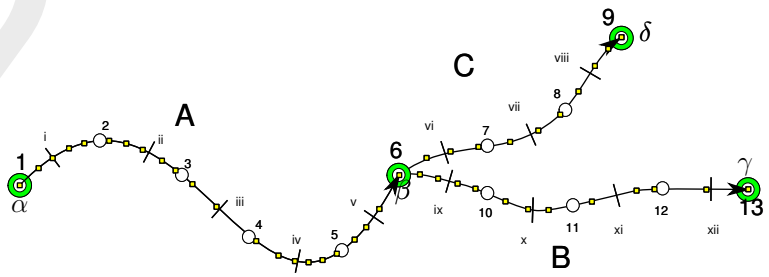


Figure B.3: *Simple 1D network and mesh, all branches do have intermediate nodes; 3 branches (A, B, C), 4 network nodes (green circles, α, β, γ, δ), 13 nodes.*

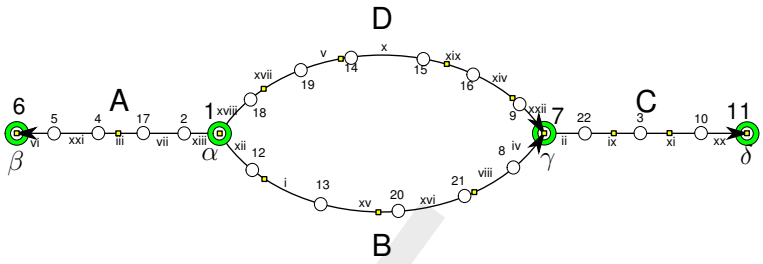


Figure B.4: *Simple 1D network and mesh, all branches do have intermediate nodes; 4 branches (A, B, C, D), 4 network nodes (green circles, α, β, γ, δ), 22 nodes (latin numbers), 22 edges (roman numbers).*

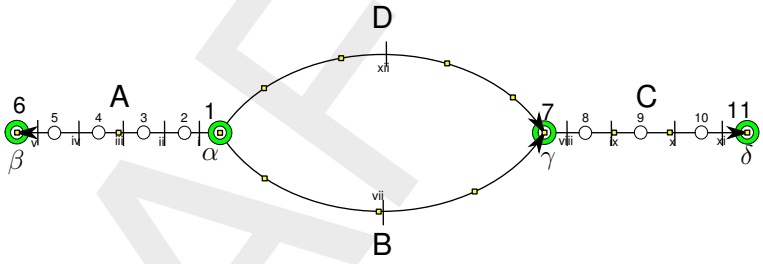


Figure B.5: *Simple 1D network and mesh, 2 branches do not have intermediate nodes (so only end points); 4 branches (A, B, C, D), 4 network nodes (green circles, α, β, γ, δ), 11 nodes (latin numbers), 12 edges (roman numbers).*

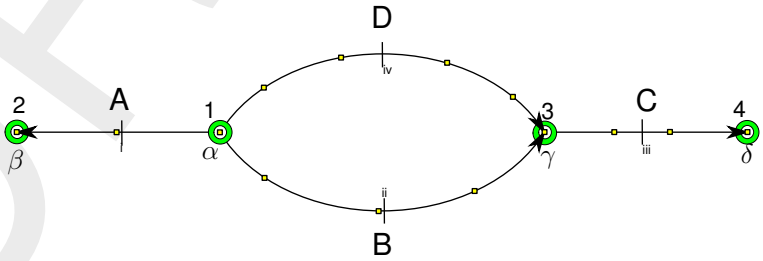


Figure B.6: *Simple 1D network and mesh, all branches do not have intermediate nodes (so only end points); 4 branches (A, B, C, D), 4 network nodes (green circles, α, β, γ, δ), 4 nodes (latin numbers), 4 edges (roman numbers).*

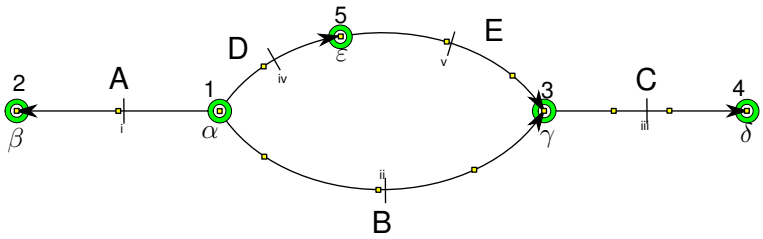


Figure B.7: *Simple 1D network and mesh, all branches do not have intermediate nodes (so only end points) and connection node ε does only have two branches connected. 5 branches (A, B, C, D, E), 5 network nodes (green circles, α, β, γ, δ, ε), 5 nodes (latin numbers), 5 edges (roman numbers).*
Network node 5 is special because it has only two edges attached. This is the same number as for a node which is not a network node.

DRAFT

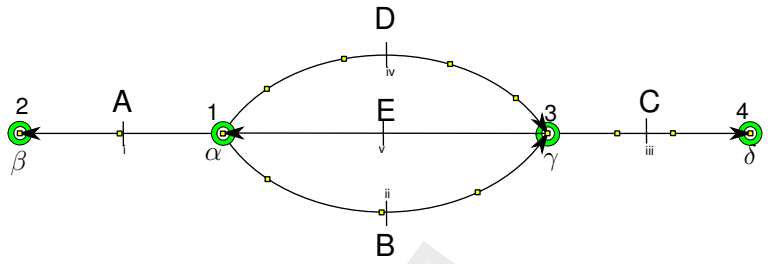


Figure B.8: *Simple 1D network and mesh, all branches do not have intermediate nodes (so only end points) and branch B, D and E have the same begin and end nodes. 5 branches (A, B, C, D, E), 4 network nodes (green circles, α, β, γ, δ), 4 nodes (latin numbers), 5 edges (roman numbers).*

In Figure B.8 the edges II, IV and V are special because they have the same begin and end node and should be separately identified by the array `mesh1d_edge_branch` array and also the node offset is not given in the array `mesh1d_node_offset` for these begin- and end-points. In the `mesh1d_node_offset` array the offset are given for the points 1 (α) and 3 (γ) which belongs to branch A and branch C.

B.2.7 Data defined on 1D meshes

Datasets that contain data variables defined on 1D meshes should define for each variable:

- ◇ The name of the mesh variable that defines the mesh, via the `mesh` attribute.
- ◇ The topological location on which the discrete data values are defined, via the `location` attribute. Allowed values are: `"node"` and `"edge"`.

This approach comes directly from the UGRID-conventions and is sufficient already.

As a result, the standard does not have to prescribe anything about assumed locations for any particular data variable. For example in hydrodynamics, relevant variables are water levels and velocities. It is up to the data producer (e.g., the simulation software) to define whether these are defined on nodes or edges, staggered or collocated. Below is an example definition of water level results on nodes (ζ-points) and velocities on edges (u-points).

```
...
double mesh1d_s1(time, mesh1d_nNodes) ;
    mesh1d_s1:location = "node" ;
    mesh1d_s1:long_name = "Water level" ;
    mesh1d_s1:mesh = "mesh1d" ;  //  this is UGRID location pointing to mesh1d topology
    mesh1d_s1:standard_name = "sea_surface_height_above_geoid" ;
    mesh1d_s1:units = "m" ;
double mesh1d_u(time, mesh1d_nEdges) ;
    mesh1d_u:location = "edge" ;
    mesh1d_u:long_name = "velocity along branch" ;
    mesh1d_u:mesh = "mesh1d" ;
    mesh1d_u:standard_name = "sea_water_speed" ;
    mesh1d_u:units = "m s-1" ;
...
```

DRAFT

B.2.8 1D Network history nc-file

To be added station locations, see for an example the D-Flow FM history file.

```
...
double s1(time=ntimes, stations=nstations);
    coordinates = "station_x station_y"
    long_name = "Water level"
    standard_name = "sea_surface_height_above_geoid"
    units = "m"
double u(time=ntimes, stations=nstations);
    coordinates = "station_xu station_yu"
    long_name = "Velocity along branch"
    standard_name = "sea_water_speed"
    units = "m s-1"
...
```

Note: The x- and y-coordinates of the station does not necessarily have the same coordinates as the nodes on the mesh or network.

B.2.9 Keyword definition for 1D Network

Table B.2: *1D network topology*

Required topology attributes	**Value**	**Example**
cf_role	"mesh_topology"	
topology_dimension	1	
edge_geometry	*variable name*	"network1d_geometry"
edge_node_connectivity	*variable name*	"network1d_edge_nodes"
node_coordinates	*variable name*	"network1d_node_x␣network1d_node_y"
Optionally required attributes		
edge_dimension	*dimension name*	"network1d_nEdges"
node_dimension	*dimension name*	"network1d_nNodes"
Optional attributes		
edge_length	*variable name*	"network1d_edge_length"
long_name		"Network topology"

Table B.3: *1D network geometry, referenced by the value of the attribute* **edge_geometry** *attribute as defined in Table B.2*

Required geometry attributes	**Value**	**Example**
geometry_type	"line"	
node_coordinates	*variable names*	"network1d_geom_x␣network1d_geom_y"
Optionally required attributes		
node_count	*variable name*	"network1d_node_count"
Optional attributes		

DRAFT

long_name		"1D Geometry"

Table B.4: *1D computational mesh*

Required mesh attributes	**Value**	**Example**
cf_role	"mesh_topology"	
topology_dimension	1	
coordinate_space	*variable name*	"network1d", *defined by Table B.2*
edge_node_connectivity	*variable name*	"mesh1d_edge_nodes"
node_coordinates	*variable names*	"mesh1d_node_branch␣mesh1d_node_offset"
Optionally required attributes		
edge_coordinates	*variable names*	"mesh1d_edge_branch␣mesh1d_edge_offset"
Optional attributes		
edge_dimension	*dimension name*	"mesh1d_nEdges"
node_dimension	*dimension name*	"mesh1d_nNodes"
long_name		"Topology data of 1D mesh"

B.2.10 Keyword definition for 1D2D Links

Table B.5: *Composite meshes, 1D2D Links*

Required attributes	**Value**	**Example**
cf_role	"mesh_topology_parent"	
meshes	*variable names*	"mesh1d␣mesh2d"
mesh_contact	*variable name*	"link1d2d"
Optional attributes		
long_name		"Topology data of composite 1D2D mesh"

Table B.6: *1D2D Links, referenced by the value of the attribute* mesh_contact *attribute as defined in Table B.5*

Required attributes	**Value**	**Example**
cf_role	"mesh_topology_contact"	
contact	*variable names*	"mesh1d:node␣mesh2d:face"
start_index	*integer*	1
Optional attributes		
long_name		"1D2D links"

DRAFT

B.2.11 Addendum: Deltares-specific agreements for 1D2D datasets

The preceding proposal was initiated by Deltares, but is intended to be generic for any 1D2D data producer or consumer. In this Section we will now additionally list a few Deltares-specific requirements that have been agreed upon as part of the functional design of the Delft3D Flexible Mesh Suite/D-HYDRO Suite software, abbreviated below as D-HYDRO.

B.2.11.1 Attributes for data variables

Variable units

The units of each variable are presented according the Climate and Forecast conventions, see http://cfconventions.org/Data/cf-standard-names/41/build/cf-standard-name-table.html, column "canonical units". Because the unit of Chézy does not fulfill the SI unit convention because the power of the length is not an integer ($C = [m^{\frac{1}{2}}/s] \equiv [m^{1/2}/s] \equiv [\sqrt{m}/s]$), we choose to represent the unit of Chézy in the netCDF file by: [m1/2 s-1]

long_name

Long names in a variable's `long_name` attribute should start with a capital character, because these names are used in plots as labels. See http://cfconventions.org/Data/cf-conventions/cf-conventions-1.7/cf-conventions.html#long-name

B.2.11.2 Non-unique relation between 1D mesh nodes and network topology edges

D-HYDRO uses one computation mesh point on each network topology node. When that network topology node is a network end point, the mesh node uniquely lies on that network topology edge (sometimes also referred to as a "'*branch*"'). However, when the network topology node is not an end point (then often referred to as a "'*connection node*"'), then the mesh node could equally well be associated with any of the connected network edges. For example, referring to Figure B.1, 1D mesh node 6 can be defined on branch A offset 2500, or on branch B offset 0, or on branch C offset 0. All are the same x, y locations. D-HYDRO has not yet defined a specific choice for network edge/branch selection, but this could for example be: the lowest branch number.

No choice has been made yet for models where the physics require multiple 1D mesh points on connection nodes (e.g. for morphology and 1D bed levels) nor where the physics require fluxes between all these multiple 1D mesh points on connection nodes (e.g., for water quality transport fluxes between all pairs of branches on a connection node).

B.2.11.3 Branch and node terminology

D-HYDRO models will often refer to network edges as branches, even though this is not in the standard. To this end, some additional attributes (and underlying variables) will be present for network edges in the net files:

- ◇ `network1d:branch_id` points to a variable name with the character IDs for all branches.
- ◇ `network1d:branch_long_name` points to a variable name with the character long names for all branches.
- ◇ `network1d:branch_order` points to a variable name with the integer branch orders for all branches, used in cross section interpolation.

The network nodes are also often referred to by their character string nodeId, for example in 1D boundary conditions (Section C.6.2.1). To this end, some additional attributes (and

underlying variables) will be present for network nodes in the net files:

- ◇ `network1d:node_id` points to a variable name with the character IDs for all nodes.
- ◇ `network1d:node_long_name` points to a variable name with the character long names for all nodes.

Contrary to the above definitions for the *network* layer, no such variables are defined for the computational *mesh* layer. Locations should not be referenced by any computational grid point number. This is motivated by the fact that model schematizations may be changed with different computation grid resolutions, making any old reference to grid point numbers obsolete. Only using network branch and node ids or even x,y coordinates to specify locations makes all model input reusable.

B.2.11.4 Water system characteristics

Most of the definitions in this entire chapter deal with geometry, but more network characteristics may need to be stored in order to represent all relevant details of the 1D water system.

B.2.11.4.1 Branch type

Urban sewer systems typically distinguish between pipes draining only storm sewage, only domestic/industrial sewage (so-called dry weather flows), or both (mixed). D-HYDRO stores this information into the network file by means of an integer branch type variable with explanatory `flag_meanings` attribute:

```
int network_branch_type(network1d_nEdges);
    long_name = "Water type in branch (network edge)";
    flag_values = [1, 2, 3, 4, 5];
    flag_meanings = "dry_weather_flow storm_water_flow mixed_flow
                     surface_water transport_water";
    _FillValue = -999;
    valid_range = [1, 5];
    mesh = "network1d";
    location = "edge";
```

The water type "transport_water" represents transport pipes that connect two subparts of the total network. Often these are pressurized pipes behind a pump. This type is mainly used in the GUI to distinguish these pipes from regular sewer pipes and open water.

The water type of a manhole (compartment), in the numerical model represented by a storage node, can be derived from the network branches connected to the corresponding network node; it's therefore not stored separately. When all connected branches are of the same type, the node is also of that type. When the connected branches have different types, the node will be of type `mixed_flow`. The storage nodes file (section C.18) includes a `nodeType` keyword to indicate the purpose of the manhole; that type represents information that is complementary to the water type.

B.2.11.5 Network geometry

The presence of network geometry (section B.2.4.3) via the `edge_geometry` attribute is optional in a network topology variable. D-HYDRO however, will always write network edge geometry points, and will also assume the presence of these when reading in 1D data files. In particular, when network edge lengths are specified (thereby overriding the (shortest) lengths of the straight network edges), it makes sense to define the precise geometrical shape of the

network edges. This is because the discrete 1D mesh points defined on each network edge using offsets can then be placed approximately at the correct x, y positions.

B.2.11.6 Positioning of 1D mesh nodes: from offset to x, y

Discrete 1D mesh points are defined in the dataset by the nodes's branch and offset variables. The corresponding x, y-coordinates of each 1D mesh node are determined as follows:

1 Determine the total length L_E of the network edge on which the mesh node lies:
   - ◇ *If* a network edge length variable is defined via the network topology's `edge_length` attribute, read the length for that edge.
   - ◇ *If not*, then calculate the direct length between the edge's endpoints, using the network topology's `edge_nodes` table, and its `node_coordinates` variables.

2 Interpolate the mesh node's offset d_n onto the network edge geometry:
   - ◇ *If* an explicit network edge geometry is defined via the network topology's `edge_geometry` attribute:
     - 2.1 Calculate the total length L_{G_E} of the geometry line string of that edge.
     - 2.2 Scale the offset d_n of the mesh node on the network edge to a corresponding offset of that mesh node on the network edge geometry: $d_{G_E} = d_n \cdot L_{G_E}/L_E$.
     - 2.3 Find the edge geometry's line segment number i_{G_E} in which the calculated offset d_{G_E} lies.
     - 2.4 Interpolate the scaled offset d_{G_E} between edge geometry points i_{G_E} and $i_{G_E} + 1$ using the cumulative edge geometry's line segment lengths.
   - ◇ *If not*:
     - 2.1 Calculate the relative offset of the mesh node on the network edge: $d_{n,\mathrm{rel}} = d_n/L_E$.
     - 2.2 Interpolate this relative offset between the network edge's start and end points (which have network node numbers $i_{E,1}$ and $i_{E,2}$, respectively): $x_n = (1 - d_{n,\mathrm{rel}}) \cdot x_{i_{E,1}} + d_{n,\mathrm{rel}} \cdot x_{i_{E,2}}$

B.2.11.7 Determine the chainage of nodes on a branch, ex. branch 1 of Figure B.4

(work in progress)
The branch offset for each node is given by the following array

```
mesh1d_node_branch = 1, 1, 3, 1, 1,
                     1, 3, 2, 4, 3,
                     3, 2, 2, 4, 4,
                     4, 1, 4, 4, 2,
                     2, 3 ;
mesh1d_node_offset =    0,  500, 1400, 1500, 2100,
                     2750, 3500, 3150, 2100, 2200,
                     3000,  700, 1400, 1050, 1400,
                     1750, 1000,  350,  700, 2100,
                     2800, 700 ;
```

The nodes 1, 6, 7 and 11 are network nodes, these nodes are assigned to a one of the branches who are connect to these nodes. In this example node 1 and 6 are assigned to branch A with offset 0 and 2750,and nodes 7 and 11 are assigned to branch C with offset of 0 and 3000.

How to determine the offset of a network node if not listed in the array `mesh1d_node_offset`

Note: The network edge length appears two times in the definition, one when defining the network edge length (array `network1d_edge_length`) and one in the array `mesh1d_node_offset`.

As example choose node 7.
For branch B, C and D the chainage value of node 7 is different:
3500.0 for branch B,
0.0 for branch C,
2500.0 for branch D
Which chainage value is assigned to this node is dependent on the branch you are looking to, the chainage for node 7 is 0.0 because node 7 is located on branch C, as given by array `mesh1d_node_branch` (C≡3).

Assume that we want to have the chainage values for branch B shown if Figure B.4.

- ◇ Look into the array `mesh1d_edge_branch` for the edges laying on branch B. The edges are: 1, 4, 8, 12, 15 and 16
- ◇ Look in the array `mesh1d_edge_nodes` for nodes who define the edges, i.e. 12–13, 8–7 , 21–8, 1–12, 13–20 and 20–21. These pairs are ordered, the positive direction is from the first to the second node, we need that in the next step.
- ◇ Sort these edges in positive direction, resulting in: 1–12, 12–13, 13–20, 20–21, 21–8 and 8–7. The first and last node are the begin and end node of the branch and have either zero or branch length as chainage.
- ◇ Node 1 is the begin of the branch and its chainage is set to 0.0 and node 7 is the end of the branch so its chainage is set to 3500.
- ◇ The chainage of the other nodes are given by array `mesh1d_node_offset`
- ◇ The chainage for branch B is given in Table B.7:

Table B.7: *Chainage table for branch B, Figure B.4*

Node	Chainage
1	0000.0
12	0700.0
13	1400.0
20	2100.0
21	2800.0
8	3150.0
7	3500.0

The same algorithm can be used for a so called route, i.e. a sequence of branches. Note that for a route the length of the preceding branches should be taken into account.

Note: If the edge direction is in the same direction as the branch direction then the offset of an edge at the beginning or end of the branch can be taken from the array `network2d_edge_length`.

B.2.11.8 1D2D link types

1D2D links are defined as mesh contacts in the network file, which have been introduced in Section B.2.5.2. The *type* of 1D2D links is not included in the concept of mesh contacts, therefore this is via an additional variable in the file. In the example below, the variable `link1d2d_contact_type` can be identified via the mesh contact's attribute `link1d2d:contact_type`.

```
uint link1d2d(nLink1D2D_edge, Two) ;
    link1d2d:cf_role = "mesh_topology_contact" ;
    link1d2d:contact = "mesh1d:node mesh2d:face" ;
    link1d2d:contact_type = "link1d2d_contact_type" ;
    link1d2d:start_index = 1 ;

uint link1d2d_contact_type(nLink1D2D_edge) ;
    link1d2d:contact_type = "link1d2d_contact_type" ;
    link1d2d:flag_values = [3, 4, 5, 7] ;
    link1d2d:flag_meaning = "lateral_1d2d_link longitudinal_1d2d_link
                             street_inlet_1d2d_link roof_gutter_1d2d_link";
    link1d2d:valid_range = [3, 7] ;
    link1d2d:_FillValue = -1 ;
```

The integer link type codes determine how the 1D2D link is treated numerically. This is further described in Section 17.7. Note that the lateral and embedded link types are both coded as type 3.

B.2.11.9 Data compression by packing

This section is not specific for 1D network input, but applies to describing packed data on any mesh or network. An example use of this is the classmap output as described in Section F.3.3. Output data may be compressed by *packing*, which is explained in the CF-conventions as "altering the data in a way that reduces its precision" [4]. For example, floating point data may be compressed by categorizing it in consecutive intervals, each represented by an integer class number. All classes together span the full data range.

Here we propose a small extension to the CF-conventions concept of flag values and flag meanings. We introduce a new data variable attribute `flag_bounds`, which gives the name of a variable containing the bounds of each category. This flag bounds variable for a set of N contiguous intervals must be an array of shape $(N, 2)$, where N is the slowest varying dimension. When an interval has an open end, that bound should be set to the fill value. The following example packs a velocity magnitude variable into 8 intervals of non-uniform size, the last of which open-ended.

```
dimensions:
  Two = 2 ;
  mesh2d_nFaces = 40 ;
  time = UNLIMITED ; // (24 currently)
  class_ucmag = 8 ;
variables:
  byte mesh2d_ucmag(time, mesh2d_nFaces) ;
    mesh2d_ucmag:mesh = "mesh2d" ;
    mesh2d_ucmag:location = "face" ;
    mesh2d_ucmag:coordinates = "mesh2d_face_x mesh2d_face_y" ;
    mesh2d_ucmag:cell_methods = "mesh2d_nFaces: mean" ;
    mesh2d_ucmag:cell_measures = "area: mesh2d_flowelem_ba" ;
    mesh2d_ucmag:standard_name = "sea_water_speed" ;
    mesh2d_ucmag:long_name = "Flow element center velocity magnitude" ;
    mesh2d_ucmag:units = "m s-1" ;
    mesh2d_ucmag:grid_mapping = "projected_coordinate_system" ;
    mesh2d_ucmag:flag_values = 1b, 2b, 3b, 4b, 5b, 6b, 7b, 8b ;
    mesh2d_ucmag:flag_bounds = "class_bounds_ucmag" ;
    mesh2d_ucmag:flag_meanings = "0.000m_s-1_to_0.200m_s-1
     0.200m_s-1_to_0.400m_s-1 0.400m_s-1_to_0.600m_s-1
     0.600m_s-1_to_0.800m_s-1 0.800m_s-1_to_1.000m_s-1
     1.000m_s-1_to_1.500m_s-1 1.500m_s-1_to_2.000m_s-1 above_2.000m_s-1" ;

  double class_bounds_ucmag(class_ucmag, Two) ;
```

[4] http://cfconventions.org/cf-conventions/cf-conventions.html#packed-data

```
data:
 mesh2d_ucmag =
  1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
  1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1,
    1, 4, 2, 1, 1, 5, 4, 2, 1, 5, 4, 2, 5, 4, 5,
  //...
 class_bounds_ucmag =
  0, 0.2,
  0.2, 0.4,
  0.4, 0.6,
  0.6, 0.8,
  0.8, 1,
  1, 1.5,
  1.5, 2,
  2, _ ;
```

DRAFT

C Attribute files

C.1 Introduction

In the following sections we describe the attribute files used in the input file (MDU-file) of D-Flow FM. Most of these files contain the quantities that describe one specific item, such as the location of open boundaries, or time dependent data of fluxes discharged in the model area by discharge stations. Most of the attribute files can be generated by the D-Flow FM GUI after defining an input scenario. Some files can almost only be generated by utility programs such as the unstructured grid generated by the D-Grid Editor. Still, we describe both type of files as it might be useful to know how the input data is structured to be able to generate (large) files, such as astronomic boundary conditions, or time-series for wind speed and direction by client specific tools.

C.2 Polyline/polygon file

D-Flow FM uses the same format for polylines and (closed) polygons. When used as a polygon file, there is *not* the requirement that the first and last point should be identical. It is good modelling practice to name files containing polygons with the extension <.pol>, and files containing polylines with the extension <.pli>. When the polylines have a third column with z-coordinates, the extension <.pliz> is advised.

File contents	The coordinates of one or more polylines. Each polyline (piecewise linear) is written in a single block of data.
Filetype	ASCII
File format	Free formatted
Filename	<*name*.pol>
Generated	RGFGRID, QUICKIN, D-Flow FM GUI, etc

Record description:

Record	Record description
	Preceding description records, starting with an asterisk ($*$), and will be ignored.
1	A non blank character string, starting in column one.
2	Two integers N_r, N_c representing the numbers of rows and number of columns for this block of data.
	Two reals representing the x, y or λ, ϕ-coordinate, followed by remaining data values at that location (if $N_c > 2$).

Example:

```
*
* Polyline L007
*
L007
6  2
            132400.0    549045.0
            132345.0    549030.0
            132165.0    549285.0
            131940.0    549550.0
```

DRAFT

```
            131820.0    549670.0
            131585.0    549520.0
*
* Polyline L008
*
L008
4  2
            131595.0    549685.0
            131750.0    549865.0
            131595.0    550025.0
            131415.0    550175.0
*
* Polyline L009
*
L009
6  2
            131595.0    549655.0
            148975.0    564595.0
            150000.0    564935.0
            152105.0    565500.0
            153150.0    566375.0
            154565.0    567735.0
```

C.3 Sample file

Sample file are used for several of D-Flow FM's input files.

File contents	The location and value of samples.
Filetype	ASCII
File format	Free formatted
Filename	<*name*.xyz>
Generated	Manually or Offline with QUICKIN or D-Flow FM GUI and data from digitised charts or GIS-database.

Record description:

Filetype	Record description
Free formatted	Location and sample value per row Two reals representing the x, y or λ, ϕ-coordinate and one real representing the sample value

Example:

Sample file with 12 sample values with their location (free formatted file).

```
   213813.2       603732.1      -4.053000
   214686.0       607226.1      -4.522000
   214891.7       610751.2      -5.000000
   210330.8       601424.1      -2.169000
   211798.0       604444.8      -2.499000
   212460.0       607475.7      -2.760000
   212436.9       610362.5      -2.865000
   185535.4       606607.9       1.360000
   186353.0       603789.4       1.122000
   187959.2       601197.6      0.9050000
   190193.0       599101.5      0.7050000
   208578.7       602513.7     -0.7990000
```

DRAFT

C.4 Time series <tim> file (ASCII)

Warning:

- ◇ Deprecated: The ASCII <tim> format is a legacy file format without any metadata. It is recommended to use the <bc> format instead (see Section C.5).

Time series files are used for several of D-Flow FM's input files. There is no header, except for optional comment lines (starting with * or #). There should be two or more columns containing numbers (integer or floating point): the first column contains time in minutes since the model's reference date, all remaining columns contain the data values. Columns can be separated using one or more whitespace or tab characters. Each line contains a single time with its (space-uniform) values. A simple example is shown below:

```
* comments
0  1.23
60 1.5
* ...
1440 1.5
```

C.5 Time-varying data <bc> file (ASCII)

Time-varying input is used in several places, for example for **B**oundary **C**onditions. The <*.bc> file format was initially introduced for one specific purpose, but its use has since expanded. Now, <*.bc> files can generally be used as well for meteorological forcings, hydraulic structure control, laterals, source-sinks, and time-varying parameter input (cf. Chapter D). The external forcings file may point to this type of file via the `forcingFile` keyword (see section C.6.2). Similary, a structure INI file may refer to this type of file for the individual hydraulic structure (geometrical) parameters (see Section C.13.12).

The time-varying data may be provided as timeseries or as periodic signals (see `harmonic` and `astronomic` in Table C.1.

The <*.bc> format enables the combination of multiple forcing signals within a single ASCII file. Multiple **Boundary**, **Meteo**, **Lateral**, **SourceSink** blocks in the external forcings file can reference the same file. Similarly, all hydraulic structure dynamics can be collected in the same file, although this is optional and left for the modeller to decide.

The file consists of a **General** block followed by one or more **Forcing** blocks. Each forcing block consists of a column-wise table (the data) and a header specifying how this table should be interpreted. A wide range of functions and forcings data is supported, so care must be taken to provide correct and complete header data.

Table C.1: *Description of <bc> format.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`1.01`	File version. Do not edit this.
`fileType`	String	`boundConds`	File type. Do not edit this.
`[Forcing]`			*Repeat as needed*

(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
`name`	String		Location name. Possible values are: ◇ `global`: Spatially uniform values, e.g., for meteo forcing. ◇ `<polyline point>`: Boundary polyline (point) reference. ◇ `<location Id>`: Location id, e.g., for meteo stations, hydraulic structures and other "point" locations. More details for each of these options can be found in Section C.5.1.
`function`	String		Function type. The supported options are `timeSeries`, `harmonic`, `astronomic`, `harmonic-Correction`, `astronomic-Correction`, `t3D`, `QHTable`, and `constant`. More details for each of these options can be found in Section C.5.2.
`offset`	Double	0	If `function` equals `timeSeries`, `t3D` or `constant` all values in the table are increased by the offset (after multiplication by factor).
`factor`	Double	1	If `function` equals `timeSeries`, `t3D` or `constant` all values in the table are multiplied with the factor. If `function` equals `harmonic` or `astronomic`) only the amplitude column is multiplied by the given factor.
`verticalPositions`	Double[]		The vertical position is specified as a space- or comma-separated list of floats, the interpretation of which varies according to the vertical position type. The order of the positions in the list becomes relevant when referring to them from the quantity blocks. However, no specific ordering of these positions (ascending or descending) is assumed.
`verticalInterpolation`	String		Possible values are: ◇ `linear`: linear interpolation between vertical positions. ◇ `log`: logarithmic interpolation between vertical positions (e.g. vertical velocity profiles). ◇ `block`: equal to the value at the directly lower specified vertical position.
			(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
`verticalPositionType`	String		Possible values are: ◇ `percBed`: Percentage [0-100%] or Fraction [0-1] w.r.t. water depth from the bed upward. ◇ `ZDatum`: z-coordinate with respect to the reference level of the model. ◇ `ZBed`: Absolute distance from the bed upward. ◇ `ZSurf`: Absolute distance from the free surface downward.
`timeInterpolation`	String		Possible values are: ◇ `linear`: linear interpolation between times. ◇ `Block-From`: equal to that at the start of the time interval (latest specified time value). ◇ `Block-To`: equal to that at the end of the time interval (upcoming specified time value). ◇ `amountToRate`: Input data is interpreted as absolute amount (e.g., millimeters) in past time interval, and returned as an average rate in that time interval (e.g., millimeters/second). Typically used for (but not restricted to) rainfall quantity.
The next three keywords repeated for every column in this data block			
`quantity`	String		Name of quantity, directly related to which type of (forcing) data is provided here. More details about all supported quantities can be found in Section C.5.3.
`unit`	String		Unit of quantity
`verticalPositionIndex`	Integer		This is a (one-based) index into the vertical-position-specification, assigning a vertical position to the quantity (`t3D`-blocks only)
The data block			
`...`	This block holds the actual data in columns. The interpretation of the columns is determined by the order of the aforementioned quantity keywords, i.e. the n-th quantity-block refers to the n-th column. The columns should contain valid floating point numbers (scientific notation is allowed). The only exception is the `astronomicComponent` column, which should contain strings representing the component names.		

DRAFT

C.5.1 Name in a bc block

The `name` keyword in the above Table C.1 depends on the type of forcing. The following subsections define the details for each of these.

C.5.1.1 Global forcings

The location name `global` is reserved for a global, spatially uniform forcing signal, e.g., uniform rainfall.

Example:

<*.ext> file block:

```
[Meteo]
quantity    = rainfall
forcingFile = meteo.bc
# ...
```

<*.bc> file Forcing block:

```
[Forcing]
name     = global
function = timeSeries
quantity = time
unit     = minutes since 2006-12-25 0:00:00
quantity = rainfall_rate
unit     = mm day-1
```

C.5.1.2 Boundary forcings

Boundary conditions can be prescribed by one or more **Forcing** blocks, each of which is uniquely tied to an individual boundary support point. For a boundary location defined as a polyline, the support points are the ones listed in the <.pli>-file. In that case the name in the bc-file should read `<polyline label>_<point number>`. The polyline label is defined in the first header line of the <*.pli> file. The point number should be formatted as a 4-digit zero-padded number. It is not needed to provide a **Forcing** block for all polyline points. In particular, when only a single block is provided, for example for point number 0001, the boundary signal is constant in space along the entire boundary polyline. Blocks with the function `QHTable` are an exception to this rule, because in qh-boundaries no data is required for individual support points. For this specific type of boundary condition, the name in the <.bc>-file should read `<polyline label>` without any point number.

If the boundary location is defined as a node for a 1D network, then the name should equal its `<nodeId>`.

Example:

<*.ext> file block:

```
[Boundary]
locationFile = upstream.pli
forcingFile  = boundaries.bc
# ...
```

<*.bc> file Forcing block:
(See page 368 for a complete example.)

```
[Forcing]
name = left01_0001
```

<upstream.pli> file:

```
left01
12 2
   30571.716797   30301.445312
   30585.171875   30315.066406
...
```

DRAFT

C.5.1.3 Meteo forcings on stations

Future functionality: Time series for meteorological forcings may be specified for individual locations, after which some spatial interpolation should be done. The `name` should specify the `<stationId>`, but additionally x- and y- coordinates must be specified. Currently, as the only alternative, station time series from netCDF files must be used. See Section E.2.4 for more details.

C.5.1.4 Forcings on "point" locations

Time series on independent objects/locations should be provided with the <*.bc> block `name` keyword equal to the <*.ext>/structure INI block `id` keyword to uniquely identify each object/location. This holds for lateral discharges and source-sinks, as well as hydraulic structures.

Example:

<*.ext> file block:

```
[SourceSink]
id        = sourcesinkABC
discharge = sources.bc
salinityDelta = sources.bc
# ...
```

<*.bc> file Forcing block:

```
[Forcing]
name     = sourcesinkABC
function          = timeseries
timeInterpolation = linear
quantity = time
unit     = minutes since 2006-12-25 0:00:00
quantity = sourcesink_discharge
unit     = m3 s-1
quantity = sourcesink_salinityDelta
unit     = ppt
```

See page 405 for a complete example for a hydraulic structure.

C.5.2 Function in a bc block

The `function` keyword specifies how the **Forcing** block should be interpreted. The `function` also affects which `quantity-unit` pairs should be present in the header. The possible values for the function are:

`timeSeries` — Value[s] as a function of time. One of the quantities must be called `Time` (see remarks below for unit).

`harmonic` — Period, amplitude, phase. One of the quantities (e.g., the first one) must be called `Harmonic Component` [-]. The first colum is the period in minutes. The other quantities should come in pairs:

- ◇ `<quantity> amplitude` (the unit depends on the quantity)
- ◇ `<quantity> phase` (degrees)

`astronomic` — Component-name, amplitude, phase. One of the quantities must be called `Astronomic Component` [-]. The other quantities should come in pairs:

- ◇ `<quantity> amplitude` (the unit depends on the quantity)
- ◇ `<quantity> phase` (degrees)

DRAFT

`Harmonic-Correction`	Period, amplitude-factor, phase-offset. One of the quantities must be called `Harmonic Component` [-]. The other quantities should come in pairs: ◇ `<quantity> amplitude` (the unit depends on the quantity) ◇ `<quantity> phase` (degrees)
`Astronomic-Correction`	Component-name, amplitude-factor, phase-offset. One of the quantities must be called `Astronomic Component` [-]. The other quantities should come in pairs: ◇ `<quantity> amplitude` (the unit depends on the quantity) ◇ `<quantity> phase` (degrees)
`t3D`	Values on multiple vertical positions versus time. One of the quantities must be called `Time` (see remarks below for unit).
`QHTable`	Discharge as a function of water level. The quantities definition should be exactly the following two: ◇ `<quantity> waterLevel` [m] ◇ `<quantity> discharge` [m^3/s]
`constant`	Constant value

Remarks:

- ◇ Although typical files will use UpperCamelCase keywords for chapters and lowerCamelCase for keywords and constants, all strings (including user-defined location and quantity names) will be verified by case insensitive comparison.
- ◇ Valid time units are string indicating the time unit (`days`, `hours`, `minutes`, or `seconds`) `since` a reference date (`YYYY-MM-DD`) and optional time (`HH-MM-SS`) and time zone (`+/-HH`). As in ISO 6801, `T` may be used between date and time; `Z` may be used as time zone indicator.
- ◇ All quantities in a harmonic or astronomic block are forced using the same components. All quantities in a time series block are forced using the same time points.

Example:

```
[General]
    fileVersion         = 1.01
    fileType            = boundConds

[Forcing]
    name                = left01_0001
    function            = Harmonic
    quantity            = Harmonic Component
    unit                = -
    quantity            = waterlevelbnd amplitude
    unit                = m
    quantity            = waterlevelbnd phase
    unit                = rad/deg/minutes
    0.0  4.114  0.0

[Forcing]
    name                = left01_0001
    function            = Harmonic
    quantity            = Harmonic Component
```

DRAFT

```
    unit                = -
    quantity            = normalvelocitybnd amplitude
    unit                = m/s
    quantity            = normalvelocitybnd phase
    unit                = rad/deg/minutes
    0.0  1.215  0.0

[Forcing]
    name                = right01_0001
    function            = TimeSeries
    timeInterpolation   = Linear
    quantity            = time
    unit                = minutes since 2001-01-01
    quantity            = waterlevelbnd
    unit                = m
       0.000000    2.50
    1440.000000    2.50
```

C.5.3 Quantity names in a bc block

The `quantity` keywords specify for each data column the specific quantity that it contains. Most quantities will be actual data quantities (more on that in the table below). Generally, the first quantity concerns the time dimension. The possible values for this are `time`, `harmonic component` and `astronomic component`, all explained before in Section C.5.2. The remainder of this section concerns the actual *data* quantities.

The name of a quantity is uniquely determined by the type of forcing (e.g., `waterlevelbnd`), and—in case of "objects/locations"—also a specific parameter of that object (e.g., `sourcesink_discharge`). The table below lists all possible values:

Table C.3: *Quantity names for use in <*.bc> files.*

Forcing type/object parameter	Quantity name in <*.bc> header
Boundary conditions	
All quantities listed in Table C.9 for boundary conditions.	Identical to <*.ext> `quantity` value, e.g., `waterlevelbnd`.
Lateral discharges	
`discharge`	`lateral_discharge`
Meteorological forcings	
Current support:	
`rainfall`	`rainfall` (will become `rainfall_amount` in a future release)
`rainfall_rate`	`rainfall_rate`
`windxy`	`windxy`
`windx`	`windx`
`windy`	`windy`
Future support:	
All quantity names listed in Table C.9 for meteorological fields.	

(continued on next page)

Forcing type/object parameter	Quantity name in <*.bc> header
	(continued from previous page)
Source and sinks	
discharge	sourcesink_discharge
salinityDelta	sourcesink_salinityDelta
temperatureDelta	sourcesink_temperatureDelta
sedFrac<fractionname>Delta	sourcesink_sedFrac<fractionname>Delta
tracer<tracername>Delta	sourcesink_tracer<tracername>Delta
Hydraulic structures	
weir	
crestLevel	weir_crestLevel
culvert	
valveOpeningHeight	culvert_valveOpeningHeight
pump	
capacity	pump_capacity
orifice	
crestLevel	orifice_crestLevel
gateLowerEdgeLevel	orifice_gateLowerEdgeLevel
gate	
crestLevel	gate_crestLevel
gateLowerEdgeLevel	gate_gateLowerEdgeLevel
gateOpeningWidth	gate_gateOpeningWidth
generalStructure	
crestLevel	general_structure_crestLevel
gateLowerEdgeLevel	general_structure_gateLowerEdgeLevel
gateOpeningWidth	general_structure_gateOpeningWidth
damBreak	
No <.bc> support yet.*	

C.6 The external forcings file

Two definition files for the external forcings are supported, each with their own format.

C.6.1 Old style external forcings

The name of the file is specified in the MDU-file as `ExtForceFile`. External forcings are specified in in blocks key-value pairs, e.g.

```
* comments
* comments

QUANTITY =waterlevelbnd
FILENAME =tfl_01.pli
FILETYPE =9
METHOD   =3
```

```
OPERAND  =O

QUANTITY = ...
FILENAME = ...
...
```

The format is not case-sensitive, but key-value pairs are expected in the *particular order*

The majority of the keywords speak for themselves or are optional. The `operand` needs some explanation. This keyword determines the operation to be performed with a new connected forcing for a quantity already set by another forcing from a different source: 'O' to replace the existing value with the new one and '+' to add the new to the existing value. Though generally applicable to all external forcings, it is most commonly used when working with spatial fields (such as wind and atmospheric pressure) originating from different types of sources. For instance a spatially uniform wind field could be set as a timeseries, but overwritten locally by a values interpolated from regional atmospheric model output, upon which a cyclone is superimposed from a spiderweb-type file. This means that the <ext>-file likely has three entries for quantity wind in the aforementioned order: a uniform timeseries with an operand 'O', a netCDF file with operand 'O' and a spiderweb file with operand '+'. The netCDF file might not cover the entire flow grid. In the uncovered regions, the value from the uniform wind is left unaltered. The same is done in the case of the spiderweb file with operand 'O'. Regions for which data is missing in spatial fields are treated as non-existent in the file (currently no interpolation is by default active to fill gaps, therefore it is highly recommendable to complete these files in a preprocessing step). In netCDF files, missing data are recognized as values equal to the fill value designated in the `_FillValue`-attribute. In files for the Curvi-type (ASCII) it is the `NODATA`-value in the header.

A comprehensive list of keywords is given in the table below.

DRAFT

quantity	character	Name of the quantity, see the list below
filename	character	File associated with this forcing
varname*	character	Variable name used in filename associated with this forcing; some input files may contain multiple variables
sourcemask*	character	File containing a mask
filetype	integer	Indication of the filetype: 1. Time series (C.4) 2. Time series magnitude and direction 3. Spatially varying weather 4. ArcInfo 5. Spiderweb data (cyclones) (C.14.3) 6. Curvilinear data (C.14.2) 7. Samples (C.3) 8. Triangulation magnitude and direction 9. Polyline (<∗.pli>-file, C.2) 11. NetCDF grid data (e.g. meteo fields) 14. NetCDF wave data
method	integer	Method of interpolation: 1. Pass through (no interpolation) 2. Interpolate time and space 3. Interpolate time and space, save weights (+ optional extrapolation) 4. Interpolate space 5. Interpolate time 7. Interpolate/Extrapolate time
extrapolation_method*	integer	Optionally allow nearest neighbour extrapolation in space (0: no, 1: yes). Default off.
maxsearchradius*	float	Max search radius for nearest neighbor extrapolation in space.
operand	character	Overwriting or superimposing values already set for this quantity: 'O' Values are overwritten. '+' New value is superimposed.
value*	float	custom coefficients for transformation
factor*	float	
ifrctyp*	float	
averagingtype*	float	
relativesearchcellsize*	float	
extrapoltol*	float	
area*	float	Area for source/sink

Note: Keywords marked with ∗ are optional

Remark:

◇ A polyline file specified as FILENAME for a boundary should only contain a single

DRAFT

polyline. If you need boundaries that consist of multiple unconnected polylines, provide a separate block for each of them.

C.6.2 New style external forcing

The name of this file is specified using the keyword `ExtForceFileNew` in the MDU-file (see chapter A). Multiple external forcing files may be specified by separating the file names by a space. File names containing spaces must be enclosed in double quotes. External forcings are specified in the ini-file format. Table C.4 lists the structure of this file.

Table C.4: *Definition of external forcings.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`2.02`	File version. Do not edit this.
`fileType`		`extForce`	File type. Do not edit this.
`[Boundary]` definition(s)			See Section C.6.2.1.
`[Lateral]` definition(s)			See Section C.6.2.2.
`[Meteo]` definition(s)			See Section C.6.2.3.
`[SourceSink]` definition(s)			See Section C.6.2.4.

This file may contain one or more `[Boundary]` and `[Lateral]`and `[SourceSink]` and `[Meteo]` blocks, each followed by a set of keywords that specify the location and forcing for that forcing. See the following sections for details.

Example:

```
[General]
fileVersion         = 2.02
fileType            = extForce

[Boundary]
quantity            = waterlevelbnd
locationFile        = tfl_01.pli
forcingFile         = tfl_01.bc

[Lateral]
id                  = Lateral1
name                = First Lateral
locationType        = 1d
branchId            = Channel1
chainage            = 250.000
discharge           = 27.3

[Meteo]
quantity            = rainfall
forcingFile         = precip.nc
forcingFileType     = netcdf
targetMaskFile      = roofs.pol
targetMaskInvert    = true
interpolationMethod = nearestNb

[SourceSink]
id                  = Id01
locationFile        = sourcesink01.pliz
discharge           = discharge01.bc
```

DRAFT

C.6.2.1 Boundary definitions

Table C.5: *Boundary definitions in new style external forcing file.*

Keyword	Type	Default	Description
(Part of new-style external forcings file, see Table C.5.)			
`[Boundary]`			*Repeat as needed*
`quantity`	String		Name of the quantity, see the list in Section C.6.3 (boundaries only)
`nodeId`	String		(Optional) Node id of a 1D network node.
`locationFile`	String		Name of boundary polyline $<*.\text{pli}>$. This keyword is only used if `nodeId` is not specified.
`forcingFile`	String		Name of file containing the forcing for this boundary. The file may either be a <bc> file (see section C.5) or a netCDF-file (see section E.2.4).
`returnTime` (`return_time` in earlier versions)	Double	0	(Optional) Thatcher-Harleman return time [s]. See also section 4.3.3. The default value of 0 seconds means that the Thatcher-Harleman return time has no impact.
`tracerFallVelocity`	Double	0	(Optional) Fall velocity for this tracer [m/s]. The default value 0 means that settling is disabled for this tracer.
`tracerDecayTime`	Double	0	(Optional) Decay time τ for this tracer [s]. Specifies the timescale over which the tracer decays to approximately 36.8% $(1/e)$ of its initial value. The default value 0 means that tracer decay is disabled for this tracer.
`bndWidth1D`	Double		(Optional) Custom width for boundary flow link, to override default mirrored internal flow link width.
`bndBlDepth`	Double		(Optional) Custom bed level depth below initial water level for boundary point, to override default mirrored bed level from internal pressure point.

Remarks:

- ◇ The `tracerfallVelocity` is set *per tracer*, not per boundary location. Multiple boundaries for the same tracer should not specify different values, otherwise the first value is used. It is sufficient to set the `tracerFallVelocity` only once.
- ◇ The `tracerDecayTime` is also set *per tracer* and follows the same rules as in the previous remark.
- ◇ A polyline file specified as `locationFile` for a boundary should only contain a single polyline. If you need boundaries that consist of multiple unconnected polylines, provide a separate `[Boundary]`+`locationFile` for each of them.
- ◇ When multiple `[Boundary]` blocks are present with the same `quantity` name, the behavior depends on the `locationFile` name:

DRAFT

- ◇ When also the `locationFile` full paths are the same, the boundaries are applied in the order in which they appear in the file, and are *added* on top of each other (comparable to implicitly setting `OPERAND=+` if this had been an old format external forcings file, see Section C.6).
- ◇ When the `locationFile` full paths are different, the boundaries are applied in the order in which they appear in the file, and are *overwriting* each other (comparable to implicitly setting `OPERAND=O` if this had been an old format external forcings file, see Section C.6).
- ◇ When using <*.bc files> the quantity names to be used in those files are described in Section C.5.3.

C.6.2.2 Lateral discharge definitions

Table C.6: *Lateral discharge definitions in new style external forcing file.*

Keyword	Type	Default	Description
(Part of new-style external forcings file, see Table C.4.)			
`[Lateral]`			*Repeat as needed*
`id`	String		Unique lateral discharge id.
`name`	String		Long name in the user interface.
`type`	String	`all`	*Deprecated.* Replaced by locationType.
`locationType`	String	`all`	(optional) Only when `xCoordinates` and `yCoordinates` are also specified. Possible values are: ◇ `1d`: only affect enclosed 1D grid points (only valid value when `nodeId` is given or `branchId` and `chainage` are given). ◇ `2d`: only affect enclosed 2D grid points. ◇ `all`: affect all enclosed grid points.
`applyTransport`	Int	0	Defines the way the concentrations of constituents at lateral locations are taken into account. ◇ `0` Only water (so without transport of constituents) is added to or removed from the model, depending on the sign of lateral discharge. For 3D models, lateral discharge is applied to the top layer of the model. ◇ `1` Transport of constituent is included. This option works only in combination with an external library via BMI-Interface. In a dimr configuration file, users can specify any constituents and layers of the lateral discharge. If the sign is positive, lateral discharge is passed to the model with a concentration set via the BMI-interface. When negative, discharge is passed from the model with a concentration equal to that of the ambient water in the model.

(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
nodeId	String		(optional) Node id of a 1D network node. The lateral will be connected to the grid point on/closest to that network node.
branchId	String		(optional) For a 1D point lateral: branch on which the lateral discharge is located, in combination with chainage.
chainage	Double		(optional) For a 1D point lateral: location on the branch of the point-lateral discharge [m], in combination with branchId.
numCoordinates	Int		(optional) number of values in xCoordinates and yCoordinates. This value should be ≥ 1.
xCoordinates	Double[]		(optional) x-coordinates of the lateral discharge location polygon [m]. (number of values = numCoordinates).
yCoordinates	Double[]		(optional) y-coordinates of the lateral discharge location polygon [m]. (number of values = numCoordinates).
locationFile	String		(optional) Name of lateral polygon $<*$.pol$>$.
discharge	*		The prescribed discharge for this lateral [m^3/s].

Remarks:

- ◇ Three types of location specifications are available:
  - ◇ When nodeId is given, it will be used to snap the lateral to the grid point closest to that network node (only for 1D).
  - ◇ When branchId and chainage are given, they will be used for the location specification (only for 1D).
  - ◇ When numCoordinates, xCoordinates and yCoordinates are given, or when locationFile is given, the points will be used to define a polygon. When numCoordinates ≥ 3, the lateral discharge will be distributed over all grid points inside that polygon. When instead the number of polygon points $= 1$, only the 2D grid cell encompassing that single point will receive the full lateral discharge.
- ◇ Items marked with (*) can either be a scalar value, or a time series file ($<$*.tim$>$ (deprecated)/$<$*.bc$>$) or realtime, to indicate that this parameter gets it values supplied by, e.g., realtime control.
- ◇ When using $<$*.bc files$>$ the quantity names to be used in those files are described in Section C.5.3.

C.6.2.3 Meteorological forcings

Table C.7: *Meteorological forcings in new style external forcing file.*

Keyword	Type	Default	Description
(Part of new-style external forcings file, see Table C.4.)			
[Meteo]			*Repeat as needed*
			(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
`quantity`	String		Name of the quantity, see the list in Section C.6.3 (meteo only).
`forcingFile`	String		Name of file containing the forcing for this meteo quantity.
`forcingFileType`	String		Type of `forcingFile`. Supported types: ◇ `bcAscii`: Space-uniform time series in <*.bc> (Section C.5). ◇ `uniform`: *Deprecated: Space-uniform time series in <*.tim> (Section C.4).* Use `bcAscii` instead. ◇ `netcdf`: NetCDF, either with gridded data, or multiple station time series (Section E.2.4). ◇ `uniMagDir`: Space-uniform timeseries of vector data in <*.tim> (Section C.4). Time, magnitude and direction in three columns. ◇ `ArcInfo`: Time- and space-varying scalar data in a variant of gridded ARCINFO ASCII <*.amu/amv/amp/amr> (More info in: (Deltares, 2026g, Section B.7.1)). ◇ `curviGrid`: Time- and space-varying scalar data in defined on a separate curvilinear grid in <*.amu/amv/amp/amr> or <*.apwxwy> (More info in: (Deltares, 2026g, Section B.7.2)). ◇ `spiderWeb`: Time- and space-varying wind and pressure on a spiderweb grid <*.spw> (More info in: (Deltares, 2026g, Section B.7.3)).
`forcingVariableName`	String		(Optional) name of data variable in the forcing file, typically used with NetCDF files.
`locationType`	String		*Obsolete! Maybe needed in the future for multi-station data in* `bcAscii` *files.*
`targetMaskFile`	String		(optional) Name of <*.pol> file to be used as mask. Grid parts inside any polygon will receive the meteo forcing.
`targetMaskInvert`	Logical	`no`	(optional) Flag indicating whether the target mask should be inverted, i.e., outside of all polygons: `no` or `yes`.
`interpolationMethod`	String		Type of (spatial) interpolation. Supported methods: ◇ `linearSpaceTime`: linear interpolation, both in space and time. ◇ `nearestNb`: nearest neighbor in space (linear in time), only with station data in `forcingFileType=netcdf`.
			(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
extrapolation ↩ Allowed	Logical	no	*Note: this setting is subject to future change*. Option for (spatial) extrapolation: no or yes. Used to fill holes in input forcing data, but not beyond the bounding box of that input forcing data. Uses nearest-neighbour extrapolation. Only available for forcingFileType=netcdf.
operand	String	O	(optional) How this data is combined with previous data for the same quantity (if any). Supported operands: ◇ O, override any previous data. ◇ V, override any previous data *only where* the provided forcing data file contains actual values. Only available for forcingFileType=netcdf. ◇ +, adds the provided values to the existing values.

Remarks:

- ◇ Spatially uniform meteo forcing is available as forcingFileType=bcAscii, when that file contains a scalar time series in a block with name=global.
- ◇ Multiple station time series is available as forcingFileType=netcdf, when that file contains a time series variable with a station dimension. The station feature is identified via the attribute :cf_role="timeseries_id". x- and y-coordinates must also be present, to support spatial interpolation.

C.6.2.4 Source and sink definitions

Table C.8: *Source and sink definitions in new style external forcing file.*

Keyword	Type	Default	Description
(Part of new-style external forcings file, see Table C.4.)			
[SourceSink]			*Repeat as needed*
id	String		Unique id.
name	String		Long name in the user interface.
locationFile	String		(optional) Name of source sink polyline $<*$.pli$>$ or $<*$.pliz$>$.
numCoordinates	Int		(optional) Number of values in xCoordinates and yCoordinates. This value should be ≥ 1.
xCoordinates	Double[]		(optional) x-coordinates of the source/sink discharge location polygon [m] (number of values = numCoordinates).
yCoordinates	Double[]		(optional) y-coordinates of the source/sink discharge location polygon [m] (number of values = numCoordinates).
			(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
zSource	Double[]		(optional) 1 (or 2) value(s) to specify in which vertical layer (range) the source is located (see section 6.4.10).
zSink	Double[]		(optional) 1 (or 2) value(s) to specify in which vertical layer (range) the sink is located (see section 6.4.10).
discharge	*		The prescribed discharge for the source [m^3/s].
area	Double		(optional) If present: used to compute the momentum released at the source location (see section 6.4.10).
salinityDelta	*		(optional) The prescribed salinity difference between source and sink.
temperatureDelta	*		(optional) The prescribed temperature difference between source and sink.
sedFrac<fractionname>Delta	*		(optional) The prescribed difference in concentration for sediment <fractionname> between source and sink.
tracer<tracername>Delta	*		(optional) The prescribed difference in concentration for tracer <tracername> between source and sink.

Remarks:

- ◇ Items marked with (*) can either be a scalar value (Double), or the name of a time series file <*.bc> (String) or realtime, to indicate that this parameter gets it values supplied by, e.g., realtime control. Keyword realtime is not (yet) available for sediment fractions and tracers.
- ◇ When using <*.bc files> the quantity names to be used in those files are described in Section C.5.3.
- ◇ Names of sediment fractions and tracers may contain special characters. For the keywords above, it is expected that spaces and forward slashes are replaced by underscores.

C.6.3 Accepted quantity names

The list of accepted quantity names is subdivided into six categories:

- ◇ Boundary conditions (old and new <*.ext> file)
- ◇ Meteorological fields (old and new <*.ext> file)
- ◇ Structure parameters (old <*.ext> file and Structure INI file)
- ◇ Initial fields (old <*.ext> file and initial field/parameter file)
- ◇ Spatial physical properties (old <*.ext> file and initial field/parameter file)
- ◇ Miscellaneous (mainly old <*.ext> file)

DRAFT

Table C.9: *List of accepted external forcing quantity names.*

Quantity	pg.	Description
Boundary conditions: *(old and new ext)*		
`waterlevelbnd`	161	Water level
`neumannbnd`	163	Water level gradient
`riemannbnd`	164	Riemann invariant
`outflowbnd`		
`velocitybnd`	162	Velocity
`dischargebnd`	161	Discharge
`salinitybnd`	60	Salinity
`temperaturebnd`	60	Temperature
`sedfracbnd`<*fractionname*>	see[1]	Suspended sediment concentration for fraction <*fractionname*>
`uxuyadvectionvelocitybnd`		
`normalvelocitybnd`, `tangentialvelocitybnd`	170	Normal and tangential velocity (only in 2D)
`qhbnd`	165	Discharge-water level dependency
`tracerbnd`<*tracername*>	60	User-defined tracer
`absgenbnd`		Absorbing-generating boundary
Meteorological fields: *(Items marked with † are deprecated, available in old ext file only)*		
`windx, windy, windxy`	236	Wind components, wind vector
`windspeedfactor`†		Dimensionless coefficient to downscale wind speeds (e.g. near wind farms).
`windstresscoefficient`†		Spatially varying wind drag coefficient C_d, constant for all wind speeds. Not yet in new <*.ext> file, currently available via initial fields and parameters file.
`airpressure`, `atmosphericpressure`	236	Atmospheric pressure
`airpressure_windx_windy`	236	Atmospheric pressure and wind components
`charnock`		Charnock coefficient
`airpressure_windx_windy_charnock`		Meteo forcing + Charnock coefficient
`airpressure_stressx_stressy`		Atmospheric pressure and direct input of wind stress
`pseudoAirPressure`		Water level correction expressed as pseudo atmospheric pressure fields
`waterLevelCorrection`		Water level correction fields
`rainfall` or `rainfall_rate`	239	Precipitation
`humidity, cloudiness`		
`airdensity, airpressure, airtemperature, dewpoint`	227	Varying air density
`humidity_airtemperature... ..._cloudiness`	**??**	Combined heat flux terms
`humidity_airtemperature... ..._cloudiness_solarradiation`		

[1] Deltares (2026d)

DRAFT

Quantity	pg.	Description
dewpoint_airtemperature... ..._cloudiness		
dewpoint_airtemperature... ..._cloudiness_solarradiation		
longwaveradiation	**??**	Long wave radiation
sensibleheatflux	456	Sensible heat flux
latentheatflux	456	Latent heat flux
netsolarradiation	**??**	Net solar radiation forcing. (No albedo will be applied)
solarradiation	**??**	Solar radiation
solarradiationfactor†	**??**	Solar radiation factor
Sources and sinks:		In new <*.ext> file, use specific keyword to supply forcing for each quantity (See Section C.6.2.4).
discharge_salinity... ..._temperature_sorsin†	59	Discharge, salinity and heat sources
Structure parameters: (old ext only)		*Deprecated, use preferred structure INI file instead (See Section C.13).*
pump	202	via type=pump, capacity=...
damlevel		via type=weir, crestLevel=...
gateloweredgelevel		via type=gate, gateLowerEdgeLevel=...
generalstructure		via type=generalStructure, crestLevel=..., etc.
Initial fields: (old ext only)		*Deprecated, see section D.2.*
initialwaterlevel	449	
initialsalinity	449	
initialsalinitytop	449	
initialtemperature	450	
initialverticaltemperatureprofile	450	
initialverticalsalinityprofile	450	
initialtracer<*tracername*>	450	
initialsedfrac<*fractionname*>	see[1]	Initial suspended sediment concentration for fraction <*fractionname*>
Spatial physical properties: (old ext only)	450	*Deprecated, see section D.2.*
frictioncoefficient		
horizontaleddyviscositycoefficient		
horizontaleddydiffusivitycoefficient		
backgroundverticaleddydiffusivitycoefficient		
advectiontype		
ibotlevtype		

DRAFT

Quantity	pg.	Description
Water quality processes:	**??**	
waqparameter	**??**	Processes input that varies in space
waqfunction	**??**	Processes input that varies in time
waqsegmentfunction	**??**	Processes input that varies in time
waqmassbalancearea	**??**	Definition of mass balance area
waqparameterProcessesInactive	**??**	Area where processes are switch off
waqsegmentnumber	**??**	Global segmentnumbers for D-Water Quality processes
Miscellaneous:		
shiptxy		
movingstationxy	463	Moving observation point for output (time,x,y)
wavesignificantheight	**??**	Wave related input
waveperiod	**??**	Wave related input

Table C.10: *List of quantity name and required standard name supported in NetCDF format.*

Quantity name	Standard name
Meteorological fields:	
airdensity	air_density
airpressure	air_pressure
airpressure_stressx_stressy	air_pressure
	surface_downward_eastward_stress
	surface_downward_northward_stress
airpressure_windx_windy	air_pressure
	eastward_wind
	northward_wind
airpressure_windx_windy_charnock	air_pressure
	eastward_wind
	northward_wind
	charnock
airtemperature	air_temperature
atmosphericpressure	air_pressure
bedrock_surface_elevation	bedrock_surface_elevation
charnock	charnock
cloudiness	cloud_area_fraction
dewpoint	dew_point_temperature
dewpoint_airtemperature_cloudiness	dew_point_temperature
	air_temperature
	cloud_area_fraction
dewpoint_airtemperature_cloudiness_solarradiation	dew_point_temperature

DRAFT

	air_temperature
	cloud_area_fraction
	surface_downwelling_shortwave_flux_in_air
friction_coefficient ↩ _time_dependent	friction_coefficient
humidity	relative_humidity
humidity_airtemperature ↩ _cloudiness	relative_humidity
	air_temperature
	cloud_area_fraction
humidity_airtemperature ↩ _cloudiness_solarradiation	relative_humidity
	air_temperature
	cloud_area_fraction
	cloud_area_fraction
	surface_downwelling_shortwave_flux_in_air
longwaveradiation	surface_net_downward_longwave_flux
sensibleheatflux	surface_upward_sensible_heat_flux
latentheatflux	surface_upward_latent_heat_flux
pseudoairpressure	air_pressure
rainfall	precipitation_amount
rainfall_rate	rainfall_rate
solarradiation	surface_downwelling_shortwave_flux_in_air or
	solar_irradiance
stressx	surface_downward_eastward_stress
stressxy	surface_downward_eastward_stress
	surface_downward_northward_stress
stressy	surface_downward_northward_stress
totalwaveenergydissipation	total_energy_dissipation
waterlevelcorrection	sea_surface_height
windx	eastward_wind
windxy	eastward_wind
	northward_wind
windy	northward_wind

C.6.4 Overview of deprecated and removed keywords in external forcing files

The following table lists in alphabetical order per data block the deprecated keywords (which are currently still supported by D-Flow FM, but may be removed in a future release) and the removed keywords (which are no longer processed). When the functionality of the keyword has is now supported via another keyword then the new keyword is specified.

Table C.11: *Deprecated and obsolete keywords in external forcings file.*

Keyword	Action
[Lateral]	
flow	Replace with keyword discharge.
type	Either remove or replace with keyword locationType.

`[Meteo]`	
`locationFile`	Replace with keyword `targetMaskFile`.

C.7 Initial and parameter fields

Initial and parameter fields are time-independent spatial fields that can describe initial conditions and spatially varying model parameters. The name of this file is specified using the keyword `IniFieldFile` in the MDU-file (see chapter A). This file supersedes the old-style external forcings file for these quantities. More details can be found in section D.2.

C.8 Trachytopes

The trachytope functionality allows for the usage of different types of roughness formulations at different locations within the computational domain. Multiple formulation may be active in the same grid cell. Several keywords in the MDU file influence the functioning. All keywords below should be placed underneath the `[trachytopes]` section in the MDU file.

Keyword	Value	Description	Default
`TrtRou`	Y or N	Trachytope option activated	N
`TrtDef`	<name.ttd>	Definition file trachytopes	
`Trtl`	<name.arl>	Area Roughness on Link file trachytopes	
`DtTrt`	pos. real	Time step in seconds for updating roughness and resistance coefficients based on trachytopes. Must be a multiple of `DtUser`.	1 `DtUser`
`Make spatially varying input calibration possible TrtCll`	*#name#*	Spatial calibration factor for trachytopes	uniform 1
`TrtMnH`	pos. real	Minimum water depth in roughness computation	.2 `Epshu`
`TrtMth`	1 or 2	Area averaging method	1
`TrtAsr`	real $\in [0, 1]$	Serial factor in averaging of area roughnesses	0.6
`TrtMxR`	integer	Maximum recursion depth for mixed trachytope definitions	8

Limitation: `DtTrt` must be a multiple of `DtUser`

DRAFT

C.8.1 Area Roughness on Links (ARL-file)

File contents	The Area Roughness on Links file (ARL-file) is the input file for the spatial distribution of the alluvial and vegetation roughness which are handled by the trachytopes module.
Filetype	ASCII
File format	Space separated file format
Filename	<*name*.arl>

The ARL file has the following properties:

- ◇ A single line comment, starts with a hash "`#`" or an asterisk "`*`".
- ◇ Each line has the format
  "`xu yu zu TrachytopesNr Fraction`",
  - ▫ where `xu`, `yu`, `zu` is the coordinate of the midpoint of the netlink,
  - ▫ `TrachytopeNr` is an integer corresponding to the number as described in the TrachyTopes Definition File (cf. section C.8.2),
  - ▫ and the meaning of `Fraction` depends on the `TrachytopeNr` (see below).
- ◇ A combination of multiple trachytopes can be applied by specifying multiple consecutive lines with the same set of midpoint-netlink-coordinates `xu`, `yu`, `zu` (ignoring any comment lines). If lines for the same set of coordinates `xu`, `yu`, `zu` are *not* consecutive, then the last (set of consecutive) line(s) will overrule all earlier lines.
- ◇ If the `TrachytopeNr` refers to an area trachytope, then the `Fraction` represents the fraction of the momentum control volume associated with that trachytope. This fractions should be between `0.0` and `1.0`. The sum of the `Fraction` values of all *area* trachytopes associated with one link, should sum to a value of at most `1.0`. If the sum is less than `1.0` the background roughness prescribed in the `[physics]` chapter in the MDU file is prescribed for the remaining `Fraction`, cf. chapter A. Only a single roughness type is allowed in combination with the Trachytopes module.
- ◇ If the `TrachytopeNr` refers to a linear trachytope, then the `Fraction` represents the length of the linear roughness element projected onto the netlink, relative to the length of the netlink (flow width). This value should be `0.0` or larger. The total projected length of multiple lines, e.g. multiple rows of hedges, may sum to a value larger than `1.0`.
- ◇ If the `TrachytopeNr` refers to a point trachytope, then the `Fraction` refers to the product of n, the number of trees per unit area, and D, the characteristic tree diameter. This value should be `0.0` or larger, but significantly smaller than $1/D$.

C.8.1.1 Example

An example of the <arl> file is given below based on the <ttd> file given in section C.8.2. In this case the netlink with u point located at x_u = 10.542 and y_u = 11.6, which is covered for 30 %, 30 % 20 % and 20 % for `TrachytopeNr` = 1, 2, 4 and 3 respectively.

```
(continued)
...
10.542    11.6    0    1      0.3
10.542    11.6    0    2      0.3
10.542    11.6    0    4      0.2
10.542    11.6    0    3      0.2
...
(continued)
```

C.8.2 Trachytope Definition file (TTD-file)

The trachytope definition file contains lines of the following format defining the different types of trachytopes. The types which are supported are general, discharge dependent and water-level dependent formats.

C.8.2.1 General format

The general format is formatted as follows:

```
TrachytopeNr    FormulaNr     ...Parameters...
```

where `...Parameters...` indicates a space separated list of formula specific parameters; the parameters required and their order are specified in section C.8.2.5. The user must specify for each trachytope (combination of formula number and parameters) a unique positive trachytope number. This trachytope number is used in the area files (see section C.8.1) to indicate the roughness types on a net link.

C.8.2.2 Example

An example of such a file where the first three codes 1–3 with a Nikuradse roughness height are defined, and the fourth (code 4) is Chézy roughness height.

```
1       51      0.1
2       51      0.2
3       51      0.3
4       52      35
```

C.8.2.3 Discharge dependent format

The discharge dependent format is formatted as follows:

```
TrachytopeNr    DISCHARGE    "Cross-section name"
TrachytopeNr    Q1      FormulaNr     ...Parameters...
TrachytopeNr    Q2      FormulaNr     ...Parameters...
TrachytopeNr    Q3      FormulaNr     ...Parameters...
TrachytopeNr    Q4      FormulaNr     ...Parameters...
...
```

which again expects a user defined `TrachytopeNr`. The first row contains the keyword “DISCHARGE” and subsequently a cross-section name occuring in CrsFile in the mdu-file.

The cross-section name can be enclosed by quotation marks, for instance when the cross-section name is made up of more than one word. The subsequent rows all have the same `TrachytopeNr` as the first row and furthermore every `FormulaNr` should be the same. The list of discharges Q1, Q2, Q3, Q4 should be monotonically increasing.

The `...Parameters...` for each formula type are specified in section C.8.2.5.

C.8.2.4 Water level dependent format

The water-level dependent format is similar to the discharge dependent format and is formatted as follows:

```
TrachytopeNr    WATERLEVEL    "Observation-station name"
TrachytopeNr    ZS1      FormulaNr     ...Parameters...
TrachytopeNr    ZS2      FormulaNr     ...Parameters...
TrachytopeNr    ZS3      FormulaNr     ...Parameters...
TrachytopeNr    ZS4      FormulaNr     ...Parameters...
...
```

which again expects a user defined `TrachytopeNr`. The first row contains the keyword “WATERLEVEL” and subsequently an observation-station name occuring in ObsFile in the mdu-file. The observation-statopm name can be enclosed by quotation marks, for instance when the observation-station name is made up of more than one word. The subsequent rows all have the same `TrachytopeNr`

DRAFT

as the first row and furthermore every `FormulaNr` should be the same. The list of waterlevels ZS1, ZS2, ZS3, ZS4 should be monotonically increasing.

The `...Parameters...` for each formula type are specified in section C.8.2.5.

DRAFT

C.8.2.5 Supported roughness formulations

The roughness formulations specified in the table below are supported by D-Flow FM. These formulations are specified in the .ttd by `FormulaNr` and `...Parameters...` in the order in which they appear in the table below. The related formulae are presented in the Technical Reference Manual

FormulaNr	Description	Parameters
Special classes (1–50)		
1	flood protected area, reduces the effective area of the grid cell. It has no influence on the continuity equation (i.e. it does not decrease the surface area of the grid cell).	–
2	composite trachytope: fraction α of type T_1 and fraction β (generally $\beta = 1 - \alpha$) of type T_2	T_1, T_2, α, β
Area trachytope classes: simple type (51–100)		
51	constant White-Colebrook/Nikuradse value	k [m]
52	constant Chézy value	C [m$^{1/2}$/s]
53	constant Manning value	n [s/m$^{1/3}$]
54	constant z_0 value	z_0 [m]
61	constant White-Colebrook/Nikuradse values for ebb and flood	k_{ebb} [m], k_{flood} [m]
62	constant Chézy values for ebb and flood	C_{ebb} [m$^{1/2}$/s], C_{flood} [m$^{1/2}$/s]
63	constant Manning values for ebb and flood	n_{ebb} [s/m$^{1/3}$], n_{flood} [s/m$^{1/3}$]
64	constant z_0 values for ebb and flood	$z_{0,ebb}$ [m], $z_{0,flood}$ [m]
Area trachytope classes: alluvial type (101–150)		
101	simplified Van Rijn	A [m$^{0.3}$], B [m$^{0.3}$]
102	power relation	A [m$^{1/2}$/s], B [-]
103[2]	Van Rijn predictor	-
104[2]	Struiksma predictor	A_1 [m$^{1/2}$/s], A_2 [-], θ_c [-], θ_m [-], $C_{\min}$ [m$^{1/2}$/s]
105[2]	bedforms quadratic	-
106[2]	bedforms linear	-
Area trachytope classes: vegetation type (151–200)		
151	Barneveld 1	h_v [m], n [1/m]
152	Barneveld 2	h_v [m], n [1/m], C_D [-], k_b [m]
153	Baptist 1	h_v [m], n [1/m], C_D [-], C_b [m$^{1/2}$/s]
154	Baptist 2	h_v [m], n [1/m], C_D [-], C_b [m$^{1/2}$/s]

[2] Formulae 103, 104, 105 and 106 use the D_{50} and D_{90} data from the sediment transport and morphology module if available. In case of a simulation without sediment, these values are obtained from the keywords `BdfD50` and optionally `BdfD90` in the **[bedform]** of the mdu-file. The values of `BdfD50` and `BdfD90` are a constant value for the whole grid, or the file name of a file containing a spatial varying field. The default value for D_{50} is 0.2 mm, and the default value for D_{90} is $1.5D_{50}$.

DRAFT

FormulaNr	Description	Parameters
Linear trachytope classes: various (201–250)		
201 202	hedges 1 hedges 2	h_v [m], n [1/m] h_v [m], n [1/m]
Point trachytope classes: various (251–300)		
251	trees	h_v [-], C_D [-]

C.9 Fixed weirs

A fixed weir has many quantitites. On a polyline several quantitites has to be specified. In the table below an overview is given of all these quantities.

Keyword	Description	Default value
`X-coordinate`	X-coordinate of polyline point	-
`y-coordinate`	Y-coordinate of polyline point	-
`Crest level`	Absolute crest level	- [mAD]
`Ground height left`	Difference between crest level and toe level at left side	0.0 [m]
`Ground height right`	Difference between crest level and toe level at right side	0.0 [m]
`Crest width`	Width of weir in perpendicular direction	3.0 [m]
`Slope left`	Slope of weir at left side	4.0 [-]
`Slope right`	Slope of weir at right side	4.0 [-]
`Roughness code`	Roughness code for vegetation on weir	0.0 [-]
`Weir type`	(v)illemonte or (t)abellenboek	v [-]

in which 'mAD' stands for 'meter Above Datum', which denotes a level relative to the vertical reference system. The last column with the weir type (Villemonte or Tabellenboek) is optional. In this way, individual polyline can be given another weir type. For example, if FixedweirType=9 (Villemonte) has been specified in the MDU-file, then individual polylines can be set to Tabellenboek by adding a "t" to all points in the polyline; see the example below.

The default value for the ground height levels (left and right) is 0.0 m. However, in case of the Tabellenboek a minimum value for the ground heights of 0.1 m is applied, because the empirical Tabellenboek relation doesn't allow values smaller than 0.1 m. Noted that in WAQUA also a minimum value of 0.1 m is applied in case of the Tabellenboek. For fixed weirs also a shape file can be generated, see section 6.4.13. In case of the Tabellenboek the ground heights in the shape file can be slightly different from the value in the input file, because a minimum value of 0.1 m is applied.

For some of the fixed weir input quantities upper and lower limits are applied, because realistic input values are required for an accurate computation of the energy losses of fixed weirs. This involves a maximum crest level of 10000 [m], a minimum slope of 0.0000001 [-], a maximum slope of 1000 [-],

DRAFT

a minimum ground height of 0.0 [m] and a maximum ground height of 500 [m]. If one of these values isn't between the lower and upper limit, then an error message is written to the diagnostic file and the simulation will stop. Then, the user has to check the fixed weirs input file for erroneous values. Below an example is given of an error message for fixed weirs in the diagnostic file, in which a negative slope is applied.

```
    ** INFO   : Opened file : harlingen-1_fxw_error.pliz
** INFO   : Closed file : harlingen-1_fxw_error.pliz
** WARNING: Slope left -0.10000D+01 is smaller than the minimum limit   0.10000D-07.
** WARNING: Slope left is located at line 2 in PLI FixedWeir01 of file harlingen-1_fxw_error.pliz.
** ERROR  : Some fixed weirs have values outside of limits. See messages above.
```

Example

An example of a polyline with fixed weir input is given below. It consists of a polyline with two points

```
(continued)
...
weir
     2     10
     399.999420    99.997688    2.0   0.2   0.8   3.0   4.0   4.0   0   t
     399.999420   100.999542    2.1   0.3   0.9   3.0   4.0   4.0   0   t

...
(continued)
```

In general, fixed weir polylines do not coincide with the computational grid. The polylines are snapped to flow links, which is illustrated in Figure 6.15. This is computed by the computational kernel of D-Flow FM.

In practice, it is possible that a certain flow link contains multiple snapped fixed weirs. This is for example the case when polylines with fixed weirs cross each other. Then, the fixed weir with the highest crest level is taken, because the crest level is used in the drying-flooding algorithm. In this way, overtopping of a weir the water level only occurs if the water level is above the weir with the highest crest level.

C.10 Calibration Factors

The calibration factor functionality is a multiplier for the rouhgness at different locations within the computational domain. Multiple formulation may be active in the same grid cell. Several keywords in the MDU file influence the functioning. All keywords below should be placed under the `[calibration]` section in the MDU file.

Keyword	Value	Description	Default
`UseCalibration`	1 or 0	Calibration factor option activated	N
`DefinitionFile`	<name.cld>	Calibration factor definition file	
`AreaFile`	<name.cll>	Calibration factor area file	

C.10.1 Calibration factor definition file (CLD-file)

The calibration class definition file contains lines of the following format defining the different calibration classes. It supports constant values and values that depend on the discharge at a named cross section or the water level at a named location.

C.10.1.1 Header of the CLD-file

The file starts with the following header

```
# [FileInformation]
#   FileType     = CalibrationFactorsDefinitionFile
#   FileVersion  = 1.0
# [CalibrationFactors]
```

C.10.1.2 Constant values

The format for constant values is as follows:

```
CalibrationClassNr  ConstantValue
```

where `ConstantValue` indicates the calibration value to be used (use 1.0 to use the uncalibrated roughness). The user must specify for each calibration class (line in this file) a unique calibration class number `CalibrationClassNr`. This calibration class number is used in the calibration area <.cll> file to indicate the area of influence of this class.

C.10.1.3 Discharge dependent format

The discharge dependent line is formatted as follows:

```
CalibrationClassNr DISCHARGE "Cross-section name"
CalibrationClassNr Q1 ConstantValueAtQ1
CalibrationClassNr Q2 ConstantValueAtQ2
CalibrationClassNr Q3 ConstantValueAtQ3
CalibrationClassNr Q4 ConstantValueAtQ4
...
```

which again expects a user defined calibration class number `CalibrationClassNr`. The first line contains the keyword `DISCHARGE` and subsequently a cross-section name occurring in the CrsFile in the mdu-file. The cross-section name can be enclosed by quotation marks, for instance when the name contains spaces. The subsequent lines all start with the same calibration class number as the first line. The list of discharges `Q1`, `Q2`, `Q3`, and `Q4` should be monotonically increasing. For each discharge value a calibration value should be specified. Between the discharges specified, the calibration values are linearly interpolated. Below the minimum and above the maximum discharge the first and last values are used respectively.

C.10.1.4 Water level dependent format

The water-level dependent format is similar to the discharge dependent format and is formatted as follows:

```
CalibrationClassNr WATERLEVEL "Observation-station name"
CalibrationClassNr ZS1 ConstantValueAtZS1
CalibrationClassNr ZS2 ConstantValueAtZS2
CalibrationClassNr ZS3 ConstantValueAtZS3
CalibrationClassNr ZS4 ConstantValueAtZS4
...
```

which again expects a user defined calibration class number `CalibrationClassNr`. The first line contains the keyword `WATERLEVEL` and subsequently an observation-station name occurring

DRAFT

in ObsFile in the mdu-file. The observation-station name can be enclosed by quotation marks, for instance when the name contains spaces. The subsequent lines all start with the same calibration class number as the first line. The list of water levels `ZS1`, `ZS2`, `ZS3`, and `ZS4` should be monotonically increasing. For each water level value a calibration value should be specified. Between the water levels specified, the calibration values are linearly interpolated. Below the minimum and above the maximum water level the first and last values are used respectively.

C.10.1.5 Example

The example file given below defines three calibration classes. In the area associated with the first class, the roughness values are used without any adjustment. In the area associated with the second class, the roughness value is decreased by 20%. In the area associated with the third and final class, the increase in the roughness value varies between 0% and 30% depending on the discharge at a specific location.

```
# [FileInformation]
#   FileType     = CalibrationFactorsDefinitionFile
#   FileVersion  = 1.0
# [CalibrationFactors]

# areas without any calibration adjustment
1 1.0

# areas with 20% decreased roughness value
2 0.8

# area with calibration depending on discharge at crsX
10 DISCHARGE 'crsX'
10 500  1.0
10 1000 1.1
10 3000 1.3
```

C.10.2 Calibration Class Area on Links (CLL-file)

C.10.2.1 Header of the CLL-file

The file starts with the following header

```
# [FileInformation]
#   FileType     = CalibrationAreasFile
#   FileVersion  = 1.0
# [CalibrationAreas]
```

C.10.2.2 Body of the CLL-file

The body of the CLL-file consists of comment lines (any line starting with # or *) or data lines of the following format:

```
xu yu zu CalibrationClassNr AreaFraction
```

where `xu`, `yu`, `zu` is the coordinate of the midpoint of the netlink, `CalibrationClassNr` is an integer corresponding to the number as described in the Calibration Class Definition File (cf. section C.10.1), and `AreaFraction` is an area fraction between 0.0 and 1.0.

All lines that list the same `xu`, `yu`, `zu` coordinates are combined to a weighted average of multiple calibration classes, provided the sum of the fractions does not add to a value greater than 1.0.

DRAFT

If the sum of the areas specified for a certain netlink is less than 1.0, then the a constant calibration factor of 1.0 is assumed for the remaining area (i.e. no calibration effect). The result is that if for a specific netlink no line is specified at all, then the roughness of this netlink will be unaffected by the calibration process.

C.10.2.3 Example

The example of the <CLL>-file is given below based on the <CLD>-file given in section C.10.1. In this case the roughness at netlink with midpoint located at `xu` = 10.542 and `yu` = 11.6 is completely affected by calibration class 1, the netlink with midpoint at `xu` = 11.120 and `yu` = 12.1 is affected for 60 % by calibration class 1 and 40 % by calibration class 10, and the final location listed uses only the value resulting from calibration class 10.

```
...
10.542 11.6 0 1 1.0
11.120 12.1 0 1 0.6
11.120 12.1 0 10 0.4
12.635 12.6 0 10 1.0
...
```

C.11 Sources and sinks

Warning:

- ◇ This section describes deprecated functionality using the old external forcings format. Use the new external forcings file for source sinks instead, see Section C.6.2.4.

Sources and sinks (section 6.4.10) are defined as part of the $<*$.ext$>$ file, as follows:

```
QUANTITY=discharge_salinity_temperature_sorsin
FILENAME=chan1_westeast.pli  # A file 'chan1_westeast.tim', with same basename
                             # as the polyline should be present
FILETYPE=9
METHOD  =1
OPERAND =O
AREA    =1.5
```

The $<*$.pli(z)$>$ file is a polyline file (section C.2) with two or three or five columns. 2D models only need two (x, y) columns. A 3D model with extraction and discharge in a single layer needs three (x, y, z) columns. It is also possible to extract and/or discharge into a range of vertical layers. This must be specified in a five-column file $(x, y, z_{\text{min}}, z_{\text{max}}, 0)$, where the fifth column is a dummy unused value and $z_{\text{min}}, z_{\text{max}}$ define the range of layers across which the discharge is averaged. Along with the $<*$.pli(z)$>$-file, there has to be a time series file (section C.4) with the same basename as the $<*$.pli(z)$>$-file, and extension $<$.tim$>$. The columns in the $<*$.tim$>$ are as follows: time in minutes, discharge Q in [m^3/s], and *all* constituents in the model. The order of consituents is: salinity in [ppt], temperature T in [°C], sediment concentrations, spiral flow intensity and tracers. For example, if we only have temperature and two tracers we get 5 columns: time, discharge, temperature, tracer 1, tracer 2. For more information, see section 6.4.10.

C.12 Dry points and areas

Dry points can either be defined by a basic sample file (see section C.3), or a polygon file (see section C.2).

Special attention must be given to the optional third column of a polygon file: when the first point has a z-value of -1, the polygon mask is inverted, i.e., all points outside the polygon are set as dry points.

More details can be found in section 6.4.2.7.

C.13 Structure INI file

Hydraulic structures are defined in a <structures.ini> file. Each structure is defined in its own `[Structure]` section, followed by a set of key-value parameters that depend on the type of structure:

Table C.13: *General part of structure file.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`3.01`	File version. Do not edit this.
`fileType`	String	`structure`	File type. Do not edit this.
`[Structure]`			
`id`	String		Unique structure id (max. 256 characters).
`name`	String		Given name in the user interface.
`branchId`	String		(optional) Branch on which the structure is located.
`chainage`	Double		(optional) Chainage on the branch [m].
`numCoordinates`	int		(optional) Number of values in `xCoordinates` and `yCoordinates`. This value should be greater or equal 2.
`xCoordinates`	Double		(optional) x-coordinates of the location of the structure. (number of values = `numCoordinates`)
`yCoordinates`	Double		(optional) y-coordinates of the location of the structure. (number of values = `numCoordinates`)
`locationFile`	String		Filename of a <*.pli> file containing x-/y-points of the structure's position.
`type`	String		Structure type. Possible values: ◇ `weir` (for specific input refer to C.13.1) ◇ `universalWeir` (for specific input refer to C.13.2) ◇ `culvert` (for specific input refer to C.13.3) ◇ `longCulvert` (for specific input refer to C.13.4) ◇ `bridge` (for specific input refer to C.13.5) ◇ `pump` (for specific input refer to C.13.6) ◇ `orifice` (for specific input refer to C.13.7) ◇ `gate` (for specific input refer to C.13.8) ◇ `generalStructure` (for specific input refer to C.13.9) ◇ `dambreak` (for specific input refer to C.13.10) ◇ `compound` (for specific input refer to C.13.11)
...			Structure type specific data

Remarks:

DRAFT

- ◇ When `branchId` and `chainage` are given, they will be used for the location specification (only for 1D).
- ◇ When `numCoordinates`, `xCoordinates` and `yCoordinates` are given, or when `locationFile` is given, the points will be used to define a polyline. All flow links crossing that polyline will fall under the influence of that hydraulic structure.
- ◇ The structure id/name may contain spaces, with no quoting needed.
- ◇ All structure types require a location specification as above, except for the compound structure. The latter should not have a location specification; it is inherited from its underlying structures (Section C.13.11).
- ◇ When direction is of importance (e.g., for structures with an `allowedFlowDir`, and for all flux quantities output), the `positive` direction is oriented as follows:
  - ◇ When placed on 1D branch, in the direction of the branch.
  - ◇ When placed via a polyline (`x/yCoordinates` or `locationFile`), the positive direction is oriented 90 degrees clockwise w.r.t. the polyline.
- ◇ The universal weir, culvert and bridge can only be used in a single 1D channel.
- ◇ All other structures will be split up over the intersecting flow links.
- ◇ When using <*.bc files> to control some structure parameters, the quantity names to be used in those files are described in Section C.5.3.

C.13.1 Weir

The data block for a weir structure in the <structures.ini> file is described in this section. The geometrical shape and hydraulic treatment is further described in Section 10.4.

Table C.14: *Weir definition.*

Keyword	Type	Default	Description
`[Structure]`			
`type`	String	`weir`	Structure type; must read `weir`.
`allowedFlowdir`	String	`both`	Possible values: `both`, `positive`, `negative`, `none`.
`crestLevel`	*		Crest level of weir [m AD].
`crestWidth`	Double	$\sum w_u$	Width of weir [m].
`corrCoeff`	Double	0.0	Correction coefficient [-].
`useVelocityHeight`	logical	true	Flag indicates whether the velocity height is to be calculated or not.

Remarks:

- ◇ Items marked with (*) can either be a scalar value, or a time series file (<*.bc>, see more details in Section C.13.12/(deprecated) <*.tim>) or `realtime`, to indicate that this parameter gets it values supplied by, e.g., realtime control.
- ◇ `crestWidth` is optional. Default is (the sum of) the width[s] of the flow link[s]. If the given value is larger than this sum, it is automatically lowered to this default.
- ◇ If the `crestWidth` of a 2d structure is smaller than the sum of the widths of the 2d links, the `crestWidth` of the individual structure elements is reduced symmetrical from the outside inwards.

C.13.2 Universal Weir

The data block for a universal weir structure in the <structures.ini> file is described in this section. The geometrical shape and hydraulic treatment is further described in Section 10.10.

DRAFT

Table C.15: *Universal Weir definition.*

Keyword	Type	Default	Description
`[Structure]`			
`type`	String	`universalWeir`	Structure type; must read `universalWeir`.
`allowedFlowDir`	String	`both`	Possible values: `both`, `positive`, `negative`, `none`.
`numLevels`	Int		Number of yz-Values.
`yValues`	Double[]		y-values of the cross section [m]. (number of values = `numLevels`)
`zValues`	Double[]		z-values of the cross section [m]. (number of values = `numLevels`)
`crestLevel`	Double		Crest level of weir [m AD].
`dischargeCoeff`	Double		Discharge coefficient c_e [-], as used in the formulas in Section 10.10.

Remark:

- ◇ The `zValues` should be considered as relative values, merely defining the shape of the cross section. The absolute z-values are determined by shifting the entire cross section such that the lowest point is exactly at `crestLevel`.

C.13.3 Culvert

The data block for a culvert structure in the <structures.ini> file is described in this section. The geometrical shape is described by the parameters in the following table and these refer to the symbols in Figure 10.8. The entrance and exit losses are further described in Section 10.7.

Table C.16: *Culvert*

Keyword	Type	Default	Description
`[Structure]`			
`type`	String	`culvert`	Structure type; must read `culvert`.
`allowedFlowdir`	String	`both`	Possible values: `both`, `positive`, `negative`, `none`.
`leftLevel`	Double		Left invert level of the culvert (z_{c1}) [m AD].
`rightLevel`	Double		Right invert level of the culvert (z_{c2}) [m AD].
`csDefId`	String		Id of cross section definition.
`length`	Double		Length (L) [m].
`inletLossCoeff`	Double		Inlet loss coefficient (ξ_i) [-].
`outletLossCoeff`	Double		Outlet loss coefficient (ξ_o) [-].
`valveOnOff`	Int		Flag for having valve or not (0=no valve, 1=valve).
`valveOpeningHeight`	Double		Valve opening height (h_{valve}) [m].
			Table for loss coefficients of valve.
`numLossCoeff`	Int		Number of rows in table.
`relOpening`	Double[]		Relative valve opening (0.0 — 1.0) [-]. (number of values = `numLossCoeff`)

(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
lossCoeff	Double[]		Loss coefficients (ξ_v) [-]. (number of values = numLossCoeff)
bedFrictionType	String		Friction type, possible values are listed in Section C.16.
bedFriction	Double		Friction Value.
subType	String	culvert	Defines the behaviour of the culvert. Possible values: culvert and invertedSiphon.
bendLossCoeff	Double		Bend loss coefficient (ξ_b) of the inverted siphon.

Remarks:

- Items marked with (*) can either be a scalar value, or a time series file (<*.bc>, see more details in Section C.13.12/(deprecated) <*.tim>) or realtime, to indicate that this parameter gets it values supplied by, e.g., realtime control.
- valveOpeningHeight is a relative height with respect of the invert level of the culvert. Thus, a positive value should be prescribed. **Note that D-Flow FM doesn't check on negative values. This may result into a crash of the simulation.**
- bendLosses is mandatory in case of an inverted siphon, but is not allowed in combination with a culvert.

C.13.4 Long culvert

The data block for a long culvert structure in the <structures.ini> file is described in this section. The geometrical shape is described by the parameters in the following table and these refer to the symbols in Figure 10.11. The geometrical details are further described in Section 10.8.1.

Table C.17: *Long culvert*

Keyword	Type	Default	Description
[Structure]			
type	String	longCulvert	Structure type; must read longCulvert.
id,numCoordinates, xCoordinates, yCoordinates			See Table C.13.
zCoordinates	Double		z-coordinates for the vertical placement of the structure. (number of values = numCoordinates) [m AD].
allowedFlowdir	String	both	Possible values: both, positive, negative, none.
width	Double		Width of the culvert's cross section shape [m].
height	Double		Height of the culvert's cross section shape [m].
frictionType	String		Roughness type associated with this cross section (see section C.16 for encoding).
frictionValue	Double		Roughness value; its meaning depends on the roughness type selected (only used if frictionType specified).
valveRelativeOpening	*	1.0	Relative valve opening $[0.0, 1.0]$ [-].

Remarks:

- ◇ Items marked with (*) can either be a scalar value, or a time series file (<*.bc>, see more details in Section C.13.12/(deprecated) <*.tim>) or `realtime`, to indicate that this parameter gets it values supplied by, e.g., realtime control.
- ◇ The location specification for a long culvert is only supported via x/yCoordinates or `locationFile`, and these should define the actual placement of the pipe in positive flow direction (i.e., not perpendicular to the flow direction).
- ◇ When the optional friction parameters are not set, they default to the global MDU-settings `UnifFrictType` and `UnifFrictCoef1D`.
- ◇ `valveRelativeOpening` is *different* from the normal culvert's `relOpening`. The former directly reduces the discharge through a long culvert, with respect to the fully open state.

C.13.5 Bridge

The data block for a bridge structure in the <structures.ini> file is described in this section. The geometrical shape and hydraulic treatment is further described in Section 10.9.

Table C.18: *Bridge Structure definition.*

Keyword	Type	Default	Description
`[Structure]`			
`type`	String	`bridge`	Structure type; must read `bridge`.
`allowedFlowdir`	String	`both`	Possible values: `both`, `positive`, `negative`, `none`.
`Pillar definition:`			
`pillarWidth`	Double		Total width of pillars in cross direction [m].
`formFactor`	Double		Shape factor [-].
`Abutment definition:`			
`csDefId`	String		Id of Cross-Section Definition.
`shift`	Double		Vertical shift of the cross section definition [m]. Defined positive upwards.
`inletLossCoeff`	Double		Inlet loss coefficient [-], ξ_i in Equation (10.58).
`outletLossCoeff`	Double		Outlet loss coefficient [-], k in Equation (10.59).
`frictionType`	String		Friction type, possible values are listed in Section C.16.
`friction`	Double		Friction Value, used in friction loss in Equation (10.60).
`length`	Double		Length [m], L in Equation (10.60).

Remarks:

- ◇ At least one of the sections "Pillar definition" and "Abutment definition" must be present, but the other is optional.
- ◇ In case the "Pillar definition" is provided the effect of pillars are taken into account. In that case all of the items in the "Pillar definition" section are required parameters
- ◇ In case the "Abutment definition" is provided, the bridge results to a bridge with an own cross section definition. In that case all of the items in the "Abutment definition" section are required parameters
- ◇ The bridge treatment is equivalent to the one known from an *abutment bridge*, see Section 10.9 for details.
- ◇ The cross section is suggested to be an open cross section (that is, without bridge deck), but it can be of any type, because it is merely used to compute the wet area and perimeter.
- ◇ The `friction` and `frictionType` values are required in case of an abutment bridge, and will override the cross section's own roughness.

DRAFT

C.13.6 Pump

The data block for a pump structure in the <structures.ini> file is described in this section. The pump staging and triggering, and its hydraulic treatment is further described in Section 10.11.

Table C.19: *Pump definition.*

Keyword	Type	Default	Description
[Structure]			
type	String	pump	Structure type; must read pump.
orientation	String	positive	Pump orientation. Possible values: positive, negative.
controlSide	String		On which side[s] built-in pump triggering is active. *Only required/used when numStages > 0.* Possible values: ◇ suctionSide: Suction side control. ◇ deliverySide: Delivery side control. ◇ both: Suction and Delivery side control.
numStages	Int	0	Number of trigger stages in pump. Set to 0 to disable built-in triggering.
capacity	*		Pump capacity [m^3/s]. (number of values = max(1, numStages))
startLevelSuctionSide	Double[]		Start level suction side [m AD]. (number of values = numStages)
stopLevelSuctionSide	Double[]		Stop level suction side [m AD]. (number of values = numStages)
startLevelDeliverySide	Double[]		Start level at delivery side [m AD]. (number of values = numStages)
stopLevelDeliverySide	Double[]		Stop level at delivery side [m AD]. (number of values = numStages)
numReductionLevels	Int	0	Number of levels in reduction table.
head	Double[]		Head, the difference in waterlevel (delivery side - suction side) [m]. (number of values = numReductionLevels)
reductionFactor	Double[]		Reduction factor [-]. (number of values = numReductionLevels)

Remarks:

- ◇ Items marked with (*) can either be a scalar value, or a time series file (<*.bc>, see more details in Section C.13.12/(deprecated) <*.tim>) or realtime, to indicate that this parameter gets it values supplied by, e.g., realtime control.
- ◇ A pump's orientation defines the direction of positive capacity with respect to the spatial orientation. For example, when a pump is pumping against a network branch's direction, it may be convenient to set orientation = negative and use positive capacity values (as opposed to using negative capacity values).
- ◇ Built-in triggering based on stages can not be combined with time series or controlling by RTC. Therefore, if the capacity is a time series filename or is controlled by RTC, the number of stages (numStages) may only be equal to 0.
- ◇ When built-in triggering is on (numStages > 0), then all capacity values must be nonnegative (use orientation when needed). On the other hand, a (non-triggered) capacity as a scalar, or timeseries, or via RTC may be negative.
- ◇ The discharge of a 2D pump is equally distributed over the different flow links.

DRAFT

C.13.7 Orifice

The data block for an orifice structure in the <structures.ini> file is described in this section. The geometrical shape and hydraulic treatment is further described in Section 10.6.

Table C.20: *Orifice definition.*

Keyword	Type	Default	Description
`[Structure]`			
`type`	String	`orifice`	Structure type; must read `orifice`.
`allowedFlowdir`	String	`both`	Possible values: `both`, `positive`, `negative`, `none`.
`crestLevel`	*		Crest level [m AD]
`crestWidth`	Double	$\sum w_u$	Crest width [m]
`gateLowerEdgeLevel`	*		Position of gate door's lower edge. [m AD]
`corrCoeff`	Double	0.0	Correction coefficient [-]
`useVelocityHeight`	logical	`true`	Flag indicates whether the velocity height is to be calculated or not.
`useLimitFlowPos`	logical	`false`	Flag for setting the flow limiter for positive flow to either unlimited or limited (`false` = unlimited, `true` = limited).
`limitFlowPos`	Double		Maximum positive flow [m^3/s], only applied when `uselimitflowpos` is `true`.
`useLimitflowNeg`	logical	`false`	Flag for setting the flow limiter for negative flow to either unlimited or limited (`false` = unlimited, `true` = limited).
`limitFlowneg`	Double		Maximum negative flow [m^3/s], only applied when `uselimitflowneg` is `true`.

Remarks:

- ◇ Items marked with (*) can either be a scalar value, or a time series file (<*.bc>, see more details in Section C.13.12/(deprecated) <*.tim>) or `realtime`, to indicate that this parameter gets it values supplied by, e.g., realtime control.
- ◇ `crestWidth` is optional. Default is (the sum of) the width[s] of the flow link[s]. If the given value is larger than this sum, it is automatically lowered to this default.
- ◇ If the `crestWidth` of a 2d structure is smaller than the sum of the widths of the 2d links, the `crestWidth` of the individual structure elements is reduced symmetrical from the outside inwards.
- ◇ The maximum flow limitation by settings `(use)LimitflowPos` and `(use)LimitflowNeg` is further described in Table 10.6.

C.13.8 Gate

The data block for a gate structure in the <structures.ini> file is described in this section.

Table C.21: *Gate definition.*

Keyword	Type	Default	Description
`[Structure]`			
`type`	String	`gate`	Structure type; must read `gate`.
`crestLevel`	*		Crest level. [m AD]

(continued on next page)

DRAFT

Keyword		Default	Description
			(continued from previous page)
crestWidth	Double	$\sum w_u$	Crest width. [m]
gateLowerEdgeLevel	*		Position of gate door's lower edge. [m AD]
gateHeight	Double		Height of gate door. Needed for possible overflow across door. [m]
gateOpeningWidth	*	0.0	Opening width between gate doors, should be smaller than (or equal to) crestWidth. Use 0.0 for a vertical door. [m]
gateOpeningHorizontal Direction	String	symmetric	Horizontal opening direction of gate door[s]. Possible values are: symmetric, fromLeft, fromRight.

Remarks:

- ◇ Items marked with (*) can either be a scalar value, or a time series file (<*.bc>, see more details in Section C.13.12/(deprecated) <*.tim>) or realtime, to indicate that this parameter gets it values supplied by, e.g., realtime control.
- ◇ crestWidth is optional. Default is (the sum of) the width[s] of the flow link[s]. If the given value is larger than this sum, it is automatically lowered to this default.
- ◇ If the crestWidth of a 2d structure is smaller than the sum of the widths of the 2d links, the crestWidth of the individual structure elements is reduced symmetrical from the outside inwards.

C.13.9 General Structure

The data block for a general structure in the <structures.ini> file is described in this section. The geometrical shape is described by the parameters in the following table and these refer to the symbols in Figures 10.2 and 10.3. The hydraulic treatment is further described in Section 10.3.

For some of the keywords no default value has been specified. Then, the computational kernel of D-Flow FM uses a value of 0.0 for these keywords. However, some of these keywords are levels (upstream1Level, upstream2Level, downstream1Level, downstream2Level), which means that these levels are at the reference level, which is most likely an inaccurate value. Therefore, it is important that a user pays attention to this topic of specifying values for these upstream and downstream energy levels.

Table C.22: *Specific input for the general structure.*

Keyword	Type	Default	Description
[Structure]			
type	String	generalStructure	Structure type; must read generalStructure.
allowedFlowdir	String	both	Possible values: both, positive, negative, none.
upstream1Width	Double	10.0	w_{u1} [m]
upstream1Level	Double	0.0	z_{u1} [m AD]
upstream2Width	Double	10.0	w_{u2} [m]
upstream2Level	Double	0.0	z_{u2} [m AD]
crestWidth	Double	10.0	w_s [m]
crestLevel	*	0.0	z_s [m AD]

(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
crestLength	Double	0.0	The crest length across the general structure. When the crest length $>$ 0, the extra resistance for this structure will be $l_s \cdot g/(C^2 \cdot \mathrm{waterdepth})$. [m]
downstream1Width	Double	10.0	w_{d1} [m]
downstream1Level	Double	0.0	z_{d1} [m AD]
downstream2Width	Double	10.0	w_{d2} [m]
downstream2Level	Double	0.0	z_{d2} [m AD]
gateLowerEdgeLevel	*	11.0	Position of gate door's lower edge. [m AD]
posFreeGateFlowCoeff	Double	1.0	Positive free gate flow corr.coeff. c_{gf} [-].
posDrownGateFlowCoeff	Double	1.0	Positive drowned gate flow corr.coeff. c_{gd} [-].
posFreeWeirFlowCoeff	Double	1.0	Positive free weir flow corr.coeff. c_{wf} [-].
posDrownWeirFlowCoeff	Double	1.0	Positive drowned weir flow corr.coeff. c_{wd} [-].
posContrCoefFreeGate	Double	1.0	Positive gate flow contraction coefficient μ_{gf} [-].
negFreeGateFlowCoeff	Double	1.0	Negative free gate flow corr.coeff. c_{gf} [-].
negDrownGateFlowCoeff	Double	1.0	Negative drowned gate flow corr.coeff. c_{gd} [-].
negFreeWeirFlowCoeff	Double	1.0	Negative free weir flow corr.coeff. c_{wf} [-].
negDrownWeirFlowCoeff	Double	1.0	Negative drowned weir flow corr.coeff. c_{wd} [-].
negContrCoefFreeGate	Double	1.0	Negative gate flow contraction coefficient μ_{gf} [-].
extraResistance	Double	0.0	Extra resistance [-].
gateHeight	Double	1d10	
gateOpeningWidth	*	0.0	Opening width between gate doors, should be smaller than (or equal to) crestWidth. [m]
gateOpening↩ HorizontalDirection	String	symmetric	Horizontal opening direction of gate door[s]. Possible values are: symmetric, fromLeft, fromRight.
useVelocityHeight	logical	true	Flag indicates whether the velocity height is to be calculated or not.

Remarks:

- ◇ For an extensive description on these parameters see Section 10.3.
- ◇ Items marked with (*) can either be a scalar value, or a time series file (<*.bc>, see more details in Section C.13.12/(deprecated) <*.tim>) or realtime, to indicate that this parameter gets it values supplied by, e.g., realtime control.
- ◇ crestWidth is optional. Default is (the sum of) the width[s] of the flow link[s]. If the given value is larger than this sum, it is automatically lowered to this default.
- ◇ If the crestWidth of a 2d structure is smaller than the sum of the widths of the 2d links, the crestWidth of the individual structure elements is reduced symmetrical from the outside inwards.

C.13.10 Dambreak

The data block for a dambreak in the <structures.ini> file is described in this section. A dambreak is not really a hydraulic structure, but it has so many similarities with hydraulic structure input that it is also placed in the same structure file. The dam break behavior and hydraulic treatment is further described in Section 10.2.

DRAFT

Table C.23: *Specific input for the dambreak.*

Keyword	Type	Default	Description
`[Structure]`			
`type`	String	`dambreak`	Structure type; must read `dambreak`.
`startLocationX`	Double		x-coordinate of breach starting point.
`startLocationY`	Double		y-coordinate of breach starting point.
`algorithm`	Int		Breach growth algorithm. Possible values are: ◇ `1`: van der Knaap (2000) ◇ `2`: Verheij–van der Knaap (2002) ◇ `3`: Predefined time series, see `dambreakLevelsAndWidths`
`crestLevelIni`	Double		Initial breach level $z_{\text{crest level}}$ [m AD].
`breachWidthIni`	Double		Initial breach width B_0 [m].
`crestLevelMin`	Double		Minimal breach level z_{min} [m AD].
`t0`	Double		Breach start time T_{start} [s].
`timeToBreachToMaximumDepth`	Double		$t_{\text{Phase 1}}$ [s].
`f1`	Double		Factor f_1 [-].
`f2`	Double		Factor f_2 [-].
`uCrit`	Double		Critical flow velocity u_c for erosion [m/s].
`waterLevelUpstreamLocationX`	Double		(optional) x-coordinate of custom upstream water level point.
`waterLevelUpstreamLocationY`	Double		(optional) y-coordinate of custom upstream water level point.
`waterLevelDownstreamLocationX`	Double		(optional) x-coordinate of custom downstream water level point.
`waterLevelDownstreamLocationY`	Double		(optional) y-coordinate of custom downstream water level point.
`waterLevelUpstreamNodeId`	String		(optional) Node Id of custom upstream water level point.
`waterLevelDownstreamNodeId`	String		(optional) Node Id of custom downstream water level point.
`dambreakLevelsAndWidths`	String		(only when `algorithm=3`) Filename of <*.tim> file (Section C.4) containing the breach levels and widths.

Remarks:

- ◇ The location specification for a dambreak currently only supports polyline coordinates via `x/yCoordinates` or `locationFile` (see Table C.13).
- ◇ On the same location as the dambreak polyline, a fixed weir polyline must be present (see Section C.9).
- ◇ The dambreak supports 2D grids, and also has basic support for 1D2D connections. For a 1D river with lateral couplings to 2D flood plains, the polyline must lie alongside the river, intersecting the 1D2D connections. Currently, the resolution of the 1D network must be similar to the 2D grid resolution, i.e., typically one 1D2D link per 2D grid cell.
- ◇ Breach start time `t0` is considered relative to the model's `[time] RefDate` (see Chapter A).
- ◇ `crestLevelIni` is also required for time series dambreaks.
- ◇ When `algorithm=1`, Van der Knaap(2000), currently only the default material type clay is supported. This means that the related coefficients in Equation (10.13) are for clay (see Table 10.3).

DRAFT

- ◇ When `algorithm=3`, the time series provided in the filename via `dambreakLevelsAndWidths` in minutes (also see Section C.4) are considered relative to breach start time `t0` (which is in in seconds).
- ◇ When `algorithm=2`, Verheij–van der Knaap (2002), the up- and downstream water levels h_{up} and h_{down} are computed as the average water levels for all grid cell[s] upstream or downstream of the breach gap, respectively.
  Alternatively, users can specify two points so that the water levels at these points are used for h_{up} and h_{down}. (This solves any unwanted dependency on local grid cell size.) These two points can be provided either by node Ids or coordinates. For instance, the point used for upstream water level can be given by `waterLevelUpstreamNodeId` or `waterLevelUpstreamLocationX`. (same for `Down`).

A example of a timfile for a dambreach is:

```
*  Time breachlevel breachwidth
      0     18.5          0
     60     17.5         20
    120     16.5         50
    240     15.0        100
    480     15.0        200
   2880     15.0        200
* end input
```

C.13.11 Compound Structure

The data block for a compound structure in the <structures.ini> file is described in this section. A compound structure consists of a number of structures defined in the structure file that are combined to one compound structure. The hydraulic treatment is further described in Section 10.12.

Table C.24: *Specific input for the compound structure.*

Keyword	Type	Default	Description
`[Structure]`			
`type`	String	`compound`	Structure type; must read `compound`.
`numStructures`	int		Number of individual structures in compound structure.
`structureIds`	String		Semicolon separated list of structure ids.

Remarks:

- ◇ `structureIds` is a semicolon separated list, where leading and trailing spaces will be trimmed from each structure id in the list.
- ◇ `structureIds` can contain spaces in the individual structure ids, no quotation is needed.
- ◇ A compound structure should *not* contain a location specification. This will be retrieved from the individual structures.
- ◇ A compound structure *does* require its own `id` and `name` fields, because the output in the history file will also contain total output for the compound's underlying structures together and this will appear under its own compound `id`.
- ◇ All individual structures in a compound structure must be snapped to exactly the same flow link[s].
- ◇ The referenced individual structures may appear anywhere in the structure file.

The structure INI file is the preferred format for hydraulic structure input, and supersedes some of the old structure quantity names in Table C.9 under "Structure parameters".

```
[Structure]
type                  = weir                # Type of structure
```

DRAFT

```
id                    = weir01              # Name of the structure
locationFile          = weir01.pli          # *.pli; Polyline location for structure
crestLevel            = weir01_crest.bc    # Crest level in [m]

[Structure]
type                  = gate                # Type of structure
id                    = gate01              # Name of the structure
locationFile          = gate01.pli          # *.pli; Polyline location for structure
crestLevel            = gate01_crest.bc     # Crest level in [m]
crestWidth            = 39.5                # (Optional) Crest width in [m], between
                                            # the fixed side walls.
gateHeight            = 8.54                # Gate door height in [m], used for
                                            # detecting flow across door.
gateLowerEdgeLevel    = gate01_ledge.bc     # Position of gate's lower edge in [m],
                                            # used for flow underneath door.
gateOpeningWidth      = gate01_opening.bc   # (Optional) opening width between two
                                            # sideways closing gate doors.
gateHorizontalOpeningDirection = from_left/from_right/symmetric # (Optional) Opening
                                            # direction of the gate door(s).

[Structure]
type                  = pump                # Type of structure
id                    = pump01              # Name of the structure
locationFile          = pump01.pli          # *.pli; Polyline location for structure
capacity              = pump01_cap.bc       # Pumping capacity in [m3/s]
```

C.13.12 Structure defined with a time series

Some quantities of a certain structure can be given as time dependent values, using a time series. We use an example of defining a pump's capacity as a time series, following the steps below:

- ◇ 1: in the structure INI file (which was specified as `StructureFile` in the MDU file), when defining a structure in one `[Structure]` block, specify a BC file as value of the quantity keyword. For example, the following `[Structure]` block defines a pump, whose capacity is specified by `structureforcing.bc` file.

```
[Structure]
id                    = Pump_bc_tim
name                  = Pump_bc_tim
branchId              = T3_B1
chainage              = 1050.000
type                  = pump
capacity              = structureforcing.bc
```

- ◇ 2: In the specified BC file, when defining a `[forcing]` block, keyword `name` should have the same name as the `id` of the structure in the structure <*.ini> file. Keyword `function` should be given as `timeSeries`. Keyword `quantity` should have a quantity name, which must be equal to one of the quantity names in Table C.25, other quantity names will not be recognized. Using the pump example again, to specify a time series in the corresponding *.bc file, i.e. file `structureforcing.bc`, the `[forcing]` block can be defined as below.

```
[forcing]
name                  = Pump_bc_tim
function              = timeseries
time-interpolation    = linear
quantity              = time
unit                  = minutes since 2010-01-01 00:00:00
quantity              = pump_capacity
unit                  = m3 s-1
0 0
60 0
65 1
120 1
125 2.5
180 2.5
240 0
1440 0
```

Remark:

DRAFT

- Quantities of structures that can be specified as a time series are listed in Table C.25. Keyword `quantity` in the *.bc file must be exactly equal to one of the quantities in Table C.25. Otherwise it cannot be recoganized.

Table C.25: *Quantities that can be set by a time series.*

structure	quantity keywords in the .bc file
`pump`	`pump_capacity`
`culvert`	`culvert_valveOpeningHeight`
`weir`	`weir_crestLevel`
`orifice`	`orifice_crestLevel`, `orifice_gateLowerEdgeLevel`
`longCulvert`	`culvert_valveRelativeOpening`
`gate`	`gate_crestLevel`, `gate_gateLowerEdgeLevel`, `gate_gateOpeningWidth`
`generalStructure`	`general_structure_crestLevel`, `general_structure_gateLowerEdgeLevel`, `general_structure_gateOpeningWidth`

C.13.13 File version history

Below are the most important file versions that have existed for the structure INI file. File versions are formatted as *<major>.<minor>*. Major version changes indicate that older versions of the software are no longer readable. Minor version changes indicate that the software can still read the older (minor) file versions.

- 3.01: Rename `polylineFile` to `locationFile` for location specification.
  Note: when doing this renaming in an existing input file, all changes listed below between versions 2.00 and 3.00 must also be made.
  Also, add new `type=gate`.
- 3.00: Remove bridge field `bedLevel`, superseded by `shift`.
- 2.01: Add new types `weir`, `universalWeir`, `culvert`, `bridge`, `pump`, `orifice`, `generalStructure`, `longCulvert`. Remove weir field keyword `lat_contr_coeff`, superseded by `corrCoeff` (cf. Table 10.4).
- 2.00: Add `branchId+chainage` or `numCoordinates/xCoordinates/yCoordinates` for location specification.
  Note: as of this version, using the old keyword `polylineFile` *will disable the reading of all new keywords and fall back to the 1.00 version reading in D-Flow FM itself.*
- 1.00: Consistent renaming of variables and attribute names. For pump: renamed `nrStages` to `numStages`, `reductionFactorLevels` to `numReductionLevels`; for culvert: renamed `lossCoeffCount` to `numLossCoeff`; for universal weir: renamed `levelsCount` to `numLevels`.
- Unversioned ("0.00"): First INI-formatted version of structures.ini for D-Flow FM. No `[General]` header and location specification via `polylinefile` only. Offer a subset of structures for 2D3D modelling only.

Warnings:

- Upgrading the structure file from Unversioned or version 1.0 to the most recent version can result in different results for the general structure. The main reasons are:
  - In versions up to 1.0, an extra friction loss on the crest is added and the crest length is taken as the distance between the two adjoining water level points. In version 3.0, the user can explicitly define the crest length (a value of 0 means no extra friction loss is applied).
  - A general structure with doors that block part of the waterway is solved in a different way in version 3.0. This may lead to significant differences in results compared to earlier versions.

DRAFT

- ◇ Structure file version 3.0 does not yet support 3D applications.
- ◇ Although version 1.0 is planned to be phased out, users are currently advised not to change structure file versions for 2D/3D applications. Continue using the Unversioned or 1.0 format in these cases.
- ◇ For 1D/2D applications, users are advised to adopt the 3.0 format. This version is more consistent with the SOBEKs and supports culverts, staged pumps and bridges, which are not supported in version 1.0.

C.14 Space varying wind and pressure

Warning:

- ◇ This functionality contains bugs and therefore the user is advised to check the space varying wind and pressure via the map output file. Some meteo files may contain metadata for custom grid orientations and when misinterpreted, this can cause significantly different results. Compare a wind and pressure field, at a certain time stamp on the map file, with the wind and pressure field you supplied at that time stamp. If the space varying wind and pressure appears to be wrong for your application, then please contact Delft3D Support.

In many cases the space varying wind data is provided as gridded data based on meteorological stations or meteorological forecast models. This data is often defined on a different grid than the computational grid used in D-Flow FM. Translating these files into files defined on the grid of the computational engine is often a lengthy process and can result in huge files. This feature facilitates the reading of the meteorological data on its own grid and interpolates the data internally to the grid of D-Flow FM. D-Flow FM can handle four types of space-varying meteorological input:

1 Time- and space-varying wind on an equistant grid.
2 Time- and space-varying wind on a curvilinear grid.
3 Time- and space-varying wind on a Spiderweb grid.
4 Time- and space-varying wind in a netCDF file.

C.14.1 Meteo on equidistant grids

File contents	Time-series of a space varying wind and atmospheric pressure defined on an equidistant (Cartesian or spherical) grid.
File format	Free formatted or unformatted, keyword based.
Generated	Some offline program.

Remark:

- ◇ The keywords are case insensitive.

Header description for the wind velocity files:

Keywords	**Value**	**Description**
`FileVersion`	1.03	version of file format
`Filetype`	meteo_on_equidistant_grid	meteo input on equidistant grid
`NODATA_value`	*free*	value used for input that is to be neglected
`n_cols`	*free*	number of columns used for wind datafield
`n_rows`	*free*	number of rows used for wind datafield
`grid_unit`	m *or* degree	unit of distances on the grid in both x- and y-direction

Keywords	Value	Description
x_llcorner	*free*	x-coordinate of lower left corner of lower left grid cell (in units specified in grid_unit
y_llcorner	*free*	y-coordinate of lower left corner of lower left grid cell (in units specified in grid_unit
x_llcenter	*free*	x-coordinate of centre of lower left grid cell (in units specified in grid_unit
y_llcenter	*free*	y-coordinate of centre of lower left grid cell (in units specified in grid_unit
dx	*free*	gridsize in x-direction in units specified in grid_unit
dy	*free*	gridsize in y-direction in units specified in grid_unit
n_quantity	1	number of quantities specified in the file
quantity1	x_wind *or* y_wind	the velocity component given in unit unit1
unit1	m s-1	unit of quantity1: metre/second

The user must specify the location of the equidistant grid on which the meteorological data is specified. While the user can specify the meteorological grid by means of grid cell's corners or grid cell's centres, the meteorological data is always specified in the cell centre. If one has the location of the lower left corner of the lower left grid cell, one can specify the starting point of the grid using keywords x_llcorner and y_llcorner. If one has the location of the cell centre of the lower left grid cell, one should use the keywords x_llcenter and y_llcenter. Using the first option, the first data value is placed at (x_llcorner$+\frac{1}{2}dx$, y_llcorner$+\frac{1}{2}dy$), which is the cell centre of cell (1,1). Using the latter option, the first data value is placed at (x_llcenter, y_llcenter), which is again the cell centre of cell (1,1), i.e. the data values are always placed at the cell centres of the meteorological grid. Note that the lower left grid cell is defined to be the grid cell with index (1,1). When using the option of meteorological data on a separate curvilinear grid, the origin and orientation of the data set can be chosen freely with respect to the grid on which it is specified, see section C.14.2 for details.

Time definition and data block description for the wind velocity files

Keywords	Value	Description
Time	*fixed format described below*	time definition string

The time definition string has a fixed format, used to completely determine the time at which a dataset is valid. The time definition string has the following format:

TIME *minutes/hours since* YYYY-MM-DD HH:MM:SS TIME ZONE, e.g.

```
360 minutes since 2008-07-28 10:55:00 +01:00
```

DRAFT

The format of the string is completely fixed. No extra spaces or tabs can be added between the different parts of the definition. The time definition is followed by the datablock of input values corresponding to the specified time. The data block contains values for the wind velocity in x- or y-direction for *n_cols* by *n_rows* points, starting at the top left point. The time definition and the data block are repeated for each time instance of the time-series.

The atmospheric pressure file

The header for the atmospheric pressure is similar to that of the wind velocity files, except for the following differences.

Keywords	Value	Description
`quantity1`	air_pressure	air pressure
`unit1`	Pa *or* mbar	unit of `quantity1`: Pascal or millibar

The specification of the time definition and the data block is fully conform the wind velocity files.

File version and conversion

The current description holds for `FileVersion` 1.03. The table below shows the latest modifications in the file format (and version number).

FileVersion	Modifications
`1.03`	Use of keyword `Value_pos` to indicate the position of the lower left corner of the grid replaced by use of the combination of keywords: `x_llcorner` and `y_llcorner` or `x_llcenter` and `y_llcenter`
`1.02`	No changes for this meteo input type, but for the meteo type *meteo_on_spiderweb_grid*
`1.01`	Changed keyword `MeteoType` to `FileType` Changed fixed value of input type (Keyword `Filetype`) from *ArcInfo* to *meteo_on_equidistant_grid*

Restrictions:

- ◇ The contents of the file will not be checked on its domain.
- ◇ Keywords are followed by an equal sign '=' and the value of the keyword.
- ◇ When a keyword has value *free*, the value of this keyword is free to choose by the user. When only one value is given for a keyword, this keyword has a fixed value and when 2 or more options are shown, the user can choose between those values.
- ◇ Times must be specified exactly according to the time definition. See the examples shown in this section.
- ◇ The atmospheric pressure file must use the same grid definition and time frame as the files for the wind velocity components.
- ◇ The unit of the meteo grid must be the same as the computational grid, i.e. both with `grid_unit` = [m] or both with `grid_unit` = [degree].
- ◇ Input items in a data block are separated by one or more blanks.
- ◇ The wind components are specified at the cell centres (water level points) of the numerical grid.
- ◇ The wind components are specified in the west-east (`x_wind`) and south-north directions (`y_wind`).

Remarks:

DRAFT

- ◇ The time definition in the meteo files contains the number of minutes or hours since a reference date and time in a certain time zone. The reference time and time zone may differ from those of the simulation. During a simulation the computational engine will search in the meteo file for the current simulation time and interpolate between neighbouring times if necessary. Possible differences in time zone will be accounted for by shifting the meteo input data. The reference times within the time definition string may vary in a meteo file, i.e. it is possible to attach new input with a different reference time, behind the last data block. Consecutive times must always be increasing in the input file.
- ◇ Comments can be added after pound signs (#). These are not read.

Example of a file containing wind in x-direction (west-east)

The data blocks in this example are the result of the following FORTRAN statements:

```
do j = nrows,1,-1
   write(out,*) (xwind(i,j),i=1,ncols)
enddo
```

The x-wind velocity file for a 3 (n_cols) by 4 (n_rows) grid has the following layout:

```
FileVersion      =    1.03
filetype         =    meteo_on_equidistant_grid
NODATA_value     =    -999.000
n_cols           =    3
n_rows           =    4
grid_unit        =    degree
x_llcenter       =    -12.000
y_llcenter       =     48.000
dx               =    0.12500
dy               =    0.083333333
n_quantity       =    1
quantity1        =    x_wind
unit1            =    m s-1
TIME =      0.0 hours since 2008-01-15 04:35:00 +00:00
2   3.0  3.6
3   4.5  2
2.2 1    2.3
1.2 0.7 -0.4
TIME =      6.0 hours since 2008-01-15 04:35:00 +00:00
-1.1 -2.3 -3.6
-3.2  0.8  1.1
 2.2 -1   -1.6
 1.2 -0.7 -0.4
```

This results in an x-component of wind velocity given - in [m/s] - on a spherical, 3 by 4, equidistant grid, with grid sizes given by *dx* and *dy* (in degrees) and where the centre point of the lower left cell of the grid lies in (longitude, latitude) (-12.0, 48.0) on the globe. Data is given at two times: 0 and 6 hours since January 15th, 2008, 4:35 AM, in UTC+0.

Remark:

- ◇ The layout of the data blocks is from north to south (whereas most of the other files in Delft3D-FLOW, such as the curvilinear grid file, are ordered from south to north).
  Usually the signal corresponds to Mean Sea Level. One actually wants to prescribe an input signal corresponding to the local pressure prescribed by the space varying meteo input. To this end, it is possible to specify an average pressure - which should correspond to your input signal on the open boundaries - which is then used to determine local pressure gradients that need to be applied along the open boundaries to obtain an input signal that is consistent with the local atmospheric pressure. Using this keyword one can specify an average pressure that is used on all open boundaries, independent of the type of wind input. The pressure must be specified in N/m^2. An example:

DRAFT

C.14.2 Curvilinear data

File contents	Time-series of a space varying wind and atmospheric pressure defined on a curvilinear (Cartesian or spherical) grid.
File format	Free formatted or unformatted, keyword based.
Generated	Some offline program.

Remark:

- ◇ The keywords are case insensitive.

Header description for the wind velocity files:

Keywords	Value	Description
`FileVersion`	1.03	version of file format
`Filetype`	meteo_on_curvilinear_grid	meteo input on curvilinear grid
`NODATA_value`	*free*	value used for input that is to be neglected
`grid_file`	*free*.grd	name of the curvilinear grid file on which the data is specified
`first_data_value`	grid_llcorner *or* grid_ulcorner *or* grid_lrcorner *or* grid_urcorner	*see example below*
`data_row`	grid_row *or* grid_column	*see example below*
`n_quantity`	1	number of quantities specified in the file
`quantity1`	x_wind *or* y_wind	the velocity component given in unit `unit1`
`unit1`	m s-1	unit of `quantity1`: metres/second

Time definition and data block description for the wind velocity files

For a description of the time definition and data block see section C.14.1.

The atmospheric pressure file

For a description of the atmospheric file see section C.14.1.

DRAFT

File version and conversion

The current description holds for `FileVersion` 1.03. The table below shows the latest modifications in the file format (and version number).

FileVersion	Modifications
`1.03`	Fixed bug in interpolation of data from meteo grid to computational grid: Conversion script mirrored data set erroneously. This was treated correctly by meteo module. Fixed both the conversion script and the meteo module together: Required modification in meteo input file: Change `first_data_value` = grid_llcorner into grid_ulcorner or vice versa or Change `first_data_value` = grid_lrcorner into grid_urcorner or vice versa
`1.02`	No changes for this meteo input type, but for the meteo type *meteo_on_spiderweb_grid*
`1.01`	Changed keyword `MeteoType` to `FileType` Changed keyword `Curvi_grid_file` to `Grid_file` Changed fixed value of input type (Keyword `Filetype`) from *Curvi* to *meteo_on_curvilinear_grid*

Restrictions:

- ◇ The restrictions for space varying wind and pressure on a separate curvilinear grid are the same as for space varying wind and pressure on an equidistant grid, described in section C.14.1. A differerence is that the data values on the curvilinear grid are not specified in the cell centres, but in the grid points (cell corners).
- ◇ The unit of the meteo grid must be the same as the computational grid, i.e. both with `grid_unit` = [m] or both with `grid_unit` = [degree].

Remark:

- ◇ The remarks for space varying wind and pressure on a separate curvilinear grid are the same as for space varying wind and pressure on an equidistant grid, described in section C.14.1.

DRAFT

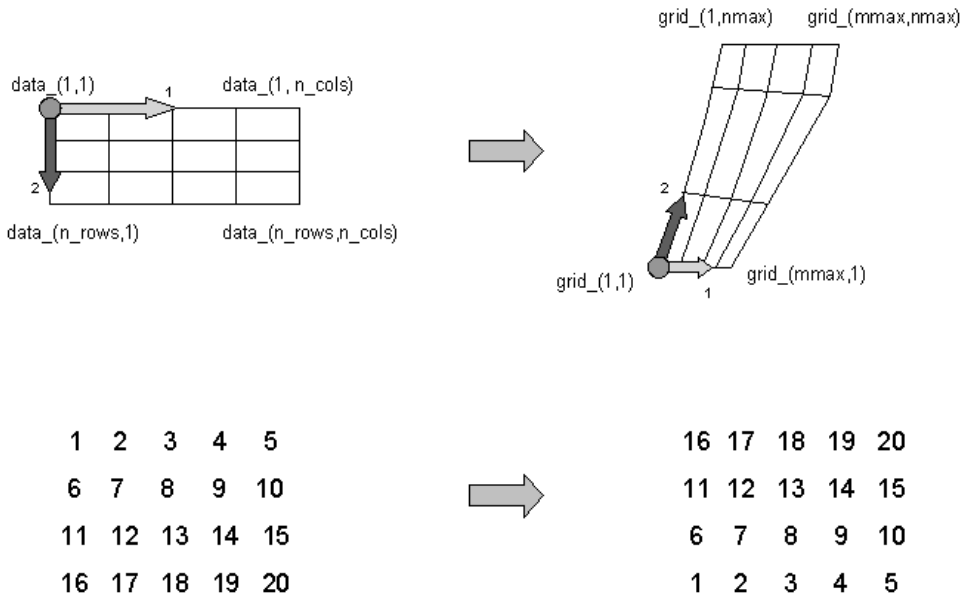


Figure C.1: *Illustration of the data to grid conversion for meteo input on a separate curvilinear grid*

Example:

A file for input of x-velocity (in west-east direction) on a 4 (n_rows) by 5 (n_cols) curvilinear grid, where the meteorogical data is mirrored vertically with respect to the grid, has the following layout:

```
FileVersion     =    1.03
filetype        =    meteo_on_curvilinear_grid
NODATA_value    =    -999.000
grid_file       =    curviwind.grd
first_data_value =   grid_llcorner
data_row        =    grid_row
n_quantity      =    1
quantity1       =    x_wind
unit1           =    m s-1
TIME =      0.0 minutes since 1993-06-28 14:50:00 -02:00
 1   2   3   4   5
 6   7   8   9  10
11  12  13  14  15
16  17  18  19  20
TIME =      600.0 minutes since 1993-06-28 14:50:00 -02:00
 1   2   3   4   5
 6   7   8   9  10
11  12  13  14  15
16  17  18  19  20
```

This results in an x-component of velocity given - in [m/s] - on the curvilinear grid specified in file <curviwind.grd>. The data set will be mirrored such that the first value of the data (upper left corner, in the example '1') corresponds to the lower left corner of the grid (point (1,1)) and a row of data corresponds to a row on the grid, see Figure C.1. Data is given at two times: 0 and 600 minutes since June 28th, 1993, 14:50 PM, in UTC-2.

C.14.3 Spiderweb data

Remarks:

- ◇ The spiderweb file format used in D-Flow FM forms a subset of the the format described below. The extensive format is accepted by D-Flow FM, but not all keywords are required and/or meaningful.
- ◇ The keyword `air_pressure_reference` is ignored.
- ◇ The keyword `n_quantity` is ignored; the number of quantities is always 3.
- ◇ The keywords `quantity1`, `quantity2` and `quantity3` are ignored, the order of the variables in the file is assumed to be wind speed, wind direction and pressure drop.
- ◇ The keywords `unit1` and `unit2` are ignored.
- ◇ The keyword `unit3` is optional; if omitted or different from '`mbar`', Pa is silently assumed

Cyclone winds are governed by a circular motion, combined with a cyclone track. This type of wind is generally very difficult to implement on a curvilinear grid. This feature facilitates the reading of the so-called Spiderweb files and interpolates the wind and pressure data internally to the computational grid. A special feature of the space varying wind and pressure on the Spiderweb grid is that it can be combined with one of the other meteorological input options described in this manual, i.e. to either uniform wind and pressure, or to one of the space varying wind and pressure options, see section C.14.

File contents	Time-series of a space varying wind and atmospheric pressure defined on a Spiderweb grid. This grid may be specified in Cartesian or spherical coordinates.
File format	Free formatted or unformatted, keyword based.
Generated	Some offline program.

Remarks:

- ◇ The keywords are case insensitive.
- ◇ Space varying wind and pressure on a Spiderweb grid is added to other wind input and the wind fields are interpolated and combined in and around the cyclone.

Header description of the Spiderweb wind and pressure file:

Keywords	**Value**	**Description**
`FileVersion`	1.04	version of file format
`Filetype`	meteo_on_spiderweb_grid	meteo input on Spiderweb grid
`NODATA_value`	*free*	value used for input that is to be neglected
`n_cols`	*free*	number of gridpoints in angular direction
`n_rows`	*free*	number of gridpoints in radial direction
`grid_unit`	m *or* degree	unit of the Spiderweb grid
`spw_utm_zone_target`	format 'ZZH', where ZZ is the UTM zone number, and H is the hemisphere (N or S)	UTM zone of the grid on which the meteo field is interpolated
`spw_radius`	*free*	radius of the spiderweb given in units given by `spw_rad_unit`
`spw_rad_unit`	m	unit of the Spiderweb radius

DRAFT

Keywords	Value	Description
spw_merge_frac	[0.0,1.0]	fraction of the Spiderweb radius where merging starts of the background wind with the Spiderweb wind. Default is 0.5
air_pressure _reference	air_pressure_default_from _computational_engine *or free*	Both keyword and value are too long to fit on one line. Reference value related to p_drop is the default air pressure of the computional engine *or* the value specified. If missing, p_drop is extracted from the actual atmospheric pressure.
n_quantity	3	number of quantities specified in the file
quantity1	wind_speed	wind speed given in unit unit1
quantity2	wind_from_direction	direction where the wind is coming from given in unit unit2
quantity3	p_drop	drop in atmospheric pressure given in unit unit3
unit1	m s-1	unit of quantity1: metres/second
unit2	degree	unit of quantity2: degrees
unit3	Pa *or* mbar	unit of quantity3: Pascal or millibar

Time definition and data block description

For a description of the time definition see section C.14.1.

Cyclone track information:

For each time in the time series of space varying wind and pressure on a Spiderweb grid, the position of the cyclone eye (and thus also the spiderweb grid) must be given, as well as the drop of atmospheric pressure in the cyclone eye.

File version and conversion

The current description holds for FileVersion 1.04. The table below shows the latest modifications in the file format (and version number).

DRAFT

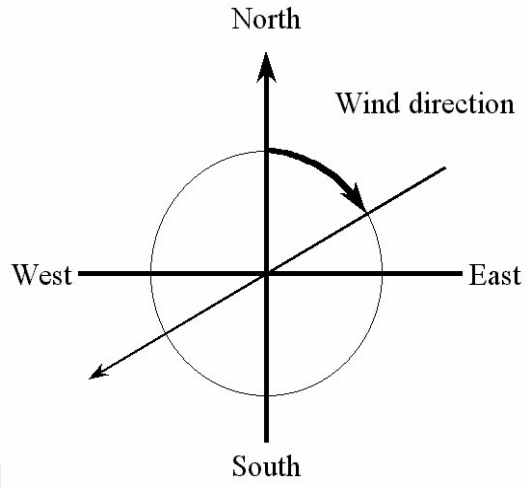


Figure C.2: *Wind definition according to Nautical convention*

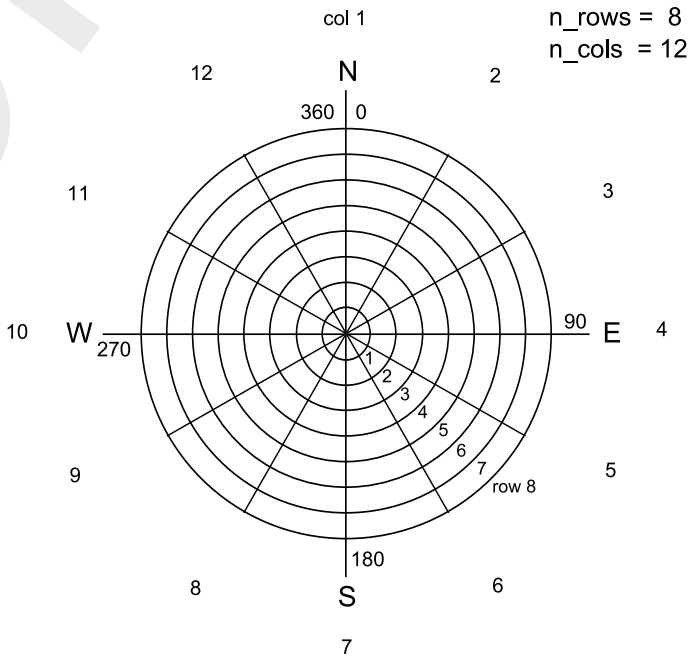


Figure C.3: *Spiderweb grid definition*

DRAFT

FileVersion	Modifications
1.04	Addition of optional keyword spw_utm_zone_target, which allows to project spiderweb wind fields on flow and wave grids defined using UTM coordinates. Only taken into account when grid_unit is degree.
1.03	No changes for this meteo input type
1.02	Changed the use of keyword n_rows and n_cols. The radius of the cyclone is divided in *n_rows* rings of width: spw_radius/n_rows [m] and the circle is divided in *n_cols* parts of $2\pi/n_cols$ [rad].
1.01	Changed keyword MeteoType to FileType Changed fixed value of input type (Keyword Filetype) from *Spiderweb* to *meteo_on_spiderweb_grid*

Restriction:

- ◇ The restrictions for space varying wind and pressure on a Spiderweb grid are the same as for space varying wind and pressure on an equidistant grid, described in section C.14.1.

Remarks:

- ◇ The remarks for space varying wind and pressure on a separate curvilinear grid are the same as for space varying wind and pressure on an equidistant grid, described in section C.14.1.
- ◇ The Spiderweb grid is circular and the definitions of the number of rows n_rows and the number of columns n_cols is therefore different then for the other meteo input formats. For the Spiderweb grid, the number of rows determines the grid size in radial direction. The number of columns defines the grid size in angular direction. See Figure C.3.
- ◇ The wind is specified according to the nautical convention, i.e. wind from the true North has direction zero and the wind turns clockwise with an increasing angle. See Figure C.2.

Example:

A file for input of space varying wind and pressure on a 5x3 Spiderweb grid, has the following layout:

```
FileVersion            = 1.04
filetype               = meteo_on_spiderweb_grid
NODATA_value           = -999.000
n_cols                 = 3
n_rows                 = 5
grid_unit              = degree
spw_utm_zone_target    = 49N
spw_radius             = 600000.0
spw_rad_unit           = m
air_pressure_reference = air_pressure_default_from_computational_engine
n_quantity             = 3
quantity1              = wind_speed
quantity2              = wind_from_direction
quantity3              = p_drop
unit1                  = m s-1
unit2                  = degree
unit3                  = Pa
TIME          =    0.0 hours since 1997-07-14 03:00:00 -06:00
x_spw_eye     = 115.1
y_spw_eye     = 18.9
p_drop_spw_eye = 5300.0
1.38999       1.38261       1.38315
1.28251       1.34931       1.22571
1.27215       1.31214       1.32451
1.38999       1.86592       2.87732
1.43899       1.24912       2.21519
60.0000       180.0000      270.0000
28.7500       20.0000       31.2500
```

DRAFT

```
42.5000       53.7500       65.0000
49.3400       60.2400       81.5200
51.4100       62.0000       43.1200
5301.280      5294.490      5156.240
5043.460      5112.040      5264.020
5140.020      5202.520      5411.210
5294.730      5285.760      5235.250
5242.530      5156.190      5124.240
TIME          =      1.0 hours since 1997-07-14 03:00:00 -06:00
x_spw_eye     = 114.8
y_spw_eye     = 18.8
p_drop_spw_eye = 5250.0
1.35763       1.35763       1.35763
1.35763       1.87273       2.24784
1.92214       2.47836       2.17266
1.87662       2.72116       2.82375
1.26585       2.24146       2.38722
159.0000      346.5200      290.6400
342.3200      282.1400      20.2400
10.7500       25.5300       36.4500
61.8400       81.6200       45.5100
49.5250       56.7500       75.1300
5314.520      5104.490      5287.240
5124.240      5285.760      5252.420
5152.460      5247.040      5222.020
5242.020      5223.520      5475.210
5244.270      5211.210      4998.110
```

This results in the following set of meteo data. Velocities given in [m/s] and pressure drops in [Pa] on a Spiderweb grid which is given in spherical coordinates (`grid_unit` = degree), but that is projected on a UTM 49N zone flow/wave grid. The cyclone and spiderweb grid have a radius of 600 km. The grid is 5x3, which means the radius is divided in five parts of 120 km and the 360 degrees are divided in 3 parts of 120 degrees each. Wind speeds, wind directions and pressure drops are given at two times: 0 and 1.0 hour since July 14th, 1997, 03:00 AM, in UTC-6. Between these two times the cyclone eye moves from (longitude, latitude) (115.1, 18.9) to (114.8, 18.8) on the globe and the pressure drop in the cylcone eye decreases from 5300.0 [Pa] to 5250.0 [Pa].

C.14.4 Meteo on a netCDF file

Scalar meteorological forcing variables can be provided as gridded fields in NetCDF files. The file and metadata details are described in Section E.2.4.2.

C.15 Statistical output (Fourier, minimum, maximum and average)

File contents	All parameters required to execute an online statistical analysis for specified quantities, a specified period and for specified frequencies. This analysis can be a Fourier analysis or another form of statistical analysis e.g. computation of the minimum, maximum or average value.
Filetype	ASCII
File format	Fix formatted for text variables, free formatted for real and integer values.
Filename	<*name*.fou>
Generated	Manually offline

Each line in the file represents a record.

DRAFT

Record description:

Record	Record description
each line	1 Id of the quantity on which the analysis is to be performed: bs bed stress (1D/2D field) cn n-th constituent in the MDU-file cs salinity, will be skipped if temperature is not modelled ct temperature, will be skipped if temperature is not modelled eh energy height (1D/2D field) fb freeboard (1D only) q1 discharge through a flow link sul water levels at a flow links (1D/2D field) ux velocity in x-direction uxa velocity in x-direction, column average (1D/2D field) uy velocity in y-direction uya velocity in y-direction, column average (1D/2D field) uc velocity magnitude vog volume on ground (1D only) wd water depth (1D/2D field) wdog waterdepth on ground (1D only) wl water level (1D/2D field) ws wind magnitude (1D/2D field) 2 Analysis start time, `tstart`, in Tunit [seconds, minutes, hours] after the simulation Reference Date. Use -1 to indicate the simulation start time. 3 Analysis stop time, `tstop`, in Tunit [seconds, minutes, hours] after the simulation Reference Date. Use -1 to indicate the simulation stop time. 4 Integer number of cycles (i.e. mode) within the analysis time frame. Alternatively, the length of the running mean filter. 5 Nodal amplification factor. 6 Phase shift [degrees]. 7 Layer number for the analysis of 3D quantities. This number should if and only if the selected quantity is not marked as a 1D or 1D/2D field. When specified, it should be a valid layer number. However, this value is currently not used. In 3D models, the analysis is done on the whole 3D field, ignoring the layer number. 8 Flag specifying the type of the analysis: ◇ default (no keyword): perform Fourier analysis ◇ `min`/`max`/`avg`, requests the computation of the minimal, maximal or average of the selected quantity over the selected period. The number of cycles now is the length of an optional running mean filter. An additional quantity id can be supplied to report the requested quantity when the `min`/`max` of that additional quantity occurs (see last record of Example 2, extensions for min/max). ◇ `last`, requests the last value, e.g. when anticipating convergence to a steady-state solution. The number of cycles now is the number of time steps before the end of the analysis period to average. ◇ `time below` / `time above` followed by threshold value, requests the computation of the total time passed during which the selected quantity strictly below or above the specified threshold.

Remarks:

- ◇ The analysis is performed for all grid points individually.
- ◇ If the number of cycles is set equal to 0, the *mean* value of the quantity over the interval specified by the start and stop time is determined.
- ◇ The computed Fourier amplitudes differ slightly from the amplitude of the corresponding tidal component. When comparisons with co-tidal maps have to be made, this factor can be determined using the subsystem ASCON of Delft3D-TIDE, the tidal analysis package of Deltares.

DRAFT

- ◇ The computed Fourier phases are by default associated with the reference date/time of the FLOW simulation. For comparisons with co-tidal maps the user may specify a non-zero phase shift.
- ◇ The layer number should not be specified for 1D or 1D/2D quantities such as water levels. In 3D models, the analysis is done on the whole 3D field, ignoring the layer number.
- ◇ In case of `max` and `min`, this keyword maybe followed by the keyword `time` to indicate that also the time at which the maximal or minimal value occurs, should be written to file.
- ◇ In case of `max` computation for water level, water depth or energy height, an additional keyword `inidryonly` can be specified to perform the analysis only for points that are initially dry.
- ◇ Depending on the flag `FouUpdateStep` the analysis is done every user time step (`FouUpdateStep = 0`, default), or every computation time step (`FouUpdateStep = 1`), or with the same time step as the history output (`FouUpdateStep = 2`). The keyword `FouUpdateStep` is defined in the chapter `[output]` of the mdu-file.

Restrictions:

- ◇ If `FouUpdateStep = 0 or 2`, the start and stop times of the analysis frame specified must be a multiple of the user time step or the history output time step.
- ◇ The start and stop times of the analysis frame specified must be a valid time, i.e. must fit in the simulation time frame of the FLOW computation.
- ◇ Items in a record must be separated by one or more blanks.
- ◇ The quantity name (first item on a line) must start on position one.
- ◇ The starting time of the simulation is *excluded* from the analysis.

Running mean

The minimum and maximum values may be filtered with a running mean filter, to minimize the effect of spikes. Column 4 is used for this and is the length of the filter in fourier output time steps. Therefore, this option can only be used if `FouUpdateStep = 0` or `2`.

The running mean filter is defined by:
Suppose we have a time serie $x[1], x[2], ...x[N]$.
Then the running mean is:
$\overline{x}[i] = (x[i+1-L] + x[i+2-L] + .. + x[i])/L$, with L the length of the filter and $i \geq L \wedge L > 0$.

The minimum of the running mean is defined as: $\min \overline{x}[i], i \in [\text{tstart}, \text{tstop}]$, the same holds for other functions.

The running mean filter is applied for all points in the model. Note that we need a buffer of length L times the number of grid points to calculate this. This buffer is needed for each quantity in the fou-file using the running mean, except if you ask both the $\min$ and $\max$ for the same field with the same start and stop time. Then the same buffer is used.

It is possible to get the velocity at the moment that we have the highest water level, or the time of the maximum water level (see the last two records of Example 2, extensions for min/max). Or just the latest time(s), in case you are running to a steady state solution (check out the `last` option).

Example 1

A statistical analysis is requested for:

- ◇ Water level: mean value and the first two harmonics.
- ◇ Velocity in x-direction: first harmonic, in the top layer.
- ◇ Velocity in y-direction: first harmonic, in the top layer.
- ◇ Velocity magnitude: first harmonic, in the top layer.
- ◇ Temperature: first harmonic, in the third layer.
- ◇ Salinity: mean value of the third layer.
- ◇ Four constituents: mean value for a slightly shifted time period in the top layer for 3 constituents; and the maximum for the fourth constituent.

DRAFT

```
wl      720.    1440.     2    1.000   0.000
wl      720.    1440.     1    1.000   0.000
wl      720     1440.     0    1.000   0.000
ux      720.    1440.     1    1.000   0.000      1
uy      720.    1440.     1    1.000   0.000      1
uc      720.    1440.     1    1.000   0.000      1
ct      720.    1440.     1    1.000   0.000      3
cs      720.    1440.     0    1.000   0.000      3
c1      710.    1430.     0    1.000   0.000      1
c2      710.    1430.     0    1.000   0.000      1
c3      710.    1430.     0    1.000   0.000      1
c4      710.    1430.     0    1.000   0.000      1    max
```

Remark:

- ◇ In this example the start and stop times are in minutes because the Tunit of this model (as defined in the MDU file) is in minutes.

Example 2, extensions for min/max

A statistical analysis is requested for:

- ◇ Water level: maximum with a filter with length 25, starting t = 720.
- ◇ Water level: minimum with a filter with length 13, stopping at t = 1440.
- ◇ Water level: average for the last 5 steps of the simulation.
- ◇ Water level: maximum with a filter with length 25, with extra output of the time at which this maximum occurs.
- ◇ Velocity magnitude as the water level reaches this maximum, starting at t = 720 and stopping at t = 1440.

```
wl      720.    -1          25   1.000   0.000      max
wl      -1       1440.      13   1.000   0.000      min
wl      -1       -1          5   1.000   0.000      last
wl      -1       -1         25   1.000   0.000      max time
uc      720.    1440.        1   1.000   0.000      1 max wl
```

Remarks:

- ◇ In this example the start and stop times are in minutes because the Tunit of this model is in minutes. A start time of -1 means start of the simulation; A stop time of -1 means the end of the simulation.
- ◇ In this example the columns 5 and 6 are completely ignored.
- ◇ Quantities that give combined results, must have the same dimension. Therefore we introduced sul (water level at flow links) to compute q1 (discharge through flow link) at the time of the maximum water level. Note that this combination does not work for 3D applications.

C.16 1D roughness INI file

The roughness settings for the 1D network may be specified directly in the cross section definitions (see section C.17.1) or by reference to roughness variables defined here. The keyword `FrictFile` should be used to point to one or more roughness files as defined below (see chapter A). Each file defines one roughness variable. The following roughness types have been implemented

- ◇ `Chezy`: Chézy C [m$^{1/2}$/s]
- ◇ `Manning`: Manning n [s/m$^{1/3}$]
- ◇ `wallLawNikuradse`: Nikuradse k_n [m]
- ◇ `WhiteColebrook`: Nikuradse k_n [m]
- ◇ `StricklerNikuradse` = Nikuradse k_n [m]
- ◇ `Strickler`: Strickler k_s [m$^{1/3}$/s]
- ◇ `DeBosBijkerk`: De Bos-Bijkerk γ [1/s]

See section 3.3.6 for details on these formulations.

The file description is given below.

Table C.31: *Roughness definition.*

Keyword	Type	Default	Description
[General]			
fileVersion	String	3.01	File version. Do not edit this.
fileType	String	roughness	File type. Do not edit this.
frictionValuesFile	String		Name of <*.bc> file containing the time-series with friction values. Only needed for functionType = timeSeries.
[Global]			*One global block per file (see Remarks for an exception)*
frictionId	String		Name of the roughness variable
frictionType	String		The global roughness type for this variable which is used if no branch specific roughness definition is given. See the table at the beginning of this section for the encoding of the different roughness types.
frictionValue	Double		The global default value for this roughness variable
[Branch]			*Optional: one block per branch*
branchId	String		The name of the branch
frictionType	String		The roughness type to be used on this branch. See the table at the beginning of this section for the encoding of the different roughness types.
functionType	String	constant	Function type for the calculation of the value, possible values ◇ constant: Constant (in time) ◇ timeSeries: Time series (see remark below) ◇ absDischarge: Function of absolute discharge (not dependent on direction) ◇ waterLevel: Function of water level
timeSeriesId	String		Refers to a data block in the <*.bc> frictionValuesFile. Only if functionType = timeSeries.
numLevels	Int		Number of levels in table. Only if functionType is not constant.
levels	Double[]		Discharge (m^3/s) or water level [m AD] values. Only if functionType is not equal to constant.
numLocations	Int	0	Number of locations on branch. The default 0 value implies branch uniform values.
chainage	Double[]		Space separated list of locations on the branch [m]. Locations sorted by increasing chainage. The keyword must be specified if numLocations>0.

(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
frictionValues	Double[][]		numLevels lines containing space separated lists of roughness values: numLocations values per line. If the functionType is equal to constant, then a single line is required. For a uniform roughness per branch (numLocations = 0) a single entry per line is required. The meaning of the values depends on the roughness type selected (see frictionType).

Remarks:

- If the roughness varies along a branch, the roughness values are interpolated *linearly* along the branch in between roughness locations and assumed constant beyond the first and last location specified. If the roughness varies as a function of discharge or water level, the roughness values are interpolated *linearly* between the specified values and assumed constant below the first and above the last value.
- Multiple roughness variable definitions that don't vary spatially (i.e. that don't require Branch blocks) may be combined in one file. In that case there will one General block, multiple instances of the Global block and no Branch blocks in the file. See the second example.
- The function of discharge has been implemented as a function of absolute discharge.
- The frictionId may contain spaces, with no quoting needed.
- The branchId may contain spaces, with no quoting needed.
- This file does not include an option to specify roughness locations via x and y.
- The number of frictionValues to be specified per line equals the number of locations on that branch. If numLocations is not specified or equal to 0, one value is expected which will be applied uniformly along the branch.
- The number of data lines following frictionValues is one for functionType = constant and equal to numLevels for the other function types.
- When at least one [Branch] block has frictionType = timeSeries, then:
  - the [General] block must specify a <*.bc> file via frictionValuesFile. The <*.bc> may contain multiple time series (in [Forcing] blocks);
  - the branch keyword timeSeriesId must refer to a timeseries block in that file (i.e., must match with the name);
  - the quantity in that file must equal friction_coefficient_*<friction type>*, where *<friction type>* is the type as given after frictionType, e.g., quantity = friction_coefficient_Manning.
- A branch that has functionType = timeSeries can only use one time series. In other words: the friction coefficient is constant along the branch.

Example:

```
[General]
    fileVersion           = 3.01
    fileType              = roughness

[Global]
    frictionId            = Main
    frictionType          = Chezy
    frictionValue         = 45.000

[Branch]
    branchId              = Channel1
    frictionType          = Manning
    functionType          = constant
    numLocations          = 2                    # at two locations
    chainage              = 0.000 100.000
    frictionValues        = 0.20000 0.30000
```

```
[Branch]
    branchId              = Channel4
    frictionType          = Chezy
    functionType          = constant
    frictionValues        = 40.00000

[Branch]
    branchId              = Channel7
    frictionType          = Chezy
    functionType          = absDischarge
    numLevels             = 3                     # for three values
    levels                = 100.000 200.000 300.000
    numLocations          = 2                     # at two locations
    chainage              = 0.000 300.000
    frictionValues        = 45.00000 55.00000     # values for |Q|=100
                            41.00000 52.00000     # values for |Q|=200
                            40.00000 50.00000     # values for |Q|=300
```

Example Multiple Constant Roughness Variables:

```
[General]
    fileVersion           = 3.01
    fileType              = roughness

[Global]
    frictionId            = Brick
    frictionType          = WhiteColebrook
    frictionValue         = 0.1

[Global]
    frictionId            = Steel
    frictionType          = WhiteColebrook
    frictionValue         = 0.005
```

Example time-dependent roughness:

```
[General]
    fileVersion           = 3.01
    fileType              = roughness
    frictionValuesFile    = roughness_timeseries.bc

[Global]
    frictionId            = Main
    frictionType          = Chezy
    frictionValue         = 45.000

[Branch]
    branchId              = Channel3
    frictionType          = deBosBijkerk
    functionType          = timeSeries
    timeSeriesId          = rural_secondary

[Branch]
    branchId              = Channel9
    frictionType          = deBosBijkerk
    functionType          = timeSeries
    timeSeriesId          = rural_secondary
```

The accompanying <roughness_timeseries.bc>:

```
[General]
    fileVersion           = 1.01
```

DRAFT

```
    fileType              = boundConds

[Forcing]
    name                  = rural_secondary
    function              = timeSeries
    timeInterpolation     = linear
    quantity              = time
    unit                  = days since 2001-01-01
    quantity              = friction_coefficient_deBosBijkerk
    unit                  = s-1
    0  15
    31 21
    59 28
    90 25
    # ...
```

C.17 Cross section files

Cross sections are defined in two files. The first file (`CrossDefFile`) contains the cross section definitions with the geometrical parameters and the main roughness settings. This file is described in section C.17.1. The second file (`CrossLocFile`) defines where in the 1D network the cross sections are actually used, and it also specifies the vertical position of the cross sections at those locations. This file is described in section C.17.2. The cross section location file is not used for XYZ cross sections since the XYZ definition already includes a full specification of the location. The keyword `CrossDefFile` should be used to point to the cross section definition file and the keyword `CrossLocFile` should be used to point to the cross section location file (see chapter A).

C.17.1 Cross section definition file

The definition file contains the geometrical parameters for each cross section. It defines the shape and roughness of the cross section. The shape may be defined either by a parametrized shape or a more flexible table (containing either ZW, YZ, or XYZ data). The roughness settings may be specified directly (by means of type and value) or by reference to roughness variables. If no roughness setting is specified, the roughness is in most cases derived from the roughness variable "Main" (see the subsections below for exceptions). If both `frictionId` references are given, and also `frictionType`+`frictionValue` are given in the cross section definition, then the latter two are ignored: roughess variables (roughness sections) take precedence over direct values. The input for roughness variables is described in section C.16.

Table C.32: *Cross section definition file.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`3.00`	File version. Do not edit this.
`fileType`		`crossDef`	File type. Do not edit this.
`[Global]`			
`leveeTransitionHeight`	Double	`0.50`	Levee transition height [m]. Only applicable in case of cross section type `zwRiver`.
`[Definition]`			
`id`	String		Unique cross-section definition id.

(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
`type`	String		Cross section type. Possible values are: ◇ `circle`: circular profile ◇ `rectangle`: rectangular profile ◇ `zwRiver`: tabulated river profile ◇ `zw`: ZW profile ◇ `xyz`: XYZ profile ◇ `yz`: YZ profile For specific profile shapes such as arch, egg, mouth-shape, and trapezium see the description of the tabulated ZW profiles.
`thalweg`	Double		Transverse Y coordinate at which the cross section aligns with the branch **(Keyword used by GUI only)**
`...`			Keywords specific for the cross section type

The type specific keywords and descriptions can be found in the following subsections.

Remark:
- ◇ The `id` may contain spaces, with no quoting needed.

Example:

```
[General]
    fileVersion           = 3.00
    fileType              = crossDef

[Definition]
    id                    = Prof1
    type                  = circle
    thalweg               = 0.0
    diameter              = 2.0
    frictionId            = Brick

[Definition]
    id                    = Prof1A
    type                  = circle
    thalweg               = 0.0
    diameter              = 2.0
    frictionType          = WhiteColebrook
    frictionValue         = 0.1

[Definition]
    id                    = Prof2
    type                  = zw
    thalweg               = 0.0
    template              = steelMouth          # template name and
    height                = 0.950               # values used by GUI
    r                     = 0.600
    r1                    = 0.500
    r2                    = 0.000
    r3                    = 0.000
    a                     = 0.000
    a1                    = 0.000
    numLevels             = 43                  # zw levels used by kernel
    levels                = 0.00000 0.00873 0.01746 0.02619 0.03492 0.04365
    0.05238 0.06111 0.06984 0.07857 0.08730 0.09603 0.10476 0.11349 0.12222
    0.13095 0.13968 0.14841 0.15714 0.16587 0.17460 0.18333 0.19206 0.20079
    0.20952 0.21825 0.22698 0.23571 0.24444 0.25317 0.26190 0.27063 0.27937
    0.28810 0.29683 0.30556 0.31429 0.32302 0.33175 0.34048 0.34921 0.35794
    0.36667
    flowWidths            = 0.00000 0.18605 0.26196 0.31940 0.36716 0.40863
    0.44559 0.47907 0.50976 0.53814 0.56455 0.58927 0.61249 0.63439 0.65508
```

DRAFT

```
    0.67470 0.69331 0.71102 0.72787 0.74393 0.75925 0.77388 0.78785 0.80119
    0.80964 0.79011 0.76970 0.74832 0.72589 0.70231 0.67746 0.65120 0.62335
    0.59367 0.56190 0.52763 0.49036 0.44934 0.40341 0.35067 0.28738 0.20396
    0.00000
    frictionId            = Steel

[Definition]
    id                    = xyzProf1
    type                  = xyz
    thalweg               = 283.746
    xyzCount              = 6
    xCoordinates          = 1920.254 1925.508 1925.508 1925.508 1930.761
    1930.761
    yCoordinates          = 1452.223 1168.526 1026.677 953.126 900.589
    884.828
    zCoordinates          = 10.000 0.004 4.994 7.585 9.445 10.000
    conveyance            = segmented
    sectionCount          = 3
    frictionPositions     = 0.000 158.032 344.304 567.706
    frictionIds           = LeftBank; Main; RightBank
```

DRAFT

C.17.1.1 Circle cross section definition

This section describes the additional keywords for the circular shaped cross sections in the **Definition** block compared to the general description in section C.17.1. This cross section type is always closed.

Table C.33: *Circle cross section definition.*

Keyword	Type	Default	Description
`[Definition]`			
`type`	String	`circle`	Cross section type; must read `circle`.
`diameter`	double		Internal diameter of the circle [m].
`frictionId`	String		Name of the roughness variable associated with this cross section. Either this parameter or `frictionType` should be specified. If neither parameter is specified, the `frictionId` defaults to "Main".
`frictionType`	String		Roughness type associated with this cross section (see section C.16 for encoding). Either this parameter or `frictionId` should be specified.
`frictionValue`	Double		Roughness value; its meaning depends on the roughness type selected (only used if `frictionType` specified).

A ground layer will be included in the future; keywords not yet defined.

C.17.1.2 Rectangle cross section definition

This section describes the additional keywords for the rectangular cross sections in the **Definition** block compared to the general description in section C.17.1. A typical rectangular cross section is shown in the figure on the right. The rectangular profile may be used for both open and closed channels. Use the keyword `closed` to switch between these two cases.

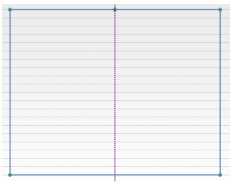


Table C.34: *Rectangular cross section definition.*

Keyword	Type	Default	Description
`[Definition]`			
`type`	String	`rectangle`	Cross section type; must read `rectangle`.
`width`	Double		Width of the rectangle [m].
`height`	Double		Height of the rectangle [m].
`closed`	Logical	`yes`	`no`: Open channel, `yes`: Closed channel
`frictionId`	String		Name of the roughness variable associated with this cross section. Either this parameter or `frictionType` should be specified. If neither parameter is specified, the `frictionId` defaults to "Main".
`frictionType`	String		Roughness type associated with this cross section (see section C.16 for encoding). Either this parameter or `frictionId` should be specified.

(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
`frictionValue`	Double		Roughness value; its meaning depends on the roughness type selected (only used if `frictionType` specified).

A ground layer will be included in the future; keywords not yet defined.

C.17.1.3 Tabulated river cross section definition

This section describes the additional keywords for the tabulated river cross sections in the **Definition** block compared to the general description in section C.17.1. The tabulated river cross section is very similar to the ZW cross section (see section C.17.1.4). Compared to the standard ZW cross section, the tabulated river profile will always be open and the width tables should be monotonically increasing. Furthermore, the roughness over the flow width may be defined separately for three sections: main, floodplain 1 and floodplain 2. This cross section type is always processed using the lumped conveyance type per section.

Table C.35: *Tabulated river cross section definition.*

Keyword	Type	Default	Description
`[Definition]`			
`type`	String	`zwRiver`	Cross section type; must read `zwRiver`.
`numLevels`	Int		Number of levels in the table.
`levels`	Double[]		Space separated list of monotonic increasing heights/levels [m AD].
`flowWidths`	Double[]		Space separated list of flow widths at the selected heights [m)].
`totalWidths`	Double[]		Space separated list of total widths at the selected heights [m]. Equal to `flowWidths` if not specified. If specified, the `totalWidths` should be larger than `flowWidths`.
`leveeCrestLevel`			Relative crest level of levee [m].
`leveeBaseLevel`			Relative base level of levee [m].
`leveeFlowArea`			Flow area behind levee [m^2].
`leveeTotalArea`			Total area behind levee [m^2].
`mainWidth`	Double	*	Width of main section [m]. Default value: max(flowWidths).
`fp1Width`	Double	*	Width of floodplain 1 section [m]. Default value: max(flowWidths)-mainWidth
`fp2Width`	Double	*	Width of floodplain 2 section [m]. Default value: max(flowWidths)-mainWidth-fp1Width
`frictionIds`	String[]		Semicolon separated list of roughness variable names associated with the roughness sections. Either this parameter or `frictionTypes` should be specified. If neither parameter is specified, the `frictionIds` default to "Main", "FloodPlain1" and "FloodPlain2".
			(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
`frictionTypes`	String[]		Semicolon separated list of roughness types associated with the roughness sections (see section C.16 for encoding). Either this parameter or `frictionIds` should be specified. Can be specified as a single value if all roughness sections use the same type.
`frictionValues`	Double[]		Space separated list of roughness values; their meaning depends on the roughness types selected (only used if `frictionTypes` specified).

Remarks:
- ◇ The optional levee feature was called “summer dike” in SOBEK.
- ◇ The parameters `leveeCrestLevel` and `leveeBaselevel` are relative levels with respect to the cross section location.

C.17.1.4 ZW (tabulated) cross section definition

This section describes the additional keywords for the ZW cross sections in the **Definition** block compared to the general description in section C.17.1. The ZW profile may be used for both open and closed channels, so this type may be open or closed. Use a zero total width at the last (highest) level to define a closed cross section. This profile type allows you to distinguish between flow area and storage area; the difference between the total width and the flow width is the storage width. This cross section type is always processed using the lumped conveyance type.

Table C.36: *ZW cross section definition.*

Keyword	Type	Default	Description
`[Definition]`			
`type`	String	`zw`	Cross section type; must read `zw`.
`template`	String		(Optional) Name of ZW cross section template; see description below. **(Keyword used by GUI only)** Possible values are: ◇ `arch`: arch profile ◇ `mouth`: mouth-shaped profile ◇ `egg`: egg profile ◇ `ellipse`: ellipse profile ◇ `invEgg`: inverted egg profile ◇ `steelMouth`: steel mouth-shaped profile ◇ `trapezium`: trapezium profile ◇ `uShape`: U-shaped profile
`...`			Keywords specific for the template.
`numLevels`	Int		Number of levels in the table.
`levels`	Double[]		Space separated list of monotonic increasing heights/levels [m AD].
`flowWidths`	Double[]		Space separated list of flow widths at the selected heights [m].
`totalWidths`	Double[]		Space separated list of total widths at the selected heights [m].
			(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
`frictionId`	String		Name of the roughness variable associated with this cross section. Either this parameter or `frictionType` should be specified. If neither parameter is specified, the `frictionId` defaults to "Main".
`frictionType`	String		Roughness type associated with this cross section (see section C.16 for encoding). Either this parameter or `frictionId` should be specified.
`frictionValue`	Double		Roughness value; its meaning depends on the roughness type selected (only used if `frictionType` specified).

A ground layer will be included in the future; keywords not yet defined.

Many pipes and channels follow standardized design procedures and as a result these pipes and channels have a standardized shape. For the simulation kernel these shapes are represented by a ZW profile description. To support the user in creating these profiles, the user interface supports templates for common cross section geometries. In these cases, the keyword `template` is used to specify the templated profile being used and additional keywords are included.

Remarks:

- If you manually modify this file, please note that the user interface and simulation kernel process this cross section type differently if the `template` is included. So, do not include that keyword ... or make sure that the information of the template is consistent with the information provided by the `numLevels`, `levels`, `flowWidths` and `totalWidths` keywords.
- When importing a cross section definition containing the `template` keyword, the user interface will not process the keywords `numLevels`, `levels`, `flowWidths` and `totalWidths`.
- When the `template` is empty, the user interface will treat the ZW profile record as a generic ZW cross section and thus it will process the keywords `numLevels`, `levels`, `flowWidths` and `totalWidths`.
- The simulation engine will ignore `template` and associated keywords, and always read the keywords `numLevels`, `levels`, `flowWidths` and `totalWidths`.

The following tables give an overview of the various templates recognized and the associated keywords **(All used by GUI only)**.

Arch profile template

Additional definition items for the arch profile template. A typical arch profile is shown in the figure on the right. The `height` is the total height of the profile. The `arcHeight` is the height of the curved arch. The height of the straight base is consequently `height − arcHeight`. The `arcHeight` must thus be less or equal `height`.

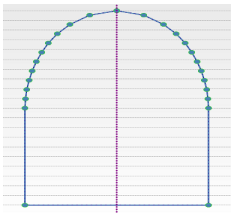


Table C.37: *Arch profile definition.*

Keyword	Type	Default	Description
`[Definition]`			
`template`	String	`arch`	Name of ZW cross section template
`width`	Double		Width of the cross section [m].
			(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
`height`	Double		Total height of cross section [m].
`arcHeight`	Double		Height of arch [m].

Mouth-shaped profile template

Additional definition items for the mouth-shaped profile template. A typical mouth-shaped profile is shown in the figure on the right. This profile was called "Cunette" in SOBEK.

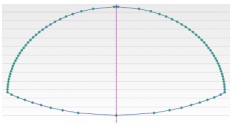


Table C.38: *Mouth-shaped profile definition.*

Keyword	Type	Default	Description
`[Definition]`			
`template`	String	`mouth`	Name of ZW cross section template
`width`	Double		Width of the mouth profile [m].
`height`	Double	*	Height of mouth profile [m]. The default height is 0.634 * `width`

Egg profile template

Additional definition items for the egg profile template. A typical egg profile is shown in the figure on the right. The egg profile has the widest end at the top.

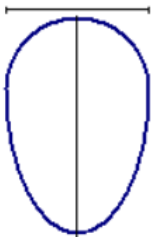
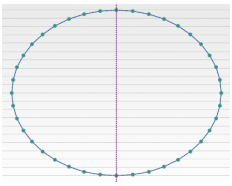


Table C.39: *Egg profile definition.*

Keyword	Type	Default	Description
`[Definition]`			
`template`	String	`egg`	Name of ZW cross section template
`width`	Double		Width of the egg profile [m].
`height`	Double	*	Height of the egg profile [m]. The default height is 1.5 * `width`

Ellipse profile template

Additional items for the ellipse profile template. A typical ellipse profile is shown in the figure on the right.



DRAFT

Table C.40: *Ellipse profile definition.*

Keyword	Type	Default	Description
`[Definition]`			
`template`	String	`ellipse`	Name of ZW cross section template
`width`	Double		Width of the ellipse profile [m].
`height`	Double		Height of the ellipse profile [m].

Inverted egg profile template

Additional definition items for the inverted egg profile template. A typical inverted egg profile is shown in the figure on the right. The inverted egg profile has the widest end at the base.

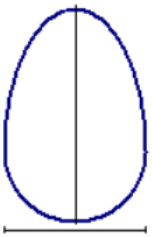


Table C.41: *Inverted egg profile definition.*

Keyword	Type	Default	Description
`[Definition]`			
`template`	String	`invEgg`	Name of ZW cross section template
`width`	Double		Width of the egg profile [m].
`height`	Double	*	Height of the egg profile [m]. The default height is 1.5 * `width`.

Steel mouth-shaped profile template

Additional definition items for the steel mouth-shaped profile template. A typical steel mouth-shaped profile is shown in the figure on the right. This profile was called "SteelCunette" in SOBEK.

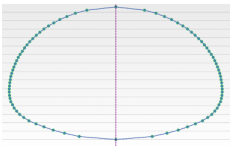


Table C.42: *Steel mouth-shaped profile definition.*

Keyword	Type	Default	Description
`[Definition]`			
`template`	String	`steelMouth`	Name of ZW cross section template
`height`	Double		Height of steel mouth profile [m].
`r`	Double		Radius r [m].
`r1`	Double		Radius r1 [m].
`r2`	Double		Radius r2 [m].
`r3`	Double		Radius r3 [m].
`a`	Double		Angle a [degrees].
`a1`	Double		Angle a1 [degrees].

Trapezium profile template

DRAFT

Additional items for the trapezium profile template. A typical trapezium profile is shown in the figure on the right.

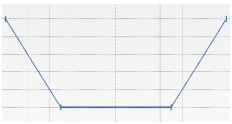


Table C.43: *Trapezium profile definition.*

Keyword	Type	Default	Description
`[Definition]`			
`template`	String	`trapezium`	Name of ZW cross section template
`slope`	Double		Slope of trapezium: dZ per 1 m of width [m].
`width`	Double		Maximum flow width of trapezium [m].
`baseWidth`	Double	0	Base width of trapezium [m].
`closed`	Logical	`no`	`no`: Open channel, `yes`: Closed channel

U-shaped profile template

Additional definition items for the U-shaped profile template. A typical U-shaped profile is shown in the figure on the right. The `height` is the total height of the profile. The `arcHeight` is the height of the curved arch. The height of the straight top is consequently `height − arcHeight`. The `arcHeight` must thus be less or equal `height`.

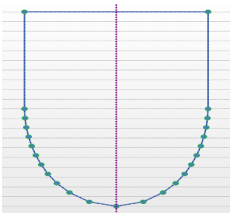


Table C.44: *U-shaped profile definition.*

Keyword	Type	Default	Description
`[Definition]`			
`template`	String	`uShape`	Name of ZW cross section template
`width`	Double		Width of the cross section [m].
`height`	Double		Height of cross section [m].
`arcHeight`	Double		Height of U-arch [m].

C.17.1.5 XYZ cross section definition

This section describes the additional keywords for the XYZ cross sections in the **Definition** block compared to the general description in section C.17.1. The distance along the XY-line will be converted to a transverse coordinate Y, and subsequently this cross section type will be treated as a YZ-cross section. XYZ cross sections are always open. The cross section may be processed using lumped or vertically segmented conveyance method; please note that this choice influences the available options for the roughness definition.

Table C.45: *XYZ cross section definition.*

Keyword	Type	Default	Description
`[Definition]`			
`type`	String	`xyz`	Cross section type; must read `xyz`.
`branchId`	String		(optional) Branch on which the cross section is located.
`xyzCount`	Int		Number of XYZ-coordinates

(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
xCoordinates	Double[]		X-coordinates [m or degrees East].
yCoordinates	Double[]		Y-coordinates [m or degrees North].
zCoordinates	Double[]		Z-coordinates [m AD].
conveyance	String	lumped	lumped: Lumped, segmented: Vertically segmented. In the case of lumped conveyance, only a single uniform roughness for the whole cross section is allowed, i.e., sectionCount must equal 1.
sectionCount	Int	1	Number of roughness sections. If the lumped conveyance is selected then sectionCount must equal 1.
frictionPositions	Double[]		Locations where the roughness sections start and end. Always one location more than sectionCount. The first value should equal 0 and the last value should equal the cross section length. Keyword may be skipped if sectionCount = 1.
frictionIds	String[]		Semicolon separated list of roughness variable names associated with the roughness sections. Either this parameter or frictionTypes should be specified. If sectionCount = 1 and neither parameter is specified, the frictionIds defaults to "Main".
frictionTypes	String[]		Semicolon separated list of roughness types associated with the roughness sections (see section C.16 for encoding). Either this parameter or frictionIds should be specified. Can be specified as a single value if all roughness sections use the same type.
frictionValues	Double[]		Space separated list of roughness values; their meaning depends on the roughness types selected (only used if frictionTypes specified).

Remark:

- ◇ The branchId keyword is only needed if there is a possibility for confusion, i.e. an XYZ definition that intersects multiple branches. The XYZ cross section is always positioned at (the chainage given by) the intersection of the XYZ cross section and the polyline of the branch.

C.17.1.6 YZ cross section definition

Additional items for the YZ cross section definition. YZ profiles may be used represent an open channel (Z as a single-valued function of monotonic increasing Y) or a closed pipe or semi-closed channel of arbitrary cross sectional shape (Z is not a single valued function of Y, YZ is not self-intersecting); the keyword singleValuedZ is used to distinguish between these two cases, but only the first option is currently implemented. A cross section of the second type is considered closed if the first and last YZ coordinate pair are equal. YZ cross sections for open channels (singleValuedZ = yes) may be processed using lumped or vertically segmented conveyance method; please note that this choice influences the available options for the roughness definition. If singleValuedZ = no then only the lumped conveyance and a single uniform roughness is allowed.

DRAFT

Table C.46: *YZ cross section definition.*

Keyword	Type	Default	Description
[Definition]			
type	String	yz	Cross section type; must read yz.
singleValuedZ	Logical	yes	no: Not self-intersecting YZ curve, yes: Monotonic increasing Y, single valued Z(Y) (implemented as any value <>0). **Note:** only the second option is currently available.
yzCount	Int		Number of YZ-coordinates
yCoordinates	Double[]		Y-coordinates [m].
zCoordinates	Double[]		Z-coordinates [m AD].
conveyance	String	segmentedlumped: Lumped, segmented: Vertically segmented. Only the default lumped option is allowed if singleValuedZ = no. In the case of lumped conveyance, only a single uniform roughness for the whole cross section is allowed, i.e., sectionCount must equal 1.	
sectionCount	Int	1	Number of roughness sections. If the lumped conveyance is selected then sectionCount must equal 1.
frictionPositions	Double[]		Locations where the roughness sections start and end. Always one location more than sectionCount. The first value should equal 0 and the last value should equal the cross section length. Keyword may be skipped if sectionCount = 1
frictionIds	String[]		Semicolon separated list of roughness variable names associated with the roughness sections. Either this parameter or frictionTypes should be specified. If sectionCount = 1 and neither parameter is specified, the frictionIds defaults to "Main".
frictionTypes	String[]		Semicolon separated list of roughness types associated with the roughness sections (see section C.16 for encoding). Either this parameter or frictionIds should be specified. Can be specified as a single value if all roughness sections use the same type.
frictionValues	Double[]		Space separated list of roughness values; their meaning depends on the roughness types selected (only used if frictionTypes specified).

C.17.2 Cross section location file

The location file contains a listing of all the locations where cross sections are used in the 1D network. For every location included, it points to the cross section definition used at that particular location. The file does not include any records for locations where XYZ cross sections are used; all the information for these cross sections is already included in their definition. Consequently, if a model contains only XYZ cross sections, this file doesn't have to be specified.

DRAFT

Table C.47: *Cross section location.*

Keyword	Type	Default	Description
[General]			
fileVersion	String	1.01	File version. Do not edit this.
fileType		crossLoc	File type. Do not edit this.
[CrossSection]			
id	String		Unique cross-section location id
branchId	String		(optional) Branch on which the cross section is located.
chainage	Double		(optional) Location on the branch [m].
x	Double		(optional) x-coordinate of the location of the cross section.
y	Double		(optional) y-coordinate of the location of the cross section.
shift	Double	0.0	Vertical shift of the cross section definition [m]. Defined positive upwards.
definitionId	String		Id of cross section definition

Remarks:

- The id, branchId and definitionId may contain spaces, with no quoting needed.
- Two types of location specifications are available.
  - When branchId is given, the keywords branchId and chainage will be used for the location specification.
  - When branchId is *not* given, the keywords x and y coordinates will be used.
  - The value for shift is relative in meters with respect to the absolute bed level of the cross section definition (for example as in yz). When the associated cross section definition is of a type that has no absolute bed level (for example circle), then its absolute bed level defaults to 0.0, which means that the resulting actual bed level becomes exactly equal to the shift.

Example:

```
[General]
    fileVersion           = 1.01
    fileType              = crossLoc

[CrossSection]
    id                    = Channel1_50.000
    branchId              = Channel1
    chainage              = 50.000
    shift                 = 1.0
    definitionId          = Prof1

[CrossSection]
    id                    = Channel2_300.000
    branchId              = Channel2
    chainage              = 300.000
    shift                 = 0.0
    definitionId          = Prof2
```

C.18 Storage node file

By default the storage area of a 1D node is equal to half the sum of the surface areas of the connected flow links (i.e. cross sectional width times branch length up to the neighbouring calculation point). The

DRAFT

storage node file can be used to assign additional storage area to a calculation point, e.g. the area of a manhole (compartments), a small lake or a pond. The keyword `StorageNodeFile` should be used to point to the storage node file (see chapter A).

Table C.48: *Definition of Storage Nodes.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`2.02`	Major file version. Do not edit this.
`fileType`	String	`storageNodes`	File type. Do not edit this.
`useStreetStorage`	Logical	`true`	Indicates whether above `streetLevel` the `streetStorageArea` must be used (`true`) or the regular storage `area` continues (`false`). This option is only applicable for storage nodes with `useTable = false`.
`[Global]`			*Loss coefficients in the Global section can be overridden by their counterparts in specific StorageNode sections.*
`angleCount`	Int		Number of values in angle lookup table.
`angles`	Double		Angles in lookup table, must be monotonically increasing between 0 and 180 degrees [deg].
`angleLossCoefficient`	Double		Angle loss coefficients corresponding to the angles in the lookup table.
`entranceLossCoefficient`	Double		Entrance loss coefficient.
`exitLossCoefficient`	Double		Exit loss coefficient.
`expansionLossCoefficient`	Double		Expansion loss coefficient.
`[StorageNode]`			
`id`	String		Unique id of the storage node.
`name`	String		Long name in the user interface.
`manholeId`	String		(optional) Unique id of manhole that this (compartment) node is part of.
`nodeType`	String	`unspecified`	(optional) Type of the node. Possible values are: ◇ `inspection`: inspection chamber ◇ `soakawayDrain`: soakaway drain (infiltration) ◇ `compartment`: manhole compartment ◇ `unspecified`: general storage node of unspecified type
`nodeId`	String		(optional) Connection node on which the storage node is located.
`branchId`	String		(optional) Branch on which the storage node is located.
`chainage`	Double		(optional) Location on the branch, distance from start of the branch [m].
`x`	Double		(optional) x-coordinate of the storage node.
`y`	Double		(optional) y-coordinate of the storage node.

(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
useTable	Logical		Switch to select a simple (false) or tabulated (true) storage area.
If useTable = false then specify:			
bedLevel	Double		Bed level of the storage area [m AD].
area	Double		Storage area from bedLevel up to streetLevel (and beyond if useStreetStorage = false) [m^2].
streetLevel	Double		Street level of the storage area [m AD].
streetStorageArea	Double		Storage area from streetLevel upwards if useStreetStorage = true [m^2].
storageType	String	reservoir	Possible values: ◇ reservoir: Above streetLevel the storage area of this node is also taken into account. ◇ closed: Above streetLevel this storage node has no storage area.
If useTable = true then specify:			
numLevels	Int		Number of levels in storage area table.
levels	Double []		Levels in storage area table [m AD].
storageArea	Double []		Areas in storage area table [m^2].
interpolate	String	linear	Interpolation type for storage area table. Possible values: linear or block.
To override Global loss coefficients for specific StorageNode:			
angleCount	Int		Number of values in angle lookup table.
angles	Double		Angles in lookup table, must be monotonically increasing between 0 and 180 degrees [deg].
angleLoss ↩ Coefficient	Double		Angle loss coefficients corresponding to the angles in the lookup table.
entranceLoss ↩ Coefficient	Double		Entrance loss coefficient.
exitLoss ↩ Coefficient	Double		Exit loss coefficient.
expansionLoss ↩ Coefficient	Double		Expansion loss coefficient.

Remarks:

- ◇ Three types of location specifications are available.
  - ◇ When nodeId is given, the storage node will be placed on the computational grid point located on the connection node in the network with that nodeId (only for 1D).
  - ◇ When branchId is given, the keywords branchId and chainage will be used for the location specification (only for 1D).
  - ◇ When nodeId and branchId are *not* given and x and y are given, the storage node will be placed on the computational grid point closest to this (x,y) location.
- ◇ Either the nodeId or the combination branchId and chainage can be specified, but not both.
- ◇ The coordinates (x, y) will only be used when nodeId and branchId are omitted.

DRAFT

- ◇ To define a manhole storage node (in urban applications):
  - ◇ Use `useTable = false`, then the storage area will be constant between `bedLevel` and `streetLevel`, and constant above `streetLevel`. So, the storage area will not linearly vary between `area` and `streetStorageArea`.
  - ◇ If `useStreetStorage = false`, then `area` is the storage area from `bedLevel` upwards. Hence the `streetStorageArea` value will not be used.
  - ◇ Additionally, if `useStreetStorage = false`, and `storageType = closed` the storage area above `streetLevel` is set to a small nonzero slot area.
  - ◇ The `streetLevel` will be used as the local "ground level" for output in the map file (more details on page 512).
- ◇ To define a lumped water storage node (in rural applications): use `useTable = true`.
- ◇ The last `levels` value will be used as the local "ground level" for output in the map file (more details on page 512).
- ◇ The loss coefficients are only considered for storage nodes with `nodeType = compartment`.
- ◇ Overriding loss coefficients or inheriting global values is handled for each loss parameter separately. For example: one may inherit a global angle loss table, and yet override the bend loss coefficient.

C.19 1D-2D Link file

The 1D-2D links in a model can be configured for geometry and other parameters. Global parameters are available, but additionally, this can also be done for selected links. This configuration is done via the `1D2DLinkFile`. This approach is somewhat comparable to placing cross sections on 1D network branches. Note that the 1D-2D links themselves (their location) is stored as mesh contacts in the `NetFile` (Section B.2.5.2).

Table C.49: *1D-2D link definitions.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`1.00`	File version. Do not edit this.
`fileType`		`1D2DLink`	File type. Do not edit this.
`[MeshContactParams]`			*Repeat as needed*
`contactType`	String	`all`	(optional) Only affect links of specified type. Only relevant when polygon `xCoordinates` and `yCoordinates` are also specified. Possible values are: ◇ `streetInlet`: links between 2D street and 1D node (gullies). ◇ ...: *Note: later, other link types might be supported here as well.* ◇ `all`: affect all enclosed 1D-2D links.
`contactId`	String		(optional) Directly select the link with specified Id.
`numCoordinates`	int		(optional) number of values in `xCoordinates` and `yCoordinates`. This value should be ≥ 3.
`xCoordinates`	Double[]		(optional) x-coordinates of the link selection polygon. (number of values = `numCoordinates`).
`yCoordinates`	Double[]		(optional) y-coordinates of the link selection polygon. (number of values = `numCoordinates`).
Parameters for `contactType = streetInlet` *(gullies)*			
`openingWidth`	Double		Opening width of street inlet (also used for pipe cross section definition).

(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
openingHeight	Double		Opening height of street inlet (also used for pipe cross section definition).

Remark:

- Two types of location specifications are available. One and just one must be given.
  - When contactId is given, it will be used to configure that identified mesh contact.
  - When instead, the keywords numCoordinates, xCoordinates and yCoordinates are given, these will be used to define a polygon. All 1D-2D links inside that polygon (and matching with the specified contactType, if present) will be configured with the same specified parameters.

C.20 Cache file

A cache file may be used to speed up the initialization of a model significantly. It is automatically filled with the geometrical discretization of some spatial input of a model schematization. The name of this file is always equal to <*mdu_name*.cache> and located in the same directory as the mdu-file. It is a binary file.

The currently supported model features for caching are:

- Fixed weirs (see Section 10.1);
- Observation points (see Section 6.4.2.2).
- Observation cross-sections (see section 6.4.2.3).
- Dry points and dry areas (see Section 6.4.2.7)
- Thin dams (see Section 6.4.2.4)

In principle, caching can also be applied to other features like weirs, gates, sources/sinks and embankments. However, in practice we observe that in general the initialization of these features isn't time consuming. Therefore, caching hasn't been implemented for these features. On the other hand, in particular for fixed weirs the initialization can be time consuming.

The behavior of caching is as follows:

1. Upon initialization of the model, when a cache file is present, it is checked on the following requirements:
   - The file version of the cache file should match the one supported by the current D-Flow FM version, see Section C.20.1
   - The MD5 hash stored in the cache file should match the one of the current NetFile.
   - The cached count of observation points and their x- and y-coordinates should match the current set of observation points.
   - The cached count of fixed weirs and their x- and y-polyline coordinates should match the current set of fixed weirs.
2. If any of these checks fail, the cache file is entirely ignored. In this case (and also when no cache file is present at all) the model input is completely initialized onto the model grid. The result is then saved in the same (or new) cache file, for a future model run.
3. If all checks succeeded, the feature discretization is skipped, and instead directly loaded from the cache file.

Caching can be enabled or disabled via the MDU setting UseCaching=1 or 0, respectively. The default or MDU setting for caching can be disabled from the commandline using the option --no-geom-cache.

The cache file is platform-independent and as such can be transferred to other hosts, along with the entire set of model input files.

DRAFT

The cache file should not be edited in any way.

The default value is `UseCaching=1`, which means that caching is applied automatically. Although there is no reason for this, it is possible to switch off caching via keyword `UseCaching=0`.

C.20.1 *File version and description*

The description here holds for the current `FileVersion` 1.01. The table below shows the latest modifications in the features being cached and version number.

FileVersion	Description
`1.00`	Features being cached: fixed weirs, observation points, observation cross-sections, dry points and dry areas
`1.01`	Added to cache: thin dams

DRAFT

D Initial conditions and spatially varying input

D.1 Introduction

Spatially varying input is input data that varies in space such as initial conditions (water levels, etc.) or spatially varying model parameters (friction coefficients, etc.). D-Flow FM uses input data in *model-independent coordinates*, that is, the input files should contain their own spatial coordinates, and do not correspond with the D-Flow FM grid cell numbering. As a result, the model grid can always be changed, without the need to change all other spatial input. The spatial input will be interpolated onto the active model grid. Spatial input has to be specified in the ext-file (section C.6). All supported quantities are listed in Table C.9 under "Initial fields" and "Spatial physical properties".
Remark:

- ◇ Spatial input has to be specified in the `IniFieldFile` (section D.2). Formerly, this was specified in the old-style ext-file, which is now only available for backwards compatibility.

Initial conditions can also be set using a restart-file, which assigns the exact values in the file to the current model grid. No interpolation is performed, and the restart file should correspond exactly with the current model. This is completely unrelated to the remainder of this chapter; please consult Section 7.8 instead.

D.2 Initial and parameter fields

Initial and parameter fields are spatial fields that can describe initial conditions and spatially varying model parameters. The name of this file is specified using the keyword `IniFieldFile` in the MDU-file (see chapter A). This file supersedes the old-style external forcings file for these quantities.

The fields are specified in the ini-file format. The order of blocks is relevant, since multiple data sets maybe combined for the same quantity (see `operand` in Table D.1). The order of keywords within a block is *not* relevant. Keywords and constants are case insensitive, but camel case is used as a naming convention. String values (e.g., filenames) are case sensitive.

This file may contain one or more **Initial** blocks or **Parameter** blocks, each followed by a set of keywords that specify the spatial data for that field. See the following table for details.

Table D.1: *Initial and parameter fields.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`2.02`	File version. Do not edit this.
`fileType`	String	`iniField`	File type. Do not edit this.
`[Initial]` or `[Parameter]`			*Repeat as needed*
`quantity`	String		Name of the quantity, see Table D.2 hereafter.
`dataFile`	String		Name of file containing the field data values.
`dataFileType`	String		Type of `dataFile`. Supported types: ◇ `ArcInfo` ◇ `GeoTIFF` (section D.4.4) ◇ `sample` (section C.3) ◇ `netcdf` (only valid for `quantity = frictioncoefficient`) ◇ `1dField` (section D.4.1), for this type the following keywords are not necessary. ◇ `polygon` (section C.2)

(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
`interpolationMethod`	String		Type of (spatial) interpolation. Supported methods: ◇ `constant`, only with `dataFileType=polygon`. ◇ `triangulation`, Delaunay triangulation+linear interpolation. ◇ `averaging`, grid cell averaging. ◇ `linearSpaceTime`, linear interpolation in space and time (for time-dependent parameters)
`operand`	String	`O`	(optional) How this data is combined with previous data for the same quantity (if any). Supported operands: ◇ `O`, override any previous data. ◇ `A`, append, sets only where data is still missing. ◇ `+`, adds the provided values to the existing values. ◇ `*`, multiplies the existing values by the provided values. ◇ `X`, takes the maximum of the existing values and the provided values. ◇ `N`, takes the minimum of the existing values and the provided values.
`averagingType`	String	`mean`	(optional) Type of averaging, if `interpolationMethod=averaging`. Supported types: ◇ `mean`, arithmetic mean ◇ `nearestNb`, nearest neighbour value ◇ `max`, highest ◇ `min`, lowest ◇ `invDist`, inverse-distance-weighted average ◇ `minAbs`, smallest absolute value ◇ `median`, median value
`averagingRelSize`	Double	1.01	(optional) Relative search cell size for averaging.
`averagingNumMin`	Integer	1	(optional) Minimum number of points in averaging. Must be ≥ 1.
`averagingPercentile`	Double	0.0	(optional) Percentile value for which data values to include in averaging. 0.0 means off.
`extrapolationMethod`	Logical	`no`	Option for (spatial) extrapolation: `no` or `yes`.
`locationType`	String	`all`	(optional) Target location of interpolation, if `quantity` is 1d2d distinction possible (see Table D.2). Supported locations: ◇ `1d`, interpolate only to 1d nodes/links ◇ `2d`, interpolate only to 2d nodes/links ◇ `all`, interpolate to all nodes/links
`value`	Double		(optionally required) Only for `dataFileType=polygon`. The constant value to be set inside for all model points inside the polygon.
			(continued on next page)

DRAFT

Keyword	Type	Default	Description
			(continued from previous page)
`frictionType`	String		(optional) Only for `quantity=frictionCoefficient`. See remark below.
`tracerFallVelocity`	Double	0	(Optional) Only for `initialtracer` ↩ <*tracername*>. Fall velocity for this tracer [m/s]. The default value 0 means that settling is disabled for this tracer.
`tracerDecayTime`	Double	0	(Optional) Only for `initialtracer` ↩ <*tracername*>. Decay time τ for this tracer [s]. Specifies the timescale over which the tracer decays to approximately 36.8% $(1/e)$ of its initial value. The default value 0 means that tracer decay is disabled for this tracer.

Example:

```
# comments
[Initial]
quantity            = initialWaterLevel
dataFile            = iniwaterlevels.asc
dataFileType        = ArcInfo
interpolationMethod = triangulation
locationType        = 2d

[Initial]
quantity            = bedLevel
dataFile            = inibedlevel.ini
dataFileType        = 1dField

[Parameter]
quantity            = frictionCoefficient
dataFile            = manning.xyz
dataFileType        = sample
interpolationMethod = triangulation

[Parameter]
quantity            = frictionCoefficient
dataFile            = calibration1.pol
dataFileType        = polygon
interpolationMethod = constant
value               = 0.03
operand             = multiply
```

Remarks:

- ◇ Friction coefficients may be provided of a different type than the model global type (i.e., MDU setting `UnifFrictCoef`). To do so, add the extra keyword `frictionType` to the `[Parameter]` block. Valid values are listed below:
  - ◇ `Chezy`: Chézy C [m$^{1/2}$/s]
  - ◇ `Manning`: Manning n [s/m$^{1/3}$]
  - ◇ `wallLawNikuradse`: Nikuradse k_n [m]
  - ◇ `WhiteColebrook`: Nikuradse k_n [m]
- ◇ D-Flow FM supports NetCDF format to specify spatially varying friction coefficients. User can specify the friction coefficients over a grid or at sample points. In the latter case, D-Flow FM will interpolate the friction coefficients to the computational grid using linear interpolation via triangulation; no extrapolation is supported yet.
- ◇ If the input is a NetCDF file, then the friction coefficient can also be defined as a time-dependent variable. There are two options for specifying time-dependent friction coefficients. The first option is to specify the friction coeffient fields at two or more time levels. This is for example useful

DRAFT

for modelling changes in the bathymetry after a storm event. The second option is to specify friction coefficients in the form of harmonic constituents (amplitude, phase and frequency). This is useful for cases in which the bathymetry changes periodically following tides or seasonal effects.

◇ An example of an ASCII representation of a NetCDF file used to specify a time series of friction coefficients is:

```
netcdf friction {
dimensions:
        time = 2 ;
        y = 2 ;
        x = 3 ;
variables:
        double time(time) ;
                time:time_origin = "01-Jan-2001 00:00:00" ;
                time:long_name = "Time - minutes since 01-Jan-2001 00:00:00 +00:00" ;
                time:standard_name = "time" ;
                time:calender = "gregorian" ;
                time:units = "minutes since 2001-01-01 00:00:00 +00:00" ;
        double Y(y) ;
                Y:standard_name = "latitude" ;
                Y:units = "degrees_north" ;
        double X(x) ;
                X:standard_name = "longitude" ;
                X:units = "degrees_east" ;
        float friction_coefficient(time, y, x) ;
                friction_coefficient:coordinates = "y x" ;
                friction_coefficient:long_name = "friction coefficient" ;
                friction_coefficient:standard_name = "" ;
                friction_coefficient:units = "s/[m1/3]" ;
data:

 time = 0,   // time(1)
    1440;  // time(2)

 Y = -0.01,   // Y(1)
    0.11;  // Y(2)

 X = -0.01,   // X(1)
    0.5,    // X(2)
    1.01;  // X(3)

 friction_coefficient =
  0.02,   // friction_coefficient(1,1,1)
    0.02,   // friction_coefficient(2,1,1)
    0.025,  // friction_coefficient(3,1,1)
    0.02,   // friction_coefficient(1,2,1)
    0.02,   // friction_coefficient(2,2,1)
    0.025,  // friction_coefficient(3,2,1)
    0.03,   // friction_coefficient(1,1,2)
    0.03,   // friction_coefficient(2,1,2)
    0.035,  // friction_coefficient(3,1,2)
    0.03,   // friction_coefficient(1,2,2)
    0.03,   // friction_coefficient(2,2,2)
    0.035;  // friction_coefficient(3,2,2)
    }
```

◇ Below is an example of an ASCII representation of a NetCDF file used to specify a periodic friction coefficient. Note that the information about frequency and phase reference is specified as global attributes in the NetCDF file. When using more harmonic components, each component should be in a NetCDF file separately from the others.

```
        netcdf friction_coefficient_at_sample_points_harmonic {
                dimensions:
                point = 6 ;
                variables:
                double y(point) ;
                y:standard_name = "latitude" ;
                y:units = "degrees_north" ;
                double x(point) ;
                x:standard_name = "longitude" ;
                x:units = "degrees_east" ;
                float amplitude(point) ;
                amplitude:coordinates = "y x" ;
```

DRAFT

```
        amplitude:long_name = "amplitude_of_friction_coefficient" ;
        amplitude:standard_name = "friction_coefficient" ;
        amplitude:units = "-" ;
        float phase(point) ;
        phase:coordinates = "y x" ;
        phase:long_name = "phase_of_component" ;
        phase:standard_name = "phase" ;
        phase:units = "deg" ;

        // global attributes:
        :component = "M2" ;
        :component_period_in_seconds = "44712" ;
        :reference_time_of_component_phase = "2020-01-01 00:00:00 +00:00" ;
        data:

        y = -0.01,   // y(1)
        0.11,   // y(2)
        0.11,   // y(3)
        -0.01,   // y(4)
        -0.01,   // y(5)
        0.11;  // y(6)

        x = -0.01,   // x(1)
        -0.01,   // x(2)
        0.4,   // x(3)
        0.6,   // x(4)
        1.01,   // x(5)
        1.01;  // x(6)

        amplitude = 0.02,   // amplitude(1)
        0.02,   // amplitude(2)
        0.02,   // amplitude(3)
        0.03,   // amplitude(4)
        0.03,   // amplitude(5)
        0.03;  // amplitude(6)

        phase = -90,   // phase(1)
        -90,   // phase(2)
        -90,   // phase(3)
        90,   // phase(4)
        90,   // phase(5)
        90;  // phase(6)
}
```

◇ It is possible to combine friction coefficients specified in different NetCDF files. This is done by using multiple **Parameter** blocks with `quantity = frictioncoefficient` for the different files, each with a corresponding `operand`. This is useful for example for superpositioning gridded harmonic file with a gridded time series file. At model area that is not covered by the gridded values, the value of `UnifFrictCoef` will be used.
Below is a configuration example for superpositioning of two NetCDF files with friction coefficients:

```
[Parameter]
quantity            = frictionCoefficient
dataFile            = frictioncorrection_constant.nc
dataFileType        = netcdf
interpolationMethod = linearSpaceTime
operand             = override

[Parameter]
quantity            = frictionCoefficient
dataFile            = frictioncorrection_periodic_Sa.nc
dataFileType        = netcdf
interpolationMethod = linearSpaceTime
operand             = add
```

Note: It is currently not possible to combine time-constant and time-varying files. As a workaround, the user can prepare a NetCDF file containing time-constant friction coefficients at two time levels covering the simulation period.

DRAFT

D.3 Supported quantities

D.3.1 Accepted quantity names in initial field files

The list of accepted quantity names in initial field files is subdivided into two categories:

- ◇ Initial fields
- ◇ Spatial model parameters

Table D.2 shows the accepted quantity names, the corresponding names in old external forcing files, the expected units, the possibility of 1d/2d distinction and on which page to find more details. 1d/2d distinction allows to limit a dataset to either the 1D grid points, or 2D, or allow both. This is achieved by setting the `locationType` parameter in the same data block.

Table D.2: *List of accepted field quantity names in initial field file.*

Quantity	Name in old external forcing file	Unit	1d/2d distinction possible?	Page
Initial fields:				
`bedLevel`	`bedlevel`, `bedlevel1D`, `bedlevel2D`	[m]	yes	
`initialWaterLevel`	`initialwaterlevel`, `initialwaterlevel1d`, `initialwaterlevel2d`	[m]	yes	449
`initialWaterDepth`	N/A	[m]	yes	
`initialSalinity`	`initialsalinity`	[ppt]	yes	449
`initialSalinityTop`[1]	`initialsalinitytop`	[ppt]	yes	449
`initialSalinityBot`	`initialsalinitybot`	[ppt]	yes	449
`initialVertical` ↩ `SalinityProfile`	`initialvertical` ↩ `salinityprofile`	[ppt]	no	450
`initialTemperature`	`initialtemperature`	[°C]	yes	450
`InitialVertical` ↩ `TemperatureProfile`	`initialvertical` ↩ `temperatureprofile`	[°C]	no	450
`initialTracer` ↩ *<tracername>*	`initialtracer` ↩ *<tracername>*		yes	450
`initialUnsaturated` ↩ `ZoneThickness`	`initialunsatured` ↩ `zonethickness`	[m]	no	
`N/A`	`initialvelocity`	[m/s, m/s]	no	
`initialVelocityX`	`initialvelocityx`	[m/s]	no	449
`initialVelocityY`	`initialvelocityy`	[m/s]	no	449
Spatial model parameters:				450
General hydrodynamics:				
`frictionCoefficient`	`frictioncoefficient`	depends on its type, see Section 3.3.6	N/A	

[1] `initialSalinityTop` and `initialSalinityBot` can not be combined. Only one can be given per model, and must be combined with `initialSalinity`.

DRAFT

Quantity	name in old external forcing file	Unit	1d/2d distinction possible?	Page
	or friction ↩ _coefficient ↩ _time_dependent			
horizontalEddyVis ↩ cosityCoefficient	horizontaleddyvis ↩ cositycoefficient	[m^2/s]	Intended for 2D only	
horizontalEddyDif ↩ fusivityCoefficient	horizontaleddydif ↩ fusivitycoefficient	[m^2/s]	N/A	
backgroundVertical ↩ EddyDiffusivity ↩ Coefficient	backgroundVertical ↩ EddyDiffusivity ↩ Coefficient	[m^2/s]	N/A	
advectionType	advectiontype	[-]	N/A	See AdvecType in Table A.1
Hydrology:				206
InterceptionLayer ↩ Thickness	N/A	[m]	yes	208
PotentialEvaporation	N/A	[mm/hr]	yes	209
InfiltrationCapacity[2]	infiltrationcapacity	[mm/day]	yes	208
HortonMaxInfCap[3]	N/A	[mm/hr]	yes	209
HortonMinInfCap	N/A	[mm/hr]	yes	209
HortonDecreaseRate	N/A	[1/hr]	yes	209
HortonRecoveryRate	N/A	[1/hr]	yes	209

D.3.2 Water levels

Water levels can be provided through all file types listed in section D.4 (except for vertical profiles, obviously). The restart file sets the water level both at the old and new time step. Non-restart files should be specified using `quantity=initialWaterlevel`.

D.3.3 Initial velocities

Initial velocities are recommended to be set via restart files, such that they exactly correspond with the model's waterdepths. They *can* also be set in the `IniFieldFile`, using `quantity=initialVelocityX/Y`, but this is not advised.

D.3.4 Salinity

Salinity concentrations from non-restart files are possible as spatially varying in the horizontal plane and with two depth options:

- ◇ depth-averaged values, so as a 2D scalar field (in case of 3D models applied uniformly in the vertical);
- ◇ or (for 3D models) as linearly interpolated values between a top layer concentration and a bottom layer concentration. This is achieved by specifying two `quantity` blocks: `initialSalinityTop` and `initialSalinity`, respectively;

[2] Only for constant infiltration (`InfiltrationModel = 2`).
[3] Only for Horton infiltration (`InfiltrationModel = 4`).

DRAFT

◇ or (for 3D models) a scalar 2D field constant in the vertical below a depth. This is achieve by providing at least a `quantity` block with `initialSalinityBot`, and the extra parameter `uniformSalinityBelowZ = ...`.

For 3D models, alternatively, spatially constant, but depth-varying salinities can be set using the `quantity=initialverticalsalinityprofile`, and a vertical profile file (see section D.4.5).

D.3.5 Temperature

Temperature values from non-restart files are possible as depth-averaged values, using `quantity=temperature`.

For 3D models, alternatively, spatially constant, but depth-varying temperatures can be set using the `QUANTITY=initialverticaltemperatureprofile`, and a vertical profile file (see section D.4.5).

D.3.6 Tracers

Initial tracer concentrations from non-restart files are similar to temperature values, now specified using `quantity=tracer<`*tracername*`>`. The set of tracer name(s) is not listed in the MDU-file explicitly, they are inferred from all supplied initial tracer definitions *and* the tracer boundary conditions (see also **??**). The <*tracername*> postfix can uniquely associate an initial `tracer` quantity with a `tracerbnd` quantity.

D.3.7 Physical coefficients

A second group of spatially varying input are the physical and numerical coefficients listed below. They allow changing of model parameters in space or in certain sub-regions of the domain, and as such these values override the uniform values from the MDU file. The input data can come from the inside-polygon file type (section D.4.2, the sample file type (section D.4.3), or the GeoTIFF raster file type (section D.4.4).

quantity name	MDU uniform equivalent	description
`frictioncoefficient`	`UnifFrictCoef`	Friction coefficient (use `IFRCTYP` and `UnifFrictType`, resp. to set the friction type.
`horizontaleddy -viscositycoefficient`	`Vicouv`	Horizontal eddy viscosity coefficient (m^2/s).
`horizontaleddy -diffusivitycoefficient`	`Dicouv`	Horizontal eddy diffusivity coefficient (m^2/s).
`backgroundverticaleddy -diffusivitycoefficient`	`Dicoww`	Background vertical eddy diffusivity coefficient (m^2/s).
`advectiontype`	`AdvecType`	Type of advection scheme.
`ibotlevtype`	`BedlevType`	Type of bed-level handling.

D.4 Supported file formats

All file formats below, except for restart files, are in *model-independent coordinates* and will be cropped and/or interpolated onto the model grid.

DRAFT

D.4.1 1D field INI file

This section describes the file format that can be used for defining a scalar quantity on a 1D network. This file format is for instance used for the initial conditions defined on a network, in which case it should be referenced as a `dataFile` inside the initial field file (section D.2).

Table D.3: *1D field definition.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`2.00`	File version. Do not edit this.
`fileType`	String	`1dField`	File type. Do not edit this.
`[Global]`			*One global block per file*
`quantity`	String		The name of the quantity
`unit`	String		The unit of the quantity
`value`	Double		The global default value for this quantity
`[Branch]`			*Optional: one block per branch*
`branchId`	String		The name of the branch
`numLocations`	Int	0	Number of locations on branch. The default 0 value implies branch uniform values.
`chainage`	Double[]		Space separated list of locations on the branch (m). Locations sorted by increasing chainage. The keyword must be specified if `numLocations`>0.
`values`	Double[]		Space separated list of `numLocations` values; one for each chainage specified. One value required if `numLocations`=0

Remarks:

- The `branchId` may contain spaces, with no quoting needed.
- This file does not include an option to specify value locations via `x` and `y`.
- The number of `values` to be specified per line equals the number of locations on that branch. If `numLocations` is not specified, 0 or 1, one value is expected which will be applied uniformly along the branch. If 2 or more locations have been specified, the values are interpolated *linearly* along the branch in between the locations and assumed constant beyond the first and last location specified.

Example:

```
[General]
    fileVersion           = 2.00
    fileType              = 1dField

[Global]
    quantity              = waterDepth
    unit                  = m
    value                 = 10.000               # default value

[Branch]
    branchId              = Channel1             # overrule for Channel1
    numLocations          = 2                    # at two locations
    chainage              = 0.000 100.000
    values                = 7.000 7.500          # 7 at 0m and 7.5 at 100m
```

DRAFT

```
[Branch]
    branchId             = Channel4             # overrule for Channel4
    values               = 8.000                # uniform value
```

D.4.2 Inside-polygon option

A spatially constant value in a restricted part of the domain can be set by specifying a polygon file (section C.2) in the ext-file as follows:

```
QUANTITY=frictioncoefficient
FILENAME=winterbed.pol
FILETYPE=10
METHOD  =4
VALUE   =55
IFRCTYP =1  # Optional, override uniform [physics] UnifFrictType=..
```

The interpolation option `METHOD=4` simply specifies the no-interpolation is to be performed, only inside-polygon cropping. For `QUANTITY=frictioncoefficient` the default friction type may be overridden with the keyword `IFRCTYP`.

D.4.3 Sample file

Spatial data from sample files (<∗.xyz>, see section C.3) is interpolated by triangle interpolation or sample averaging. The following options control the type of interpolation:

interpolation method	options	description
`METHOD=5`		Triangle interpolation
	`EXTRAPOLTOL=20`	Allow extrapolation from convex hull of samples up to . . . meters (default: 0).
`METHOD=6`		Sample averaging inside control volumes:
	`AVERAGINGTYPE=1`	Normal (unweighted) averaging
	`AVERAGINGTYPE=2`	Nearest neighbour
	`AVERAGINGTYPE=3`	Maximal value
	`AVERAGINGTYPE=4`	Minimal value
	`AVERAGINGTYPE=5`	Inverse weighted distance averaging
	`AVERAGINGTYPE=6`	Minimal absolute value (Use with care. It takes the absolute value.)
	`RELATIVESEARCHCELLSIZE =...`	Control size of search volume: 1=actual cell, 1.5=50 % larger in all directions.

D.4.4 GeoTIFF file

GeoTIFF[4] is a file format for georeferenced rasterized datasets. When providing a GeoTIFF file as an initial or parameter field (Section D.2), the model coordinate system in that file must match the coordinate system of the D-Flow FM model (e.g., geographic or projected coordinates). Also, only single layer (or band) files are currently supported.

[4] https://trac.osgeo.org/geotiff/

DRAFT

D.4.5 Vertical profile file

Initial values that are constant in 2D space, but vary in depth can be specified using vertical profile files, which are basic polyline files (section C.2). For example for a linearly varying salinity from 30 to 20 ppt from -10 to 0 m :

```
L1
2 2
-10 30
 0 20
```

The `METHOD` keyword must be specified, but is further ignored: linear interpolation is always used.

D.4.6 Map file

Initial field data from a map file may be used as non-restart input, but this is only supported currently for the legacy `MapFormat = 1` (i.e., UGRID map files are not yet supported). Values are read in from the map files as if they were samples, and further treated and interpolated as such. All interpolation options from section D.4.3 can be used here. This option may be used as a non-exact restart state when the model grid has been changed (e.g., locally refined or cropped to a subdomain).

D.4.7 Restart file

Restart files are discussed in section F.3.4. They should be included in your model in the MDU-file as follows:

```
[restart]
RestartFile     = mdu_name_yyyymmdd_HHMMSS_rst.nc
RestartDateTime = # Restart time (YYYYMMDDHHMMSS), only relevant
                  # in case of restart from *_map.nc
```

Concerning parallel runs, each subdomain can generate its own restart/map file. To restart a parallel run, one can use any of the following approaches:

- ◇ On each subdomain use its own restart/map file, provided that the partition does not change. (This means that in each partition MDU file specify its own restart/map file.)
- ◇ Merge the subdomain restart/map files to one global file (see 7.4.4.2). This file enables to restart a parallel run with a different or the same partition. (This means that specify the merged file for all the partition MDU files.) One can also run a sequential simulation with this merged restart/map file.

DRAFT

E Boundary conditions specification

Boundary conditions are part of the external forcing of a model and and as such declared in the external-forcings file (section C.6).

E.1 Supported boundary types

E.1.1 Open boundaries

E.1.2 Astronomic boundary conditions

Boundary values can be specified for any location in terms of astronomical components in attribute files with extension <cmp> (the BC-format, discussed furtheron, is supported as an alternative). Tidal motion are described as a series of simple harmonic constituent motions, each with its own characteristic period:

$$H(t) = A_0 + \sum_{i=1}^{k} A_i F(c_i) \cos\left(\frac{2\pi}{T(c_i)} t + (V_0 + u)(c_i) - G_i\right) \tag{E.1}$$

in which:

c_i	i-th component, specified by label (name)
A_i	amplitude of the i-th component
G_i	phase of the i-th component
$T(c_i)$	period
$H(t)$	boundary value at time t
A_0	constant offset value
k	number of relevant components
$F(c_i)$	nodal amplitude factor
$(V_0 + u)(c_i)$	astronomical argument

The component (by label) c_i, amplitude A_i and phase G_i for each component i are required in the <cmp>-file In addition to the primary constituents, compound and higher harmonic constituents may have to be used. This is the case in shallow water areas for example, where advection, large amplitude to depth ratio, and bottom friction give rise to non-linear interactions.

F, V_0 and u are also *time-dependent*. For a given period, their values are easily calculated or obtained various tidal year books. V_0 is a frequency dependent phase correction factor relating the local time frame of the observations to an internationally agreed celestial time frame. F_i and u_i are slowly varying amplitude and phase corrections, The variations depend on the frequency, often with a cyclic period of 18.6 years. By default, the nodal amplitude factors and astronomical arguments are re-calculated every 6 hours.

E.1.3 Astronomic correction factors

For individual astronomic components, correcting factor α_i for amplitude and offset δ_i may be specified by the user so that the effective amplitude for that component becomes $\alpha_i\, A_i$ and the effective phase becomes $G_i + \delta_i$.

E.1.4 Harmonic flow boundary conditions

Harmonic boundary conditions are specified in a manner similar to their astronomical counterpart, except that the periods (first column of a .cmp file) are declared explicitly in minutes, rather than using component names. Equation (E.1) applies with $T(c_i) = T_i$ and furthermore $F = 1$ and $(V_0 + u) = 0$.

DRAFT

E.1.5 QH-relation flow boundary conditions

In this type of boundary condition, a waterlevel is prescribed as a function of the integrated discharge through a crossection defined by the polyline of the boundary.

The relation between waterlevel and discharge is specified in a lookup-table.

Within this table linear interpolation is applied.

Note that only one waterlevel is set for the entire QH-boundary, and that its behaviour is specified by a *single* QH-table for the entire boundary.

E.1.6 Time-series flow boundary conditions

Time-series in the general time-series file format (section C.4) can be used to specify values on boundaries.

Boundary values are provided at discrete points in time, which are then used for interpolation at times in between.

Extrapolation beyond the range of specified times is not supported, Time-series are also supported at multiple vertical levels for quantities that are defined on layers (tracers, salinity, temperature, velocity). These need to be specified in the BC-format (section E.2.3).

E.1.7 Time-Series for Varying Air Density

In the computation of wind stress, it is possible to use a time- and/or spatially varying air density instead of the constant value specified by `rhoAir` in the MDU file. This air density can either be directly prescribed or computed from three other time-varying meteorological fields.

- To directly prescribe a time- and spatially varying air density, the quantity `airdensity` must be provided in the <ext> file. This file must be in netCDF format.
- To compute the air density instead, set the keyword `computedAirdensity` in the input block `[wind]` of the MDU file to 1. In this case, the quantities `airpressure`, `airtemperature`, and `dewpoint` must be available in the <ext> file. These quantities can be provided either as individual variables (e.g., `quantity = airpressure`) or as part of combined quantities (e.g., `quantity = airpressure_windx_windy`).

E.2 Boundary signal file formats

E.2.1 The <cmp> format

Warning:

- The <cmp> format is a legacy file format without any metadata. It is recommended to use the <bc> format instead `function=harmonic` or `astronomic` (see Section C.5).

```
* comments
* ...
c[0]      amplitude[0]    phase[0]
c[1]      amplitude[1]    phase[1]
...
```

where `c` represents a period in minutes or the name of a valid astronomic component. Amplitudes are assumed to be in the unit of the quantity.

E.2.2 The <qh> format

Warning:

- The <qh> format is a legacy file format without any metadata. It is recommended to use the <bc> format instead, with `function=QHTable` (see Section C.5).

```
* comments
* ...
discharge[0]      waterlevel[0]
discharge[1]      waterlevel[1]
...
```

E.2.3 The <bc> format

The external forcings file may point to this type of file via the `forcingFile` keyword (see section C.6.2). The type of forcing is not restricted to boundary conditions, but may also contain meteo time series (global or stations).

E.2.4 The netCDF-format

E.2.4.1 Scalar quantity for point-location time-series

The external forcings file may point to this type of file (for boundary condition support points or meteo station time series) via the `forcingFile` keyword (see section C.6.2).

NetCDF files that provide forcing data on point locations should meet the following structure:

- The time series have to be stored in a double variable with the following specifications:
  - it has the name `'time'`
  - it has the attribute `standard_name = 'time'`
  - it has one dimension, the time dimension

  The labels do not have to be unique.
- The location labels have to be stored in a character variable with the following specifications:
  - it has the name `'station_id'`
  - it has the attribute `cf_role = 'timeseries_id'`
  - it has two dimensions, the primary being the series dimension and the secondary the string length dimension.

  The labels do not have to be unique.
- The variable containing the timeseries data is identified by its attribute `standard_name = <quantity-name>` and has two dimensions, the series dimension and the time dimension. The latter can be the unlimited dimension.

In analogy with the ASCII BC-format (Section C.5), location label and quantity name uniquely identify the timeseries.

Example header of a netCDF file with point data:

```
netcdf timeseries {
dimensions:
        time = UNLIMITED ; // (5 currently)
        two = 2 ;
        name_len = 256 ;
        station = 3 ;
variables:
        double time(time) ;
                time:units = "seconds since 2011-12-22 00:00:00 +00:00" ;
                time:standard_name = "time" ;
```

DRAFT

```
                time:bounds = "time_bds" ;
        char station_id(station, name_len) ;
                station_id:cf_role = "timeseries_id" ;
}
```

E.2.4.2 Scalar Quantity for gridded data

The external forcings file may point to this type of file via the `forcingFile` keyword (see section C.6.2).

Gridded data is used to apply per surface external forcing and should therefor spatially be 2D. There are two different ways to supply this data: Either with a timeseries or a harmonic component.

E.2.4.2.1 timeseries

NetCDF files that provide forcing data on a grid with timeseries should meet the following structure:

- ◇ The variable containing the timeseries data is identified by its attribute `:standard_name = <quantity-name>` and has three dimensions, the time dimension and the grid coordinates. See Table C.10 for a list of quantity names and their required standard names. If no suitable standard name is available, the modeller can provide a custom variable name in the <*.ext> file using `forcingVariableName=...` (see section C.6.2.3).

Example header of a netCDF file with a gridded timeseries:

```
netcdf forcing_timeseries {
dimensions:
        time = 8761 ;
        x = 9 ;
        y = 2 ;
variables:
        double time(time) ;
                time:time_origin = "2013-01-01 00:00:00" ;
                time:long_name = "Time - minutes since 2013-01-01 00:00:00 +00:00" ;
                time:standard_name = "time" ;
                time:calender = "gregorian" ;
                time:units = "minutes since 2013-01-01 00:00:00 +00:00" ;
        double x(x) ;
                x:standard_name = "longitude" ;
                x:long_name = "longitude" ;
                x:units = "degrees_east" ;
        double y(y) ;
                y:standard_name = "latitude" ;
                y:long_name = "latitude" ;
                y:units = "degrees_north" ;
        double air_pressure(time, y, x) ;
                air_pressure:coordinates = "Time y x" ;
                air_pressure:long_name = "Atmospheric Pressure" ;
                air_pressure:standard_name = "air_pressure" ;
                air_pressure:units = "Pa" ;
}
```

E.2.4.2.2 Harmonic component

When the external per surface forcing is periodic in nature, an alternate format can be used in which the data is expressed in the form of a harmonic component. This form generally allows the forcing file to be much smaller while it also provides flexibility in extrapolation.

NetCDF files that provide forcing data on a grid with a harmonic component should meet the following structure:

- ◇ The variable containing the amplitude data is identified by its attribute `standard_name = <quantity-name>` and has two dimensions, the grid coordinates.
- ◇ The phase offset is identified by the variable name `phase` and should have the same coordinate dimensions as the amplitude data variable and `phase:units = "deg"`.
- ◇ The reference time for which the phase offset has been supplied should be defined a global attribute named `reference_time_of_component_phase`.
- ◇ The component's period in seconds should be defined a global attribute named `component_period_in_seconds`.

When using this harmonic component format, the timeseries in each grid point i will be given by:

$$H_i(t) = A_i \cos\left(\frac{2\pi}{T}(t - t_{\mathtt{ref}}) - \frac{\pi}{180}G_i\right) \tag{E.2}$$

in which:

A_i	amplitude of the component at grid point i
T	period of the component (seconds)
$t_{\mathtt{ref}}$	reference time of the component phase (seconds)
G_i	phase of the component at grid point i (degrees)

Example header of a netCDF file with a gridded harmonic component:

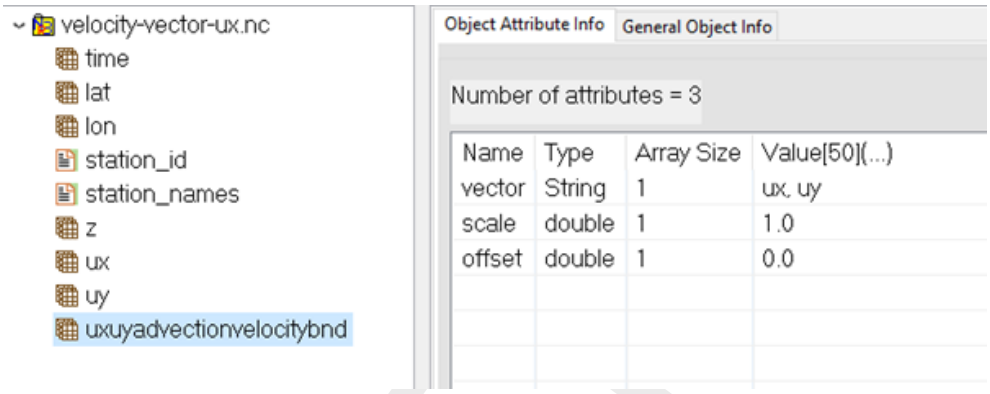
```
netcdf forcing_component_s1 {
dimensions:
        x = 9 ;
        y = 2 ;
variables:
        double x(x) ;
                x:standard_name = "longitude" ;
                x:long_name = "longitude" ;
                x:units = "degrees_east" ;
        double y(y) ;
                y:standard_name = "latitude" ;
                y:long_name = "latitude" ;
                y:units = "degrees_north" ;
        double amplitude(y, x) ;
                amplitude:coordinates = "y x" ;
                amplitude:long_name = "amplitude_of_airpressure" ;
                amplitude:standard_name = "air_pressure" ;
                amplitude:units = "Pa" ;
        double phase(y, x) ;
                phase:coordinates = "y x" ;
                phase:long_name = "phase_of_component" ;
                phase:standard_name = "phase" ;
                phase:units = "deg" ;

        // global attributes:
                :component = "S1" ;
                :component_period_in_seconds = "86400" ;
                :reference_time_of_component_phase = "2013-01-01 00:00:00 +00:00" ;
}
```

E.2.4.3 Vector quantity

NetCDF file can hold data for a vector quantity. For example, **uxuyadvectionvelocitybnd** boundary condition has X and Y velocity components. To hold this vector quantity, NetCDF file has to have an empty quantity with the name **uxuyadvectionvelocitybnd** in addition to scalar quantities for the velocity components. This empty quantity has to have a string attribute named **vector** that contains the vector element names: X and Y components. These names are separated by comma. The picture below shows an example when the string contains quantities' names **ux** and **uy**. As well the string can contain standard and/or long names.

DRAFT



Remark:

- Be sure that names in the string are not truncated due the string size.

DRAFT

F Output files

D-Flow FM can produce several types of output into several different files. This chapter summarizes the types of output and how to configure them in them model definition. We distinguish five types of output here:

1 The diagnostics file, a log file with live details of a single model run.
2 NetCDF output files for history, map and restart data.
3 Timings file with performance statistics.
4 Shapefiles.

F.1 Diagnostics file

The diagnostics (dia) file is the log file of a single model simulation run. It is an important file to check for warning or error messages, when suspecting a faulty model run. The filename is automatically chosen as <*mdu_name*.dia>. For parallel model runs, each process writes its own file <*mdu_name*_000X.dia>. The dia file has the following global structure:

```
Various INFO/DEBUG messages:
** INFO   : Opened file : unstruc.hlp
[..]
The version of D-Flow FM used for this calculation:
* Deltares, D-Flow FM Version 1.1.145.41271M, Aug 11 2015, 18:05:31
Date and time of model run start:
File creation date: 18:27:44, 06-09-2015
[..]

Check for possible model initialization errors:
** WARNING: readMDUFile: [numerics] cflwavefrac=0.1 was in file, but not used.
[..]

Upon successful initialization, a full printout of the effective model settings is given:
** INFO   : ** Model initialization was successful **
** INFO   : * Active Model definition:
# Generated on 18:27:45, 06-09-2015
# Deltares, D-Flow FM Version 1.1.145.41271M, Aug 11 2015, 18:05:31
[general]
[..]

Next, the time loop is started:
** INFO   : **
** INFO   : Writing initial output to file(s)...
** DEBUG  : Opened netCDF file 'DFM_OUTPUT_035hammen/035hammen_his.nc' as #3.
** DEBUG  : Opened netCDF file 'DFM_OUTPUT_035hammen/035hammen_map.nc' as #4.
** INFO   : Done writing initial output to file(s).
[..]

Finishing summary of model run:
** INFO   : simulation period     (s)  :          6400.0000000000
** INFO   : nr of timesteps       ( )  :           771.0000000000
** INFO   : average timestep      (s)  :             8.3009079118
[..]
Some basic run time statistics
** INFO   : time steps            (s)  :            76.2840000003
** INFO   : Computation started  at: 18:27:45, 06-09-2015
** INFO   : Computation finished at: 18:29:04, 06-09-2015
** INFO   :
Which parallellization options have been active in this run:
** INFO   : MPI    : no.
** INFO   : OpenMP : yes.
        #threads max : 8
```

F.2 Demanding output

The configuration of what output should be produced during a model run is via the MDU file, and several attribute files for history files.

DRAFT

F.2.1 The MDU-file

The MDU-file has an `[output]`-section, with a range of key-value-pair options. See Table A.1 on page 333 for all available output options. Enabling the various output files begins with setting the output interval to a value > 0, see for example `MapInterval` and `HisInterval` in Table A.1.

F.2.2 Observation points

The observation point file defines the locations for time series output of flow "point" variables such as water level and water depth. Observation points in 2D (or 3D) parts of the model domain are used to monitor quantities at grid cell centres, and equivalently at the 1D grid points if the station is snapped to a 1D grid point.

The keyword `ObsFile` should be used to point to the observation point file; optionally a space separated list of files can be specified (see chapter A).

Two types of observation point files are supported: $<*$_obs.ini$>$ files and old $<*$.xyn$>$ files. Both file types are described in the subsections below; the first file type is recommended. The following general remarks hold for both types of input:

Remarks:
- ◇ The observation points are stationary by default.
- ◇ Observation points may have non-unique names (also together with moving observation points), but the results can only be distinguished by the column number in the output file, and this practice is not advised.
- ◇ In 2D/3D, the x,y-locations are snapped to the grid cell in which they lie. When an observation point lies on a grid cell edge, snapping results are unpredictable.
- ◇ In 1D, the observation point locations are snapped to the closest computational point; this may not always match the grid cell in which the point is located.
- ◇ Time series are reported for the cell-centered solution data, that is: *no* interpolation to the exact x,y-location takes place.

F.2.2.1 The observation point file with extension $<*$_obs.ini$>$

This file can be used to define observation points in a more detailed way, amongst others in 1D using (`branchId`, `chainage`), but see below for more possibilities.

Table F.1: *Definition of observation points.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`2.00`	File version. Do not edit this.
`fileType`		`obsPoints`	File type. Do not edit this.
`[ObservationPoint]`			
`name`	String		Name of the observation point (max. 255 characters).
`locationType`	String	`2d`	(optional) Only when `x` and `y` are also specified. `1d`: snap to closest 1D grid point, `2d`: snap to closest 2D grid cell centre, `all`: snap to closest 1D or 2D point.
`branchId`	String		(optional) Branch on which the observation point is located.
`chainage`	Double		(optional) Location on the branch (m).
`x`	Double		(optional in 1D) x-coordinate of the location of the observation point.

(continued on next page)

Keyword	Type	Default	Description
			(continued from previous page)
y	Double		(optional in 1D) y-coordinate of the location of the observation point.
locationFile	String		(optional) Specifically for moving observation points: name of a <∗.bc> containing the timeseries for the x and y coordinates. See Section F.2.3 for an example.
locationFileType	String		Type of locationFile. Supported type: ◇ bcAscii: Space-uniform time series for x and y in <*.bc> (Section C.5).

Remarks:

- ◇ The station id/name may contain spaces, with no quoting needed.
- ◇ If locationType = all then the observation point is first checked whether it lies inside a 2D grid cell. When this is not the case, in a second attempt it is snapped to the nearest 1D grid point.
- ◇ Three types of location specifications are available.
  - ◇ When branchId is given, the keywords branchId and chainage will be used for the location specification (only for 1D, the locationType is automatically assumed to be 1d).
  - ◇ When branchId is *not* given, but keywords x and y are, then these fixed coordinates will be used. By default this is assumed to be a 2D/3D station location (i.e. locationType = 2d). Optionally, this location can be snapped to 1D calculation points using locationType = 1d.
  - ◇ Finally, when only locationFile is given, that is considered to be a file with timeseries for the x,y coordinates of a moving observation station. See Section F.2.3 for an example.

Example:

```
[General]
    fileVersion  = 2.00
    fileType     = obsPoints
[ObservationPoint]
    name         = ABDN
    x            = -2.06250000e+00
    y            = 5.71583333e+01
[ObservationPoint]
    name         = AVMH
    x            = -2.73750000e+00
    y            = 5.15083334e+01
[ObservationPoint]
    name         = St Helier Jersey
    x            = -2.11250000e+00
    y            = 4.91750000e+01
[ObservationPoint]
    name         = Obs4
    branchId     = Channel1
    chainage     = 275.0
[ObservationPoint]
    name         = Obs5
    locationType = 1d
    x            = -2.31250000e+00
    y            = 5.01750000e+01
```

F.2.2.2 The observation point file with extension <∗.xyn>

The observation point file with extension <∗.xyn> is basically like a sample file (section C.3), but now with station id strings in the third column:

DRAFT

```
-2.06250000e+00 5.71583333e+01 ABDN
-2.73750000e+00 5.15083334e+01 AVMH
-2.11250000e+00 4.91750000e+01 'St Helier Jersey'
```

Remarks:

- ◇ The station id/name may contain spaces, in which case it should be surrounded by single quotes, `'as␣follows'`. Maximum length is 255 characters.
- ◇ Each observation point is first checked whether it lies inside a 2D grid cell. When this is not the case, in a second attempt it is snapped to the nearest 1D grid point. (Equivalent to `locationType = all`.)

F.2.3 Moving observation points

Moving observation points are specified via the <*_obs.ini>, using `locationFile=example.bc` Below is an example of a <*.bc> file containing the moving point coordinates for a point with `name = ship1`:

```
[Forcing]
name             = ship1
function         = timeseries
timeInterpolation = linear
quantity         = time
unit             = minutes since 1992-08-31 00:00:00 +00:00
vector           = movingstationtxy:xCoordinate,yCoordinate
quantity         = xCoordinate
unit             = -
quantity         = yCoordinate
unit             = -
0      1140.          100.
1      1140.           50.
2      180.           50.
3      180.          100.
4      140.          100.
5      140.           50.
6      180.           50.
7      180.          100.
8      140.          100.
```

Warning:

- ◇ This example below describes deprecated functionality using the old external forcings format. Use the new <*_obs.ini> file format instead, with keyword `locationFile`, see Section F.2.2.1.

Moving observation points are specified via the <∗.ext>-file:

```
QUANTITY=movingstationtxy
FILENAME=movingstation1.tim
FILETYPE=1
METHOD=1
OPERAND=O
```

The <∗.tim> file (one for each moving observation point) is a standard time series file (section C.4) with three columns, containing the time in minutes since the reference data, the x- and y-position of the moving station at that time.

Remarks:

DRAFT

- ◇ Stationary and moving observation points form one large set of observation points in the output his file.
- ◇ No space or time interpolation is done: the reported values are instantaneous values for the grid cell in which the moving observation point lies at that each output time.

F.2.4 Observation cross sections

Observation cross sections (or cross sections for short) define the location for time series output of flow "flux" variables such as discharge and velocity. Other typical output quantities are instantaneous as well as cumulative, space-integrated discharges and constituent transport. The keyword `CrsFile` should be used to point to the observation cross section file; optionally a space separated list of files can be specified (see chapter A).

The location of an observation cross section can be specified using polylines (typically in 2D) or single points (only in 1D). Two types of cross section files are supported: <∗.ini> files and old <∗_crs.pli> files. Both file types are described in the subsections below; the first file type is recommended. The following general remarks hold for both types of input:

Remarks:

- ◇ Observation cross sections may have non-unique names, but the results can only be distinguished by the column number in the output file, and this practice is not advised.
- ◇ A cross section polyline is snapped to all grid cell edges (i.e., velocity points), whose flow link is intersected by the polyline.
- ◇ The flow data on velocity points is integrated along the polyline and across all vertical layers, resulting in a single time series per cross section and flow quantity.
- ◇ When a single polyline happens to intersect both 1D and 2D flow links, all of these are included and flow data is integrated for 1D and 2D together.

F.2.4.1 Observation cross section file with extension <∗_crs.ini>

This file can be used to define cross sections in a more detailed way, amongst others in 1D using (`branchId`, `chainage`), but see below for more possibilities.

Table F.2: *Definition of observation cross sections.*

Keyword	Type	Default	Description
`[General]`			
`fileVersion`	String	`2.00`	File version. Do not edit this.
`fileType`		`obsCross`	File type. Do not edit this.
`[ObservationCrossSection]`			
`name`	String		Name of the cross section (max. 255 characters).
`branchId`	String		(optional) Branch on which the cross section is located
`chainage`	Double		(optional) Location on the branch (m)
`numCoordinates`	int		(optional) number of values in xCoordinates and yCoordinates. This value should be greater or equal 2.
`xCoordinates`	Double []		(optional) x-coordinates of the cross section line. (number of values = `numCoordinates`)
`yCoordinates`	Double []		(optional) y-coordinates of the cross section line. (number of values = `numCoordinates`)

DRAFT

Remarks:

- ◇ The cross section id/name may contain spaces, with no quoting needed.
- ◇ Two types of location specifications are available.
  - ◇ When `branchId` is given, the keywords `branchId` and `chainage` will be used for the location specification (only for 1D).
  - ◇ When `branchId` is *not* given, the keywords `numCoordinates`, `xCoordinates` and `yCoordinates` will be used to define a polyline. Output will be accumulated over all flow links that the polyline intersects.

Example:

```
[General]
    fileVersion  = 2.00
    fileType     = obsCross
[ObservationCrossSection]
    name           = CrossSec1
    numCoordinates = 4
    xCoordinates   = 132345.0 132165.0 131940.0 131820.0
    yCoordinates   = 549030.0 549285.0 549550.0 549670.0
[ObservationCrossSection]
    name           = CrossSec2
    numCoordinates = 3
    xCoordinates   = 131750.0 131595.0 131415.0
    yCoordinates   = 549865.0 550025.0 550175.0
[ObservationCrossSection]
    name           = CrossSec3
    branchId       = Channel1
    chainage       = 300.000
```

F.2.4.2 Observation cross section file with extension <*_crs.pli>

The format of the <*_crs.pli> file is identical to that of a general polyline/polygon file (section C.2).

```
CrossSec1
4  2
            132345.0    549030.0
            132165.0    549285.0
            131940.0    549550.0
            131820.0    549670.0
CrossSec2
3  2
            131750.0    549865.0
            131595.0    550025.0
            131415.0    550175.0
```

Remarks:

- ◇ The first header line of each polyline block contains the name of each cross section.
- ◇ The cross section id/name may contain spaces, no quotes or other delimiters must be used.

F.3 NetCDF output files

All flow data output produced by D-Flow FM is written in the netCDF format. This cross-platform storage format is widely used in the world. The contents of a file can be documented using variable attributes and standardized meta data.

The following conventions are used in some of our output file types:

- ◇ CF-conventions for both the timeseries (his) files and the spatial (map/class map/Fourier) files. More information on: https://cfconventions.org/.
- ◇ UGRID-conventions for the unstructured grids in the spatial (map/class map/Fourier) files. More information on: http://ugrid-conventions.github.io/ugrid-conventions/.

DRAFT

Additionally, for 1D and 1D2D grids, the Deltares-extension on top of UGRID is used. More information about this is in Section B.2.

- ACDD-conventions for the geospatial and time bounds in the spatial (map/class map/Fourier) files. More information on: https://wiki.esipfed.org/Category:Attribute_Conventions_Dataset_Discovery.

Spherical coordinates in NetCDF output files

The coordinate variables in all NetCDF output files are in the same coordinate system as the input grid file (<_net.nc>). If this input file contains metadata describing grid mapping/coordinate transformation, that metadata is also copied over into most output NetCDF files (specifically: into all UGRID-type map/class map/Fourier files, and the history files). For models using a Cartesian/metric coordinate system, an option is available to automatically add the equivalent latitude and longitude coordinate values for all locations in the NetCDF output files. To enable this feature:

- Verify that the input <_net.nc> file contains a grid mapping variable with coordinate transformation parameters. In order to be found, the grid mapping variable should be called `projected_coordinate_system`, or have the attribute `:grid_mapping_name`. Additionally, it must have an attribute `:proj4_params` containing a PROJ-compatible coordinate transformation string[1].
- If the input contains the above metadata, switch on the lat-lon conversion in the MDU file using keyword `[output] NcWriteLatLon = 1`.

The fragment below shows an example of NetCDF output for a map file, where the input contained a detailed grid mapping variable for UTM zone 32N:

```
netcdf example_map {
// [..]
variables:
        int projected_coordinate_system ;
                projected_coordinate_system:name = "ETRS89 / UTM zone 32N" ;
                projected_coordinate_system:epsg = 25832 ;
                projected_coordinate_system:grid_mapping_name = "transverse_mercator" ;
                projected_coordinate_system:longitude_of_prime_meridian = 0. ;
                projected_coordinate_system:semi_major_axis = 6378137. ;
                projected_coordinate_system:semi_minor_axis = 6356752.31414036 ;
                projected_coordinate_system:inverse_flattening = 298.257222101 ;
                projected_coordinate_system:EPSG_code = "EPSG:25832" ;
                projected_coordinate_system:value = "value is equal to EPSG code" ;
                projected_coordinate_system:proj4_params =
                        "+proj=utm +zone=32 +ellps=GRS80 +towgs84=0,0,0,0,0,0,0 +units=m +no_defs" ;
                projected_coordinate_system:projection_name = "unknown" ;
                projected_coordinate_system:wkt = "PROJCS[\"ETRS89 / UTM zone 32N\",\n",
                        "    GEOGCS[\"ETRS89\",\n",
                        "        DATUM[\"European_Terrestrial_Reference_System_1989\",\n",
                        "            SPHEROID[\"GRS 1980\",6378137,298.257222101,\n",
                        "                AUTHORITY[\"EPSG\",\"7019\"]],\n",
                        "            TOWGS84[0,0,0,0,0,0,0],\n",
                        "            AUTHORITY[\"EPSG\",\"6258\"]],\n",
                        "        PRIMEM[\"Greenwich\",0,\n",
                        "            AUTHORITY[\"EPSG\",\"8901\"]],\n",
                        "        UNIT[\"degree\",0.0174532925199433,\n",
                        "            AUTHORITY[\"EPSG\",\"9122\"]],\n",
                        "        AUTHORITY[\"EPSG\",\"4258\"]],\n",
                        "    PROJECTION[\"Transverse_Mercator\"],\n",
                        "    PARAMETER[\"latitude_of_origin\",0],\n",
                        "    PARAMETER[\"central_meridian\",9],\n",
                        "    PARAMETER[\"scale_factor\",0.9996],\n",
                        "    PARAMETER[\"false_easting\",500000],\n",
                        "    PARAMETER[\"false_northing\",0],\n",
                        "    UNIT[\"metre\",1,\n",
                        "        AUTHORITY[\"EPSG\",\"9001\"]],\n",
                        "    AXIS[\"Easting\",EAST],\n",
                        "    AXIS[\"Northing\",NORTH],\n",
                        "    AUTHORITY[\"EPSG\",\"25832\"]]" ;
// [..]
        double mesh2d_node_x(mesh2d_nNodes) ;
                mesh2d_node_x:units = "m" ;
```

[1] https://proj.org/usage/projections.html

```
                mesh2d_node_x:standard_name = "projection_x_coordinate" ;
                mesh2d_node_x:long_name = "x-coordinate of mesh nodes" ;
        double mesh2d_node_y(mesh2d_nNodes) ;
                mesh2d_node_y:units = "m" ;
                mesh2d_node_y:standard_name = "projection_y_coordinate" ;
                mesh2d_node_y:long_name = "y-coordinate of mesh nodes" ;
        double mesh2d_node_lon(mesh2d_nNodes) ;
                mesh2d_node_lon:units = "degrees_east" ;
                mesh2d_node_lon:standard_name = "longitude" ;
                mesh2d_node_lon:long_name = "longitude coordinate of mesh nodes" ;
        double mesh2d_node_lat(mesh2d_nNodes) ;
                mesh2d_node_lat:units = "degrees_north" ;
                mesh2d_node_lat:standard_name = "latitude" ;
                mesh2d_node_lat:long_name = "latitude coordinate of mesh nodes" ;
```

Detailed per NetCDF output file type

The following sections describe the specific details for each type of output file.

F.3.1 Timeseries as netCDF his-file

The history file <*mdu_name*_his.nc> contains the dimensions and variable definitions in its header, followed by the actual data. For parallel runs, all data is gathered by domain #0 into <*mdu_name*_0000_his.nc> (section 7.4.4).

F.3.1.1 Dimensions

The following dimensions are defined in a his file header:

```
netcdf weirfree_his {
dimensions:
       time = UNLIMITED ; // (64 currently)
       name_len = 64 ;
       stations = 18 ;
       cross_section = 4 ;
       cross_section_name_len = 64 ;
       cross_section_pts = 3 ;
       npumps = 2 ;
       // and possibly nweirgens, nweirs, ngategens, ngates, nsources/sinks
```

F.3.1.2 Location variables

The original location and IDs for stations, cross sections and sources/sinks are also included in the his file:

```
variables:
 double station_x_coordinate(stations) ;
   station_x_coordinate:units = "m" ;
   station_x_coordinate:standard_name = "projection_x_coordinate" ;
   station_x_coordinate:long_name = "x-coordinate" ;
 double station_y_coordinate(stations) ;
   station_y_coordinate:units = "m" ;
   station_y_coordinate:standard_name = "projection_y_coordinate" ;
   station_y_coordinate:long_name = "x-coordinate" ;
 char station_name(stations, name_len) ;
   station_name:cf_role = "timeseries_id" ;
   station_name:long_name = "Observation station name" ;

 double cross_section_x_coordinate(cross_section, cross_section_pts) ;
   cross_section_x_coordinate:units = "m" ;
   cross_section_x_coordinate:standard_name = "projection_x_coordinate" ;
   cross_section_x_coordinate:long_name = "x-coordinate" ;
 double cross_section_y_coordinate(cross_section, cross_section_pts) ;
   cross_section_y_coordinate:units = "m" ;
```

```
    cross_section_y_coordinate:standard_name = "projection_y_coordinate" ;
    cross_section_y_coordinate:long_name = "y-coordinate" ;
  char cross_section_name(cross_section, cross_section_name_len) ;

  double source_sink_x_coordinate(source_sink, source_sink_pts) ;
    source_sink_x_coordinate:units = "m" ;
    source_sink_x_coordinate:standard_name = "projection_x_coordinate" ;
    source_sink_x_coordinate:long_name = "x-coordinate" ;
  double source_sink_y_coordinate(source_sink, source_sink_pts) ;
    source_sink_y_coordinate:units = "m" ;
    source_sink_y_coordinate:standard_name = "projection_y_coordinate" ;
    source_sink_y_coordinate:long_name = "y-coordinate" ;
  char source_sink_name(source_sink, source_sink_name_len) ;
```

F.3.1.3 Overview of output quantities in a netCDF his-file

An overview of output quantities in a his-file is given in Table F.3 and the following subsections. These output quantities are present in the his-file only when the output object exists in the model. For example, quantities on observation stations are present only when observation station(s) exists in the model.

DRAFT

Table F.3: *Global overview of output quantities in the history file.*

Quantity name	MDU keywords for output	More information
Basic condition for his output	`HisInterval > 0`	See Section F.2.1.
Quantities on observation stations.		See section F.3.1.4.
Quantities on observation cross sections.		See section F.3.1.5.
Quantities on runup gauges.		
Quantities on sources and sinks.	`wriHis_sourcesink > 0`.	
Quantities for mass balance.	`wriHis_balance > 0`.	See section F.3.1.6.
Quantities on hydraulic structures.		See section F.3.1.9.
Quantities on laterals.	`wriHis_lateral > 0`.	See section F.3.1.7.
Quantities for dredging and dumping.	`[sediment] DredgeFile` is specified.	
Quantities for checkerboard monitor.	`[numerics] checkerboardmonitor = 1`.	
`time`		Instant time.
`timestep`		Latest computational timestep size in each output interval [s].

DRAFT

F.3.1.4 Variables on stations

Variables on observations station are typically defined as follows:

```
double waterlevel(time, stations) ;
  waterlevel:standard_name = "sea_surface_level" ;
  waterlevel:long_name = "Water level" ;
  waterlevel:units = "m" ;
  waterlevel:coordinates = "station_x_coordinate station_y_coordinate station_name" ;
  waterlevel:_FillValue = -999. ;
```

Output for observation stations is available as time series in the history file. The output is shown in history file when one or more observation stations are defined in the model. The available output quantities are listed in Table F.4. These output variables are quantities locating on the observation stations. Such output can be switched on/off by setting value 1/0, respectively, to MDU `[output]` keywords, as listed in the third column of Table F.4 (More details about these MDU keywords refer to Table A.1.) If for one variable there is no MDU keyword shown in the third column, then the output of this variable is always switched on.

DRAFT

Table F.4: *Output quantities of observation stations in history file.*

Quantity name	Description [Unit]	MDU `[output]` keyword to switch on/off this output
`station_id`	Identification string of the observation stations.	
`station_name`	Names of the observation stations.	
`station_x_coordinate`	Original x-coordinates of stations (non-snapped) [m].	
`station_y_coordinate`	Original y-coordinates of stations (non-snapped) [m].	
# The following variables are only for 3D models:		
`zcoordinate_c`	Vertical coordinate at center of flow element and layer [m].	`wriHis_zcor` (default=1)
`zcoordinate_w`	Vertical coordinate at centre of flow element and at layer interface [m].	`wriHis_zcor` (default=1)
`zcoordinate_wu`	Vertical coordinate at edge of flow element and at layer interface [m].	`wriHis_zcor` (default=1)
# The above variables are only for 3D models.		
`station_geom`	Geometry variables, see Section F.3.1.10.	
# End of time-independent variables.		
`waterlevel`	Water level [m].	`wriHis_waterlevel_s1` (default=1)
`bedlevel`	Bottom level [m].	`wriHis_bedlevel` (default=1)
`waterdepth`	Water depth [m].	`wriHis_waterdepth` (default=0)
`tausx`, `tausy`	x-, y-components of mean bottom shear stress vector [Pa].	`wriHis_taucurrent` (default=1)
`x_velocity`, `y_velocity`	x-, y-components of flow element center velocity vector [m/s].	`wriHis_velocity_vector` (default=1)
# Following are only for 3D models:		
`z_velocity`	z-component of flow element center velocity vector [m/s].	`wriHis_velocity_vector` (default=1)
`depth-averaged_x_velocity`, `depth-averaged_y_velocity`	x-, y-components of flow element depth-averaged center velocity vector [m/s].	`wriHis_velocity_vector` (default=1)
`tke`	Turbulent kinetic energy [m^2/s^2] when MDU keyword `[numerics] Turbulencemodel >= 3.`	`wriHis_turbulence` (default=1)
`vicww`	Turbulent vertical eddy viscosity [m^2/s] when MDU keyword `[numerics] Turbulencemodel > 1.`	`wriHis_turbulence` (default=1)
`eps`	Turbulent energy dissipation [m^2/s^3] when MDU keyword `[numerics] Turbulencemodel = 3.`	`wriHis_turbulence` (default=1)

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	Description	MDU `[output]` keyword to switch on/off this output
`tau`	Turbulent time scale [1/s] when MDU keyword `[numerics] Turbulencemodel = 4`.	`wriHis_turbulence` (default=1)
`rich`	Richardson Nr [-].	`Richardsononoutput` (default=0)
# The above are only for 3D models.		
`salinity`	Salinity [ppt] when MDU keyword `[physics] Salinity > 0`.	`wriHis_salinity` (default=1)
`velocity_magnitude`	Velocity magnitude [m/s]. Or Eulerian velocity magnitude [m/s] when MDU keyword `[output] EulerVelocities` equals 1.	`wriHis_velocity` (default=1)
`discharge_magnitude`	Average discharge magnitude [m^3/s].	`wriHis_discharge` (default=1)
`R`	Roller energy per square meter [J/m^2] when MDU keyword `[waves] Wavemodelnr = 4`.	
# The following variables are for models when MDU keyword `[waves] Wavemodelnr > 0`:		
`hwav`	Significant wave height [m]. Or root mean square wave height based on wave energy [m] when MDU keyword `[waves] jahissigwav = 0`.	`wriHis_waves` (default=1)
`twav`	Wave period [s].	`wriHis_waves` (default=1)
`phiwav`	Wave from direction [deg from N].	`wriHis_waves` (default=1)
`rlabda`	Wave length [m].	`wriHis_waves` (default=1)
`uorb`	Orbital velocity [m/s].	`wriHis_waves` (default=1)
`ustokes`, `vstokes`	x-, y-components of Stokes drift [m/s].	`wriHis_waves` (default=1)
# The above variables are for models when MDU keyword `[waves] Wavemodelnr > 0`.		
`wtau`	Mean bed shear stress [Pa].	`wriHis_taucurrent` (default=1)
# The following variables are for models with temperature:		
`temperature`	Temperature [°C] when MDU keyword `[physics] Temperature > 0`.	`wriHis_temperature` (default=1)
`wind`	Wind speed [m/s] when MDU keyword `[physics] Temperature > 1`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`Tair`	Air temperature [°C] when MDU keyword `[physics] Temperature > 1`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`rhum`	Relative humidity [-] when MDU keyword `[physics] Temperature = 5`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`clou`	Cloudiness [-] when MDU keyword `[physics] Temperature = 5`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	Description	MDU `[output]` keyword to switch on/off this output
`Qsun`	Solar influx [W/m^2] when MDU keyword `[physics] Temperature = 5`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`Qeva`	Evaporative heat flux [W/m^2] when MDU keyword `[physics] Temperature = 5`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`Qcon`	Sensible heat flux [W/m^2] when MDU keyword `[physics] Temperature = 5`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`Qlong`	Long wave back radiation [W/m^2] when MDU keyword `[physics] Temperature = 5`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`Qfreva`	Free convection evaporative heat flux [W/m^2] when MDU keyword `[physics] Temperature = 5`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`Qfrcon`	Free convection sensible heat flux [W/m^2] when MDU keyword `[physics] Temperature = 5`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
`Qtot`	Total heat flux [W/m^2] when MDU keyword `[physics] Temperature > 1`.	`wriHis_temperature` and `wriHis_heat_fluxes` (default=1)
# The above variables are for models with temperature.		
`density`	Density [kg/m^3] when MDU keyword `[physics] Temperature` or `[physics] Salinity` or `[sediment] Sedimentmodelnr` is larger than 0.	`wriHis_density` (default=1)
Names of all constituents	All constituents [unit depends] when there is at least one constituent.	`wriHis_constituents` (default=1)
Names of all water quality bottom variables	All water quality bottom variables [unit depends] when there is at least one water quality bottom variable.	`wriHis_wqBot` (default=1)
Names of all water quality bottom variables with suffix `_3D`	All water quality bottom variables [unit depends] when there is at least one water quality bottom variable.	`wriHis_wqBot3d` (default=0)
`water_quality_output_j, j=1,...,noout_user`	Water quality output (from 1 to number of user outputs) [unit depends] when water quality process is either substances initiated or processes activated.	
`water_quality_stat_j, j=1,...,noout_statt`	Water quality statistic variables (from 1 to number of statistic outputs) [unit depends] when water quality process is either substances initiated or processes activated.	
# The following are sediment transport variables:		
# for model when both MDU keywords `[sediment] SedFile` and `MorFile` are not empty, and `Sedimentmodelnr` is 4.		
`sedfrac_name`	Sediment fraction identifier [-] when the computed first sediment fraction is positive.	`wriHis_sediment` (default=1)

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	Description	MDU `[output]` keyword to switch on/off this output
`seddif`	(Only for 3D models) sediment vertical diffusion [m^2/s] when the computed first sediment fraction is positive.	`wriHis_sediment` (default=1)
`sed`	Sediment concentration [kg/m^3] when the computed first sediment fraction is positive.	`wriHis_sediment` (default=1)
`ws`	Sediment settling velocity [m/s] when the computed first sediment fraction is positive.	`wriHis_sediment` (default=1)
`taub`	Bed shear stress for morphology [Pa] when MDU keywords `[sediment] Sedimentmodelnr` is positive, and MorFile keyword `[Output] taub` is true.	`wriHis_sediment` (default=1)
`sbcx`, `sbcy`	x-, y-components of current related bedload transport, with unit [kg/s/m] if MorFile keyword `[Output] TranspType = 0`, or [m^3/s/m] if MorFile keyword `[Output] TranspType = 1 or 2`. Output of these variables is available when MDU keywords `Sedimentmodelnr` is positive, and MorFile keyword `[Output] BedTranspDueToCurrentsAtZeta` is true.	`wriHis_sediment` (default=1)
`sbwx`, `sbwy`	x-, y-components of wave related bedload transport, with unit [kg/s/m] if MorFile keyword `[Output] TranspType = 0`, or [m^3/s/m] if MorFile keyword `[Output] TranspType = 1 or 2`. Output of these variables is available when MorFile keyword `[Output] BedTranspDueToWavesAtZeta` is true, and MDU keywords `[waves] Wavemodelnr > 0`, and `[waves] flowWithoutWaves` is false.	`wriHis_sediment` (default=1)
`sswx`, `sswy`	x-, y-components of wave related suspended transport, with unit [kg/s/m] if MorFile keyword `[Output] TranspType = 0`, or [m^3/s/m] if MorFile keyword `[Output] TranspType = 1 or 2`. Output of these variables is available when MorFile keyword `[Output] SuspTranspDueToWavesAtZeta` is true, and MDU keywords `[waves] Wavemodelnr > 0`, and `[waves] flowWithoutWaves` is false.	`wriHis_sediment` (default=1)
`sscx`, `sscy`	x-, y-components of current related suspended transport, with unit [kg/s/m] if MorFile keyword `[Output] TranspType = 0`, or [m^3/s/m] if MorFile keyword `[Output] TranspType = 1 or 2`. Output of these variables is available when when MorFile keyword `[Output] SuspTranspDueToCurrentsAtZeta` is true.	`wriHis_sediment` (default=1)
`sourse`	Source aterm suspended sediment transport [kg/m^3/s], when MorFile keyword `[Output] SourceSinkTerms` is true.	`wriHis_sediment` (default=1)
`sinkse`	Sink term suspended sediment transport [1/s], when MorFile keyword `[Output] SourceSinkTerms` is true.	`wriHis_sediment` (default=1)
`bodsed`	Available sediment mass in the bed [kg/m^2], when MorFile keyword `[Underlayer] IUnderLyr = 1`.	`wriHis_sediment` (default=1)

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	Description	MDU `[output]` keyword to switch on/off this output
`dpsed`	Sediment thickness in the bed [m], when MorFile keyword `[Underlayer] IUnderLyr = 1.`	`wriHis_sediment` (default=1)
`msed`	Available sediment mass in a layer of the bed [kg/m^2], when MorFile keyword `[Underlayer] IUnderLyr = 2.`	`wriHis_sediment` (default=1)
`thlyr`	Thickness of a layer of the bed [m], when MorFile keyword `[Underlayer] IUnderLyr = 2.`	`wriHis_sediment` (default=1)
`poros`	Porosity of a layer of the bed [-], when MorFile keywords `[Underlayer] IUnderLyr = 2` and `[Underlayer] IPorosity > 0.`	`wriHis_sediment` (default=1)
`lyrfrac`	Volume fraction in a layer of the bed [m], when MorFile keyword `[Underlayer] IUnderLyr = 2.`	`wriHis_sediment` (default=1)
`frac`	Availability fraction in top layer [-], when MorFile keyword `[output] frac` is true.	`wriHis_sediment` (default=1)
`mudfrac`	Mud fraction in top layer [-], when MorFile keyword `[output] MudFrac` is true.	`wriHis_sediment` (default=1)
`sandfrac`	Sand fraction in top layer [-], when MorFile keyword `[output] SandFrac` is true.	`wriHis_sediment` (default=1)
`fixfac`	Reduction factor due to limited sediment thickness [-], when MorFile keyword `[output] FixFac` is true.	`wriHis_sediment` (default=1)
`hidexp`	Hiding and exposure factor [-], when MorFile keyword `[output] HidExp` is true.	`wriHis_sediment` (default=1)
`mfluff`	Sediment mass in fluff layer [-], when MorFile keyword `[FluffLayer] Type > 0` and number of suspended sediment fractions is positive.	`wriHis_sediment` (default=1)
# The above are sediment transport variables.		
# for model when both MDU keywords `[sediment] SedFile` and `MorFile` are not empty, and `Sedimentmodelnr` is 4.		
`sediment_concentration`	Sediment concentration [kg/m^3], when MDU keywords `[waves] Wavemodelnr > 0` and, either MDU keywords `[sediment] SedFile` or `MorFile` is empty or `Sedimentmodelnr` is not 4.	`wriHis_sediment` (default=1)
# Variables for wind models		
`patm`	Atmospheric pressure [N/m^2], when atmospheric pressure is provided via the EXT file, for example with quantity `airpressure_windx_windy` (see Table C.9).	`wriHis_wind` (default=1)
`windx`, `windy`	x-, y-components of the wind vector [m/s], when wind is provided via the EXT file, for example with quantity `airpressure_windx_windy` (see Table C.9), or when `[external forcing] WindExt > 0.`	`wriHis_wind` (default=1)

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	Description	MDU `[output]` keyword to switch on/off this output
`rain`	Precipitation rate [mm/day], when rainfall is provided via the EXT file, for example with quantity `rainfall_rate` (see Table C.9), or when MDU keyword `[external forcing] Rainfall > 0`.	`wriHis_rain` (default=1)
`infiltration_cap`, `infiltration_actual`	Infiltration capacity and actual infiltration rate [mm/hr], respectively. Output of these variables are available when MDU keyword `[grw] Infiltrationmodel = 2 or 4`.	`wriHis_infiltration` (default=1)

DRAFT

F.3.1.5 Variables on cross sections

Variables on cross sections are typically defined as follows:

```
double cross_section_discharge(time, cross_section) ;
cross_section_discharge:units = "m3 s-1" ;
cross_section_discharge:coordinates = "cross_section_name" ;
```

Output on cross sections is available as time series in the history file. The output is only included when one or more cross sections are defined in the model. The available output quantities are listed in Table F.5. These output variables are quantities located on the cross sections. Output of these variables is always switched on.

DRAFT

Table F.5: *Output quantities on cross sections in history file.*

Quantity name	Description [Unit]
`cross_section_name`	Names of cross sections [-].
`cross_section_geom`	Geometry variables, see Section F.3.1.10
# End of time-independent variables.	
`cross_section_discharge`	Discharge through cross sections [m^3/s].
`cross_section_cumulative_discharge`	Cumulative discharge through cross sections [m^3]
`cross_section_area`	Area of cross sections [m^2]
`cross_section_velocity`	Averaged velocity (by area) through cross sections [m/s].
# The following variables are for transport models,	
`cross_section_cumulative_⟨name of constituent⟩`	Cumulative flux (based on upwind flow cell) for each constituent. If the constituent is sediment, then the unit is [kg] when MorFile keyword `[output] TranspType` is 0, and is [m^3] when this keyword is 1 or 2. For non-sediment constituent, the unit is [constituent unit m^3] where [constituent unit] is specified by users, if it is not specified, then the unit is [-].
`cross_section_⟨name of constituent⟩`	Flux (based on upwind flow cell) for each constituent. If the constituent is sediment, then the unit is [kg/s] when MorFile keyword `[output] TranspType` is 0, and is [m^3/s] when this keyword is 1 or 2. For non-sediment constituent, the unit is [constituent unit m^3/s] where [constituent unit] is specified by users, if it is not specified, then the unit is [-].
# The above variables are for transport models.	
# The following variables are for sediment models,	
# when MDU keyword `[sediment] Sedimentmodelnr = 4` and number of sediment tractions is positive.	
`cross_section_bedload_sediment_transport`	Cumulative bed load sediment transport [kg].
`cross_section_suspended_sediment_transport`	Cumulative suspended load sediment transport [kg], when number of suspended sediment fractions is positive.
`cross_section_bedload_sediment_transport_⟨name of sediment fraction⟩`	cumulative bed load sediment transport per fraction [kg]

DRAFT

F.3.1.6 Mass balance output

The history file may also contain model-global, time-integrated mass balance output, for example:

```
double WaterBalance_total_volume(time) ;
  WaterBalance_total_volume:units = "m3" ;
```

The available output quantities are listed in Table F.6. Output of these variables is switched on or off by setting MDU keyword `[output] wriHis_balance = 1 or 0`, respectively (default is `1`).

DRAFT

Table F.6: *Output quantities for the mass balance in history file.*

Quantity name	Description	[Unit]
water_balance_total_volume	Total volume at the end of timestep.	[m^3]
water_balance_storage	Net storage since Tstart.	[m^3]
water_balance_volume_error	Volume error since Tstart	[m^3]
water_balance_boundaries_in	Cumulative inflow through boundaries since Tstart.	[m^3]
water_balance_boundaries_out	Cumulative outflow through boundaries since Tstart.	[m^3]
water_balance_boundaries_total	Net inflow through boundaries since Tstart.	[m^3]
water_balance_exchange_with_1D_in	Cumulative inflow via SOBEK–D-Flow FM 1D–2D boundaries since Tstart.	[m^3]
water_balance_exchange_with_1D_out	Cumulative outflow via SOBEK–D-Flow FM 1D–2D boundaries since Tstart.	[m^3]
water_balance_exchange_with_1D_total	Net inflow via SOBEK–D-Flow FM 1D–2D boundaries since Tstart.	[m^3]
water_balance_precipitation_total	Total precipitation volume since Tstart.	[m^3]
water_balance_evaporation	Evaporation.	[m^3]
water_balance_source_sink	Net inflow via source–sink elements since Tstart.	[m^3]
InternalTidesDissipation	Total internal tides dissipation, when `internaltidesfrictioncoefficient` in EXT file is specified.	[TJ].
Gravitational_Input	Total Gravitational Input (incl. SAL), when MDU keyword `[physics] TidalForcing = 1`.	[TJ]
SAL_Input	Total input of self attraction and loading, when MDU keyword `[physics] SelfAttractionLoading > 0`.	[TJ]
SAL_Input_2	Total input of self attraction and loading with different formulation, when MDU keyword `[physics] SelfAttractionLoading > 0`.	[TJ]
water_balance_groundwater_in	Cumulative volume received from groundwater.	[m^3]
water_balance_groundwater_out	Cumulative volume out towards groundwater.	[m^3]
water_balance_groundwater_total	Total net inflowing volume from groundwater.	[m^3]
water_balance_laterals_in	Cumulative inflowing volume through laterals since Tstart.	[m^3]
water_balance_laterals_out	Cumulative outflowing volume through laterals since Tstart.	[m^3]
water_balance_laterals_total	Total net inflowing volume through laterals since Tstart.	[m^3]
water_balance_laterals_in_1D	Cumulative inflowing volume through 1D-laterals since Tstart.	[m^3]

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	Description	[Unit]
water_balance_laterals_out_1D	Cumulative outflowing volume through 1D-laterals since Tstart.	[m^3]
water_balance_laterals_total_1D	Total net inflowing volume through 1D-laterals since Tstart.	[m^3]
water_balance_laterals_in_2D	Cumulative inflowing volume through 2D-laterals since Tstart.	[m^3]
water_balance_laterals_out_2D	Cumulative outflowing volume through 2D-laterals since Tstart.	[m^3]
water_balance_laterals_total_2D	Total net inflowing volume through 2D-laterals since Tstart.	[m^3]
water_balance_Qext_in	Cumulative inflowing volume through external discharge since Tstart.	[m^3]
water_balance_Qext_out	Cumulative outflowing volume through external discharge since Tstart.	[m^3]
water_balance_Qext_total	Total net inflowing volume through external discharge since Tstart.	[m^3]
water_balance_Qext_in_1D	Cumulative inflowing volume through 1D external discharge since Tstart.	[m^3]
water_balance_Qext_out_1D	Cumulative outflowing volume through 1D external discharge since Tstart.	[m^3]
water_balance_Qext_total_1D	Total net inflowing volume through 1D external discharge since Tstart.	[m^3]
water_balance_Qext_in_2D	Cumulative inflowing volume through 2D external discharge since Tstart.	[m^3]
water_balance_Qext_out_2D	Cumulative outflowing volume through 2D external discharge since Tstart.	[m^3]
water_balance_Qext_total_2D	Total net inflowing volume through 2D external discharge since Tstart.	[m^3]
water_balance_total_volume_interception	Total volume of the interception layer at the end of timestep.	[m^3]
water_balance_evaporation_interception	Total evaporated volume from interception layer since Tstart.	[m^3]
water_balance_precipitation_on_ground	Total volume of rain fallen onto the ground (i.e., not intercepted) since Tstart.	[m^3]

DRAFT

F.3.1.7 Variables on laterals

The history file may also contain variables on laterals. This is controlled by the MDU setting `[output] wriHis_lateral`. These variables are listed in Table F.7.

Table F.7: *Output quantities for laterals.*

Quantity name	Description	Unit
`lateral_id`	Id of lateral.	
`lateral_prescribed_discharge_instantaneous`	Prescribed discharge through lateral at current time step (instantaneous).	[m^3/s]
`lateral_prescribed_discharge_average`	Prescribed discharge through lateral, average over the last history time interval.	[m^3/s]
`lateral_realized_discharge_instantaneous`	Realized discharge through lateral at current time step (instantaneous).	[m^3/s]
`lateral_realized_discharge_average`	Realized discharge through lateral, average over the last history time interval.	[m^3/s]
`lateral_geom`, `lateral_geom_node_count`, `lateral_geom_node_coordx`, `lateral_geom_node_coordy`	Geometry variables of laterals, see Section F.3.1.10.	

F.3.1.8 Variables on sources/sinks

Variables on sources/sinks are typically defined as follows:

```
double source_sink_prescribed_discharge(time, source_sink) ;
source_sink_prescribed_discharge:units = "m3 s-1" ;
source_sink_prescribed_discharge:coordinates = "source_sink_name" ;
```

Other quantities that can be written are:

- source sink prescribed salinity increment;
- source sink prescribed temperature increment;
- source sink current discharge;
- source sink cumulative volume;
- source sink discharge average;
- geometry variables, see Section F.3.1.10.

These variables are automatically written to his-file if any source/sink exists. One can switch it off by setting in MDU-file that `[output] wriHis_sourcesink = 0`.

F.3.1.9 Hydraulic structure output

A history file can also contain automatic output for hydraulic structures, without the need to add an explicit cross section at the same location. An example variable is:

```
double weirgen_discharge(time, weirgens) ;
  weirgen_discharge:long_name = "Discharge through weir" ;
```

Other quantities that can be written are:

- weir crest level at current time;

- ◇ Total discharge through gate at current time;
- ◇ gate sill level at current time;
- ◇ gate sill width at current time;
- ◇ gate door lower edge level at current time;
- ◇ gate door opening width at current time;
- ◇ Total pump discharge at current time;
- ◇ Max. pump capacity at current time;
- ◇ Geometry variables, see Section F.3.1.10.

Quantities for different hydraulic structures are described in the following subsections.

F.3.1.9.1 Pump

Output for pump structures is available as time series in the history file. The output can be switched on or off using the MDU setting `[output] wriHis_structure_pump` (cf. Table A.1). The available output quantities are listed in Table F.8.

Table F.8: *Output quantities of pumps in history file.*

Quantity name	Description	Unit
`pump_id`	Id of the pump.	
`pump_xmid`, `pump_ymid`	x- and y-coordinates of representative mid point of the pump location (snapped polyline).	[m]
`pump_structure_discharge`*	Discharge through the pump.	[m^3/s]
`pump_capacity`	Prescribed capacity of the pump.	[m^3/s]
`pump_discharge_dir`*	Discharge w.r.t. pump orientation.	[m^3/s]
`pump_s1up`	Water level at upstream of the pump.	[m AD]
`pump_s1dn`	Water level at downstream of the pump.	[m AD]
`pump_s1_delivery_side`	Water level at delivery side of pump.	[m AD]
`pump_s1_suction_side`	Water level at suction side of pump.	[m AD]
`pump_structure_head`	Head difference across the pump structure, this is computed by `pump_s1up` subtracting `pump_s1dn`.	[m]
`pump_head`	Head difference in pumping direction, this is computed by `pump_s1_delivery_side` subtracting `pump_s1_suction_side`.	[m]
`pump_actual_stage`	Actual stage of the pump. It gives valid values only when number of stages is larger than 0.	
`pump_reduction_factor`	Reduction factor of the pump.	

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the pump covers more than one flow link, variable `pump_structure_discharge` is summed across all wet links.
- ◇ `pump_s1up` shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to `pump_s1dn` considering downstream cells. Moreover, `pump_s1_delivery_side`/`pump_s1_suction_side` also follows the similar rule, depending on if its delivery/suction side is wet or not, respectively.
- ◇ `pump_structure_head` and `pump_head` have a valid value only when both sides are wet. Otherwise it shows a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.

DRAFT

◇ The pump_capacity does *not* include any reduction factor yet. The actual discharge is available in pump_discharge_dir and only this variable includes a possible reduction factor, as well as volume-based relaxation (cf. Section 10.11.6).

F.3.1.9.2 General structure

Output for general structures is available as time series in the history file. The output can be switched on or off using the MDU setting [output] wriHis_structure_gen (cf. Table A.1). The available output quantities are listed in Table F.9.

Table F.9: *Output quantities of general structures in history file.*

Quantity name	Description	Unit
general_structure_id	Id of the general structure.	
general_structure_discharge*	Total discharge through the general structure.	[m^3/s]
general_structure_crest_level	Actual crest level that is used in the computation.	[m AD]
general_structure_crest_width	Actual crest width that is used in the computation.	[m]
general_structure_gate_lower_edge_level	Actual gate lower edge level that is used in the computation.	[m AD]
general_structure_gate_opening_width	Actual gate opening width that is used in the computation.	[m]
general_structure_s1up	Water level at upstream of the general structure.	[m AD]
general_structure_s1dn	Water level at downstream of the general structure.	[m AD]
general_structure_discharge_through_gate_opening*	Discharge through gate opening.	[m^3/s]
general_structure_discharge_over_gate*	Discharge over gate.	[m^3/s]
general_structure_discharge_under_gate*	Discharge under gate.	[m^3/s]
general_structure_gate_opening_height	Gate opening height.	[m]
general_structure_gate_upper_edge_level	Gate upper edge level.	[m AD]
general_structure_head	Head difference across the general structure.	[m]
general_structure_velocity_through_gate_opening*	Velocity through gate opening.	[m/s]
general_structure_velocity_over_gate*	Velocity over gate.	[m/s]
general_structure_velocity_under_ gate*	Velocity under gate.	[m/s]
general_structure_flow_area*	Total flow area.	[m^2]
general_structure_flow_area_in_gate_opening*	Flow area in gate opening.	[m^2]
general_structure_flow_area_over_gate*	Flow area over gate.	[m^2]
general_structure_flow_area_under_gate*	Flow area under gate.	[m^2]

(continued on next page)

DRAFT

Quantity name	Description	Unit
(continued from previous page)		
general_structure_state*	Flow state through the general structure. Possible values: ◇ 0: No flow ◇ 1: free weir flow ◇ 2: submerged weir flow ◇ 3: free gate flow ◇ 4: submerged gate flow	
general_structure_s1_on_crest*	Water level on crest.	[m AD]
general_structure_velocity*	Average velocity through the general structure.	[m/s]
general_structure_force_difference*	Force difference per unit width at the general structure.	[N/m]

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the general structure covers more than one flow link, the written variables are either summed across all wet links (e.g. discharges and areas), or a weighted average, with the weights being the widths of the wet flow links (e.g. water level on crest and force difference per unit width). Specially, velocity is the summed discharge divided by the summed area on wet links.
- ◇ general_structure_s1up shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to general_structure_s1dn considering downstream cells.
- ◇ general_structure_head has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, general_structure_head has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.
- ◇ general_structure_state shows a fill value (-999) if the flow link is dry.
  If the flow link is wet, on this flow link, three states of the flow are computed: state of flow under gate, state of flow between gate and state of flow over gate. The maximal value of these integers will be used as the flow state of the link. When the general structure covers more than one flow link, then the state may differ per flow link. If not all of the flow links have the same state value, the state is written to file as a fill value (-999).

F.3.1.9.3 Weir

Output for weir structures is available as time series in the history file. The output can be switched on or off using the MDU setting [output] wriHis_structure_weir (cf. Table A.1). The available output quantities are listed in Table F.10.

Table F.10: *Output quantities of weirs in history file.*

Quantity name	Description	Unit
weirgen_id	Id of the weir.	
weirgen_discharge*	Discharge through the weir.	[m^3/s]
weirgen_crest_level	Actual crest level that is used in the computation.	[m AD]
weirgen_crest_width	Actual crest width that is used in the computation.	[m]
weirgen_s1up	Water level at upstream of the weir.	[m AD]
(continued on next page)		

Quantity name	Description	Unit
(continued from previous page)		
weirgen_s1dn	Water level at downstream of the weir.	[m AD]
weirgen_structure_head	Head difference across the weir.	[m]
weirgen_flow_area*	Flow area at the weir.	[m^2]
weirgen_velocity*	Velocity through the weir.	[m/s]
weirgen_state*	Flow state at the weir. Possible values: ◇ 0: No flow ◇ 1: free weir flow ◇ 2: submerged weir flow	
weirgen_force_difference*	Force difference per unit width at the weir.	[N/m]
weirgen_s1_on_crest*	Water level on crest of the weir.	[m AD]

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the weir covers more than one flow link, the written variables are either summed across all wet links (e.g. discharge and area), or a weighted average, with the weights being the widths of the wet flow links (e.g. water level on crest and force difference per unit width). Specially, velocity is the summed discharge divided by the summed area on wet links.
- ◇ weirgen_s1up shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to weirgen_s1dn considering downstream cells.
- ◇ weirgen_structure_head has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, weirgen_structure_head has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.
- ◇ weirgen_state shows a fill value (-999) if the flow link is dry. When the weir covers more than one wet flow link, then the state may differ per flow link. If not all of the flow links have the same state value, the state is written to file as a fill value (-999).

F.3.1.9.4 Orifice

Output for orifice structures is available as time series in the history file. The output can be switched on or off using the MDU setting [output] wriHis_structure_orifice (cf. Table A.1). The available output quantities are listed in Table F.11.

Table F.11: *Output quantities of orifices in history file.*

Quantity name	Description	Unit
orifice_id	Id of the orifice.	
orifice_discharge*	Discharge through the orifice.	[m^3/s]
orifice_crest_level	Actual crest level that is used in the computation.	[m AD]
orifice_crest_width	Actual crest width that is used in the computation.	[m]
orifice_gate_lower_edge_level	Actual gate lower edge level that is used in the computation.	[m AD]
orifice_s1up	Water level at upstream of the orifice.	[m AD]
(continued on next page)		

Quantity name	Description	Unit
(continued from previous page)		
orifice_s1dn	Water level at downstream of the orifice.	[m AD]
orifice_gate_opening_height	Gate opening height of the orifice.	[m]
orifice_head	Head difference across the orifice.	[m]
orifice_flow_area*	Flow area at the orifice.	[m^2]
orifice_state*	Flow state at the orifice. Possible values: ◇ 0: No flow ◇ 1: free weir flow ◇ 2: submerged weir flow ◇ 3: free gate flow ◇ 4: submerged gate flow	
orifice_velocity*	Velocity through the orifice.	[m/s]
orifice_s1_on_crest*	Water level on crest of the orifice.	[m AD]
orifice_force_difference*	Force difference per unit width at the orifice.	[N/m]

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the orifice covers more than one flow link, the written variables are either summed across all wet links (e.g. discharge and area), or a weighted average, with the weights being the widths of the wet flow links (e.g. water level on crest and force difference per unit width). Specially, velocity is the summed discharge divided by the summed area on wet links.
- ◇ orifice_s1up shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to orifice_s1dn considering downstream cells.
- ◇ orifice_head has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, orifice_head has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.
- ◇ orifice_state shows a fill value (-999) if the flow link is dry. When the orifice covers more than one wet flow link, then the state may differ per flow link. If not all of the flow links have the same state value, the state is written to file as a fill value (-999).

F.3.1.9.5 Bridge

Output for bridge structures is available as time series in the history file. The output can be switched on or off using the MDU setting [output] wriHis_structure_bridge (cf. Table A.1). The available output quantities are listed in Table F.12.

Table F.12: *Output quantities of bridges in history file.*

Quantity name	Description	Unit
bridge_id	Id of the bridge.	
bridge_discharge*	Discharge through the bridge.	[m^3/s]
bridge_s1up	Water level at upstream of the bridge.	[m AD]
bridge_s1dn	Water level at downstream of the bridge.	[m AD]
bridge_head	Head difference across the bridge.	[m]
(continued on next page)		

DRAFT

Quantity name	Description	Unit
(continued from previous page)		
bridge_flow_area*	Flow area at the bridge.	[m^2]
bridge_velocity*	Velocity through the bridge.	[m/s]
bridge_blup	Bed level at upstream of the bridge.	[m AD]
bridge_bldn	Bed level at downstream of the bridge.	[m AD]
bridge_bl_actual	Actual bed level of bridge (crest) that is used in the computation.	[m AD]

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the bridge covers more than one flow link, the written variables are summed across all wet links (e.g. discharge and area). Specially, velocity is the summed discharge divided by the summed area on wet links.
- ◇ bridge_s1up shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to bridge_s1dn considering downstream cells.
- ◇ bridge_head has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, bridge_head has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.
- ◇ When the bridge covers more than one flow link, variables of bed level (with bl) are weighted averages, with the weights being the widths of the flow links (both dry and wet).
- ◇ The actual bed level of the bridge is a crest-like bed level, i.e., at the velocity point.

F.3.1.9.6 Culvert

Output for culvert structures is available as time series in the history file. The output can be switched on or off using the MDU setting [output] wriHis_structure_culvert (cf. Table A.1). The available output quantities are listed in Table F.13.

Table F.13: *Output quantities of culverts in history file.*

Quantity name	Description	Unit
culvert_id	Id of the culvert.	
culvert_discharge*	Discharge through the culvert.	[m^3/s]
culvert_crest_level	Crest level.	[m AD]
culvert_gate_lower_edge_level	Gate lower edge level.	[m AD]
culvert_s1up	Water level at upstream of the culvert.	[m AD]
culvert_s1dn	Water level at downstream of the culvert.	[m AD]
culvert_gate_opening_height	Gate opening height of the culvert.	[m]
culvert_head	Head difference across the culvert.	[m]
culvert_flow_area*	Flow area at the culvert.	[m^2]
(continued on next page)		

DRAFT

Quantity name	Description	Unit
(continued from previous page)		
culvert_state*	Flow state at the culvert. Possible values: ◇ 0: No flow ◇ 1: free culvert flow ◇ 2: submerged culvert flow	
culvert_velocity*	Velocity through the culvert.	[m/s]

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the culvert covers more than one flow link, the written variables are summed across all wet links (e.g. discharge and area). Specially, velocity is the summed discharge divided by the summed area on wet links.
- ◇ culvert_s1up shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to culvert_s1dn considering downstream cells.
- ◇ culvert_head has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, culvert_head has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.
- ◇ culvert_state shows a fill value (-999) if the flow link is dry. When the culvert covers more than one wet flow link, then the state may differ per flow link. If not all of the flow links have the same state value, the state is written to file as a fill value (-999).

F.3.1.9.7 Long culvert

Output for long culvert structures is available as time series in the history file. The output can be switched on or off using the MDU setting [output] wriHis_structure_longculvert (cf. Table A.1). The available output quantities are listed in Table F.14.

Table F.14: *Output quantities of long culverts in history file.*

Quantity name	Description	Unit
longculvert_id	Id of the long culvert.	
longculvert_discharge*	Discharge through the long culvert.	[m^3/s]
longculvert_s1up	Water level at upstream of the long culvert.	[m AD]
longculvert_s1dn	Water level at downstream of the long culvert.	[m AD]
longculvert_head	Head difference across the long culvert.	[m]
longculvert_flow_area*	Flow area at the long culvert.	[m^2]
longculvert_velocity*	Velocity through the long culvert.	[m/s]
longculvert_valve_relative_opening*	Valve relative opening in long culvert.	[1]

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the long culvert covers more than one flow link, the written variables are summed across all wet links (e.g. discharge and area). Specially, velocity is the summed discharge divided by the summed area on wet links.
- ◇ longculvert_s1up shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being

the widths of the flow links whose corresponding upstream cells are wet. This also applies to `longculvert_s1dn` considering downstream cells.
- ◇ `longculvert_head` has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, `longculvert_head` has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.
- ◇ In the case of a long culvert that consists of more than one wet flow link (refer to Figure 10.11):
  - ◇ the discharge is used from the first flow link, i.e., the inlet side.
  - ◇ the upstream and downstream water levels are taken from the 2D end points of the long culvert structure. No output on any intermediate pressure points is written in the his file. Those can be found in the map file.

F.3.1.9.8 Dam break

Output for dam break structures is available as time series in the history file. The output can be switched on or off using the MDU setting `[output] wriHis_structure_damBreak` (cf. Table A.1). The available output quantities are listed in Table F.15.

Table F.15: *Output quantities of dam breaks in history file.*

Quantity name	Description	Unit
`dambreak_id`	Id of the dam break.	
`dambreak_s1up`	Water level at upstream of the dam break.	[m AD]
`dambreak_s1dn`	Water level at downstream of the dam break.	[m AD]
`dambreak_discharge`*	Discharge through the dam break.	[m^3/s]
`dambreak_cumulative_discharge`*	Cumulative discharge through the dam break.	[m^3/s]
`dambreak_breach_width_time_derivative`	Breach width time derivative of the dam break.	[m/s]
`dambreak_water_level_jump`	Breach water level jump across the dam break. See remark below.	[m]
`dambreak_normal_velocity`	Normal velocity through the dam break.	[m/s]
`dambreak_structure_head`	Head difference across the dam break.	[m]
`dambreak_flow_area`*	Flow area at the dam break.	[m^2]
`dambreak_crest_level`	Actual crest level that is used in the computation. When there is no breach yet, the crest level is the bed level ("bob") at the breach starting location.	[m AD]
`dambreak_crest_width`	Actual crest width that is used in the computation. When there is no breach yet, equal to 0.	[m]

Remarks:
- ◇ Contrary to all other hydraulic structures, the dam break output only includes flow through the breach hole. In a 2D model, when multiple flow links are crossed by the dambreak polyline, there may additionally be flow across the fixed weir crest levels of flow links even though the breach growth has not reached them yet. When the total discharge across the entire polyline is desired, it is recommended to include an additional observation cross section (Section F.2.4) in the model.
- ◇ In a similar way as in the previous point, water levels and other quantities are only averaged or summed over the flow links in the breach hole.

DRAFT

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the dambreak covers more than one flow link, the written variables are summed across all wet links (e.g. discharge and area).
- ◇ `dambreak_s1up` shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to `dambreak_s1dn` considering downstream cells.
- ◇ `dambreak_structure_head` has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, `dambreak_structure_head` has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.
- ◇ The quantities `dambreak_structure_head` and `dambreak_water_level_jump` are different. The head is the basic difference between up- and downstream water levels. The water level jump is equal to $\max(0, \zeta_{up} - z_s) - \max(0, \zeta_{dn} - z_s)$ and represents the average water level jump at the breach crest, often used in breach growth formulas.

F.3.1.9.9 Universal weir

Output for universal weir structures is available as time series in the history file. The output can be switched on or off using the MDU setting `[output] wriHis_structure_uniWeir` (cf. Table A.1). The available output quantities are listed in Table F.16.

Table F.16: *Output quantities of universal weirs in history file.*

Quantity name	Description	Unit
`uniweir_id`	Id of the universal weir.	
`uniweir_discharge`*	Discharge through the universal weir.	[m^3/s]
`uniweir_s1up`	Water level at upstream of the universal weir.	[m AD]
`uniweir_s1dn`	Water level at downstream of the universal weir.	[m AD]
`uniweir_head`	Head difference across the universal weir.	[m]
`uniweir_flow_area`*	Flow area at the universal weir.	[m^2]
`uniweir_velocity`*	Velocity through the universal weir.	[m/s]
`uniweir_crest_level`	Crest level of the universal weir.	[m AD]

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the universal weir covers more than one flow link, the written variables are summed across all wet links (e.g. discharge and area). Specially, velocity is the summed discharge divided by the summed area on wet links.
- ◇ `uniweir_s1up` shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to `uniweir_s1dn` considering downstream cells.
- ◇ `uniweir_head` has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, `uniweir_head` has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.

DRAFT

F.3.1.9.10 Compound structure

Output for compound structures is available as time series in the history file. The output can be switched on or off using the MDU setting `[output] wriHis_structure_compound` (cf. Table A.1). The available output quantities are listed in Table F.17.

Table F.17: *Output quantities of compound structures in history file.*

Quantity name	Description	Unit
`cmpstru_id`	Id of the compound structure.	
`cmpstru_discharge`*	Total discharge through the compound structure.	[m^3/s]
`cmpstru_s1up`	Water level at upstream of the compound structure.	[m AD]
`cmpstru_s1dn`	Water level at downstream of the compound structure.	[m AD]
`cmpstru_head`	Head difference across the compound structure.	[m]
`cmpstru_flow_area`*	Total flow area at the compound structure.	[m^2]
`cmpstru_velocity`*	Average velocity through the compound structure.	[m/s]

Remarks:

- ◇ Variables with notation $*$ show a fill value (-999) if the flow link is dry. When the compound structure covers more than one flow link, the written variables are summed across all wet links (e.g. discharge and area). Specially, velocity is the summed discharge divided by the summed area on wet links.
- ◇ `cmpstru_s1up` shows a fill value (-999) when the upstream cell is dry. If there is more than one upstream cell, then the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream cells are wet. This also applies to `cmpstru_s1dn` considering downstream cells.
- ◇ `cmpstru_head` has a valid value only when both upstream and downstream cells are wet. Otherwise it shows a fill value (-999). This means that, when the upstream cell is dry while downstream cell is wet, `cmpstru_head` has a fill value (-999). In the situation when there is more than one upstream/downstream cell, the value is a weighted average, with the weights being the widths of the flow links whose corresponding upstream and downstream cells are both wet.

F.3.1.9.11 Summary

Table F.18 summarizes the above output quantities for different hydraulic structures, where the existence of output is denoted by $\times$.

Table F.18: *Summary of output in history files of quantities for different structures.*

Quantity name	Pump	General structure	Weir	Orifice	Bridge	Culvert	Dam break	Universal Weir	Compound structure	Long culvert
Id of structure	×	×	×	×	×	×	×	×	×	×
Total discharge through structure	×	×	×	×	×	×	×	×	×	×
Discharge through gate opening		×								
Discharge over gate upper edge level		×								
Water level at upstream of the structure	×	×	×	×	×	×	×	×	×	×

(continued on next page)

Quantity name	Pump	General structure	Weir	Orifice	Bridge	Culvert	Dam break	Universal Weir	Compound structure	Long culvert
(continued from previous page)										
Water level at downstream of the structure	×	×	×	×	×	×	×	×	×	×
Crest level of structure		×	×	×		×	×	×		
Gate opening height of structure		×		×		×				
Gate lower edge level of structure		×		×		×				
Gate upper edge level of structure		×								
Crest width of structure		×	×	×			×			
Force difference per unit width		×	×	×						
Water level on crest of structure		×	×	×						
Head difference across structure	×	×	×	×	×	×	×	×	×	×
Velocity through structure		×	×	×	×	×		×	×	×
Velocity through gate opening		×								
Velocity over gate upper edge level		×								
Flow area at structure		×	×	×	×	×	×	×	×	×
Flow area in gate opening		×								
Flow area above upper edge level		×								
Flow state at structure		×	×	×		×				
Actual stage of pump	×									
Capacity of pump	×									
Discharge w.r.t. pump orientation	×									
Head difference in pumping direction	×									
Reduction factor of pump	×									
Water level at delivery side of pump	×									
Water level at suction side of pump	×									
Coordinates of representative of pump	×									
Gate opening width		×								
Cumulative discharge through dambreak							×			
Breach width time derivative of dambreak							×			
Breach water level jump of dambreak							×			
Normal velocity through dambreak							×			
Bed level at upstream of bridge					×					
Bed level at downstream of bridge					×					
Actual bed level of bridge					×					
Valve relative opening										×
Geometry variables	×	×	×	×	×	×		×		×

F.3.1.10 Geometry variables

Geometry variables[2], that are also written to the his-files, give the geometry information for those locations for which time series are reported in the his-file. NOTE: the location information of geometry variables does not give the original input location of the geometries, but rather the locations that are snapped to the computational mesh.

[2]For more details about geometry variables please see: https://github.com/cf-convention/cf-conventions/blob/main/ch07.adoc#geometries

DRAFT

For a set of geometries, there are four geometry variables. Take a set of general structures as an example. The geometry variables names contain "_geom" for readability, but this is not required, and they should be identified via CF-compliant attributes. For the set of general structures these are:

- "general_structure_geom",
- "general_structure_geom_node_count",
- "general_structure_geom_node_coordx",
- "general_structure_geom_node_coordy".

The description of the above geometry variables are as follows:

- "general_structure_geom" is a geometry container variable. Such a variable can typically be identified when it is referenced in the `:geometry` attribute of another data variable, and the container variable itself has the `:geometry_type` attribute. This variable contains all the attributes that describe this set of geometries, including the following attributes:
  - "geometry_type": tells the type of the geometry. It can be "point", "line" or "polygon".
  - "node_count": tells the name of a variable that indicates the count of nodes per geometry. The name is "general_structure_geom_node_count" in our example, and it will contain the number of geometry points for each of the general structures in the model input.
  - "node_coordinates": contains the names of variables that contain the node coordinates of the geometry. In our example, it has two names "general_structure_geom_node_coordx" and "general_structure_geom_node_coordy".
- "_node_count": indicates the total number of nodes on this set of geometries. For instance, if variable "general_structure_geom_node_count" contains values "2, 3", it means that there are two general structures, and the first and the second general structures have 2 and 3 nodes, respectively.
- "_node_coordx": gives x-coordinates of all the nodes. Take the above example, then this array will contain 5 points in total; the first 2 values of this variable are the x-coordinates of the 2 nodes of the first general structure, and next 3 values of this variable are the x-coordinates of the 3 nodes of the second general structure.
- "_node_coordy": gives y-coordinates of all the nodes.

The following geometries in his files have geometry variables:

- Observation points.
- Observation cross sections.
- Hydraulic structures, including:
  - Weirs.
  - Orifices.
  - General structures.
  - Universal weirs.
  - Culverts.
  - Gates.
  - Pumps.
  - Bridges.
  - Long culverts.
- Source and sinks.
- Laterals.

DRAFT

F.3.2 Spatial fields as netCDF map-file

The map file <*mdu_name*_map.nc> contains data on the entire model grid in 1D, 2D and 3D. Three formats are currently supported, selected by the `MapFormat` keyword in the mdu-file:

```
[output]
MapFormat = 4  # Map file format, 1: netCDF, 4: netCDF-UGRID
```

D-Flow FM is using the more standardized UGRID[3] format. This is the default option. D-Flow FM GUI and Delft3D-QUICKPLOT also support this format. In the examples below the older netCDF format is still shown, but most of the concepts discussed hereafter apply equally well to UGRID output (4), only the variable names will be slightly different.

For parallel runs, each process writes a separate file for each partition as <*mdu_name*_000X_map.nc>.

The map file contains the dimensions and variable definitions in its header, followed by the actual data.

Dimensions

The following dimensions are defined in a map file header:

```
netcdf weirfree_map {
dimensions:
       nNetNode = 310 ;
       nNetLink = 486 ;
       nNetLinkPts = 2 ;
       nBndLink = 252 ;
       nNetElem = 180 ;
       nNetElemMaxNode = 4 ;
       nNetLinkContourPts = 4 ;
       nFlowElem = 180 ;        // Nr.
of grid cells
       nFlowElemMaxNode = 4 ;
       nFlowElemContourPts = 4 ;
       nFlowLink = 246 ;        // Nr.
of flow links (open cell edges)
       nFlowLinkPts = 2 ;
       time = UNLIMITED ; // (64 currently)
```

For plotting solution data, the grid cells (flow nodes) are important, and the flow links (open cell edges). All `Net*` dimensions relate the flow grid tot the original unstructured network. More explanation of these topological naming conventions can be found in section 17.1.

Location variables

The location and shape of grid cells is stored in several x,y variables:

```
 double FlowElem_xcc(nFlowElem) ;
   FlowElem_xcc:units = "m" ;
   FlowElem_xcc:standard_name = "projection_x_coordinate" ;
   FlowElem_xcc:long_name = "Flow element circumcenter x" ;
   FlowElem_xcc:bounds = "FlowElemContour_x" ;
 double FlowElem_ycc(nFlowElem) ;
   FlowElem_ycc:units = "m" ;
   FlowElem_ycc:standard_name = "projection_y_coordinate" ;
   FlowElem_ycc:long_name = "Flow element circumcenter y" ;
   FlowElem_ycc:bounds = "FlowElemContour_y" ;
```

[3] https://github.com/ugrid-conventions/ugrid-conventions

```
// [..]
 double FlowElemContour_x(nFlowElem, nFlowElemContourPts) ;
   FlowElemContour_x:units = "m" ;
   FlowElemContour_x:standard_name = "projection_x_coordinate" ;
   FlowElemContour_x:long_name = "List of x-points forming flow element" ;
   FlowElemContour_x:_FillValue = -999.
;
// [..]
 double FlowElem_bl(nFlowElem) ;
   FlowElem_bl:units = "m" ;
   FlowElem_bl:positive = "up" ;
   FlowElem_bl:standard_name = "sea_floor_depth" ;
   FlowElem_bl:long_name = "Bottom level at flow element\'s circumcenter." ;
```

In 3D output the map file also contains the vertical dimensions `laydim` for the layer centers and `wdim` for the layer interfaces. Corresponding to these dimensions, vertical coordinates `LayCoord_cc` and `LayCoord_w`, respectively are added:

```
double LayCoord_cc(laydim) ;
   LayCoord_cc:standard_name = "" ;
   LayCoord_cc:long_name = "z layer coordinate at flow element center" ;
   LayCoord_cc:positive = "up" ;
   LayCoord_cc:units = "m" ;
double LayCoord_w(wdim) ;
   LayCoord_w:standard_name = "" ;
   LayCoord_w:long_name = "z layer coordinate at vertical interface" ;
   LayCoord_w:positive = "up" ;
   LayCoord_w:units = "m" ;
```

These variable contain time- and space-independent layer positions (either in absolute z, or relative sigma coordinates).

In the case the FullGridOutput option is active (`FullGridOutput = 1`), these vertical coordinates `LayCoord_cc` and `LayCoord_w` are excluded from the map-file and replaced by variables `FlowElem_zcc` and `FlowElem_zw`. These variables contain the vertical positions of layer centers and interfaces, respectively, written for each flow cell and each timestep separately. Currently, this is only the case for the flow cell (centre) positions and not for the horizontal flow link positions:

```
double FlowElem_zcc(time, nFlowElem, laydim) ;
   FlowElem_zcc:coordinates = "FlowElem_xcc FlowElem_ycc FlowElem_zcc" ;
   FlowElem_zcc:standard_name = "" ;
   FlowElem_zcc:long_name = "flow element center z" ;
   FlowElem_zcc:units = "m" ;
double FlowElem_zw(time, nFlowElem, wdim) ;
   FlowElem_zw:coordinates = "FlowElem_xcc FlowElem_ycc FlowElem_zw" ;
   FlowElem_zw:standard_name = "" ;
   FlowElem_zw:long_name = "flow element z at vertical interface" ;
   FlowElem_zw:units = "m" ;
```

Variables on grid cells/pressure points

Variables on grid cells (represented by the pressure point/cell circumcenter) are typically defined as follows:

```
 double s1(time, nFlowElem) ;
   s1:coordinates = "FlowElem_xcc FlowElem_ycc" ;
   s1:standard_name = "sea_surface_level" ;
   s1:long_name = "waterlevel" ;
   s1:units = "m" ;
   s1:grid_mapping = "projected_coordinate_system" ;
```

DRAFT

All quantities that are available in the map file are listed in Table F.19 and Table F.20. These quantities are output when the model contains the related object. For example, quantities about waves are output in map file only when the model contains wave computation.

DRAFT

Table F.19: *Output variables in map file of UGRID format. Column 'Location' shows the grid location: CN means cell corners. L means net links. S means pressure points, U means velocity points. S3D and U3D are the corresponding locations in 3D networks. Two more locations in 3D networks are: W, vertical velocity point on all layer interfaces, and WU, vertical viscosity point on all layer interfaces.*

Quantity name	MDU option	Unit	Location
Water level at end of time step.	`wriMap_waterlevel_s1`	[m AD]	S
Water depth at pressure points.	`wriMap_waterdepth`	[m]	S
Water level at start of time step.	`wriMap_waterlevel_s0`	[m AD]	S
Numlimdt: number of times each cell was limiting the time step.	`wriMap_numlimdt`	[–]	S
Temperature.	`wriMap_temperature`	[$^{\circ}$C]	S or S3D
Tracers and other constituents.	`wriMap_constituents`	[-]	S or S3D
# The followings are hydrological quantities (Chapter 11):			
Rainfall intensity.	`wriMap_rain`	[m/s]	S
Interception layer water depth.	`wriMap_interception`	[m]	S
Potential evaporation.	`wriMap_evaporation`	[m/s]	S
Actual evaporation.	`wriMap_evaporation`	[m/s]	S
Prescribed evaporation rate.	`wriMap_evaporation`	[m/s]	S
# The above are hydrological quantities.			
# The following variables are only for 1D network:			
Freeboard.	`wriMap_freeboard`	[m]	S
Waterdepth on ground.	`wriMap_waterdepth_on_ground`	[m]	S
Volume on ground.	`wriMap_volume_on_groundn`	[m^3]	S
Cumulative time at each node when water is above ground level.	`wriMap_time_water_on_ground`	[s]	S
Current total 1D2D net inflow.	`wriMap_total_net_inflow_1d2d`	[m^3/s]	S
Cumulative total 1D2D net inflow.	`wriMap_total_net_inflow_1d2d`	[m^3]	S
Current total lateral net inflow.	`wriMap_total_net_inflow_lateral`	[m^3/s]	S
Cumulative total lateral net inflow.	`wriMap_total_net_inflow_lateral`	[m^3]	S
Water level gradient.	`wriMap_water_level_gradient`	[1]	U
# The above variables are only for 1D network.			

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
# The following variables are only for 3D models:			
Vertical coordinate of layer centres at pressure points.	`FullGridOutput`	[m]	S3D
Vertical coordinate of layer interfaces at pressure points.	`FullGridOutput`	[m]	W
Bounds of vertical coordinate of layers at pressure points.	`FullGridOutput`	[m]	S3D
Vertical coordinate of layer centres at velocity points.	`FullGridOutput`	[m]	U3D
Bounds of vertical coordinate of layers at velocity points.	`FullGridOutput`	[m]	U3D
Turbulent kinetic energy, if `[numerics] Turbulencemodel >= 3`.	`wriMap_turbulence`	[m^2/s^2]	WU
Turbulent vertical eddy viscosity, if `[numerics] Turbulencemodel >= 3`.	`wriMap_turbulence`	[m^2/s]	WU
Turbulent energy dissipation, if `[numerics] Turbulencemodel = 3`.	`wriMap_turbulence`	[m^2/s^3]	WU
Turbulent time scale, if `[numerics] Turbulencemodel = 4`.	`wriMap_turbulence`	[1/s]	WU
Flow element center velocity vector, z-component.	`wriMap_velocity_vector`	[m/s]	S3D
Flow element center GLM depth-averaged velocity if `[output] EulerVelocities = 1` and `[waves] Wavemodelnr > 0`. Otherwise, flow element center depth-averaged velocity. This includes x- and y-components.	`wriMap_velocity_vector`	[m/s]	S
Flow element center depth-averaged GLM velocity magnitude if `[output] EulerVelocities = 1` and `[waves] Wavemodelnr > 0`. Otherwise, flow element center depth-averaged velocity magnitude.	`wriMap_velocity_magnitude`	[m/s]	S
Upward velocity on vertical interface, n-component.	`wriMap_upward_velocity_component`	[m/s]	W
Flow element center mass density.	`wriMap_density_rho`	[kg/m^3]	S3D
# The above variables are only for 3D models.			
Sum of all water influx.	`wriMap_Qin`	[m^3/s]	S
Volume of water in grid cell.	`wriMap_volume1`	[m^3]	S or S3D
Time step per cell based on CFL.	`wriMap_DTcell`	[s]	S or S3D
Water depth at velocity points.	`wriMap_waterdepth_hu`	[m]	U
Wet flow area between two neighbouring grid cells.	`wriMap_flowarea_au`	[m^2]	U or U3D
Velocity at velocity points, n-component.	`wriMap_velocity_component_u1`	[m/s]	U or U3D

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Velocity at velocity points at previous time step, n-component.	`wriMap_velocity_component_u0`	[m/s]	U or U3D
Flow element center eulerian velocity vector if `[output] EulerVelocities = 1` and `[waves] Wavemodelnr > 0` and `[waves] flowWithoutWaves` is false. Otherwise, flow element center velocity vector. This includes x- and y-components.	`wriMap_velocity_vector`	[m/s]	S or S3D
Flow element center eulerian velocity magnitude if `[output] EulerVelocities = 1` and `[waves] Wavemodelnr > 0` and `[waves] flowWithoutWaves` is false. Otherwise, flow element center velocity magnitude.	`wriMap_velocity_magnitude`	[m/s]	S or S3D
Flow element center eulerian velocity vector based on discharge if `[output] EulerVelocities = 1` and `[waves] Wavemodelnr > 0` and `[waves] flowWithoutWaves` is false. Otherwise, flow element center velocity vector based on discharge. This includes x- and y-components.	`wriMap_velocity_vectorq`	[m/s]	S or S3D
Discharge through flow link at current time.	`wriMap_flow_flux_q1`	[m^3/s]	U or U3D
Main channel discharge through flow link at current time.	`wriMap_flow_flux_q1_main`	[m^3/s]	U or U3D
Horizontal eddy viscosity.	`wriMap_horizontal_viscosity_viu`	[m^2/s]	U or U3D
Horizontal eddy diffusivity.	`wriMap_horizontal_diffusivity_diu`	[m^2/s]	U or U3D
Total bed shear stress vector, including x- and y-components.	`wriMap_taucurrent`	[N/m^2]	S
Total bed shear stress magnitude.	`wriMap_taucurrent`	[N/m^2]	S
Bed shear stress magnitude for morphology if both MDU keywords `[sediment] SedFile` and `MorFile` are not empty, and `Sedimentmodelnr` is 4.	`wriMap_taucurrent`	[N/m^2]	S
# The following variables are about roughness:			
Chézy roughness:			
Effective Chézy roughness in flow element center.	`wriMap_chezy`	[$m^{0.5}$/s]	S
Effective Chézy roughness on flow links.	`wriMap_chezy_on_flow_links`	[$m^{0.5}$/s]	U
Input roughness on flow links.	`wriMap_input_roughness`	Depends on roughness type.	U
Input roughness type.	`wriMap_input_roughness`	[-], see `UnifFrictType` in Table A.1.	U

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Energy loss for fixed weirs in case of Tabellenboek or Villemonte if `[numerics] FixedWeirScheme = 8 or 9`	`wriMap_fixed_weir_energy_loss`	[m]	U or U3D
Salinity in flow element if `[physics] Salinity > 0`.	`wriMap_salinity`	[ppt]	S or S3D
# The following variables are about streamline curvature and spiral flow intensity (section 3.7):			
(For non-3D models only) Flow streamline curvature if `[physics] SecondaryFlow` is positive.	`wriMap_spiral_flow`	[1/m]	S
Spiral flow intensity if `[physics] SecondaryFlow > 0`.	`wriMap_spiral_flow`	[m/s]	S
# The above variables are about streamline curvature and spiral flow intensity.			
Names of all water quality bottom variables in flow element.		[depends]	S
Names of all water quality bottom variables in flow element with suffix `_3D`.	`wriMap_wqBot3d`	[depends]	S3D
Extra water quality output.		[depends]	S or S3D
All water quality statistic variables.		[depends]	S or S3D
Water quality mass balance areas.		[-]	S
Atmospheric pressure near surface.	`wriMap_wind`	[N/m^2]	S
Wind velocity on flow element center, including x- and y-components.	`wriMap_wind`	[m/s]	S
Wind velocity on flow links, including x- and y-components.	`wriMap_wind`	[m/s]	U
Wind stress on flow element center, including x- and y-components.	`wriMap_windstress`	[N/m^2]	S
# The following variables are about heat flux quantities, air temperature, relative humidity, cloudiness and more detailed fluxes (**??**):			
Air temperature near surface.	`wriMap_heat_fluxes`	[$^\circ$C]	S
Relative humidity near surface.	`wriMap_heat_fluxes`	[-]	S
Cloudiness.	`wriMap_heat_fluxes`	[1]	S
Solar influx.	`wriMap_heat_fluxes`	[W/m^2]	S
Evaporative heat flux.	`wriMap_heat_fluxes`	[W/m^2]	S
Sensible heat flux.	`wriMap_heat_fluxes`	[W/m^2]	S
Long wave back radiation.	`wriMap_heat_fluxes`	[W/m^2]	S
Free convection evaporative heat flux.	`wriMap_heat_fluxes`	[W/m^2]	S
Free convection sensible heat flux.	`wriMap_heat_fluxes`	[W/m^2]	S

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Total heat flux.	`wriMap_heat_fluxes`	[W/m^2]	S
# The above variables are about heat flux quantities, air temperature, relative humidity, cloudiness and more detailed fluxes.			
Time-varying bottom level in flow cell center, if `[sediment] Sedimentmodelnr > 0` and `[sediment] SedFile` and `MorFile` are not empty, and `Sedimentmodelnr` is 4, or if the EXT file contains `bedrock_surface_elevation`.	`wriMap_sediment`	[m]	S
Cumulative subsidence/uplift, if the EXT file contains `bedrock_surface_elevation`.		[m]	S, U or CN
Current related roughness height.	`wriMap_z0`	[m]	U
Current-wave related roughness height.	`wriMap_z0`	[m]	U
# The following variables are for models with sediment (Deltares (2026d)),			
# when `[sediment] Sedimentmodelnr > 0` and `[sediment] SedFile` and `MorFile` are not empty, `Sedimentmodelnr` is 4.			
Time interval over which cumulative transports are calculated, if MorFile keyword `[output] AverageAtEachOutputTime` is false.	`wriMap_sediment`	[s]	-
Average morphological factor over elapsed morphological time.	`wriMap_sediment`	[-]	-
Current morphological time.	`wriMap_sediment`	[s]	-
Sediment fraction name.	`wriMap_sediment`	[-]	-
Suspended sediment fraction name.	`wriMap_sediment`	[-]	-
(For 3D models only) Bottom layer for sed calculations.	`wriMap_sediment`	[-]	S
Sediment settling velocity.	`wriMap_sediment`	[m/s]	W for 3D, otherwise S
(For non-3D models only) Equilibrium sediment concentration.	`wriMap_sediment`	[kg/m^3]	S
Near-bed reference concentration height, if MorFile keyword `[Output] ReferenceHeight` is true.	`wriMap_sediment`	[m]	S
Near-bed reference concentration, if MorFile keyword `[Output] ReferenceHeight` is true.	`wriMap_sediment`	[kg/m^3]	S
Source term suspended sediment fractions, if MorFile keyword `[Output] SourceSinkTerms` is true.	`wriMap_sediment`	[kg/m^3/s]	S
Sink term suspended sediment fractions, if MorFile keyword `[Output] SourceSinkTerms` is true.	`wriMap_sediment`	[1/s]	S

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
(For 3D models only) Near-bed transport correction in face-normal direction, if MorFile keyword `[Output] NearBedTranspCorrAtFlux` is true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	U
Sediment concentration.	`wriMap_sediment`	[kg/m^3]	S or S3D
Suspended load transport, including n- and t-components.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	U
Bed load transport, including n- and t-components.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	U
Bed slope, including n- and t-components, if MorFile keyword `[Output] Bedslope` is true.	`wriMap_sediment`	[-]	U
Characteristic velocity in cell centre, including n- and t-components, if MorFile keyword `[Output] VelocAtZeta` is true.	`wriMap_sediment`	[m/s]	S
Characteristic velocity magnitude, if MorFile keyword `[Output] VelocMagAtZeta` is true.	`wriMap_sediment`	[m/s]	S
Height above bed for characteristic velocity, if MorFile keyword `[Output] VelocZAtZeta` is true.	`wriMap_sediment`	[m]	S
Bed shear velocity, if MorFile keyword `[Output] ShearVeloc` is true.	`wriMap_sediment`	[m/s]	S
Bed load transport due to currents, including x- and y-components, if MorFile keyword `[Output] BedTranspDueToCurrentsAtZeta` and `[Output] RawTransportsAtZeta` are true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Bed load transport due to currents (reconstructed), including x- and y-components, if MorFile keyword `[Output] BedTranspDueToCurrentsAtZeta` is true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Bed load transport due to waves, including x- and y-components, if MorFile keyword `[Output] BedTranspDueToWavesAtZeta` and `[Output] RawTransportsAtZeta` are true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Bed load transport due to waves (reconstructed), including x- and y-components, if MorFile keyword `[Output] BedTranspDueToWavesAtZeta` is true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Suspended load transport due to currents, including x- and y-components, if MorFile keyword `[Output] SuspTranspDueToCurrentsAtZeta` and `[Output] RawTransportsAtZeta` are true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Suspended load transport due to currents (reconstructed), including x- and y-components, if MorFile keyword `[Output] SuspTranspDueToCurrentsAtZeta` is true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Suspended load transport due to waves, including x- and y-components, if MorFile keyword `[Output] SuspTranspDueToWavesAtZeta` and `[Output] RawTransportsAtZeta` are true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Suspended load transport due to waves (reconstructed), including x- and y-components, if MorFile keyword `[Output] SuspTranspDueToWavesAtZeta` is true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Total sediment transport (reconstructed), including x- and y-components.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Time-averaged bed load, including x- and y-components, if MorFile keyword `[Output] AverageAtEachOutputTime` is true.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Time-averaged suspended load transport, including x- and y-components.	`wriMap_sediment`	[kg/s/m] if MorFile keyword `[Output] TranspType` is 0, or [m^3/s/m] if MorFile keyword `[Output] TranspType` is 1 or 2.	S
Available sediment mass in the bed, if MorFile keyword `[Underlayer] IUnderLyr = 1`.	`wriMap_sediment`	[kg/m^2]	S
Sediment thickness in the bed, if MorFile keyword `[Underlayer] IUnderLyr = 1`.	`wriMap_sediment`	[m]	S
Available sediment mass in a layer of the bed, if MorFile keyword `[Underlayer] IUnderLyr = 2`.	`wriMap_sediment`	[kg/m^2]	S
Thickness of a layer of the bed, if MorFile keyword `[Underlayer] IUnderLyr = 2`.	`wriMap_sediment`	[m]	S
Porosity of a layer of the bed, if MorFile keyword `[Underlayer] IUnderLyr = 2` and `[Underlayer] IPorosity > 0`.	`wriMap_sediment`	[-]	S
Volume fraction in a layer of the bed, if MorFile keyword `[Underlayer] IUnderLyr = 2`.	`wriMap_sediment`	[-]	S
Bed shear stress for morphology, if MorFile keyword `[Output] Taub` is true.	`wriMap_sediment`	[N/m^2]	S
Excess bed shear ratio, if MorFile keyword `[Output] taurat` is true.	`wriMap_sediment`	[-]	S
Arithmetic mean sediment diameter, if MorFile keyword `[Output] Dm` is true.	`wriMap_sediment`	[m]	S
Geometric mean sediment diameter, if MorFile keyword `[Output] Dg` is true.	`wriMap_sediment`	[m]	S
Geometric standard deviation of particle size mix, if MorFile keyword `[Output] Dgsd` is true.	`wriMap_sediment`	[m]	S
All sediment diameter percentile, if MorFile keyword `[Output] Percentiles` is true.	`wriMap_sediment`	[m]	S
Availability fraction in top layer, if MorFile keyword `[Output] Frac` is true.	`wriMap_sediment`	[-]	S
Mud fraction in top layer, if MorFile keyword `[Output] MudFrac` is true.	`wriMap_sediment`	[-]	S
Sand fraction in top layer, if MorFile keyword `[Output] SandFrac` is true.	`wriMap_sediment`	[-]	S

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Reduction factor due to limited sediment thickness, if MorFile keyword `[Output] FixFac` is true.	`wriMap_sediment`	[-]	S
Hiding and exposure factor, if MorFile keyword `[Output] HidExp` is true.	`wriMap_sediment`	[-]	S
Sediment mass in fluff layer, if MorFile keyword `[FluffLayer] Type > 0`.	`wriMap_sediment`	[kg/m^2]	S
(For 1D only) Time-varying cross-section points level, if MorFile keyword `[Morphology] BedUpd` is true.	`wriMap_sediment`	[m]	S
(For 1D only) Time-varying cross-section points width, if MorFile keyword `[Morphology] BedUpd` is true.	`wriMap_sediment`	[m]	S
(For 1D only) Name of cross-section, if MorFile keyword `[Morphology] BedUpd` is true.	`wriMap_sediment`	[-]	S
(For 1D only) Main channel averaged bed level, if MorFile keyword `[Output] MainChannelAveragedBedLevel` is true.	`wriMap_sediment`	[m]	S
(For 1D only) Main channel cell area, if MorFile keyword `[Output] MainChannelCellArea` is true.	`wriMap_sediment`	[m^2]	S
(For 1D only) Main channel cell width, if MorFile keyword `[Output] MainChannelWidthAtFlux` is true.	`wriMap_sediment`	[m]	U
# The above variables are for models with sediment (Deltares (2026d)),			
# when `[sediment] Sedimentmodelnr > 0` and `[sediment] SedFile` and `MorFile` are not empty, `Sedimentmodelnr` is 4.			
# The following variables are for models with sediment,			
# when `[sediment] Sedimentmodelnr > 0`, and either `[sediment] SedFile` or `MorFile` is empty or `Sedimentmodelnr` is not 4.			
All sediment concentrations.	`wriMap_sediment`	[kg/m^3]	S
All erodable layer thickness per size fraction in flow element centers,if MDU keyword `[sediment] Jaceneqtr = 1`.	`wriMap_sediment`	[m]	S
Flow element center bedlevel (bl),if MDU keyword `[sediment] Jaceneqtr = 1`.	`wriMap_sediment`	[m]	S
All erodable layer thickness per size fraction in flow element corners,if MDU keyword `[sediment] Jaceneqtr` is not 1.	`wriMap_sediment`	[m]	CN
Flow element corner bedlevel (zk),if MDU keyword `[sediment] Jaceneqtr` is not 1.	`wriMap_sediment`	[m]	CN

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
# The above variables are for models with sediment,			
# when `[sediment] Sedimentmodelnr > 0`, and either `[sediment] SedFile` or `MorFile` is empty or `Sedimentmodelnr` is not 4.			
# The following variables are for flow without waves, when `[waves] flowWithoutWaves` is true.			
RMS wave height, if EXT file contains `hrms` or `wavesignificantheight`, and MDU keyword `[waves] jamapsigwav = 0`.	`wriMap_waves`	[m]	S
Significant wave height, if EXT file contains `hrms` or `wavesignificantheight`, and MDU keyword `[waves] jamapsigwav` is non-zero.	`wriMap_waves`	[m]	S
Peak wave period, if EXT file contains `tp` or `tps` or `rtp` or `waveperiod`.	`wriMap_waves`	[s]	S
Mean direction of wave propagation relative to ksi-dir. ccw, if EXT file contains `dir` or `wavedirection`.	`wriMap_waves`	[deg]	S
Mean direction of wave propagation relative to ksi-dir. ccw, if EXT file contains `dir` or `wavedirection`.	`wriMap_waves`	[deg]	S
Surface layer wave forcing term, x-component, if EXT file contains `fx`.	`wriMap_waves`	[N/m^2]	S
Surface layer wave forcing term, y-component, if EXT file contains `fy`.	`wriMap_waves`	[N/m^2]	S
Bottom layer wave forcing term, x-component, if EXT file contains `wsbu`.	`wriMap_waves`	[N/m^2]	S
Bottom layer wave forcing term, y-component, if EXT file contains `wsbv`.	`wriMap_waves`	[N/m^2]	S
Wave-induced volume flux in x-direction, if EXT file contains `mx`.	`wriMap_waves`	[m^3/s/m]	S
Wave-induced volume flux in y-direction, if EXT file contains `my`.	`wriMap_waves`	[m^3/s/m]	S
Wave energy dissipation rate at the free surface, if EXT file contains `dissurf`.	`wriMap_waves`	[w/m^2]	S
Wave energy dissipation rate due to white capping, if EXT file contains `diswcap`.	`wriMap_waves`	[w/m^2]	S
Wave orbital velocity, if EXT file contains `ubot`.	`wriMap_waves`	[m/s]	S
# The above variables are for flow without waves, when `[waves] flowWithoutWaves` is true.			
# The following variables are for flow with waves, when `[waves] flowWithoutWaves` is false.			
Wave energy per square meter, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[J/m^2]	S

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Roller energy per square meter, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[J/m^2]	S
Roller energy dissipation per square meter, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[W/m^2]	S
Wave breaking energy dissipation per square meter, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[W/m^2]	S
Wave bottom energy dissipation per square meter, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[W/m^2]	S
Radiation stress, including x- and y-components, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[N/m^2]	S
Radiation stress, xy-component, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[N/m^2]	S
Sea surface wave phase celerity, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[m/s]	S
Sea surface wave group celerity, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[m/s]	S
Sea surface wave mean frequency, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[rad/s]	S
Sea surface wave wavenumber, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[rad/m]	S
Sea surface wave ratio group phase speed, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[-]	S
Sea surface wave refraction celerity, if MDU keyword `[waves] Wavemodelnr = 4`.	`wriMap_waves`	[rad/s]	S
Sea surface wave wavelength, if MDU keyword `[waves] Wavemodelnr = 4` and `windmodel`, read from the specified file in MDU keyword `[waves] SurfbeatInput`, is 1.	`wriMap_waves`	[m]	S
Wind source term on wave energy, if MDU keyword `[waves] Wavemodelnr = 4`, and `windmodel` and `windsource` that are read from the specified file in MDU keyword `[waves] SurfbeatInput`, are 1.	`wriMap_waves`	[J/m^2/s]	S

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Wind source term on wave period, if MDU keyword `[waves] Wavemodelnr = 4`, and `windmodel` and `windsource`, read from the specified file in MDU keyword `[waves] SurfbeatInput`, are 1.	`wriMap_waves`	[s/s]	S
(For 3D models only) Surface layer wave forcing term, including x- and y-components, if MDU keyword `[waves] Wavemodelnr` is 3 or 4.	`wriMap_waves`	[N/m^2]	S
(For 3D models only) Water body wave forcing term, including x- and y-components, if MDU keyword `[waves] Wavemodelnr` is 3 or 4.	`wriMap_waves`	[N/m^2]	S
RMS wave height, if MDU keyword `[waves] Wavemodelnr > 0` and `[waves] jamapsigwav = 0`.	`wriMap_waves`	[m]	S
Significant wave height, if MDU keyword `[waves] Wavemodelnr > 0` and `[waves] jamapsigwav` is non-zero.	`wriMap_waves`	[m]	S
Wave orbital velocity, if MDU keyword `[waves] Wavemodelnr > 0`.	`wriMap_waves`	[m/s]	S
Stokes drift including x- and y-components, if MDU keyword `[waves] Wavemodelnr > 0`.	`wriMap_waves`	[m/s]	S or S3D
Stokes drift including n- and t-components, if MDU keyword `[waves] Wavemodelnr > 0`.	`wriMap_waves`	[m/s]	U or U3D
Wave from direction, if MDU keyword `[waves] Wavemodelnr > 0`.	`wriMap_waves`	[deg from N]	S
Wave period, if MDU keyword `[waves] Wavemodelnr > 0`.	`wriMap_waves`	[s]	S
Wave force including x- and y-components, if MDU keyword `[waves] Wavemodelnr` is 3 or 4.	`wriMap_waves`	[N/m^2]	S or S3D
Wave force at velocity point including n- and t-components, if MDU keyword `[waves] Wavemodelnr` is 3 or 4.	`wriMap_waves`	[N/m^2]	U or U3D
# The above variables are for flow with waves, when `[waves] flowWithoutWaves` is false.			
Time-varying dune height, if BedformFile keyword `[bedform] BdfOut` and `[bedform] Bdf` are true.		[m]	S
Time-varying dune length, if BedformFile keyword `[bedform] BdfOut` and `[bedform] Bdf` are true.		[m]	S
Ripple roughness height, if BedformFile keyword `[bedform] BdfOut` is true and, either MDU keywords `[waves] Wavemodelnr > 0` and `[waves] Rouwav = VR04`, or Van Rijn 2004 transport formula is used.		[m]	S

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
Megaripple roughness height, if BedformFile keyword `[bedform] BdfOut` is true and, either MDU keywords `[waves] Wavemodelnr > 0` and `[waves] Rouwav = VR04`, or Van Rijn 2004 transport formula is used.		[m]	S
Dune roughness height, if BedformFile keyword `[bedform] BdfOut` is true and, either MDU keywords `[waves] Wavemodelnr > 0` and `[waves] Rouwav = VR04`, or Van Rijn 2004 transport formula is used.		[m]	S
Bedform roughness height, if BedformFile keyword `[bedform] BdfOut` is true and, either MDU keywords `[waves] Wavemodelnr > 0` and `[waves] Rouwav = VR04`, or Van Rijn 2004 transport formula is used.		[m]	S
# The following variables are for trachytope roughness:			
Chezy roughness from trachytopes, if MDU keyword `[physics] UnifFrictType = 0`.	`wriMap_trachytopes`	[$m^{0.5}$/s]	L
Manning roughness from trachytopes, if MDU keyword `[physics] UnifFrictType = 1`.	`wriMap_trachytopes`	[s/$m^{0.333}$]	L
White-Colebrook roughness from trachytopes, if MDU keyword `[physics] UnifFrictType` is 2 or 3.	`wriMap_trachytopes`	[m]	L
Roughness from trachytopes, if MDU keyword `[physics] UnifFrictType` is not equal to 0, 1, 2, or 3.	`wriMap_trachytopes`	[-]	L
Stem density of vegetation.		[1/m^2]	S
Stem diameter of vegetation.		[m]	S
Stem height of vegetation.		[m]	S
Calibration factor for roughness.	`wriMap_calibration`	[$m^{0.5}$/s]	L
# The above variables are for trachytope roughness.			
# The following variables are about nudging, when the EXT file contains `nudge_salinity_temperature`.			
Nudging relaxing time.	`wriMap_nudging`	[s]	S
Nudging temperature.	`wriMap_nudging`	[$^\circ$C]	S3D
Nudging salinity.	`wriMap_nudging`	[1e-3]	S3D
Difference of nudging temperature with temperature.	`wriMap_nudging`	[$^\circ$C]	S3D
Difference of nudging salinity with salinity.	`wriMap_nudging`	[1e-3]	S3D

(continued on next page)

DRAFT

(continued from previous page)

Quantity name	MDU option	Unit	Location
# The above variables are about nudging, when the EXT file contains `nudge_salinity_temperature`.			

DRAFT

Additionally, some variables specific for 1D networks can be written. Most are related to the concept of *ground level* in 1D. The ground level is not to be confused with bed level. The ground level of a 1D node is equal to:

- the `streetLevel`, if a storage node is defined on that node (Section C.18) and if `useTable=false`. Storage on ground is allowed if `storageType`≠`closed`.
- the highest `levels` value, if a storage node is defined on that node (Section C.18) and if `useTable=true`.
- the highest cross section value (interpolated from the two nearest cross sections) when no storage node is defined on that node. For closed cross sections (see section C.17.1), this level is only used to compute freeboard; no storage above ground level is allowed.

The additional available output variables on 1D nodes are:

- Freeboard at each node (only for 1D nodes):
  - Freeboard is calculated as ground level minus water level. It is the height still available for further water level increase.
  - When no storage node is present nor cross sections are nearby, a fill value is written.
  - Freeboard can be negative when the water level is above the ground level (only relevant for non-closed storage nodes with a street level or open water cross sections).
  - Freeboard has a minimal value of 0 if the storage node or nearest cross sections are closed.
- Waterdepth on ground at each node (only for 1D nodes):
  - This is calculated as water level minus ground level, only when ground level is defined and storage on ground is allowed on that node.
  - When no storage on ground is allowed, a fill value is written.
  - Its minimal value is 0 (for instance when the water level is below ground level).
- Volume on ground at each node (only for 1D nodes):
  - This is calculated as current volume minus the maximum volume below ground level.
  - When no storage on ground is allowed, a fill value is written.
  - Its minimal value is 0 (for instance when current volume is smaller than the maximum volume below ground level, i.e., when the water level is below ground level).
- Cumulative time at each node (only for 1D nodes) when water is above ground level, or more precisely: when waterdepth on ground is larger than `wriMap_wet_waterdepth_threshold` (this allows to use a different 'wet' threshold for output files than the computational flooding threshold `Epshu`).
- Current total 1D2D net inflow (discharge) and cumulative total 1D2D net inflow (volume), (only for 1D nodes):
  - The current total 1D2D net inflow (discharge) at a 1D node is the summation of instantaneous discharge through all the connected 1D2D links at each output time.
  - The cumulative total 1D2D net inflow (volume) at a 1D node is the cumulative total net volume that has flowed into this node through all the connected 1D2D links since the starting time of the simulation. At each computational time step it is incremented by the summation of the net volume through all the connected 1D2D links.
- Current total lateral net inflow (discharge) and cumulative total lateral net inflow (volume), (only for 1D nodes):
  - The current total lateral net inflow (discharge) at a 1D node is the summation of instantaneous discharge through all the connected laterals at each output time.
  - The cumulative total lateral net inflow (volume) at a 1D node is the cumulative total net volume that has flowed into this node through all the connected laterals since the starting time of the simulation. At each computational time step it is incremented by the summation of the net volume through all the connected laterals.

The additional available output variables on 1D links are:

- water level gradient (only for 1D links). This is the gradient in flow link direction. Depending on the input model grid, flow links may sometimes be oriented opposite to the branch orientation.

DRAFT

Variables on flow links/velocity points

Variables on grid cell edges (represented by the velocity point) are typically defined as follows:

```
double q1(time, nFlowLink) ;
  q1:coordinates = "FlowLink_xu FlowLink_yu" ;
  q1:long_name = "Flow flux" ;
  q1:units = "m3 s-1" ;
```

Other quantities that can be written are:

- ◇ Normal velocities at the old and new time level;
- ◇ Horizontal viscosity and diffusivity;
- ◇ Edge-centered wind speed components;
- ◇ Effective roughness values from trachytopes/vegetation (section 12.3)

Variables for flow analysis

By selecting `wriMap_flow_analysis = 1` under `[Output]` in the MDU file, a number of flow analysis variables will be written to the map file. These variables can be used to find possible bottlenecks in the model.

Table F.20: *Flow analysis variables in the map file. Column 'Location' shows the grid location: S means pressure points, U means velocity points.*

Quantity name	Unit	Location
Number of times negative depth was calculated.	[-]	S
Cumulative number of times negative depth was calculated.	[-]	S
Number of times no nonlinear convergence was caused	[-]	S
Cumulative number of times no nonlinear convergence was caused	[-]	S
Number of times a node was limiting for the computational time step	[-]	S
Cumulative number of times a node was limiting for the computational time step	[-]	S
Courant number	[-]	S

Remarks:

- ◇ During the simulation, the non-linear iteration can calculate negative depths. In general an occasional negative depth does not affect the simulation. However frequently calculated negative depths can be the cause of strange results or instabilities.
- ◇ The report of 'no convergence in the non linear iteration' (counted in `Number of times no nonlinear convergence is caused`), is a strong indicator that something might be wrong with the model. This count is incremented for the flow node that showed the largest water level difference in the last nonlinear iteration.
- ◇ The performance of a model is affected by the maximal timestep that can be used during the simulation. In `Number of times a node was limiting for the computational time step` the locations that are bottlenecks for the maximal timestep can be found. Using this information, the locations that are restricting for the maximal timestep can be investigated. Subsequently an attempt can be made to find a solution to reduce these bottlenecks.
- ◇ The Courant number is the limiting factor for the maximal timestep. This variable can be used to investigate whether only a few locations are limiting the timestep or whether it uniformly distributed across the complete model domain.
- ◇ The `Courant number` is an instantaneous value, valid for the moment the output is written to the map file.

DRAFT

◇ The other output variables come in two forms. The first of each two variables contains the number of occurrences during the past map output interval, whereas the second (denoted with "Cumulative"/_cum) contains the total number of occurrences since the start of the simulation until the current map output time.

F.3.3 Class map files as netCDF clm-file

The class map option gives output in a netCDF file, where the size of the output file is significantly less then the size of a map-file. This is achieved by storing the field of interest (currently only water levels and water depths) not as a double (8 bytes), but as a byte that represents the class it is in. As the class will not change every time, compressing is used to get even smaller files than the factor 8 based on the ratio double/byte.

This format is very suitable for making animations of dryfall and flooding.

The class map file is defined in the mdu-file by:

```
[output]
...
ClassMapFile               = weirfree_clm.nc   # Filename for class map output
WaterlevelClasses          = -3.0 2.0 5.0      # Water level classes (m)
WaterdepthClasses          = 0.5 1.0 5.0 10.0  # Water depth classes (m)
ClassMapInterval           = 5.0               # Class map output interval (s)
```

The class map file is always in UGRID format. When running in parallel, a class map file is generated for each process. Use mapmerge to collect them in a single file.

F.3.4 Restart files as netCDF rst-file

Restart files <*mdu_name*_yyyymmdd_HHMMSS_rst.nc> are almost identical to map files, except that they contain all data for just a single time. The grid and flow geometry information is not present, except for the flow cell/link coordinates. Moreover, restart files resulted from a parallel run contains parallelization information. For this reason, restart files are less suitable for plotting, but efficient for restarting a computation.

F.3.5 Volume tables per- gridpoint output as netCDF file

Dimensions

The per- gridpoint output net file contains all 1D flow geometry dimensions, plus the volume table levels and increment:

```
        netcdf PerGridpoint_TestModel_01 {
dimensions:
        Two = 2 ;
        network1d_nEdges = 4 ;
        network1d_nNodes = 5 ;
        network1d_nGeometryNodes = 141 ;
        strLengthIds = 40 ;
        strLengthLongNames = 80 ;
        mesh1d_nNodes = 28 ;
        mesh1d_nEdges = 27 ;
        nmesh1d_FlowElemContourPts = 7 ;
        levels = 45 ;
        increment = 1 ;
```

Furthermore full network information is included:

```
int network1d ;
        network1d:cf_role = "mesh_topology" ;
        network1d:long_name = "Topology data of 1D network" ;
        network1d:edge_dimension = "network1d_nEdges" ;
        network1d:edge_geometry = "network1d_geometry" ;
        network1d:edge_node_connectivity = "network1d_edge_nodes" ;
        network1d:node_coordinates = "network1d_node_x network1d_node_y" ;
        network1d:node_dimension = "network1d_nNodes" ;
        network1d:topology_dimension = 1 ;
        network1d:node_id = "network1d_node_id" ;
        network1d:node_long_name = "network1d_node_long_name" ;
        network1d:branch_id = "network1d_branch_id" ;
        network1d:branch_long_name = "network1d_branch_long_name" ;
        network1d:edge_length = "network1d_edge_length" ;
int network1d_edge_nodes(network1d_nEdges, Two) ;
        network1d_edge_nodes:cf_role = "edge_node_connectivity" ;
        network1d_edge_nodes:long_name = "Start and end nodes of network edges" ;
char network1d_branch_id(network1d_nEdges, strLengthIds) ;
        network1d_branch_id:long_name = "ID of branch geometries" ;
char network1d_branch_long_name(network1d_nEdges, strLengthLongNames) ;
        network1d_branch_long_name:long_name = "Long name of branch geometries" ;
double network1d_edge_length(network1d_nEdges) ;
        network1d_edge_length:long_name = "Real length of branch geometries" ;
        network1d_edge_length:units = "m" ;
char network1d_node_id(network1d_nNodes, strLengthIds) ;
        network1d_node_id:long_name = "ID of network nodes" ;
char network1d_node_long_name(network1d_nNodes, strLengthLongNames) ;
        network1d_node_long_name:long_name = "Long name of network nodes" ;
double network1d_node_x(network1d_nNodes) ;
        network1d_node_x:units = "m" ;
        network1d_node_x:standard_name = "projection_x_coordinate" ;
        network1d_node_x:long_name = "x-coordinate of network nodes" ;
double network1d_node_y(network1d_nNodes) ;
        network1d_node_y:units = "m" ;
        network1d_node_y:standard_name = "projection_y_coordinate" ;
        network1d_node_y:long_name = "y-coordinate of network nodes" ;
int network1d_geometry ;
        network1d_geometry:geometry_type = "line" ;
        network1d_geometry:long_name = "1D Geometry" ;
        network1d_geometry:node_count = "network1d_geom_node_count" ;
        network1d_geometry:node_coordinates = "network1d_geom_x network1d_geom_y" ;
int network1d_geom_node_count(network1d_nEdges) ;
        network1d_geom_node_count:long_name = "Number of geometry nodes per branch" ;
double network1d_geom_x(network1d_nGeometryNodes) ;
        network1d_geom_x:units = "m" ;
        network1d_geom_x:standard_name = "projection_x_coordinate" ;
        network1d_geom_x:long_name = "x-coordinate of branch geometry nodes" ;
double network1d_geom_y(network1d_nGeometryNodes) ;
        network1d_geom_y:units = "m" ;
        network1d_geom_y:standard_name = "projection_y_coordinate" ;
        network1d_geom_y:long_name = "y-coordinate of branch geometry nodes" ;
int network1d_branch_order(network1d_nEdges) ;
        network1d_branch_order:long_name = "Order of branches for interpolation" ;
        network1d_branch_order:mesh = "network1d" ;
        network1d_branch_order:location = "edge" ;
int network1d_branch_type(network1d_nEdges) ;
        network1d_branch_type:long_name = "Type of branches" ;
        network1d_branch_type:mesh = "network1d" ;
        network1d_branch_type:location = "edge" ;
```

As is the 1D mesh topology information:

```
int mesh1d ;
        mesh1d:cf_role = "mesh_topology" ;
        mesh1d:long_name = "Topology data of 1D mesh" ;
        mesh1d:topology_dimension = 1 ;
        mesh1d:coordinate_space = "network1d" ;
        mesh1d:edge_node_connectivity = "mesh1d_edge_nodes" ;
        mesh1d:node_dimension = "mesh1d_nNodes" ;
        mesh1d:edge_dimension = "mesh1d_nEdges" ;
        mesh1d:node_coordinates = "mesh1d_node_branch mesh1d_node_offset
```

```
        mesh1d_node_x mesh1d_node_y" ;
        mesh1d:edge_coordinates = "mesh1d_edge_branch mesh1d_edge_offset
        mesh1d_edge_x mesh1d_edge_y" ;
        mesh1d:node_id = "mesh1d_node_id" ;
        mesh1d:node_long_name = "mesh1d_node_long_name" ;
int mesh1d_node_branch(mesh1d_nNodes) ;
        mesh1d_node_branch:long_name = "Index of branch on which mesh nodes are
        mesh1d_node_branch:start_index = 0 ;
double mesh1d_node_offset(mesh1d_nNodes) ;
        mesh1d_node_offset:long_name = "Offset along branch of mesh nodes" ;
        mesh1d_node_offset:units = "m" ;
double mesh1d_node_x(mesh1d_nNodes) ;
        mesh1d_node_x:units = "m" ;
        mesh1d_node_x:standard_name = "projection_x_coordinate" ;
        mesh1d_node_x:long_name = "x-coordinate of mesh nodes" ;
        mesh1d_node_x:bounds = "mesh1d_FlowElemContour_x" ;
double mesh1d_node_y(mesh1d_nNodes) ;
        mesh1d_node_y:units = "m" ;
        mesh1d_node_y:standard_name = "projection_y_coordinate" ;
        mesh1d_node_y:long_name = "y-coordinate of mesh nodes" ;
        mesh1d_node_y:bounds = "mesh1d_FlowElemContour_y" ;
int mesh1d_edge_branch(mesh1d_nEdges) ;
        mesh1d_edge_branch:long_name = "Index of branch on which mesh edges are
        mesh1d_edge_branch:start_index = 0 ;
double mesh1d_edge_offset(mesh1d_nEdges) ;
        mesh1d_edge_offset:long_name = "Offset along branch of mesh edges" ;
        mesh1d_edge_offset:units = "m" ;
double mesh1d_edge_x(mesh1d_nEdges) ;
        mesh1d_edge_x:units = "m" ;
        mesh1d_edge_x:standard_name = "projection_x_coordinate" ;
        mesh1d_edge_x:long_name = "Characteristic x-coordinate of the mesh edge ;
double mesh1d_edge_y(mesh1d_nEdges) ;
        mesh1d_edge_y:units = "m" ;
        mesh1d_edge_y:standard_name = "projection_y_coordinate" ;
        mesh1d_edge_y:long_name = "Characteristic y-coordinate of the mesh edge ;
char mesh1d_node_id(mesh1d_nNodes, strLengthIds) ;
        mesh1d_node_id:long_name = "ID of mesh nodes" ;
char mesh1d_node_long_name(mesh1d_nNodes, strLengthLongNames) ;
        mesh1d_node_long_name:long_name = "Long name of mesh nodes" ;
int mesh1d_edge_nodes(mesh1d_nEdges, Two) ;
        mesh1d_edge_nodes:cf_role = "edge_node_connectivity" ;
        mesh1d_edge_nodes:long_name = "Start and end nodes of mesh edges" ;
        mesh1d_edge_nodes:start_index = 1 ;
double mesh1d_FlowElemContour_x(mesh1d_nNodes, nmesh1d_FlowElemContourPts) ;
        mesh1d_FlowElemContour_x:units = "m" ;
        mesh1d_FlowElemContour_x:standard_name = "projection_x_coordinate" ;
        mesh1d_FlowElemContour_x:long_name = "list of x-coordinates forming flow
        mesh1d_FlowElemContour_x:_FillValue = -999. ;
double mesh1d_FlowElemContour_y(mesh1d_nNodes, nmesh1d_FlowElemContourPts) ;
        mesh1d_FlowElemContour_y:units = "m" ;
        mesh1d_FlowElemContour_y:standard_name = "projection_y_coordinate" ;
        mesh1d_FlowElemContour_y:long_name = "list of y-coordinates forming flow
        mesh1d_FlowElemContour_y:_FillValue = -999. ;
```

Finally it contains the actual volume table data written. For the per- gridpoint output the volume and surface arrays are two dimensional, containing a value per volume table level for every 1D node.

```
double volume(mesh1d_nNodes, levels) ;
        volume:long_name = "Volume" ;
        volume:units = "m3" ;
double surface(mesh1d_nNodes, levels) ;
        surface:long_name = "Surface" ;
        surface:units = "m2" ;
int numlevels(mesh1d_nNodes) ;
        numlevels:long_name = "Number of levels" ;
        numlevels:units = "" ;
double bedlevel(mesh1d_nNodes) ;
        bedlevel:long_name = "Bedlevel" ;
        bedlevel:units = "m" ;
double topheight(mesh1d_nNodes) ;
        topheight:long_name = "TopHeight" ;
        topheight:units = "m" ;
```

DRAFT

```
double increment(increment) ;
        increment:long_name = "Increment" ;
        increment:units = "m" ;
```

F.3.6 NetCDF versions and file formats

The supported netCDF file formats are netCDF 3 and 4, configurable in the MDU via the `[output] NcFormat` keyword. Format 3 is the default for all netCDF output files, except for the classmap file, which always uses netCDF 4, because it uses compression.

The precision of the data written to the map-file and his-file can be configured in the MDU using the keywords `[output] NcMapDataPrecision` and `[output] NcHisDataPrecision`. Note that the station coordinates are always written in double precision for accuracy.

Compression of the Map (*_map.nc) and Fourier (*_fou.nc) output files can be enabled by setting `[output] NcCompression = 1`. Note that this feature only works when `[output] NcFormat=4`.

The history files, on the other hand, always use netCDF 3, because that was found beneficial for I/O performance.

To improve I/O performance, the following non-default settings may be considered. `NcNoUnlimited` writes the time dimension directly at full length, instead of at an unlimited length. This is only possible for regular model runs with start and stop time set (or at least the map output start and stop time set). The entire file size will be allocated directly upon initialization of the model. This setting is recommended (but not always required) to be combined with `NcFormat=4`, such that the variable size is not limited.

I/O performance can sometimes benefit greatly from disabling flushing every output timestep, using MDU setting `[output] NcNoForcedFlush`. The only consideration here is that during a running simulation, the buffered results may take some time before being visible in the output data on disk.

F.4 Shapefiles

Shapefiles[4] are widely used for visualization in geographic information system (GIS) software. A shapefile stores geometric locations and corresponding attributes information for the spatial features. D-Flow FM can generate shapefiles by setting keyword in the MDU file. Table F.21 shows what features can be written to a shapefile by D-Flow FM, and the corresponding MDU settings. (By default these keywords are zero, which disables writing the shapefile.) Moreover, this function is also valid in parallel simulation.

[4] https://www.esri.com/library/whitepapers/pdfs/shapefile.pdf

DRAFT

Table F.21: *Features and MDU settings for generating shapefiles*

Features for shapefiles	MDU setting
Cross sections	`[output] Wrishp_crs = 1`
Dambreaks	`[output] Wrishp_dambreak = 1`
Dry area	`[output] Wrishp_dryarea = 1`
Embankments	`[output] Wrishp_ebm = 1`
Fixed weirs	`[output] Wrishp_fxw = 1`
Gates	`[output] Wrishp_gate= 1`
General structures	`[output] Wrishp_genstruc = 1`
Observation stations	`[output] Wrishp_obs = 1`
Pump	`[output] Wrishp_pump = 1`
Source-sinks	`[output] Wrishp_src = 1`
Thin dams	`[output] Wrishp_thd = 1`
Weirs	`[output] Wrishp_weir= 1`

F.5 Post processing on his-file

There are three options for post processing on the history file. Two options are for finding the maximum value and one for the running mean maximum value for each history station or cross section. They are options of the program `dfmoutput`.

F.5.1 Max25 function

The max25 function computes statistics for all stations in the his-file. Basic statistics includes the `first`, `minimum`, `maximum` and `last` value. After determining these values, max25 computes an average over the surrounding values. The number of surrounding values is determined by the filter length. By default, two filter lengths are used: 13 and 25. The average over 25 values around the maximum (12 before, 12 after and the maximum itself) is abbreviated to `max25`. The max25 function differs from the `max_running_mean` function (See F.5.3) because it first determines the statistics and then computes an average. `max_running_mean` first computes a running mean for the whole timeseries and subsequently computes the maximum.

Basic usage for max25 is:

```
dfmoutput max25 --infile [hisfile.nc]
```

There are three optional arguments:

```
--outfile [name of the outputfile], default max25.out
--varname [variable to find maximum for], default waterlevel
--filterlength [list of numbers of points in time], default two values 13, 25
```

F.5.2 Maximum based on a generic filter

The second option is to find the maximum value after applying a 4th order monotone filter on the time series for each station. This filter has three parameters: the filter length and two coefficients. These parameters can be given by the user, see below.

Basic usage for maximum based on a generic filter is:

```
dfmoutput genfilter --infile [hisfile.nc]
```

There are five optional arguments:

```
--outfile [name of the outputfile], default max.out
--varname [variable to find maximum for], default waterlevel
--intval [filter period], default 6
--coefimpl [filter coefficient implicit part], default 0.3
--coefexpl [filter coefficient explicit part], default 0.3
```

F.5.3 Maximum based on a running mean filter

The third option is a maximum based on a running mean filter, the same as can be used in the fou-file. See C.15 for a definition of this filter.

Basic usage for maximum based on a running mean is:

```
dfmoutput max_running_mean --infile [hisfile.nc]
```

There are three optional arguments:

```
--outfile [name of the outputfile], default max_running_mean.out
--varname [variable to find maximum for], default waterlevel
--filterlength [filter lengths], default two values 13, 25
```

F.6 Time step analysis

Setting `[time] timeStepAnalysis = 1` in the MDU file generates a text file named <"runid".steps> at the level of the MDU file with information about the time step during the simulation. The information given is:

- `time0/60`: simulated time since start [min].
- `dts`: internal computational timestep [s]. This is the timestep used to advance the solution from the current time level to the next time level.
- `dtsc`: maximum timestep based on Courant number at the limiting cell `kkcflmx` [s].
- `kkcflmx`: 2D flow node number of the cell that is limiting the timestep based on Courant number [-]. It is equal to zero if no 2D flow node is limiting the timestep because the timestep is limited by `dtmax`.
- `kcflmx-kbot(kkcflmx)+1`: Layer (1 is the bottom-most layer) which is limiting the timestep. Variable `kcflmx` is the 3D flow node number of the cell that is limiting the timestep based on Courant number [-]. Variable `kbot` is the bottom layer 3D flow node number for each 2D flow node number. I.e., for a 2D flow node number `index`, variable `kbot(index)` provides the 3D flow node number of the bottom layer at that 2D location.
- `vol1(kcflmx)`: Volume of the limiting cell at end of the timestep [m^3].
- `squ2D(kcflmx)`: Outgoing 2D flux of the limiting cell [m^3/s]. Only if `autotimestep = 3` or `autotimestep = 4`. Otherwise, this value is equal to `squ(kcflmx)` (see next item). Note that by default `autotimestep = 1` in 2D and `autotimestep = 5` in 3D.
- `squ(kcflmx)`: Outgoing flux of the limiting cell [m^3/s].
- `sqi(kcflmx)`: Incoming flux of the limiting cell [m^3/s].

DRAFT

G Model generation

G.1 Introduction

Parts of the model building process can also be automated, to support model generation. This comes in addition to the full modelling process as described in chapter 6.

G.2 Grid generation

Automated grid generation is limited to a few basic types, described hereafter. More specific grid generation should be done by the modeller in RGFGRID. Automated grid generation is currently done from the commandline using the `dflowfm-cli.exe` program (`dflowfm` on Linux).

G.2.1 Uniform Cartesian grid

A uniform Cartesian (i.e., rectangular) grid can be generated by specifying the bounding box and desired grid resolution:

```
> dflowfm-cli.exe --gridgen:x0=x_LL:y0=y_LL:dx=Δx:dy=Δy:ncols=M:nrows=N
```

This produces a file <out_net.nc>. When the coordinate system for the grid should be spherical (WGS84), add the option `:spherical=1`.

G.2.2 Locally refined Cartesian grid

An existing grid can be locally refined based on an additional input file that defines the refinement locations. Here, we will now consider a uniform Cartesian (i.e., rectangular) that needs to be refined based on a raster file (e.g. an ArcInfo file). The grid refinement is currently based on 'Courant' refinement, but this can be applied to different refinement metrics as well. The command for refinement is:

```
> dflowfm-cli.exe --refine:hmin=Δx_min:dtmax=Δt_max:connect=1:outsidecell=1
                  unif_net.nc refineC.asc
```

The input file <unif_net.nc> is a file that has been produced by the `--gridgen` command (for this Cartesian case, see section G.2.1), for example with a grid resolution $\Delta x_{\max}$. The input file <refineC.asc> should contain values 1 for locations that need the finest size $\Delta x_{\min}$, 4 for one step coarser, 16 for two steps coarser. In general: 2^{2n}, with n the number of coarsening steps. The value of $\Delta t_{\max}$ is a fictitious value here, and should be set to $\Delta t_{\max} = \Delta x_{\min}/\sqrt{9.81}$. When entering only a few decimal digits for $\Delta t_{\max}$, make sure to round it up.

This again produces a file <out_net.nc>.

DRAFT

H Spatial editor

H.1 Introduction

The spatial editor is a generic feature of the User Interface for editing spatial data (e.g. bed level, roughness, viscosity, initial conditions, sediment availability). The spatial editor supports both point clouds and coverages (e.g. data on a grid or network). Therefore, you can use the spatial editor both to edit spatial data in general and to prepare model input. This Chapter describes the general features of the spatial editor (section H.2), (spatial) quantity selection (section H.3), geometry operations (section H.4), spatial operations (section H.5) and the operation stack (section H.6). The examples given below are typically focusing on use of the spatial editor in the D-Flow FM User Manual.

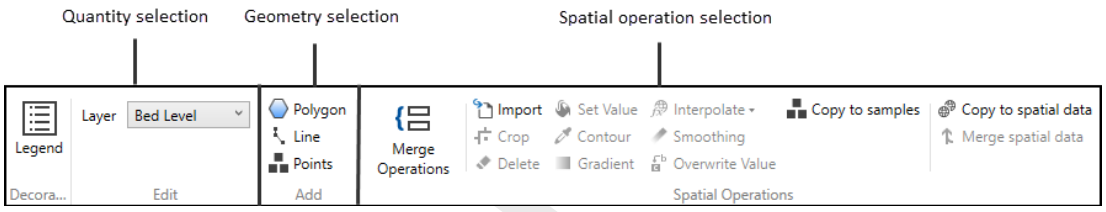


Figure H.1: *Overview of spatial editor functionality in Map ribbon*

H.2 General

H.2.1 Overview of spatial editor

The spatial editor functionality can be accessed from the “Spatial Operations” ribbon (Figure H.1). Typically, you first select which data set or quantity (either a point cloud or a coverage) to work on (e.g. bed level, roughness, viscosity, initial conditions, sediment availability), then a geometry (e.g. point, line, polygon) and finally which spatial operation to perform (e.g. crop, delete, set value, contour, interpolate, smoothing). Typical workflows are as follows:

Working on a point cloud dataset:

1 Import the dataset as point cloud (section H.2.2)
2 Activate/select the dataset (quantity) in the spatial editor (section H.3)
3 Select a geometry to perform an operation on (section H.4)
4 Select the spatial operation for this geometry (section H.5)
5 Repeat steps 3 and 4 until you are satisfied with the data
6 Export the dataset (section H.6.7)

Working on a coverage (e.g. model input):

1 Activate/select the spatial quantity to work on in the spatial editor (section H.3)
2 Optional: import a dataset as point cloud (section H.5.1)
3 Select a geometry to perform an operation on (section H.4)
4 Select the spatial operation for this geometry (section H.5)
5 Repeat steps 3 and 4 until you are satisfied with the data
6 Interpolate the point cloud to the grid or network (section H.5.10)
7 Optional: export the dataset (section H.6.7)

Upon performing a spatial operation, the ‘Operations’ panel will open (see Figure H.53) with the operations stack. This stack keeps track of the workflow of spatial operations that you performed. This helps you to make transparent how you arrived at your ‘final’ dataset without having to save all the intermediate datasets (steps) separately. Moreover, the stack is reproducible and easily editable without having to start all over again. When working on a coverage (e.g. the second workflow), point clouds can be used to construct the coverage. In this case the coverage (for example ‘Bed level’) is the ‘trunk’ of the workflow and the point clouds (appearing as sets in the stack) are ‘branches’ or subsets of this trunk (see Figure H.53). By selecting the set or coverage in the ‘Operations’ panel you determine on which

dataset you are working. The interpolate operation (section H.5.10) allows you to bring data from the point cloud (branch) to the coverage (trunk). For more information on the stack you are referred to section section H.6.

H.2.2 Import/export dataset

To import a (point cloud) dataset use the context menu on “project” in the “Project Tree”, select “import” from the context menu and select the option "Points from XYZ-file" (Figure H.2). There is yet another method to import the point cloud. Click on "Import" icon under the Home ribbon to obtain a drop-down menu with the list of importers. Select the option "Points from XYZ-file" under Spatial data section (Figure H.3) to launch the import wizard.

After the import, the point cloud will be added to the project tree. To activate the point cloud in the spatial editor, either double click the dataset in the project tree (Figure H.4) or select it from the drop down box in the spatial editor ribbon (Figure H.5).

Note: Exporter still to be implemented

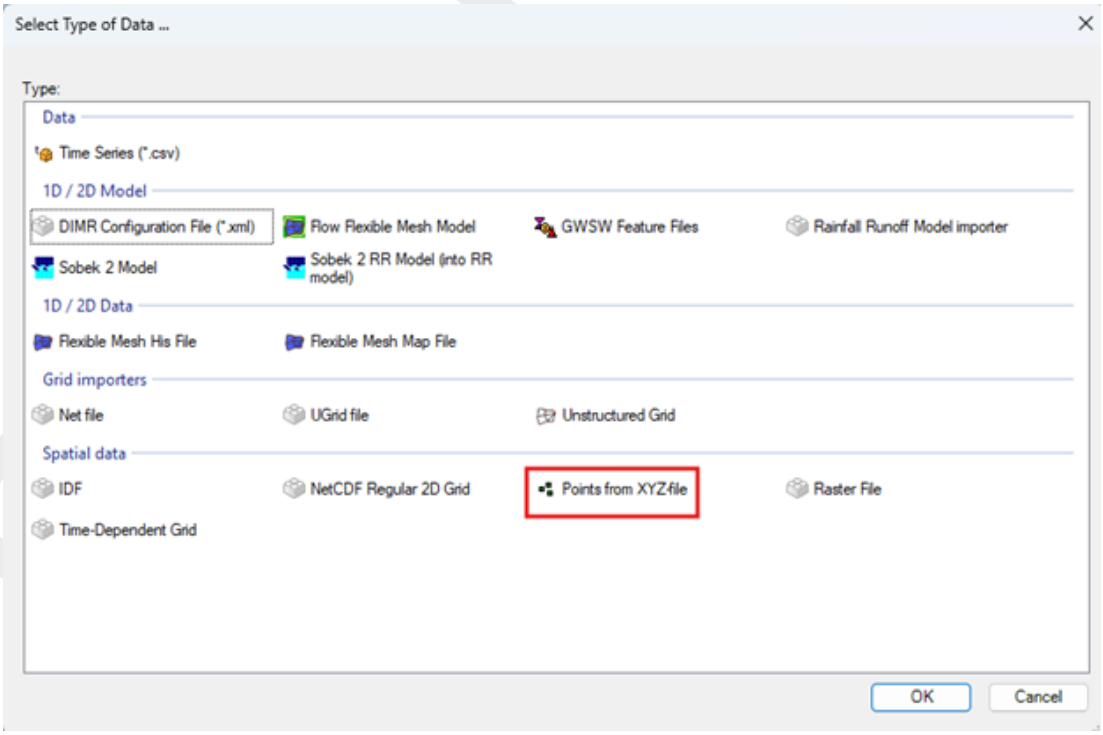


Figure H.2: *Importing a point cloud into the project using the context menu on “project” in the project tree*

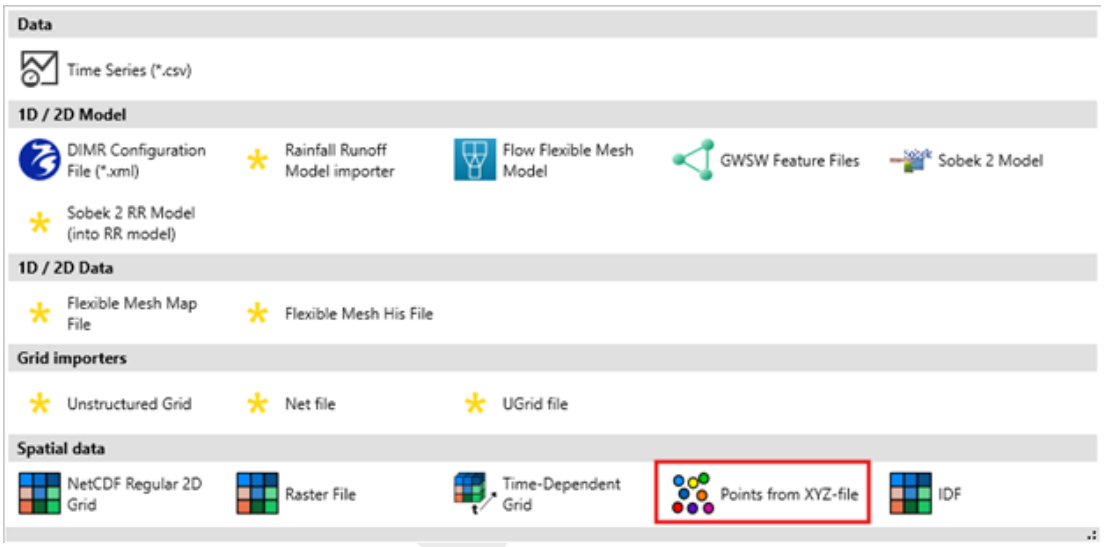


Figure H.3: *Importing a point cloud into the project using the "Import" drop-down menu in the Spatial Operations ribbon*

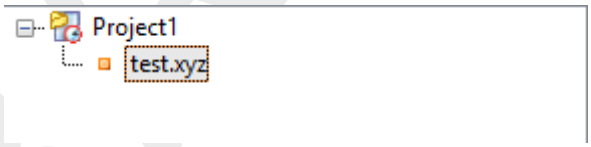


Figure H.4: *Activate the imported point cloud in the spatial editor by double clicking it in the project tree*

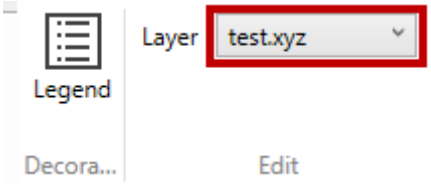


Figure H.5: *Activate the imported point cloud in the spatial editor by selecting it from the dropdown box in the Map ribbon*

H.2.3 Activate (spatial) model quantity

Similar to activating an imported dataset in the spatial editor, you can also activate a (spatial) model quantity (e.g. bed level, initial conditions, roughness, viscosity) in the spatial editor by double clicking the quantity in the project tree or selecting it from the dropdown box in the “Spatial Operations” ribbon.

H.2.4 Colorscale

In the spatial editor the colorscale for a spatial quantity can be made visible in the map window by clicking on the ”Legend" button in the “Spatial Operations” ribbon (Figure H.6). Then, in the Map window a colorscale is activated (Figure H.7). You can (de-)activate the color scale by clicking on the “Legend” button again. By default the colorscale is ranging from the minimum to the maximum value of the active dataset.

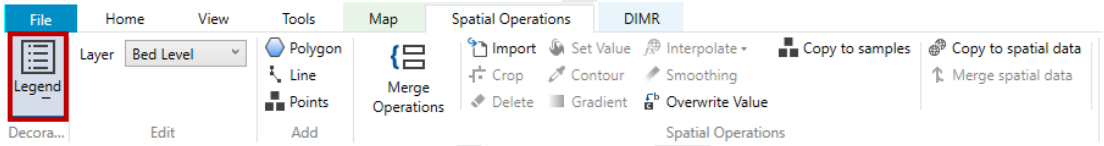


Figure H.6: *Legend button in Map ribbon*

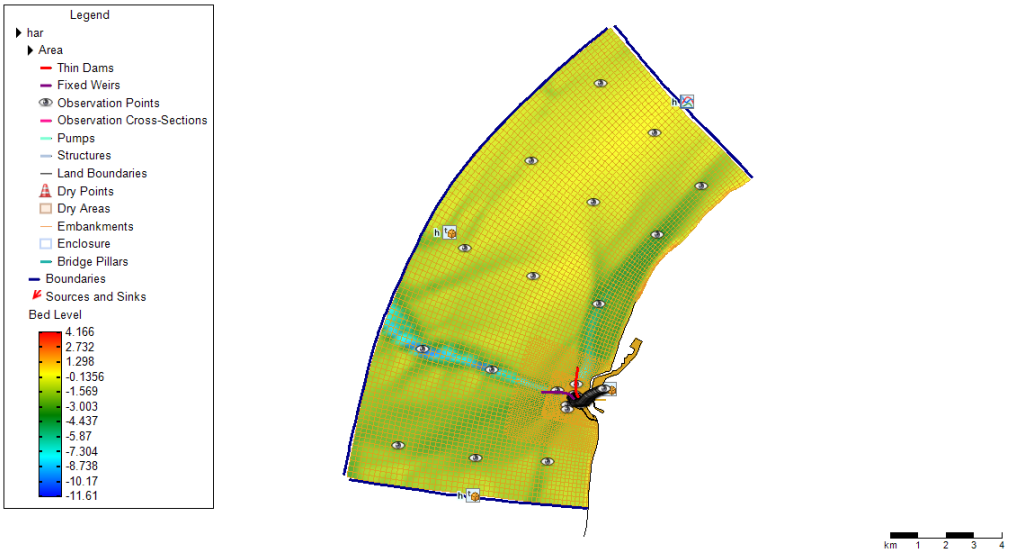


Figure H.7: *Activate the colorscale by using the Legend in the map ribbon and the colorscale will become visible in the map window.*

You can adjust the colormap and classes of the colorscale using the context menu on the spatial quantity in the “Map tree” and selecting “Legend Properties” (Figure H.8 left panel). A layer properties editor will open in which you can set the colormap to your own preferences (Figure H.8 right panel).

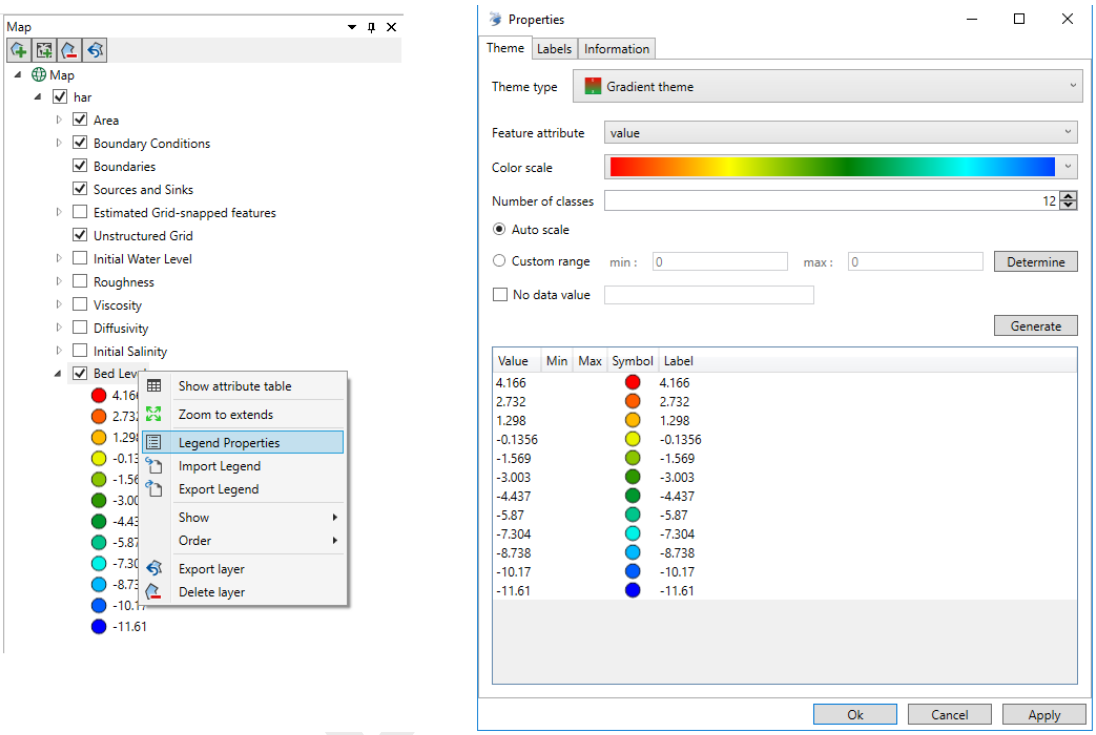


Figure H.8: *Edit the colorscale properties using the context menu on the active layer in the Map Tree*

H.2.5 Render mode

By default point clouds are rendered as (colored) points and coverages as shades (e.g. 'FillSmooth'). The render mode can be edited using the properties of the active layer Figure H.9. The User Interface offers the following render modes:

- Points
- Lines (only for coverages)
- Shades or 'FillSmooth' (only for coverages)
- Colored numbers
- Mono colored numbers

An example of a coverage rendered as colored numbers is given in Figure H.10.

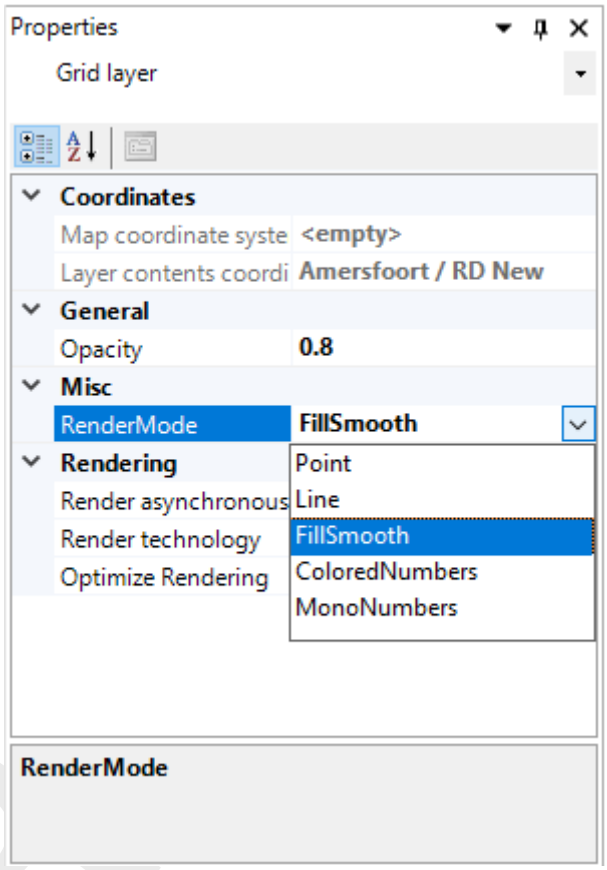


Figure H.9: *Select the rendermode for the active layer in the property grid.*

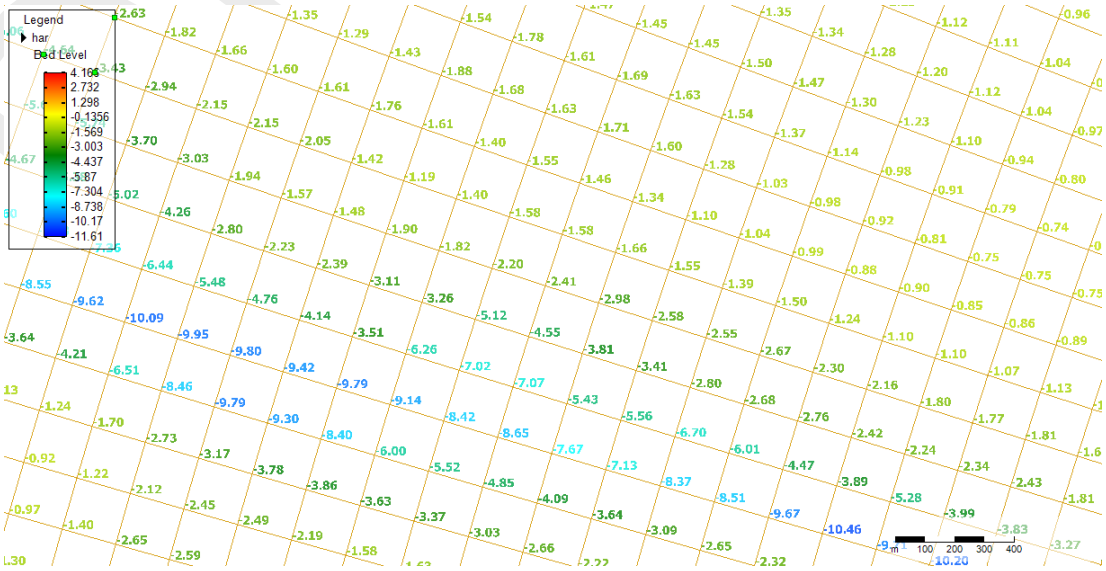


Figure H.10: *Example of a coverage rendered as colored numbers.*

H.2.6 Context menu

In addition to the selection of spatial operation from the 'Spatial Operations' ribbon (see section H.5), you can also select spatial operations using the context menu (e.g. context menu). After drawing a geometry and clicking the context menu all spatial operation available for the geometry will pop-up (see Figure H.11). The spatial operation will become active upon selecting it from the context menu.

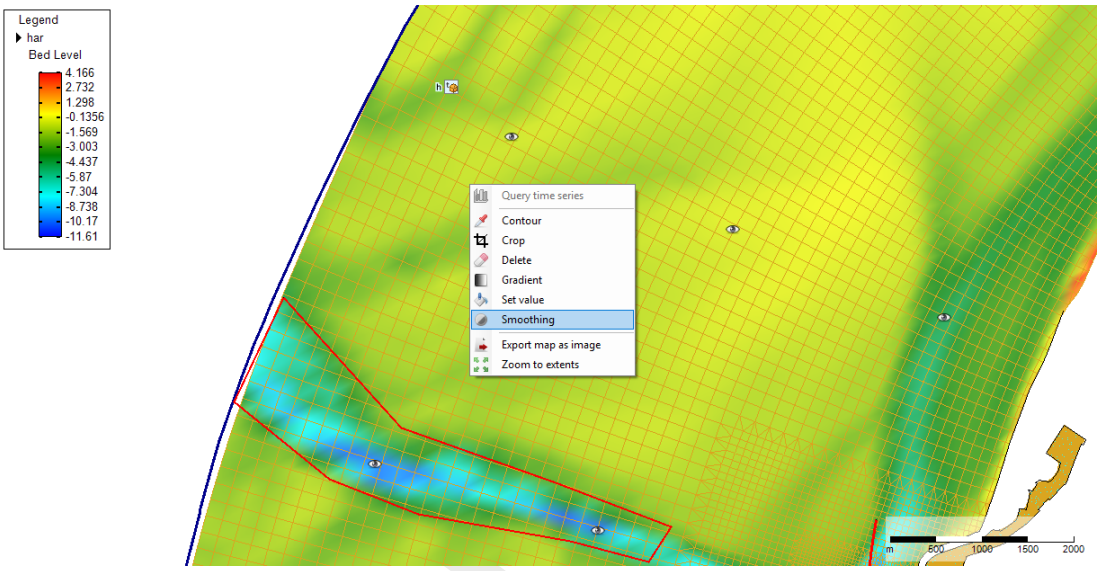


Figure H.11: *Selecting a smoothing operation for a polygon geometry from the context menu (using context menu)*

H.3 Quantity selection

A spatial quantity can be activated/selected either by double clicking it in the project tree (Figure H.12) or by selecting it from the dropdown box in the “Spatial Operations” ribbon (Figure H.13). Upon selecting the spatial quantity it will be shown as a point cloud (for a dataset) or coverage (for model input) on the central map. Typically, you start from a point cloud (either obtained from import or by generating samples yourself) which will eventually be interpolated to a grid or network (e.g. coverage). The spatial editor will keep track of both the changes made to the point cloud(s) and coverage of the selected spatial quantity. The information will be saved in the User Interface project and available the next time you open the project. **Note:** The operations are not yet saved in a human-readable/editable file

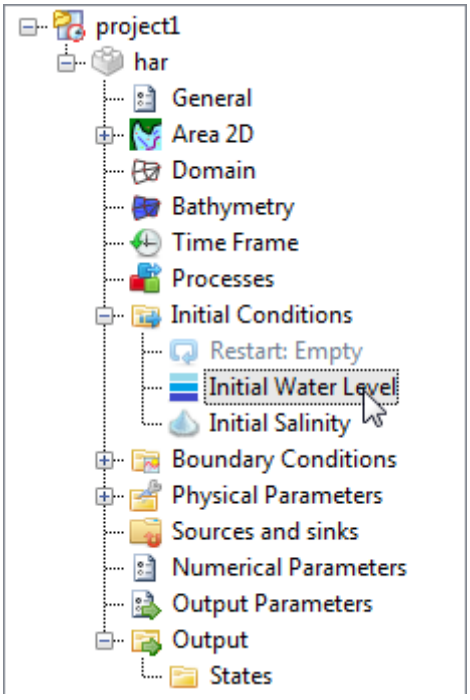


Figure H.12: *Activating a spatial quantity by double clicking it in the project tree (in this example ‘Initial Water Level’)*

DRAFT

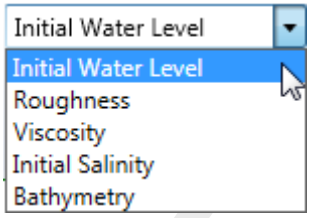


Figure H.13: *Activating a spatial quantity by selecting it from the dropdown box in the 'Spatial Operations' ribbon*

H.4 Geometry operations

The spatial editor supports three types of geometry operations: (1) polygons, (2) lines and (3) points (see also Figure H.14). The following sub-sections subsequently describe how these geometries can be selected and edited. If you do not select any of these three geometry operations, the spatial operation automatically applies to all the data.

Note: Please note that the drawn geometries are not yet persistent, implying that the geometries once drawn cannot be edited yet. Upon pressing the "Esc" button while in editing mode all drawn geometries will disappear.

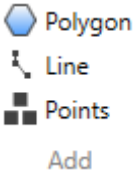


Figure H.14: *Overview of the available geometry operations in the 'Spatial Operations' ribbon*

H.4.1 Polygons

Upon selecting "Polygon" from the "Map" ribbon you can draw one or multiple polygons (Figure H.15). Each polygon point is defined by a single click with the LMB. The polygon is closed by double clicking the LMB. After drawing the (first) polygon, the available spatial operations for polygons are enabled in the "Spatial Operations" ribbon. The following spatial operations are available for polygons:

- Crop (section H.5.2)
- Delete (section H.5.3)
- Set Value (section H.5.4)
- Contour (section H.5.5)
- Gradient (section H.5.9)
- Smoothing (section H.5.11)
- Interpolate - only in case samples and a grid/network are available (section H.5.10)
- Copy to samples (section H.5.6)
- Copy to spatial data (section H.5.7) - only for grid coverages

The selected spatial operation applies to all the drawn polygons.

DRAFT

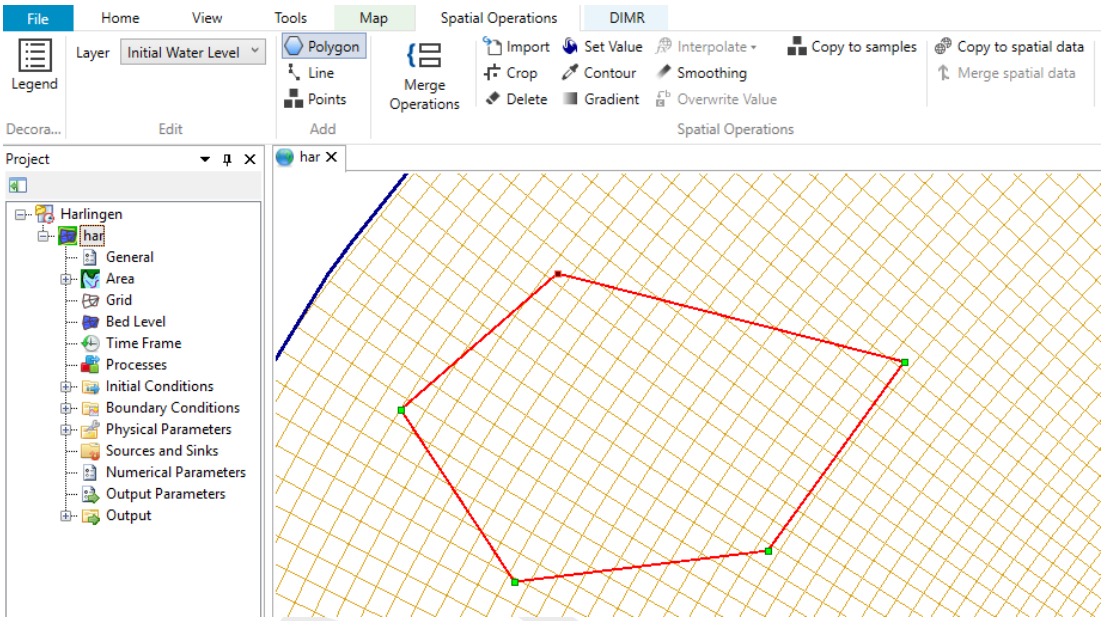


Figure H.15: *Activating the polygon operation and drawing polygons in the central map.*

H.4.2 Lines

Upon selecting “Line” from the “Map” ribbon you can draw one or multiple lines (Figure H.16). Each line point is defined by a single click with the LMB. The line is completed by double clicking the LMB. After drawing the (first) line, the available spatial operations for lines are enabled in the “Map” ribbon. The following spatial operations are available for lines:

- Contour (section H.5.5)
- Copy to samples (section H.5.6)
- Copy to spatial data (section H.5.7) - only for grid coverages

The selected spatial operation applies to all the drawn lines.

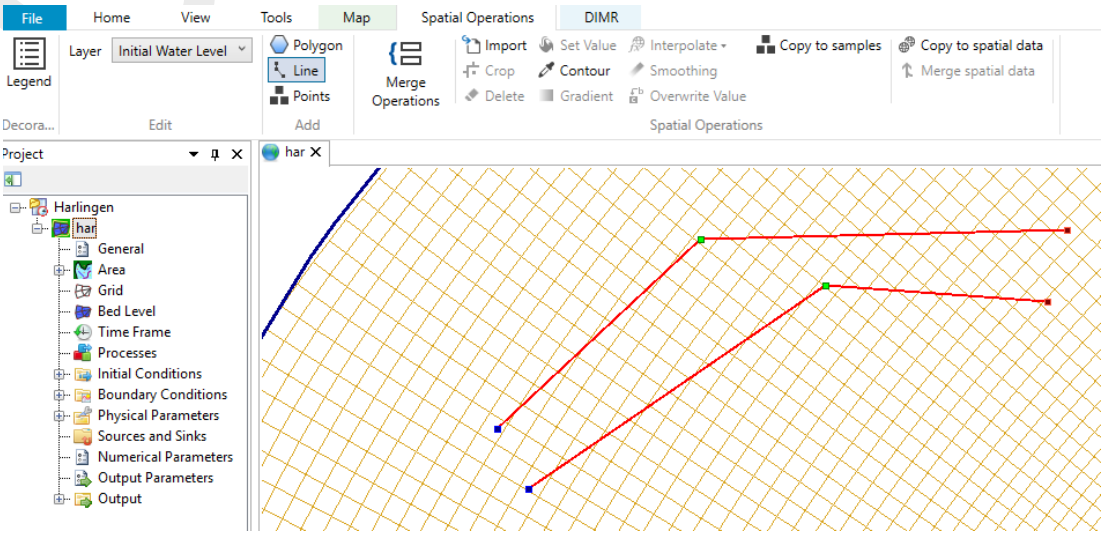


Figure H.16: *Activating the line operation and drawing lines in the central map.*

H.4.3 Points

Upon selecting “Add points” from the “Map” ribbon you can draw one or multiple points and assign a uniform value to them (Figure H.17). Each point is defined by a single click with the LMB. The group of points is completed by double clicking the LMB. Upon double clicking a popup appears in which you can assign a value to the points.

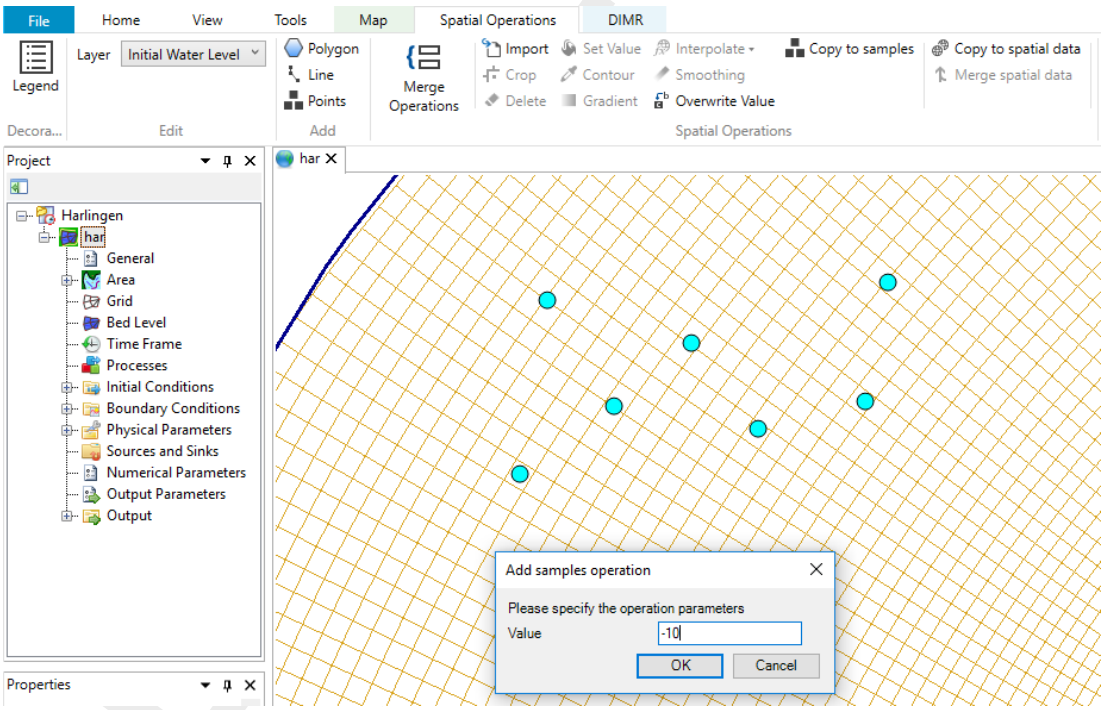


Figure H.17: *Activating the ‘Add points’ operation, drawing them in the central map and assigning a value to them.*

H.5 Spatial operations

The spatial editor supports the following spatial operations (see also Figure H.18):

- Import (section H.5.1) - only for point clouds
- Crop (section H.5.2)
- Delete (section H.5.3)
- Set Value (section H.5.4)
- Contour (section H.5.5) - only for point clouds
- Gradient (section H.5.9)
- Interpolate (section H.5.10) - only for point clouds
- Smoothing (section H.5.11)
- Change single value (section H.5.12) - only for grid coverages
- Merge spatial data (section H.5.8) - only for grid coverages
- Copy to samples (section H.5.6) - only for grid coverages
- Copy to spatial data (section H.5.7) - only for grid coverages

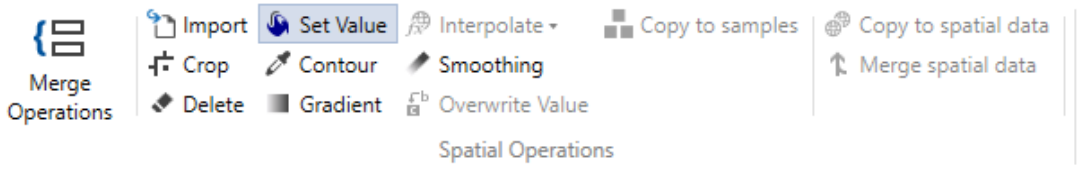


Figure H.18: *Overview of the available spatial operations in the ‘Spatial Operations’ ribbon*

The sections below provide a description of each operation.

H.5.1 Import point cloud

With the import operation you can import a point cloud for the selected spatial quantity. For this operation no geometry is required. The import operation is activated from the 'Spatial Operations' ribbon (Figure H.19). Upon importing a point cloud you are asked whether a coordinate transformation should be applied to the imported dataset (Figure H.20). By default it will be assumed that the imported data is in the same coordinate system as the model. If not, you can indicate from which to which coordinate system the data should be transformed. After import the point cloud is added to the operations stack (Figure H.21). The difference between this importer and importing a point cloud on the project level in the project tree (section H.2.2) is that for this importer the point cloud is directly assigned to the selected spatial quantity (e.g. model input) instead of being treated as a separate dataset.

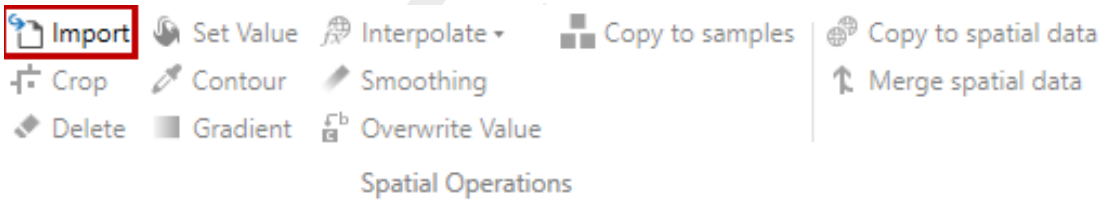


Figure H.19: *Importing a point cloud using the 'Import' operation from the 'Spatial Operations' ribbon*

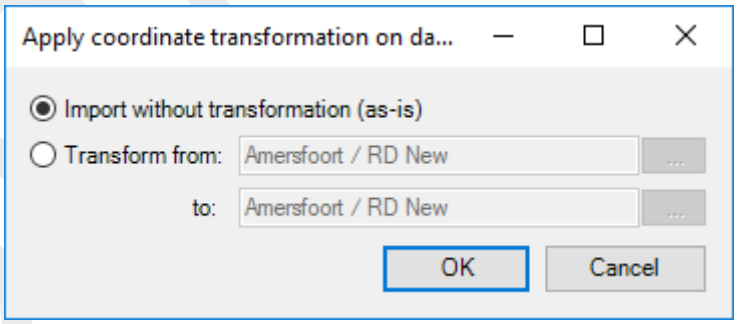


Figure H.20: *Option to perform a coordinate transformation on the imported point cloud*

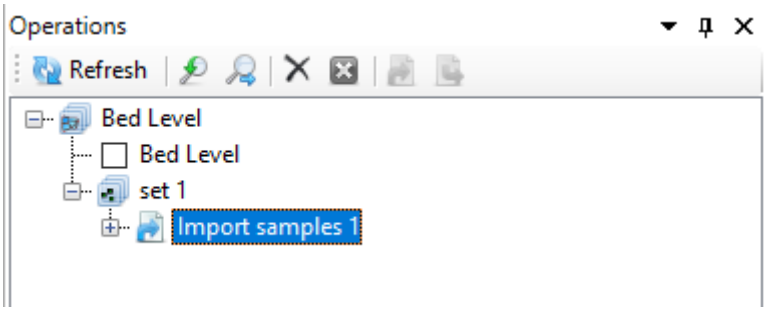


Figure H.21: *Appearance of import point cloud operation in the operations stack*

H.5.2 Crop

The crop operation crops a point cloud or coverage (depending on which one is active). The crop operation is activated from the 'Spatial Operations' ribbon and only available for polygon geometries. You can control which part of the data should be deleted by using polygons. If you provide a polygon outside the point cloud or coverage, all data will be deleted. For an example see Figure H.22.

After cropping (part of) the point cloud of coverage the operation is added to the operations stack (Figure H.23).

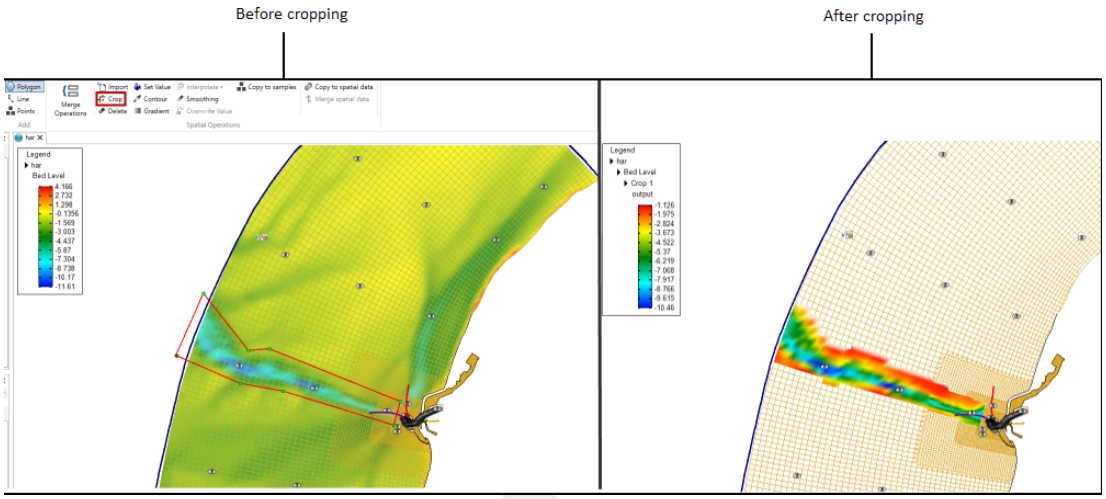


Figure H.22: *Performing a crop operation on a point cloud with a polygon using 'Crop' from the 'Spatial Operations' ribbon*

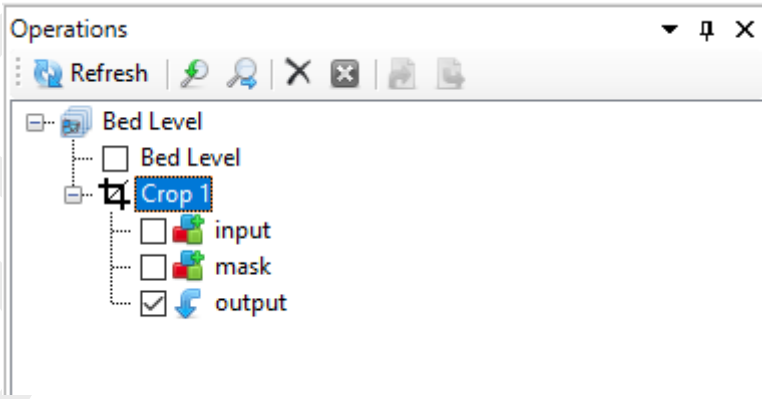


Figure H.23: *Appearance of crop operation in the operations stack*

H.5.3 Delete

The delete operation deletes (part of) a point cloud or coverage (depending on which one is active). The delete operation is activated from the 'Spatial Operations' ribbon. You can control which part of the data should be deleted by using polygons. If no polygons are provided, the total dataset will be deleted. For an example see Figure H.24. After erasing (part of) the point cloud or coverage the operation is added to the operations stack (Figure H.25).

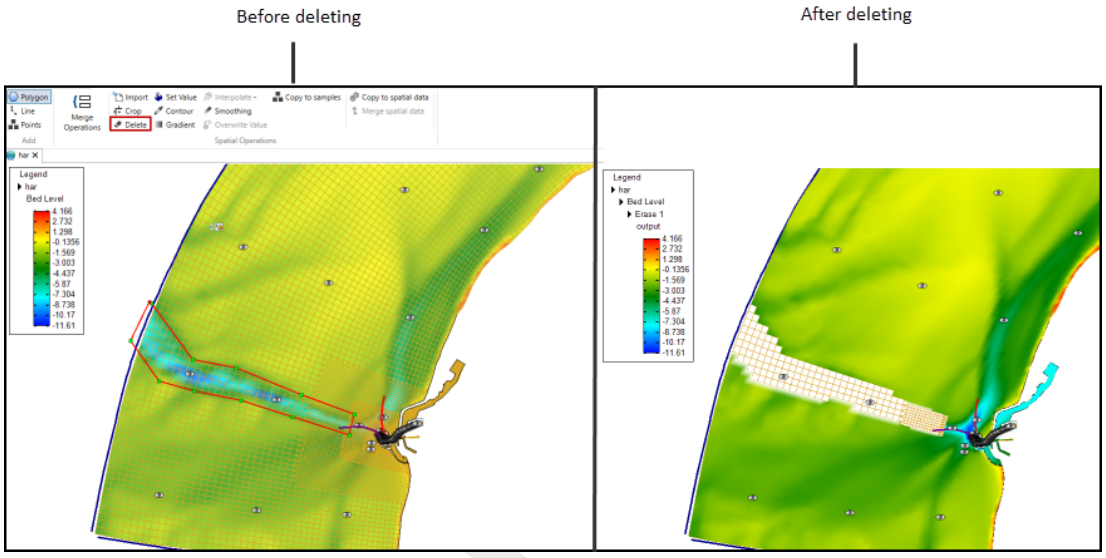


Figure H.24: *Performing an delete operation on a point cloud with a polygon using 'Delete' from the 'Spatial Operations' ribbon*

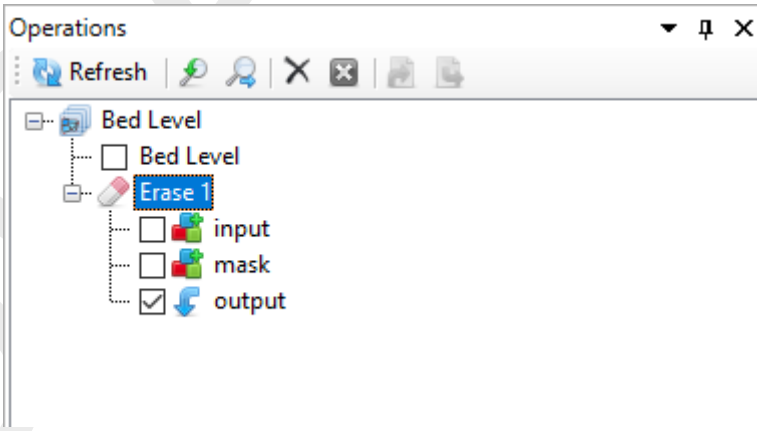


Figure H.25: *Appearance of delete operation in the operations stack*

H.5.4 Set value

The set value operation assigns a value to a point cloud or coverage (depending on which one is active). The set value operation is activated from the 'Spatial Operations' ribbon and only available for polygon geometries or for the total data set if no polygon is provided. By assigning a value, the user can choose from the following operations:

- **Overwrite** : overwrites all existing points within the polygon (excluding no data values) with the uniform value
- **Overwrite where missing (only for coverages)** : overwrites all missing values within the polygon with the uniform value
- **Add** : Adds the uniform value to all existing points within the polygon (excluding no data values)
- **Subtract** : Subtracts the uniform value from all existing points within the polygon (excluding no data values)
- **Multiply** : Multiplies all existing points within the polygon (excluding no data values) with the uniform value
- **Divide** : Divides all existing points within the polygon (excluding no data values) by the uniform value
- **Maximum** : Sets all existing points within the polygon (excluding no data values) to the maximum of its current value and the uniform value

◇ **Minimum** : Sets all existing points within the polygon (excluding no data values) to the minimum of its current value and the uniform value

For an example see Figure H.26. After performing a set value operation to (part of) the point cloud or coverage the operation is added to the operations stack (Figure H.27).

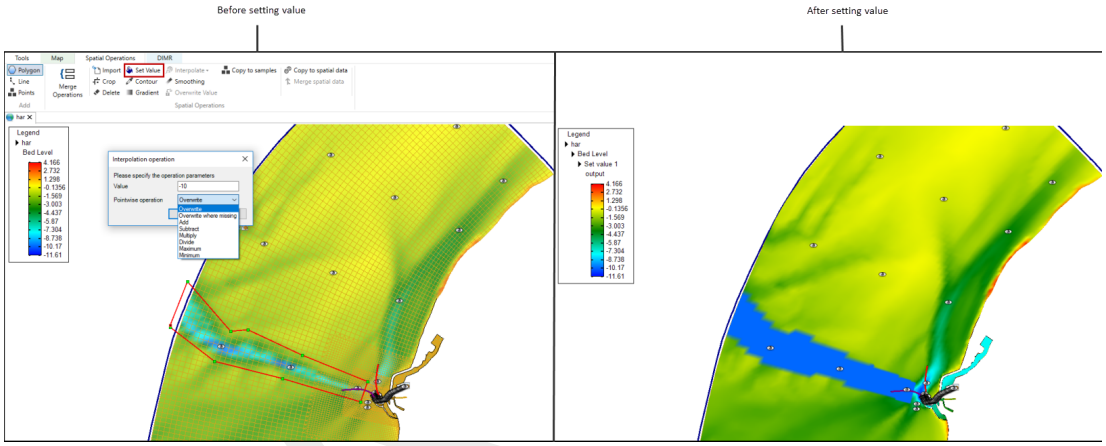


Figure H.26: *Performing a set value operation (e.g. overwrite) on a point cloud with a polygon using 'Set Value' from the 'Spatial Operations' ribbon*

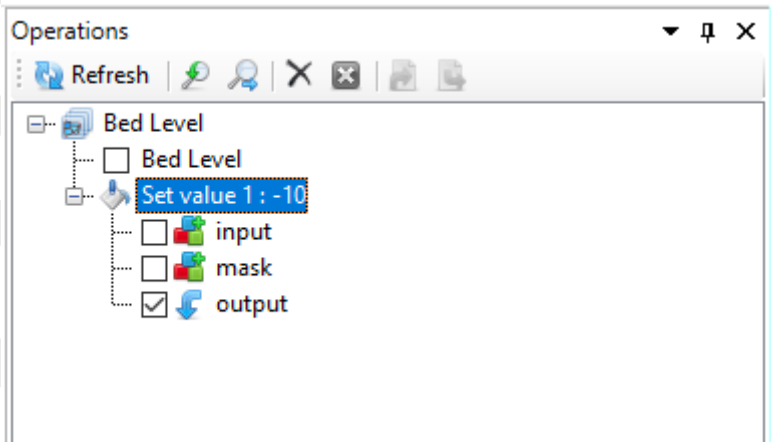


Figure H.27: *Appearance of set value operation in the operations stack*

H.5.5 Contour

The contour operation creates a point cloud with a uniform value along a line or polygon (depending on which one is active). The contour operation is activated from the 'Spatial Operations' ribbon. After drawing the lines or polygons you have to assign the uniform value (argument) and the sampling interval in m. This spatial operation can be useful to digitalize information from nautical charts for example. In this case you first have to import the nautical chart as a geotiff (Figure H.28), set the right map coordinate system (Figure H.29) and then use the contour operation Figure H.30. Sometimes the samples are created behind the geotiff. Then you can use the context menu on the samples layer in the Map tree to bring the samples to the front (Figure H.31). After applying the contour operation it is added to the stack (Figure H.32).

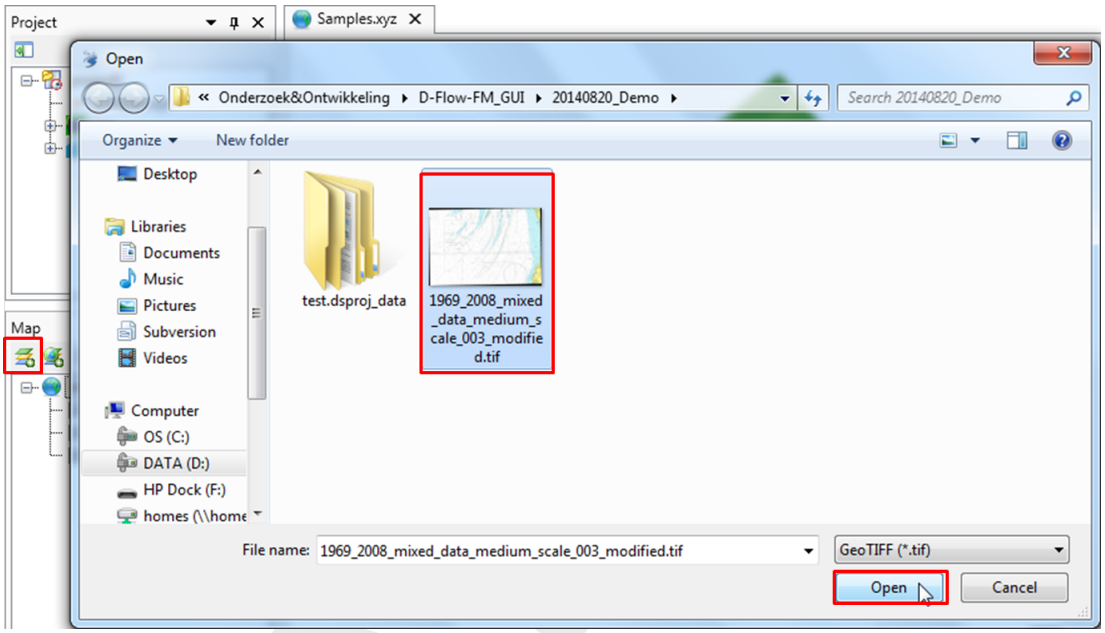


Figure H.28: *Import a nautical chart as a georeferenced tiff file*

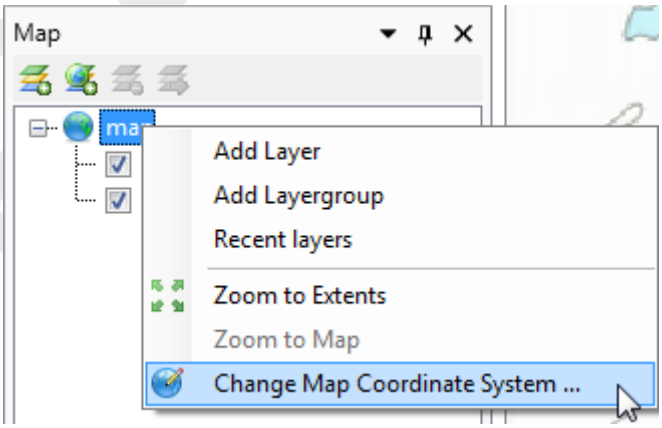


Figure H.29: *Set the right map coordinate system for the geotiff*

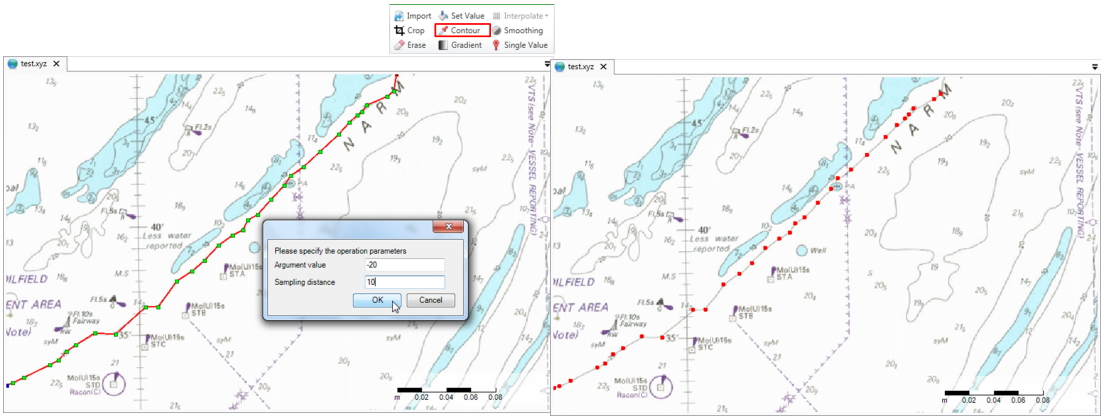


Figure H.30: *Performing a contour operation on a nautical chart using lines to define the contours and 'Contour' from the 'Spatial Operations' ribbon*

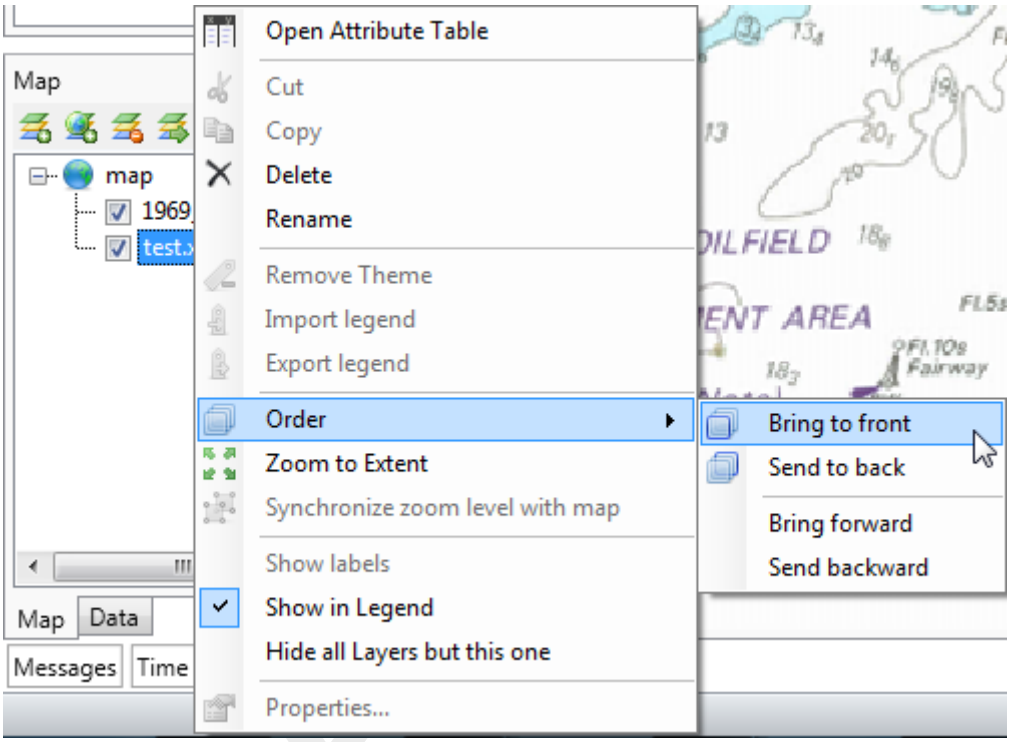


Figure H.31: *Bring the sample set to the front if it appears behind the nautical chart*

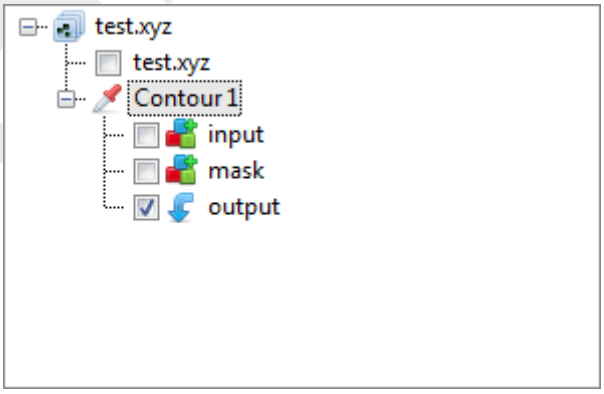


Figure H.32: *Appearance of contour operation in the operations stack*

H.5.6 Copy to samples

This operation converts the currently selected grid coverage to a sample set, which becomes that starting point of a new subset. If polygons have been drawn, the operation will confine the copy to the interiors. The operation will not convert missing values to samples. Note that the operation does keep a reference to the original copied grid coverage; if it is changes by a re-evaluation of the stack, the changes will affect the output point cloud.

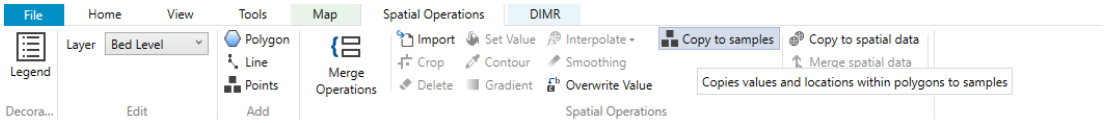


Figure H.33: *Applying the copy to samples operation*

Figure H.34: *Copy to samples operation result*

H.5.7 Copy to spatial data

This operation simply clones the grid coverage, starting a new subset with a snapshot of the currently selected operation output. Similarly to H.5.6, it keeps a reference to the input data and will perform the clone again upon re-evaluation.

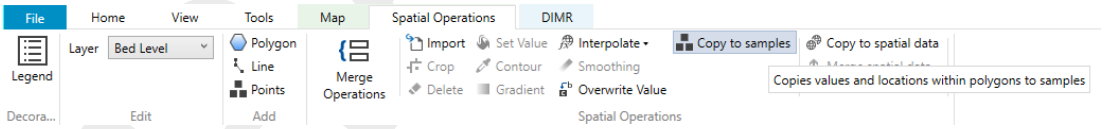


Figure H.35: *Applying the copy spatial data operation*

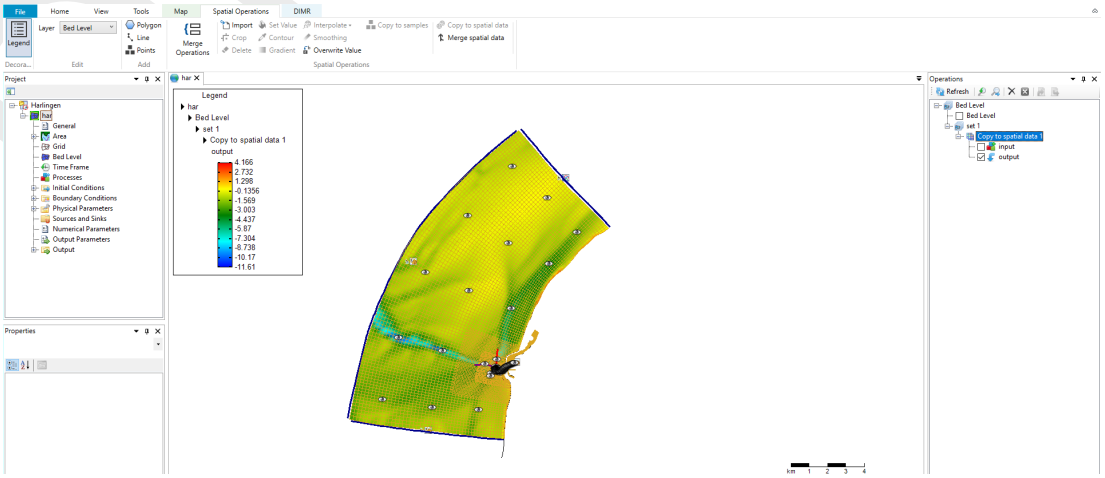


Figure H.36: *Copy spatial data operation result*

H.5.8 Merge spatial data

Whenever a subset contains a grid coverage as its editing data (after applying e.g. H.5.7) on the same grid as the main operation set, its result can be combined with the main set by applying this operation, similarly to the interpolation operation for sample sets, discussed below. Combining the grid coverages can be achieved with the usual point-wise methods.

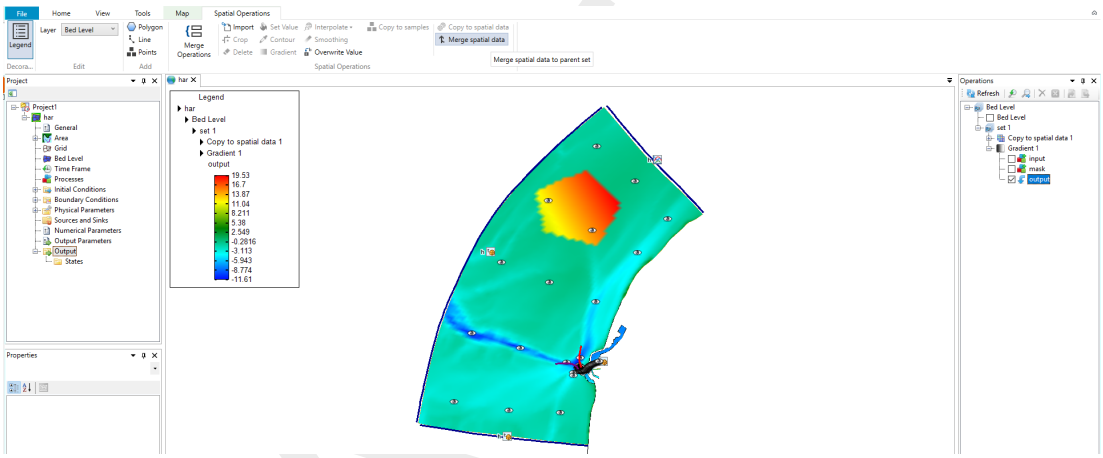


Figure H.37: *Activating the merge spatial data tool from the ribbon*

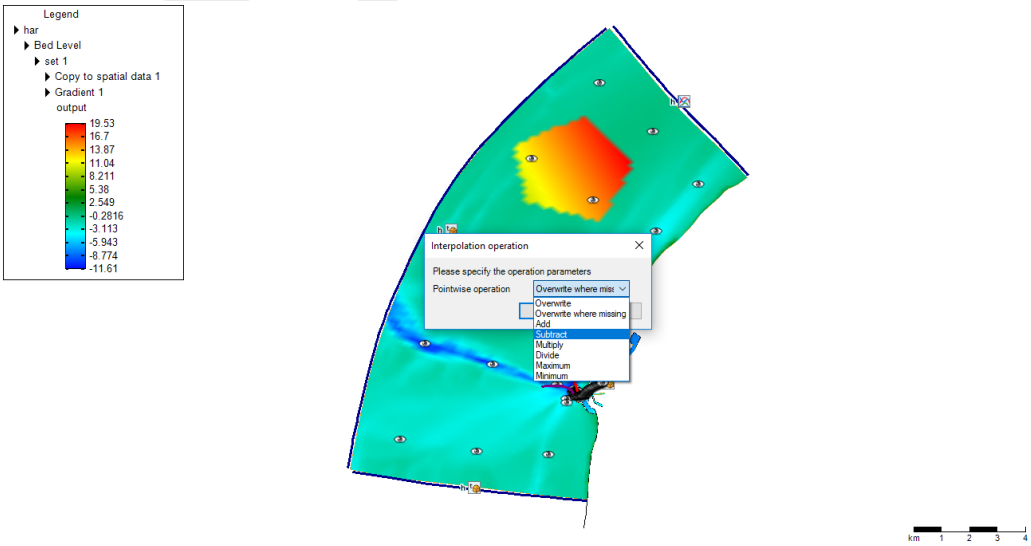


Figure H.38: *The merge operation requests a pointwise combination method*

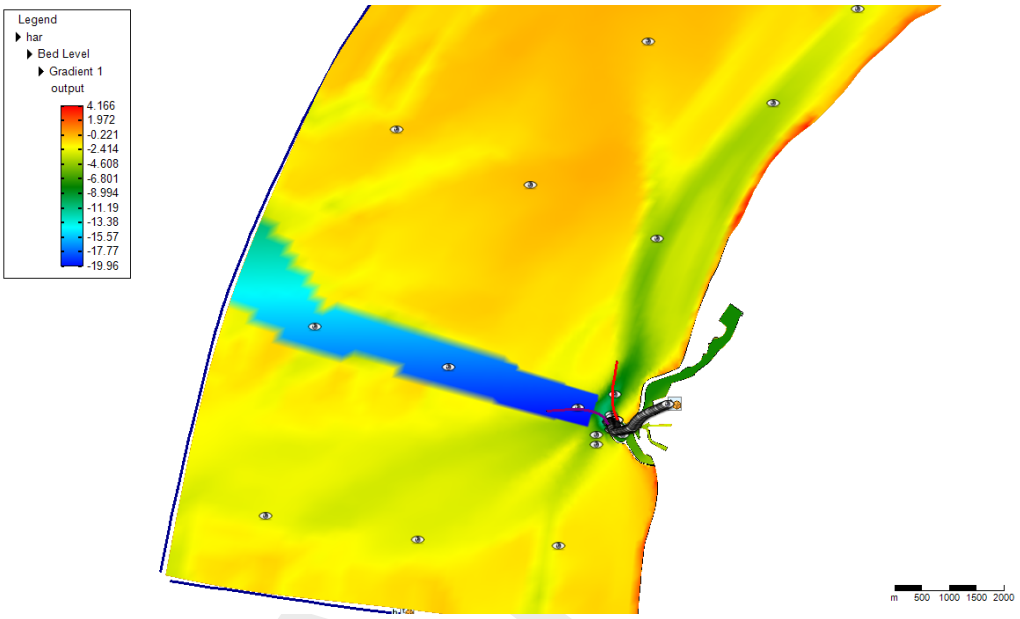


Figure H.39: *Resulting grid coverage*

H.5.9 Gradient

The gradient operation applies a gradient to a point cloud or grid coverage (depending on which one is active). The gradient operation is activated from the 'Spatial Operations' ribbon and only available for polygon geometries or for the total data set if no polygon is provided. You have to assign the initial (start) value, the final (end) value and the going to angle (according to the Cartesian convention with 0 degrees is East, 90 degrees is North, etc **Note:** This is not working properly yet). For an example see Figure H.40. After applying a gradient to (part of) the point cloud or coverage the operation is added to the operations stack (Figure H.41).

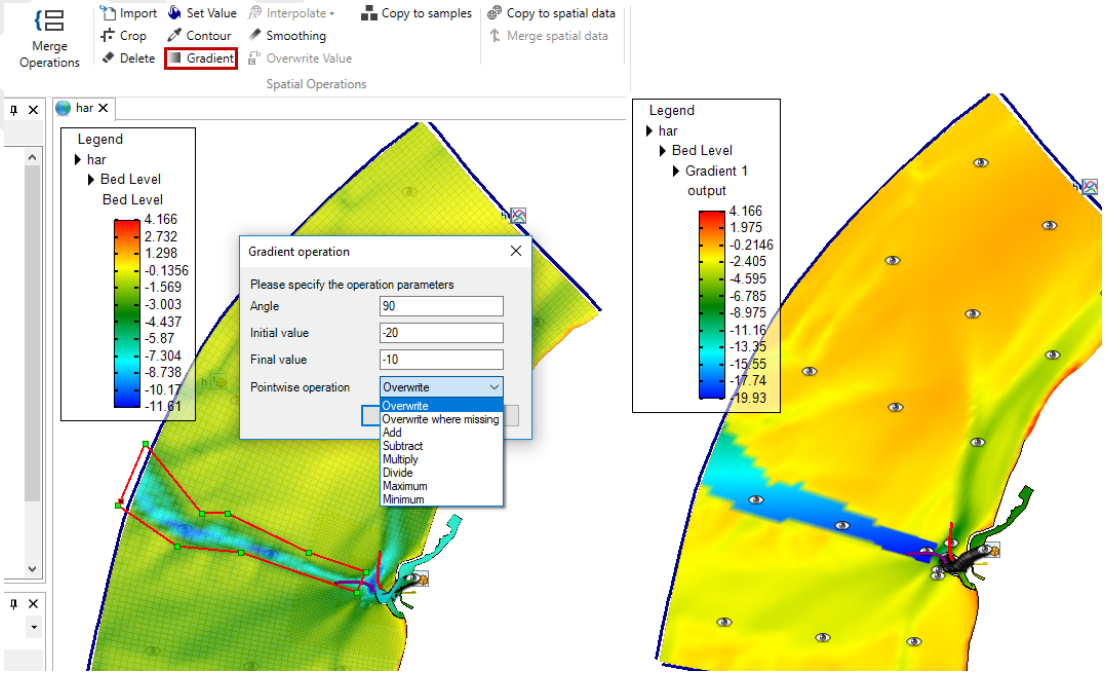


Figure H.40: *Performing a gradient operation on a point cloud with a polygon using 'Gradient' from the 'Spatial Operations' ribbon*

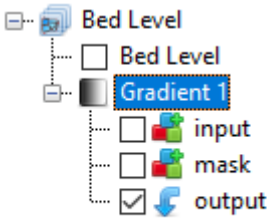


Figure H.41: *Appearance of gradient operation in the operations stack*

H.5.10 Interpolate

The interpolate operation is the way to get sample set(s) to a grid or network (e.g. coverage). In the operation stack this means that we are actively switching from working on a point cloud (helping to construct the coverage) to working on the selected coverage (or spatial quantity). The interpolation is performed on the data within a polygon or polygons (if provided) or all the data (if no polygons are provided). The interpolate operation can be performed on a single (selected) sample set or on multiple sample sets. Both methods are discussed below. The methods for interpolation are either

- ◇ Triangulation: performs a Delauney triangulation on the sample point set before projecting onto the grid.
- ◇ Averaging: combines sample points within a possible enlarged cell according to an algorithm of choice. The user can set the search cell expansion factor (rel. search cell size) and a threshold for the number of sample points within a cell (minimum sample points), see Figure H.42.

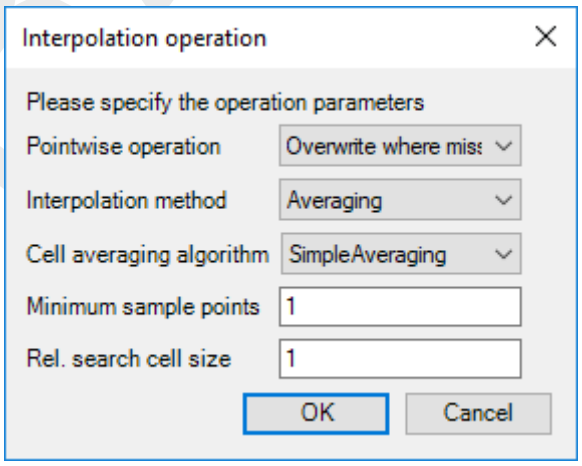


Figure H.42: *Interpolation Operation options*

Seven Cell averaging algorithms can be chosen by the user; see Figure H.43. These algorithms are explained below:

- ◇ SimpleAveraging: the arithmetic mean[1] of the samples inside the search area is taken.
- ◇ ClosestPoint: the value of the closest sample inside the search area is taken.
- ◇ MaximumValue: the maximum value of the samples inside the search area is taken.
- ◇ MinimumValue: the minimum value of the samples inside the search area is taken.
- ◇ InverseWeightedDistance: the inverse-distance-weighted mean[2] of the samples inside the search area is taken. Contrary to the arithmetic mean, in which all samples contribute equally to the mean,

[1] The arithmetic mean is given by:
$u(\mathbf{x}) = \frac{1}{N} \sum_{k=1}^{N} u_i$

[2] The inverse-distance-weighted mean is given by:

when using the inverse-distance-weighted average, the samples close to the query point have a larger contribution to the mean.

- ◇ MinAbs: the minimum of the absolute values of the samples inside the search area is taken.
- ◇ KdTree: This is an obsolete option, which will be removed in a future release.

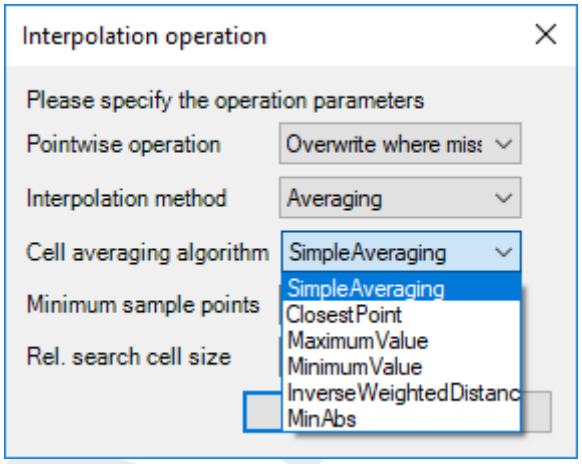


Figure H.43: *Averaging options*

By default, the interpolation will only overwrite missing values in the gridded data set. However, if the grid coverage already contains values, the user may choose to overwrite or combine the data by a pointwise arithmetic operation.

Interpolate single (selected) set

To perform interpolation on a single sample set, select the sample set (i.e. 'set1') in the operation stack and press 'Interpolate' in the 'Map' ribbon (Figure H.44). Since no polygon is provided in this example, all the samples will be interpolated to the grid. Use polygons if you would like to have more control over the interpolation. After the interpolation the operation is added to the operations stack (Figure H.45). Please note that after performing the interpolation the workflow in the stack is shifting from the sample set (i.e. 'set1' - which was a side step to construct the coverage) to the coverage (i.e. 'bed level').

$$u(\mathbf{x}) = \begin{cases} \frac{\sum_{i=1}^{N} w_i(\mathbf{x}) u_i}{\sum_{i=1}^{N} w_i(\mathbf{x})}, & \text{if } d(\mathbf{x}, \mathbf{x}_i) \neq 0 \text{ for all } i, \\ u_i, & \text{if } d(\mathbf{x}, \mathbf{x}_i) = 0 \text{ for some } i, \end{cases}$$

$$w_i(\mathbf{x}) = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)}$$

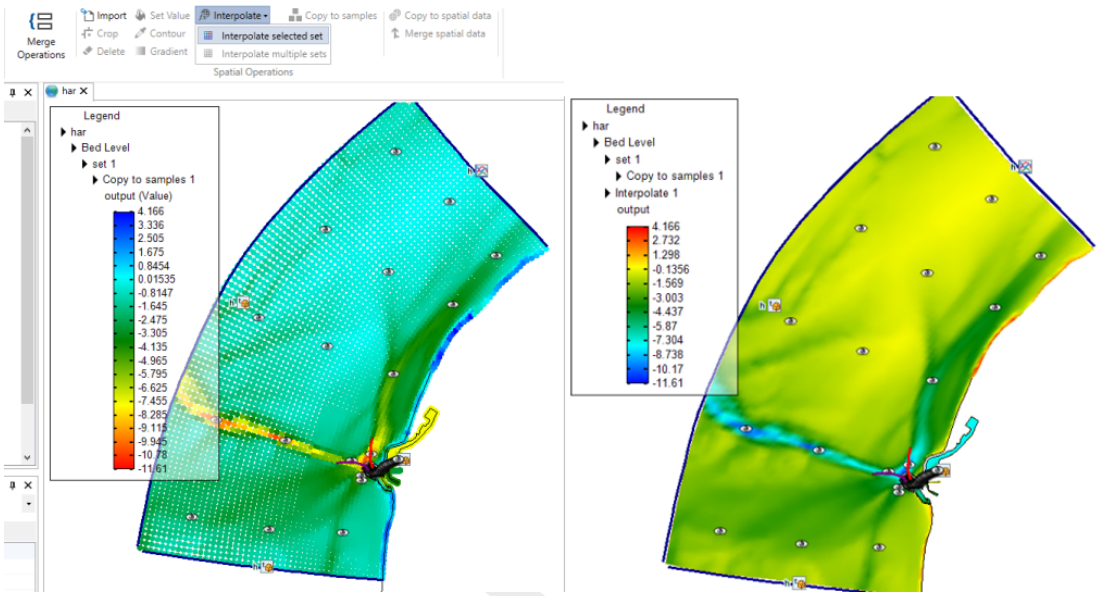


Figure H.44: *Performing an interpolation operation on a single sample set (without using a polygon) using 'Interpolate' from the 'Spatial Operations' ribbon*

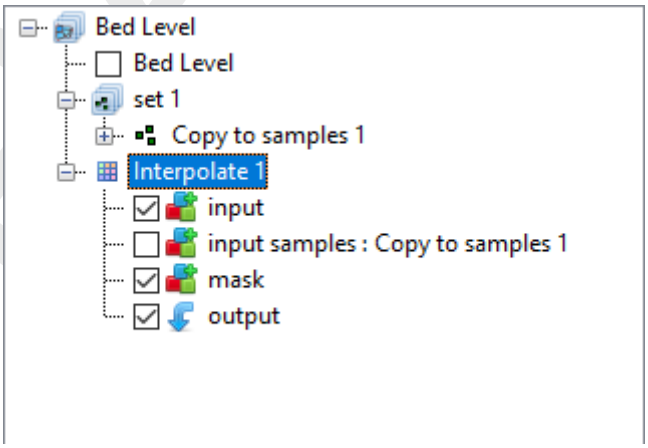


Figure H.45: *Appearance of interpolation of 'set1' to the coverage 'bed level' in the operations stack*

Interpolate multiple sets

To perform interpolation on multiple sample sets, select the active coverage (i.e. 'bed level') in the operation stack and press 'Interpolate' in the 'Map' ribbon (Figure H.46). In the popup you can select which sample sets to include in the interpolation (in this example both). Since no polygon is provided, all the samples (from the two sets) will be interpolated to the grid. Use polygons if you would like to have more control over the interpolation. After the interpolation the operation is added to the operations stack (Figure H.47). Again note that after performing the interpolation the workflow in the stack is shifting from the sample set (which was a side path to construct the coverage) to the coverage (i.e. bed level).

Note: Please note that interpolation of multiple sample sets can also be achieved by importing/combining different sample sets into the same set in the stack instead of using two separate sets. In this case you can just interpolate the single (selected) set.

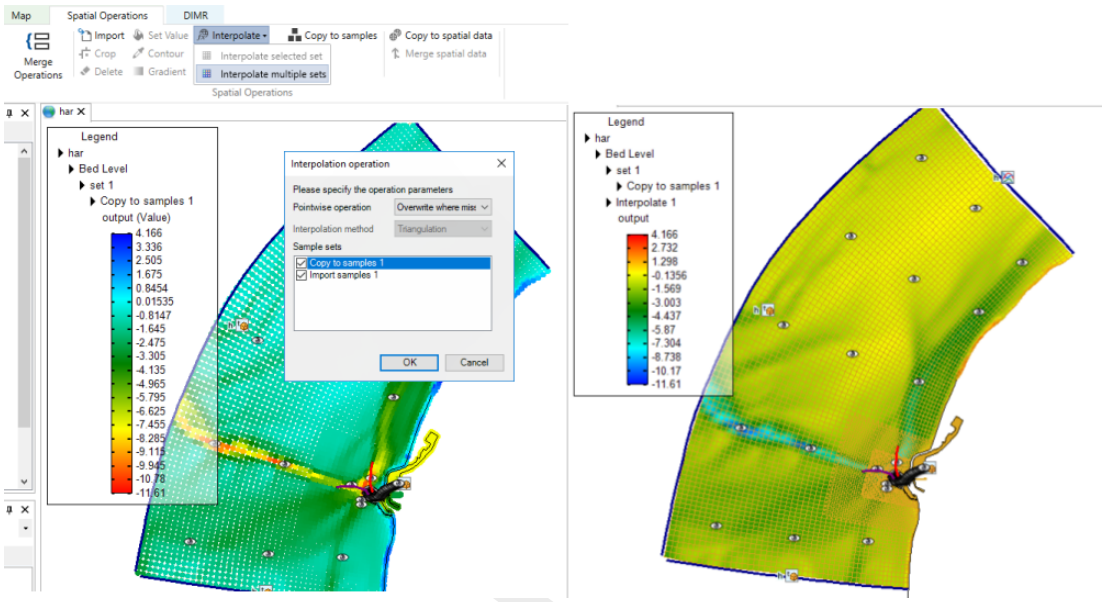


Figure H.46: *Performing an interpolation operation on multiple sample sets (without using a polygon) using 'Interpolate' from the 'Spatial Operations' ribbon*

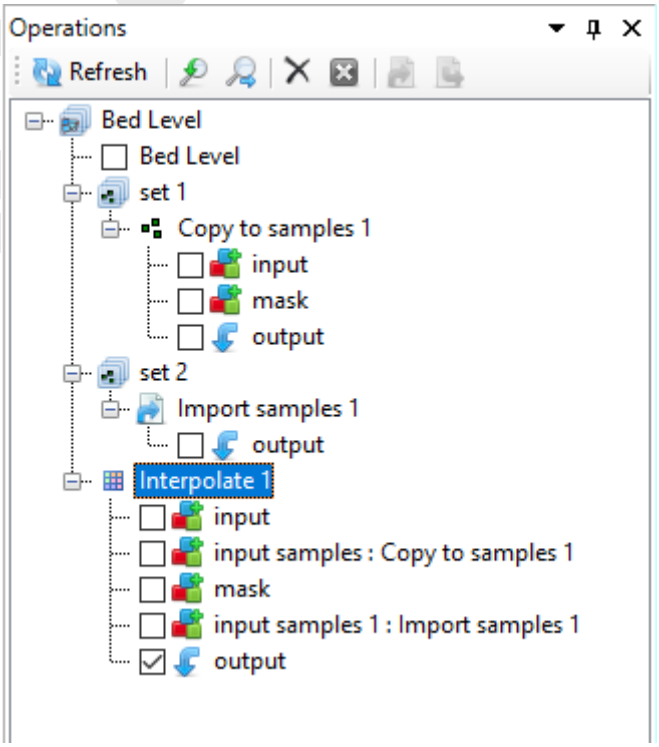


Figure H.47: *Appearance of interpolation of 'set1' and 'set2' to the coverage 'bed level' in the operations stack*

H.5.11 Smoothing

The smoothing operation smooths out (steep) gradients in a point cloud or coverage (depending on which one is active). The smoothing operation is activated from the 'Spatial Operations' ribbon and only available for polygon geometries or for the total data set if no polygon is provided. You have to assign the smoothing exponent and number of smoothing steps. The higher the exponent and the number of smoothing steps, the heavier the smoothing. For an example see Figure H.48. After

applying smoothing to (part of) the point cloud or coverage the operation is added to the operations stack (Figure H.49).

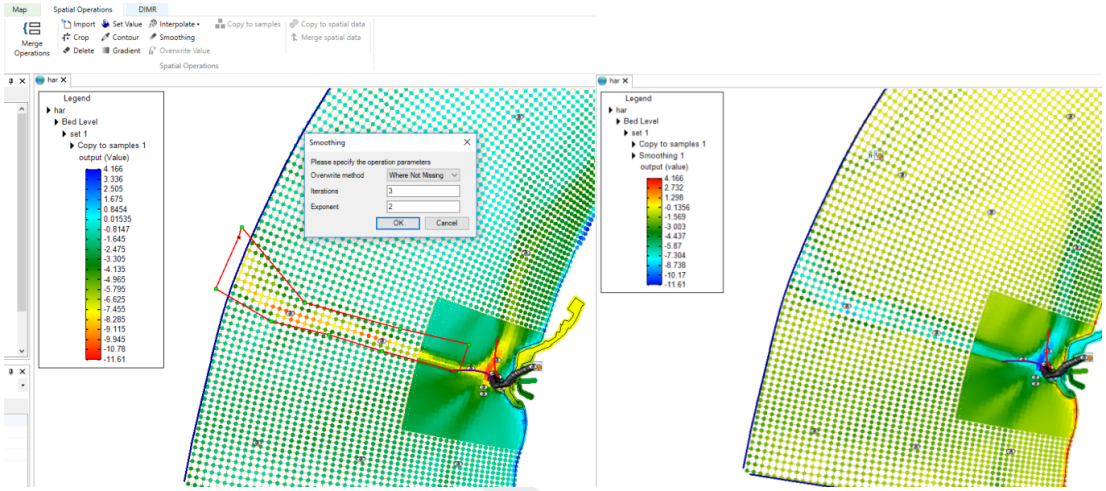


Figure H.48: *Performing a smoothing operation on a point cloud with a polygon using 'Smoothing' from the 'Spatial Operations' ribbon*

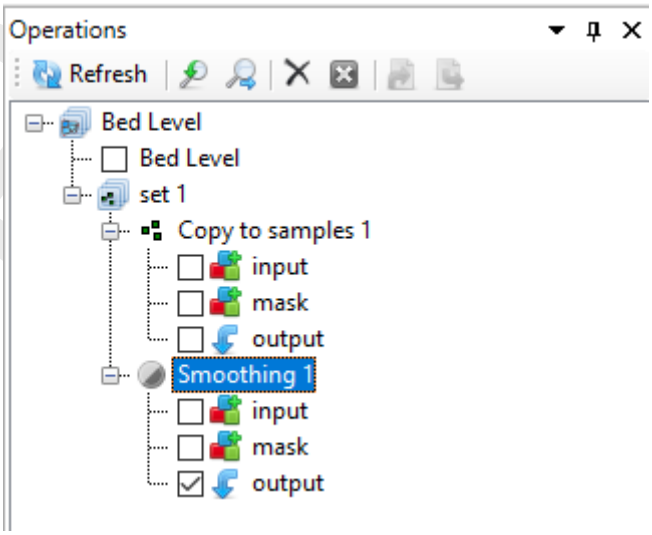


Figure H.49: *Appearance of smoothing operation in the operations stack*

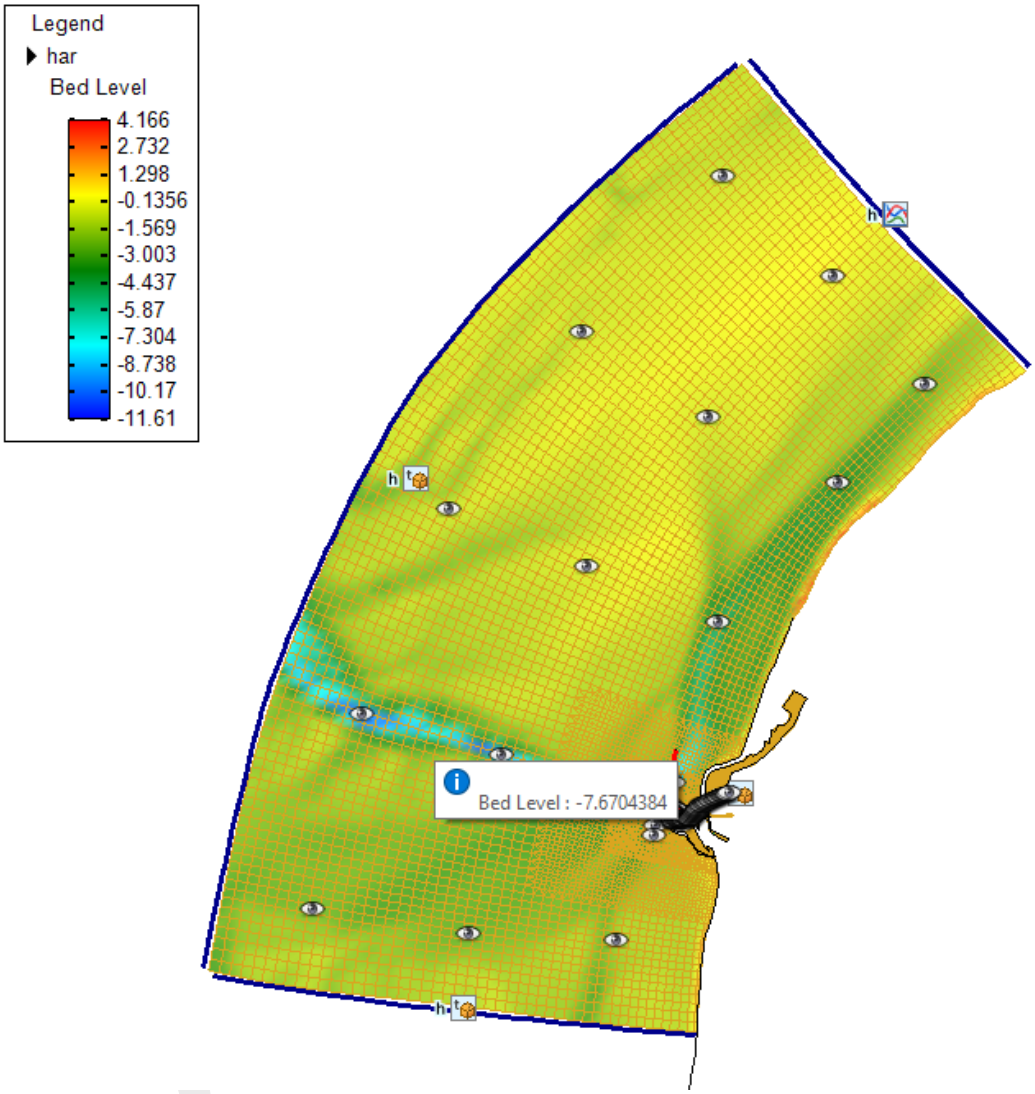


Figure H.50: *The cursor for the overwrite operation showing the value of the closest coverage point*

H.5.12 Overwrite (single) value

The ‘overwrite (single) value’ operation allows you to edit single values on the active coverage after the interpolation. The ‘overwrite (single) value’ operation is activated from the ‘Spatial Operations’ ribbon. There is no geometry required for this operation. Upon selecting the operation from the ribbon a cursor will become active showing the coverage value closest to the cursor in a tooltipstring (Figure H.50). Upon clicking LMB a popup appears in which you can overwrite the value of this coverage point Figure H.51. After applying the overwrite operation it is added to the operations stack (Figure H.52).

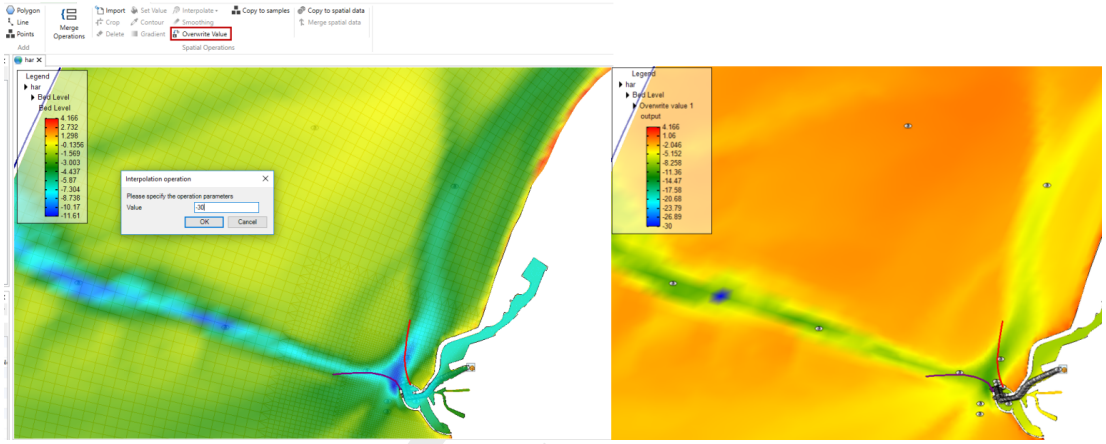


Figure H.51: *Performing an overwrite operation on a coverage point using 'Single Value' from the 'Spatial Operations' ribbon*

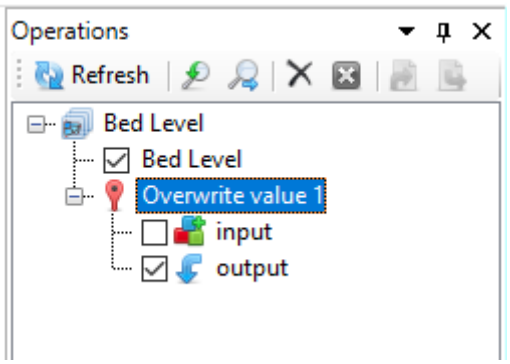


Figure H.52: *Appearance of overwrite operation in the operations stack*

H.6 Operation stack

The operation stack keeps track of the workflow of spatial operations that you performed. This helps you to make transparent how you arrived at your 'final' dataset without having to save all the intermediate datasets (steps) separately. Moreover, the stack is reproducible and easily editable without having to start all over again. This section describes the stack workflow (section H.6.1), how to edit operation properties (section H.6.2), how to enable/disable (section H.6.3), delete (section H.6.4), refresh(section H.6.5) operations, quick links (section H.6.6) and import/export functionality (section H.6.7).

Note: Currently, the stack is saved in the User Interface project upon saving the project. The next time you open the project, the stack will reappear. The stack is not (yet) saved in a human readable/editable file.

H.6.1 Stack workflow

Upon performing a spatial operation, the 'Operations' panel will open (see Figure H.53) with the operations stack (tree). The stack first shows on which point cloud or coverage you are working (in this example 'bed level'). Subesquently, all the operations on this dataset are listed. For each operation you can inspect what the input, mask (e.g. the geometry used for the operation) and output are for the operation (Figure H.54). By default the stack continues from the last operation that you performed. If you wish to work on a different dataset or operation within a dataset, you have to select that dataset or operation in the 'Operations' panel with the LMB.

When working on a coverage, point clouds (or sets) can be used to construct the coverage. In that case the stack jumps from the 'trunk' to a 'branch' and the subsequent operations are performed on the

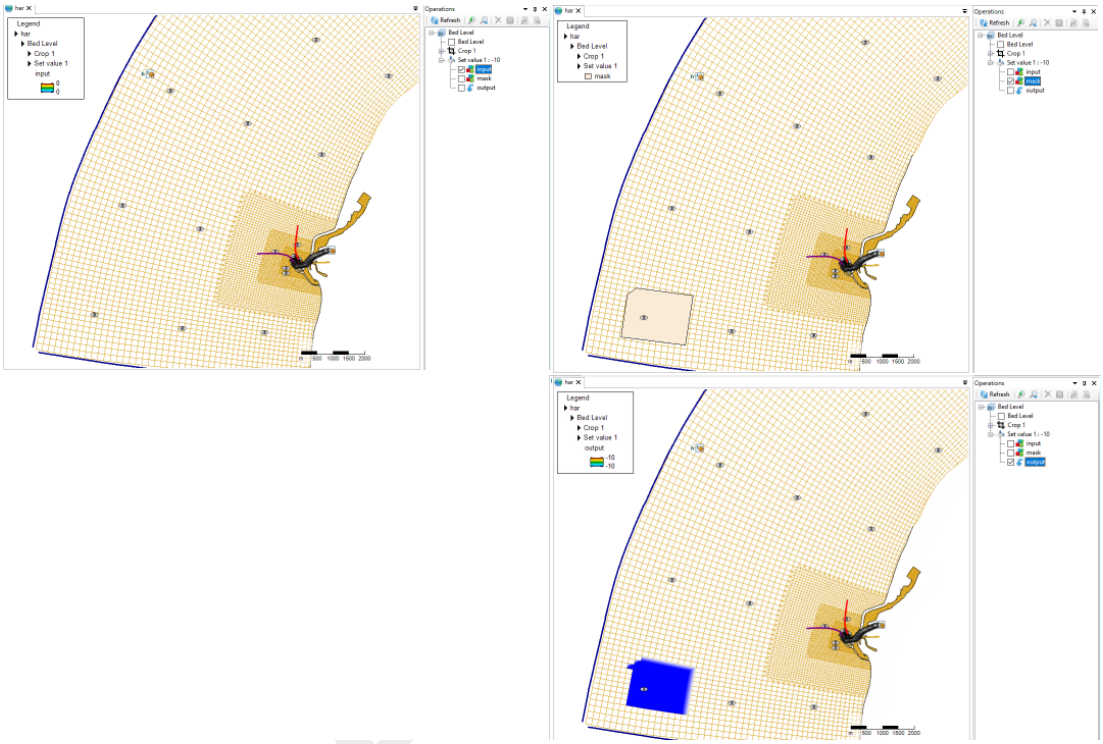


Figure H.54: *Input for the operation (top panel), mask for the operation (middle panel) and output of the operation (bottom panel)*

point cloud (see Figure H.53). By selecting the set or coverage in the 'Operations' panel you determine on which dataset you are working. The interpolate operation (section H.5.10) allows you to bring data from the point cloud (branch) to the coverage (trunk). See also Figure H.53.

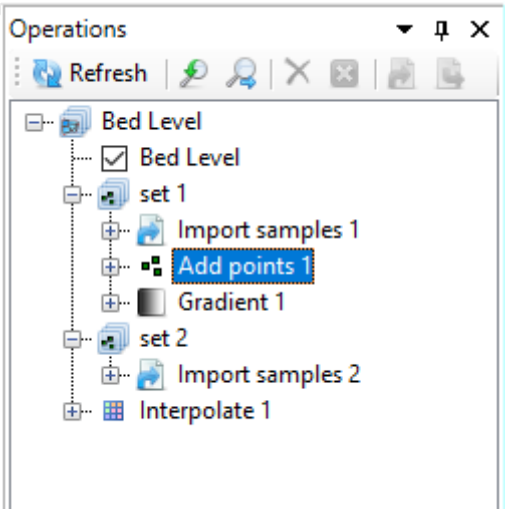


Figure H.53: *The 'Operations' panel with the operations stack. In this example 'bed level' is the coverage (e.g. trunk) that is edited. The point clouds 'set 1' and 'set 2' (e.g. branches) are used to construct the 'bed level' coverage.*

H.6.2 Edit operation properties

For each operation that you performed the properties (such as the value or 'Pointwise operation' of a 'Set Value' operation) can be edited using the 'Properties' window (Figure H.55). **Note:** Please

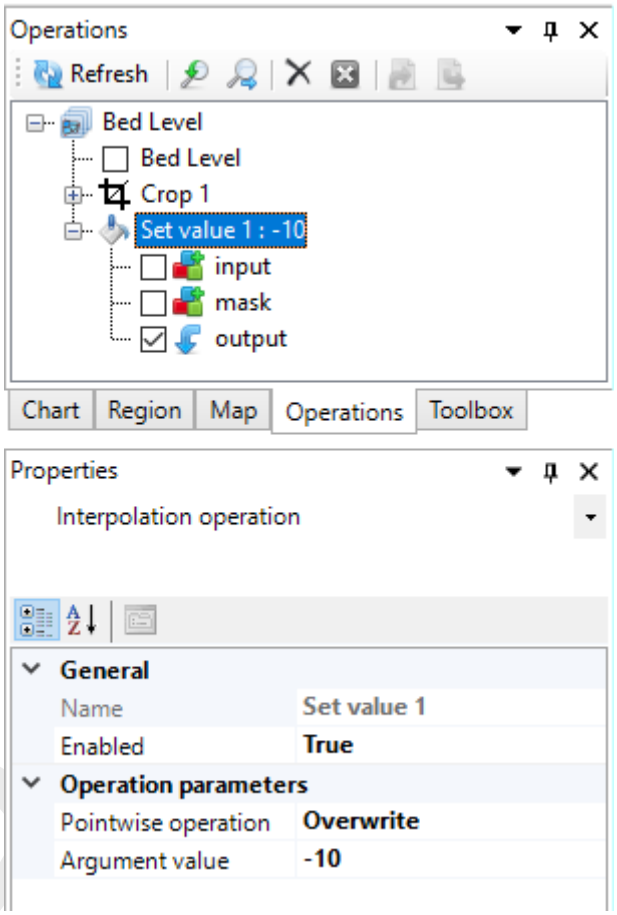


Figure H.55: *Editing the value or 'Pointwise operation' of a 'Set Value' operation using the properties panel*

note that the mask of an operation cannot (yet) be edited. By editing the operation properties the operation stack becomes 'out of sync' and has to be refreshed for the changes to become active (see section H.6.5).

H.6.3 Enable/disable operations

You can (temporarily) enable/disable operations by selecting the operation and pressing boxed cross icon in the stack menu (Figure H.56). Upon disabling an operation the operation will be made grey in the stack and the operation is not taken into account anymore upon evaluation of the overall result. The result of disabling an operation is not directly activated. This is indicated in the stack with the 'out of sync' exlamation mark (Figure H.56). You need to refresh the stack (see section H.6.5) for the changes to become active.

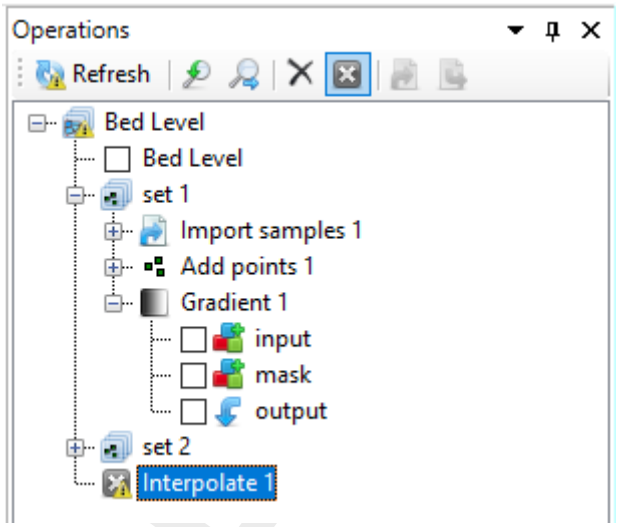


Figure H.56: *Disabling an operation using the boxed cross icon in the stack menu. The operation will become grey. Note the exlamation marks marking the stack 'out of sync'.*

H.6.4 Delete operations

To delete an operation permanently you have to select the operation and either press the cross icon (Figure H.57) or use the context menu and select delete (Figure H.58). The operation will be removed from the stack. The result of deleting an operation is not directly activated. This is indicated in the stack with the 'out of sync' exlamation mark. You need to refresh the stack (see section H.6.5) for the changes to become active.

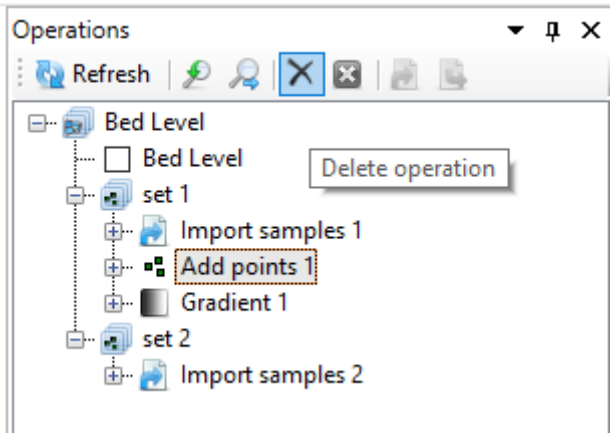


Figure H.57: *Removing an operation from the stack using the cross icon in the stack menu*

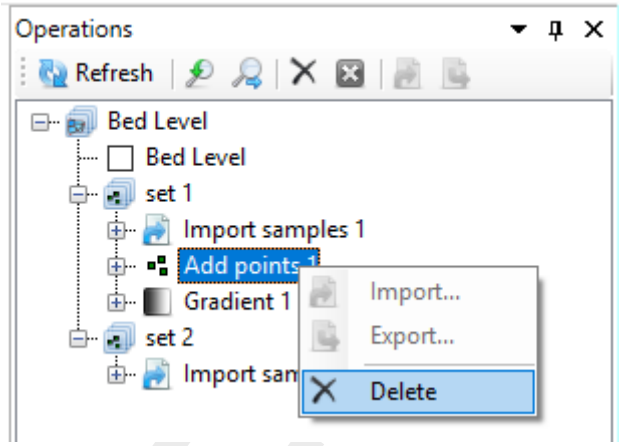


Figure H.58: *Removing an operation from the stack using the context menu on the selected operation*

H.6.5 Refresh stack

When the stack is marked 'out of sync' by exclamation marks, you can refresh the stack by pressing the 'Refresh' button for the changes to become active (Figure H.59). Upon refreshing the stack all the (enabled) operations in the stack will be (re-)evaluated. **Note:** Please note that refreshing the stack can take some time when large datasets are (re-)evaluated!

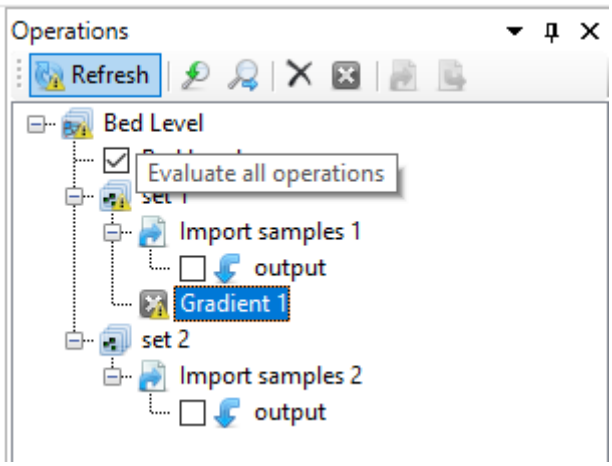


Figure H.59: *Refresh the stack using the 'Refresh' button so that all operation are (re-) evaluated*

H.6.6 Quick links

The stack menu contains two quick links to quickly show the original dataset (e.g. where you started from, Figure H.60) and the end result of the spatial operations (Figure H.61).

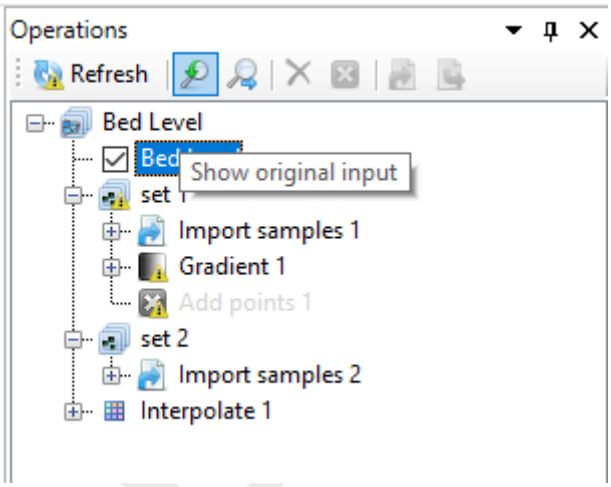


Figure H.60: *Quick link to the original dataset before performing any spatial operations*

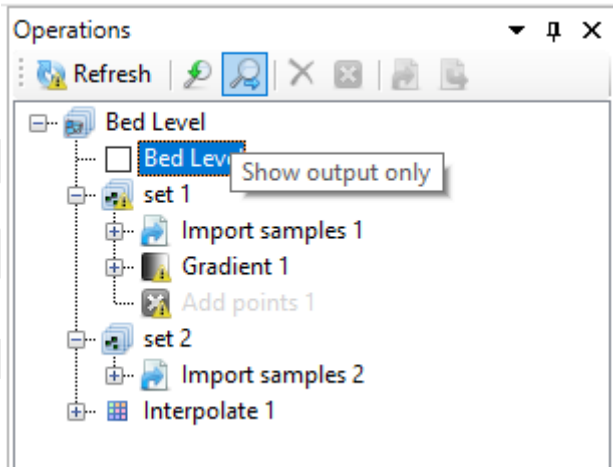


Figure H.61: *Quick link to the output after performing all (enabled) operations*

H.6.7 Import/export

Note: Importing and exporting data into or from the stack is still under construction

I Grid editing

A new grid editor is being developed, which is called D-Grid Editor. Currently, D-Grid Editor is embedded only in D-HYDRO Suite, where it can be used to generate and edit meshes for D-Flow FM 1D2D. RGFGRID is still delivered with Delft3D Flexible Mesh Suite, suitable for D-Flow FM 2D3D. In the future, the new editor will replace the RGFGRID editor, which is already in use for many decades for grid generation in 2D.

D-Grid Editor is not (yet) distributed as stand-alone application, nor is its User Manual. Therefore, the main features of D-Grid Editor will be described briefly in this appendix.

The main reasons for developing the new grid generator D-Grid Editor are:

- As RGFGRID was developed decades ago, it is increasingly hard to maintain and keep (cyber) secure.
- The Delta Shell framework enables us to embed mesh editing in the GUIs, instead of starting up a separate application.
- Enabling Multi-step Undo of mesh operations, while the Undo functionality is missing in RGFGRID.
- Enabling a background map - for example OpenStreetMap - while generating a mesh, which is not possible in RGFGRID.

Note: that D-Grid Editor generates - to what we will refer to as - *Meshes*, as they are compatible with D-Flow Flexible Mesh. A mesh can consist of curvilinear parts and unstructured parts as can be seen below: an example of a mesh with only quadrangles and another example with both triangles and quadrangles, respectively. For an example of a 1D2D mesh, generated by D-Grid Editor, see Figure 18.16.

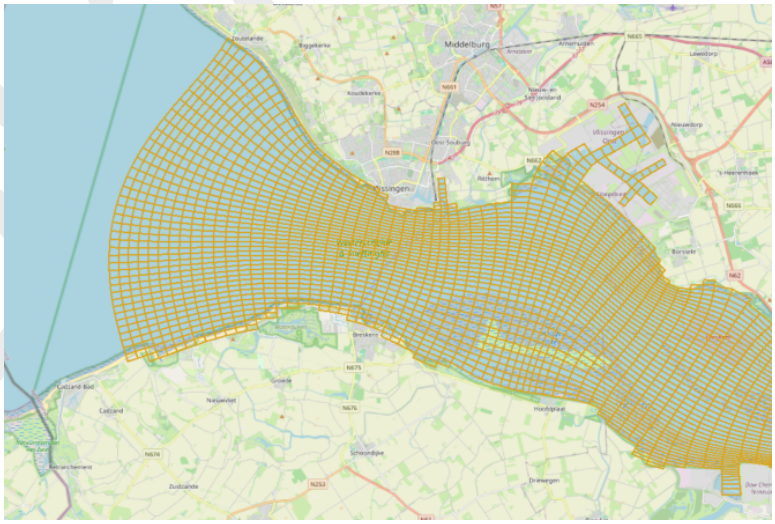


Figure I.1: *Illustration of a* mesh *with only quadrangles generated by D-Grid Editor*

DRAFT

Figure I.2: *Illustration of a* mesh *with triangles and quadrangles generated by D-Grid Editor*

I.1 Overview of the toolbars

After activation of D-Grid Editor a toolbar appears with high level functions; see Figure I.3.

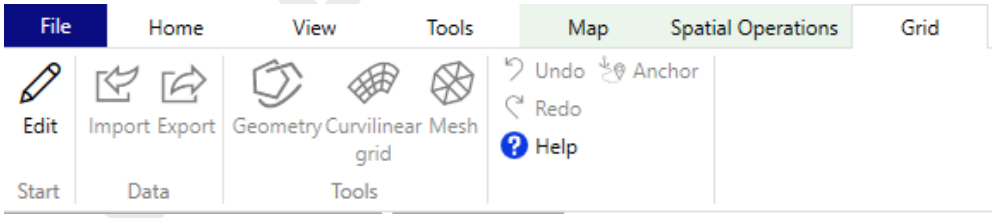


Figure I.3: *Main toolbar in D-Grid Editor*

Next, different functions can be started via this toolbar, which involves:

- Importing (geometrical) data, *Curvilinear grids* and/or *Meshes*
- Exporting (geometrical) data, *Curvilinear grids* and/or *Meshes*
- Working with *polygons* - for example to limit the area for which mesh operations will be applied; see Figure I.4.
- Working with /menuSplines, which can be used to generate *Curvilinear grids*; see Figure I.5.
- Working with *Land boundaries*; see Figure I.6.
- Generating a *Curvilinear grid* with only quadrangles; see Figure I.7.
- Generating an unstructured *Mesh* with triangles and/or quadrangles; see Figure I.8.

Figure I.4: *Toolbar for Polygons*

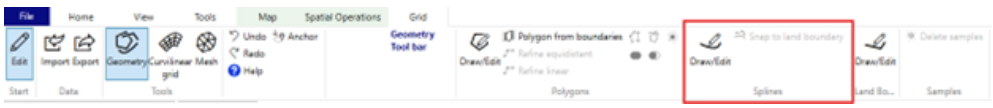


Figure I.5: *Toolbar for Splines*

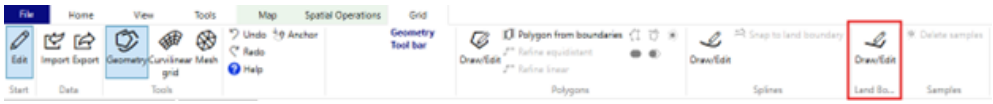


Figure I.6: *Toolbar for Land Boundaries*

Figure I.7: *Toolbar for Curvilinear Grids*

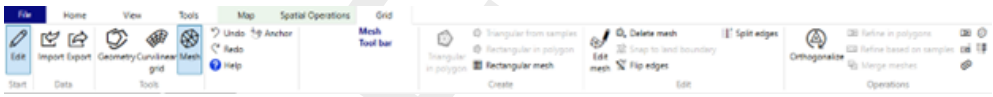


Figure I.8: *Toolbar for Meshes*

I.2 How to start D-Grid Editor?

D-Grid Editor is started by double clicking on *Grid* in the subfolder *2D* of the **Project** window. Then, the Toolbar of D-Grid Editor pops up, as shown in Figure I.3. Next, start editing by clicking the *Edit* button, after which one of the functions can be selected. For example, the *Splines* toolbar can be activiated so that splines can be drawn and a first grid generated via for example the *Curvilinear grid* section.

I.3 How to save a mesh generated by the D-Grid Editor?

In order to save a mesh, do not forget to use the *Convert to mesh* button. Now, by again clicking on the *Edit* button the unstructured mesh is saved in the project, D-Grid Editor is closed and you are back in the main menu of the GUI.

DRAFT

J Astronomical constituents

J.1 Introduction

In this Appendix we list the astronomical constituents that you can use when prescribing astronomical boundary conditions for the flow. In total you can select from 235 tidal constituents. See Section 6.4.7.2 for details on how to use them in boundary conditions.

J.2 List of astronomical constituents

The set of constituents can be divided into the primary constituents, which appear in the equilibrium tide (no land masses; only one deep ocean), and compound constituents. The latter are linear combinations of the primary constituents.

The names and frequencies of all 235 internally available constituents in D-Flow FM are specified below. We remark that 234 of these constituents are also in the Deltares tidal analysis program Delft3D-TIDE. Only component N4 has been added to the predefined list in D-Flow FM and this component isn't present in Delft3D-TIDE. For the primary constituents the relative magnitude in the equilibrium tide is given as well.

If you have only a short time-series for a tidal analysis you can use astronomical coupling between nearby constituents. To this end also the *equilibrium amplitude relation* with their main constituents is given. The *equilibrium phase relation* is equal to zero.

Remark:

- ◇ If you want to apply astronomical coupling, first try to find the amplitude and phase relations based on a long period analysis of a neighbouring station. Only if such information is not available, you may resort to the equilibrium tide relations.

Table J.1: *Astronomical constituents*

Constituent name	Angular frequency [degrees/hour]	Amplitude in equilibrium tide [m]	Amplitude coupling relation
SA	0.0410686	0.01156	
SSA	0.0821373	0.07281	
MSM	0.4715211	0.01579	
MM	0.5443747	0.08254	
MSF	1.0158958	0.01369	
MS0	1.0158958		
MF	1.0980331	0.15647	
KO0	1.0980331		
MK0	1.0980331		
SNU	1.4874169		
SN	1.5602705		
MSTM	1.5695542	0.00569	
MFM	1.6424078	0.02996	
2SM	2.0317916		
MSQM	2.1139289	0.00478	
MQM	2.1867825	0.00396	
2SMN	2.5761663		

continued on next page

DRAFT

Table J.1 — continued from previous page

Constituent name	Angular frequency [degrees/hour]	Amplitude in equilibrium tide [m]	Amplitude coupling relation
2OK1	12.8450025		
2Q1	12.8542862	0.00955	0.025 × O1
NJ1	12.8542862		
SIGMA1	12.9271398	0.01152	
MUK1	12.9271398		
NUJ1	12.9271398		
Q1	13.3986609	0.07343	0.191 × O1
NK1	13.3986609		
RO1	13.4715145	0.01395	0.036 × O1
NUK1	13.4715145		
O1	13.9430356	0.38358	
TAU1	14.0251728	0.00504	
MP1	14.0251728		
M1B	14.4874103	0.01065	0.350 × M1A
M1C	14.4920521		
M1A	14.4966939	0.02964	
M1	14.4966939	0.03150	0.082 × O1
NO1	14.4966939		
CHI1	14.5695476	0.00580	
LP1	14.5695476		
PI1	14.9178647	0.01028	
TK1	14.9178647		
P1	14.9589314	0.17543	0.328 × K1
SK1	14.9589314		
S1	15.0000000	0.00416	
K1	15.0410686	0.53496	
MO1	15.0410686		
SP1	15.0410686		
PSI1	15.0821353	0.00109	
RP1	15.0821353		
FI1	15.1232059	0.00755	
KP1	15.1232059		
THETA1	15.5125897	0.00578	
LABDAO1	15.5125897		
J1	15.5854433	0.03022	0.079 × O1
MQ1	15.5854433		
2PO1	15.9748272		
SO1	16.0569644		
OO1	16.1391017	0.01939	0.051 × O1
2KO1	16.1391017		

continued on next page

DRAFT

Table J.1 — continued from previous page

Constituent name	Angular frequency [degrees/hour]	Amplitude in equilibrium tide [m]	Amplitude coupling relation
UPSILON1	16.6834764	0.00372	
KQ1	16.6834764		
2MN2S2	26.4079379		
3MKS2	26.8701753		
2NS2	26.8794590		
3MS2	26.9523126		
OQ2	27.3416964		
MNK2	27.3416964		
EPSILON2	27.4238337	0.00671	
MNS2	27.4238337		
2ML2S2	27.4966873		
MNUS2	27.4966873		
MNK2S2	27.5059710		
2MS2K2	27.8039339		
O2	27.8860711		
NLK2	27.8860711		
2MK2	27.8860711		
2N2	27.8953548	0.02303	0.132 $\times$ N2
MU2	27.9682084	0.02776	0.031 $\times$ M2
2MS2	27.9682084		
SNK2	28.3575922		
NA2	28.3986628		
N2	28.4397295	0.17398	0.191 $\times$ M2
KQ2	28.4397295		
NB2	28.4807962		
NU2	28.5125831	0.03304	0.194 $\times$ N2
3MSN2	28.6040041		
2KN2S2	28.6040041		
OP2	28.9019669		
MSK2	28.9019669		
GAMMA2	28.9112506	0.00273	
ALFA2	28.9430356	0.00313	
MPS2	28.9430356		
MA2	28.9430356		
M2	28.9841042	0.90872	
KO2	28.9841042		
MSP2	29.0251728		
MB2	29.0251728		
DELTA2	29.0662415		
MKS2	29.0662415		

continued on next page

DRAFT

Table J.1 — continued from previous page

Constituent name	Angular frequency [degrees/hour]	Amplitude in equilibrium tide [m]	Amplitude coupling relation
M2(KS)2	29.1483788		
2SN(MK)2	29.3734880		
LABDA2	29.4556253	0.00466	0.005 × M2
SNM2	29.4556253		
2MN2	29.5284789		
L2	29.5284789	0.02663	0.029 × M2
L2A	29.5284789	0.02569	
L2B	29.5377626	0.00704	0.274 × L2A
2SK2	29.9178627		
T2	29.9589333	0.02476	0.059 × S2
S2	30.0000000	0.42248	
KP2	30.0000000		
R2	30.0410667	0.00366	0.009 × S2
K2	30.0821373	0.12004	0.284 × S2
MSNU2	30.4715211		
MSN2	30.5443747		
ZETA2	30.5536584	0.00134	
ETA2	30.6265120	0.00702	
KJ2	30.6265120		
MKN2	30.6265120		
2KM(SN)2	30.7086493		
2SM2	31.0158958		
SKM2	31.0980331		
2MS2N2	31.0887494		
2SNU2	31.4874169		
2SN2	31.5602705		
SKN2	31.6424078		
MQ3	42.3827651		
NO3	42.3827651		
MO3	42.9271398		
2MK3	42.9271398		
2MP3	43.0092771		
M3	43.4761563	0.01780	
NK3	43.4807981		
SO3	43.9430356		
MP3	43.9430356		
MK3	44.0251728		
SP3	44.9589314		
2MQ3	44.5695476		
SK3	45.0410686		

continued on next page

DRAFT

Table J.1 — continued from previous page

Constituent name	Angular frequency [degrees/hour]	Amplitude in equilibrium tide [m]	Amplitude coupling relation
2SO3	46.0569644		
K3	45.1232059		
4MS4	55.9364168		
2MNS4	56.4079379		
3MK4	56.8701753		
N4	56.8794591		
MNLK4	56.8701753		
3MS4	56.9523126		
MSNK4	57.3416964		
MN4	57.4238337		
MNU4	57.4966873		
2MLS4	57.4966873		
2MSK4	57.8860711		
M4	57.9682084		
2MKS4	58.0503457		
SN4	58.4397295		
3MN4	58.5125831		
2SMK4	58.9019669		
MS4	58.9841042		
MK4	59.0662415		
2SNM4	59.4556253		
2MSN4	59.5284789		
SL4	59.5284789		
S4	60.0000000		
SK4	60.0821373		
2SMN4	60.5443747		
3SM4	61.0158958		
2SKM4	61.0980331		
MNO5	71.3668693		
3MK5	71.9112440		
3MP5	71.9933813		
M5	72.4649024		
MNK5	72.4649024		
2MP5	72.9271398		
MSO5	72.9271398		
3MO5	73.0092771		
MSK5	74.0251728		
3KM5	74.1073101		
2(MN)S6	84.8476674		
3MNS6	85.3920421		

continued on next page

DRAFT

Table J.1 — continued from previous page

Constituent name	Angular frequency [degrees/hour]	Amplitude in equilibrium tide [m]	Amplitude coupling relation
4MK6	85.8542796		
2NM6	85.8635632		
4MS6	85.9364168		
2MSNK6	86.3258006		
2MN6	86.4079379		
2MNU6	86.4807915		
3MSK6	86.8701753		
M6	86.9523126		
MSN6	87.4238337		
MNK6	87.5059710		
4MN6	87.4966873		
MKNU6	87.5788246		
2(MS)K6	87.8860711		
2MS6	87.9682084		
2MK6	88.0503457		
2SN6	88.4397295		
3MSN6	88.5125831		
MKL6	88.5947204		
2SM6	88.9841042		
MSK6	89.0662415		
S6	90.0000000		
2MNO7	100.3509735		
2NMK7	100.9046319		
M7	101.4490066		
2MSO7	101.9112440		
MSKO7	103.0092771		
2(MN)8	114.8476674		
3MN8	115.3920421		
3MNKS8	115.4741794		
M8	115.9364168		
2MSN8	116.4079379		
2MNK8	116.4900752		
3MS8	116.9523126		
3MK8	117.0344499		
2SNM8	117.4238337		
MSNK8	117.5059710		
2(MS)8	117.9682084		
2MSK8	118.0503457		
3SM8	118.9841042		
2SMK8	119.0662415		

continued on next page

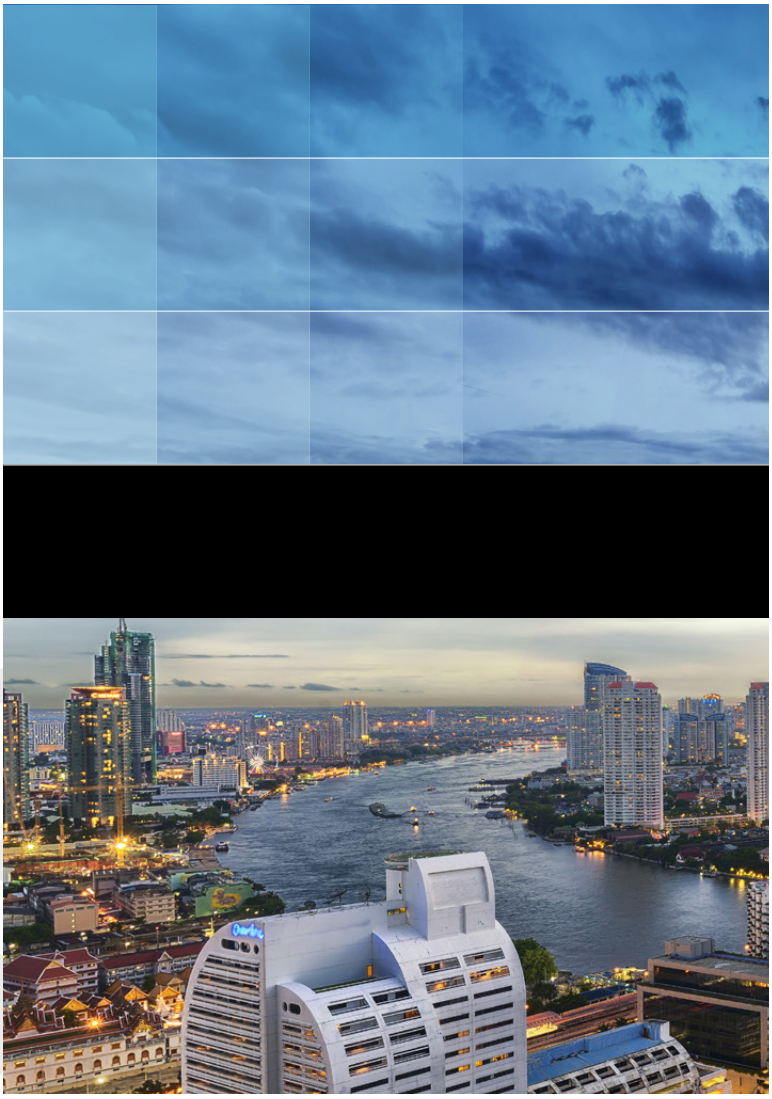
DRAFT

Table J.1 — continued from previous page

Constituent name	Angular frequency [degrees/hour]	Amplitude in equilibrium tide [m]	Amplitude coupling relation
S8	120.0000000		
2(MN)K9	129.8887361		
3MNK9	130.4331108		
4MK9	130.9774855		
3MSK9	131.9933813		
4MN10	144.3761463		
M10	144.9205210		
3MSN10	145.3920421		
4MS10	145.9364168		
2(MS)N10	146.4079379		
2MNSK10	146.4900752		
3M2S10	146.9523126		
4MSK11	160.9774855		
M12	173.9046253		
4MSN12	174.3761463		
5MS12	174.9205210		
3MNKS12	175.4741794		
4M2S12	175.9364168		

DRAFT

DRAFT



s systems

+31 (0)88 335 81 88
software@deltares.nl
www.deltares.nl/software

DRAFT